《数据结构》讲义

2026–2027学年第一学期

本科生（大二上）课程

2025级数据科学与大数据技术专业5班、6班

赵金华

E-mail: zhaojh@m.scnu.edu.cn

华南师范大学 数据科学与工程学院

November 12, 2026

CONTENTS

0. 课程介绍 9

0.1. 个人介绍 9

0.2. 课程介绍与参考资料 9

0.3. 礼仪、纪律、建议 14

1. 绪论 18

1.1. 计算与计算工具 18

1.2. 编程语言 19

1.3. 数据结构与算法 20

1.4. 日常生活中的算法 22

2. 搭建工作流程 24

2.1. 工具：编程和计算 24

1. 编译器 24

2. 文本编辑器 26

3. IDE 27

2.2. 工具：作图和可视化 29

2.3. 如果你是macOS用户 31

2.4. 工具：编辑和生成文档 32

1. 生成演示文稿 34

2.5. 在文档中插入代码 34

2.6. 利用数据文件来作图 34

习题 37

3. 编程语言和项目 38

3.1. C++编程语言 38

3.2. 自洽方程不动点的求解和分析 40

1. 函数零点的二分法 40

2. 自洽方程的离散序列法 41

3. 不动点的稳定性分析 42

4. 例子：连续相变和不连续相变 45

3.3. 投针问题 50
1. 投针问题的几何描述 50
2. 投针问题的模拟实现 53
3. 解析理论：蒲丰投针问题 53
4. 解析理论：蒲丰-拉普拉斯投针问题 56
5. 模拟和解析结果 61
3.4. Vicsek模型 63
习题 66

4. 算法的复杂度 68
4.1. 算法复杂度的测量 68
4.2. 时间复杂度的渐近分析：概念和符号 69
4.3. 时间复杂度的渐进分析：计算 71
4.4. 递归算法的时间复杂度 75
4.5. 计算复杂性 79
习题 83

5. 向量(Vector) 85
5.1. 向量的基本性质 85
5.2. 向量的ADT 85
5.3. 动态空间管理 87
5.4. 序列的生成 89
1. 有序序列 89
2. 随机序列 90
5.5. 查找算法 91
1. 序列上的查找任务 91
2. 查找长度 93
3. 查找算法的比较 94
4. 二分查找法（版本A） 94
5. 斐波那契查找法 98
6. 插值查找法 104
7. 二分查找法（版本B） 109
8. 二分查找法（版本C） 112

5.6. 排序算法 115
1. 序列上的排序任务 115
2. 排序算法的比较 116
3. 插入排序 116
4. 选择排序 121
5. 冒泡排序 125
6. 归并排序 129
7. 快速排序 137
习题 144

6. 列表(List) 150
6.1. 列表的基本性质 150
6.2. 列表的ADT 153
6.3. 比较向量和列表上的操作时间 161
6.4. 列表的应用 164
习题 166

7. 字符串(String) 168
7.1. 字符串的基本性质 168
7.2. 字符串的ADT 169
7.3. 随机字符串的生成 169
1. 为何需要随机字符串? 169
2. 问题的描述 169
3. 生成序列的算法 171
4. 代码实现 172
7.4. 串匹配问题 175
7.5. 串匹配问题的蛮力算法 178
7.6. 串匹配问题的KMP算法 181
1. Next表的含义 181
2. Next表的直接构造法 185
3. Next表的迭代构造法 186
4. 基于Next表的KMP算法 192
5. 更新版Next表 192

6. 更新版Next表的直接构造法 194
7. 更新版Next表的迭代构造法 194
8. 基于更新版Next表的KMP算法 196
9. KMP算法和蛮力算法的比较 197
习题 200

8. 栈与队列(Stack and Queue) 202
8.1. 栈 202
1. 栈的基本性质 202
2. 栈的ADT 202
3. 栈的应用 205
8.2. 队列 210
1. 队列的基本性质 210
2. 队列的ADT 211
3. 队列的应用 212
8.3. 栈与队列的比较 212
习题 213

9. 散列表(Hashtable) 216
9.1. 应用场景：词典 216
9.2. 基本性质 216
9.3. 散列函数(hashing function) 216
9.4. 冲突排解 217
9.5. 散列表的应用 218
习题 221

10. 优先级队列(Priority Queue) 223
10.1. 基本性质 223
10.2. ADT 223
10.3. 应用 223

11. 二叉树(Binary Tree) 224
11.1. 树的基本性质 224

11.2. 二叉树的ADT 228
11.3. 二叉树的先序、中序、后序遍历 236
1. 遍历过程的定义 236
2. 遍历过程的递归版本 236
3. 递归算法和迭代算法的比较 242
4. 遍历过程的非递归版本 245
11.4. 二叉树的层次遍历 245
11.5. 二叉树的遍历序列与节点的顺序性 245
11.6. 从二叉树的遍历序列反推其构型 250
11.7. 二叉树的作图 261
习题 264

12. 二叉搜索树(Binary Search Tree) 267
12.1. 二叉搜索树的基本性质 267
12.2. 二叉搜索树的ADT 267
12.3. 渐近平衡与旋转操作 274
12.4. AVL树 275
12.5. 构造AVL树的重平衡策略 279
习题 285

13. 图(Graph) 286
13.1. 图的基本性质 286
13.2. 无向图的ADT 302
13.3. 有向图的ADT 306
13.4. 图的遍历 309
13.5. 图的遍历：多个连通单元的情形 312
13.6. 顶点间的最短路径 316
13.7. 图的最小生成树 321
13.8. 无向图上的移叶过程 323
13.9. 有向图上的强连通单元 326
习题 328

14. 随机图模型 330

14.1. 随机正规图(Random regular graph) 330
1. 生成过程 331
2. 度分布的推导 331
14.2. ER随机图(Erdös-Rényi random graph) 332
1. 生成过程 332
2. 度分布的推导 333
14.3. 度分布的无标度性质 336
14.4. 纯无标度网络(Pure scale-free networks) 337
14.5. BA模型(Barabási-Albert Model) 337
1. 生成过程 337
2. 度分布的推导 338
3. 计算结果 338
14.6. 静态模型(Static Model) 338
1. 生成过程 339
2. 度分布的推导 340
3. 计算结果 343
14.7. 构型模型(Configuration Model) 344
1. 生成过程 344
2. 度分布的推导 345
3. 计算结果 345
14.8. 图的随机化 345
习题 346

15. 图上的动力学过程 348
15.1. 图上的渗流现象(Percolation phenomena) 348
15.2. 最大连通单元的生成 349
1. 模型 349
2. 解析理论 352
3. 自洽方程的不动点分析 353
4. ER随机图上的结果 354
15.3. K-核渗流(K-core percolation) 355
1. 模型 355

2. 解析理论 358
3. 解析理论公式的简化 359
4. 自洽方程的不动点分析 360
5. ER随机图上的结果 362
15.4. 核渗流(Core percolation) 364
1. 模型 364
2. 解析理论 365
3. 自洽方程的不动点分析 367
4. ER随机图上的结果 367
15.5. 有向图的强连通单元 370
1. 模型 370
2. 解析理论 370
3. ER随机图上的结果 371
15.6. Random walks 372
习题 374

16. 作业安排 375
16.1. 平时作业的安排 375
16.2. 实验课作业的安排 376

References 379

0.

0.1. 个人介绍

■ 个人介绍

授课老师：赵金华

所属学院：数据科学与工程学院

E-mail：zhaojh@m.scnu.edu.cn

办公室：汕尾校区学院综合楼724室

教育背景：

- 理学学士，华中科技大学物理系，2006年；
- 理学博士（理论物理专业），中国科学院理论物理研究所，2014年；

研究兴趣：复杂系统的统计物理学，包括：

- 无序系统的统计物理学、及其交叉学科应用；
- 图论组合优化问题的统计物理方法和求解算法；
- 图论与网络科学，比如图渗流问题。

0.2. 课程介绍与参考资料

■ 授课内容：《数据结构》

背景知识：高等数学、概率论、C/C++编程语言。

课程覆盖了数据结构和算法的基础知识及一些应用，包括：

（1）算法时间复杂度的渐近分析；

（2）常见数据结构的抽象数据类型与使用，包括向量、列表、字符串、散列表、栈与队列、二叉树、二叉搜索树、图；

（3）一些基本的算法问题，比如查找、排序、匹配、遍历等。

■ 编程语言的选择

推荐大家使用C语言或者C++语言，或者其它能够实现面向对象编程的语言。

本课程的讲解用的是C++语言。

以下是本课程需要大家预先掌握的编程语言的相关知识：

- 变量：基本数据类型。
- 基本运算符：算术、逻辑、比较等。

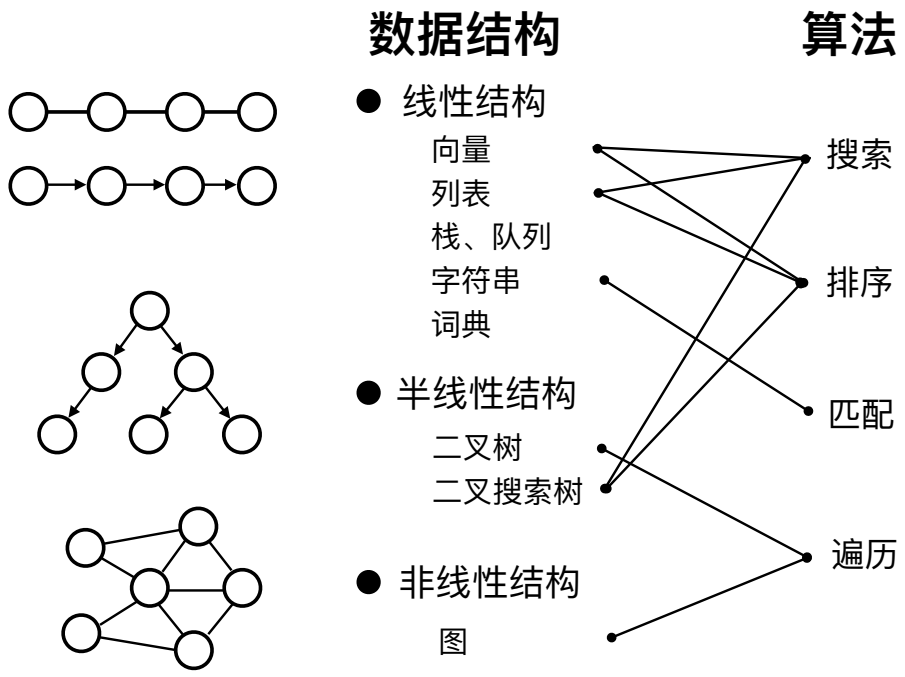


FIG. 0.1. 《数据结构》课程的若干主题。

• 流程控制语句：循环（for和while指令）、分支（if和if-else指令）、跳转（break和continue指令）等。

• 函数：声明、定义、使用。

• 面向对象编程：结构体(struct)和类(class)的定义和使用。

■ 本课程的定位和设置

定位：利用编程语言来解决一些实际的计算问题。

在课程设计中的位置：

（1）程序设计语言：学习基本的编程方法，比如C/C++、Python。代码行数：$10^1 \sim 10^2$。

（2）**数据结构与算法**：基本的数据结构和算法求解方法。代码行数：$10^1 \sim 10^3$。

（3）数值方法和模拟方法：数值方法一般用来处理数学里难以解析求解的问题，包括矩阵、微分方程、积分方程、最优化问题等；模拟方法一般用来处理物理和工程领域里难以在物理上直接实现的过程，比如随机过程的蒙特卡洛方法等等。代码行数：$10^1 \sim 10^3$。

（4）专业领域：信息检索、自然语言处理、计算机视觉、大规模机器学习等等。一般使用具体的数据处理工具或平台。代码行数：$10^1 \sim 10^3$。

■ 本课程的推荐教材

• 《数据结构教程》（第7版 · 微课视频 · 题库 · AI赋能版），李春葆主编，清华大学出版社，2022年

• 《数据结构教程》（第6版 · 微课视频 · 题库版），李春葆主编，清华大学出版社，2022年

• 《数据结构（C++语言版）》（第3版），邓俊辉著，清华大学出版社，2013年

• 《数据结构习题详解》（第3版），邓俊辉著，清华大学出版社，2013年

• 课程网站: `https://dsa.cs.tsinghua.edu.cn/~deng/ds/dsacpp/index.htm`

• 学习视频：在bilibili上搜索“邓俊辉 数据结构”

注意：本课堂的教学的出发点和研究生考试没有关系。有计划考研的同学请自行查阅当年的“全国计算机专业数据结构联考大纲”。此外，本课程的讲义在内容覆盖和专业名词上和“联考大纲”有些差别。



FIG. 0.2. 若干参考资料：数据结构和算法。

■ 参考资料：C++编程语言

• *C++ Primer*, Fifth Edition, Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo, Addison-Wesley, 2013

◦《C++ Primer》（第5版，中文版），Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo著，王刚、杨巨峰译，电子工业出版社，2013年

• *C++ Primer Plus*, Sixth Edition, Stephen Prata, Addison-Wesley, 2012

◦《C++ Primer Plus》（第6版，中文版），Stephen Prata著，张海龙、袁国忠译，人民邮电出版社，2020年

• *The C++ Programming Language*, Fourth Edition, Bjarne Stroustrup, Addison-Wesley, 2013

◦《C++程序设计语言》（第4版，中文版），Bjarne Stroustrup著，王刚、杨巨峰译，机械工业出版社，2016年

◦《C++程序设计语言》（第4版，中文版）（上下册），Bjarne Stroustrup著，何荣华、王文斌等译，人民邮电出版社，2026年

个人建议：当你对C++编程语言本身很感兴趣的时候，可以参考上述这些书。

• *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*, Third Edition, Scott Meyers, Addison-Wesley, 2005

◦《Effective C++: 改善程序与设计的55个具体做法》（第3版，中文版），Scott Meyers著, 电子工业出版社，2011年

• Google C++ Style Guide（谷歌C++编程规范）

→ `https://google.github.io/styleguide/cppguide.html`

中文教材推荐（够用）：

• 《C++程序设计教程》（第3版）通用版，钱能著，清华大学出版社，2019年。

• 《C++程序设计语言–现代C++（23标准）》，庞雄文、李乡儒、张金区著，清华大学出版社，2026年。

■ 参考资料：数据结构与算法

• *Algorithms*, Fourth Edition, Robert Sedgewick and Kevin Wayne, Pearson Education, Inc. 2011

◦《算法》（第4版，中文版），Robert Sedgewick and Kevin Wayne著，谢路云译，人民邮电出版社，2012年;

• *Algorithm Design*, Jon Kleinberg and Éva Tardos, Pearson Education, Inc., 2006

◦《算法设计》，Jon Kleinberg and Éva Tardos著，王海鹏译，人民邮电出版社，2021年。

• *Algorithms*, Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani, McGraw Hill Higher Education, 2008.

◦《算法概论》，Sanjoy Dasgupta, Christos Papadimitriou, Umesh Vazirani著，王沛、

唐扬斌、刘齐军译，清华大学出版社，2008年。

• *Introduction to Algorithms*, Fourth Edition, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein (aka. CLRS), The MIT Press, 2022

◦《算法导论》（第3版，中文版），Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein著，殷建平、徐云、王刚等译，机械工业出版社，2012年。

■ 算法的在线测试平台

• Leetcode算法题库: `https://leetcode.cn/`

■ 课程的考核方式

▷ 《数据结构》

考试形式：闭卷考试。

总评分数构成（暂定）：

（1）“平时”：分数占比∼ 40%。∼ 4次平时作业的平均表现。

（2）“期末”：分数占比∼ 60%。期末考试的卷面分数。

▷ 《数据结构实验》

考试形式：考核。

总评分数构成（暂定）：

（3）“期末”：包括∼ 10次实验课作业。

■ 一些特别的注意事项

（1）和《算法设计与分析》课程的关系

▷ 两门课都是研究设计具体的逻辑结构来尽可能快快速和准确地处理计算问题，但它们的侧重点有些不一样。前者侧重逻辑结构本身的不同，后侧侧重整体的处理思路和方法。 但为具体问题设计数据结构、计算复杂度的讨论等是两者都关注的问题。

▷ 本质上，这两门应该整合在一起。

▷ 大家的《算法设计与分析》被安排在大三上学期。

（2）培养计划的修改

2026级开始执行新的培养计划。《数据结构》和《数据结构实验》合并为一门课。学时结构为理论课48学时、实验课16学时。

0.3. 礼仪、纪律、建议

■ 和老师打交道时的基本礼仪

（1）邮件：有事情需要给老师写邮件时，注意写好邮件的题目、抬头、称呼等基本信息。注意：缺乏基本礼仪的邮件，我一概不回。

（2）微信：近几年，我一般不加学生的微信。绝大多数的事情，比如请假、询问课程信息等等，大家可以通过邮件联系我、或者在实验课期间问我。如果对课程和知识本身有比较多的问题，可以加我微信。

（3）课程微信群：主要用来发布资料、以及供大家讨论使用。建议大家有问题需要讨论或者询问的可以直接发在群里。

■ 课堂纪律

（1）课堂里可能会点名，也可能没有。希望大家为自己的学习负责。

（2）课程学习的路径有很多。当你决定来课堂时，最好有个必须来的理由。如果你来了教室，希望不要浪费自己的时间和精力，同时也要尊重课堂里的一些基本纪律。

（3）一点基本的课堂礼仪：

- 教室、寝室、食堂、操场的差别不仅仅在于它们所在的地方。
- 不要在教室吃东西。尤其是第1节课。
- 迟到的同学从后门进教室。
- 理论课期间请勿用手机、平板、或电脑。
- 上课（以及考试）期间，教师示意可以离开之后，大家才能离开教室。

（4）实验课里，一般有具体的编程任务，请大家带上自己的电脑和草稿纸。

■ 平时作业和实验课作业

（1）平时作业侧重理论推导和编程。

（2）作业的第一要求：独立完成。作业里的方法和流程可以和同学讨论，但代码和结果必须用自己的方式来完成。

（3）所有作业统一在励儒云平台上提交。每次作业都有提交的“到期时间”和“截止时间”。因为特殊原因需要延期交作业的话，请提前告诉我。

（4）作业改进：在“到期时间”之后、“截止日期”之前，作业可以继续修改，并继续交过来。

（5）作业评分：期末的作业分数大致是大家平时作业分数的总体表现。

（6）作业的参考答案只作为参考，不必作为标准答案。大家只需要保证自己解答的每

一步都是清晰的、有理由可循即可，无需修改成参考答案的形式。

■ 作业文档的格式

（1）文档格式：文档格式为pdf。如果是使用word所写，请转成pdf格式。推荐使用文本编辑工具Tex来编辑和生成文档。

（2）作业文件名：包括个人姓名、专业、班级、作业次序，再加上文档版本号（依次为v1、v2等），例如“赵小华-25级大数据1班-第1次作业-v1.pdf”。

（3）大部分编程题属于开放性题目。计算结果展示的篇幅不限，但其组织结构需要很清晰。

（4）编程题的作答格式： 对于每道题，需要用清晰的语言描述主要解决方法和结论。可以按照方法、代码、结果的方式来组织内容。具体内容需要包括：

• 计算步骤：包括计算过程的整体流程、算法流程和示意图、主要参数等等。

• 主要代码：和计算直接相关的源代码。交作业时暂时**无需提交源代码文件本身**。作业文档里源代码可选择使用**打了行号的代码截图**。你可以在自己的编辑器环境里显示代码的行号，然后进行**截屏**。此外，使用作图工具绘图时所使用的代码或脚本不用展示。

• 计算结果：包括算法的输出结果。可以使用屏幕截图。有时候需要使用作图工具把计算结果可视化。评价下计算结果，以及可能改进的地方等等。

请避免这样的作答格式：只有代码、或者只有代码和计算结果。

■ 平时作业的修改

（1）作业修改的“截止时间”一般为参考答案发布后3 ∼ 4天。

（2）在修改后的作业文档开头用红色字体逐项列出来所有的修改内容。

（3）在修改后的作业文档里用红色字体标记所有修改过的内容。

（4）在励儒云上提交修改版本时，不要删除之前的作业，直接上传新的版本即可。

■ 实验课作业的记分

（1）实验课本身的出发点是大家能够当堂完成。

（2）实验报告统一在励儒云上提交。

（3）如果只是在砺儒云上按时提交实验课作业的话，在没错误的情况下，只能得实验课作业的基本分数，比如60% ∼ 70% 。如果你希望多拿一些分数，请尽量在实验课里当堂完成。

■ 一些学习建议

（1）课前请预习。课堂里能学到的东西是你在整个课程里能学到的大概30%。课后需

要重复阅读相应的章节、并写代码运行。

（2）如果你想跟上课堂的进度，请不要逃课。

（3）课件本身主要是知识点的索引。大家不用指望它会教你所有的东西。

（4）管理好本课程的参考文档、代码、作业等等。

（5）多讨论：课程微信群除了发布课程消息和作业，还供大家讨论问题。

（6）多写程序：概念+代码。两者缺一不可。多写代码，偶尔试试用笔写。

（7）完善学习习惯：学习都是个学习+测试+反馈的迭代过程。通过讨论和测试，有了对知识点掌握程度上的反馈，才能知道自己是否真的做到了理解，之后再进行纠正和改进。整个学习生涯也是如此，你可以不断判断自己的学习策略是否得当、然后改进。

■ A final note (1)

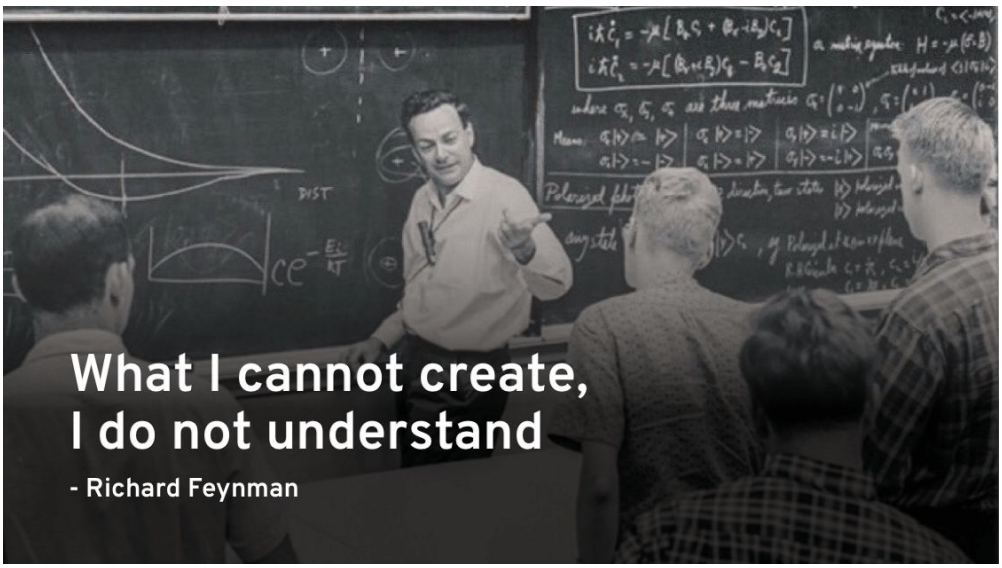


FIG. 0.3. Richard P. Feynman (1918-1988)，Nobel Laureate of Physics 1965。

Richard P. Feynman: “What I cannot create, I do not understand. ”

里查德·费曼：“那些我无法创造的东西，我无法理解。”

图片链接：`https://digital.archives.caltech.edu/collections/Images/1.10-29/`

我的理解是：对于知识点，我们除了理解它本身，还需要了解它被发现出来的具体场景。想象一下自己处在当时的场景下，试下自己能够想到的最好的解决办法。当你能想到的解决办法和知识点的思路类似、甚至更好，那么才能做到对知识点有更深的理解。

■ A final note (2)



FIG. 0.4. 南宋陆游（1125-1210）。

此诗完成于1199年，为陆游对幼子治学上的告诫。

冬夜读书示子聿

– 【南宋】陆游

古人学问无遗力，少壮工夫老始成。
纸上得来终觉浅，绝知此事要躬行。

1.

参考内容：邓俊晖教材第1.5节。李春葆教材第1.1、1.2、1.4节。

1.1. 计算与计算工具

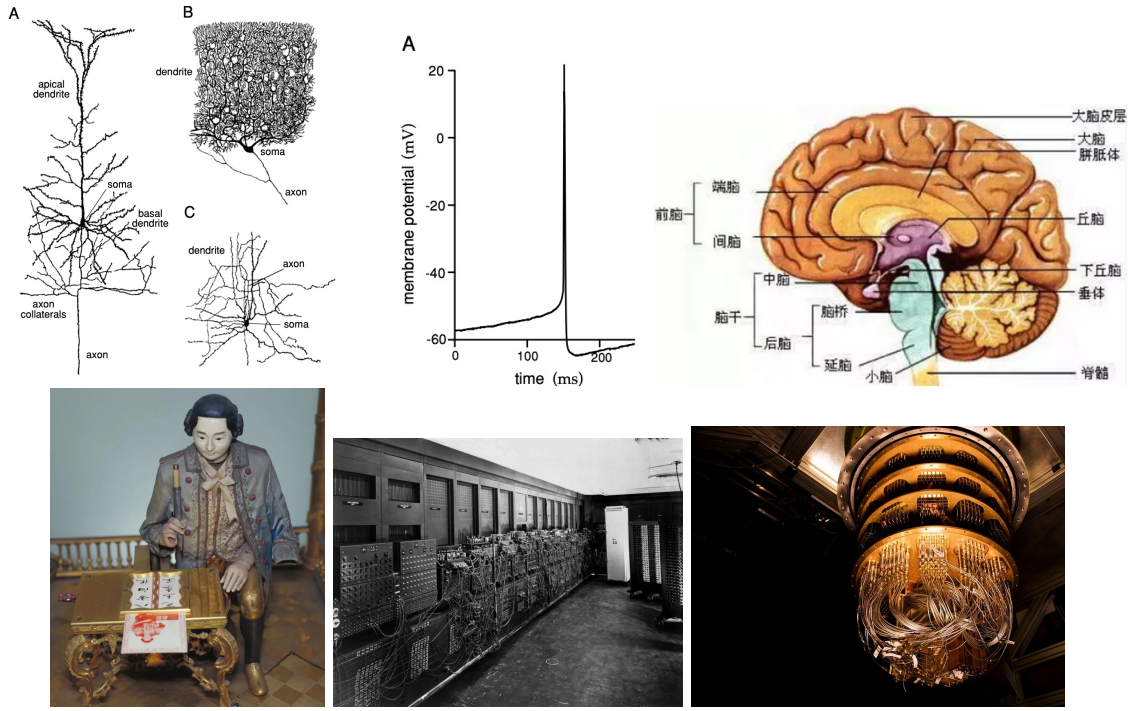


FIG. 1.1. 计算工具的各种形式。 **上排**：神经元、神经元产生神经电位脉冲、大量的（10^9的量级）的神经元构成的大脑。 **下排（左）**：故宫博物院钟表馆的铜镀金写字人钟。文字为“八方向化、九土来王”。清朝乾隆年间（1736–1796）。 **下排（中）**：ENIAC (Electronic Numerical Integrator And Computer)，1946。 **下排（右）**：Google's Sycamore (谷歌悬铃木），2019。

- 人脑(brain)

人脑本身就是一个计算和信息处理工具。巨量的神经元最终实现了从到计算、记忆、理解等功能的神经系统。这是计算神经科学(computational neuroscience)的研究领域。

人脑是一个典型的生物智能(biological intelligence)或自然智能(natural intelligence)的例子。

参考人脑神经系统的结构和运行个点来设计更好的计算工具：类脑计算(neuromorphic computing)。

• 机械驱动的计算装置

这是电子计算机出现之前的计算工具类型。比如用来执行计算的差分仪。

• 电子计算机

人类为了追求更快的计算工具，发明基于各种介质的计算工具。

超级计算机排名：Top500 → `https://top500.org/lists/top500/2025/06/`

一些单位：Giga = 10^9， Tera = 10^{12}， Peta = 10^{15}， Exa = 10^{18}。

强大的计算机可以用来做什么：国家超级计算广州中心 → `http://nscc-gz.cn/`

• 量子计算机(quantum computer)

超越经典电子计算机的量子计算器件：量子优越性(quantum supremacy)

→ `https://www.nature.com/articles/s41586-019-1666-5`

通用量子计算的路线图

当下的量子计算时代：NISQ (Noisy Intermediate-Scale Quantum) Computation

推荐阅读读物 [5, 6]

◦ 郭光灿，“量子信息技术研究现状与未来”，《中国科学：信息科学》，2020年。

◦ 潘建伟，“量子信息科技的发展现状与展望”，《物理学报》，2024年。

1.2. 编程语言

■ 编程语言能用来做什么?

按照一定的语法规则来完成字符组成的代码，以利用硬件来按照你的设计来完成计算任务。

▷ 一些常见的计算任务：

• **数值计算**：算术计算、矩阵计算等等。大家可以想象想一下代数课里有关求函数零点、高数课里的导数和定积分计算，等等。

• **模拟/仿真**：基于基本规律来实现物理过程，适合那些重复性的、或者昂贵的过程。比如通过模拟来测试高层建筑在地震或者狂风下的反应。

• **数据处理**：分析数据的统计规律、寻找数据内隐藏的规律等等。 比如利用购物平台上的顾客和购物数据，来估算出来顾客的购物习惯、并推荐顾客可能购买的商品。

• **可视化**：以二维、三维等多种形式来直观地展示数据。

■ 编程语言的流行程度

常见的两个排名：

(1) IEEE Spectrum Top Programming Languages

https://spectrum.ieee.org/top-programming-languages-2025

(2) tiobe指数

https://www.tiobe.com/tiobe-index/

1.3. 数据结构与算法

■ 数据与数据结构

数据(data)：客观事物的的符号表示，能输入到计算机中并被计算机程序处理的符号的总称。比如数字、符号、音频、图像等等。

数据结构(data structure)：相互之间存在一种或多种特定逻辑关系的数据元素的集合。

基本结构：集合、线性、树状结构、图状或网状结构。

数据结构的定义形式：二元组Data_Structure $= (D, S)$。D为数据元素的有限集；S为D元素内部逻辑关系的有限集。

例子：复数Complex $= (C, R)$。$C = \{c1, c2\}$, $R = \{\langle c1, c2\rangle\}$，为有序对⟨实部、虚部⟩。

▷ 数据的逻辑结构：数据之间的逻辑关系

▷ 数据的物理结构（存储结构）：数据结构在计算机硬件（内存）中的位置;

–顺序存储结构：借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系;

–非顺序存储结构：借助指示元素存储位置的指针(pointer)表示数据元素之间的逻辑关系。

■ 抽象数据类型

抽象数据类类型(abstract data type，ADT)实现了内部实现和外部接口的分离。它可以用来在代码上实现数据结构。

▷ 一个数学模型，以及定义在该模型上的一组操作。其定义取决于它的一组逻辑特性，而与其在计算机内部如何表示和实现无关。

抽象数据类型可用三元组表示：(D, S, P)。D为数据对象；S为D上的关系集；P为对D的基本操作。

▷ 实现了程序的模块化：模块内部是数据的表示和操作细节，模块外部使用的只是抽象的数据和抽象的操作。

数据结构通过统一接口和内部封装，分层次从整体上加以设计、实现和使用。

封装：将数据项与相关的操作结合为一个整体，并将其从外部的可见性划分为若干级别，从而将数据结构的外部性与其内部实现相分离，提供一直而且标准的对外接口，隐藏

内部的实现细节。

抽象数据类型可无关于具体的程序设计语言、具体的实现方式。 它催生了面向对象(object-oriented)程序设计语言。

■ 数据结构和算法

现代的计算机提供了数据存储以及重复计算。现代计算机语言背景下的算法例子:

▷ 排序(sorting)问题：给定元素和约定的次序，按非降序排列它们。 比如把学号按升序排列、按学生的生日按前后排列。

▷ 字符查找问题：在文档里搜索文字。

• 计算机作为一种计算工具 (Nicklaus Wirth, Turing Award 1984)

数据结构(data structures) + 算法(algorithms) = 程序(programs)

▷ 数据结构：数据在计算机系统内的组织、存储和操作，涉及到数据单元逻辑上的关系。

▷ 算法：基于特定的计算模型（数据的基本操作），旨在解决某一信息处理问题而设计的一个指令序列。

■ 使用算法求解问题的步骤:

（1）理解问题;

（2）设计算法、构建相应的数据结构;

（3）测试、调校和改进。

■ 算法的要素

以"对学生学号进行排序"这个问题来说明以下要素。

（1）**输入**(input)：待计算问题的实例。

→ 学号列表。

（2）**输出**(output)：输出问题实例的答案；可能存放在单独的存储空间，也可能直接存放在原输入所占的存储空间。

→ 排好序后的学号列表。

（3）**确定性**、**可行性**：算法可以描述为由若干语义明确的基本操作组成的指令序列，且每一基本操作在对应的计算模型里均可兑现。当且仅当算法可以通过程序设计语言（内部已经定义了基本操作）精确地描述，它满足确定性和可行性。

→ 对学号列表进行排序的算法及代码。

（4）**有穷性**和**正确性**：在执行有限次基本操作之后终止，并给出输出，而且给出的输出应该能够符合由问题本身在事先确定的条件。

→ 算法能够用有限的时间来给学号正确排序，不会出现错排、或者死循环的情况。

如何证明有穷性和单调性？寻找算法过程中的单调性和不变性。

- 单调性：问题的有效规模随算法的推进不断递减。
- 不变性：在算法初始状态下满足，而且应与最终的正确性相呼应。

（5）**退化**(degeneracy)和**鲁棒性**(robustness)：处理意外情况并保证程序的争取运行，比如处理一些意外情形下的输入实例和参数。

→ 如果输入的学号有缺失、格式错误、或者把数字当成了字符等等，算法能够给予提示、乃至处理相应的问题。

（6）**重用性**：算法的总体框架能否便捷地推广到其它场合。

→ 此学号排序算法可否用来给其它应用场景下的整数序列、乃至实数序列排序。

1.4. 日常生活中的算法

以下是一些大数据时代的若干算法例子：

（1）搜索引擎(search engine)： 当下搜索信息的主要途径包括通用搜索引擎以及比如社交网络平台等的内置搜索功能。 两个入口能够接触的信息资源有很大差别。 搜索引擎需要进行大量的线下处理过程，比如利用爬虫下载网页信息、对网页进行分析和索引。搜索引擎根据词条搜索结果、按照相关性的高低对结果进行排序并返回。 上述过程涉及到处理包含各类字符的网页信息、 索引不断增加的网页文件、选择返回结果的排列顺序等等。 这些过程都需要设计合理的数据存储结构、词条索引、和排序算法。

（2）推荐系统(recommendation systems)： 此过程常见于购物平台、视频网站、社交网络等等。 推荐系统的整体思路是在物品、服务、或账号和用户构成的二分图结构里发掘两类节点之间的潜在连接。 推荐系统需要使用算法对用户和物品进行刻画、并衡量两者之间的连接程度， 依此来估计用户的潜在需求。 刻画两者的算法可以通过用户购买和浏览物品或服务的记录、跨平台账号信息、 账号的好友信息等等多维度多来源的信息。推荐系统涉及到如何存储用户和物品的大量连接信息、 如何在多维度信息中选择重要的低维特征来描述用户和物品、 选择测度来描述物品和用户之间的相关性、并对推荐物品进行合理排序。 同时推荐系统也需要根据用户对推荐结果的反馈来动态地调整算法及其参数。

（3）移动出行平台(mobility platforms)： 常见的移动出行平台有滴滴打车、高德地图

内的打车功能等等。 出行平台连接了出行服务的两端：车辆载体和出行旅客。出行旅客下订单之后， 平台需要通过旅客所在精确地理位置和具体需求把订单分派给附近的可用车辆供司机进行抢单。 车辆接单、顾客上车后，平台需要为车辆提供到达终点的具体路线。 因为终点距离的远近、路段拥挤程度不同、沿途路段突发状况、 单行道或快速路等不同路段类型的存在等等原因，导致可能存在多种最优路线可供选择。 在这个服务问题里，实时的基于地理位置的订单的派送和锁定、最优路线的计算等等都是很直接的算法问题。

请大家自行搜索以下概念：

（4）Large Language Model（LLM，大语言模型）

（5）Scaling Laws（标度率）

（6）Foundation Model（基础模型）

（7）World Model（世界模型）

（8）Information Cocoons（信息茧房）

2.

一般情况下，基于编程的完整工作流程里一般涉及到以下三方面：

（1）**编程和计算**：选择编程语言进行计算，并直接输出计算结果，或者按照格式输出数据文件。在一些能够进行解析计算的问题，还需要事先得到一些理论预测或者近似结果。

（2）**作图和可视化**：将程序运行得到的数据绘制成图。此时需要为数据结果设置合适的组织和呈现方式，并且把数据文件按照指定格式进行输出。这样可以用来验证计算结果是否正确或者和其他结果（比如理论预测）进行对比。

（3）**编辑和生成文档**：基于计算推导和编程计算结果，使用文本生成器来整理推导公式、示意图、数据结果图等等各类信息来编辑和生成论文或报告文档。

我们按照上述三个步骤来列举常用的工具。

2.1. 工具：编程和计算

在实现代码的编辑和运行上，一般有两种场景可以选择：

（1）使用编译器(compiler)和文本编辑器(text editor)。 两者一般都为轻量级的开源软件，在使用上会相对自由。此时你可以在文本编辑器里修改代码文件，然后使用编译器对代码文件进行调试和运行。 总体上，这个组合摈弃掉了工业级开发所需要的繁复功能，可以让你更加聚焦程序本身。这个组合用法更适合编程语言的学习阶段。

（2）使用IDE(Integrated Development Environment, 集成开发环境)。 此时你可以在同一个工作环境里完成从编辑代码源文件、编译代码、到运行程序的全部流程。 IDE更适合工业级的软件开发。一般情况下，IDE是软件公司维护的大型软件。

以下以C++语言为例。

1. 编译器

推荐使用GCC(GNU Compiler Collection)。

可查阅以下链接：

→ `https://gcc.gnu.org/`

GCC的最新版本为GCC 16.2（更新于2026年8月7日）。

■ Windows系统上推荐使用编译器MinGW-w64

MinGW-w64的核心即为GCC。

可查阅以下链接：

→ `https://www.mingw-w64.org`

下载：从sourceforge中下载安装文件

→ `https://sourceforge.net/projects/mingw-w64`

文件路径：Files → mingw-w64 → mingw-w64-release

注意：MinGW-w64的最新版本为mingw-w64-v13.0.0（更新于2025年6月8日）。

如果在sourceforge上无法下载，一般可以重新选择镜像地址。此时可参考以下处理方式：

→ `https://www.cnblogs.com/xupeixuan/p/17081115.html`

下面的安装过程无需安装程序，只通过设置解压缩文件的默认路径的方式来直接运行编译器。

安装过程可参考：

→ `https://zhuanlan.zhihu.com/p/12243887447`

相应的可下载编译器文件如下列表：

→ `https://github.com/niXman/mingw-builds-binaries/releases`

安装流程如下：

(1)：下载并解压缩安装文件：下载适合Windows系统的最新版本，并解压缩到合适的路径。

(2)：设置默认路径：在解压缩后的文件里找到mingw64/bin文件夹。里面放置了GCC可使用的编译命令。拷贝这个路径，然后添加到系统的默认路径。这个做法是为了能够在命令行窗口直接调用GCC的编译命令。

设置方式：选择“电脑”图标点右键 → “属性” → “系统高级设置” → “环境变量” → “用户变量” → Path。点击Path，选择“编辑”，添加上述bin的路径名，最终保存修改。

(3)：编译和运行程序：把课堂里使用的源代码文件夹放在合适的路径。在代码文件夹里直接打开命令行界面/终端。

如何打开代码文件所在路径的命令行窗口？以下方法从易到难。请逐个试一下。

（1）在文件夹空白处点右键，在右键选项里选择“在终端中打开”。

（2）在地址栏的空白处点击，然后直接输入cmd、回车。之后命令行窗口会直接弹出。

（3）直接打开命令行窗口。此时对应文件夹为用户的默认文件夹，并不是代码文件所在的文件夹。此时你可以使用cd(chang directory)命令把默认文件夹路径改到代码所在的文件夹路径。

当你在代码文件夹打开命令行窗口后，就可以运行编译器来编译和运行程序。 对于代码源文件`xx.cc`，其编译代码的命令格式为：

```
g++ -o yy xx.cc
```

其运行代码对应可执行文件的命令格式为：

```
./yy
```

2. 文本编辑器

文本编辑器类似Windows里的文本文档。它能够识别和编辑各类后缀的文本文档。 它可以除了用来处理代码文档之外，还可以用来编辑数据文件、作图工具需要的脚本文件等等。

▶ Vim

可查阅以下链接：

→ `https://www.vim.org`

Vim的最新版本为9.2.0（更新于2026年2月14日）。

安装文件的下载页面可查阅以下链接：

→ `https://www.vim.org/download.php`

使用Vim打开文档时，你可以把文档直接拖入已经打开的Vim窗口。

▶ Emacs

可查阅以下链接：

→ `https://www.gnu.org/software/emacs/`

Emacs的最新版本为31.1（更新于2026年8月24日）。

安装文件的下载页面如下：

→ https://ftp.gnu.org/gnu/emacs/windows/emacs-31/

▶ gedit

可查阅以下链接：

→ https://gedit-text-editor.org

3. IDE

▶ Dev-C++

一个轻量级的IDE。适合C++语言的学习和简单使用。

可查阅以下链接：

→ https://sourceforge.net/projects/orwelldevcpp/

文件路径：Files → Setup Releases → Dev-Cpp 5.11 TDM-GCC 4.9.2 Setup.exe

注意：Orwell Dev-C++项目在2016年已经终止、不再更新。它的最新版5.11基于GCC 4.9.2。

安装过程可参考如下链接：

→ https://blog.csdn.net/zdoeo/article/details/145093225

安装完成后，你可以直接把代码文件拖到编辑窗口、也可以使用界面的“文件”选项打开代码文件，然后使用“运行”选项分别编译和运行程序。流程如Figure 2.1所示。

你在Dev-C++里打开代码文件后，可能会发现代码。这一般是因为代码里使用了中文，但Dev-C++无法识别。Dev-C++的版本很久没有更新，其使用的可能是比较古老的ANSI编码格式，但当下文件流行的是更先进的UTF-8格式。解决乱码问题时，可以先使用文本文档打开代码文档、把它的编码格式从UTF-8改到ANSI，之后再用Dev-C++打开。具体操作可参考以下链接：

→ https://blog.csdn.net/qq_43450413/article/details/101982006

▶ Microsoft的Visual Studio的Community版本、或者Visual Studio Code

VS为微软针对工业级开发的免费版IDE，VSCode是轻量级的IDE。

可查阅以下链接：

→ https://visualstudio.microsoft.com/zh-hans/

VSCode的安装，可参考如下链接：

→ https://blog.csdn.net/msdcp/article/details/127033151

■ 例子：奇数求和和平方

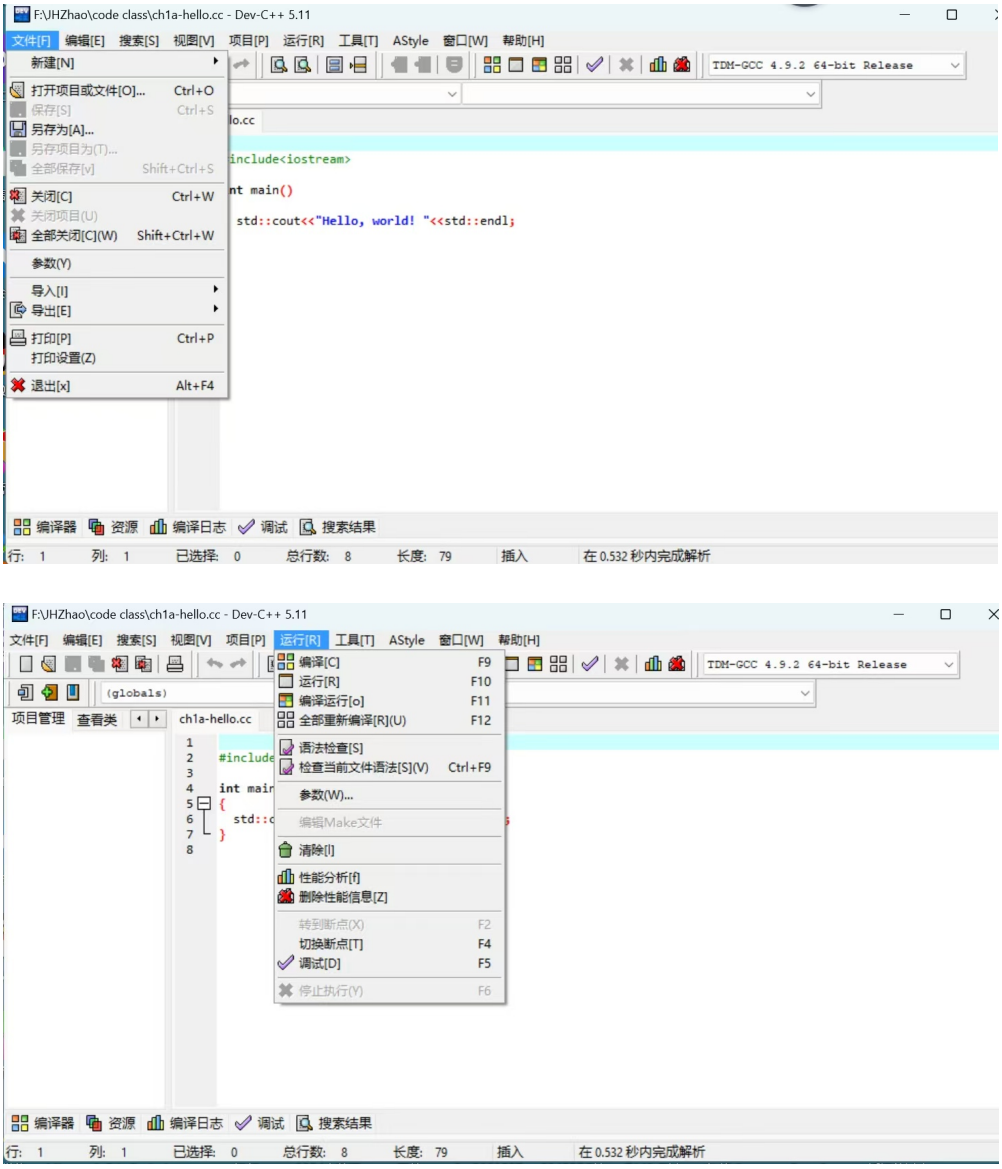


FIG. 2.1. Dev-C++的操作界面。（上图）打开已有的代码文档，或者新建代码文档。（下图）编译和运行代码。

对于奇数组成的一个序列$\{1,3,5,7,9,\cdots\}$，当我们对它们进行求和时，容易得知：

$$1 = 1^2, \tag{2.1}$$

$$1+3 = 4 = 2^2, \tag{2.2}$$

$$1+3+5 = 9 = 3^2, \tag{2.3}$$

$$1+3+5+7 = 16 = 4^2, \tag{2.4}$$

$$1+3+5+7+9 = 25 = 5^2. \tag{2.5}$$

奇数序列可以使用$\{2n-1, n \geqslant 1\}$来表示。容易证明：

$$\sum_{n=1}^{N}(2n-1) = N^2. \tag{2.6}$$

这里我们从数值计算的角度来逐个验证给定任意整数N后，其对应的奇数序列的和与平方N^2之间的关系。

请使用代码来完成下述计算任务：给定一个整数N，对于每一个$n \leqslant N$，请计算序列$\{1, 3, 5, \cdots, 2n-1\}$的和，并和$n^2$做对比。

代码可参考：`ch2a-square.cc`。

2.2. 工具：作图和可视化

程序计算的结果除了可以直接输出到屏幕，也可以以文件形式输出到硬盘。之后我们可以利用数据文件来进行作图或者可视化以直观地展示计算结果。

对于所有的作图工具，除了能够利用已有数据文件来作图，也能直接利用给定的函数直接作图。

C++语言的编译器一般不提供配套的作图工具。我们需要使用独立的作图工具。

▶ 推荐使用gnuplot

可查阅如下链接：

→ `http://www.gnuplot.info`

gnuplot的最新版本为6.0.5（更新于2026年7月）。

下载页面如下：

→ `https://sourceforge.net/projects/gnuplot`

文件的下载路径：Files → gnuplot → 6.0.5 → gp605-win64-mingw.exe

• gnuplot的使用方法

当你安装好exe文件后，可以通过以下两种方式来运行gnuplot：

（1）**通过命令行窗口运行**：选择gnuplot/bin所在文件夹，为gnuplot设置好默认路径。具体方式可参考MinGW-w64里的做法。在需要用来作图的plt文件所在文件夹里运行终端，调用gnuplot即可。

（2）**通过桌面图标来运行**：打开gnuplot的桌面快捷方式，通过“文件”选项选择并打开相应的plt文件。

上述两个过程如Figure 2.2所示。

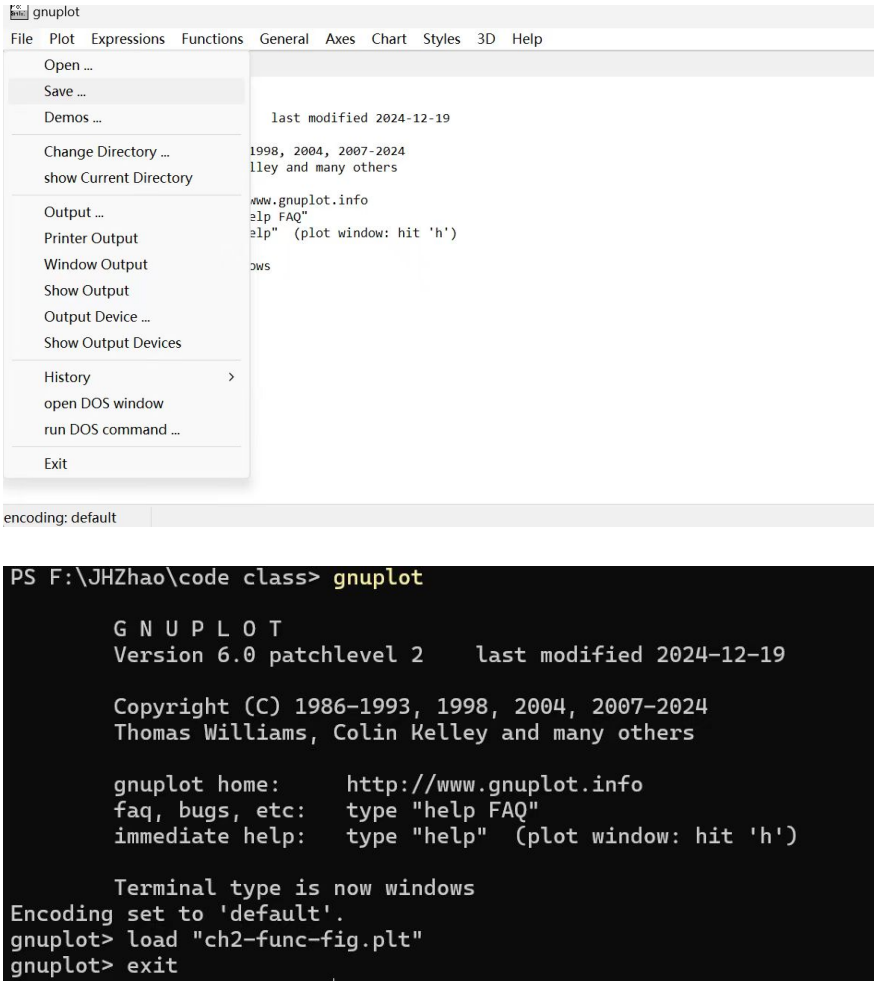


FIG. 2.2. 调用gnuplot的两种方式。（上图）在作图脚本所在文件夹里打开命令行窗口，并在窗口里直接调用gnuplot。这里展示了调用gnuplot、导入作图脚本文件、退出gnuplot的完整流程。（下图）使用图形界面的“文件”选项导入作图脚本。

- gnuplot的用法

Figure 2.3从绘图脚本`ch2b-fx-fig.plt`得来。它们给出了一个对函数本身、以及利用数据文件来做图的例子。请对比一下作图格式和相应的脚本代码。

Figure 2.4以对两组数据作图，列举了一些常见的格式设置。

使用gnuplot时，请熟悉以下格式的设置:

- 设置图的长和宽
- 设置多子图格式中各个子图的相对位置和相对大小
- 设置图例(legend)的格式(set key)
- 设置坐标轴(axis)的范围和刻度

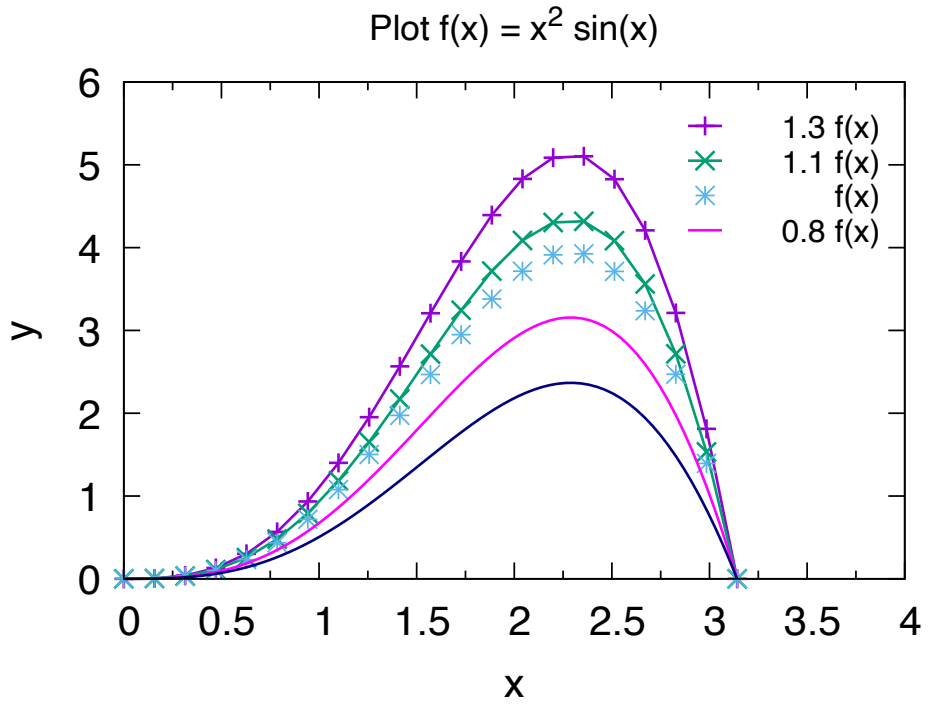


FIG. 2.3. 使用作图工具gnuplot对函数$f(x) = x^2 \sin x$进行作图。图中从上到下依次显示了$\{1.3, 1.1, 1, 0.8\} \times f(x)$对应的曲线。作图脚本中既用到了代码计算输出的数据文件，也使用了函数本身。

- 设置数据对应点状符号(points)的格式(pt, pointtype)和大小(ps, pointsize)
- 设置数据对应线条(lines)的格式(lt, linetype)、宽度(lw, linewidth)、颜色(lc, linecolor)
- 设置数据对应点线符号(linespoints)中符号和线段的格式
- 选择数据文件中相应的列来做图
- 设置或者忽略数据对应的说明(title, notitle)

2.3. 如果你是macOS用户

在macOS系统上，你可以利用软件包管理器Homebrew或者MacPorts来安装上述的程序。

- Homebrew

可查阅如下链接:

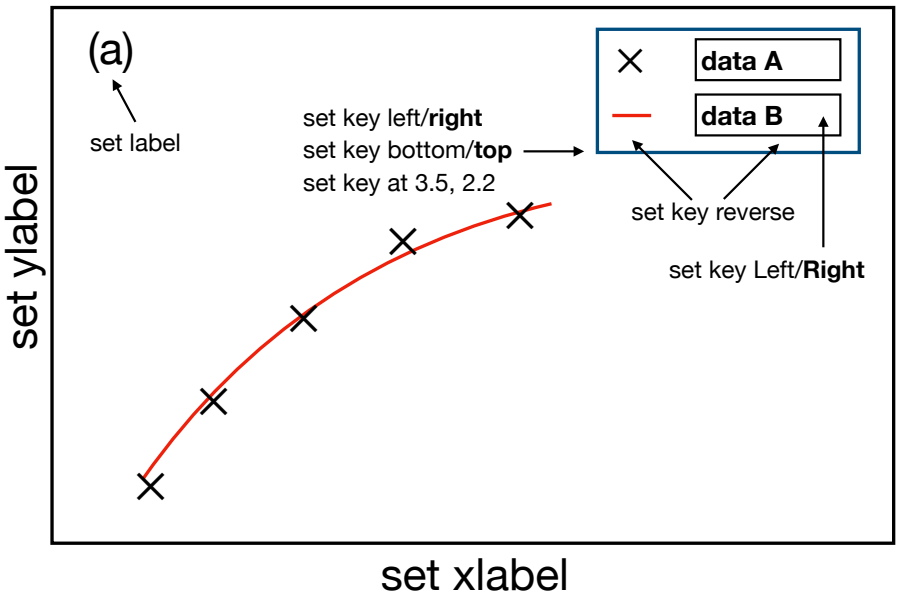


FIG. 2.4. 作图工具gnuplot中的一些格式设置。 图中涉及到坐标轴(xlabel和ylabel)的设置、图例的位置和格式的设置。 图中key设置里的黑体参数为设置的默认值。

→ `https://brew.sh`

安装后Homebrew后，使用提示的eval指令设置好调用brew命令的默认路径。

- MacPorts

可查阅如下链接:

→ `https://www.macports.org`

■ 使用Homebrew来安装工具

安装GCC: brew install gcc

安装Vim: brew install vim

安装gnuplot: brew install gnuplot

2.4. 工具：编辑和生成文档

推荐使用Tex文本生成工具。你可以使用在线形式使用Tex工具、或者在个人电脑上安装Tex工具。

- 选择在线生成文件时，推荐使用overleaf。

这是一个在线编辑、生成文档的Tex工具。无需在自己电脑上安装Tex工具。此网站提供了很多文档的模板(templates)可供选择。

请查阅如下链接:

→ `https://www.overleaf.com`

推荐参考Overleaf的官方LaTex介绍文档。链接如下:

→ `https://cn.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`

相应的中文翻译版本如下:

→ `https://blog.csdn.net/EchoooZhang/article/details/108235233`

→ `https://zhuanlan.zhihu.com/p/513218186`

• Windows系统上可使用TeX Live。

可查阅链接:

→ `http://tug.org/texlive/`

下载页面如下:

→ `https://tug.org/texlive/windows.html`

注意：最新版本为Tex Live 2026（更新于2026年3月1日）。

上述链接可以通过installer来链接镜像网站、下载安装文件来安装。注意这个安装过程会持续比较久。

如果下载通过镜像文件iso来安装Tex Live，可参考:

→ `https://blog.csdn.net/m0_67428547/article/details/158701416`

• macOS系统上可使用MacTeX，即TeX Live的macOS版本。

可查阅链接:

→ `https://www.tug.org/mactex/`

下载页面如下:

→ `https://tug.org/mactex/mactex-download.html`

注意：最新版本为MacTeX 2026（更新于2026年3月24日）。

▶ 例子：使用MacTex实现一些基本的文本编辑功能。

代码可参考文件夹：`ch2c-Tex-sample`。

注意以下常用的Tex命令:

- 在文档里实现多级目录，比如节(section)和小节(subsection)。
- 常用的数学公式，比如方程和方程组、特殊数学符号、集合、矩阵等。

- 常用的特殊符号，比如希腊字母等。
- 在文档中放入图片、调整结果图的位置和大小，并在正文中引用图片。
- 引用期刊论文、会议论文、书籍等不同类型的文献条目，并在正文中引用文献。

1. 生成演示文稿

你可以使用beamer生成演示文稿。beamer使用的也是LaTeX语言，可以认为是一个特别的tex格式。

可查阅如下链接:

→ `https://latex-beamer.com`

2.5. 在文档中插入代码

• 在tex文档中

推荐使用listings宏包。 链接如下:

→ `https://ctan.org/pkg/listings`

其使用方式可参考:

→ `https://zhuanlan.zhihu.com/p/261667944`

→ `https://blog.csdn.net/horsee/article/details/128623180`

• 在word文档中

推荐在线转制代码的工具。链接如下:

→ `https://word.wd1x.com`

使用方式可查阅以下链接:

→ `https://www.wd1x.com/word/40052.html`

上述在线工具可以实现代码的纯文本代码到类似编译器展示效果的转换。之后你可以把转制后的代码直接拷贝到word文档里。代码转制的效果如Figure 2.5所示。

2.6. 利用数据文件来作图

在课程的算法分析中，我们有不少渐近分析意义下的理论结果。比如在查找算法里，二分查找法的平均查找长度SL(n)和有序序列的长度n在渐近意义下为SL(n) = $\mathcal{O}(\log_2 n)$; 在排序算法里，起泡排序算法的时间复杂度为$T(n) = \mathcal{O}(n^2)$，归并排序算法和快速排序算

```
1.→#include<iostream>°°↵
2.→using°namespace°std;°°↵
3.→°°↵
4.→int°main()°°↵
5.→{°°↵
6.→°°cout<<"Hello,°world!°"<<endl;°°↵
7.→}°°↵
```

↵



```
1.  #include<iostream>
2.  using namespace std;
3.
4.  int main()
5.  {
6.    cout<<"Hello, world! "<<endl;
7.  }
```

FIG. 2.5. 代码从纯文本格式到word文档格式。（上图）把一个程序的纯文本形式的代码进行在线转制后，拷贝到word文档后的效果。（下图）把word文档转成pdf文档后的效果。

法的时间复杂度为$T(n) = \mathcal{O}(n \log_2 n)$，其中$n$为需要排序的向量的长度。这些定量的理论预测结果可以在给定序列上通过算法来验证。

在算法分析的场景下，通过模拟过程来验证理论结果首先需要考虑比较大的问题规模，比如元素尽可能多的序列、节点尽可能多的图。其次，有了模拟结果的数据点和理论预测的结果之后，可以通过合适的展示方式来更好地验证两者是否符合、以及寻找相应的理论预测结果的系数。

比如我们得到一组模拟结果的数据点$\{(n_1, f(n_1)), (n_2, f(n_2)), \cdots (n_m, f(n_m))\}$。作图时，最直接的做法是直接以$n$为$x$轴、以$f(n)$为$y$轴。但假如我们试图验证的关系类似$f(n) \propto cG(n)$，其中$G(n)$为$n$的函数、$c$为常系数。那么我们可以借用$G(n)$、而不是$n$来作为$x$轴。

Figure 2.6列举了算法课程里一些比较常见的理论关系。如Figure 2.6(b)所示，在验证二分查找法里平均查找长度SL(n)和序列长度n的关系时，可以使用$\log_2 n$来作为x轴。此时数据点大致呈现出线性的趋势。在数据点附近我们继续画出一条直线。当我们调整直线的斜率、使得它大致平行于数据点时，直线斜率可以视为常系数a的估计值，即在渐近意

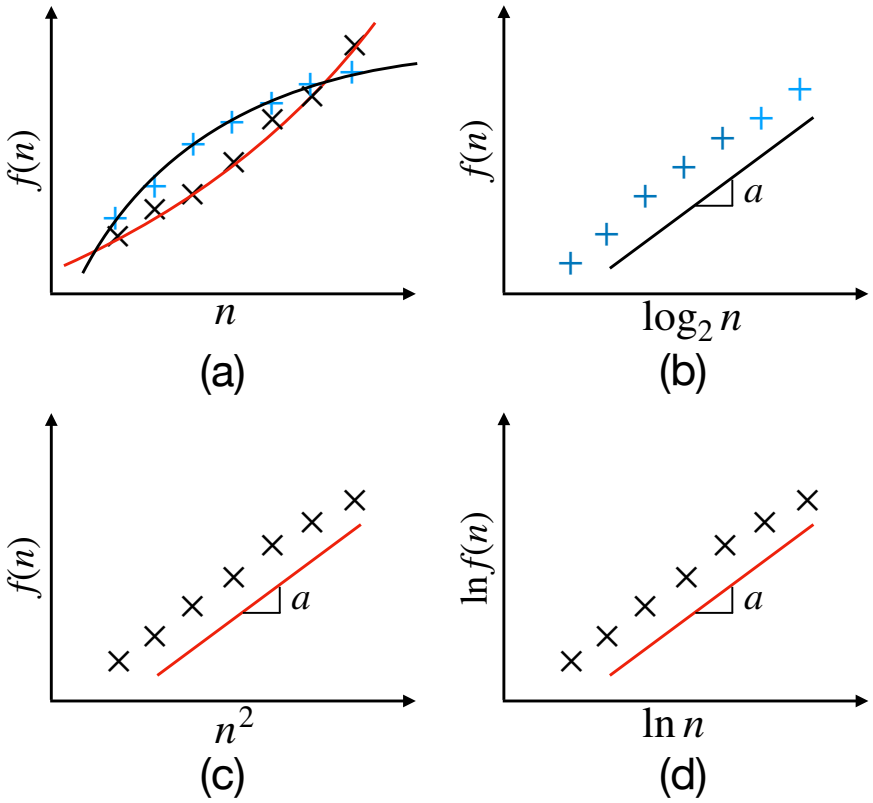


FIG. 2.6. 计算结果的作图方式。 (a)为最直接的作图方式。这里展示了两组模拟和解析结果：符号×和+标记了两类模拟结果的数据点，实线为相应的理论预测结果。 (b)把x轴写成了$\log_2 n$的形式。(c)把x轴写成了n^2的形式。(d)里x轴和y轴都写成了对数的形式。(b)-(d)里的a为直线的斜率。

义上满足$\mathrm{SL}(n) = a \log_2 n$。因为我们只考虑$n$很大时的结果，因此这里不用考虑直线的截距（对应$n = 1$的情形）。 对于Figure 2.6(c)，按照同样的逻辑，我们可以验证起泡排序算法的时间复杂度和序列长度的关系。 而在Figure 2.6(d)，我们考虑的是更加一般的多项式关系，即$f(n) \propto n^t$。此时如果多项式的阶也是未知的，可以使用如图所示的双对数坐标。相应的也可以找到相应的直线斜率a。因为$\ln f(t) \propto a \ln n$，相应地$f(t) \propto n^a$。即此时a为t的估计值。

在常见的作图流程里，我们先用程序计算结果、并输出数据文件，之后再用专门的作图工具（比如Gnuplot）导入这些数据文件来作图。程序把数据文件里的计算结果按照一列一列的格式输出，比如一列变量、一列计算结果等。为了作图方便，可以把调整之后的x轴对应的变量（比如上述$\log_2 n$、n^2、$n \log_2 n$等）作为单独的一列数据进行输出，最终选取调整之后的变量对应的一列作为x轴、计算结果对应的一列作为y轴来作图。

习题

2.1 安装常用的工具

安装以下工具。安装流程请查阅讲义。

（1）编译工具：对于C++，如果选择“编译器+文本编辑器”的方式，编译器可选择：

- Windows系统：推荐安装MinGW-w64。
- macOS系统：推荐安装GCC。

或者你可以选择IDE，比如Dev-C++。

（2）文本编辑工具：Vim。

（3）作图工具：gnuplot。

（4）文档生成工具：Tex。请按照自己的操作系统安装TexLive版本，或者使用Overleaf。作为测试，你可以试下把以下不定积分的详细推导过程用Tex写出来：

$$I = \int x^2 \sin(x) \mathrm{d}x. \tag{2.7}$$

3.

这里首先介绍下C++语言的基本概念。之后通过一些项目让大家来测试一下自己的数学和编程知识。

3.1. C++编程语言

1，基本数据类型

- 基本数据类型（bool, char, int, double）的大致内存大小和取值范围。
- bool, int, double之间的类型转换。
- 变量的基本命名规则。
- const限定符的常见使用场合。

2，表达式

- 常见的运算符及其使用，包括算术运算符、逻辑运算符、关系运算符、赋值运算符、递增递减运算符、成员访问运算符、条件运算符、位运算符、sizeof运算符、逗号运算符等。
- 注意运算符在数学和代码上的写法的差别。
- 基本的结合律和优先级。建议多用括号。
- 算术、逻辑、关系运算符的搭配使用，以及它们的优先级。

3，语句

- 条件语句(if, if-else, switch)、迭代语句(for, while, do-while)、跳转语句(continue, break)的基本概念和实现方法。
- 了解变量作用域分析的一些常见情形：全局变量、局域变量、多层循环下的变量作用域、函数定义里的变量作用域等。

4，函数

- 函数的结构和写法。函数的声明和定义。形参和实参的区别。
- 函数中参数的传递方式：传值、传指针、传引用的区别。
- 递归函数的结构和写法。其优点和缺陷。
- 函数重载的概念和实现。
- 函数中设置默认参数的常用方法。

5，数组

- 数组的定义和常用初始化方式。

- 使用下标、指针、迭代器来访问数组元素。
- 在函数中调用数组。
- 二维数组的定义和使用。

6，标准模板库

- 向量vector的定义、初始化、基本使用。
- 迭代器的概念，及其在vector中的使用。
- 字符串string的定义和使用。

7，标准输入和输出

- 用文件流的指令来实现数据文件的输出和读入。

8，结构体

- 结构体的组成、声明、定义。
- 构造函数和成员函数的写法和区别。
- 结构体对象的赋值方式。
- 如何访问结构体对象里的各个成员。
- 结构体和数组/向量的搭配使用，以及相应场景下指针和迭代器的使用。

9，类

- 类的组成、 声明、 定义。
- 构造函数和成员函数的写法和区别。
- 类对象的访问和调用方式。
- 类的访问控制符（public、private）的概念和区别。
- 结构体和类之间的区别。
- 类的定义文件作为头文件时的程序文件结构。

3.2. 自洽方程不动点的求解和分析

对于理工科里的现象或者问题，我们一般希望能够通过数学理论来进行描述。其最终的数学理论的结果往往呈现为方程的形式。但对于很多类型的方程，我们往往是没法来直接解析求解的。其中一种比较难解的方程是方程的变量构成了自洽或迭代方程(self-consistent or iterative equations)。其形式如下：

$$\vec{x} = f(\vec{x}; \vec{a}). \tag{3.1}$$

其中，$\vec{x}$为方程的变量，$\vec{a}$为可以设定的参数。 在一般情况下我们可以把$\vec{x}$和$\vec{a}$视为高维的变量或参数。在这里我们只考虑他们均为标量（一维变量或参数）的情形。 比如对于以下方程：

$$x = f(x) \equiv 2x^2 - 1. \tag{3.2}$$

它本质上是一个一元二次方程，其解析求解会比较简单。 但如果一个自洽方程有如下形式，

$$x = f(x) \equiv \mathrm{e}^{2(1-x)} + x^3 - 1. \tag{3.3}$$

容易直接看出来$x = 1$是它的一个解。但它很可能不是唯一的解。此时求解全部的解会非常困难、乃至无法进行。 对于一般的自洽方程$x = f(x; a)$，我们可以利用数值方法(numerical methods)在一定精度的意义下来求解x的值。以下我们讨论自洽方程$x = f(x; a)$的数值求解方法和不动点分析。

1. 函数零点的二分法

直观上，我们可以把求解方程$x = f(x; a)$视为求解函数$F(x, a) \equiv -x + f(x; a)$的零点解的问题。其所有零解均为对应自洽方程的不动点(fixed points)。

求方程零点的方法有很多，这里讨论最简单直观的二分法(bisection method)。我们首先把x的取值范围分成很多小区间，比如$x_0, x_1, x_2, \cdots, x_{n+1}$，其中$x_0$和$x_{n+1}$分别为取值范围的左界和右界。在我们的问题里，$x_0 = 0, x_{n+1} = 1$。在每段区间的端点上可以计算对应的$F(x_t; a), t \in \{1, 2, \cdots, n\}$。考虑一个普适的情形：$[x_L, x_R] \equiv [x_t, x_{t+1}]$。 如果$F(x_L; a)$和$F(x_R; a)$同号，则可以认为函数的零点不在这个区间； 如果$F(x_L; a)$和$F(x_R; a)$异号，则区间$(x_L, x_R)$里必有一个零点。 之后我们可以取这段区间的

中点，利用这个迭代过程不断逼近零点的位置。 考虑情形$F(x_L;a)<0$、$F(x_R;b)>0$。区间中点为$x_M=(x_L+x_R)/2$。 如果$F(x_M;a)>0$，则下一次迭代的区间为(x_L,x_M)； 如果$F(x_M;a)<0$，则下一次迭代的区间为(x_M,x_R)。 如果区间$[x_L,x_R]$内情形为$F(x_L;a)>0$、$F(x_R;a)<0$，可依类似逻辑处理。 当$F(x_M;a)$足够小时，比如$|F(x_M;a)|<\epsilon$ (例如$\epsilon\approx 10^{-8}$)时， 可认为x_M为方程$F(x;a)$的一个零点。 其过程如Figure 3.1所示。

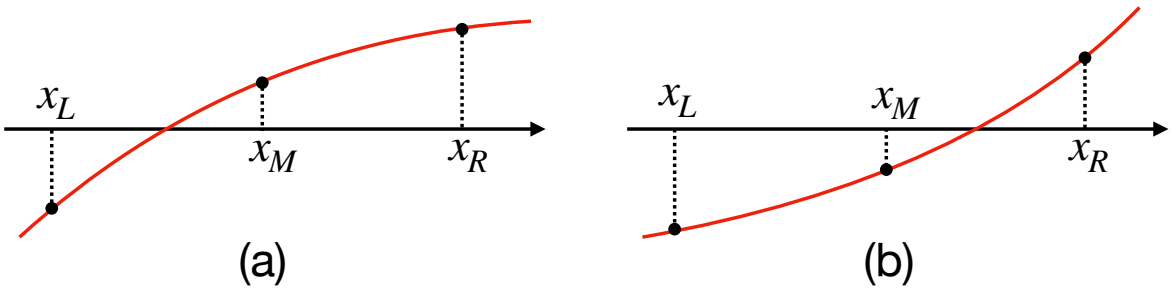


FIG. 3.1. 求解方程$F(x;a)=-x+f(x;a)=0$的二分法。 曲线对应$F(x;a)$的值。在(a)里，$[a,b]$为x范围内的一小段，且$f(a)<0$、$f(b)>0$。 取$[a,b]$的中点$t=(a+b)/2$。因为$f(t)>0$，则在二分法的下一迭代步骤取范围为$[a,t]$。 在(b)里，因为$f(t)<0$，则在下一迭代步骤取范围为$[t,b]$。

2. 自洽方程的离散序列法

除了上述函数零点的角度，我们还可以从自洽方程的含义出发，从离散迭代序列的角度来数值求解自洽方程。方程$x=f(x;a)$可以写成如下离散形式：

$$x^{(n+1)}=f(x^{(n)};a). \tag{3.4}$$

其中$n=0,1,2,\cdots$为迭代序列$\{x^{(n)}\}$的标号。给定一个初始值$x^{(0)}$，代入上式可以得到$x^{(1)}$；给定一个初始值$x^{(2)}$，代入上式可以得到$x^{(3)}$。依次类推，可以得到$x^{(4)},x^{(5)},\cdots$。如下所示：

$$x^{(1)}\leftarrow f(x^{(0)};a); \tag{3.5}$$

$$x^{(2)}\leftarrow f(x^{(1)};a); \tag{3.6}$$

$$x^{(3)}\leftarrow f(x^{(2)};a); \tag{3.7}$$

$$x^{(4)}\leftarrow f(x^{(3)};a); \tag{3.8}$$

$$\cdots \quad \cdots \tag{3.9}$$

$$x^{(n+1)} \leftarrow f(x^{(n)}; a); \tag{3.10}$$

对于一些形式的$f(x;a)$，当如果n足够大时，序列$x^{(n)}$可能趋近于某个具体的值。在迭代序列满足$|x^{(n+1)} - x^{(n)}| < \varepsilon$（比如$\varepsilon = 10^{-8}$），我们可以中断迭代过程，并认为序列收敛于$x^{(n)}$。此时，$x^{(n)}$即为自洽方程的一个不动点(fixed point)。

代码层面上，我们如何从一个初始值$x^{(0)}$来找到它对应的迭代序列的不动点?

代码里我们通过for循环语句来实现这个过程。为了判断迭代序列是否收敛，代码设置了迭代最大次数为N_iter、收敛判据中的小数为eps（对应ε）。循环过程之前，x的初始值为x0（对应题$x^{(0)}$）。每次for循环开始后，首先使用当前的x计算函数值newx= $f(x;\vec{a})$。之后比较当前的x和newx的差的绝对值。如果绝对值小于eps，则此迭代序列收敛，循环即可退出；否则，继续更新x为newx，开始下一次循环。这个循环过程如果在N_iter之前退出循环，当下的x即为序列收敛值（对应不动点x^*）、iter为序列收敛时的上标t。如果循环运行了最大循环次数（通过iter是否等于N_iter来判断），则说明在设定的eps情况下迭代序列并不收敛，此时输出"序列未收敛"的提示信息。后续可以设置更大的N_iter。

代码层面上，如何判断找到的不动点是新的不动点?

在计算给定给定参数对应的不动点时，我们首先使用不同的$x^{(0)}$来计算它们对应的所有收敛不动点x^*。给定不同$x^{(0)}$后，如果迭代序列收敛，可能得到不同的收敛值x^*。但如果它是一个稳定不动点，则计算得到的多个收敛值本质上是同一个不动点。 为了解决这个问题，我们使用一个向量来存储使用已有的$x^{(0)}$找到的所有收敛不动点x^*。每次通过$x^{(0)}$计算得到一个x^*时，我们会首先遍历这个向量、并和已有的不动点进行比较：如果它和某个已有不动点足够接近（比如设定阈值为$100 \times \epsilon$），则我们认为新得到的不动点和某个已有不动点其实是同一个；但如果新得到的不动点离已有的所有不动点都超过了设定的阈值，则认为它是一个新的不动点，并把它放入向量中。

3. 不动点的稳定性分析

为了更直观地展示迭代序列$\{x^{(t)}\}$的演化过程， 我们将一维的迭代值$\{x^{(t)}, t \geqslant 0\}$的演化过程在二维平面上展示出来。具体来说，我们可以计算并依次输出了迭代过程中的中间数据点$(x^{(t)}, x^{(t+1)}), (x^{(t+1)}, x^{(t+1)}), t \geqslant 0$，并把它们用直线连结起来。一维序列$\{x^{(t)}, t \geqslant 0\}$的演化体现在数据点在$y = f(x;a)$上的连续移动。 Figure 3.2给出了不同情形下迭代序列对应的演化过程。

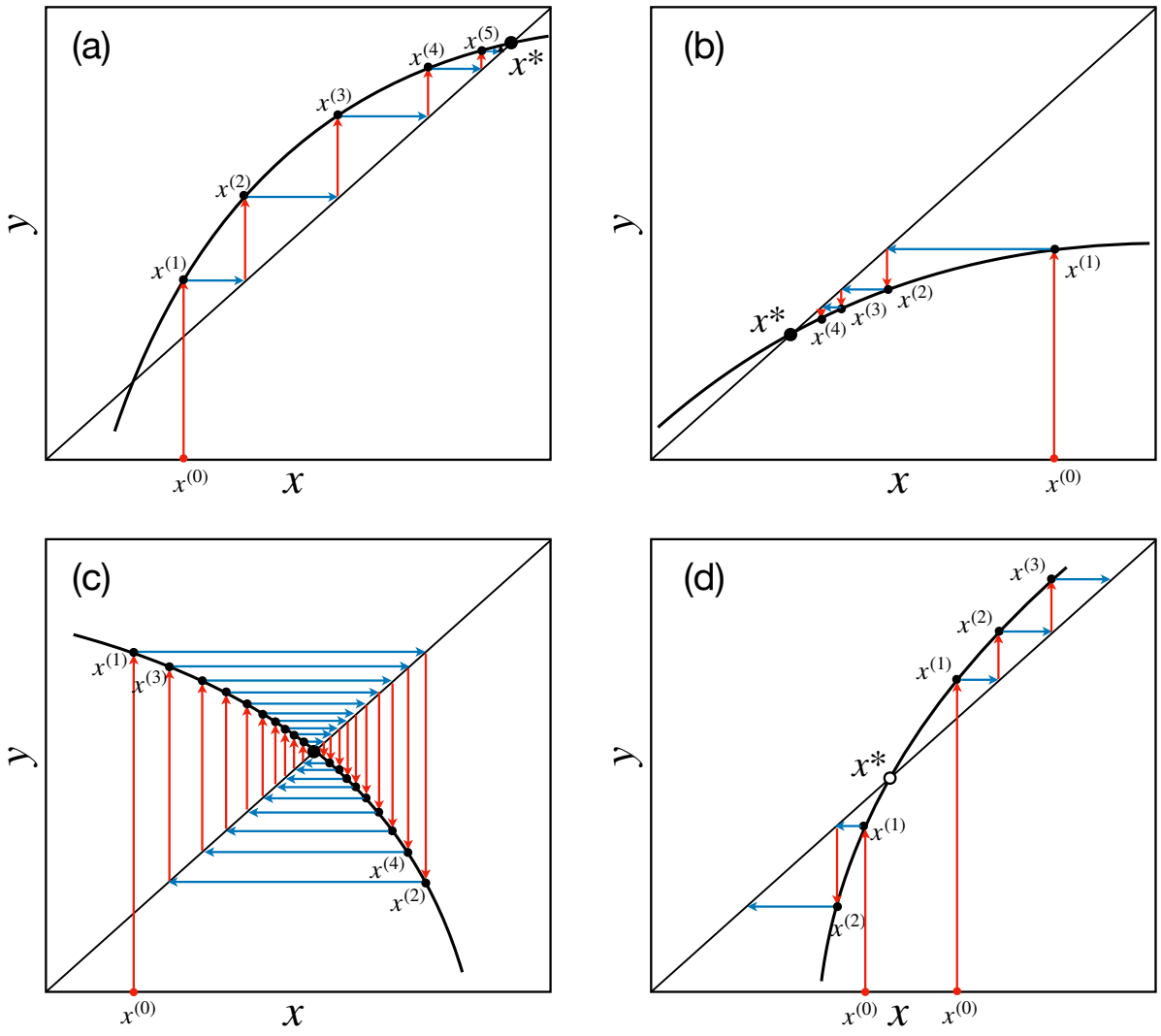


FIG. 3.2. 求解自洽方程$x = f(x; a)$的离散序列法。 图中曲线对应着$y = f(x; a)$，对角直线对应$y = x$。两者的交点即为方程的不动点，由x^*表示。 自洽方程的离散序列$\{x^{(i)}|i \geqslant 0\}$以$x^{(0)}$为初始值。图中红色垂直直线对应着$x = x^{(n)}$，蓝色水平直线对应着$y = f(x^{(n)}; a) = x^{(n+1)}$。 图中$x^{(0)}$表示序列初始值的横坐标，$x^{(n)}(n \geqslant 1)$表示二维坐标点$(x^{(n-1)}, x^{(n)})$。 沿着曲线$y = f(x; a)$，我们可以从折线的位置变化读取$x^{(n)}$的趋势。 易知，(a)-(c)中迭代序列都趋近于$x^*$，(d)中迭代序列都远离了$x^*$。

容易可知，利用二分法可以找到自洽方程的所有零点，即不动点。 利用离散迭代序列我们可以找到自洽方程的不动点，但可能不是所有的不动点。 Figure 3.2(a)中，点x^*对应着方程的一个不动点。其实在x^*附近的区域选择多个值作为$x^{(0)}$，其对应迭代序列都可以收敛于x^*。 但对于Figure 3.2(d)中的不动点x^*，如果在x^*附近的区域选择一个

值$x^{(0)}$，只要$x^{(0)} \neq x^*$，其迭代序列逐渐远离x^*。也就是说如果要利用离散迭代序列发来找到x^*作为一个不动点，需要精确地选择x^*作为离散序列的初始值。 我们把前一类不动点叫做稳定不动点(stable fixed point)，后一类叫做不稳定不动点(unstable fixed point)。用更精确的语言，我们可以通过不动点的局域微扰性质(local perturbative property of the fixed points)来对不动点进行分类： 对于自洽方程的某个不动点x^0，现在对它做一些扰动$x^{(0)} = x^0 + \delta x$，其中δx为某个很小的数；把初始值$x^{(0)}$代入迭代序列，如果序列依然收敛于x^0，则x^0为稳定不动点；如果序列偏移了x^0，则x^0为不稳定不动点。

容易得知，不动点x^*是否稳定依赖于$\partial f/\partial x|_{x=x^*}$的取值。当

$$-1 < \frac{\partial f}{\partial x}|_{x=x^*} < 1 \tag{3.11}$$

时，x^*为稳定的，否则为不稳定。 Figure 3.2(a - c)中的三个不动点都为稳定不动点，(d)中对应一个不稳定不动点。

这里对于不动点在微扰意义下的稳定性给出一个直观的解释。 想象成在光滑曲线上放置直径很小的一个球形。我们一般可以找到一个导数为0的位置以放置球体，比如上凸部分的顶点和下凹部分的底部。曲线和球形接触的位置可以理解为迭代方程的零点。 对于停在下凹曲线的底部的球，经过轻微的扰动之后，如果考虑摩擦，球体经过若干次滑动之后依然会回复到曲线底部。 但对于放置在上凸曲线顶点的球，如果把球的位置稍微晃动一下，则球会马上滑离顶点、不会回到并停留在原来的位置。 此时下凹曲线的底部可以视为一个方程的稳定不动点，上凸曲线的顶点可以视为方程的不稳定不动点。

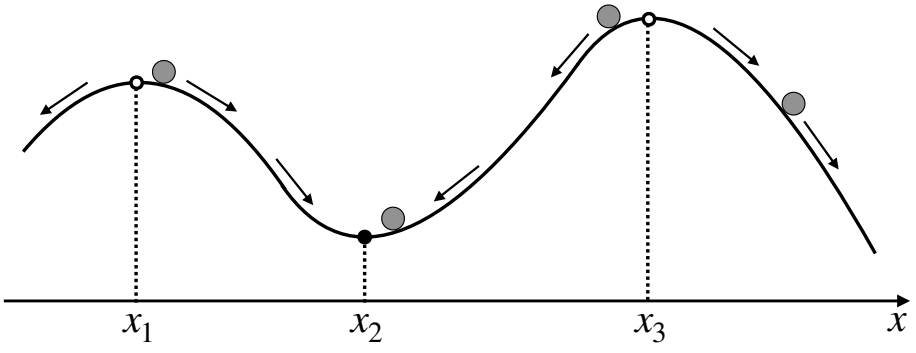


FIG. 3.3. 自洽方程的不动点的直观解释。 图中曲线代表一个可供球体移动的平面，横轴x代表位置。图中深灰色圆形表示平面上移动的球，箭头表示球沿着曲面斜率而移动的方向。容易得知x_1、x_2、x_3都为球在平面上的平衡点。但其中只有x_2对应的位置才是稳定的，而x_1和x_3是不稳定的。

在真实的物理系统中，假设某个系统的动力学过程能被某个含时的自洽方程所描述。如果此系统处在稳定态，那么系统能够通过其自洽方程的稳定不动点来描述。一般来说，自洽方程可能有不止一个稳定不动点。但在物理系统里，我们往往只能看到一个状态。原因在于不同稳定不动点对应的某些物理量（比如能量）很不一样，导致对应物理状态出现的概率相差极大。

给定自洽方程$x = f(x; a)$的参数a，我们可以数值得到其稳定不动点x^*。连续的稳定不动点可以构成一条分支(branch)。当调节参数a时，相应的稳定不动点x^*的值以及个数都可能发生变化。稳定不动点的数目随着系统参数的变化会有多种情形，比如有的稳定不动点可能会消失，或者会出现新的稳定不动点等等。后者也就是稳定不动点的分叉现象(branching phenomena)。无论是那种情况，稳定不动点的变化对应着自洽方程所描述的物理系统内部发生了剧烈的、根本性的变化，往往对应着物理系统发生了相变现象(phase transitions)。发生这种相变现象的系统参数a^*也就是系统的临界点(critical point)。

4. 例子：连续相变和不连续相变

这里考虑以下自洽方程的求解：

$$x = 1 - \mathrm{e}^{-cx} \sum_{k=0}^{K-2} \frac{(cx)^k}{k!} \equiv f(x; K, c), \tag{3.12}$$

其中实数变量$x \in [0, 1]$，整数参数$K \geqslant 2$，实数参数$c \geqslant 0$。

下面我们将分别考虑$K = 2$和$K = 3$的情形。 可知$K = 2$时，我们有：

$$x = 1 - \mathrm{e}^{-cx} \equiv f(x; c). \tag{3.13}$$

Figure 3.4给出了$K = 2$时$x = f(x; K, c)$的分析结果。 在一般情况下，如果把曲线$y = f(x; K, c)$和直线$y = x$画在一个图里，两者相交的点就是自洽方程的不动点，此时可以直观地判断不动点的个数和大致位置。 从Figure 3.4(a)中，容易得知，$x_1^* = 0$一直为方程的不动点； 当$c < 1$时，自洽方程只有一个不动点$x_1^* = 0$； 当$c > 1$时，除了$x_1^* = 0$，自洽方程有了第二个不动点$x_2^* > 0$。 Figure 3.4(b)(c)给出了在不同的c下，给定$x^{(0)}$后迭代序列的演化。 可知，当$c < 1$时，$x_1^* = 0$为方程的稳定不动点。而当$c > 1$时，第二个不动点$x_2^* > 0$才是稳定的。此时只有当精确地取初始值$x^{(0)} = 0$时，才能找到不动点$x_1^* = 0$。此时$x_1^* = 0$变成了不稳定不动点。 Figure 3.4(d)给出了在不同的c下，利用迭代序列求得的

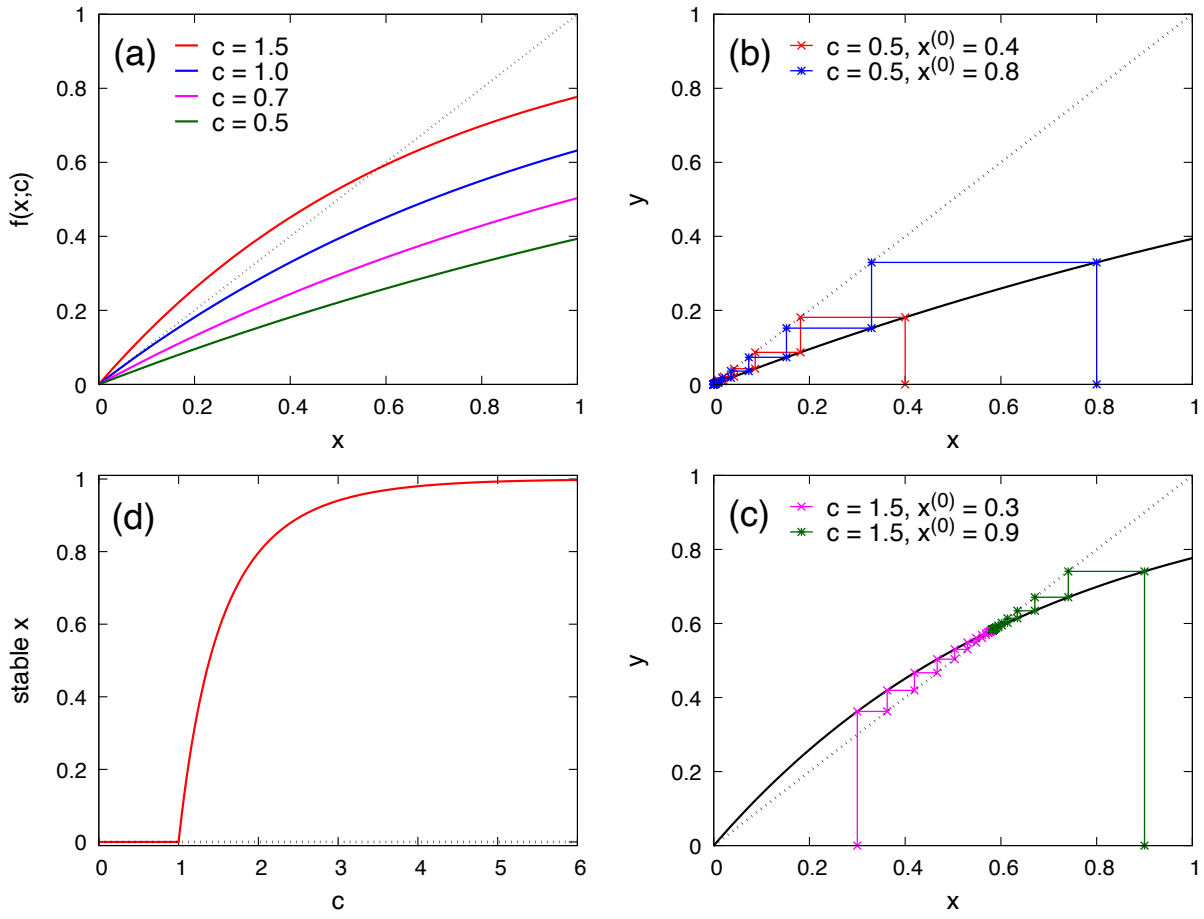


FIG. 3.4. 自洽方程$x = f(x; K, c)$（$K = 2$）的不动点分析。 (a)展示了$f(x; K, c)$在给定不同c后的结果。 实线对应函数$y = f(x; K, c)$，对角线虚线对应$y = x$。 (b)展示了在$c = 0.5$时，给定不同$x^{(0)}$后，迭代序列的变化。 可知$x^{(0)} = 0.4$和$x^{(0)} = 0.8$对应的迭代序列均收敛于$x_1^* = 0$。 (c)展示了在$c = 1.5$时，给定不同$x^{(0)}$后，迭代序列的变化。 可知$x^{(0)} = 0.3$和$x^{(0)} = 0.9$对应的迭代序列均收敛于$x_2^* \approx 0.582812$。 (d)展示了不同c对应的自洽方程的稳定不动点x^*。

所有稳定不动点。可知对于不同的c，稳定不动点只有一个分支：当$c < 1$时，方程只有一个稳定不动点$x = 0$；当$c > 1$时，稳定不动点从0开始不断增加、并趋近于1。

Figure 3.5给出了$K = 3$时$x = f(x; K, c)$的分析结果。 Figure 3.5(a)从下到上依次展示了$c = \{2, 3, 3.350, 4\}$时的$y = f(x; K, c)$，可知它们分别对应1个、1个、2个、3个不动点。 同样容易得知，$x_1^* = 0$一直为方程的不动点。 Fig.3.5(b)(c)给出了在不同的c下，给定$x^{(0)}$后迭代序列的演化。 可知当$c = 2.5$时，$x_1^* = 0$为方程的稳定不动点。当$c = 3.5$时，选取不同的初始值$x^{(0)}$能够得到不同的收敛值。但可以确定的是此时只有两个收敛值：一个为$x_1^* = 0$、一个为$x_2^* \approx 0.708579$。此时注意到其实方程还有一个不动点$x_3^* \approx 0.365$。但

就像$K=2, c>1.0$时的不稳定不动点$x_1^*=0$一样，只有精确地选择序列初始值$x^{(0)}$为这个值时，才能找到这个不动点。此时$x_3^*\approx 0.365$对应着不稳定不动点。 Figure 3.5(d)给出了不同c时的所有稳定不动点。可知当$c<c^*\approx 3.350$时，方程只有一支稳定不动点$x_1^*=0$；当$c>c^*$时，不动点$x_1^*=0$依然是稳定的，同时第二支稳定不动点x_2^*突然出现。第二支稳定不动点的起点为$x_2^*\approx 0.579378$，并随着c的增加逐渐趋近于1。注意此时第二支稳定不动点的出现并不像$K=2$情形下从$x_1^*=0$逐渐增加的，而是呈现出来一个跳变。

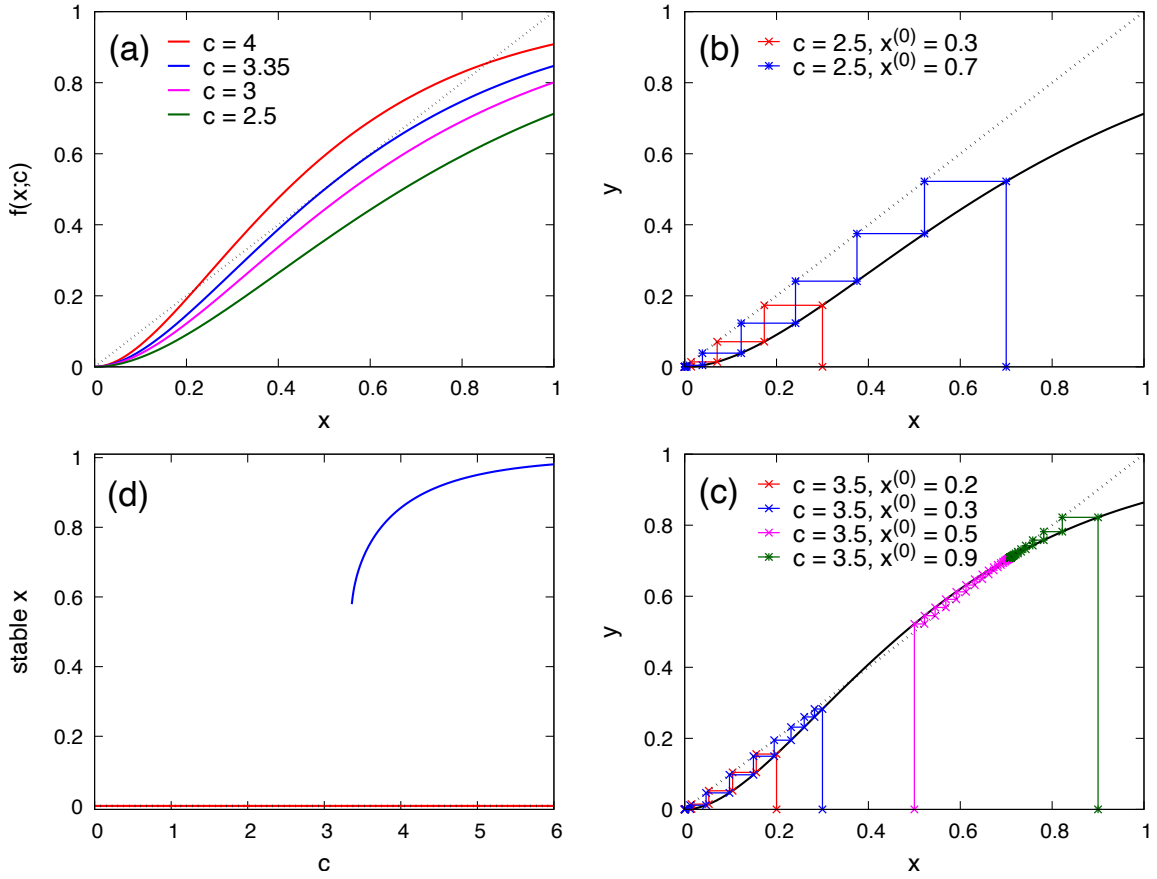


FIG. 3.5. 自洽方程$x=f(x;K,c)$（$K=3$）的不动点分析。 (a)展示了$f(x;K,c)$在给定不同c后的结果。 实线对应函数$y=f(x;K,c)$，对角线虚线对应$y=x$。 (b)展示了在$c=2.5$时，给定不同$x^{(0)}$后，迭代序列的变化。 可知$x^{(0)}=0.3$和$x^{(0)}=0.7$对应的迭代序列均收敛于$x_1^*=0$。 (c)展示了在$c=3.5$时，给定不同$x^{(0)}$后，迭代序列的变化。 可知$x^{(0)}=0.2$和$x^{(0)}=0.3$对应的迭代序列收敛于$x_1^*=0$，而$x^{(0)}=0.5$和$x^{(0)}=0.9$对应的迭代序列收敛于$x_2^*\approx 0.708579$。 (d)展示了不同c对应的自洽方程的稳定不动点x^*。 两种颜色的实线代表了稳定不动点构成的两条分支。

上述$K=2$和$K=3$对应的不动点变化行为的不同可以有定量解释，相应的临界点c^*也

可以精确求解。可知，自洽方程$x = f(x;K,c)$的不动点的数目和类型本身依赖于函数本身。 函数$f(x;K,c)$的导数为

$$\begin{aligned}\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x} &= c\mathrm{e}^{-cx}\sum_{k=0}^{K-2}\frac{(cx)^k}{k!} - c\mathrm{e}^{-cx}\sum_{k=0}^{K-2}\frac{(cx)^{k-1}}{(k-1)!} \\ &= c\mathrm{e}^{-cx}\frac{(cx)^{K-2}}{(K-2)!}.\end{aligned} \tag{3.14}$$

■ 当$K = 2$时，

$$\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x} = c\mathrm{e}^{-cx}. \tag{3.15}$$

容易得知，$\mathrm{d}f(x;K,c)/\mathrm{d}x$随着$x$的增加而减少。 可知$\mathrm{d}f(x;K,c)/\mathrm{d}x$在$x = 0$时最大，即

$$\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x}\Big|_{x=0} = c. \tag{3.16}$$

由Fig.3.4(a)可知，不动点的数目和类型依赖于$x = 0$时$f(x;K,c)$的变化趋势。

（1）当斜率$\mathrm{d}f(x;K,c)/\mathrm{d}x|_{x=0} < 1$（对应$c < 1$），曲线$f(x;K,c)$整体在直线$y = x$以下。 此时方程只有一个不动点$x_1^* = 0$，且为稳定不动点。

（2）当斜率$\mathrm{d}f(x;K,c)/\mathrm{d}x|_{x=0} > 1$（对应$c > 1$），曲线$y = f(x;K,c)$和$y = x$出现了第二个交点。此时方程有了$x_1^* = 0$之外的第二个不动点$x_2^* > 0$。 由迭代序列的演化结果可知，$x_1^* = 0$变成不稳定的，而$x_2^*$是稳定的。随着$c$的增加，$\mathrm{d}f(x;c)/\mathrm{d}x|_{x=0}$逐渐增加，稳定不动点$x_2^*$趋近于1。

（3）上述两种情况的临界参数c^*满足条件：

$$\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x}\Big|_{x=0,c=c^*} = c|_{c=c^*} = 1. \tag{3.17}$$

即$c^* = 1$。 当$c = c^*$时，自洽方程有两个简并(degenerate)不动点$x_1^* = x_2^* = 0$。

■ 当$K = 3$时，$f(x;K,c)$在$x = 0$时导数为

$$\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x}\Big|_{x=0} = 0. \tag{3.18}$$

这使得$x_1^* = 0$一直为方程的稳定不动点。 此外从Fig.3.5(a)中可知，曲线$f(x;K,c)$中部有一个凸起。当c不断增加时，这个凸起会越来越高，最终导致$f(x;K,c)$相切于$y = x$。此时的相切点x^*显然为一个非零的值，对应着第二支稳定不动点的出现是突然的。此时临界点c^*需要满足函数值和函数斜率的两个要求：

$$f(x;K,c)\Big|_{x=x^*,c=c^*} = x^*, \tag{3.19}$$

$$\frac{\mathrm{d}f(x;K,c)}{\mathrm{d}x}\Big|_{x=x^*,c=c^*} = 1. \tag{3.20}$$

上述两个方程为关于函数(x^*, c^*)的方程组，可以求解得到相应的解$c^* \approx 3.350$。

这里总结一下不同c下，函数不动点的变化情况:

（1）当$c < c^*$时，$x_1^* = 0$为唯一的不动点、且为稳定的。

（2）当$c > c^*$时，除了$x_1^* = 0$依然为稳定不动点，还出现了第二和第三个不动点x_2^*和$x_3^*(< x_2^*)$，但只有其中的x_2^*是稳定的。同时，因为自洽方程的具体形式，导致x_2^*和x_3^*的出现为一个突然的跳变、而不是从$x_1^* = 0$连续增加而得来。

（3）临界参数c^*的计算可以利用Eqs. (3.19)和(3.20)。当$c = c^*$时，自洽方程有了三个稳定不动点$\{x_1^*, x_2^*, x_3^*\}$，且其中两个为简并不动点$x_2^* = x_3^* = x^*$。

3.3. 投针问题

投针问题是经典的几何概率（geometric probability）问题[7]。以下是投针问题的两个常见版本：

• **蒲丰投针问题**(Buffon's Needle Problem, BNP)：给定一个无穷大的平面，我们不断随机地把一根针投在平面上。数学意义上，针的长度为l、宽度为0。假设平面上有着等距离的一组平行线，其间距为d。

• **蒲丰-拉普拉斯投针问题**(Buffon-Laplace Needle Problem, BLNP)：在一个无穷大的平面上，我们不断随机地把一根针投在平面上。数学意义上，针的长度为l、宽度为0。考虑平面上有着相互垂直的、等距离的两组平行线，其间距分别为a和b。为简单起见，可以设置$a \geqslant b$。

两个问题的示意图可见Figure 3.6(a)和(b)。蒲丰投针问题由蒲丰（G.-L. Buffon）于1777年提出来[8]。蒲丰-拉普拉斯投针问题由拉普拉斯（P.-S. Laplace）于1812年提出[9, 10]。他们在最初都解决了短针情形（即$l < d$和$l < b$）时的解析结果。

对于投针问题，我们可以研究针和线相交情况的统计性质，比如针和平行线相碰的概率、针和线相交数目的分布等等。其中前者是投针问题里最基本的性质。这两个问题是蒙特卡洛方法（Monte Carlo Method）[11]的一个典型应用。

那么如何处理这个问题?

首先我们按照平行线的方向来选定坐标系。两种情况下我们都不用考虑平行线和针的宽度。直观上，在每一次投针实验步骤里，我们随机往平行线或者网格的平面上投针，然后统计针线相碰的实验次数，并最终计算其相碰概率。但在具体实现上，请思考以下一些问题：

• 问题最初定义在一个无穷大的平面上。这里如何使用有限内存来表示?

• 针的位置如何表示? 针的随机位置如何生成?

• 针和线相碰的条件如何表示?

• 最终的碰撞概率的表达形式是怎样的?

下面我们给出详细的推导和计算过程。感兴趣的同学可以参考论文[12]。链接可见：

→ `https://www.sciencedirect.com/science/article/pii/S0378437124007623`

1. 投针问题的几何描述

这里我们用一个统一的几何框架来描述针和平行线的相对位置。为简单起见，我们考

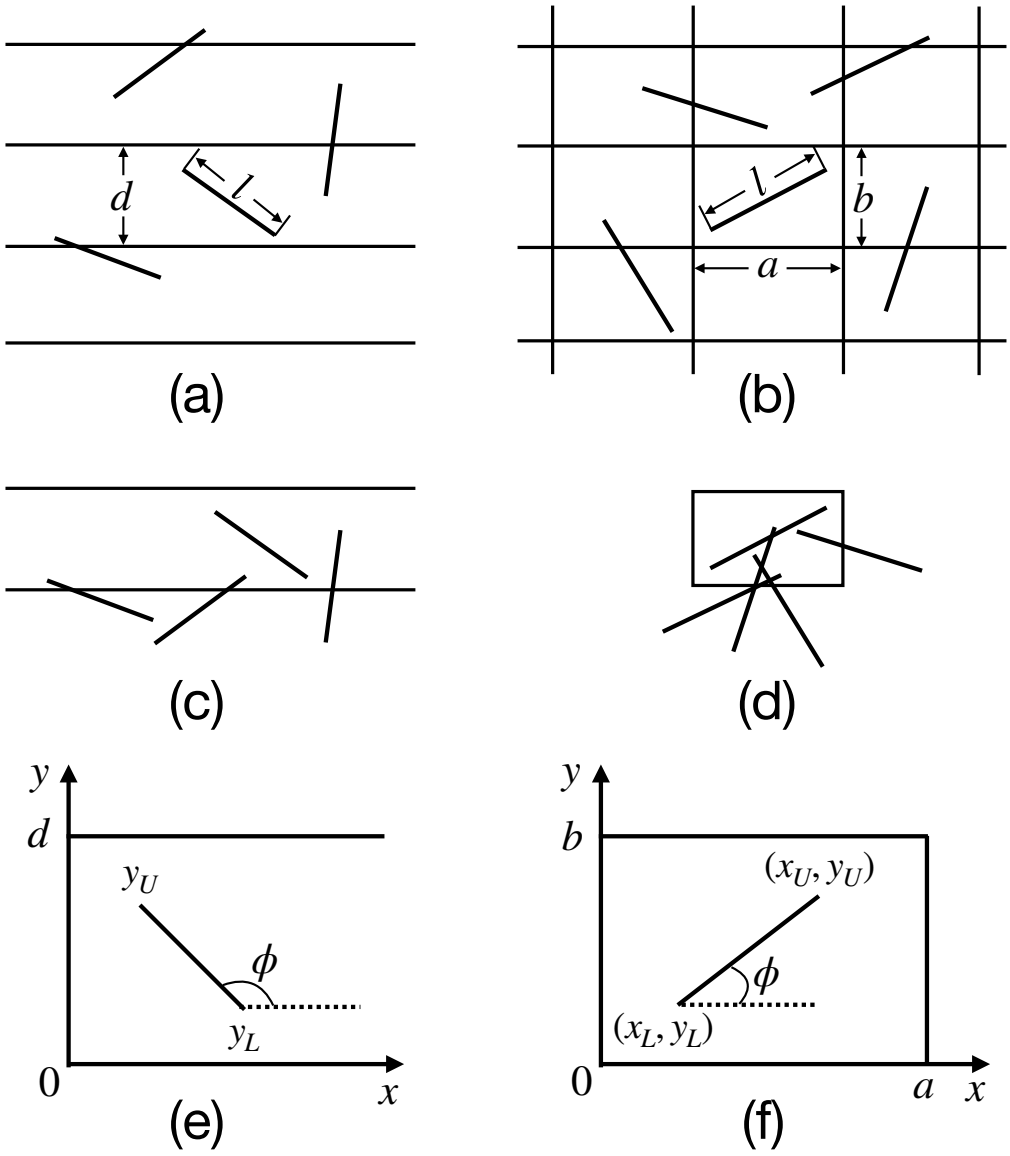


FIG. 3.6. 投针问题的描述。 (a)蒲丰投针问题的图示。针的长度为l，平行线的间隔为d。可知4根投针中有3根与平行线相碰。 (b)蒲丰-拉普拉斯投针问题的图示。平行线在横向和纵向的间隔分别为a和b。可知5根投针中有4根与平行线相碰。 (c)和(d)分别展示了把(a)和(b)中所有针挪到同一个平行线间隔或者网格后的情形。 (e)展示了蒲丰投针问题中如何在单个平行线间隔里描述一根针的位置。y_{U}和y_{L}分别表示针的上端点、下端点在y轴上的坐标。ϕ为针相对于x轴的角度。 (e)展示了蒲丰-拉普拉斯投针问题中如何在单个网格里描述一根针的位置。$(x_{\mathrm{U}}, y_{\mathrm{U}})$和$(x_{\mathrm{L}}, y_{\mathrm{L}})$分别表示针的上端点、下端点的二维坐标。$\phi$为针相对于$x$轴的角度。

虑网格情形，并把平行线情况当作网格情形的一个特例，即其中一组平行线的间距为无穷大。 对于针线相碰这个事件来说，真正重要的并不是针在坐标系里的实际位置，而是针相对于平行线的位置。针的位置在网格里的平移并不影响针和线是否相碰。在无穷大平面上投针时，最终我们只需要考虑针在单个网格里的位置。如Figure 3.6(c)-(d)所示。 对于平行线情形，我们沿着平行线方向为x轴，和它垂直的方向为y轴。 对于网格情形，我们沿着间隔为b的平行线方向作为x轴、沿着间隔为a的平行线方向作为y轴。 原点为两条垂直直线的一个相交点。

对蒲丰投针问题，我们只需要考虑针和一组平行线之间的相对位置。此时只需要两个位置参数来描述针的构型。这里选择位置参数为针的上端点y_{U}、针相对于x轴的角度ϕ。如Figure 3.6(e)所示。两者的取值范围分别为$y_{\mathrm{U}} \in [0, d)$、$\phi \in [0, \pi)$。为了生成针的随机构型，我们可以借助随机数生成器Rand()。每次调用Rand()都会返回一个$(0, 1)$之间的均匀分布的实数。 针的随机构型的生成过程可以写成

$$\begin{bmatrix} y_{\mathrm{U}} \\ \phi \end{bmatrix} = \begin{bmatrix} d \times \mathrm{Rand}() \\ \pi \times \mathrm{Rand}() \end{bmatrix}. \tag{3.21}$$

针的下端点y_{L}相应地为

$$y_{\mathrm{L}} = y_{\mathrm{U}} - l \sin \phi. \tag{3.22}$$

注意此时$y_{\mathrm{U}} \in [0, d)$，则针和线相碰的条件为

$$(y_{\mathrm{L}} < 0) \vee (d < y_{\mathrm{L}}). \tag{3.23}$$

注意此时因为$y_{\mathrm{U}} \in [0, d)$、且$\phi \in [0, \pi)$，上式的$d < y_{\mathrm{L}}$其实是不满足的。

在蒲丰-拉普拉斯投针的网格情况下，二维平面上一根针的位置可以通过三个位置参数来表示：针的每个端点需要两个参数来描述；两个端点之间距离为l，导致需要减去一个参数。本质上这个三参数可以有多个选择。这里我们选择针的上端点在x-轴的坐标x_{U}、上端点在y轴的坐标y_{U}、针相对于x轴的倾斜角。如Figure 3.6(f)所示。参数的取值范围为$x_{\mathrm{U}} \in [0, a)$, $y_{\mathrm{U}} \in [0, b)$, $\phi \in [0, \pi)$。过程如下：

$$\begin{bmatrix} x_{\mathrm{U}} \\ y_{\mathrm{U}} \\ \phi \end{bmatrix} = \begin{bmatrix} a \times \mathrm{Rand}() \\ b \times \mathrm{Rand}() \\ \pi \times \mathrm{Rand}() \end{bmatrix}. \tag{3.24}$$

针的下端点$(x_{\mathrm{L}}, y_{\mathrm{L}})$相应地为

$$\begin{bmatrix} x_{\mathrm{L}} \\ y_{\mathrm{L}} \end{bmatrix} = \begin{bmatrix} x_{\mathrm{U}} - l \cos \phi \\ y_{\mathrm{U}} - l \sin \phi \end{bmatrix}. \tag{3.25}$$

注意此时针的位置里，$x_{\mathrm{U}} \in [0, a)$， $y_{\mathrm{U}} \in [0, d)$。则针和线相碰的条件为

$$\begin{aligned} &(x_{\mathrm{L}} < 0) \vee (a < x_{\mathrm{L}}) \\ \vee &(y_{\mathrm{L}} < 0) \vee (b < y_{\mathrm{L}}) . \end{aligned} \tag{3.26}$$

类似于蒲丰投针问题，此时因为$y_{\mathrm{U}} \in [0, d)$、且$\phi \in [0, \pi)$，上式的$d < y_{\mathrm{L}}$同样是不满足的。

2. 投针问题的模拟实现

在投针问题的模拟过程中，我们利用上述的几何描述来实现。

对于蒲丰投针问题，我们利用Eq.(3.21)可以不断随机生成N个针的构型(y_{U}, ϕ)。这对应着N次的投针实验。对于其中的每一个构型，利用Eq.(3.22)得到其对应的y_L。此时可以利用针线相碰的判据Eq.(3.23)来判断这个构型和平行线是否相碰。如果针线相碰，则记录碰撞事件的个数增加一。如此可以统计出来N个针的构型中所有出现针线相碰的事件数N_{coll}。对应的针线相碰的经验性概率即为$P(l, d) = N_{\mathrm{coll}}/N$。

对于蒲丰-拉普拉斯投针问题，我们利用Eq.(3.24)可以不断随机生成N个针的构型$(x_{\mathrm{U}}, y_{\mathrm{U}}, \phi)$。这对应着$N$次的投针实验。对于其中的每一个构型，利用Eq.(3.25)得到其对应的$(x_{\mathrm{L}}, y_{\mathrm{L}})$。此时可以利用针线相碰的判据Eq.(3.26)来判断这个构型和两组平行线是否相碰。如果针线相碰，则记录碰撞事件的个数增加一。如此可以统计出来N个针的构型中所有出现针线相碰的事件数N_{coll}。对应的针线相碰的经验性概率即为$P(l, a, b) = N_{\mathrm{coll}}/N$。

3. 解析理论：蒲丰投针问题

这里我们推导出来理论框架来计算相碰概率。这里的解析理论基于上述的几何图像。计算概率的基本逻辑是：(y_{U}, ϕ)的全部取值范围内有一部分对应的针的构型导致了针线相碰，这部分参数的体积相对于全部参数取值范围的体积就是相碰概率。针线相碰的相碰概率$P(l, d)$可以写成:

$$P(l, d) = \frac{S_{\mathrm{coll}}}{S_{\mathrm{all}}}, \tag{3.27}$$

其中S_{all}为参数(y_{U}, ϕ)的取值范围的体积，S_{coll}为参数(y_{U}, ϕ)中导致针线相碰的参数范围的

体积。为简单期间，这里把y_{U}写为y。容易可知:

$$S_{\mathrm{coll}} = \int_0^{\pi} \mathrm{d}\phi \int_0^{d} \Theta(l\sin\phi - y)\mathrm{d}y, \tag{3.28}$$

$$S_{\mathrm{all}} = \pi d. \tag{3.29}$$

其中$\Theta(x)$为阶跃函数(Step Function)，定义为

$$\Theta(x) = \begin{cases} 1, & x \geqslant 0; \\ 0, & x < 0. \end{cases} \tag{3.30}$$

Eq.(3.28)中，$\Theta(l\sin\phi - y)$为1时，对应于$y_{\mathrm{L}} = y_{\mathrm{U}} - l\sin\phi \leqslant 0$，即此时的参数$y$能够导致针线相碰。考虑到$y$在积分时的取值范围$[0, d)$，可知$y$的实际取值范围为$y \in [0, l\sin\phi) \cap [0, d)$。此时可以利用图示Figure 3.7来表示这个定积分范围。按照l和d的相对大小，我们分成两种情况考虑。

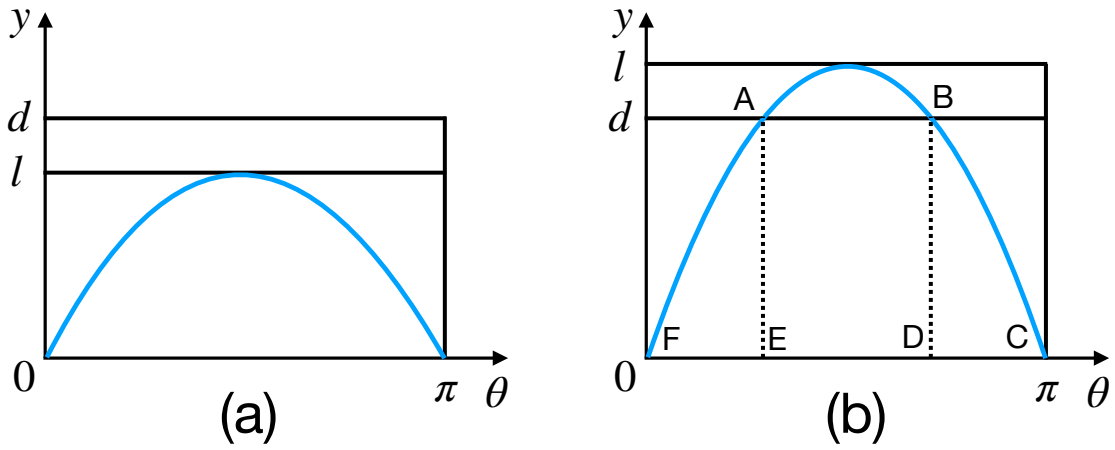


FIG. 3.7. 蒲丰投针问题中定积分S_{coll}的计算。图中曲线对应$y = l\sin\phi, \phi \in [0, \pi)$。(a)对应$l < d$的情形。此时曲线$y = l\sin\phi$和$y = 0$之间的形状的面积即为$S_{\mathrm{coll}}$。(b)对应$l < d$的情形。此时同时处在曲线$y = l\sin\phi$和$y = d$之下、和$y = 0$之间的形状的面积即为$S_{\mathrm{coll}}$，即为形状AB$\pi$0的面积。

(1) 当$l < d$时，如Figure 3.7(a)所示，$y \in [0, l\sin\phi] \cap [0, d) = [0, l\sin\phi]$，此时$\int_0^d \Theta(l\sin\phi - y)\mathrm{d}y = l\sin\phi$。可知:

$$\begin{aligned} S_{\mathrm{coll}} &= \int_0^{\pi} \mathrm{d}\phi \int_0^{d} \Theta(l\sin\phi - y)\mathrm{d}y \\ &= \int_0^{\pi} \mathrm{d}\phi \cdot l\sin\phi \\ &= 2l. \end{aligned} \tag{3.31}$$

对应着：

$$P(l,d)=\frac{S_{\text{coll}}}{S_{\text{all}}}=\frac{2}{\pi}\frac{l}{d}. \tag{3.32}$$

可知当$l=d$时，$P(l,d)=2/\pi$。

（2）当$l>d$时，如Figure 3.7(b)可知，给定l和d后，$\min\{l\sin\phi,d\}$依赖于具体的ϕ。我们可以通过添加辅助线来计算面积。Figure 3.7(b)中，A点和E点的横坐标对应着$l\sin\phi^*=d$，即

$$\phi^*=\arcsin\frac{d}{l}. \tag{3.33}$$

相应地，B点和D点的横作坐标对应$\pi-\phi^*$。当$\phi\in[0,\phi^*)\cup[\pi-\phi^*,\pi)$时，$\min\{l\sin\phi,d\}=l\sin\phi$；当$\phi\in[\phi^*,\pi-\phi^*)$时，$\min\{l\sin\phi,d\}=d$。可知$S_{\text{coll}}$由多边形AEF、ABDE、BCD组成。可知：

$$\begin{aligned}
S_{\text{coll}}=S_{\text{ABCF}}&=S_{\text{AEF}}+S_{\text{ABDE}}+S_{\text{BCD}}\\
&=\int_0^{\phi^*}\min\{l\sin\phi,d\}\mathrm{d}\phi+\int_{\phi^*}^{\pi-\phi^*}\min\{l\sin\phi,d\}\mathrm{d}\phi+\int_{\pi-\phi^*}^{\pi}\min\{l\sin\phi,d\}\mathrm{d}\phi\\
&=\int_0^{\phi^*}l\sin\phi\mathrm{d}\phi+\int_{\phi^*}^{\pi-\phi^*}d\mathrm{d}\phi+\int_{\pi-\phi^*}^{\pi}l\sin\phi\mathrm{d}\phi\\
&=2\int_0^{\phi^*}l\sin\phi\mathrm{d}\phi+\int_{\phi^*}^{\pi-\phi^*}d\mathrm{d}\phi\\
&=2l(1-\cos\phi^*)+d(\pi-2\phi^*)\\
&=2l\left[1-\sqrt{1-\frac{d^2}{l^2}}\right]+d(\pi-2\arcsin\frac{d}{l}).
\end{aligned} \tag{3.34}$$

可知

$$P(l,d)=\frac{S_{\text{coll}}}{S_{\text{all}}}=\frac{2}{\pi}\frac{l}{d}\left[1-\sqrt{1-\frac{d^2}{l^2}}\right]+\left(1-\frac{2}{\pi}\arcsin\frac{d}{l}\right). \tag{3.35}$$

综合Eqs.(3.31)和(3.35)，可知

$$P(l,d)=\begin{cases}\frac{2}{\pi}\frac{l}{d}, & \text{if } l\leqslant d;\\ \frac{2}{\pi}\frac{l}{d}\left[1-\sqrt{1-\frac{d^2}{l^2}}\right]+\left(1-\frac{2}{\pi}\arcsin\frac{d}{l}\right), & \text{if } l>d.\end{cases} \tag{3.36}$$

可知，概率$P(l,d)$并不依赖于l,d的具体数值，而依赖于它们之间的相对大小。

4. 解析理论：蒲丰-拉普拉斯投针问题

此问题的解析理论的更多计算细节可查阅[12]。

这里解析计算的基本思路和蒲丰投针问题类似。我们需要计算两个体积：S_{all}为针的构型参数$(x_{\text{U}}, y_{\text{U}}, \phi)$的所有取值范围对应的体积、$S_{\text{coll}}$为参数$(x_{\text{U}}, y_{\text{U}}, \phi)$的取值范围中导致针线相碰时的参数对应的体积。针线相碰概率$P(l, a, b)$可表示为：

$$P(l, a, b) = \frac{S_{\text{coll}}}{S_{\text{all}}}. \tag{3.37}$$

易知：

$$S_{\text{all}} = ab\pi. \tag{3.38}$$

这里的核心在于计算S_{coll}。S_{coll}可以写为：

$$S_{\text{coll}} = S_{\text{coll}}(x) + S_{\text{coll}}(y) - S_{\text{coll}}(x, y). \tag{3.39}$$

上述$S_{\text{coll}}(x)$, $S_{\text{coll}}(y)$, $S_{\text{coll}}(x, y)$分别表示针和网格边界$x = 0$或$x = a$相碰、针和网格边界$y = 0$或$y = b$相碰、针和网格两个方向上的边界同时相碰时的参数$(x_{\text{U}}, y_{\text{U}}, \phi)$的取值范围的体积。为简单起见，这里我们把$(x_{\text{U}}, y_{\text{U}})$简写为$(x, y)$。我们有：

$$\begin{aligned}
S_{\text{coll}}(x) &= \int_0^{\frac{\pi}{2}} \mathrm{d}\phi \int_0^a \Theta(l\cos\phi - x)\mathrm{d}x \int_0^b \mathrm{d}y + \int_{\frac{\pi}{2}}^{\pi} \mathrm{d}\phi \int_0^a \Theta(-l\cos\phi + x - a)\mathrm{d}x \int_0^b \mathrm{d}y \\
&= b\int_0^{\pi} \mathrm{d}\phi \int_0^a \Theta(|l\cos\phi| - x)\mathrm{d}x, \qquad (3.40) \\
S_{\text{coll}}(y) &= \int_0^{\pi} \mathrm{d}\phi \int_0^a \mathrm{d}x \int_0^b \Theta(l\sin\phi - y)\mathrm{d}y \\
&= a\int_0^{\pi} \mathrm{d}\phi \int_0^b \Theta(l\sin\phi - y)\mathrm{d}y, \qquad (3.41) \\
S_{\text{coll}}(x, y) &= \int_0^{\frac{\pi}{2}} \mathrm{d}\phi \int_0^a \Theta(l\cos\phi - x)\mathrm{d}x \int_0^b \Theta(l\sin\phi - y)\mathrm{d}y \\
&+ \int_{\frac{\pi}{2}}^{\pi} \mathrm{d}\phi \int_0^a \Theta(-l\cos\phi + x - a)\mathrm{d}x \int_0^b \Theta(l\sin\phi - y)\mathrm{d}y \\
&= \int_0^{\pi} \mathrm{d}\phi \int_0^a \Theta(|l\cos\phi| - x)\mathrm{d}x \int_0^b \Theta(l\sin\phi - y)\mathrm{d}y. \qquad (3.42)
\end{aligned}$$

在Eqs.(3.40)和(3.42)的右侧，我们使用了一次变量代换：$x - a \to -\hat{x}$。此时有：

$$\begin{aligned}
\int_0^a \Theta(-l\cos\phi + x - a)\mathrm{d}x &= \int_a^0 \Theta(-l\cos\phi - \hat{x})\mathrm{d}(-\hat{x}) \\
&= \int_0^a \Theta(-l\cos\phi - \hat{x})\mathrm{d}\hat{x}. \qquad (3.43)
\end{aligned}$$

为了计算Eqs.(3.40)-(3.42)，这里定义三个积分表达式:

$$A(l,a) \equiv \int_0^{\pi} \mathrm{d}\phi \int_0^{a} \Theta(|l\cos\phi| - x)\mathrm{d}x, \tag{3.44}$$

$$B(l,b) \equiv \int_0^{\pi} \mathrm{d}\phi \int_0^{b} \Theta(l\sin\phi - y)\mathrm{d}y, \tag{3.45}$$

$$AB(l,a,b) \equiv \int_0^{\pi} \mathrm{d}\phi \int_0^{a} \Theta(|l\cos\phi| - x)\mathrm{d}x \int_0^{b} \Theta(l\sin\phi - y)\mathrm{d}y. \tag{3.46}$$

S_{coll}相应地可写为:

$$S_{\mathrm{coll}} = bA(l,a) + aB(l,b) - AB(l,a,b). \tag{3.47}$$

为了计算Eqs.(3.44)-(3.46)、以及最终的$P(l,a,b)$，我们需要考虑参数l, a, 和b的相对大小。

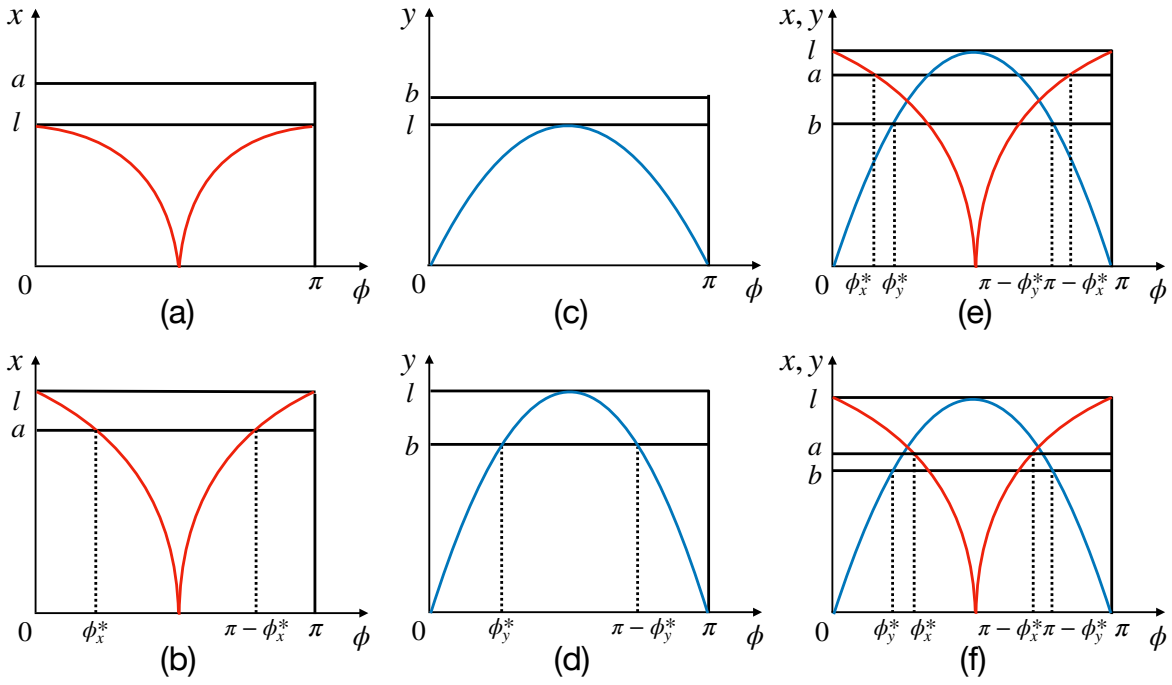


FIG. 3.8. 蒲丰-拉普拉斯投针问题中定积分S_{coll}的计算。 (a)-(b)分别展示了当$l \leqslant a$和$a < l$时的曲线$x = |l\cos\phi|$。 $\phi_x^* \in (0, \pi/2)$和$\pi - \phi_x^*$为曲线$x = |l\cos\phi|$和$x = a$相交时的ϕ坐标。 (c)-(d)分别展示了$l \leqslant b$和$b < l$时的曲线$y = l\sin\phi$。 $\phi_y^* \in (0, \pi/2)$和$\pi - \phi_y^*$为$y = l\sin\phi$和$y = b$相交时的ϕ坐标。 (e)-(f) 分别展示了$\phi_x^* \leqslant \phi_y^*$（对应于$l \leqslant L$）和$\phi_y^* < \phi_x^*$（对应于$L < l$）时的曲线$x = |l\cos\phi|$和$y = l\sin\phi$。此时定义$L \equiv \sqrt{a^2 + b^2}$。（此图为论文[12]中的Figure 4。）

注意到，Eqs.(3.44)-(3.46)中的积分有个简单形式：

$$\int_0^t \Theta(m-z)\mathrm{d}z = \min\{t, m\}, \tag{3.48}$$

其中，$(t, m, z) = (a, |l\cos\phi|, x)$和$(b, l\sin\phi, y)$.

我们把满足$l \leqslant a$和$a < l$的$A(l,a)$分别写为$A(l,a|l \leqslant a)$和$A(l,a|a < l)$。当$l \leqslant a$, $|l\cos\phi| \leqslant a$在$\phi \in [0,\pi)$时一直满足；但当$a < l$，$|l\cos\phi| \leqslant a$只有在$\phi \in [\phi_x^*, \pi - \phi_x^*]$时才满足。可参考Figure 3.8 (a) - (b)。此时我们定义：

$$\phi_x^* = \arccos\frac{a}{l}. \tag{3.49}$$

相应有：

$$\begin{aligned} A(l,a|l \leqslant a) &= \int_0^{\frac{\pi}{2}} \mathrm{d}\phi \cdot l\cos\phi + \int_{\frac{\pi}{2}}^{\pi} \mathrm{d}\phi \cdot (-l\cos\phi) \\ &= 2l, \end{aligned} \tag{3.50}$$

$$\begin{aligned} A(l,a|a < l) &= \int_0^{\phi_x^*} \mathrm{d}\phi \cdot a + \int_{\phi_x^*}^{\frac{\pi}{2}} \mathrm{d}\phi \cdot l\cos\phi + \int_{\frac{\pi}{2}}^{\pi-\phi_x^*} \mathrm{d}\phi \cdot (-l\cos\phi) + \int_{\pi-\phi_x^*}^{\pi} \mathrm{d}\phi \cdot a \\ &= 2l(1-\sin\phi_x^*) + 2a\phi_x^*. \end{aligned} \tag{3.51}$$

我们把满足$l \leqslant b$和$b < l$的$B(l,b)$分别写成$B(l,b|l \leqslant b)$和$B(l,b|b < l)$。当$l \leqslant b$时，$l\sin\phi \leqslant b$在$\phi \in [0,\pi)$时一直成立；但当$b < l$, $l\sin\phi \leqslant b$只有当$\phi \in [0,\phi_y^*] \cup [\pi - \phi_y^*, \pi)$时才满足。可参考Figure 3.8 (c) - (d)。此时我们定义：

$$\phi_y^* = \arcsin\frac{b}{l}. \tag{3.52}$$

相应有：

$$\begin{aligned} B(l,b|l \leqslant b) &= \int_0^{\pi} \mathrm{d}\phi \cdot l\sin\phi \\ &= 2l, \end{aligned} \tag{3.53}$$

$$\begin{aligned} B(l,b|b < l) &= \int_0^{\phi_y^*} \mathrm{d}\phi \cdot l\sin\phi + \int_{\phi_y^*}^{\pi-\phi_y^*} \mathrm{d}\phi \cdot b + \int_{\pi-\phi_y^*}^{\pi} \mathrm{d}\phi \cdot l\sin\phi \\ &= 2l\left(1-\cos\phi_y^*\right) + 2b\left(\frac{\pi}{2} - \phi_y^*\right). \end{aligned} \tag{3.54}$$

为了计算$AB(l,a,b)$，我们得考虑参数(l,a,b)三者之间的相对大小。此时分以下四种情况讨论。

- $l \leqslant b$**时的情形：**

我们把$l \leqslant b$时的$AB(l,a,b)$写为$AB(l,a,b|l \leqslant b)$。 此时Eq.(3.46)为

$$\begin{aligned} AB(l,a,b|l \leqslant b) &= \int_0^{\frac{\pi}{2}} \mathrm{d}\phi \cdot l\cos\phi \cdot l\sin\phi + \int_{\frac{\pi}{2}}^{\pi} \mathrm{d}\phi \cdot (-l\cos\phi) \cdot l\sin\phi \\ &= l^2. \end{aligned} \tag{3.55}$$

使用Eq.(3.47)，可知：

$$\begin{aligned} S_{\text{coll}} &= bA(l,a|l \leqslant a) + aB(l,b|l \leqslant b) - AB(l,a,b|l \leqslant b) \\ &= 2lb + 2la - l^2. \end{aligned} \tag{3.56}$$

最终：

$$P(l,a,b) = \frac{2}{\pi}\left(\frac{l}{a} + \frac{l}{b}\right) - \frac{1}{\pi}\frac{l^2}{ab}. \tag{3.57}$$

- $b < l \leqslant a$**时的情形：**

我们把$b < l \leqslant a$时的$AB(l,a,b)$写成$AB(l,a,b|b < l \leqslant a)$。此时有：

$$\begin{aligned} AB(l,a,b|b < l \leqslant a) &= \int_0^{\phi_y^*} \mathrm{d}\phi \cdot l\cos\phi \cdot l\sin\phi + \int_{\phi_y^*}^{\frac{\pi}{2}} \mathrm{d}\phi \cdot l\cos\phi \cdot b \\ &\quad + \int_{\frac{\pi}{2}}^{\pi-\phi_y^*} \mathrm{d}\phi \cdot (-l\cos\phi) \cdot b + \int_{\pi-\phi_y^*}^{\pi} \mathrm{d}\phi \cdot (-l\cos\phi) \cdot l\sin\phi \\ &= 2lb - b^2. \end{aligned} \tag{3.58}$$

则：

$$\begin{aligned} S_{\text{coll}} &= bA(l,a|l \leqslant a) + aB(l,b|b < l) - AB(l,a,b|b < l \leqslant a) \\ &= b^2 + 2la(1 - \cos\phi_y^*) + 2ab\left(\frac{\pi}{2} - \phi_y^*\right) \\ &= b^2 + 2la\left(1 - \sqrt{1 - \frac{b^2}{l^2}}\right) + 2ab\left(\frac{\pi}{2} - \arcsin\frac{b}{l}\right). \end{aligned} \tag{3.59}$$

则有：

$$P(l,a,b) = \frac{1}{\pi}\frac{b}{a} + \frac{2}{\pi}\frac{l}{b}\left(1 - \sqrt{1 - \frac{b^2}{l^2}}\right) + \frac{2}{\pi}\left(\frac{\pi}{2} - \arcsin\frac{b}{l}\right). \tag{3.60}$$

- $a < l$**时的情形：**

此时为了计算$AB(l,a,b)$，我们得考虑ϕ_x^*和ϕ_y^*之间的相对大小。 当$a < l \leqslant L$，我们有$1 - a^2/l^2 \leqslant b^2/l^2$。此时$\sin^2\phi_x^* \leqslant \sin^2\phi_y^*$，即$\phi_x^* \leqslant \phi_y^*$。 请参考Figure 3.8 (e)。 我们

把$a < l \leqslant L$时的$AB(l,a,b)$写为$AB(l,a,b|a < l \leqslant L)$。此时有：

$$\begin{aligned}
&AB(l,a,b|a<l\leqslant L)\\
&=\int_0^{\phi_x^*}\mathrm{d}\phi\cdot a\cdot l\sin\phi+\int_{\phi_x^*}^{\phi_y^*}\mathrm{d}\phi\cdot l\cos\phi\cdot l\sin\phi+\int_{\phi_y^*}^{\frac{\pi}{2}}\mathrm{d}\phi\cdot l\cos\phi\cdot b\\
&+\int_{\frac{\pi}{2}}^{\pi-\phi_y^*}\mathrm{d}\phi\cdot(-l\cos\phi)\cdot b+\int_{\pi-\phi_y^*}^{\pi-\phi_x^*}\mathrm{d}\phi\cdot(-l\cos\phi)\cdot l\sin\phi+\int_{\pi-\phi_x^*}^{\pi}\mathrm{d}\phi\cdot a\cdot l\sin\phi\\
&=2la+2lb-L^2-l^2.
\end{aligned}\tag{3.61}$$

则：

$$\begin{aligned}
S_{\mathrm{coll}}&=bA(l,a|a<l)+aB(l,b|b<l)-AB(l,a,b|a<l\leqslant L)\\
&=L^2+l^2-2lb\sin\phi_x^*-2la\cos\phi_y^*+2ab\left(\phi_x^*+\frac{\pi}{2}-\phi_y^*\right)\\
&=L^2+l^2-2\left(lb\sqrt{1-\frac{a^2}{l^2}}+la\sqrt{1-\frac{b^2}{l^2}}\right)+2ab\left(\pi-\arcsin\frac{a}{l}-\arcsin\frac{b}{l}\right).
\end{aligned}\tag{3.62}$$

相应地：

$$P(l,a,b)=\frac{1}{\pi}\left(\frac{L^2}{ab}+\frac{l^2}{ab}\right)-\frac{2}{\pi}\left(\frac{l}{a}\sqrt{1-\frac{a^2}{l^2}}+\frac{l}{b}\sqrt{1-\frac{b^2}{l^2}}\right)+\frac{2}{\pi}\left(\pi-\arcsin\frac{a}{l}-\arcsin\frac{b}{l}\right).\tag{3.63}$$

当$L<l$，我们相应有$\phi_y^*<\phi_x^*$。我们把$L<l$时的$AB(l,a,b)$写成$AB(l,a,b|L<l)$。我们有：

$$\begin{aligned}
AB(l,a,b|L<l)&=\int_0^{\phi_y^*}\mathrm{d}\phi\cdot a\cdot l\sin\phi+\int_{\phi_y^*}^{\phi_x^*}\mathrm{d}\phi\cdot a\cdot b+\int_{\phi_x^*}^{\frac{\pi}{2}}\mathrm{d}\phi\cdot l\cos\phi\cdot b\\
&+\int_{\frac{\pi}{2}}^{\pi-\phi_x^*}\mathrm{d}\phi\cdot(-l\cos\phi)\cdot b+\int_{\pi-\phi_x^*}^{\pi-\phi_y^*}\mathrm{d}\phi\cdot a\cdot b+\int_{\pi-\phi_y^*}^{\pi}\mathrm{d}\phi\cdot a\cdot l\sin\phi\\
&=2la(1-\cos\phi_y^*)+2lb(1-\sin\phi_x^*)+2ab(\phi_x^*-\phi_y^*).
\end{aligned}\tag{3.64}$$

则有：

$$\begin{aligned}
S_{\mathrm{coll}}&=bA(l,a|a<l)+aB(l,b|b<l)-AB(l,a,b|L<l)\\
&=ab\pi.
\end{aligned}\tag{3.65}$$

相应我们有：

$$P(l,a,b)=1.\tag{3.66}$$

这里有个简单的几何解释。长度为a、高度为b的网格内两点之间最长距离为它的对角线L。这样任意长度$l > L$的针，在扔在网格上后，必定和网格的边界相碰，即$P(l,a,b)=1$.

综合Eqs.(3.57)、(3.60)、(3.63)、和(3.66)，我们得到相碰概率的最终表达式：

$$P(l,a,b)=\begin{cases}\frac{2}{\pi}\left(\frac{l}{a}+\frac{l}{b}\right)-\frac{1}{\pi}\frac{l^2}{ab}, & \text{if } l\leqslant b;\\ \frac{1}{\pi}\frac{b}{a}+\frac{2}{\pi}\frac{l}{b}\left(1-\sqrt{1-\frac{b^2}{l^2}}\right)+\frac{2}{\pi}\left(\frac{\pi}{2}-\arcsin\frac{b}{l}\right), & \text{if } b<l\leqslant a;\\ \frac{1}{\pi}\left(\frac{L^2}{ab}+\frac{l^2}{ab}\right)-\frac{2}{\pi}\left(\frac{l}{a}\sqrt{1-\frac{a^2}{l^2}}+\frac{l}{b}\sqrt{1-\frac{b^2}{l^2}}\right) & \\ +\frac{2}{\pi}\left(\pi-\arcsin\frac{a}{l}-\arcsin\frac{b}{l}\right), & \text{if } a<l\leqslant L;\\ 1, & \text{if } L<l.\end{cases} \tag{3.67}$$

当$a\to\infty$时，$l/a\to 0$。Eq.(3.67)对应于蒲丰投针问题的理论结果：

$$P(l,\infty,b)=\begin{cases}\frac{2}{\pi}\frac{l}{b}, & \text{if } l\leqslant b;\\ \frac{2}{\pi}\frac{l}{b}\left(1-\sqrt{1-\frac{b^2}{l^2}}\right)+\frac{2}{\pi}\left(\frac{\pi}{2}-\arcsin\frac{b}{l}\right), & \text{if } b<l.\end{cases} \tag{3.68}$$

上述结果即为Eq.(3.36)。

5. 模拟和解析结果

Figure 3.9展示了两个投针问题的模拟结果和解析结果。可以看出两者符合的很好。有两点需要注意。第一点是，给定a,b后，相碰概率$P(l,a,b)$随着针长l的增加而单调增加，一直到达1。第二点是，当固定b、增加a时，会降低相碰概率$P(l,a,b)$。这里有个简单的直观解释：当网格单元的短边保持不点、长边增加时，针在长边方向和网格相碰的概率降低，这最终会在整体上降低针线的相碰概率。

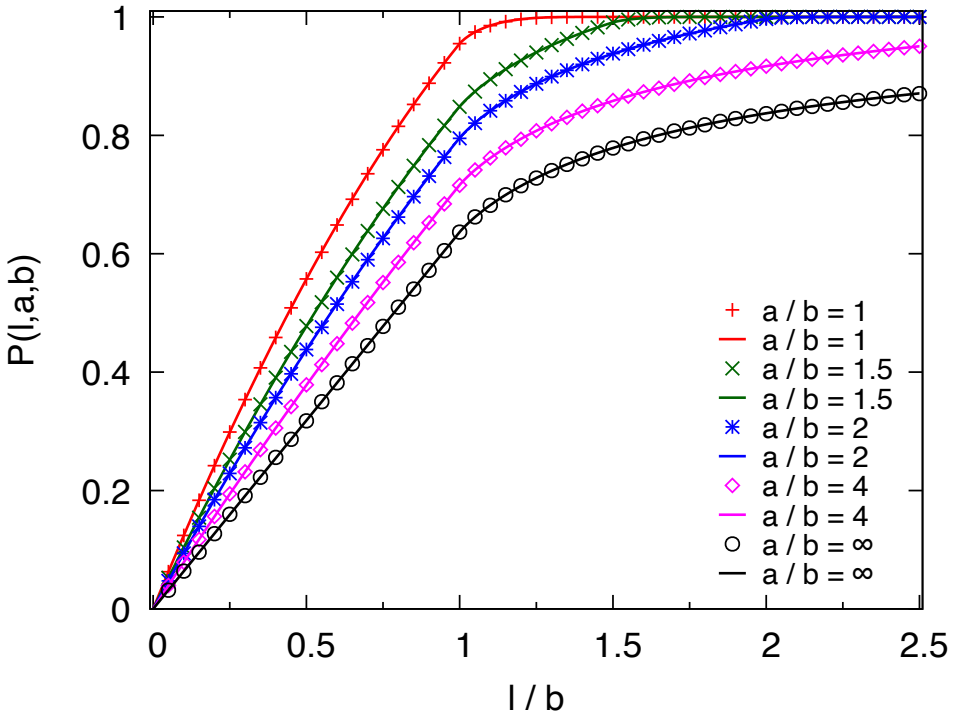


FIG. 3.9. 投针问题中针线相碰概率的计算结果。图中从上到下依次展示了蒲丰-拉普拉斯投针问题中a/b增加时的针线相碰概率$P(l,a,b)$。$a/b=\infty$的情形对应着蒲丰投针问题。 图中的符号对应模拟结果、实线对应解析结果。 模拟结果中，平行线间距$b=3.0$，且每个数据点对应于$N=10^6$次投针过程后的统计结果。 此图为论文[12]中的Figure 2(a)。

3.4. Vicsek模型

这里讨论自驱动粒子的集体运动行为的Vicsek模型。

寻找并解释复杂系统在空间–时间上的宏观尺度的集体行为规律是统计物理、系统科学等学科的研究范畴。 这些集体运动现象跨越了从细胞、化学反应、人群、到地球气候等极广的尺度和对象。 这些空间–时间上规则行为的研究可以简单称为模式形成(pattern formation) [13]。

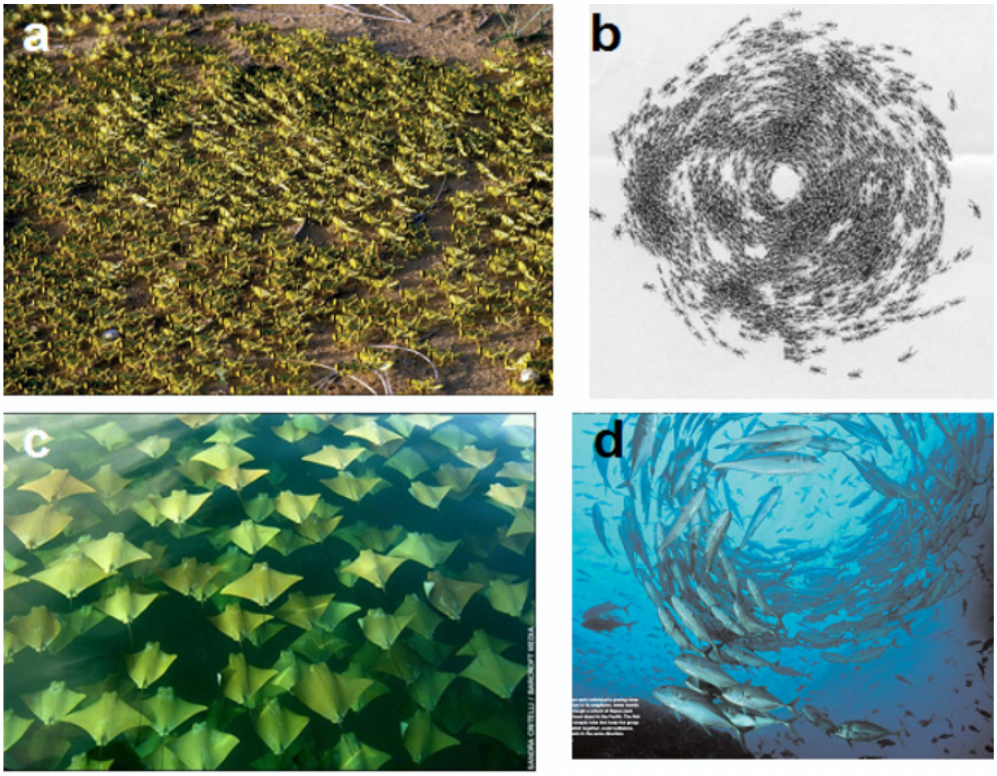


FIG. 3.10. 生物界一些典型的集体运动现象。 (a) 东非平原上的动物迁徙。 (b) 蚂蚁。 (c) 魔鬼鱼。 (d) 鱼群。

这里我们关注的是活性物质(active matter)中自驱动粒子的集体运动(collective motion of self-propelled particles)[14]。细菌、动物、人都可以认为是自驱动粒子。自驱动粒子能够通过消耗自身能量来维持和改变运动，因此粒子系统本身是一个非平衡态系统。自驱动粒子系统的集体运动建模的逻辑是宏观上的有序运动能够通过简单的局域规则、而非宏观设计来实现。这是一个典型的统计物理的研究思路。经典的集体运动模型是Vicsek model[15]。

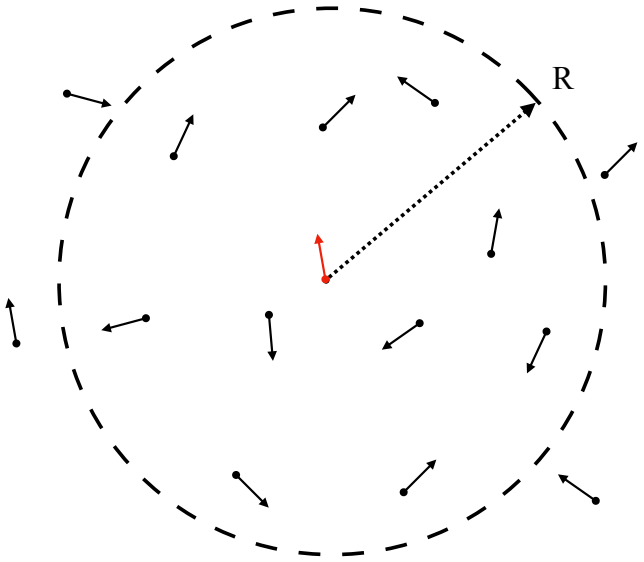


FIG. 3.11. Viseck model的基本过程：粒子i根据距离它为R之内的所有近邻粒子的平均速度来更新自己的速度。箭头的圆形端点为粒子的位置，箭头的长度和方向代表粒子的速度矢量。此模型中粒子是没有体积的。

假设在边长为L的二维平面上有N个粒子。粒子密度为$\rho = N/L^2$。 粒子的尺寸可以忽略。粒子系统的运动在平面区域里满足周期性边界条件。 粒子i的速度矢量为$\vec{v}_i(t), i = 1, 2, \cdots, N$。 我们假设所有的粒子速度的绝对值相同并且保持不变($|\vec{v}_i(t)| = v_0$)， 但方向$\theta_i(t) \in [0, 2\pi]$因为粒子间相互作用而改变。 在Vicsek model中，粒子基于距离它为R之内的粒子来更新自己的速度。 粒子i的局域运动规则为

$$\theta_i(t+1) = \langle \theta_i(t) \rangle_R + \xi_i, \tag{3.69}$$

$$\langle \theta_i(t) \rangle_R = \arctan \frac{\langle \sin \theta_i(t) \rangle_R}{\langle \cos \theta_i(t) \rangle_R}, \tag{3.70}$$

$$\xi_i \in [-\eta/2, \eta/2]. \tag{3.71}$$

上述式子里，$\langle \cdot \rangle_R$表示对距离粒子i为R内的所有近邻粒子的物理量做平均。 ξ为噪声变量，η为噪声强度。上述局域规则的大体思路是：粒子选择了R范围内近邻粒子的平均运动方向，之后计入了一个均匀分布的扰动。

在定义了粒子的基本运动规则后，经过多次的速度和位置更新，粒子系统达到了近似非平衡稳态。我们通过对粒子采样来研究集体运动的有序性。自驱动粒子系统的序参

量(order parameter)定义为

$$\varphi = \frac{1}{Nv_0}\Big|\sum_{i=1}^{N}\vec{v}_i(t)\Big|. \tag{3.72}$$

可知当粒子的运动完全无序时，即$\vec{v}_i(t)$的方向完全随机时，可知$\varphi = 0$。而当粒子的运动完全相同时，即$\vec{v}_i(t) = \vec{v}$时，可知$\varphi = 1$。容易得知$\varphi \in [0, 1]$。

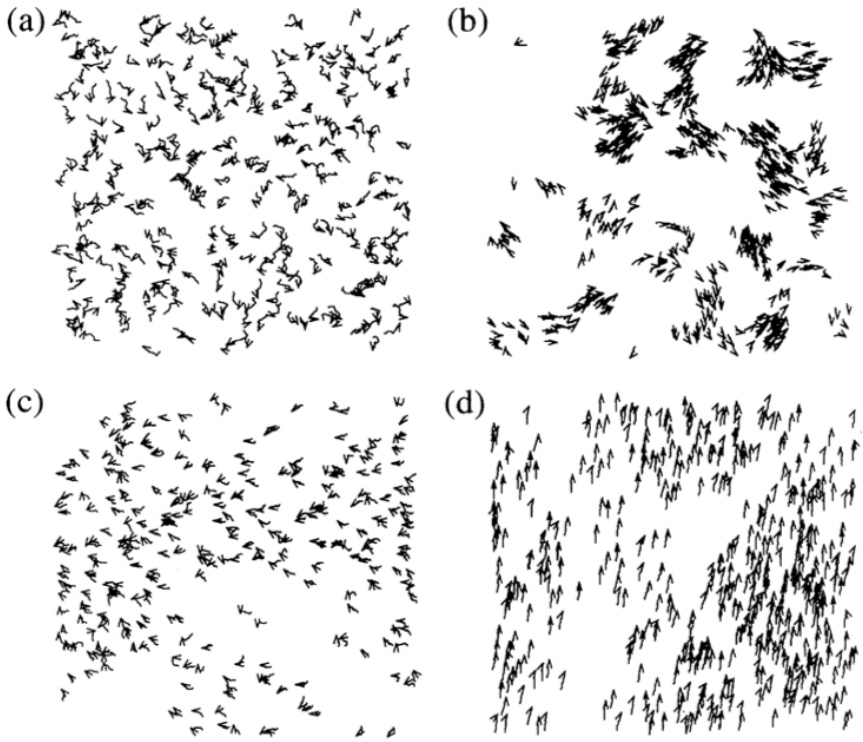


FIG. 3.12. Viseck model下不同参数对应的粒子运动构型。箭头表示粒子的运动方向。

在这里我们将使用模拟方法来观察不同粒子密度ρ下粒子系统的运动，并判断有序运动产生的条件。

编程任务：

（1）我们是在有限的区域里考虑粒子在无限空间里的运动。这里我们可以使用周期性边界条件。此时如何计算粒子之间的距离。

（2）为简单起见，我们假定无噪声的情况，即$\eta = 0$。给定R、调节ρ，请实现粒子系统的Vicsek model，并画出和φ随ρ变化的数据点。 此结果可参考论文[15]的Fig.2(b)。

习题

3.1 自洽方程的不动点求解和分析

请实现代码以实现自洽方程的迭代序列方法，并重复实现Figures 3.4和3.5里的计算结果。

这里我们使用上述数值计算方式来求解只有一个参数的的自洽方程

$$x = f(x;c) \equiv 1 - \mathrm{e}^{-cx}. \tag{3.73}$$

此时实数变量$x \in [0,1]$，实数参数$c \geqslant 0$。

编程要求：

（1）选择上述迭代序列的元素个数为$n = 10^6$、序列收敛判据中的小数为$\varepsilon = 10^{-10}$。注意这两个量可以选择其它值。首先考虑$c = 0.5$的情况。假设序列初始值可以取不同的值，如$x^{(0)} = \{0, 0.2, 0.4, 0.6, 0.8, 1\}$。请判断不同初始值对应的迭代序列在给定$n$的范围是否收敛，并输出相应的收敛值（即方程的不动点x^*）。继续计算$c > 1$（比如$c = 1.5, 2.0$等）对应的结果，并比较两者。

（2）此时我们只考虑迭代序列方法里得到的稳定不动点。考虑$c \in [0, 10]$，且此区间里c变化的步长为$\Delta c = 0.1$。请计算每个c所对应的稳定不动点x^*、序列收敛的迭代次数（题干中$x^{(t)}$里的t），并输出到屏幕或者文件中。

3.2 自洽方程的不动点求解和分析

考虑自洽方程

$$x = f(x;c) \equiv \exp(-c\exp(-cx)). \tag{3.74}$$

其中实数变量$x \geqslant 0$，实数参数$c \geqslant 0$。

请参考Figures 3.4和3.5的形式，计算并作图以下结果：

（1）给定c后函数$f(x;c)$随x的变化；

（2）给定不同初始值$x^{(0)}$后迭代序列的演化；

（3）自洽方程在不同c时的所有稳定不动点。

3.3 投针问题

请阅读讲义第3.3节“投针问题”，以重复蒲丰-拉普拉斯投针问题和蒲丰投针问题里相碰概率的模拟和解析结果，并最终实现讲义中的Figure 3.9。

注意“从问题到作业文档”的流程是：用代码实现问题的模拟过程和解析结果、代码输出数据文件、利用数据文件来生成格式为pdf的结果图、把结果图和公式等内容一起整合到作业文档以最终生成pdf文档。

要求如下：

（1）请复习一下如何实现在程序运行中输出数据文件。最终你需要利用这些程序计算得来的数据文件来生成pdf格式的结果图。

（2）对于投针问题的模拟实现，请仔细阅读“投针问题的几何描述”和“投针问题的模拟实现”小节里的文字。

（3）对于投针问题的解析结果，请直接使用Eqs.(3.67)和(3.68)计算即可。

（4）在使用程序来计算并输出数据文件时，注意参考一下你最终做图的横轴和纵轴。连续计算并输出数据文件时，可以使用循环语句，而且可以把横轴l/b作为循环过程的变量。这样生成的数据文件用来做图时会非常直接。

（5）注意结果图的横轴为l/b。在模拟结果中，b需要赋予某个值。在蒲丰投针问题里，给定l/b后，可以得到参数(l, b)，这样可以实现模拟过程。对于蒲丰-拉普拉斯投针问题，给定a/b和l/b后，可以得到确定的参数(l, a, b)，后续可以实现模拟过程。在做模拟计算时，横轴的步长可以选择为$\Delta(l/b) = 0.05$，绘图风格为符号（points）。

（6）对于蒲丰投针问题的解析结果，给定l/b后，可以直接利用Eq.(3.68)得到相碰概率的结果。对于蒲丰-拉普拉斯投针问题的解析结果，给定l/b和a/b后，可以得到相应的l/a，此时利用Eq.((3.67))可以得到相碰概率的结果。解析结果里横轴的步长可以选择为$\Delta(l/b) = 0.001$，绘图风格为线条（lines）。

（7）对于结果图的风格，你可以使用gnuplot或其它你熟悉的工具来对数据文件进行做图。你得到的结果图不必延续参考结果图中的符号和线条的风格，能区分计算结果即可。

4.

参考内容：邓俊辉教材第1.2、1.3节。李春葆教材第1.3节。

本章介绍了有关算法复杂度的内容。包括空间和时间复杂度的测量、时间复杂度的渐进分析与符号 递归算法的时间复杂度的推导。还介绍计算复杂性的一些基本概念及其常见类型。

4.1. 算法复杂度的测量

■ 算法的计算效率

(1) 评判：可以确定一套尺度，用以从时间和空间等方面来度量算法的计算成本，进而对不同算法进行比较和评判;

(2) 改进：研究和归纳算法设计和实现过程中的一般规律与技巧，以编写出效率更高、能够处理更大规模数据的程序。

▷ 时间复杂度 (time complexity)：算法操作所花费的时间。这正比于算法里基本操作的数目。

▷ 空间复杂度 (space complexity)：算法操作所消耗的空间（内存）。它是时间复杂度的一个天然上界。

■ 时间复杂度

问题：算法的计算时间依赖于哪些因素？如何度量算法所需要的计算时间?

计算成本虽然基于算法本身的执行过程和参数，但最核心的因素还是在于问题实例的规模。特定算法处理规模为n的问题所需的时间记为$T(n)$。

例子1: 对于n的整数的数组的排序，所需计算时间对序列本身的依赖关系。比如考虑在有序数组、倒序数组、随机数组上使用排序算法时的情形。

(1) 最坏情况(worst case)：考虑最长计算时间的实例;

(2) 最好情况(best case)：考虑最短计算时间的实例;

(3) 平均情况(average case)：考虑随机生成的实例;

大家可以想下什么情况下需要考虑上述三种情况?

4.2. 时间复杂度的渐近分析：概念和符号

■ 渐近分析的原因

假设$f(n)$和$g(n)$对应着两个算法的时间复杂度函数，其中n是算法所使用的问题规模，比如排序算法里序列的长度。那么如何比较两个算法的快慢?

对于具体的n，$f(n)$和$g(n)$都能容易求出来，也能进行大小的比较。但问题在于对于不同的n，两者的大小可能是不一样的，而且两者的大小可能还不是常数比例大小的关系。另一方面，在比较不同算法时，我们倾向于使用尽可能大的问题规模，这样算法的核心步骤对问题规模的依赖关系才能显现出来。出于这两方面的考虑，我们一般在n很大时函数$T(n)$随n的整体增长趋势，也就是进行**渐近分析**(asymptotic analysis)。

在渐近分析中，我们最终会忽略掉函数的常数项和n的低阶函数项，而只保留n的最高阶函数形式。比如对于一个$T(n)$，其形式为：

$$T(n) = 2n^3 + 18n^2 + 60n + 200, n \geqslant 1. \tag{4.1}$$

容易得知，当n很大时，n的二次、一次、常数项相对于n^3可以忽略不计，此时$T(n)$的趋势依赖于n^3那一项。

例子2: 比较两个函数随着n增加时的变化。

> 两个函数分别为多项式函数和指数函数，形式分别为$f(n) = pn^a$和$g(n) = qb^n$。代码可见`ch4-comparison.cc`。在代码中，我们选择$p = 10^6$, $a = 2$, $q = 0.1$, $b = 1.0001$。
>
> 由计算结果可知，在$n < 420171$时，$f(n) > g(n)$。但当n继续增加时，$f(n) < g(n)$，而且$g(n)$相对于$f(n)$的大小增加非常快、直到两者比例出现溢出。如Figure 4.1所示。

■ 渐近分析里的记号

以下是渐近分析里的常用记号，用来比较函数之间的相对变化趋势。

（1）大$\mathcal{O}$记号(big-O notation)

如果存在正的常数c，使得对于任何$n \gg 2$都有$f(n) \leqslant c \cdot g(n)$， 都可以认为，在$n$足够大之后，$g(n)$给出了$f(n)$的一个渐近上界，记之为$f(n) = \mathcal{O}(g(n))$。

（2）小o记号 (small-o notation)

如果对于任何正的常数c，当n足够大之后都有$f(n) < c \cdot g(n)$， 都可认为函数$g(n)$给出了$f(n)$增长速度的一个严格非紧的渐近上界，记作$f(n) = o(g(n))$。

（3）大Ω记号 (big-Omega notation)

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch4-comparison.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
n =        2[2^ 1]: f(n) =        4e+06, g(n) =      0.10002; g(n)/f(n) =   2.5005e-08
n =        4[2^ 2]: f(n) =      1.6e+07, g(n) =      0.10004; g(n)/f(n) =   6.2525e-09
n =        8[2^ 3]: f(n) =      6.4e+07, g(n) =      0.10008; g(n)/f(n) =  1.56375e-09
n =       16[2^ 4]: f(n) =     2.56e+08, g(n) =      0.10016; g(n)/f(n) =   3.9125e-10
n =       32[2^ 5]: f(n) =    1.024e+09, g(n) =      0.10032; g(n)/f(n) =  9.79692e-11
n =       64[2^ 6]: f(n) =    4.096e+09, g(n) =     0.100642; g(n)/f(n) =  2.45708e-11
n =      128[2^ 7]: f(n) =   1.6384e+10, g(n) =     0.101288; g(n)/f(n) =  6.18214e-12
n =      256[2^ 8]: f(n) =   6.5536e+10, g(n) =     0.102593; g(n)/f(n) =  1.56544e-12
n =      512[2^ 9]: f(n) =  2.62144e+11, g(n) =     0.105253; g(n)/f(n) =  4.01509e-13
n =     1024[2^10]: f(n) =  1.04858e+12, g(n) =     0.110782; g(n)/f(n) =   1.0565e-13
n =     2048[2^11]: f(n) =   4.1943e+12, g(n) =     0.122727; g(n)/f(n) =  2.92603e-14
n =     4096[2^12]: f(n) =  1.67772e+13, g(n) =     0.150618; g(n)/f(n) =  8.97756e-15
n =     8192[2^13]: f(n) =  6.71089e+13, g(n) =     0.226859; g(n)/f(n) =  3.38046e-15
n =    16384[2^14]: f(n) =  2.68435e+14, g(n) =     0.514651; g(n)/f(n) =  1.91722e-15
n =    32768[2^15]: f(n) =  1.07374e+15, g(n) =      2.64865; g(n)/f(n) =  2.46675e-15
n =    65536[2^16]: f(n) =  4.29497e+15, g(n) =      70.1536; g(n)/f(n) =  1.63339e-14
n =   131072[2^17]: f(n) =  1.71799e+16, g(n) =      49215.3; g(n)/f(n) =  2.86471e-12
n =   262144[2^18]: f(n) =  6.87195e+16, g(n) =  2.42214e+10; g(n)/f(n) =  3.52468e-07
n =   420171      : f(n) =  1.76544e+17, g(n) =  1.76556e+17; g(n)/f(n) =      1.00007
n =   524288[2^19]: f(n) =  2.74878e+17, g(n) =  5.86678e+21; g(n)/f(n) =      21343.2
n =  1048576[2^20]: f(n) =  1.09951e+18, g(n) =  3.44192e+44; g(n)/f(n) =   3.1304e+26
n =  2097152[2^21]: f(n) =  4.39805e+18, g(n) =  1.18468e+90; g(n)/f(n) =  2.69365e+71
n =  4194304[2^22]: f(n) =  1.75922e+19, g(n) = 1.40346e+181; g(n)/f(n) = 7.97777e+161
```

FIG. 4.1. 多项式函数和指数函数的比较。 两个函数的形式分别为$f(n) = pn^a$和$g(n) = qb^n$。

如果存在正的常数c，使得对于任何$n \gg 2$都有$f(n) \geqslant c \cdot g(n)$， 都可以认为，在$n$足够大之后，$g(n)$给出了$f(n)$的一个渐近下界，记之为$f(n) = \Omega(g(n))$。

（4）小ω记号 (small-omega notation)

如果对于任何正的常数c，当n足够大之后都有$f(n) > c \cdot g(n)$， 都可认为函数$g(n)$给出了$f(n)$增长速度的一个严格非紧的渐近下界，记作$f(n) = \omega(g(n))$。

（5）大Θ记号 (big-Theta notation)

如果存在正的常数$c_1 < c_2$和函数$f(n))$，使得对于任何$n \gg 2$都有 $c_1 \cdot g(n) \leqslant f(n) \leqslant c_2 \cdot g(n)$， 都可以认为当$n$足够大之后，$g(n)$给出了$f(n)$的一个确界。 我们记之为$f(n) = \Theta(g(n))$。

可以看出来它们之间的差别：

• 对于Θ、$\mathcal{O}$、Ω： 当$f(n) = \Theta(g(n))$时，也满足$f(n) = \mathcal{O}(g(n))$以及$f(n) = \Omega(g(n))$，但反之不一定。

比如，$2n(n+1) = \Theta(n^2)$，意味着满足$2n(n+1) = \mathcal{O}(n^2)$以及$2n(n+1) = \Omega(n^2)$。但$2n(n+1) = \mathcal{O}(n^3)$时并不满足$2n(n+1) = \Theta(n^3)$， $2n(n+1) = \Omega(n \log n)$时也不满足$2n(n+1) = \Theta(n \log n)$。

• 对于$\mathcal{O}$和o： 当$f(n) = o(g(n))$，也满足$f(n) = \mathcal{O}(g(n)$，但反之不一定。

比如，$2n = o(n \log n)$时同样满足$2n = \mathcal{O}(n \log n)$， 但$2n \log n = \mathcal{O}(n \log n)$时并不满足$2n \log n = o(n \log n)$。

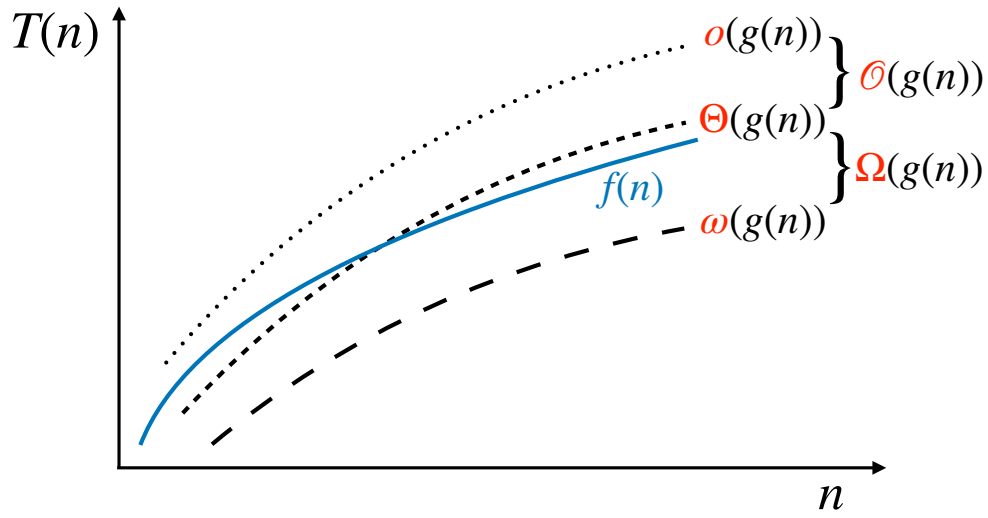


FIG. 4.2. 渐近分析里的五种记号。实线为$f(n)$，虚线表示$g(n)$。从上到下，$g(n)$依次满足$f(n) = o(g(n))$、$f(n) = \Theta(g(n))$、$f(n) = w(g(n))$。由图可知，满足$f(n) = o(g(n))$和$f(n) = \Theta(g(n))$的$f(n)$同时满足$f(n) = \mathcal{O}(g(n))$，满足$f(n) = w(g(n))$和$f(n) = \Theta(g(n))$的$f(n)$同时满足$f(n) = \Omega(g(n))$。

• 对于Ω和ω：当$f(n) = \omega(g(n))$，也满足$f(n) = \Omega(g(n)$，但反之不一定。

比如，$2n(n+1) = \omega(n\log n)$时同样满足$2n(n+1) = \Omega(n\log n)$，但$2n\log n = \Omega(n\log n)$时并不满足$2n\log n = \omega(n\log n)$。

4.3. 时间复杂度的渐进分析：计算

■ 渐近分析里的计算

注意：在本课程中，我们主要考虑$\mathcal{O}, \Omega, \Theta$这3种符号。

对于任意两个函数$f(n)$和$g(n)$，判断$f(n) = \mathcal{O}(g(n))$、或者$f(n) = \Omega(g(n))$、或者$f(n) = \Theta(g(n))$，可以通过计算$\lim_{n\to+\infty} g(n)/f(n)$的趋势。如果：

（1）$\lim_{n\to+\infty} g(n)/f(n) = \infty$，则$g(n)$的渐近趋势远远快于$f(n)$，则$f(n) = \mathcal{O}(g(n))$;

（2）$\lim_{n\to+\infty} g(n)/f(n) = 0$，则$g(n)$的渐近趋势远远慢于$f(n)$，则$f(n) = \Omega(g(n))$;

（3）$\lim_{n\to+\infty} g(n)/f(n)$为非零常数，则$g(n)$和$f(n)$的渐近趋势相同，则$f(n) = \mathcal{O}(g(n))$以及$f(n) = \Omega(g(n))$，即$f(n) = \Theta(g(n))$。

在后续渐近分析的求极限计算中，很多计算没法直接得到结果。我们会频繁使用两种

方法：洛必达法则（L'Hôpital's Rule）、夹逼定理（Squeeze/Sandwich Theorem）。

• 洛必达法则：它用来处理$\frac{\infty}{\infty}$的情形。 如果$\lim_{n\to+\infty} f(n) = +\infty$、$\lim_{n\to+\infty} g(n) = +\infty$，而且$f(n)$和$g(n)$对$n$均可导，则

$$\lim_{n\to+\infty} \frac{g(n)}{f(n)} = \lim_{n\to+\infty} \frac{g'(n)}{f'(n)}. \tag{4.2}$$

为了给分子分母不断降阶、一直到能够得到结果，可多次使用这个法则。

• 夹逼定理：本质上它借助了能够容易求解极限的式子来求解形式上稍显困难的问题。 如果

$$a(n) \leqslant \frac{g(n)}{f(n)} \leqslant z(n), \tag{4.3}$$

且

$$\lim_{n\to+\infty} a(n) = \lim_{n\to+\infty} z(n) = t, \tag{4.4}$$

其中t为一个与n无关的常数。则可得

$$\lim_{n\to+\infty} \frac{g(n)}{f(n)} = t. \tag{4.5}$$

例子3: 在渐近分析中，请选择以下式子$f(n)$和$g(n)$之间的符号。

(1) $f(n) = n - 100$, $g(n) = n - 200$;

(2) $f(n) = n^{1/2}$, $g(n) = n^{2/3}$;

(3) $f(n) = n2^n$, $g(n) = 3^n$。

解答:

(1) 可知:

$$\lim_{n\to+\infty} \frac{g(n)}{f(n)} = \lim_{n\to+\infty} \frac{n-200}{n-100} = \lim_{n\to+\infty} \frac{1-200/n}{1-100/n} = 1. \tag{4.6}$$

即$f(n) = \Theta(g(n))$。

(2) 可知:

$$\lim_{n\to+\infty} \frac{g(n)}{f(n)} = \lim_{n\to+\infty} \frac{n^{2/3}}{n^{1/2}} = \lim_{n\to+\infty} n^{1/6} = +\infty. \tag{4.7}$$

即$f(n) = \mathcal{O}(g(n))$。

(3) 可知:

$$\lim_{n\to+\infty} \frac{g(n)}{f(n)} = \lim_{n\to+\infty} \frac{3^n}{n\cdot 2^n} = \lim_{n\to+\infty} \frac{(1.5)^n}{n} \overset{\Delta}{=} \lim_{n\to+\infty} \frac{(1.5)^n \cdot \ln 1.5}{1} = +\infty. \tag{4.8}$$

则有$f(n) = \mathcal{O}(g(n))$。

在普遍意义上的多项式、指数、对数形式的式子之间，我们也可以进行渐近分析。

例子4: 证明以下渐近分析中的表达式:

（1）对于任意实数参数$a < b$，满足$n^a = \mathcal{O}(n^b)$;

（2）对于任意实数参数$a > 0, b > 1$，满足$n^a = \mathcal{O}(b^n)$;

（3）对于任意实数参数$a > 0, b > 0$，满足$(\ln n)^a = \mathcal{O}(n^b)$。

证明:

（1）可知:

$$\lim_{n\to+\infty} \frac{n^b}{n^a} = \lim_{n\to+\infty} n^{b-a} = +\infty. \tag{4.9}$$

则$n^a = \mathcal{O}(n^b)$。

（2）可知:

$$\begin{aligned}
\lim_{n\to+\infty} \frac{b^n}{n^a} &\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b^n \cdot \ln b}{a \cdot n^{a-1}} \\
&\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b^n \cdot (\ln b)^2}{a(a-1) \cdot n^{a-2}} \\
&\overset{\Delta}{=} \cdots \\
&\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b^n \cdot (\ln b)^{\lceil a \rceil}}{\frac{a!}{(a-\lceil a \rceil)!} \cdot n^{a-\lceil a \rceil}}.
\end{aligned} \tag{4.10}$$

上式中，等号上的Δ表示此等号右方使用了一次洛必达法则(L'Hôpital's Rule)。

$\lceil a \rceil$表示a的ceiling，表示不小于实数a的数里最小的一个整数。 当a的小数部分严格为0时，$a = \lceil a \rceil$，此时$a - \lceil a \rceil = 0$。 当a的小数部分严格不为0时，$a < \lceil a \rceil$，此时$a - \lceil a \rceil < 0$。 容易可知，此时对应着$\lim_{n\to+\infty} n^{a-\lceil a \rceil} = 1$或者0。 此外$b > 1$，则$\lim_{n\to+\infty} b^n = +\infty$。 综合可得$\lim_{n\to+\infty} b^n / n^a = +\infty$， 即$n^a = \mathcal{O}(b^n)$。

（3）可知:

$$\begin{aligned}
\lim_{n\to+\infty} \frac{n^b}{(\ln n)^a} &\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b \cdot n^{b-1}}{a(\ln n)^{a-1} \cdot \frac{1}{n}} = \lim_{n\to+\infty} \frac{b \cdot n^b}{a(\ln n)^{a-1}} \\
&\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b^2 \cdot n^{b-1}}{a(a-1) \cdot (\ln n)^{a-2} \cdot \frac{1}{n}} = \lim_{n\to+\infty} \frac{b^2 \cdot n^b}{a(a-1) \cdot (\ln n)^{a-2}} \\
&\overset{\Delta}{=} \cdots \\
&\overset{\Delta}{=} \lim_{n\to+\infty} \frac{b^{\lceil a \rceil} \cdot n^b}{\frac{a!}{(a-\lceil a \rceil)!} \cdot (\ln n)^{a-\lceil a \rceil}}
\end{aligned} \tag{4.11}$$

如同前一小题，等号上的Δ表示此等号右方使用了一次洛必达法则。

因为$a-\lceil a\rceil \leqslant 0$，可知$\lim_{n\to+\infty}(\ln n)^{a-\lceil a\rceil}=0$或者1。 此外$b>0$，则$\lim_{n\to+\infty} n^b=+\infty$。 综合得$\lim_{n\to+\infty} n^b/(\ln n)^a=+\infty$， 即$(\ln n)^a=\mathcal{O}(n^b)$。

在上面的计算中，$f(n)$和$g(n)$都有确定的式子。但一些式子没有简明的形式，而是表示为一些递推公式的形式。对于这类式子，我们依然可以对它们进行渐近分析。

例子5: 递推公式的渐近分析

题目来源：Ref.[3], Exercise 0.3.

斐波那契序列（Fibonacci sequence）$F_0, F_1, F_2, \cdots$按照以下规则生成

$$F_0=0, F_1=1, F_n=F_{n-1}+F_{n-2}. \tag{4.12}$$

这里我们将证明这个序列按照指数函数形式增长，并推导出增长函数的上下限。 请完成以下问题:

（1）使用归纳法证明：对于$n\geqslant 6$，F_n满足$F_n\geqslant 2^{0.5n}$。

（2）找到一个常数$c(<1)$，使得对于任意$n\geqslant 0$，$F_n\leqslant 2^{cn}$均满足。 请证明你的结果的正确性。

（3）对于$F_n=\Omega(2^{cn})$，你能找到的最大的c是多少。

解答:

（1）易知$F_1=1, F_2=1, F_3=2, F_4=3, F_5=5, F_6=8, F_7=13$。可知

$$n=6: F_6=8=2^{0.5\times 6}, \tag{4.13}$$

$$n=7: F_7=13>8\times 1.60>2^3\times 2^{1/2}=2^{0.5\times 7}. \tag{4.14}$$

即对于$n=6$和7，$F_n\geqslant 2^{0.5n}$成立。

如果不等式对$n=k(\geqslant 6), k+1$均成立，则有

$$F_k\geqslant 2^{0.5k}, F_{k+1}\geqslant 2^{0.5(k+1)}. \tag{4.15}$$

相应有

$$F_{k+2}=F_k+F_{k+1}\geqslant 2^{0.5k}+2^{0.5(k+1)}=2^{0.5k}\cdot(1+\sqrt{2})>2^{0.5k}\cdot 2=2^{0.5(k+2)}. \tag{4.16}$$

即此时$n=k+2$对于不等式依然成立。

综上所述，对于$n \geqslant 6$，F_n满足$F_n \geqslant 2^{0.5n}$。

（2）假设$F_n \leqslant 2^{cn}$对于任意$n \geqslant 0$成立。 对于$k(\geqslant 0)$和$k+1$，我们有

$$F_k \leqslant 2^{ck}, F_{k+1} \leqslant 2^{c(k+1)}. \tag{4.17}$$

相应有

$$F_{k+2} = F_k + F_{k+1} \leqslant 2^{ck} + 2^{c(k+1)} = 2^{c(k+2)} \cdot (2^{-2c} + 2^{-c}). \tag{4.18}$$

因为$F_{k+2} \leqslant 2^{c(k+2)}$成立，则必满足$2^{-2c}+2^{-c} \leqslant 1$。即$2^{2c}-2^c-1 \geqslant 0$。考虑到$c > 0$，则$2^c \geqslant \frac{1+\sqrt{5}}{2}$，即$c \geqslant \log_2 \frac{1+\sqrt{5}}{2}$。

综上所述，取任意$c \geqslant \log_2 \frac{1+\sqrt{5}}{2}$，对于任意$n \geqslant 0$，$F_n \leqslant 2^{cn}$均满足。

（3）取最大的c时，F_n满足$F_n = \Theta(2^{cn})$。此时取小题（2）中c取值范围里的最小值，有$c = \log_2 \frac{1+\sqrt{5}}{2} \approx 0.694$。

4.4. 递归算法的时间复杂度

这一小节我们主要来证明主定理（Master Theorem）。

注：请参考邓俊辉的课件（2023年秋季学期），第1章“绪论”之“迭代与递归：分而治之”。

递归形式在算法的分治(Divide-and-conquer)策略中很常见。在分治策略中，我们可以把一个规模比较大的问题分成若干小的部分，然后在每一个子问题中进行求解、并最终把这些子解重新组合起来。对于递归算法，在每一个子问题上，算法的流程是一样的。比如：

（1）在有序数组的二分查找法里，我们可以选择一个有序数组的序号中点对应的值：如果查找值小于中点值，则查找值自然在小于中点的那一半序列里；如果查找值大于中点值，则查找值在大于中点的那一半序列里。如此进行下去。

（2）在无序序列的归并排序算法里，我们把序列分成两半，在每一段序列里不断使用同样的排序算法、并最终把两个有序的子序列合并成一个有序序列。

对于问题规模n，我们定义递归算法的时间复杂度为$T(n)$。$T(n)$随n的变化可以用递推公式来表示。

- 对于求阶乘$k!$的算法，算法上可以写成$k! = k \times (k-1)!$。此时满足$T(n) = T(n-1) + \mathcal{O}(1)$。
- 对于有序数组的二分查找算法，可以写成$T(n) = T(n/2) + \mathcal{O}(1)$。

• 对于无序序列的归并排序算法，可以写成$T(n) = 2T(n/2) + \mathcal{O}(n)$。

主定理：在一般情况下，作为一个问题的递归算法，我们考虑把规模大小为n的问题平均分成$b(\geqslant 2)$份，然后在这些子问题上继续地使用分治算法。在构造问题的解时，我们选择$a \in \{1, \cdots, b\}$个子问题的解进行合并。此时子问题解的合并过程耗时为$\mathcal{O}(f(n))$。则递归算法的时间复杂度函数$T(n)$可以写成

$$T(n) = aT\left(\frac{n}{b}\right) + \mathcal{O}(f(n)). \tag{4.19}$$

以下我们推导$T(n)$的表达式。可以看出来，$T(n)$的最终形式依赖于$f(n)$的具体形式。

我们的整体思路是，对$T(n)$及其子问题不断使用Eq.(4.19)。随着递归算法的进行，问题规模n会越来也小，直到n减小到能够处理的步骤。 当$n = 1, 2$时，此时可以使用很简单的算法就能解决问题。此时可以选择$T(1) = \Theta(1)$。

推导过程可参考示意图Figure 4.3。 示意图以归并排序算法为例。其时间复杂度递推关系$T(n) = 2T(n/2) + \mathcal{O}(n)$表示的是：一段长度为$n$的序列被分成两段长度均为$n/2$的序列以继续使用这个算法（相应时间复杂度为$2T(n/2)$），并最终把这两段短的有序序列合并成一个有序系列（相应时间复杂度为$\mathcal{O}(n)$）。在算法执行层面上，序列最终被不断均分成分成单个元素、之后逐渐被合并成一个有序序列。在统计算法计算时间时，算法时间自然包括了单个元素的全部操作时间以及序列合并的全部时间。

在时间复杂度的解析推导中，我们的思路类似：在给定n的递推公式上，我们不断使用递推公式以得到更小的n，直到到达递归基，也就是对应着$n = 1$的情形。最终$T(n)$会被分解成很多容易计算的项。 相应的推导过程如下。

$$\begin{aligned}
T(n) &= aT\left(\frac{n}{b}\right) + \mathcal{O}(f(n)) \\
&= a\left[aT\left(\frac{n}{b^2}\right) + \mathcal{O}\left(f\left(\frac{n}{b}\right)\right)\right] + \mathcal{O}(f(n)) \\
&= a^2T\left(\frac{n}{b^2}\right) + a\mathcal{O}\left(f\left(\frac{n}{b}\right)\right) + \mathcal{O}(f(n)) \\
&= a^2\left[aT\left(\frac{n}{b^3}\right) + \mathcal{O}\left(f\left(\frac{n}{b^2}\right)\right)\right] + a\mathcal{O}\left(f\left(\frac{n}{b}\right)\right) + \mathcal{O}(f(n)) \\
&= a^3T\left(\frac{n}{b^3}\right) + a^2\mathcal{O}\left(f\left(\frac{n}{b^2}\right)\right) + a\mathcal{O}\left(f\left(\frac{n}{b}\right)\right) + \mathcal{O}(f(n)) \\
&= a^3\left[aT\left(\frac{n}{b^4}\right) + \mathcal{O}\left(f\left(\frac{n}{b^3}\right)\right)\right] + a^2\mathcal{O}\left(f\left(\frac{n}{b^2}\right)\right) + a\mathcal{O}\left(f\left(\frac{n}{b}\right)\right) + \mathcal{O}(f(n)) \\
&= a^4T\left(\frac{n}{b^4}\right) + a^3\mathcal{O}\left(f\left(\frac{n}{b^3}\right)\right) + a^2\mathcal{O}\left(f\left(\frac{n}{b^2}\right)\right) + a\mathcal{O}\left(f\left(\frac{n}{b}\right)\right) + \mathcal{O}(f(n)) \\
&= \cdots \\
&= a^tT\left(\frac{n}{b^t}\right) + \sum_{i=0}^{t-1} a^i\mathcal{O}\left(f\left(\frac{n}{b^i}\right)\right).
\end{aligned} \tag{4.20}$$

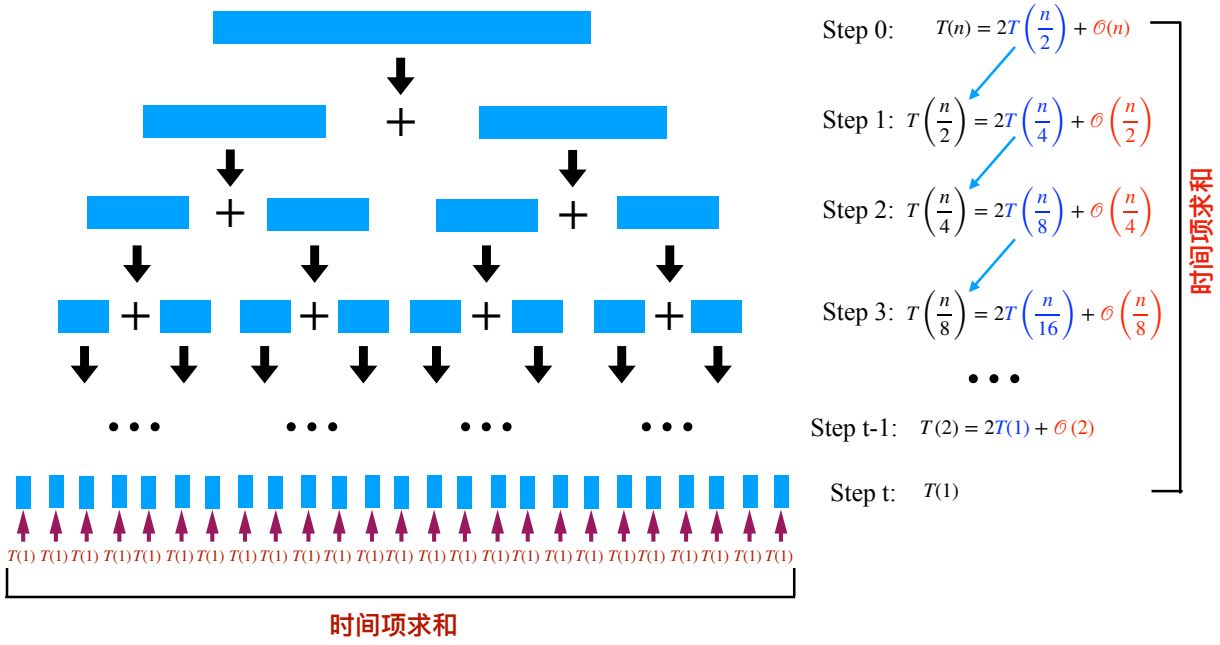


FIG. 4.3. 递归算法的时间复杂度的推导过程。这里以归并排序算法为例。图的左侧为长度为n的初始序列被逐步均分的过程，右侧对应均分过程中时间复杂度在不同序列长度上的递推关系。当n很大时，经过多次均分序列的步骤后，可以认为每段子序列的长度都为1。此时算法时间复杂度$T(n)$包括了最终的所有$T(1)$贡献的时间项，以及所有把两个短的序列合并成更长序列的时间项（所有标识为$\mathcal{O}(\cdot)$的式子）。这两个求和分别用图下方的紫色括号和图右侧的红色括号表示。

上式中，$t \in \{0, 1, 2, \cdots\}$为递推步骤的序号。

对于足够大的t，我们有$n/b^t = 1$，即$t = \log_b n$。此时对应着算法操作到了单个元素的层次。同时我们假定递归基$T(1) = \Theta(1)$，即为常数操作时间。则：

$$\begin{aligned} T(n) &= a^{\log_b n} T(1) + \sum_{i=0}^{\log_b n - 1} a^i \mathcal{O}\left(f\left(\frac{n}{b^i}\right)\right) \\ &= n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} a^i \mathcal{O}\left(f\left(\frac{n}{b^i}\right)\right). \end{aligned} \tag{4.21}$$

上式中，我们使用了等式：

$$a^{\log_b n} = n^{\log_n a \cdot \log_b n} = n^{\frac{\ln a}{\ln n} \cdot \frac{\ln n}{\ln b}} = n^{\frac{\ln a}{\ln b}} = n^{\log_b a}. \tag{4.22}$$

对于Eq.(4.21)的等号右侧，直观上来看，第一项表示单个元素操作时间（即递归基）的总和，第二项为每一次递归过程中子解合并的操作时间的总和（表示为操作次数乘以操

作时间）。此处可以和Figure 4.3中的两处求和做对比。

在递归算法中，$a \leqslant b$，相应$\log_b a \leqslant 1$。则$T(n)$最终相对于n的增长趋势依赖于a和b的取值、以及$f(n)$的具体形式。

此外，虽然上式推导用的是$\mathcal{O}$记号和$f(n)$的函数形式，但因为推导步骤涉及到的除法和求和等过程并不依赖于$\mathcal{O}$和$f(n)$的具体形式，因此在具体情况下可以按需要把$\mathcal{O}(f(n))$替换成相应的函数。

例子6: 在主定理里，参考$T(n)$的推导过程，求解以下情形对应的$T(n)$的形式。

（1）当$f(n) = n^{\log_b a - \varepsilon}$，其中$0 < \varepsilon \ll 1$。

（2）当$f(n) = n^{\log_b a + \varepsilon}$，其中$0 < \varepsilon \ll 1$。

（3）当$f(n) = n^{\log_b a} \cdot \log^k n$，其中$k > 0$。

解答:

（1）根据Eq. (4.21)，我们有

$$\begin{aligned}
T(n) &= n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} a^i \mathcal{O}\left[\left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon}\right] \\
&= n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} a^i \mathcal{O}\left[\frac{n^{\log_b a - \varepsilon}}{b^{i(\log_b a - \varepsilon)}}\right] \\
&= n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} \cancel{a^i} \mathcal{O}\left(\frac{n^{\log_b a - \varepsilon}}{\cancel{a^i} \cdot b^{-i\varepsilon}}\right) \\
&= n^{\log_b a} T(1) + \sum_{i=0}^{\log_b n - 1} \mathcal{O}\left(n^{\log_b a - \varepsilon} \cdot b^{i\varepsilon}\right) \\
&= n^{\log_b a} T(1) + \mathcal{O}\left(n^{\log_b a - \varepsilon} \cdot \sum_{i=0}^{\log_b n - 1} b^{i\varepsilon}\right). \qquad (4.23)
\end{aligned}$$

上式中的级数求和为：

$$\sum_{i=0}^{\log_b n - 1} b^{i\varepsilon} = \frac{b^{\log_b n \cdot \varepsilon} - 1}{b^\varepsilon - 1} = \frac{n^\varepsilon - 1}{b^\varepsilon - 1}. \qquad (4.24)$$

因为$\varepsilon > 0$，当n很大时，$\sum_{i=0}^{\log_b n - 1} b^{i\varepsilon} \to n^\varepsilon/(b^\varepsilon - 1)$。则，

$$\begin{aligned}
T(n) &= n^{\log_b a} T(1) + \mathcal{O}\left(n^{\log_b a - \varepsilon} \cdot \frac{n^\varepsilon}{b^\varepsilon - 1}\right) \\
&= n^{\log_b a} T(1) + \mathcal{O}\left(\frac{n^{\log_b a}}{b^\varepsilon - 1}\right) \\
&= \mathcal{O}(n^{\log_b a}). \qquad (4.25)
\end{aligned}$$

（2）略。

（3）设$f(n)=n^{\log_b a}\cdot\log^k n$，其中$k>0$。利用Eq. (4.21)，我们有

$$\begin{aligned}T(n)&=n^{\log_b a}T(1)+\sum_{i=0}^{\log_b n-1}a^i\mathcal{O}\left[\left(\frac{n}{b^i}\right)^{\log_b a}\cdot\log^k(\frac{n}{b^i})\right]\\&=n^{\log_b a}T(1)+\sum_{i=0}^{\log_b n-1}\cancel{a^i}\mathcal{O}\left[\frac{n^{\log_b a}}{\cancel{a^i}}\cdot(\log n-\log b^i)^k\right].\end{aligned}\tag{4.26}$$

上式中，$0\leqslant i\leqslant\log_b n-1$，则$b^i\leqslant b^{\log_b n-1}=\frac{n}{b}<n$。则$n$很大时，$\log n-\log b^i=\Theta(\log n)$。我们有

$$\begin{aligned}T(n)&=n^{\log_b a}T(1)+\mathcal{O}\left(\sum_{i=0}^{\log_b n-1}n^{\log_b a}\cdot\log^k n\right)\\&=n^{\log_b a}T(1)+\mathcal{O}\left(n^{\log_b a}\cdot\log^k n\cdot\log_b n\right)\\&=n^{\log_b a}T(1)+\mathcal{O}\left(n^{\log_b a}\cdot\log^{k+1}n\cdot\log_b 2\right)\\&=\mathcal{O}\left(n^{\log_b a}\cdot\log^{k+1}n\right).\end{aligned}\tag{4.27}$$

4.5. 计算复杂性

不同的算法有快慢之分，但问题本身的难度可否有定量的区分标准：这意味着对于本质上很困难的问题，是几乎不可能找到正确的快速算法。

研究问题本身难度的学科叫**计算复杂性**（Computational Complexity）。问题的计算复杂性的类别(classes)有很多。以下是一些常见的计算复杂度类别：

▷ P (Polynomial)：多项式$\mathcal{O}(n^t)(t>0)$时间内可解决。比如多个数的加法和乘法计算等。若问题存在一个复杂度在此范围以内的算法，则该问题是可有效求解的或着易解的(tractable)；不存在多项式复杂度算法的问题，称作难解的(intractable)问题。

▷ NP (Non-deterministic Polynomial)：指的是在多项式时间内可以验证其解是否正确，但此问题的求解可能很困难。它包含了一大类问题，包括P问题。此外还有：

◦ NP-complete（NP-完全）问题：指的是NP问题里最困难的那部分问题 [16–18]。

◦ NP-hard（NP-困难）问题：意味着在最坏情况下需要遍历整个解空间才能找到最优解。此时解空间内所有可能解的数目为指数级形式$\mathcal{O}(t^n)$（$t>1$）。

▷ #P (sharp-P): 最坏情况下需要$\mathcal{O}(t^{n^n})$的计算时间。比如把一个NP-困难问题的所最优解对应的所有构型都列举出来。一般NP-困难问题只需要把最优值找出来，但这类问题

需要把满足最优解的所有构型都找出来。而在一些问题里，这类最优解对应构型的数目本身是问题规模的指数级大小。

部分复杂性分类可见Figure 4.4。

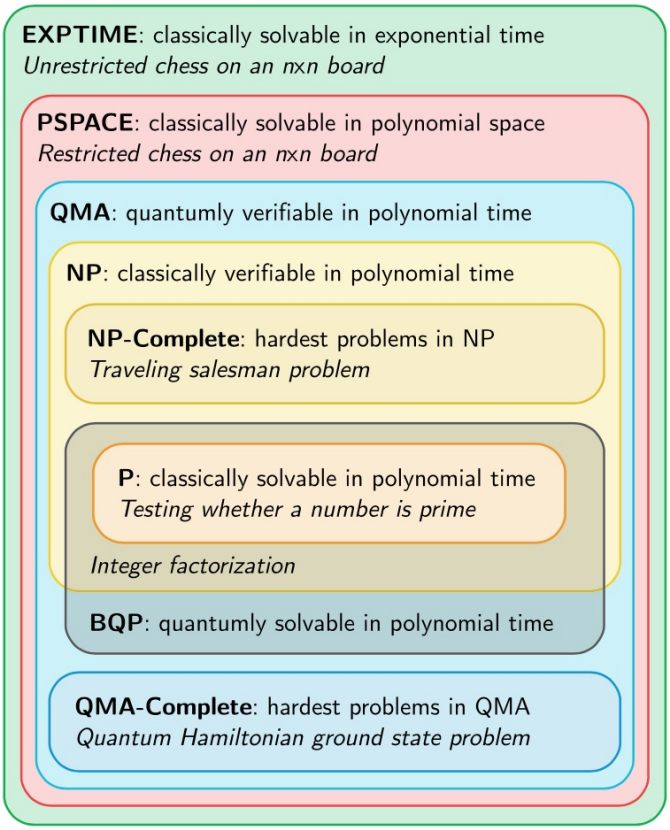


FIG. 4.4. 一些常见的计算复杂性分类。来自Ref.[19]的P4。这幅图同时涉及到量子计算问题和经典问题的复杂性分类。请注意图里的NP、NP-Complete、P类别的相对位置。

例子7: NP-完全问题的例子

21 NP-complete problems from Richard Karp, Reducibility Among Combinatorial Problems (1972)

→ `https://zhuanlan.zhihu.com/p/474342154`

Karp基于已经证明为NP-完全问题的SAT问题，借助问题之间的约化，证明了其它一系列问题能被约化为SAT问题、因而也为NP-完全问题。

■ P versus NP问题

当下我们一般认为在NP问题内部，除了P问题，还存在困难的多的、不存在快速算法的NP-完全问题。 但P versus NP问题讨论的是NP问题内部是否真的存在这类更加困难的问题。这是理论计算机科学研究里的终极问题之一。

→ `https://zhuanlan.zhihu.com/p/141884913`

■ 图论组合优化问题

这里介绍定义在图（graphs）或网络（networks）上的NP-困难问题。图和网络的内容我们将会在第15章详细介绍。这里只简单涉及一些基本概念。 一幅图或者网络由节点（nodes/vertices）和连边（links/edges）构成。连边连接着节点。在复杂系统的建模里，常常用图或者网络来表示一个复杂相互作用系统：节点表示系统的基本组成单元，连边表示组成单元之间的相互作用。连边可以不限于两个组成单元之间，对应着两体或者多体相互作用。 定义在图上的优化问题可以描述为选择一些节点集或者连边集、同时保证这些集合满足一定的结构约束。在这里节点或连边只有属于或不属于集合两种可能性，优化问题的目标在于寻找最小或者最大的集合。这类离散变量的优化问题叫做组合优化问题（combinatorial optimization problems） [20, 21]。 与次对应的是变量类型为连续的连续优化问题（continuous optimization problems）。

在连续优化问题里，我们可以通过梯度下降方法（gradient descent method）这类通用方法来求解。但对于组合优化问题，这类方法并不适用。在组合优化问题里，因为变量取值为离散值，可能出现局域上若干变量同时取某些特定值时才具有最小值的情况。但这种最小值利用局域算法是很难找到的。相对而言，很多NP-困难问题的形式表述为图论组合优化问题。

以下是一些常见的、类型为NP-困难的图论组合优化问题：

▷ Travelling Salesman Problem (TSP，旅行商问题)

问题描述：给定一个连边带权重的无向连接图，从一个格点出发遍历其它所有格点，其连边集对应了一条路径。旅行商问题在于求解路径上的连边权重之和的最小值所对应的连边集。此问题由欧拉在1759年提出来。

▷ Minimum Vertex Cover Problem（MVC，最小格点覆盖问题）

问题描述：给定一个无向图，它的一个节点覆盖（vertex cover， VC）为一个节点集，使得图中所有连边都有至少一个端点在这个节点集里。容易可知，图的所有节点本身构成了一个节点覆盖。最小格点覆盖问题需要寻找这个节点集内节点的最小值、最小节点覆盖的构型等等。

▷ Feedback Node/Vertex Set Problem (FNS/FVS Problem, 反馈节点集问题)

问题描述：给定一个无向图，我们可以删除一些节点使得图不再有环或圈（cycles or loops）结构。满足这个约束的节点集称为反馈节点集(Feedback Vertex Set, FVS)。注意这里的“反馈”对应着图中的圈结构。容易得知，图的所有节点本身也构成了一个反馈节点集。反馈节点集问题指的是寻找图的反馈节点集内节点的最小值、最小反馈节点集的构型等等。

Figure 4.5展示了图上的两个组合优化问题。

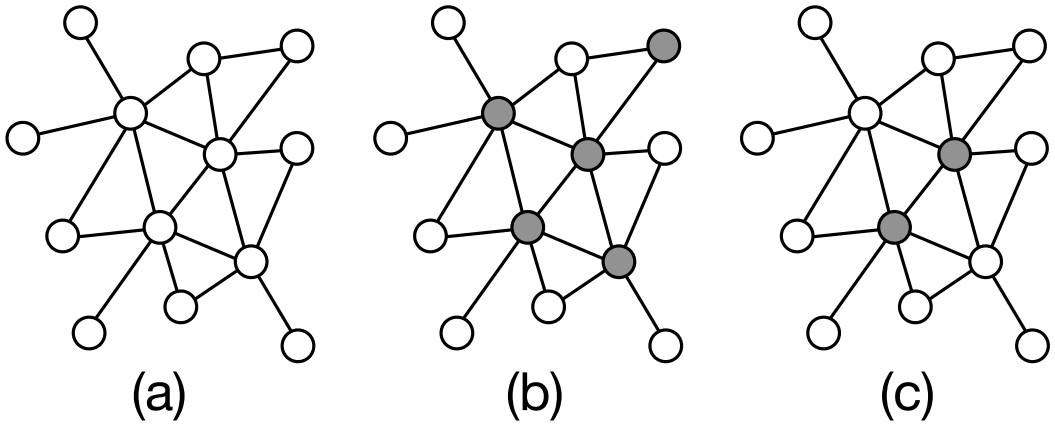


FIG. 4.5. 无向图上的两个组合优化问题。 (a) 为一幅包括13个节点、19条连边的无向图。 (b) 中的5个标记为黑色的节点为图的一个节点覆盖。此节点覆盖也对应着图的一个最小节点覆盖，其大小为5。 (c) 中的2个标记为黑色的节点为图的一个反馈节点集。此反馈节点集也对应着图的一个最小反馈节点集，其大小为2。

■ 函数的能量图景与局域算法

问题难易程度的差别首先基于维度（问题规模）的多少，其次基于问题本身涉及到的形式和约束。维度对局域算法的影响可以通过问题的函数图像或者能量图景（energy landscape）来表示，如图Figure 4.6。对于一元二次方程，它的函数图像是很简单的凹函数，其全局极小值可以很容易通过局域算法找到。但在高维函数里，函数图像会复杂的多：图像里出现大量的局域极小值，而且它们之间都有很高的能量势垒（energy barrier）。局域算法一般基于改变少数变量，很难跳出相应的能量势垒、而到达全局极小值，而是容易被困在初始值附近的某个局域极小值里。这样的能量图景导致普通的局域算法在求解这类优化问题时常常失效。

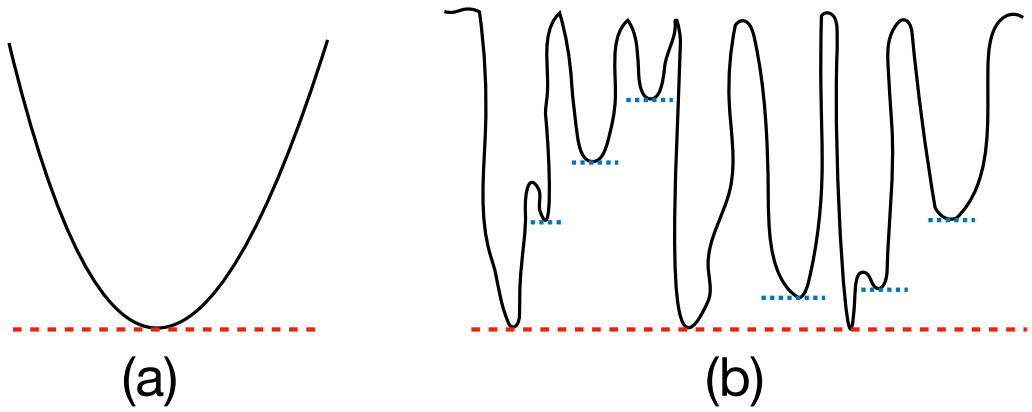


FIG. 4.6. 多维优化问题里函数的能量图景（energy landscape）。（a）为一个一元二次方程的能量图景，（b）为一个多维方程的能量图景。红色虚线表示两个函数的全局极小值，蓝色虚线表示局域极小值。可知（a）只有1个全局极小值，（b）有3个全局极小值，6个局域极小值。

习题

4.1 算法时间复杂度的渐近分析

以下每个小题中两个函数分别为$f(n)$和$g(n)$。在渐近分析中，可以写成$f(n) = ___(g(n))$的形式。请写出推导过程、并选择相应的$\mathcal{O}$、Θ、Ω符号。

注意对数的一些常用写法：$\log = \log_2$, $\lg = \log_{10}$, $\ln = \log_e$。

前17小题的来源：Ref.[3], Exercise 0.1.

(1) $n - 100$, $n - 200$;

(2) $n^{1/2}$, $n^{2/3}$;

(3) $100n + \log n$, $n + (\log n)^2$;

(4) $n \log n$, $10n \log 10n$;

(5) $\log 2n$, $\log 3n$;

(6) $10 \log n$, $\log(n^2)$;

(7) $n^{1.01}$, $n \log^2 n$;

(8) $n^2/\log n$, $n(\log n)^2$;

(9) $n^{0.1}$, $(\log n)^{10}$;

(10) $(\log n)^{\log n}$, $n/\log n$;

(11) $\sqrt{n}$, $(\log n)^3$;

(12) $n^{1/2}$, $5^{\log_2 n}$;

(13) $n2^n$, 3^n;

(14) 2^n, 2^{n+1};

(15) $n!$, 2^n;

(16) $(\log n)^{\log n}$, $2^{(\log_2 n)^2}$;

(17) $\sum_{i=1}^{n} i^k$, n^{k+1};

(18) $n^{\frac{1}{2}}$, $(\log 2n)^5$;

(19) n^2, $3^{2\log n}$;

(20) n^3, $(\log n)^{2\log n}$;

(21) $n!!$, 2^n.

4.2 递归算法的时间复杂度

参考讲义里主定理的推导过程，请推导出以下情形对应的$T(n)$:

(1) $f(n) = n^{\log_b a+\varepsilon}$，其中$0 < \varepsilon \ll 1$。

(2) $T(n) = T(\frac{n}{2}) + \mathcal{O}(n^k)$，其中$k > 0$。

(3) $T(n) = 2T(\frac{n}{2}) + \mathcal{O}(n^k)$，其中$k > 0$。

(4) $T(n) = T\left(\frac{n}{2}\right) + c$，其中$c(> 0)$为某一常数。

(5) $T(n) = 2T\left(\frac{n}{2}\right) + cn + d$，其中$c(> 0)$和$d(> 0)$均为常数。

5. (VECTOR)

参考内容：邓俊辉教材第2.1-4、2.6、2.8、10.1节。李春葆教材第2.1–2、9.2、10.3、10.5节。

本章介绍了向量作为一种线性数据结构的基本概念。之后介绍介绍了基于向量的查找算法（比如二分查找法）和排序算法（冒泡排序法、归并排序法、快速排序法）。

5.1. 向量的基本性质

■ 从数组到向量

在C语言和C++语言里，都存在数组这个数据结构。比如类似：int A[40]。 在数组里，类型和元素个数需要在申明时全部指定。在数组里，可以通过A[i]这样的写法来访问下标为i$\in \{0, 1, \cdots, n-1\}$的元素。数组作为一种典型的序列结构，其元素在内存里按顺序排列。如果A[0]的内存地址为p_0，数据类型占据的字节数为a，则元素A[k]的物理地址为$p_k = p_0 + k \times a$。

向量(vector)可以认为是数组的推广。它的数据元素在内存里也是按顺序排列。向量数据元素可以一样、也可以不一样。向量的元素可以通过秩(rank)来确定：数据元素的秩为它之前的元素个数，类似于数组的下标。

5.2. 向量的ADT

数据结构的概念在代码层面可以通过抽象数据类型(ADT)来实现。向量也一样。在C++语言里，以向量为例，抽象数据类型一般可以写成类(class)或者结构体(struct)的形式。向量的抽象数据类型中，包括其数据本身和数据操作过程两部分。

• 数据单元：向量的数据单元包括int size, int capacity, T* elem三部分，分别表示实际存储元素的个数、分配的数据单元空间、数据区的指针。

• 操作函数：表示对数据单元的访问、改变、等等各种操作。操作一般包括构造函数(constructor)、析构函数(destructor)、各类操作函数等等。构造函数表示生成对象的过程。按照具体需要可以利用函数重载功能(overload)来设置不同的形参。析构函数一般只有一个。操作函数可以按照实际需要写成任意函数。

向量class/vector的代码形式的示意图可见Figure 5.1。

内存中向量的示意图可见Figure 5.2。

■ STL Vector

```
*
* class Vector {
*  private:
*   //数据项，包括数据项的数目、类型、指针等等；
*   int size;//vector的数据单元个数；
*   int capacity;//vecor的容量；
*   T* elem;//数据区的起始指针；
*
*  public:
*   //操作函数项，包含对数据项的访问、操作、算法等等；
*   Vector(int);//使用函数重载来实现多种构造函数；
*   Vector();
*
*   ~Vector();//析构函数；
*
*   int Search(T);//给定元素来查询并返回其对应的秩；
*   void Change(int, T);//给定秩，替换相应的元素；
*   ... ...
*   ... ...
*  }
*
*  struct Vector {
*   //数据项，包括数据项的数目、类型、指针等等；
*   int size;//vector的数据单元个数；
*   int capacity;//vecor的容量；
*   T* elem;//数据区的起始指针；
*
*   //操作函数项，包含对数据项的访问、操作、算法等等；
*   Vector(int);//使用函数重载来实现多种构造函数；
*   Vector();
*
*   ~Vector();//析构函数；
*
*   int Search(T);//给定元素来查询并返回其对应的秩；
*   void Change(int, T);//给定秩，替换相应的元素；
*   ... ...
*   ... ...
*  }
*
```

FIG. 5.1. 使用类(class)或者结构体(struct)来实现数据结构的抽象数据类型。这里以向量为例。向量的数据项包括向量的数据元素个数、容量（当下可容纳的数据元素个数）、数据区的起始指针。向量的函数部分包括构造函数（构造函数的名称一般和结构体名称相同）、析构函数、各类查询或操作函数等等。函数部分可以依据实际需要进行增加。

在本课程里，我们使用C++编程语言里的STL Vector来实现向量的抽象数据结构。STL Vector属于C++的STL（标准库）的容器(containers)。上述提到的向量的大部分功能在STL Vector里都能直接实现。

对于STL Vector，大家需要掌握的包括:

（1）Vector的声明、初始化。

（2）使用下标、指针、迭代器来访问元素。

（3）添加元素、删除元素。

参考链接:

→ `https://en.cppreference.com/w/cpp/container/vector`

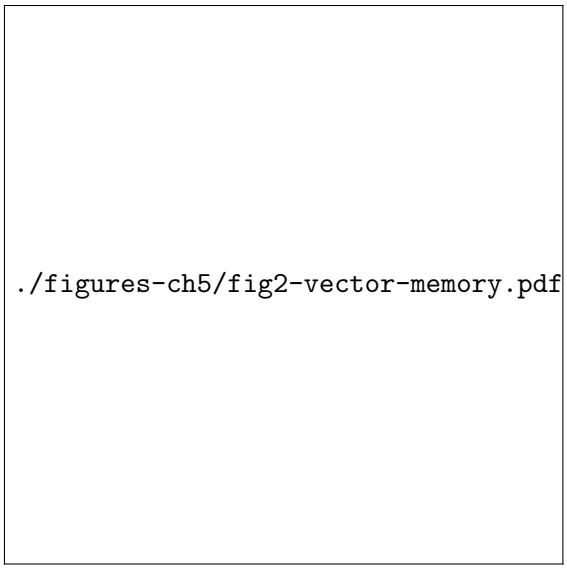


FIG. 5.2. 向量的示意图。这里的向量A存储了5个整型构成的一个序列$\{4, 8, 7, 1, 3\}$。图里依次展示了下标所对应的元素、数据元素本身、数据元素的内存地址。矢量的数据元素个数(size)为5，容量(capacity)为8、首元素指针(pelem)为A[0]的地址0x600002360100。地址单位为字节(byte)，但具体地址在不同电脑、不同编译器、不同时间上都会不一样。图中虚线方框表示系统分配了空间、但并没有存储元素。

→ `https://cplusplus.com/reference/vector/`

→ `http://c.biancheng.net/view/6749.html`

例子1: STL里容器vector上的基本操作

参考代码：`ch5a-stl-vector.cc`的函数Operations()。

请注意里面有关Vector的声明、插入和删除元素的过程。此外注意利用下标、指针、迭代器来访问元素等操作。

5.3. 动态空间管理

向量为一个典型的序列结构。

• **静态空间管理**：在向量的生命周期里，内部数组所占物理空间的容量不允许调整。

这种管理方式在内存空间比较紧凑时，会无法写入新的数据或者导致通过抹除之前数据来存入数据。一般情况下，系统会为数据结构的内存保留一个预留量。此时，数据本身

所占据空间一般会小于内存为此分配的空间。前者为数据单元数目(size）、后者为分配内存空间的容量(capacity)。相应的装填因子(load factor)可以定义为$\lambda = \text{size}/\text{capacity}$。

• **动态空间管理**：本质上是实现一个可扩充向量（extendable vector）。比如当往向量不断添加元素时，系统会重新开辟新的内存空间来放置：此时系统会划分新的一片内存，把之前的数据拷入，然后删除之前的数据、并释放相应的内存。这个过程对应着容量随数据单元个数(size)增加而阶梯式增加。在动态空间管理里，一般设置一个装填因子的临界值λ_c：当size的值大到导致大于λ_c时，容量会按某个方式增加。大致流程可见Figure 5.3。

一个好的动态空间管理方法需要在两个看似矛盾的因素之间找到一个平衡。第一，需要尽可能地利用内存，也就是把装填因子的临界值设置的尽可能高，但这意味着在增加元素时可能需要频繁增加容量。第二，保证尽可能少地改变容量，因为它涉及的内存操作过程非常耗时，但这意味着需要尽可能增加预留内存，也就是把状态因子临界值λ_c设置的尽可能小。

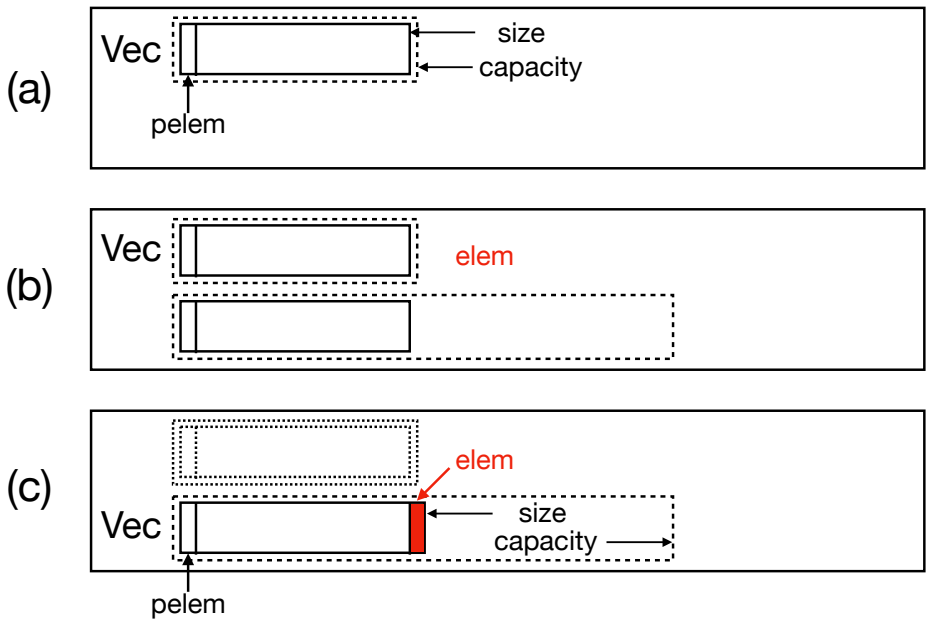


FIG. 5.3. 向量的动态空间管理的大致流程。(a)展示了内存中的一个向量Vec，其数据单元个数(size)和容量(capacity)相等，其数据区指针(pelem)指向首元素的内存地址。(b)此时需要往向量里插入一个元素elem。系统为向量在内存中开辟新的向量空间，其大小为之前capacity(容量)的两倍，之后会把原向量的元素依次拷入新的向量。此时新向量里数据单元数目依然为size。(c)之前的向量在内存中被释放，并在新向量Vec里插入新的数据单元elem。最终新向量Vec的size增加1，capacity增加为之前的两倍，pelem指向新向量所在的内存地址。

以下例子会展示STL Vector里使用的动态空间管理方法。

例子2: STL Vector的动态空间管理

代码展示了在不断给Vector添加或删除元素时，不断输出数据元素个数(size)、容量（capacity）、以及首元素内存地址(pelem)。

参考代码：`ch5a-stl-vector.cc`的函数MemoryAddress()。

从例子的结果可知，在添加元素时，一般size $\leqslant$ capacity。如果在某一步增加元素之后size = capacity、并继续添加元素时，容量会直接增加到之前的两倍。由此可知，在STL Vector的动态空间管理方式里，默认的装填因子临界值$\lambda_c = 1$、容量(capacity)的改变方式为翻倍。此外我们还会发现在容量(capacity)改变时，Vector首元素指针也发生了变化。这证实了容量(capacity)增加时，当下Vector相应地被分配到了另外一片内存区域。

在删除元素时，容量保持不变，为删除过程之前的值（实际上是整个生命周期里capacity的最大值）。期间Vector首元素的内存位置也不变，表示Vector在删减过程中一直处在之前的内存区域。

此外在删减元素时，如果需要修改容量capacity的值、使其和数据元素个数size相等（也就是删除所有冗余空间），可以使用函数shrink_to_fit()。此时size和capacity会同时减少。此外elem也在不断变化，表示系统为此也在不断开辟内存空间和转移元素。

5.4. 序列的生成

这里讨论有序序列和随机序列的生成。前者会用来测试查询算法，后者用来测试排序算法。

1. 有序序列

• 有序序列的定义：一般我们考虑非降序的序列。这意味着序列中可能存在重复的元素。 一般来说，并不是所有序列都是能够比较乃至排序的。比如对于一个由颜色构成的序列：我们能够比较两个颜色是否一样，但判断两种颜色的大小在一般情况下没有意义。有序向量一般需要定义一个值的大小的比较。

• 有序序列的生成：这里我们考虑生成一个带有随机性的有序序列。对于一个有序序列Vec，后一个元素不会比前一个元素小，对应着元素增加值increment = Vec[i] -

$\text{Vec}[i-1] \geqslant 0$，下标$i = 1, 2, \cdots$。这里的“随机”指的是元素增加值increment的大小。为了控制序列的增长速度，我们设置了元素增加值的最大值为range ($\geqslant 0$)。我们的算法可以实现如下：

（1）当$i = 0$时，$\text{Vec}[0] = \text{vec0}$。

（2）当$i \geqslant 1$时，increment = rand() % range + base;

$$\text{Vec}[i] = \text{Vec}[i-1] + \text{increment}.$$

上式中，vec0为有序序列的首元素，base($\geqslant 0$)为向量增量的最小值，rand()为一个整数随机序列，range($\geqslant 1$)为用来控制增量变化部分的值。可知向量元素增量的取值范围为[base, base + range - 1]。

• 特殊的有序序列：通过选择base和range的取值，这个生成过程可以用来生成一些特殊的有序序列，比如：

（1）元素各不同的序列：此时base $\geqslant 1$；

（2）元素全同的序列：此时base $= 0$、且range $= 1$。

例子3: 有序向量的生成

这里展示了通过设置不同的base和range来得到不同的、由整型变量构成的有序序列。

参考代码：`ch5b-seq.cc`的main()函数里的GenerateAnOrderedSeq()。

• 有序序列的唯一化（向量元素去重）的算法：两指针算法。算法实现可参考第2.6.3小节（邓俊辉教材P45-47）。

例子4: 有序向量的元素去重

在生成随机序列后使用两指针算法来去除重复的向量元素。

参考代码：`ch5b-seq.cc`的main()函数里的UniquifyAnOrderedSeq()。

2. 随机序列

这里讨论随机序列的生成算法。

随机序列的有点在于它没有内在的结构，能够用来测试算法在平均意义上的时间复杂度。

对于任意一个有限长度的序列，我们容易得知它们有一个最小值，而其它元素相对于它来说有一个正的差，且这个差有个最大值。借助这个简单的表述方式，我们可以设计一个随机序列的生成方式。随机序列的生成过程的参数如下：

- 序列长度n;
- 序列元素的最小值base;
- 元素相对于最小值的变化范围的区域为$[0, \text{range})$，range即为序列变化的上界。

随机序列的生成过程中，每一个元素独立生成。在指定基准值base之后，只需要随机生成范围在[0,range)之内的随机数。此时可以借助为C++标准库自带的随机整型数的生成器rand()。rand()的取值范围为[0,INT_MAX)。INT_MAX= $2^{31}-1 = 2147483647$，因为int类型占据内存4个字节共32位。当需要随机生成的元素的变化范围为[0, range)时，我们使用rand() % range。最终，我们可以得到以下生成随机序列的方法：

$$\text{Vec}[i] = \text{base} + \text{rand}() \ \% \ \text{range}, \tag{5.1}$$

其中$i \in [0, n)$为元素的下标/秩。

例子5: 整型随机向量的生成。

生成给定长度的、给定取值范围的、由整型元素组成的无序序列。

参考代码：`ch5b-seq.cc`中main函数里的GenerateARandomSeq()。

5.5. 查找算法

1. 序列上的查找任务

查找是序列上的一个基本算法任务。

查找一般对于有序序列来说，也就是给定义一个元素，寻找它在序列里的位置。查找算法的输入为有序序列和所要查找的元素，返回为元素在序列的位置信息;

（1）如果元素在序列里存在，则返回元素的下标;

（2）如果元素在序列里不存在，则返回指示信息（比如“不存在”）或者元素大致的位置（比如在哪两个坐标之间）。

序列上查找任务的图示如Figure 5.4所示。

举个例子，考虑以下的整型数构成的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 5,\ 5,\ 7,\ 8,\ 10,\ 12,\ 13\}. \tag{5.2}$$

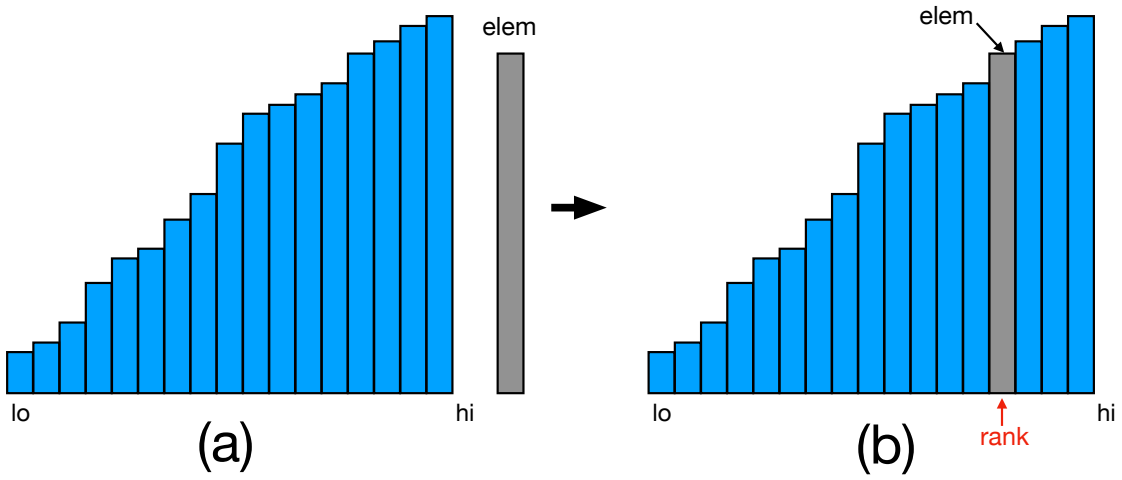


FIG. 5.4. 序列上的查找任务。 (a)为一个有序序列Vec。其柱状图的高度表示序列元素之间的相对大小。查找问题指的是给定一个待查元素elem、搜索下标或秩的区间范围[lo, hi)。 (b)查找算法返回elem所对应的下标或秩rank。

序列中元素共10个，下标分别为0到9。容易发现，下标为2、3、4的元素相同。为了区分这些重复元素，我们可以继续使用下标。此时序列写成:

$$\text{Vec} = \{2,\ 4,\ 5_{\text{a}},\ 5_{\text{b}},\ 5_{\text{c}},\ 7,\ 8,\ 10,\ 12,\ 13\}. \tag{5.3}$$

对于有序序列Vec，如果待查元素为4，则查找算法需要返回下标1。如果待查元素为6，则可以简单地返回下标-1，表示查找元素不存在。

进一步，我们可以考虑一些更加具体的查找任务:

（1）序列里的重复元素：如果有序序列里有重复元素，而且查找元素就是重复元素，那么算法最终返回多个下标中的哪一个？比如在序列Vec里查找元素5，此时查找算法需要确定返回下标2、3、还是4。我们可以额外规定查询元素为序列内的重复元素时，算法返回的是重复元素的最小下标（对应下标2）或者最大下标（对应下标4）。此时我们怎样在算法里实现?

（2）序列里不存在的元素；对于序列中不存在的元素，除了返回-1，算法可以返回有关它在序列中的位置信息。比如在序列Vec里查找元素6。容易可知6刚好在$\text{Vec}[3] = 5$和$\text{Vec}[4] = 7$之间。 在一般情况里，如果待查元素elem在数值上位于两个元素之间，比如$\text{Vec}[i] < \text{elem} < \text{Vec}[i+1], i \in [0, N)$，此时我们可以统一返回$i$或者$i+1$。此时$i$对应于小于待查元素的序列元素的最大下标，$i+1$对应于大于待查元素的序列元素的最小下标。此时在算法上如何处理这个就近下标的问题?

后面我们将会看到，为了处理上述两个问题，不同算法的执行过程会很不同。

2. 查找长度

在讨论查找算法前，如何度量查找算法的时间复杂度?

查找算法的基本操作是通过下标取值、比较元素的大小。前者在数组和向量这种数据结构上是个很小的常数。于是我们把“元素比较大小”作为查找算法里的基本操作。对于一个待查找的元素，其在得到查找结果时所需的比较元素的次数为其对应的查找长度（search length）。 容易得之，对于有序序列中的任意一个元素，其查找长度仅仅依赖于有序向量的长度和查找元素对应的下标位置，并不依赖于元素的具体取值。 查找给定元素时，元素有可能存在、也可能不存在于序列当中。相应地可以定义以下两个查找长度:

（1）成功查找长度：给定一个有序序列，如果把向量本身的元素作为查找元素，可以得到相应的查找长度。

（2）失败查找长度：给定一个有序序列，如果把一个不存在于此向量中的元素作为查找元素，可以得到相应的查找长度。

对于一个序列来说，平均查找长度(average search length, ASL)对应着所有元素的查找长度的平均值。可以发现，具有更小的平均查找长度的查找算法相对来说更有效率。 我们相应地可以定义序列层面的“平均成功查找长度”和“平均失败查找长度”。

下面我们可以简单估计以下平均查找长度的量级。 因为此时我们考虑的是有序序列，选取任意一个下标index作为，我们可以把序列分成了三部分；左侧序列（对应下标小于index的所有序列元素）、给定下标对应着的元素、右侧序列（对应下标大于index的所有序列元素）。我们可以简单地把下标index作为查找算法的切分点。 在具体的查找任务里，给定查找元素elem，我们可以得知:

（1）如果待查元素elem小于给定切分点的值，则待查元素肯定在位于左侧序列;

（2）如果待查元素elem大于给定切分点的值，则待查元素肯定在位于右侧序列;

（3）如果待查元素elem等于给定切分点的值，则待查元素已找到，此时可以直接返回下标index。

初始状态下，解空间的大小为序列长度。上述的每一步都把可能的解空间进行了缩减，一直到能够确定对应下标、或者确定待查找元素不存在于给定序列。

3. 查找算法的比较

后续我们会介绍5种查找算法:

（1）二分查找法（版本A）;

（2）斐波那契查找法;

（3）插值查找法;

（4）二分查找法（版本B）;

（5）二分查找法（版本C）。

这5种查找算法在向量长度为n的向量上的渐近时间复杂度均为$\mathcal{O}(\log_2 n)$。 但它们在查找向量里的非重复元素、重复元素、不存在元素时返回的信息稍微有些不同。具体的比较如Tab.I。

TABLE I. 5种查找算法在查找不同类型元素时返回的秩。

	序列中的非重复元素	序列中的重复元素	序列中不存在的元素
二分查找法（版本A）	元素的下标	元素的任意下标	-1
斐波那契查找法	元素的下标	元素的任意下标	-1
插值查找法	元素的下标	元素的任意下标	-1
二分查找法（版本B）	元素的下标	元素的最大下标	-1
二分查找法（版本C）	元素的下标	元素的最大下标	小于此元素的向量元素的最大秩

4. 二分查找法（版本A）

算法流程请阅读邓俊辉教材第2.6.5节（P48–51）。

这里的二分查找法(binary search)也叫折半查找法。选择切分点时，直接选择当前查找范围的中点。

二分查找法（版本A）的切分点和分段的示意图如Figure 5.5所示。

二分查找法（版本A）的参考代码如Figure 5.6所示。

二分查找法（版本A）的执行过程的具体例子如Figure 5.7所示。

这里我们可以计算在二分查找法下序列的平均查找长度。

在示意图Figure 5.8里展示了代码中用来存储查找元素所对应的下标和查找长度的结构体SLE和SLN。它们分别对应查找元素为向量元素、非向量元素时的计算结果。 非向量

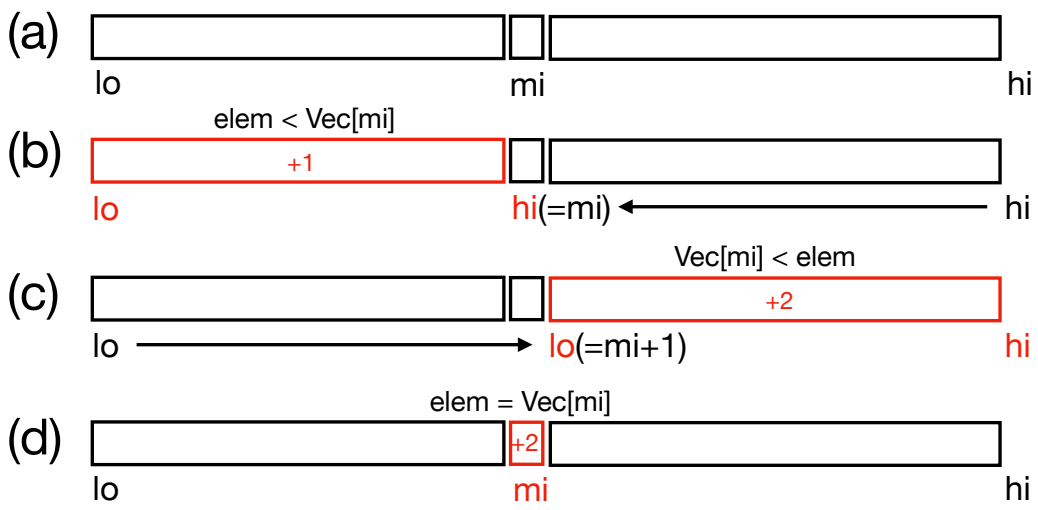


FIG. 5.5. 二分查找法（版本A）中切分点和分段。 假设有序向量为Vec、需要查找元素为elem、查找区域范围为[lo, hi)。 此时切分点下标mi = (lo + hi) /2。此算法的if-else语句把向量分成三个子区域，依次为小于、大于、等于Vec[mi]的区域。 (b)-(d)分别为查找算法进入三个区域的过程，此时需要更新新的查找区域[lo, hi)。方框中的+1和+2分别对应查找长度的增加。

```
//二分查找法（版本A）：给定需要查找的元素elem、查找范围[lo, hi);
int BinarySearchAMethod(int elem, int lo, int hi)
{
  slen_temp = 0;//对于每一个元素，重新初始化查找长度；

  while (lo < hi) {
    int mi = (lo + hi) / 2; //选择切分点，等同于lo + (hi - lo)*0.5;

    if (elem < Vec[mi]) {
      hi = mi;

      slen_temp++;//查找长度+1;
    } else if (Vec[mi] < elem) {
      lo = mi + 1;

      slen_temp += 2;//查找长度+2;
    } else {
      slen_temp += 2;//查找长度+2;

      return mi;
    }
  }

  return -1;
};
```

FIG. 5.6. 二分查找法（版本A）的参考代码。

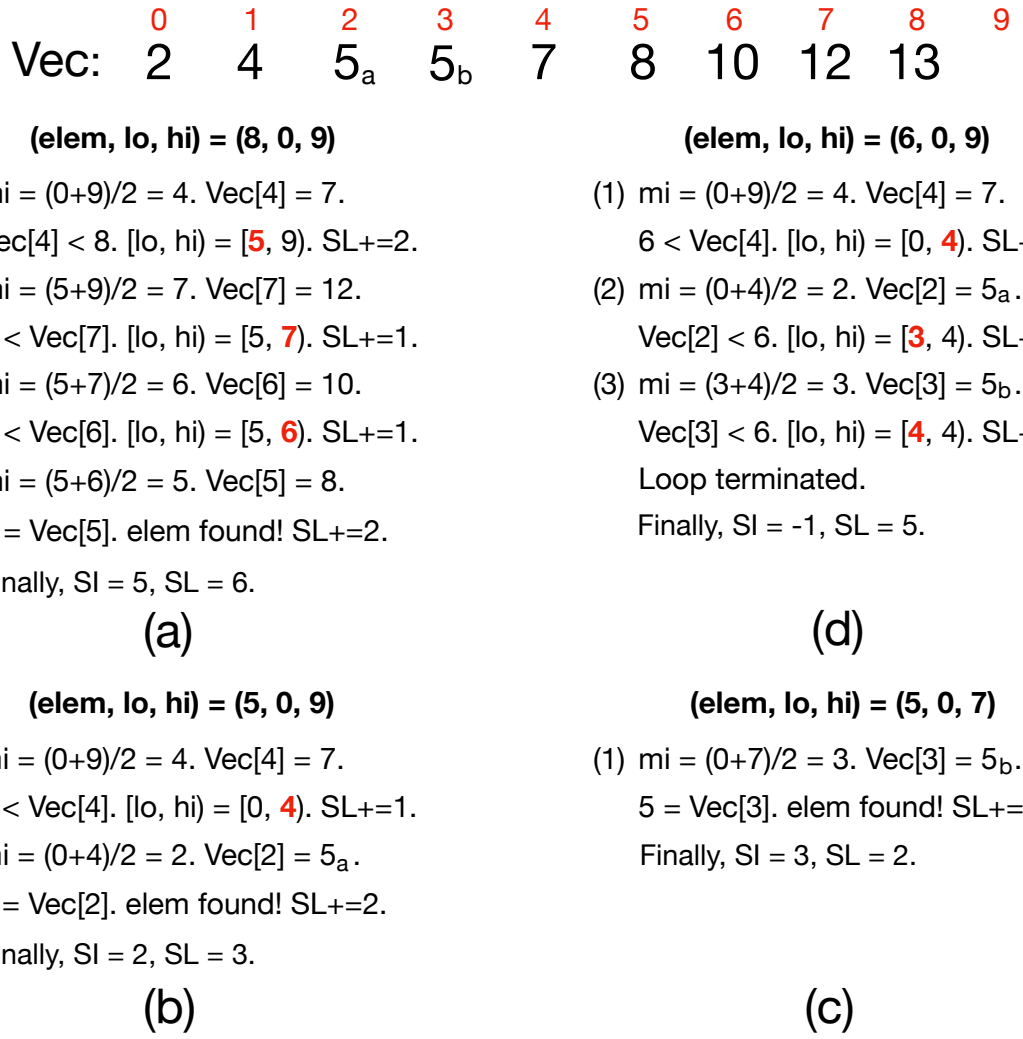


FIG. 5.7. 二分查找法（版本A）的执行过程。 有序向量为Vec，参数(elem, lo, hi)表示算法在查找范围为[lo, hi)内查找元素为elem。 (a)为查找向量已有的、非重复元素的过程。此时elem = 8 = Vec[5]。 (b)为查找向量中重复元素的过程。此时elem = 5 = Vec[2] = Vec[3]。 (c)也为查找向量中重复元素的过程，但使用了不同的[lo, hi)。 (d)为查找向量中不存在元素的过程。此时Vec[3] < elem = 6 < Vec[4]。 最终，(a)中算法返回了元素唯一的秩，(b)和(c)中算法返回了重复元素的两个不同的秩，(d)中算法返回值为−1。 图中标示出来了[lo, hi)和查找长度SL的更新，用红色数字标记了lo或者hi的改变。

元素的选择有很多。对于长度为N的有序向量Vec，我们选择的是Vec[0] - 1; (Vec[$i-1$] + Vec[i])/2, i = 1, 2 $\cdots$, N - 1; Vec[$N-1$] + 1等共$N+1$个元素。

例子6: 整型有序向量上二分查找法（版本A）的查找长度。

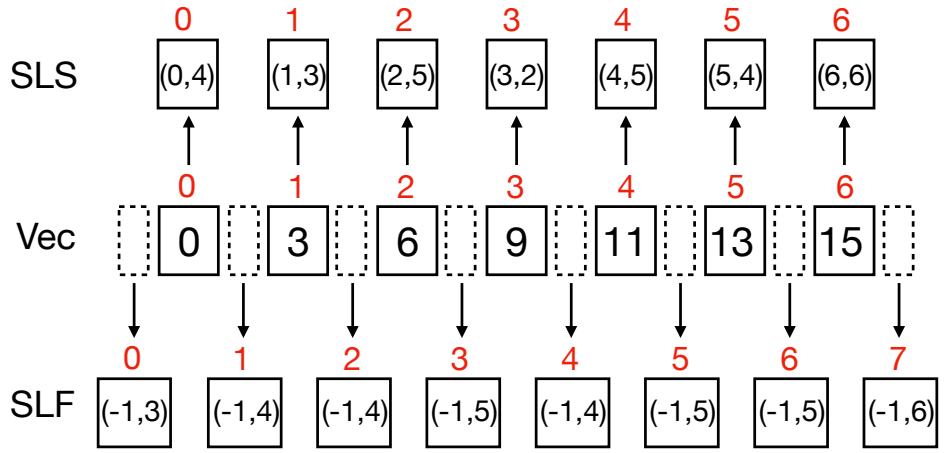


FIG. 5.8. 二分查找法（版本A）代码（`ch5c-search.cc`）中用来计算平均查找长度的结构体。Vec为给定的有序向量，其长度为7。SLS为代码用来存储每个元素作为查找元素后得到的下标和查找长度。SLF为代码用来存储非向量元素作为查找元素后得到的下标和查找长度。图中的−1表示查找元素不存在于向量。SLS的大小即为Vec的大小， SLF的大小比Vec的大小大1。图中方框上的数字表示对应向量的秩/下标。

首先生成一个有序序列，然后计算序列的平均成功查找长度和平均失败查找长度。

参考代码：`ch5c-search.cc`中main函数里的BinarySearchA()。

5. 斐波那契查找法

注：以下内容可参考邓俊辉的课件（2023年秋季学期），第2章P54-55。

这里我们遵循二分查找法（版本A）里在查找给定元素时，把区间分成三段子区间的方式。此外算法通过两个判断语句可以把所查找元素归入所在的子区间：算法进入第一段子区间需要进行一次比较、进入第二三段子区间需要两次比较。 在一个长度为n的有序数组上，我们把第一段子区间的长度比例设为$\lambda \in (0,1)$、第二段子区间的长度比例为$1-\lambda$。下面我们将估计出来查找算法的时间复杂度对于λ的依赖关系。

首先我们把基于分段的迭代算法的时间复杂度$T(n)$写成一个通用公式。 此时$T(n)$对应的是向量元素的查找过程，其意义等同于平均查找长度。 在查找算法里，搜索子区间的长度按λ和$1-\lambda$的倍数缩减到单个元素。 假设搜索深度为t，搜索元素的最深的一次查找过程的深度为$t_{\max}$、最浅的一次查找过程的深度为$t_{\min}$。 当$\lambda > 0.5$时，可知$n\lambda^{t_{\max}} = 1$、$n(1-\lambda)^{t_{\min}} = 1$； 当$\lambda < 0.5$时，可知$n(1-\lambda)^{t_{\max}} = 1$、$n\lambda^{t_{\min}} = 1$。 则查找算法的查找深度可近似写为$t = -\log_{\lambda} n$，并可以继续写为：

$$t = -\log_{\lambda} n = -\frac{\ln n}{\ln \lambda} = -\frac{\ln n}{\ln 2} \cdot \frac{\ln 2}{\ln \lambda} = (-\log_{\lambda} 2) \cdot \log_2 n. \tag{5.4}$$

我们算法计算时间可以写为$T(n) = \alpha(\lambda) \log_2 n$。$\alpha(\lambda)$为算法时间对于切分子区间长度比例的依赖关系。

$T(n)$可以写成以下迭代形式：

$$T(n) = \lambda \times [1 + T(\lambda n)] + (1-\lambda) \times [2 + T((1-\lambda)n)]. \tag{5.5}$$

上式第一项中的λ为算法进入第一段子区间的概率（查找元素为向量元素），方括号里的1表示需要经过1次比较才能进入此子区间、$T(\lambda n)$为此区间在后续算法里的时间复杂度。上式第二项合并考虑轴点本身以及第三段子区间，含义与第一项类似。

对于Eq. (5.5)，我们代入$T(n)$的函数形式，则有

$$\alpha(\lambda) \log_2 n = \lambda[1 + \alpha(\lambda) \log_2(\lambda n)] + (1-\lambda)[2 + \alpha(\lambda) \log_2((1-\lambda)n)]. \tag{5.6}$$

我们将求解以下问题：

（1）简化上述公式以得到$\alpha(\lambda)$的简明表达式，并作图展示$\alpha(\lambda)$随λ变化的曲线。

（2）解析计算$\alpha(\lambda)$的最小值及其对应的λ。此时的λ对应着二分法查找法里划分子区间时的最佳长度比例。

解答：

（1）推导$\alpha(\lambda)$。

斐波那契查找算法的时间复杂度的递推公式为

$$T(n)=\lambda[1+T(\lambda n)]+(1-\lambda)[2+T((1-\lambda)n)]. \tag{5.7}$$

定义$T(n)$的形式为$T(n)=\alpha(\lambda)\cdot\log_2 n$，代入上式后有

$$\begin{aligned}\alpha(\lambda)\cdot\log_2 n&=\lambda[1+\alpha(\lambda)\cdot\log_2\lambda n]+(1-\lambda)[2+\alpha(\lambda)\cdot\log_2(1-\lambda)n]\\&=\lambda+\lambda\alpha(\lambda)\log_2\lambda n+2(1-\lambda)+(1-\lambda)\alpha(\lambda)\log_2(1-\lambda)n\\&=2-\lambda+\lambda\alpha(\lambda)(\log_2\lambda+\log_2 n)+(1-\lambda)\alpha(\lambda)[\log_2(1-\lambda)+\log_2 n]\\&=2-\lambda+\lambda\alpha(\lambda)\log_2\lambda+(1-\lambda)\alpha(\lambda)\log_2(1-\lambda)+\alpha(\lambda)\log_2 n.\end{aligned} \tag{5.8}$$

上式两边删去~~$\alpha(\lambda)\log_2 n$~~，我们有

$$\begin{aligned}0&=2-\lambda+\alpha(\lambda)[\lambda\log_2\lambda+(1-\lambda)\log_2(1-\lambda)]\\&=2-\lambda+\frac{\alpha(\lambda)}{\ln 2}[\lambda\ln\lambda+(1-\lambda)\ln(1-\lambda)].\end{aligned} \tag{5.9}$$

则有

$$\alpha(\lambda)=-\frac{(2-\lambda)\ln 2}{\lambda\ln\lambda+(1-\lambda)\ln(1-\lambda)}. \tag{5.10}$$

函数$\alpha(\lambda)$的变化趋势见Figure 5.9。

（2）求解$\alpha(\lambda)$的极值。

为得到函数的极值，对$\alpha(\lambda)$求导。则有

$$\begin{aligned}\frac{\mathrm{d}\alpha(\lambda)}{\mathrm{d}\lambda}&=-\ln 2\cdot\left\{\frac{-1\cdot[\lambda\ln\lambda+(1-\lambda)\ln(1-\lambda)]-(2-\lambda)\cdot[\ln\lambda+1-\ln(1-\lambda)-1])}{[\lambda\ln\lambda+(1-\lambda)\ln(1-\lambda)]^2}\right\}\\&=\ln 2\cdot\frac{2\ln\lambda-\ln(1-\lambda)}{[\lambda\ln\lambda+(1-\lambda)\ln(1-\lambda)]^2}.\end{aligned} \tag{5.11}$$

假设$\frac{\mathrm{d}\alpha(\lambda)}{\mathrm{d}\lambda}=0$，则有

$$\begin{aligned}2\ln\lambda-\ln(1-\lambda)&=0,\\\lambda^2&=1-\lambda,\\\lambda^2+\lambda-1&=0,\\\lambda&=\frac{-1\pm\sqrt{5}}{2}.\end{aligned} \tag{5.12}$$

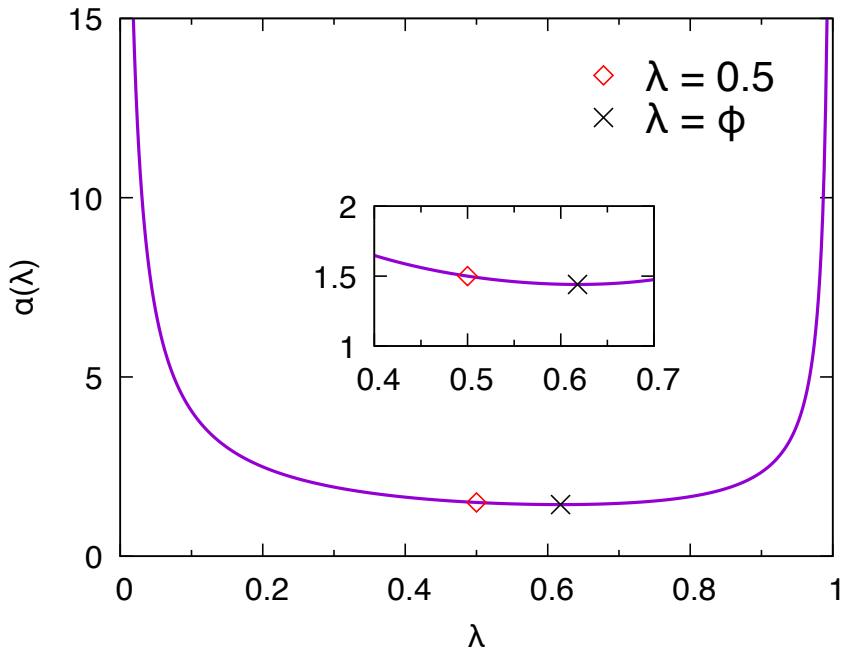


FIG. 5.9. 斐波那契查找法中的函数$\alpha(\lambda)$。 图中还分别标记了$\lambda = 0.5$和$\phi \equiv (\sqrt{5}-1)/2 \approx 0.618$对应的$\alpha(\lambda)$，其中后者对应着$\alpha(\lambda)$的极小值。 子图里更细致地展示了这两个点附近的$\alpha(\lambda)$的值。

因为$\lambda \in (0,1)$, 则唯一极值点$\lambda = (\sqrt{5}-1)/2 \equiv \phi \approx 0.618$。 此时对应

$$\alpha(\phi) \approx 1.44042. \tag{5.13}$$

作为比较，当$\lambda = 1/2$时，容易得知

$$\alpha\left(\frac{1}{2}\right) = -\frac{\frac{3}{2}\ln 2}{\frac{1}{2}\ln\frac{1}{2} + \frac{1}{2}\ln\frac{1}{2}} = 1.5 > \alpha(\phi). \tag{5.14}$$

考虑$\lambda \to 0^+$的情形。容易有

$$\lim_{\lambda\to 0^+} \lambda\ln\lambda = \lim_{\lambda\to 0^+} \frac{\ln\lambda}{1/\lambda} = \lim_{\lambda\to 0^+} \frac{1/\lambda}{-1/\lambda^2} = \lim_{\lambda\to 0^+} -\lambda \to 0^-, \tag{5.15}$$

以及

$$\lim_{\lambda\to 0^+} (1-\lambda)\ln(1-\lambda) \to 0^-. \tag{5.16}$$

综合以上两个式子，我们有

$$\lim_{\lambda\to 0^+} [\lambda\ln\lambda + (1-\lambda)\ln(1-\lambda)] \to 0^-. \tag{5.17}$$

则相应地$\lim_{\lambda\to 0^+}\alpha(\lambda)\to+\infty$。 同理也有$\lim_{\lambda\to 1^-}\alpha(\lambda)\to+\infty$。 即$\alpha(\lambda)$在变量区域的两端都为正无穷大。 此时唯一极值点$\lambda=(\sqrt{5}-1)/2$对应着$\alpha(\lambda)$的最小值。

当二分查找法以$\lambda=\phi$来切分子区间、选择轴点时，对应的查找法为斐波那契查找法。上面我们从理论角度推导出来斐波那契查找法里的黄金分割比例（Golden Ratio）是如何得到的。

二分查找法（版本A）里最优子区间比例不在$\lambda=0.5$的原因： 可以得知，二分查找法（版本A）进入两段子区间时查找长度的增加是不一样的。此外，在理论框架层面，容易得知只有当$\alpha(\lambda)=\alpha(1-\lambda)$时，才会出现$\lambda=0.5$为最优子区间比例。Eq.(5.10)不满足$\alpha(\lambda)=\alpha(1-\lambda)$的原因在于其分子，而其分子来源于Eq.(5.5)的两项里一项贡献1（第一区间对应的查找长度的增加）、一项贡献2（第二三项对应的查找长度的增加），即$2-\lambda=\lambda\cdot 1+(1-\lambda)\cdot 2$。容易得知，如果这两项相等，则$\lambda=0.5$对应着最优子区间比例。

斐波那契查找法的流程： 在类似二分查找法（版本A）的取三段子区间的框架下，如果选择分段大小$\lambda=\phi$可以在很长的向量上得到最小的平均成功查找长度。此时对应着斐波那契查找法（Fibonacci search）。对于这两个查找算法，其切分点的选择如下：

（1）二分查找法（版本A）： mi = lo + (hi - lo) ×0.5;

（2）斐波那契查找法：mi = lo + (hi - lo) ×ϕ ≈ lo + (hi - lo) ×0.618 。

斐波那契查找法的切分点和分段的示意图如Figure 5.10所示。

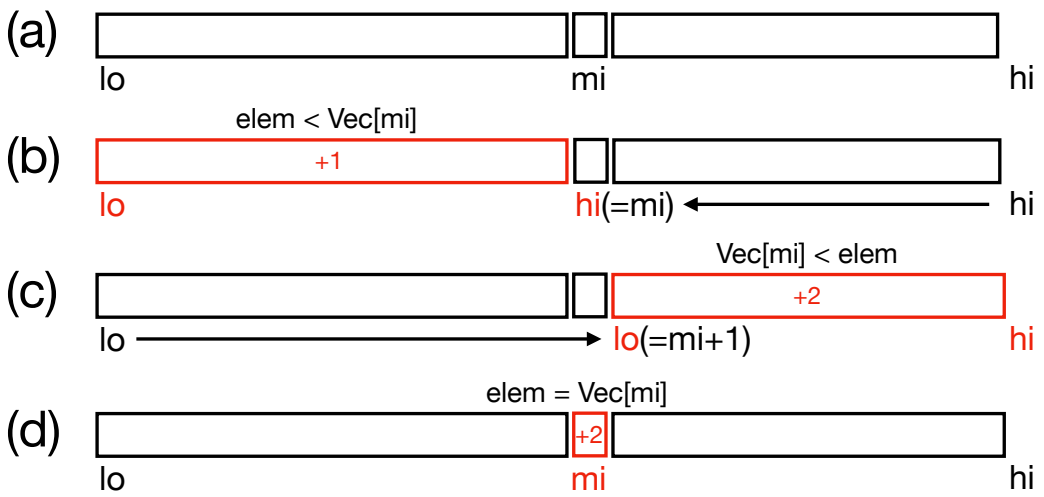


FIG. 5.10. 斐波那契查找法中的切分点和分段。 假设有序向量为Vec、需要查找元素为elem、查找区域范围为[lo, hi)。 此算法和Figure 5.5在子段切分方式上类似。此时mi = lo +(hi - lo) ×ϕ，其中黄金分割比例$\phi\equiv(\sqrt{5}-1)/2\approx 0.618$。

斐波那契查找法的参考代码Figure 5.11所示。

```
//斐波那契查找法：给定需要查找的元素elem、查找范围[lo, hi);
int FibonacciSearchMethod(int elem, int lo, int hi)
{
  slen_temp = 0;//对于每一个元素，重新初始化查找长度；

  while (lo < hi) {
    int mi = lo + static_cast<int>( (hi - lo) * g_ratio );//选择切分点；

    if (elem < Vec[mi]) {
      hi = mi;

      slen_temp++;//查找长度+1;
    } else if (Vec[mi] < elem) {
      lo = mi + 1;

      slen_temp += 2;//查找长度+2;
    } else {
      slen_temp += 2;//查找长度+2;

      return mi;
    }
  }

  return -1;
};
```

FIG. 5.11. 斐波那契查找法里查找函数的参考代码。

斐波那契查找法的执行过程如Figure 5.12所示。 其中，g_ratio赋值为$(\sqrt{5}-1)/2$。

例子7: 整型有序向量上斐波那契查找法的查找长度。

在一个随机生成有序序列上计算其平均成功查找长度和平均失败查找长度。

参考代码：`ch5c-search.cc`中main函数里的FibonacciSearch()。

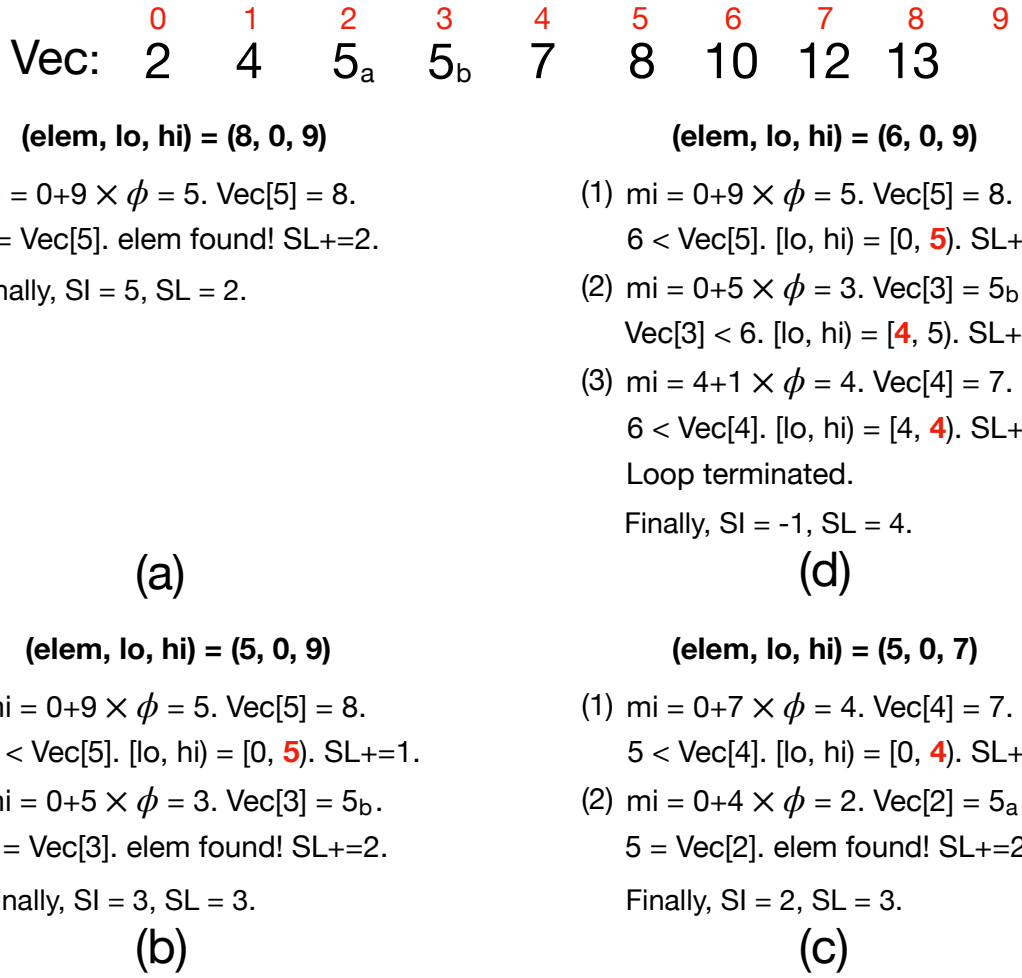


FIG. 5.12. 斐波那契查找法的执行过程。 切分点mi的更新过程使用了mi = lo + (hi - lo) ×ϕ，其中黄金分割比例$\phi = (\sqrt{5}-1)/2$。 最终，如同二分查找法（版本A），(a)中算法返回了元素唯一的秩，(b)和(c)中算法返回了重复元素的两个不同的秩，(d)中算法返回值为-1。

6. 插值查找法

阅读：邓俊辉ppt的P65里有序向量里的插值查找。

插值法（interpolation）是基于已知数据点来估计它们对应的某个函数（即插值函数）、然后利用插值函数来预测其它数据点的方法。它和拟合（fitting）的差别在于：前者需要插值函数经过所有已知数据点，但后者只需要能够近似已有数据点即可。

在查找法里，插值法使用的场景是：如果我们假设给定的有序序列是线性增长的，那么当我们知道序列Vec的两个下标（比如lo和hi）以及相应的元素值（Vec[lo]和Vec[hi]）之后，我们能够很容易得到元素下标和元素值之间的线性插值函数；这样给定查询值elem后，我们可以利用线性插值函数近似估计它对应的下标rank。

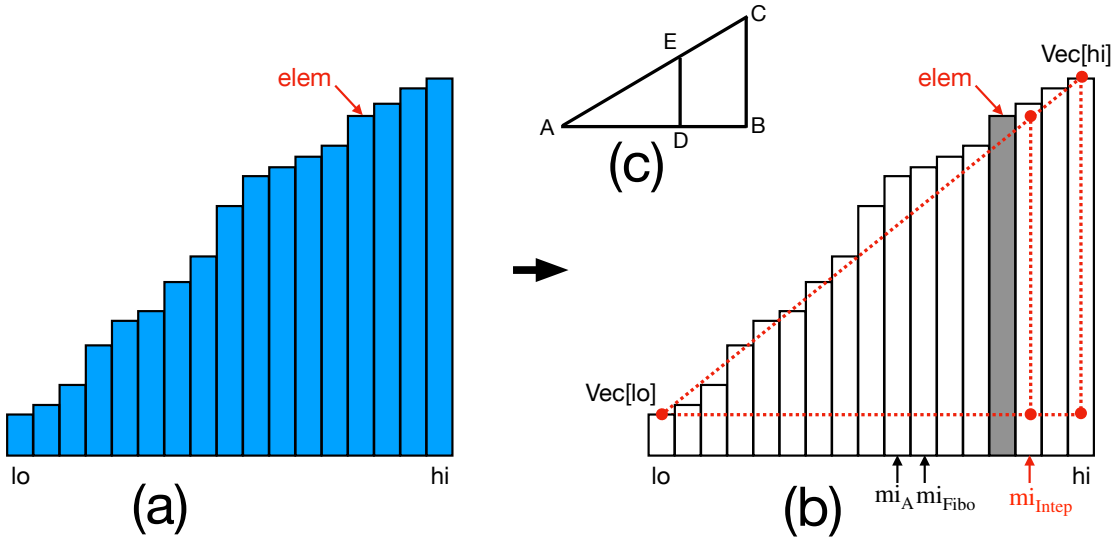


FIG. 5.13. 插值查找法中轴点下标的计算。(a)为给定的有序向量Vec，(elem, lo, hi)为相应的查找参数。注意此时考虑的查找范围为[lo, hi]。(b)基于Vec[lo], Vec[hi], elem的值，可以构造相应的三角形以估计elem大致对应的下标mi_{Interp}。作为对比，二分查找法（版本A）对应的轴点下标mi_A和斐波那契查找法对应的轴点下标mi_{Fibo}也大致标示了出来。(c)展示了(b)里虚线组成的等比三角形。此时三角形的边唱可以通过lo, hi, Vec[lo], Vec[hi], elem来表示。这样给定元素elem、下标lo和hi，可以估计出elem所对应的大致下标来作为查找过程里的轴点下标。

对于使用插值查找法的问题，我们遵循之前讨论过的查找算法的格式：我们假设需要查找的元素为elem，需要查找下的下标/秩的范围为[lo, hi]。注意这里有关下标hi的定义和其它查找法有所不同：这里用[lo, hi]、而不是[lo,hi)，仅仅在于在这里公式的写法会更加自然。在插值查找法里，我们利用Vec[lo]、Vec[hi]、elem的数值的大小：可知，如

果elem的值更接近Vec[hi]，则查找下标现在更接近hi；但如果elem的值更接近Vec[lo]，则查找下标更接近lo。在算法实现上，我们可以假设有序向量Vec按线性增加，则可以估计elem对应的大致下标，以用来作为查找过程的轴点mi。 轴点下标的计算过程可以参考Figure 5.13。 对比Figure 5.13(c)里的等比三角形ΔABC和ΔADE，可得到边长的比例

$$\frac{\mathrm{AD}}{\mathrm{AB}} = \frac{\mathrm{DE}}{\mathrm{BC}}. \tag{5.18}$$

对比Figure 5.13(b)里由Vec[lo], Vec[hi]等构成的、由虚线表示的三角形，可知

$$\frac{\mathrm{mi} - \mathrm{lo}}{\mathrm{hi} - \mathrm{lo}} = \frac{\mathrm{elem} - \mathrm{Vec[lo]}}{\mathrm{Vec[hi]} - \mathrm{Vec[lo]}}. \tag{5.19}$$

这个线性函数其实也就是给定(lo, Vec[lo])和(hi, Vec[hi])这两个数据点之后得到的元素下标和元素值之间的插值函数。 可知则有

$$\mathrm{mi} = \mathrm{lo} + (\mathrm{hi} - \mathrm{lo}) \times \frac{\mathrm{elem} - \mathrm{Vec[lo]}}{\mathrm{Vec[hi]} - \mathrm{Vec[lo]}}. \tag{5.20}$$

如果查找区域按照其它查找算法的习惯那样写成[lo, hi)、而非这里的[lo, hi]的形式，以上公式中的hi应该进行替换hi → hi - 1。相应地，mi的计算式子应该写为：

$$\mathrm{mi} = \mathrm{lo} + (\mathrm{hi} - 1 - \mathrm{lo}) \times \frac{\mathrm{elem} - \mathrm{Vec[lo]}}{\mathrm{Vec[hi} - 1] - \mathrm{Vec[lo]}}. \tag{5.21}$$

我们可以把二分查找法（版本A）、斐波那契查找法、插值查找法放在一个框架里：如果把查找法里轴点下标mi的式子写成mi = lo + (hi - lo) ×λ（λ为第一个子区间的长度比例），可知：

（1）在二分查找法（版本A）里，$\lambda = 0.5$;

（2）在斐波那契查找法里，$\lambda = \phi \approx 0.618$;

（3）在插值查找法里，$\lambda = \frac{\mathrm{elem} - A[\mathrm{lo}]}{A[\mathrm{hi}] - A[\mathrm{lo}]}$.

插值查找法的切分点和分段的示意图如Figure 5.14所示。

插值查找法的参考代码如Figure 5.15所示。

插值查找法的执行过程如Figure 5.16所示。

例子8: 整型有序向量上插值查找法的输出结果。

在给定有序序列上使用插值查找法来寻找给定元素。

参考代码：`ch5c-search.cc`中的main函数里的InterpolationSearch()。

注意在插值查找法里，会有一些其它查找法里不会发生的情形：

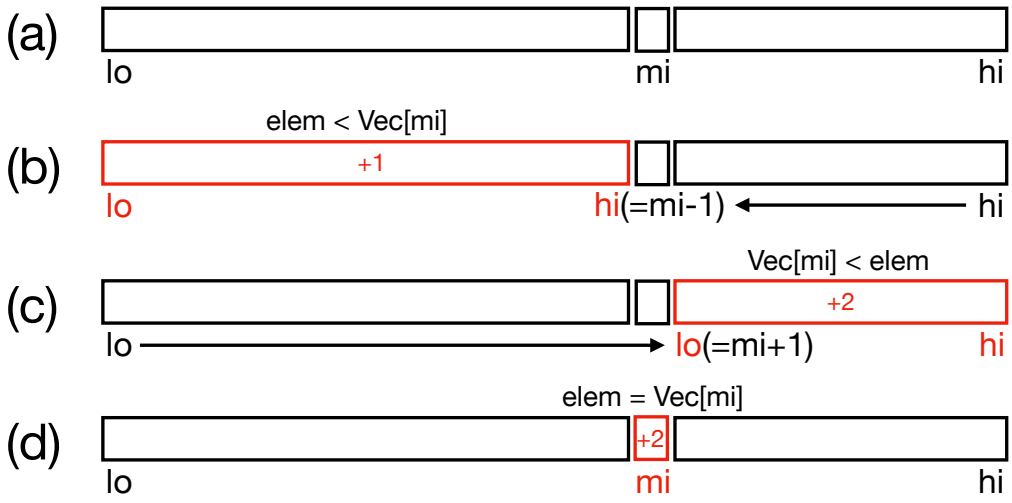


FIG. 5.14. 插值查找法中的切分点和分段。 假设有序向量为Vec、需要查找元素为elem、查找区域范围为[lo, hi]。 此算法和Figure 5.5在子段切分方式上类似。不同之处，首先在于查找元素的范围为[lo, hi]、而非[lo, hi)，其次在于mi的定义不同。

- 在查找循环里需要检查Vec[lo]是否等于Vec[hi]。此时可以直接检查elem是否等于Vec[lo]、并返回查找下标（lo或者-1）。
- 在查找循环里出现mi $<$ lo的情况。原因在于考虑序列元素具体的值时，对应的λ可能< 0。此时对应着Vec[lo - 1] $<$ elem $<$ Vec[lo]的情况，即元素在序列里实际上并不存在。此时可以输出查找下标为-1、并结束循环。

```
//插值查找法：给定需要查找的元素elem、查找范围[lo, hi];
int InterpolationSearchMethod(int elem, int lo, int hi)
{
  slen_temp = 0;//对于每一个元素，重新初始化查找长度；

  while (lo <= hi) {
    double frac = 0;
    int    mi   = 0;

    //处理特殊情况：Vec[lo] = Vec[hi];
    if (Vec[lo] == Vec[hi]) {
      if (elem == Vec[lo]) {
        return lo;
      } else {
        return -1;
      }
    } else {
      //注意lo在求和之后再转换类型，因为lo之后的一项可能为负数；
      frac = 1.0 * (elem - Vec[lo]) / (Vec[hi] - Vec[lo]);
      mi   = static_cast<int>(lo + (hi - lo) * frac);
    }

    //lo <=mi <= hi出现了矛盾；
    if (mi < lo) {
//      cout<<"Exit loop as mi < lo."<<endl;

      break;
    }

    if (elem < Vec[mi]) {
      hi = mi;

      slen_temp++;//查找长度+1;
    } else if (Vec[mi] < elem) {
      lo = mi + 1;

      slen_temp += 2;//查找长度+2;
    } else {
      slen_temp += 2;//查找长度+2;

      return mi;
    }
  }

  return -1;
};
```

FIG. 5.15. 插值查找法里查找函数的参考代码。

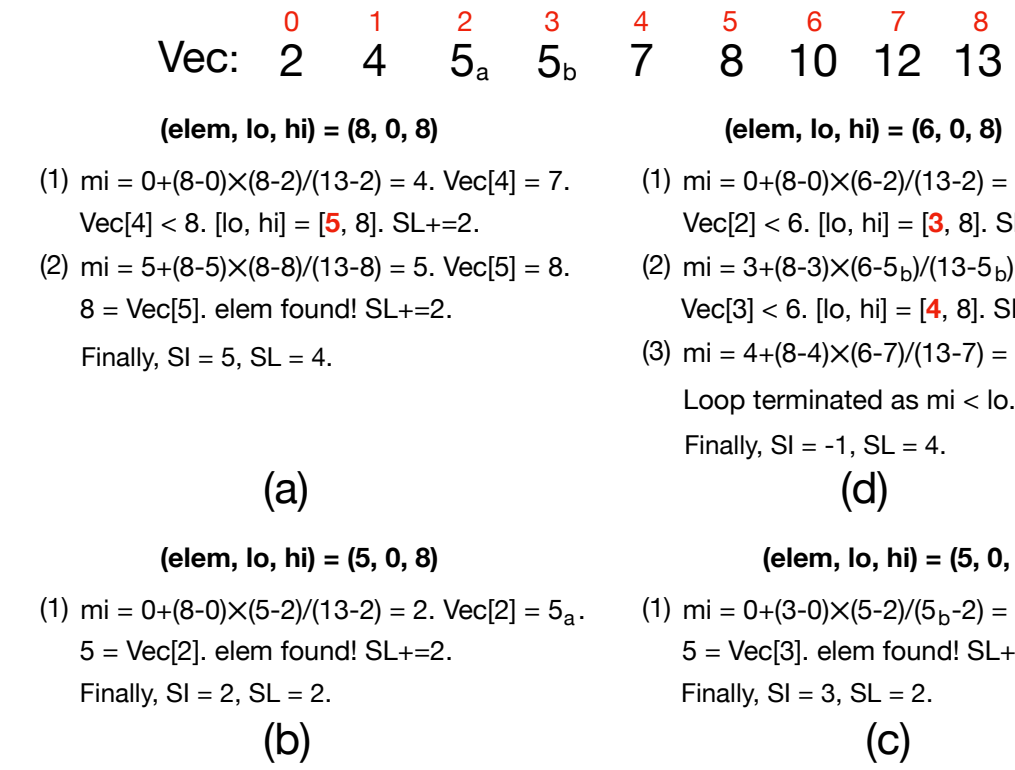


FIG. 5.16. 插值查找法的执行过程。 切分点mi的更新过程使用了Eq.5.20。注意图中的查找范围为[lo, hi)，而非其它查找算法里使用的[lo, hi)。 最终，如同二分查找法（版本A），(a)中算法返回了元素唯一的秩，(b)和(c)中算法返回了重复元素的两个不同的秩，(d)中算法返回值为-1。

7. 二分查找法（版本*B*）

这里我们会讲到二分查找法（版本A）的两个修改版本：二分查找法（版本B）和二分查找法（版本C）。前者能够处理重复元素的问题，也就是查找重复元素时返回重复元素中的最大下标。后者还能够处理查询不存在元素的问题，也就是返回查找元素的前侧序列元素的下标（即前面提到的i）。

在具体的算法实现上，二分查找法（版本B）相比于二分查找法（版本A）：

（1）在版本B里，在考虑子区间时，只考虑两段子区域，此时只需要做一次比较就能判断算法后续进入的子区间。注意，在比较元素的过程中没有判断两个值是否相等的操作。

（2）在版本B里，比较元素的过程可以视为寻找“和查找元素最接近的元素”的过程。只有在算法的最后一步才会直接地来比较这个“最接近查找元素的向量元素”和查找元素本身是否相等。

这里希望大家能够注意到一个非常反直觉的结果：就算待查元素为序列内的元素，我们依然无需使用“判断某个下标对应元素和待查元素是否相等”这个操作步骤来确定待查元素对应的序列下标。

作为例子，对于一个序列：

$$\mathrm{Vec} = \{2,\ 4,\ 5_{\mathrm{a}},\ 5_{\mathrm{b}},\ 5_{\mathrm{c}},\ 7,\ 8,\ 10,\ 12,\ 13\}.$$

如果待查元素为4，则返回下标1。如果待查元素为5，则返回下标4。如果待查元素为6，则返回下标-1。

二分查找法（版本B）的切分点和分段的示意图如Figure 5.17所示。

二分查找法（版本B）的参考代码如Figure 5.18所示。

二分查找法（版本B）的计算过程如Figure 5.19所示。

例子9: 整型有序向量上二分查找法（版本B）的查找长度。

在一个随机生成的有序序列上，计算二分查找法（版本B）对应的平均成功查找长度和平均失败查找长度。

参考代码：`ch5c-search.cc`中main函数里的BinarySearchB()。

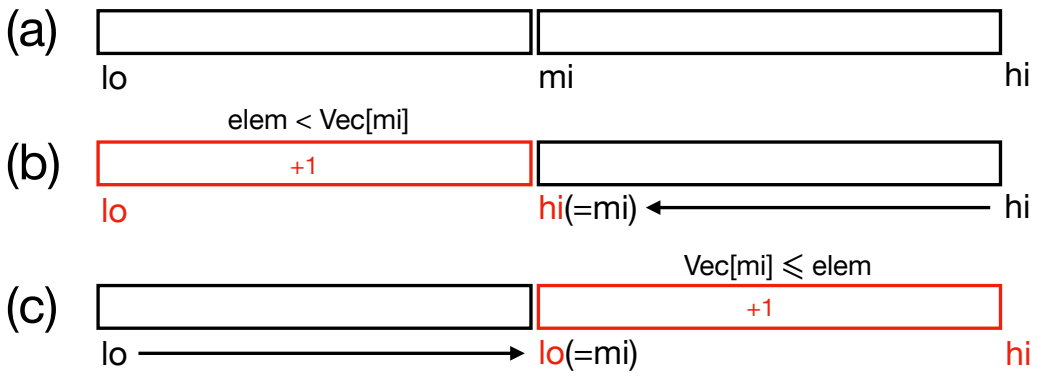


FIG. 5.17. 二分查找法（版本B）中的切分点和分段。 假设查找算法中有序向量为Vec、需要查找元素为elem、查找区域范围为lo和hi。 此算法中，mi = (lo + hi) / 2。但此时只考虑两个子区域[lo, mi)和[mi, hi)，分别表示小于、大于等于Vec[mi]的区域。

```
//二分查找法（版本B）：给定需要查找的元素elem、查找范围[lo, hi);
int BinarySearchBMethod(int elem, int lo, int hi)
{
  slen_temp = 0;//对于每一个元素，重新初始化查找长度；

  while (1 < hi - lo) {
    int mi = (lo + hi) / 2; //选择切分点；

    if (elem < Vec[mi]) {
      hi = mi;

      slen_temp++;//查找长度+1;
    } else {
      lo = mi;

      slen_temp++;//查找长度+1;
    }
  }

  return elem == Vec[lo] ? lo : -1;
};
```

FIG. 5.18. 二分查找法（版本B）里查找函数的参考代码。

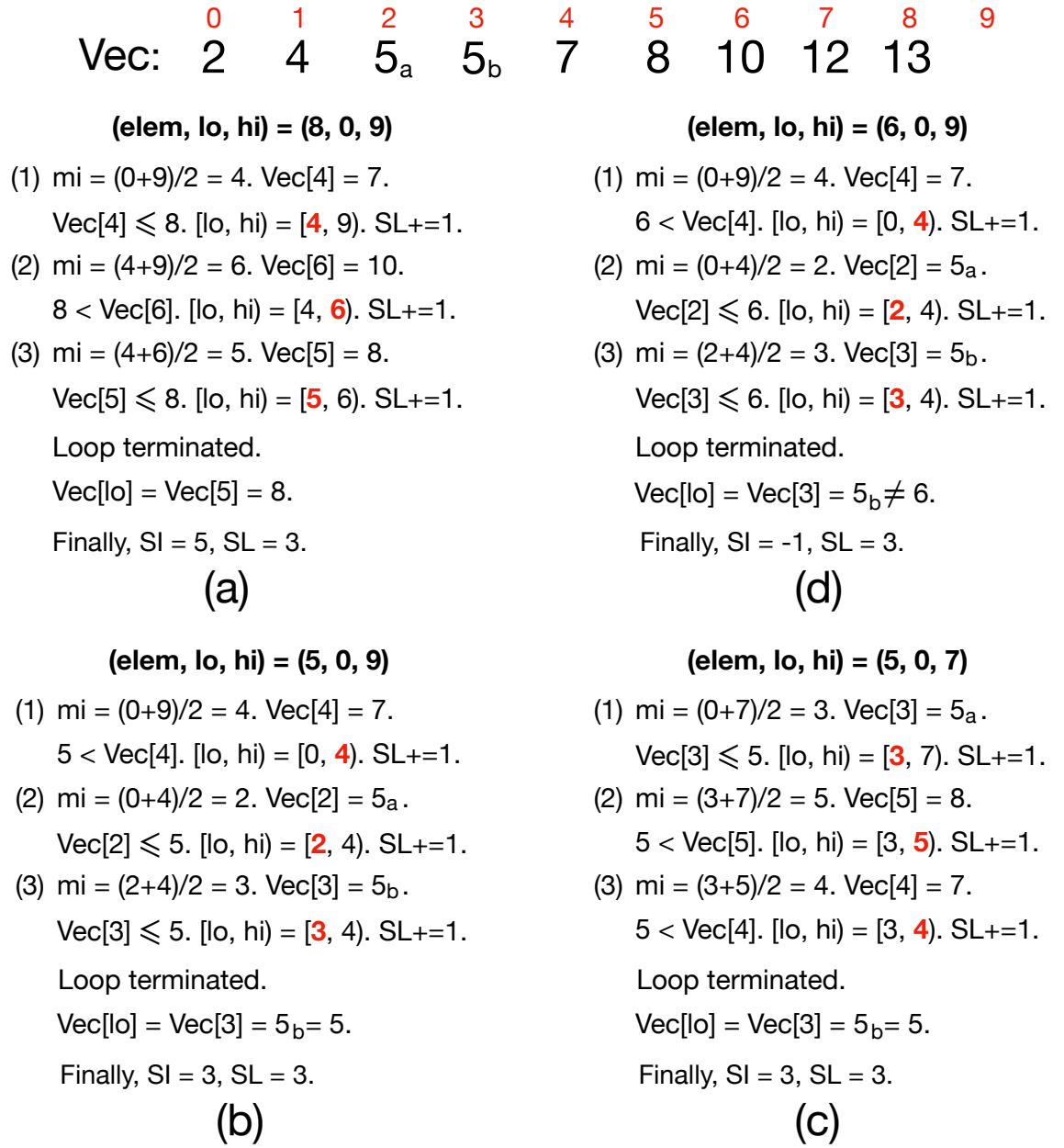


FIG. 5.19. 二分查找法（版本B）的执行过程。(a)中算法返回了元素唯一的秩。(b)和(c)中算法返回了相同的秩、即重复元素中的最大秩。(d)中算法返回值为−1。

8. 二分查找法（版本*C*）

为了处理序列里的重复元素和不存在元素的问题，我们可以做以下具体的规定：

（1）把重复元素作为查找元素时，我们可以设置返回其在序列里的**最大下标**；

（2）当查找序列不存在的元素时，我们可以返回其**小于查找元素的序列元素的最大下标**。

以上两种情况其实可以在一个框架下处理：对于任意查找元素，我们返回序列中**不大于此元素对应的最大下标**。

作为例子，对于一个序列：

$$\mathrm{Vec} = \{2,\ 4,\ 5_{\mathrm{a}},\ 5_{\mathrm{b}},\ 5_{\mathrm{c}},\ 7,\ 8,\ 10,\ 12,\ 13\}.$$

如果待查元素为4，则返回下标1。如果待查元素为5，则返回下标4。如果待查元素为6，则返回下标5。

二分查找法（版本C）的代码可查阅邓俊辉教材第2.6.8小节（P55-56）。

二分查找法（版本C）的切分点和分段的示意图如Figure 5.20所示。

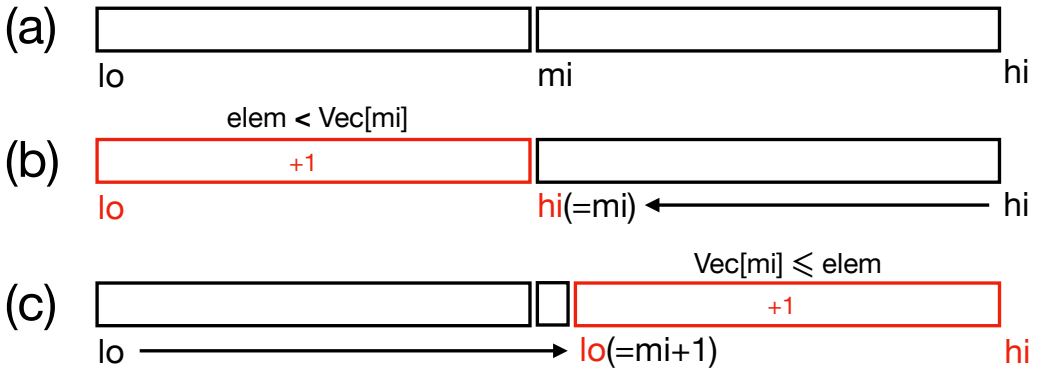


FIG. 5.20. 二分查找法（版本C）中的切分点和分段。假设查找算法中有序向量为Vec、需要查找元素为elem、查找区域范围为lo和hi。此算法在向量的子区间的划分上比较特殊：不同情况下[lo, hi)的并集并不是整段的查找区域。在这个算法里，两个子段分别为[lo, mi)和[mi + 1, hi)。

二分查找法（版本C）的参考代码如Figure 5.21所示。

一个给定向量上的算法运行过程可参考Figure 5.22。

例子10: 整型有序向量上二分查找法（版本C）的输出结果。

在一个随机生成的有序序列上，计算二分查找法（版本C）的平均成功查找长度和平均失败查找长度。

```
//二分查找法（版本C）：给定需要查找的元素elem、查找范围[lo, hi);
int BinarySearchCMethod(int elem, int lo, int hi)
{
  slen_temp = 0;//对于每一个元素，重新初始化查找长度；

  while (lo < hi) {
    int mi = (lo + hi) / 2; //选择切分点；

    if (elem < Vec[mi]) {
      hi = mi;

      slen_temp++;
    } else {
      lo = mi + 1;

      slen_temp++;
    }
  }

  return --lo;
};
```

FIG. 5.21. 二分查找法（版本C）里查找函数的参考代码。

参考代码：`ch5c-search.cc`中的main函数里的BinarySearchC()。

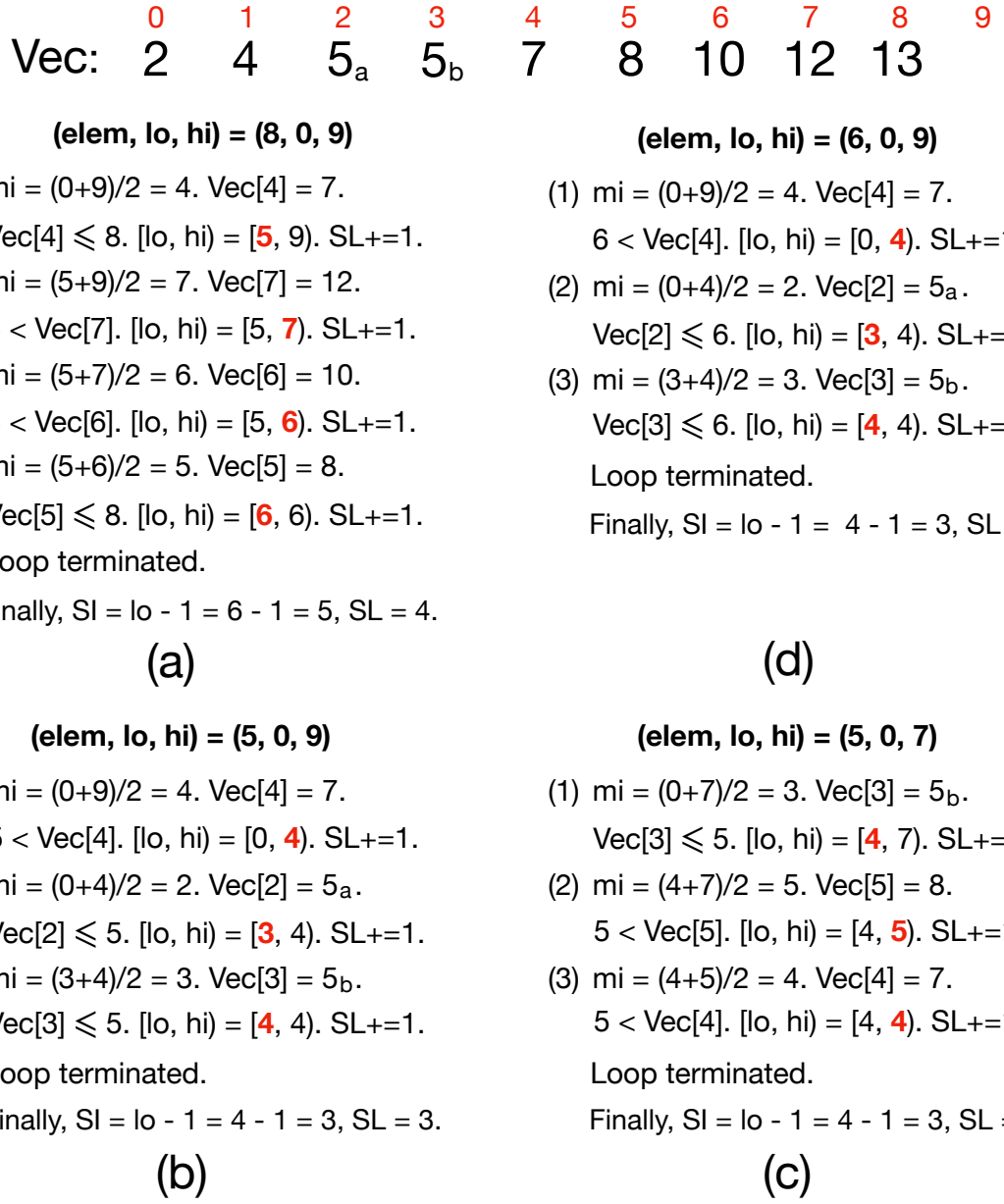


FIG. 5.22. 二分查找法（版本C）的执行过程。 (a)中算法返回了元素唯一的秩。(b)和(c)中算法返回了相同的秩，即重复元素中的最大秩。(d)中算法返回了小于此元素的向量元素的最大秩。

5.6. 排序算法

1. 序列上的排序任务

排序一般用于把任意序列按照非降序的顺序来排列。排序算法的输入为某个序列，返回值为元素重排之后的一个有序序列。序列上的排序任务如Figure 5.23所示。

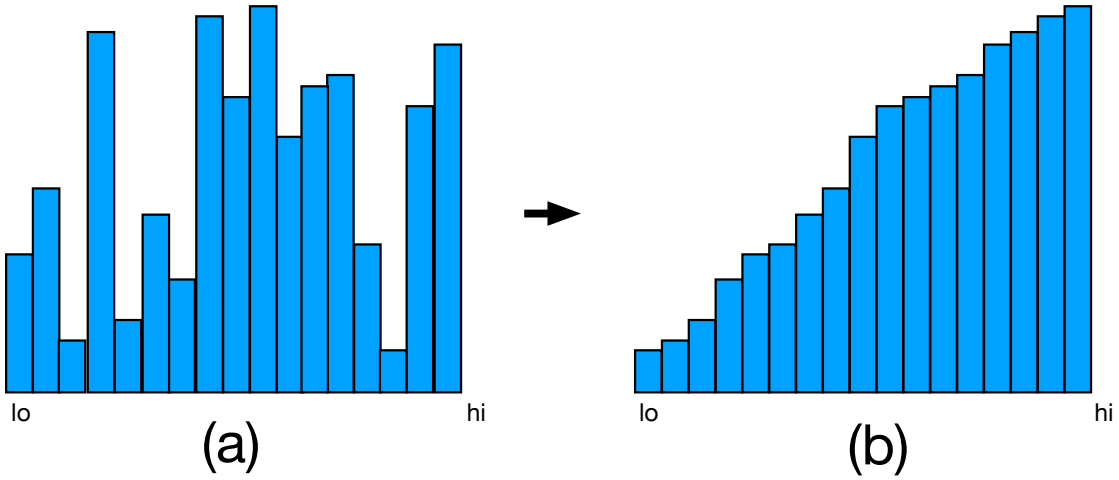


FIG. 5.23. 序列上的排序任务。 (a)给定任意一个序列。其柱状图的长度表示序列元素之间的相对大小。 (b)使用排序算法可以得到相应的有序序列。此时序列为非降序排列。

各个排序算法都有自己的出发点，但都需要处理两个问题：

Q1：算法如何使用基本的操作来降低序列的无序程度?

Q2：算法过程中的基本操作一般会只能保证序列在局域上有序。在全局层面上，算法如何实现序列的有序性?

• 排序算法的**稳定性**

排序算法的稳定性是对于序列里的重复元素来说的：如果排序之后的重复元素的相对顺序在排序之后依然保持，则此排序算法是稳定的，否则次排序算法是不稳定的。

对于一个序列：

$$\text{Vec} = \{7,\ 10,\ 5_{\text{a}},\ 4,\ 13,\ 2,\ 8,\ 5_{\text{b}},\ 12\}. \tag{5.22}$$

5作为重复元素共有2个，其右下标a和b标记了它们的相对顺序。 考虑两个排序算法的结

果，其最终排序结果分别为：

$$\mathrm{Vec1} = \{2,\ 4,\ 5_{\mathrm{a}},\ 5_{\mathrm{b}},\ 7,\ 8,\ 10,\ 12,\ 13\}. \tag{5.23}$$

$$\mathrm{Vec2} = \{2,\ 4,\ 5_{\mathrm{b}},\ 5_{\mathrm{a}},\ 7,\ 8,\ 10,\ 12,\ 13\}. \tag{5.24}$$

可知，两个序列都是有序序列，但前者序列保持了元素5的相对顺序，对应排序算法是稳定的；后者序列没有保持元素5的相对顺序，对应排序算法是不稳定的。

2. 排序算法的比较

本章里讨论的5种排序算法的比较如Tab.II。

TABLE II. 5种排序算法的复杂度和特性。

排序算法	时间复杂度			空间复杂度	稳定性
	最好情况	最坏情况	平均情况		
插入排序	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	稳定
选择排序	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	不稳定
冒泡排序	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	稳定
归并排序	$\mathcal{O}(n\log_2 n)$	$\mathcal{O}(n\log_2 n)$	$\mathcal{O}(n\log_2 n)$	$\mathcal{O}(n)$	稳定
快速排序	$\mathcal{O}(n\log_2 n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n\log_2 n)$	$\mathcal{O}(\log_2 n)$	不稳定

3. 插入排序

■ 算法的基本原理

在最简单的插入排序算法里，序列被分成了前后两个部分：靠前的已排序区和靠后的待排序区。前者包含了已确定排好序的序列部分，后者包括可能经过了一些操作、但总体上没确定是否排好序的序列部分。 在算法的初始部分，已排序区为空集，未排序区包含了全部序列元素。 之后算法从序列的第一个元素开始遍历所有元素。在遍历时，把当前元素作为待排序区的首元素放入已排序区内的合适位置。考虑在某个序列Vec使用插入排序算法。算法在寻找已排序区内的位置时，首先把元素Vec[i]取出来作为待排序元素elem，然后把已排序区的元素（即Vec[j], $\vee j < i$）按照倒序的顺序逐个和elem进行比较：

（1）如果Vec[j] > elem，则把Vec[j]移动到Vec[$j+1$]；

（2）如果Vec[j] $\leqslant$ elem，记录此时的下标为k。退出遍历元素的循环过程，并把elem放入Vec[$k+1$]。此时单次操作结束。

在算法结束时，已排序区包含了全部序列元素，待排序区的部分为空。此时得到的序列就是已经排好序的序列。

上述过程类似给洗牌后的扑克牌重新排序的过程。想象下，你手头有两叠扑克牌：一叠是排好序的扑克牌，一叠为打乱的扑克牌。一个简单直接的顺牌的方式，就是从打乱的扑克牌里拿出最上面的一张，在已排好序的牌里找到相应的位置，然后放进去。如此操作，直到把所有的打乱的扑克牌一张一张地插入已排序的那一叠。

插入排序算法的基本思路如Figure 5.24所示。

■ 算法的代码实现

插入排序过程的参考代码如Figure 5.25所示。 注意代码中，j的取值可以为-1。对应未排序区首元素在已排序区的下标为0，即首元素的位置。此时对应着待排序区的首元素比已排序区的所有元素都小。

插入排序算法的计算流程如Figure 5.26所示。

例子11: 随机序列上的插入排序算法

在一个随机生成的整型序列上，使用插入排序算法进行排序。

参考代码：`ch5e-sort.cc`中的main函数里的InsertionSort()。

■ 算法的稳定性

此时我们可以判断算法是否稳定。 注意到算法中判断是否移动已排序区中元素时所使用的关系运算符是>和$\leqslant$。 对于某个待排序元素elem，如果已排序区中的某个元素满足Vec[j] = elem，可知elem最终被放置在$j+1$的位置。也就是说序列中重复元素的相对顺序得到了保持。可知插入排序算法是稳定的。

你可以继续思考下，如果算法中判断是否移动已排序区中元素时所使用的关系运算符是$\geqslant$和<，此时的算法是否是能够实现排序？如果它依然是个排序算法，那么它是否依然是稳定的?

■ 算法的复杂度分析

插入排序算法中的基本操作是元素之间的的比较和元素的移动。为了分析算法的时间复杂度，我们分别计算不同结构序列情形下两者的操作次数。此时考虑序列的长度为n。

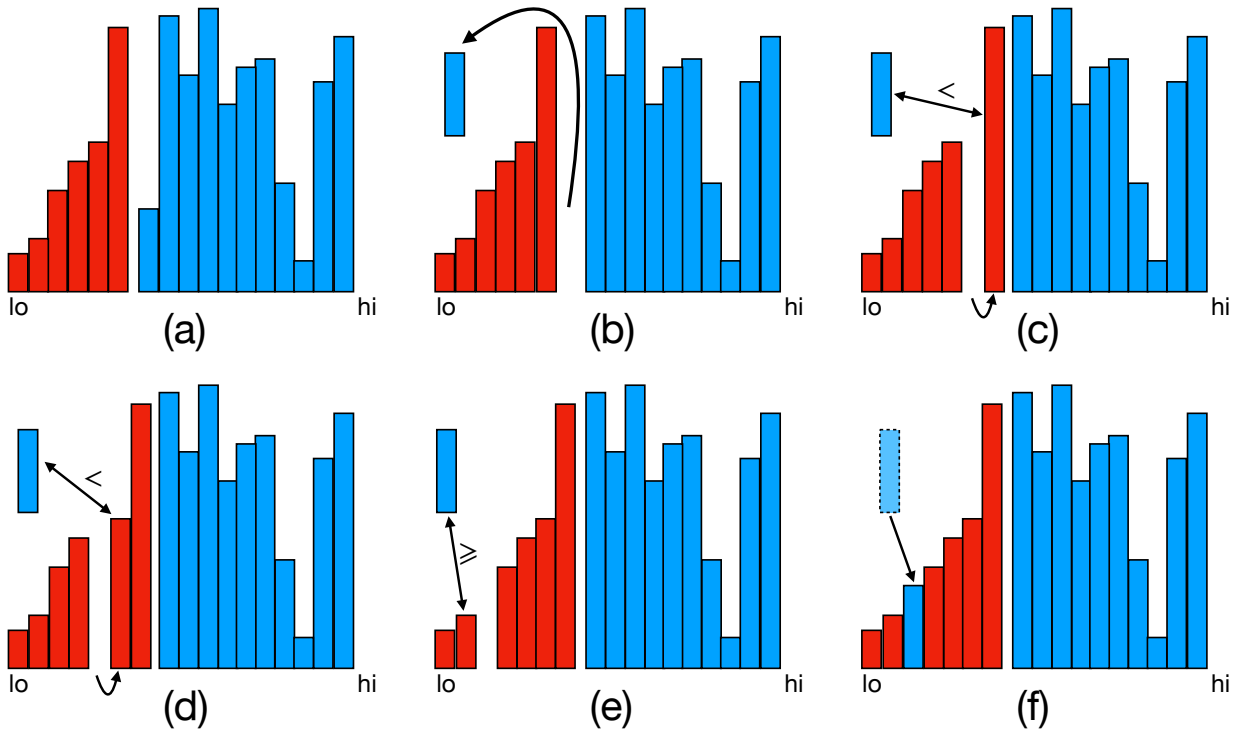


FIG. 5.24. 插入排序算法中的基本操作步骤。 图中需要排序的序列为Vec。条状表示序列元素之间的相对大小。 蓝色条状和红色条状分别表示序列中的待排序区和已排序区。 (a)为排序算法里一次基本操作前的初始状态。 (b)算法把待排序区的首元素取出来作为待排序元素。 (c)和(d)为算法中移动元素的两次操作。算法从已排序区的后端往前端遍历。如果相应元素比待排序元素大，则相应地把元素往后挪动一位，即下标从j变化为$j+1$。 (e)中算法在已排序区找到不大于待排序元素的元素的位置，比如下标k。 (f)中算法把待排序元素放置在(e)中相应位置的后一位，即下标$k+1$。 此步骤完成后，之前的已排序区以及放入合适位置的待排序元素组成了新的已排序区。

当时序列为有序时，待排序区内的元素只需和已排序区的末尾元素比较一次，而且没有移动元素的操作。此时对应的元素的比较次数和移动次数均为最少。两个操作次数分别记为$C_{\min}$和$M_{\min}$。可知:

$$C_{\min} = \sum_{i=0}^{n-1} 1 = n, \tag{5.25}$$

$$M_{\min} = 0. \tag{5.26}$$

当序列为倒序时，待排序区内的元素需要和已排序区的所有元素依次比较一次，同时需要把已排序区内所有的元素依次往后移动一位。此时对应的元素的比较次数和移动次数为最多。 对于待排序区的元素Vec$[i](i \geqslant 0)$，可知它需要和Vec[0]到Vec$[i-1]$的所有元素

```
void InsertionSortMethod_v1(int lo, int hi)
{
  //从序列范围内的第1个元素开始，依次把它们放入正确的位置；
  for (int i = lo; i < hi; i++) {
    //取出待排序元素；
    int elem = Vec[i];

    //寻找待排序元素在已排序部分的下标；
    int j = i - 1;

    for (; j >= lo; j--) {
      if (Vec[j] > elem) {
        Vec[j + 1] = Vec[j];
      } else {
        break;
      }
    }

    //重新放入待排序元素；
    Vec[j + 1] = elem;
  }
};
```

FIG. 5.25. 插入排序算法里排序函数的参考代码。 此函数实现了向量Vec上下标在范围[lo, hi)内元素的排序。

进行比较，且比较次数为i次。此时已排序区内元素的移动次数也为i。此外，还需要计入记录和重新放入当前待排序元素的两次操作。可知：

$$C_{\min} = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \mathcal{O}(n^2), \tag{5.27}$$

$$M_{\min} = \sum_{i=0}^{n-1} (i+2) = \frac{n(n+3)}{2} = \mathcal{O}(n^2). \tag{5.28}$$

当序列为随机序列时，元素的比较和移动次数处在平均情况。对于元素Vec[i]，其对应的元素比较次数为$i/2$，元素移动的次数为$i/2+2$。可知：

$$C_{\min} = \sum_{i=0}^{n-1} \frac{i}{2} = \frac{n(n-1)}{4} = \mathcal{O}(n^2), \tag{5.29}$$

$$M_{\min} = \sum_{i=0}^{n-1} \left(\frac{i}{2} + 2\right) = \frac{n(n+7)}{4} = \mathcal{O}(n^2). \tag{5.30}$$

在算法的空间复杂度上，可知算法所需的中间变量只有遍历序列时的两个下标（i和j）以及当下的待排序元素（elem）。它们的大小是常数级的，和序列大小无关。可知插入排序算法是一个就地算法。

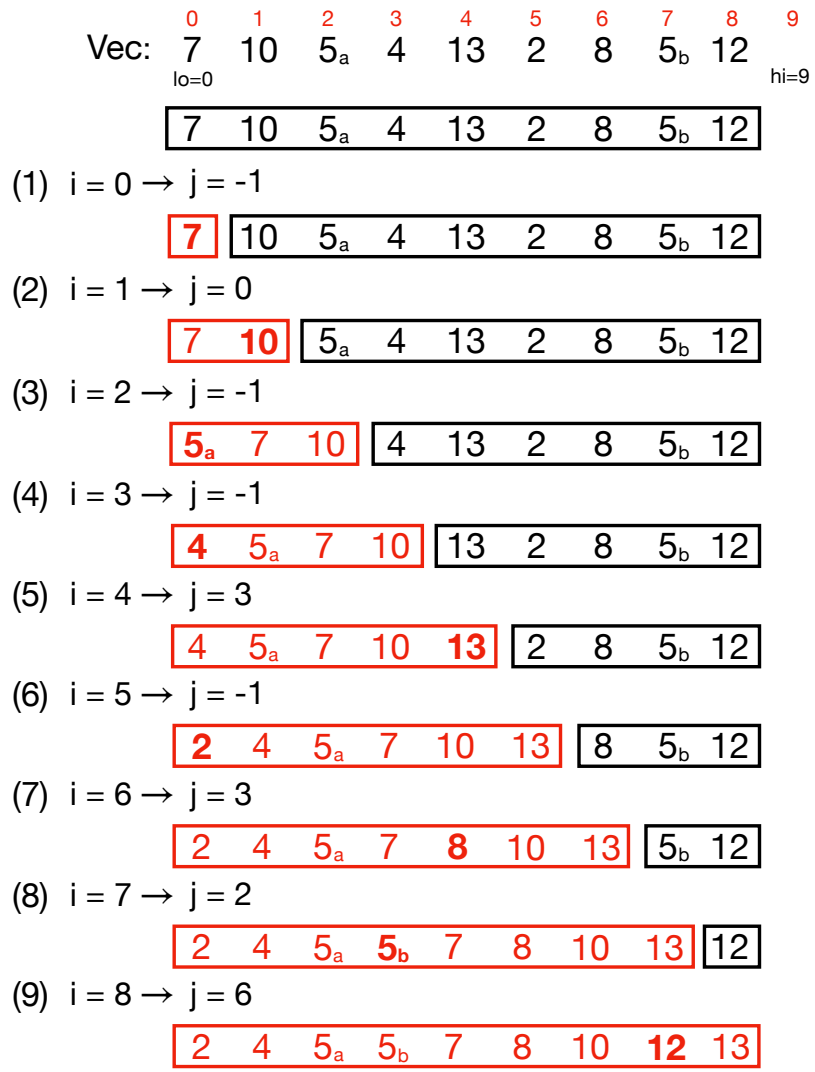


FIG. 5.26. 插入排序算法的执行过程。 待排序的序列为Vec $= \{7, 10, 5_a, 4, 13, 2, 8, 5_b, 12\}$。 图中黑色实线方框和红色实线方框分别标记了序列的待排序区和已排序区。 算法里的循环过程共9个步骤。在每一步骤，给定下标i后都会得到对应的位置j。之后待排序元素被放入下标$j+1$的位置。此时的待排序元素用粗体标记。 循环操作全部结束之后，初始序列最终成为有序序列。

4. 选择排序

■ 算法的基本原理

在选择排序算法里，序列依然分成两个部分：靠前的序列部分为已排序区，靠后的为待排序区。 在算法的开始阶段，已排序区为空集，待排序区包含了全部序列。 算法需要依次遍历序列的全部元素。 对于待排序区，首先寻找待排序区内的最小元素，将其和待排序区的首元素互换。已排序区和和待排序区的首元素合并成为新的已排序区。此时已排序区增加了一个元素，待排序区相应地减少了一个元素。容易得知，在算法的最后一步，待排序区内只有一个元素，此元素不小于已排序区的所有元素。此时可以把此元素合并入已排序区。此时序列的已排序区包括全部元素、待排序区为空。此时得到的序列就是最终的有序序列。

选择排序算法的上述思路如Figure 5.27所示。

■ 算法的代码实现

选择排序过程的参考代码如Figure 5.28所示。

选择排序算法的计算流程如Figure 5.29所示。

例子12: 随机序列上的选择排序算法

在一个随机生成的整型序列上，使用选择排序算法进行排序。

参考代码：`ch5e-sort.cc`中的main函数里的SelectionSort()。

■ 算法的稳定性

选择排序算法的稳定性依赖于寻找最小值的函数是如何实现的。 想象下，待排序区内有两个值相等的元素，而且两者均为待排序区的最小值。 在待排序区内寻找最小值的操作步骤里，只有当序列元素**小于**最小值临时变量时，才会发生最小值对应下标的替换。也就是说，这些重复的最小值在寻找待排序区的最小值的操作里，会依次成为当前未排序区的最小值，并合并到已排序区内。这个过程里，重复元素的相对顺序是不变的，也就是说选择排序算法是稳定的。

■ 算法的复杂度

最好情况下，算法的时间复杂度为$\mathcal{O}(n^2)$。 最坏情况下，算法的时间复杂度为$\mathcal{O}(n^2)$。平均情况下，算法的时间复杂度为$\mathcal{O}(n^2)$。

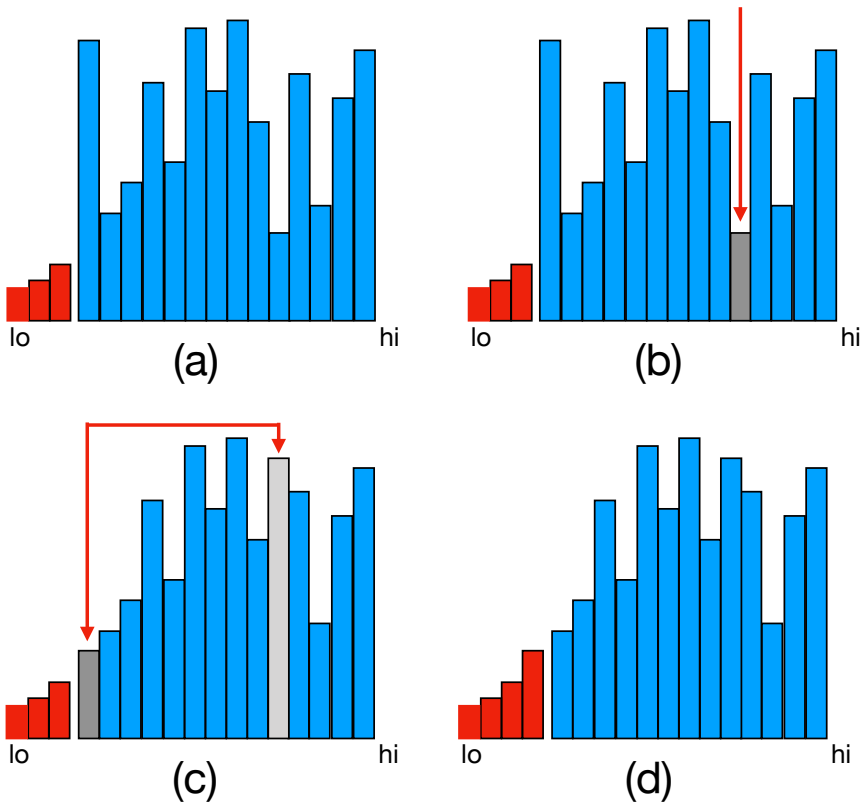


FIG. 5.27. 选择排序算法中的基本操作步骤。 图中需要排序的序列为Vec。条状表示序列元素之间的相对大小。 蓝色条状和红色条状分别表示序列中的待排序区和已排序区。 (a)为排序算法里一次基本操作前的初始状态，其中已排序区靠前、待排序区靠后。 (b)中算法在待排序区内寻找到最小值。此最小值用深色阴影色块和红色尖头来标记。 (c)中算法把(b)中的最小值和待排序区的首元素（用浅色阴影色块标记）进行了互换。此时待排序区的最小值移动到了待排序区的首位。 (d)待排序区的首元素和已排序区合并，成为了新的已排序区。

```
void SelectionSortMethod(int lo, int hi)
{
  for (int i = lo; i < hi; i++) {
    int mini = i;

    for (int j = i + 1; j < hi; j++) {
      if (Vec[j] < Vec[mini]) {
        mini = j;
      }
    }

    if (mini != i) {
      swap(Vec[mini], Vec[i]);
    }
  }
};
```

FIG. 5.28. 选择排序算法里排序函数的参考代码。 此函数实现了向量Vec上下标的范围[lo, hi)内元素的排序。

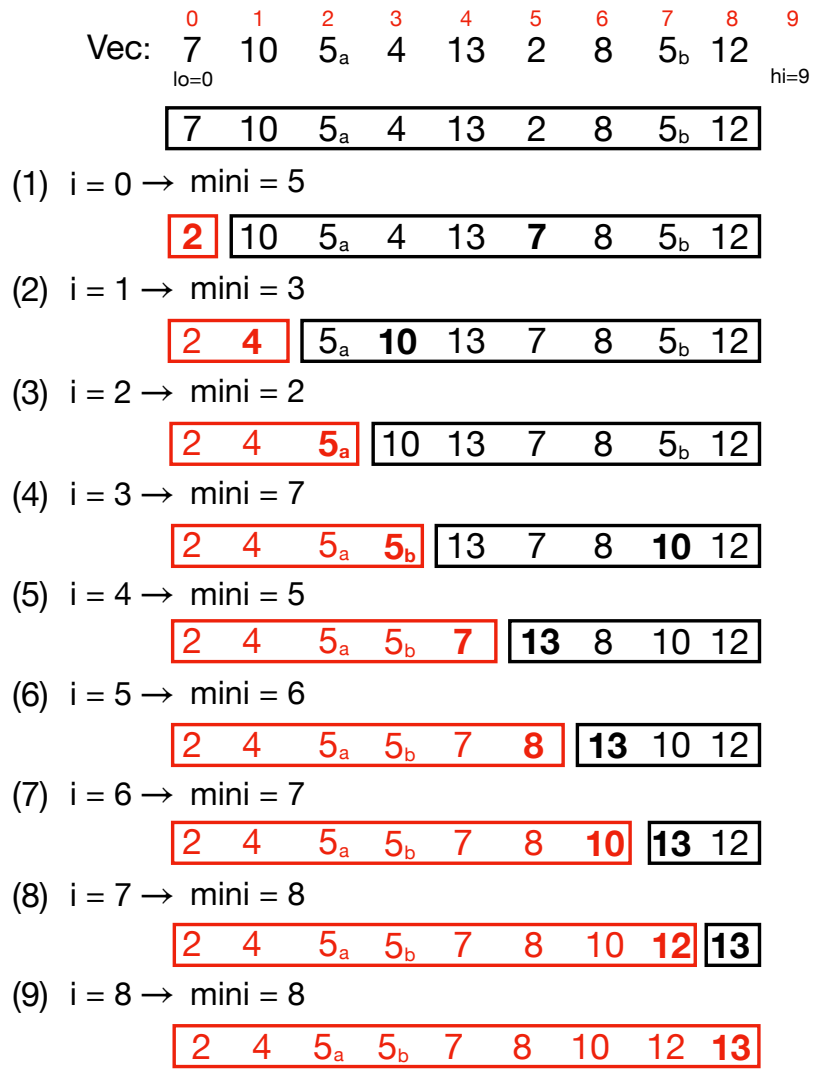


FIG. 5.29. 选择排序算法的执行过程。 待排序的序列为Vec $= \{7, 10, 5_a, 4, 13, 2, 8, 5_b, 12\}$。 图中黑色实线方框和红色实线方框分别标记了序列中的待排序区和已排序区的元素。 算法中的9个步骤对应着连续处理序列的9个元素。 在每一步骤，给定下标i后都会得到待排序区内最小值对应的下标mini。之后mini对应的元素和待排序区的首元素互换。 互换的两个元素用粗体标记。 循环操作结束之后，初始序列最终成为有序序列。

5. 冒泡排序

■ 算法的基本原理

冒泡排序算法或起泡排序算法(Bubblesort)同样把序列分成两部分：靠前的为待排序区、靠后的为已排序区。 排序算法对待排序区的元素进行多次遍历。从左到右的遍历过程中，需要判断相邻元素是否构成逆序对。如果出现逆序对，则把两个元素进行交换。不断交换相邻元素的逆序对能够降低待排序区的无序程度。此外，一个附带的结果是当前待排序区内最大的元素被移动到最右侧的位置（对应最大的下标）。

冒泡排序的基本思路如Figure 5.30所示。

沿着同样的思路，在遍历序列时，我们可以从右侧往左侧进行、并交换相邻元素的逆序对。这样的操作方式同样能够降低序列的无序度，但会把当前序列的最小元素移动到序列最左侧的位置（对应最小的下标）。这个流程类似一个反向的冒泡排序算法，同样可以实现序列的排序。

■ 算法的代码实现

冒泡排序过程的参考代码如Figure 5.31所示。

冒泡排序算法的计算流程如Figure 5.32所示。

例子13: 随机序列上的冒泡排序算法

在一个随机生成的整型序列上，使用冒泡排序算法进行排序。

参考代码：`ch5e-sort.cc`中的main函数里的BubbleSort()。

■ 算法的稳定性

冒泡排序算法的稳定性依赖于相邻逆序对的定义和交换。 在冒泡排序算法里，序列Vec上的两个相邻元素Vec$[i]$和Vec$[i+1]$ $(i \geqslant 0)$满足Vec$[i] >$ Vec$[i+1]$时，两者才构成相邻的逆序对。也就是说当Vec$[i] =$ Vec$[i+1]$时，两者并不构成逆序对，因此无需进行互换。当未排序区内有多个重复元素时，可知它们彼此之间都不构成逆序对，因而都不会进行元素的互换。最终，它们之间的相对顺序不会改变。由此可知，冒泡排序算法为稳定的。

考虑以下问题：如果两个相邻元素Vec$[i]$和Vec$[i+1]$ $(i \geqslant 0)$满足Vec$[i] \geqslant$ Vec$[i+1]$时，两个元素就进行交换。那么此时的算法是否还能实现排序? 如果此算法能够实现排序，那么它是否是稳定的?

■ 算法的复杂度

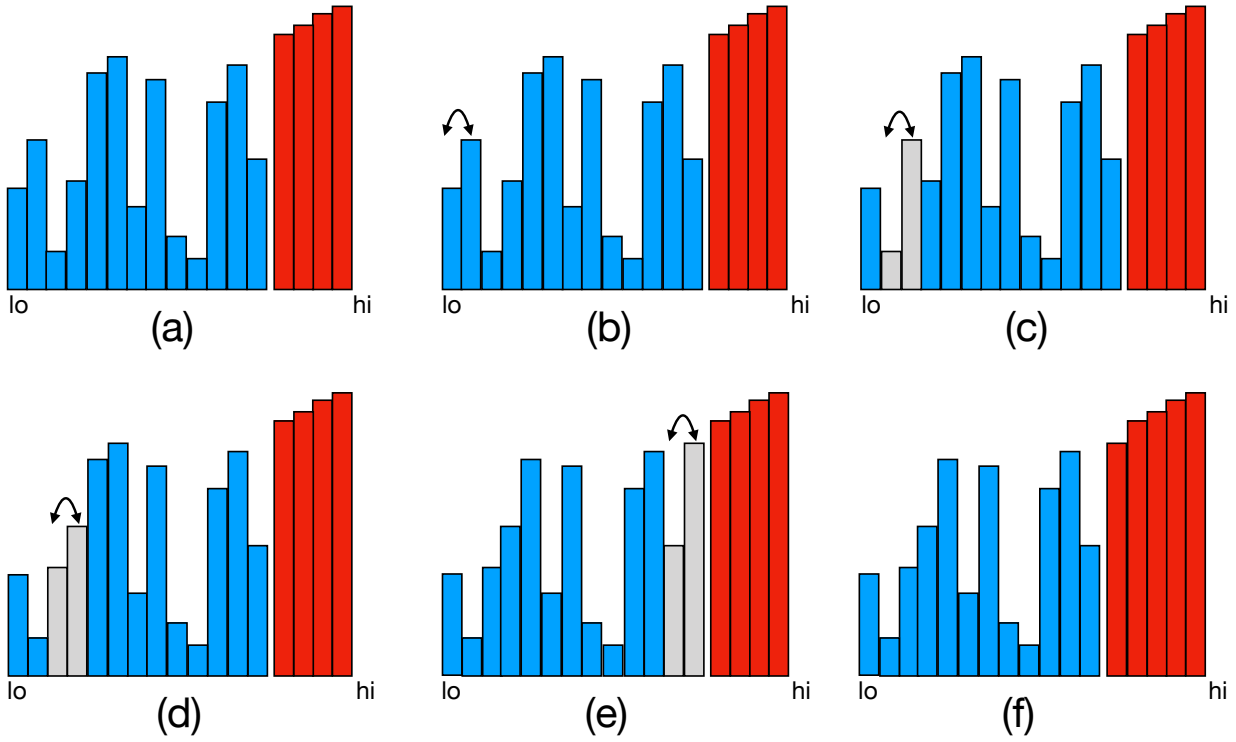


FIG. 5.30. 冒泡排序算法中的基本操作。 图中需要排序的序列为Vec。条状表示序列元素之间的相对大小。 蓝色条状和红色条状分别表示序列里的待排序区和已排序区。 (a)为排序算法里一次基本操作前的初始状态。 (b-e)表示算法中交换逆序对的第一次、第二次、第三次、以及最后一次操作。 在这些操作步骤中，算法从序列的前端往后遍历。如果相邻元素组成逆序对，则把它们进行交换。 逆序对交换后的结果用灰色条状标记。 (f)为算法遍历一次待排序区后的结果。此时待排序区内的最大元素被挪到了末尾，并和已排序区重新组成新的已排序区。

```
//使用起泡排序函数来排序Vec[lo, hi);
void BubbleSortMethod(int lo, int hi)
{
  while (lo < --hi) {
    for (int i = lo; i < hi; i++) {
      if (Vec[i] > Vec[i + 1]) {
        swap(Vec[i], Vec[i + 1]);
      }
    }
  }
};
```

FIG. 5.31. 冒泡排序算法里排序函数的参考代码。 此函数实现了向量Vec上下标的范围[lo, hi)内元素的排序。

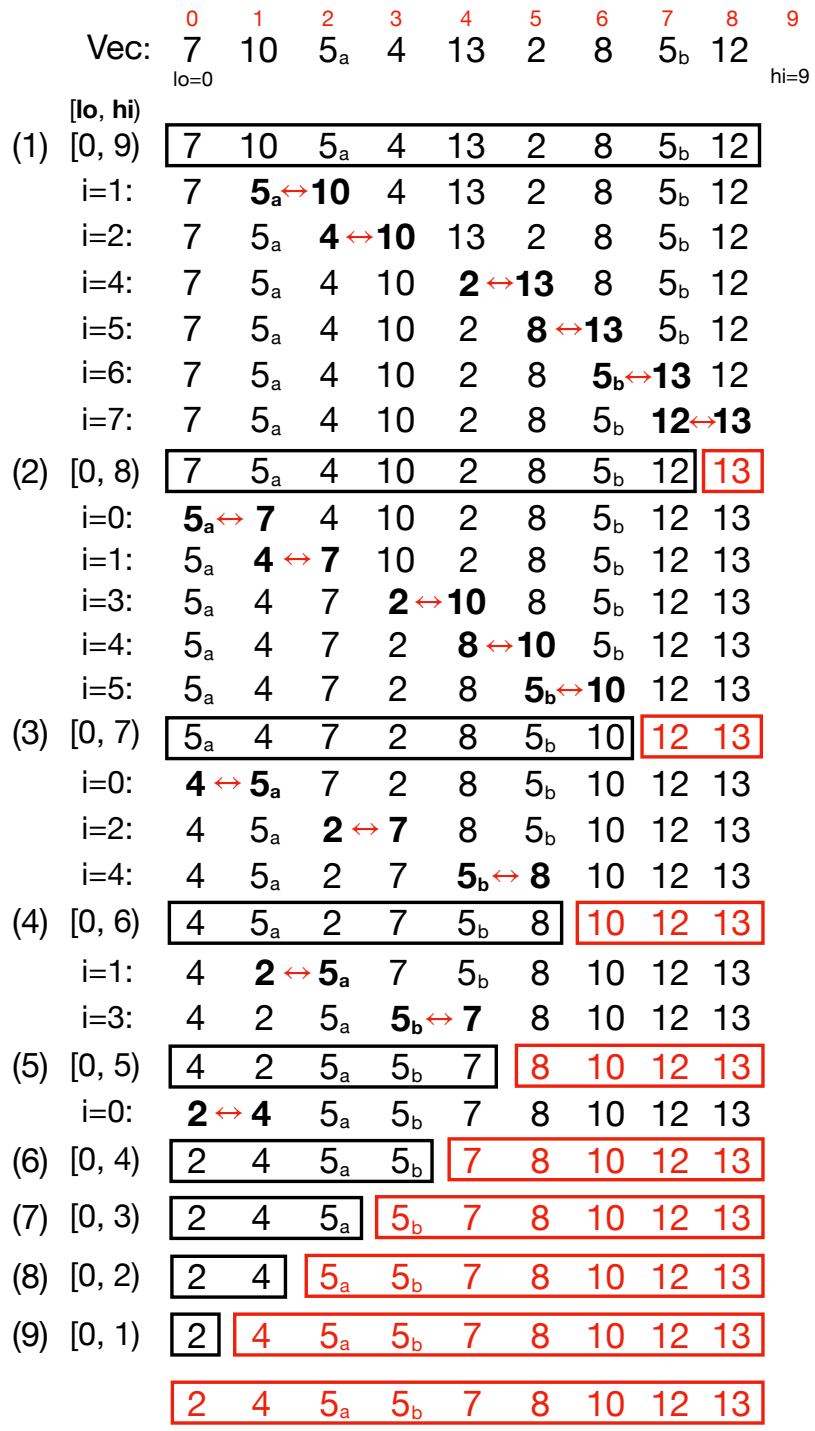


FIG. 5.32. 冒泡排序算法的执行过程。 待排序的序列为Vec $= \{7, 10, 5_a, 4, 13, 2, 8, 5_b, 12\}$。 图中共有9次第一重while循环过程，对应序列元素的遍历过程。在每次循环的开始，黑色实线方框标记了未排好序的序列元素，红色实线方框标记了已经排好序的序列元素。 被双向箭头↔相连的元素对表示相邻元素的逆序对（即Vec$[i]$和Vec$[i+1]$）已被交换。 9次遍历操作之后，初始序列最终成为有序序列。

最好情况下，算法的时间复杂度为$\mathcal{O}(n)$。 最坏情况下，算法的时间复杂度为$\mathcal{O}(n^2)$。平均情况下，算法的时间复杂度为$\mathcal{O}(n^2)$。

冒泡排序算法中交换逆序对的操作需要循环过程的两个整型下标、以及一个临时变量来存储序列元素。这些中间数据所占的内存为常数级，与序列长度无关。由此可知，冒泡排序算法是一个就地算法。

6. 归并排序

■ 算法的基本原理

归并排序(Mergesort)的出发点有两个：

（1）把短的序列（比如只有一个或两个元素的序列）进行排序，本质上是简单的。

（2）多个有序序列的合并成一个有序序列，本质上也是简单的。

归并排序算法是一个递归算法，其过程包含两个部分：

（1）迭代分段过程：把给定序列不断对半切分，一直到单个的元素。这个过程中序列元素本身没有改变。

（2）二路归并(two-way merge)过程：把切分后的有序序列合并成新的有序序列。这个算法从合并两个单个元素开始执行，新的有序序列继续合并成更长的有序序列，直到整段序列都为有序的。这个过程中序列元素发生了移动。

冒泡排序算法是一个基于交换逆序对的算法。对一个倒序序列，其交换逆序对的次数大致为$n^2/2$。但在归并排序算法里，无论序列的具体结构，从二路归并过程的第一步可知，交换逆序对的次数最多为$n/2$。归并排序算法里最耗时的部分在于有序序列的合并。从单个元素开始的有序序列的不断合并这个步骤，本质上同时处理了所有排序算法需要处理的两个问题：消除序列的无序、确保排序算法结束后序列在整体上的有序性。

归并排序算法的基本思路如Figure 5.33所示。

■ 算法的代码实现

归并排序算法的参考代码如Figures 5.34和5.34所示。 void MergeSortMethod(int lo, int hi)表示归并排序算法，实现了序列Vec的下标范围[lo, hi)内元素的排序。 void MergeTwoSet(int lo, int mi, int hi)实现了两个有序序列（Vec[lo, mi)和Vec[mi, hi)）的合并。 合并后的序列存储在序列Vec[lo, hi)。

归并排序算法的整体计算流程如Figure 5.36所示。

归并排序算法是一个迭代算法，Figure 5.37展示Figure 5.36所示的排序场景下的迭代算法中各个函数的执行顺序，

二路归并过程实现的是两个有序序列的合并过程，其计算流程如Figure 5.38所示。

例子14: 随机序列上的归并排序算法

在一个随机生成的整型序列上，使用归并排序算法进行排序。

参考代码：`ch5e-sort.cc`中的main函数里的MergeSort()。

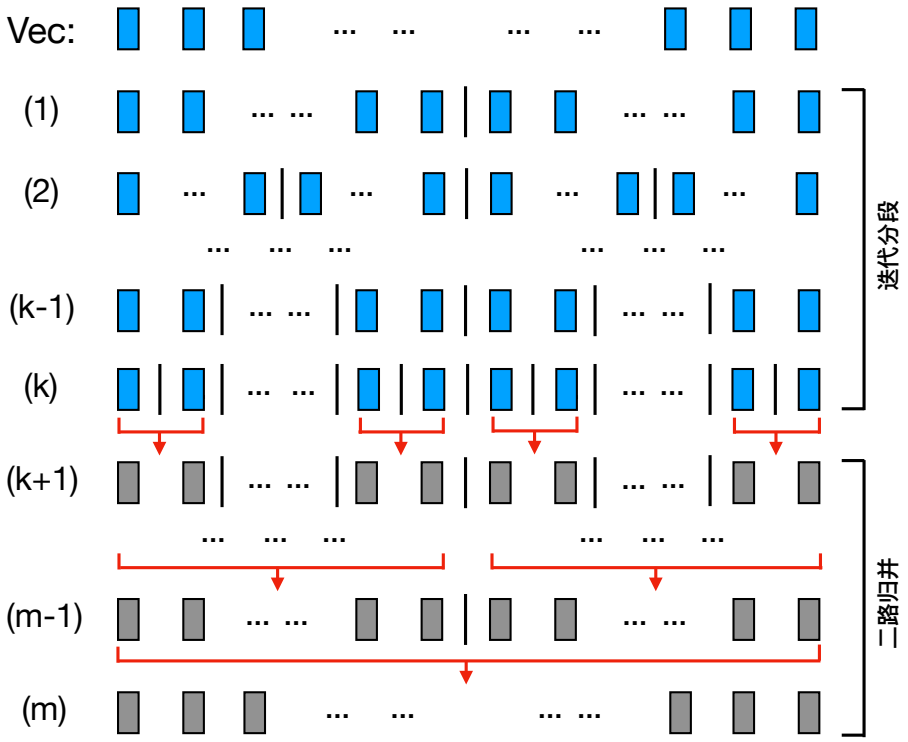


FIG. 5.33. 归并排序算法的基本思路。 需要排序的序列为Vec。 第1 ~ k步骤为迭代分段过程：每个子序列不断被对半切分成两个子序列。 第k步骤对应着序列Vec的每个元素单独构成了一个子序列。 第$k+1 \sim m$步骤为二路归并过程：从单个元素开始，相邻的有序序列合并成一个新的有序序列。 红色箭头表示子序列的合并过程。第m步骤之后，即得到排好序的序列。

```
//使用归并排序算法来排序Vec[lo, hi);
void MergeSortMethod(int lo,  int hi)
{
  //递归基；
  if (hi - lo < 2) return;

  //取中点作为轴点；
  int mi = (lo + hi) / 2;

  //对Vec[lo,mi)和Vec[mi,hi)两段分别执行归并算法；
  MergeSortMethod(lo, mi);
  MergeSortMethod(mi, hi);

  //合并两条有序序列:Vec[lo, mi)和Vec[mi,hi);
  MergeTwoSeq(lo, mi, hi);
};
```

FIG. 5.34. 归并排序算法里排序函数的参考代码。

```
//归并排序中的二路归并过程：把Vec中两段有序序列合并成一段有序序列；
void MergeTwoSeq(int lo, int mi, int hi)
{
  //把Vec[lo, mi)和Vec[mi, hi)的元素比较、并放入VecTemp;
  int i = lo;
  int j = mi;
  int k = lo;

  //把两段有序序列里较小值放入VecTemp内；
  //当两个元素的值相等时，取第一个放入VecTemp;
  while (i < mi && j < hi) {
    if (Vec[i] <= Vec[j]) {//如果使用Vec[i] < Vec[j]，结果会如何？
      VecTemp[k] = Vec[i];

      i++;
      k++;
    } else {
      VecTemp[k] = Vec[j];

      j++;
      k++;
    }
  }

  //如果Vec[lo, mi)或Vec[mi, hi)已经被全部挪到VecTemp里，
  //后续只需要把剩下的Vec[mi,hi)或Vec[lo,mi)内的元素依次放入VecTemp;
  if (i == mi) {
    while (j < hi) {
      VecTemp[k] = Vec[j];

      j++;
      k++;
    }
  } else if (j == hi) {
    while (i < mi) {
      VecTemp[k] = Vec[i];

      i++;
      k++;
    }
  }

  //把元素从VecTemp[lo, hi)拷贝回到Vec[lo, hi);
  for (int k = lo; k < hi; k++) {
    Vec[k] = VecTemp[k];
  }
};
```

FIG. 5.35. 归并排序算法里二路归并算法的参考代码。

■ 算法的稳定性

归并排序算法的稳定性依赖于二路归并过程的实现。 在合并两个有序序列的过程中，如果遍历两个序列时的两个元素分别为Vec[i]和Vec[j]。在合并过程中，如果Vec[i] $\leqslant$ Vec[j]，则Vec[i]首先被放入新合并的序列；如果Vec[i] $>$ Vec[j]，则Vec[j]首先被放入新合并的序列。容易可知，如果Vec[i] $=$ Vec[j]，首先放入新合并序列的为Vec[i]。依次类推，当待排序的序列中存在多个重复元素时，这些元素在二路归并过程中会按照它们的初始顺序依次被放入合并后的序列。也就是说，归并排序算法是稳定的。

想象一下，如果在二路归并过程中，有序序列中元素比较和合并的规则如下：如果Vec[i] $<$ Vec[j]，则Vec[i]首先被放入新合并的序列；如果Vec[i] $\geqslant$ Vec[j]，则Vec[j]首先

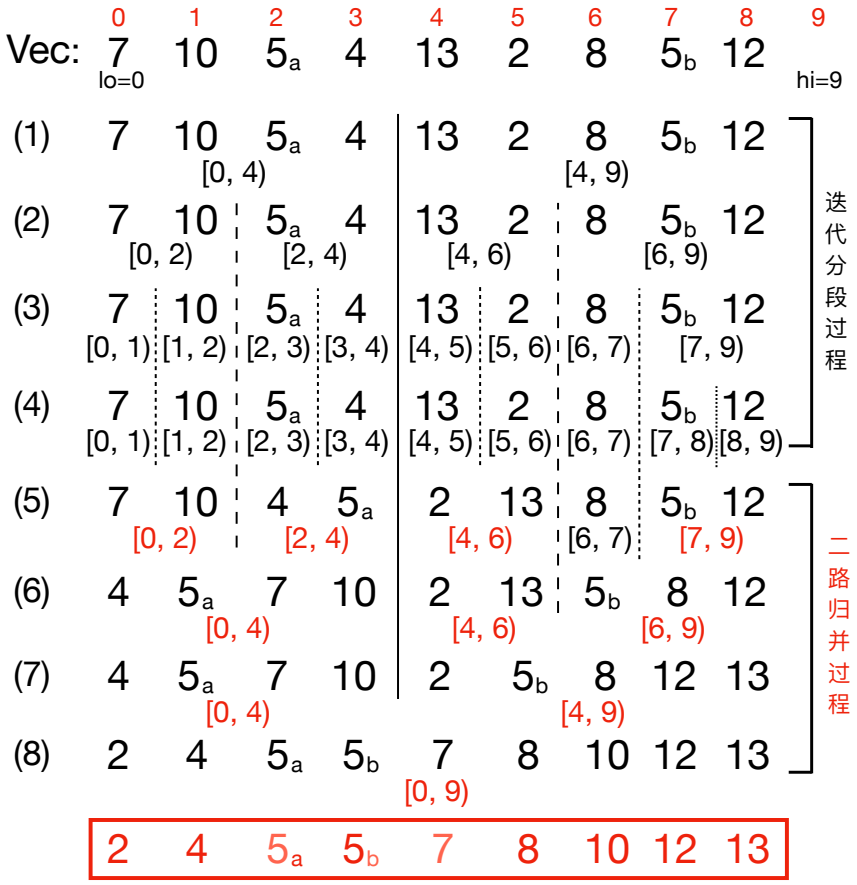


FIG. 5.36. 归并排序算法的执行过程。 初始给定向量为Vec $= \{7, 10, 5_a, 4, 13, 2, 8, 5_b, 12\}$，排序范围为[0, 9)。向量下的[lo, hi)表示各段向量对应的下标范围。在迭代分段过程中，向量被不断二分直到成为单个元素。在二路归并过程里，红色的[lo, hi)表示对应的向量分段已经是有序向量。排序之后得到的有序序列Vec $= \{2, 4, 5_a, 5_b, 7, 8, 10, 12, 13\}$。

被放入新合并的序列。那么此时的算法是否依然能够实现排序？如果它能够实现排序，那么它还是稳定的么？

■ 算法的复杂度

最好情况下，算法的时间复杂度为$\mathcal{O}(n \log_2 n)$。 最坏情况下，算法的时间复杂度为$\mathcal{O}(n \log_2 n)$。 平均情况下，算法的时间复杂度为$\mathcal{O}(n \log_2 n)$。

在归并排序算法里，序列的分段过程可以直接在初始序列上进行，并用下标的左右界来标记。 在二路归并过程里，我们需要把两个有序序列合并成为一个有序序列。这两个序列在初始序列上是连续的、可以使用不同的下标进行标记。但合并过程中的有序序列很难在初始序列上完成。 对于任意两个长度为m和n的有序序列，我们可以构造一个长度

为$m+n$的序列来存储不断合并的序列，最后把此序列拷贝回初始序列。对于整个待排序的初始序列Vec，我们直接生成一个同样长度的序列VecTemp来暂存二路归并过程中的所有序列。如Figure 5.39所示。 容易得之，归并排序算法的中间数据至少需要一个和初始序列一样长度的序列。因此归并排序算法不是一个就地算法。

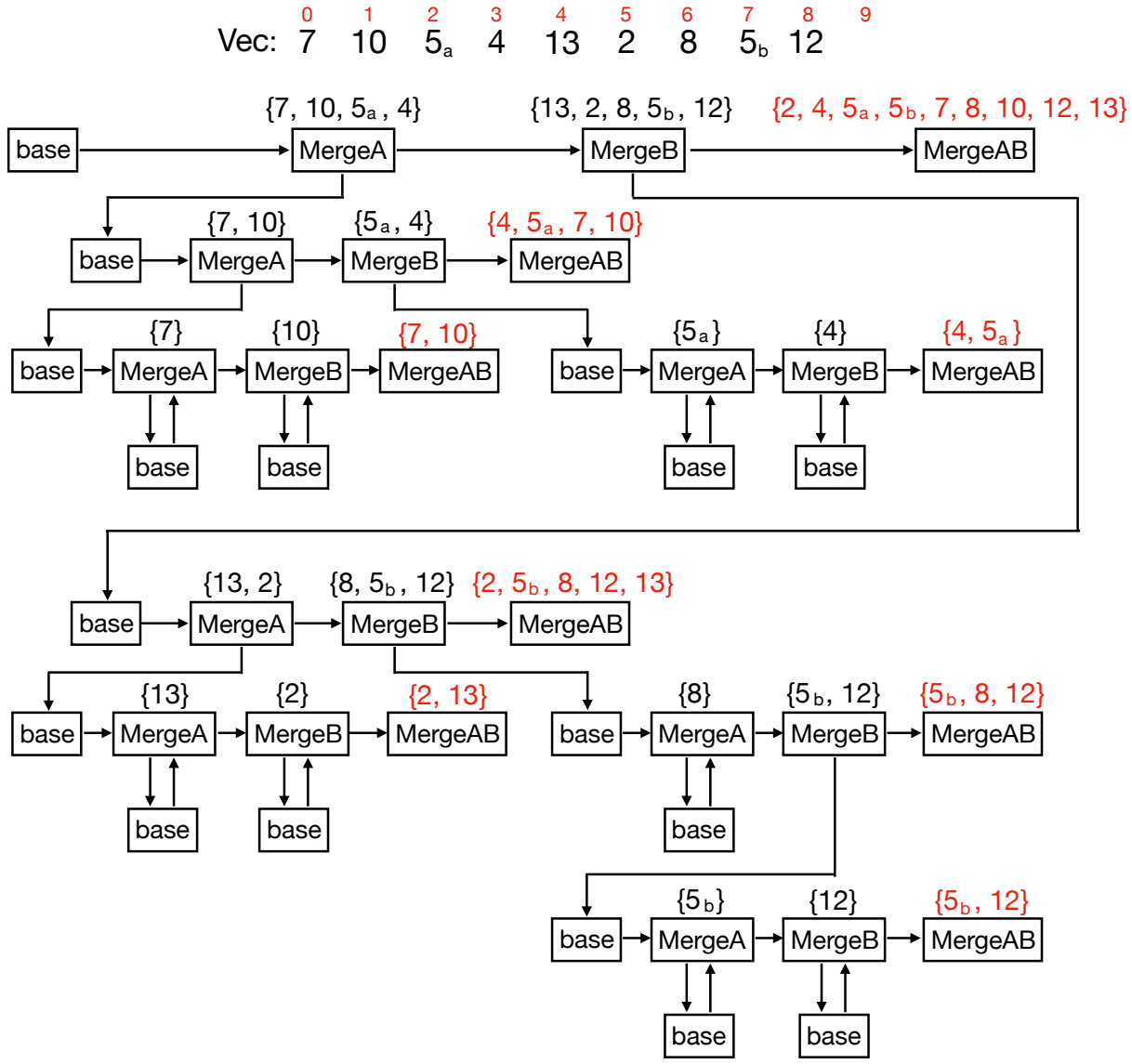


FIG. 5.37. 归并排序算法中递归过程的实现。 图中给出了给定长度为9的Vec后，Figure ??中代码执行MergeSortMethod(lo, hi)(lo = 0, hi = 9)的顺序。 base表示递归基是否执行。MergeA表示在第一段分区调用函数MergeSortMethod(lo, mi)。MergeB表示在第二段分区上调用函数MergeSortMethod(mi, hi)。 MergeAB表示两路归并函数MergeTwoSeq(lo, mi, hi)。 图中箭头表示函数执行的顺序。MergeA和MergeB方格上的序列表示相应函数要处理的序列。 MergeAB方格上的序列表示归并之后得到的有序序列。

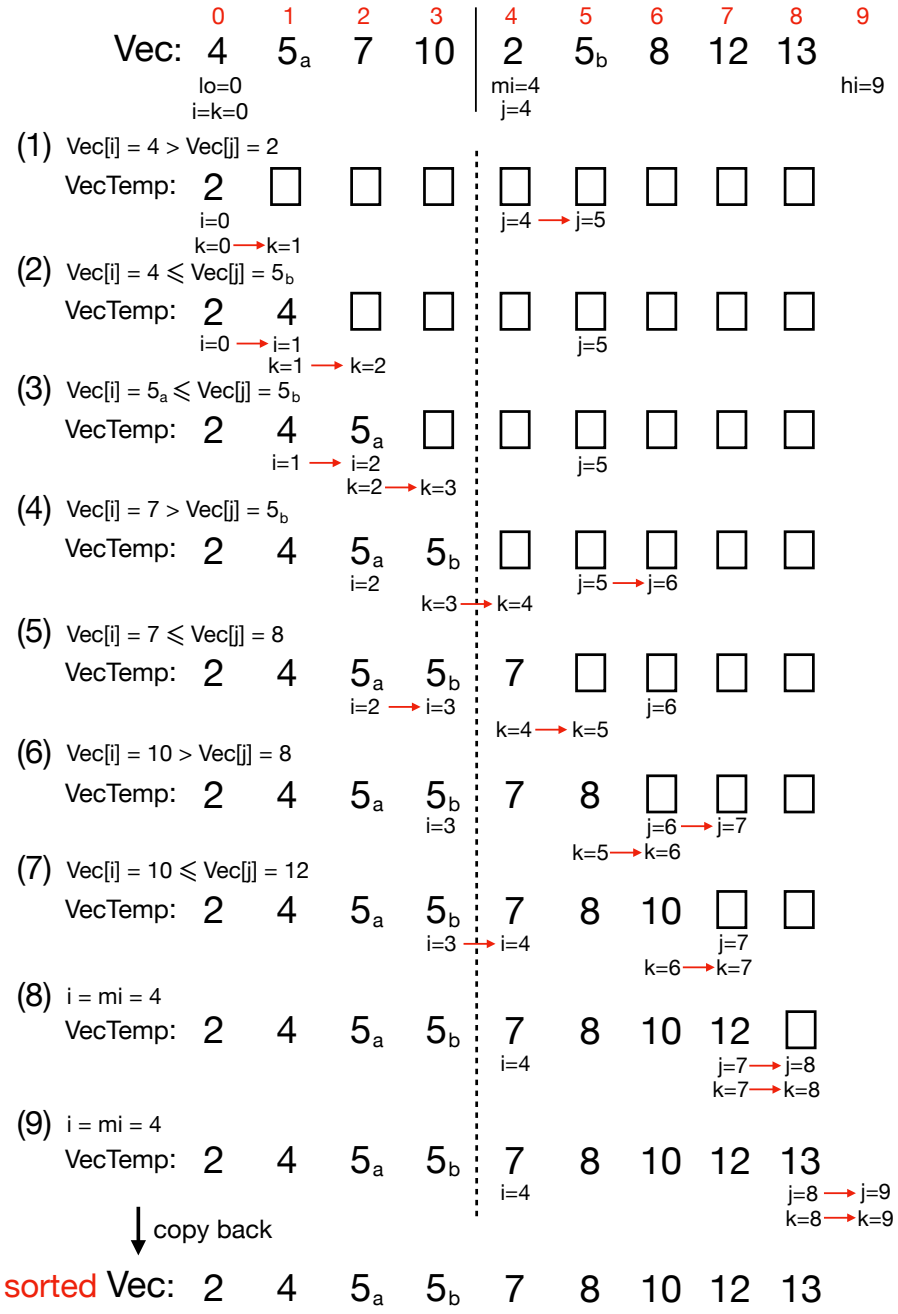


FIG. 5.38. 归并排序算法里的二路归并过程。这里对应着Figure 5.36的第7-8步骤。给定向量为Vec $= \{4, 5_a, 7, 10, 2, 5_b, 8, 12, 13\}$。两段有序序列分别为Vec[0, 4)和Vec[4, 9)。重复元素5的下标a和b用来区分两者的相对顺序。经过二路归并过程之后，得到的有序序列Vec $= \{2, 4, 5_a, 5_b, 7, 8, 10, 12, 13\}$。可知经过二路归并后，重复元素5的相对顺序没有变化。

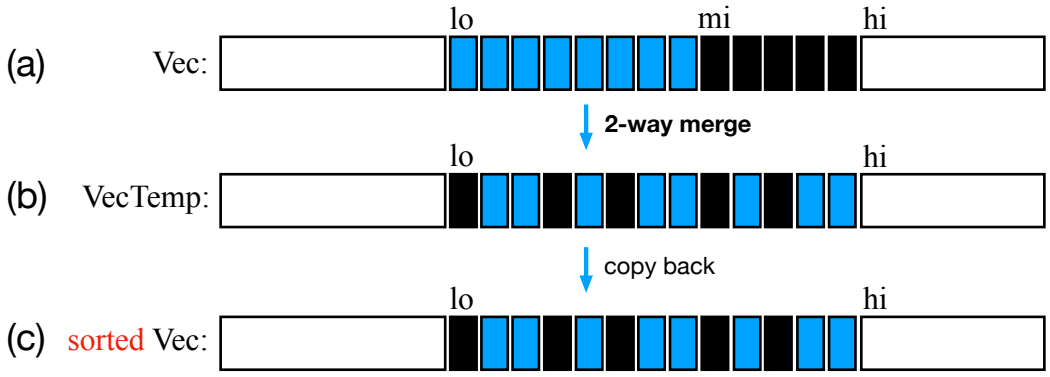


FIG. 5.39. 归并排序算法里二路归并过程中使用的数据结构。 (a)需要排序的原始向量为Vec。归并排序过程中需要合并的两段有序序列为Vec[lo, mi)和Vec[mi, hi)。 (b)归并算法中的中间变量为VecTemp，大小同样为n，用来在二路归并算法过程中缓存合并过程中的有序向量。最终得到合并后的有序向量为VecTemp[lo, hi)。 (c)有序序列VecTemp[lo, hi)被重新拷贝回到相应的Vec[lo, hi)。此时一个完整的二路归并过程结束。

7. 快速排序

■ 算法的基本原理

快速排序(Quick)算法基于序列中的轴点的概念。

• 轴点(pivot)：考虑一个向量Vec及其区间Vec[lo, hi)。对于任何lo $\leqslant$ mi $<$ hi，以元素Vec[mi]为界，可以分割出来前后两个子向量Vec[lo, mi)和Vec[mi + 1, hi)。如果对于某个mi，Vec[lo, mi)中的元素$\leqslant$Vec[mi]，且Vec[mi + 1, hi)中的元素均$\geqslant$Vec[mi]，则元素Vec[mi]称作为向量Vec的一个轴点。

• 排序算法前后的轴点：初始向量为Vec，排完序之后的序列为Vec'。轴点位置mi必然满足以下充要条件：

(1) Vec[mi] = Vec'[mi]。

(2) Vec[lo, mi)和Vec'[lo, mi)的成员完全相等，但顺序可能不一样。

(3) Vec[mi + 1, hi)和Vec'[mi + 1, hi)的成员完全相等，但顺序可能不一样。

基于轴点的概念，我们得到排序算法的一个思路：选择一个元素，将其作为轴点，通过移动元素把它放置到序列中正确的位置；对于轴点分开的两个子序列，我们可以使用确定轴点的方式来进行排序，直到子序列为空或者为单个元素。这个过程可以很自然地使用迭代函数形式来实现。当所有的子序列完成排序之后，即可得到排好序的整个序列。

上面过程就是快速排序酸的基本思路。 快速排序算法的基本思路如Figure 5.40所示。

对于快速排序算法，我们能够看到以下几点：

（1）为了“消除序列中的无序性”，快速排序算法从“部分宏观有序”的角度出发。“部分宏观有序”是对于轴点、轴点左侧元素、轴点右侧元素三部分来说的。轴点左侧元素和轴点右侧元素本身可能是无序的，但轴点左侧元素 $\leqslant$ 轴点 $\leqslant$ 轴点右侧元素的整体顺序还是存在的。这个过程是通过按照下标来移动元素来实现，不同于冒泡排序算法里的交换逆序对。

（2）为了“保证排序算法得到的序列在整体上的有序性”，快速排序算法被设计为一个递归算法，也就是在得到给定序列的轴点、轴点左侧元素、轴点右侧元素的部分宏观有序性后，分别在轴点左侧元素、轴点右侧元素上继续调用算法流程。最终序列里的所有元素都满足轴点的特性，相应地序列也就成了一个有序序列。

■ 算法的代码实现

快速排序算法的参考代码如Figure 5.41所示。 快速排序算法中实现轴点划分的函数为Figure 5.42所示。

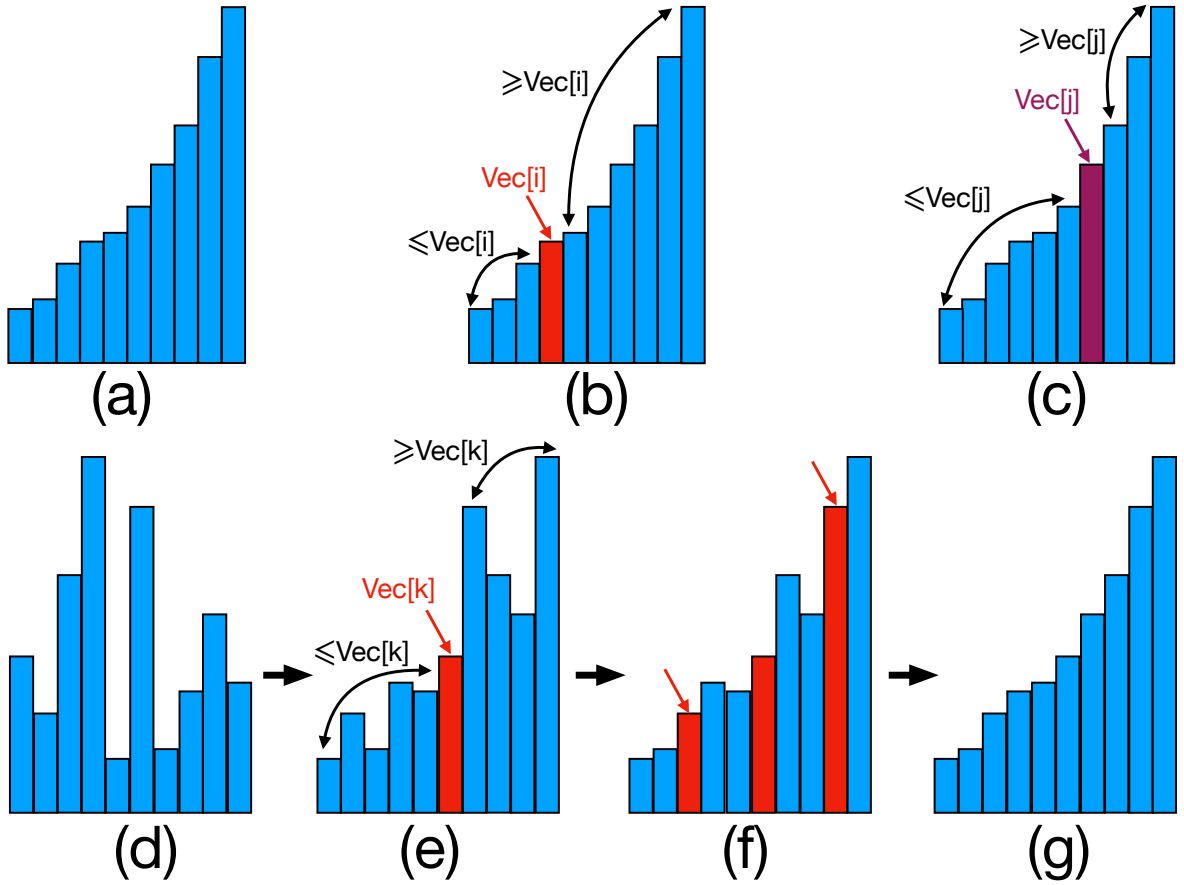


FIG. 5.40. 序列中的轴点和基于轴点的排序算法。 (a)为一个长度为n的有序向量Vec。 (b)中元素Vec[i]（红色条状所示）为序列的一个轴点，即Vec[i]（$i \in [0, n)$）满足下标小于i的元素⩽ Vec[i]、下标大于i的元素⩾ Vec[i]。 (c)中Vec[j]（深红色条状所示）为序列Vec的另外一个轴点。本质上，有序序列的所有元素都是序列的轴点。 (d)为一个无序向量Vec。 (e)中，通过元素移动的操作，使得某个元素Vec[k]成为序列Vec的一个轴点。 (f)中，轴点Vec[k]隔开的两个子序列上可以继续使用(e)中通过移动元素来放置轴点的操作。 (g)经过多次操作之后，Vec里的所有元素都为序列的轴点，相应地序列也就成为一个有序序列。

这里展示了在一个确定序列上快速排序算法的实现细节。 考虑一个序列

$$\text{Vec} = \{7,\ 10,\ 5_{\text{a}},\ 4,\ 13,\ 2,\ 8,\ 5_{\text{b}},\ 12\}, \tag{5.31}$$

在其上使用快速排序时，需要调用6次切分函数的步骤。 其完整流程如Figures 5.43 - 5.45所示。 Figure 5.43展示了一次完整的选取轴点、并基于轴点把向量进行切分的操作过程。 第1步骤为读取、并暂存轴点pivot。新的首元素8作为函数int QuickPartition(int, int)里的轴点。 第2 ∼ 5步骤为排序函数里第一重while语句的第1次循环，分别对应着往左

```
//使用快速排序法来排序Vec[lo,hi);
void QuickSortMethod(int lo, int hi)
{
  if (hi - lo < 2) return;

  int mi = QuickPartition(lo, hi - 1);

  QuickSortMethod(lo, mi);    //排序Vec[lo, mi);
  QuickSortMethod(mi + 1, hi);//排序Vec[mi+1, hi);
};
```

FIG. 5.41. 快速排序算法里排序函数的参考代码。 函数参数为序列的下标范围[lo, hi)。函数实现了此下标范围内元素的排序。

```
//快速排序的分段函数：在子区间Vec[lf, ri]内交换元素以寻找、并返回轴点下标mi;
int QuickPartition(int lf, int ri)
{
  //在向量中随机选择一个元素和首元素互换；
  int rand_ind = lf + rand() % (ri - lf + 1);//在[lf, ri]内随机选择下标；

  swap(Vec[lf], Vec[rand_ind]);

  //取出新的首元素暂存，并作为轴点；
  int pivot = Vec[lf];

  while (lf < ri) {
    //向左移动ri;
    while (lf < ri && pivot <= Vec[ri]) {
      ri--;
    }

    //把遇到的第一个比pivot小的元素挪到pivot左侧；
    Vec[lf] = Vec[ri];

    //向右移动lf;
    while (lf < ri && Vec[lf] <= pivot) {
      lf++;
    }

    //把遇到的第一个比pivot大的元素挪到pivot右侧；
    Vec[ri] = Vec[lf];
  }

  //重新放入备份的轴点；
  Vec[lf] = pivot;

  //lf此时为pivot的下标；
  return lf;
};
```

FIG. 5.42. 快速排序算法里排序函数的参考代码。 函数参数为序列中的两个下标[lf, ri)。函数在此下标范围内随机选择一个元素，继而通过移动序列元素使得此元素称为此序列内的一个轴点。函数最终返回其下标。

移动ri、赋值元素Vec[lf]、往右移动lf、赋值元素Vec[ri]。 第6 ∼ 9步骤、第10 ∼ 12步骤分别为排序函数里第一重while语句的第2次、第3次循环。 第13步骤为第一重while循环语句结束后重新把pivot赋值给Vec[lf]。 最终切分函数的返回下标为5（对应序列元素为8）。

第一次切分函数之后得到待排序的两个序列:

$$\mathrm{Vec}[0,4] = \{5_\mathrm{b},\ 7,\ 5_\mathrm{a},\ 4,\ 2\}, \tag{5.32}$$

$$\mathrm{Vec}[6,8] = \{13,\ 10,\ 12\}. \tag{5.33}$$

Figure 5.44和5.45在Vec[0, 4]和Vec[6, 8]上分别继续执行切分函数。 结合Figures 5.43 - 5.45，最终我们可以得到排序后的向量

$$\mathrm{Vec} = \{2,\ 4,\ 5_\mathrm{b},\ 5_\mathrm{a},\ 7,\ 10,\ 12,\ 13\}. \tag{5.34}$$

从这个例子里，我们能发现$\{5_\mathrm{a}, 5_\mathrm{b}\}$的顺序在排序后发生了变化。从这个例子可以直接看出来快速排序算法为不稳定的排序算法。

例子15: 随机序列上的快速排序算法

在一个随机生成的整型序列上，使用快速排序算法进行排序。

参考代码：`ch5e-sort.cc`中的main函数里的QuickSort()。

■ 算法的稳定性

■ 算法的复杂度

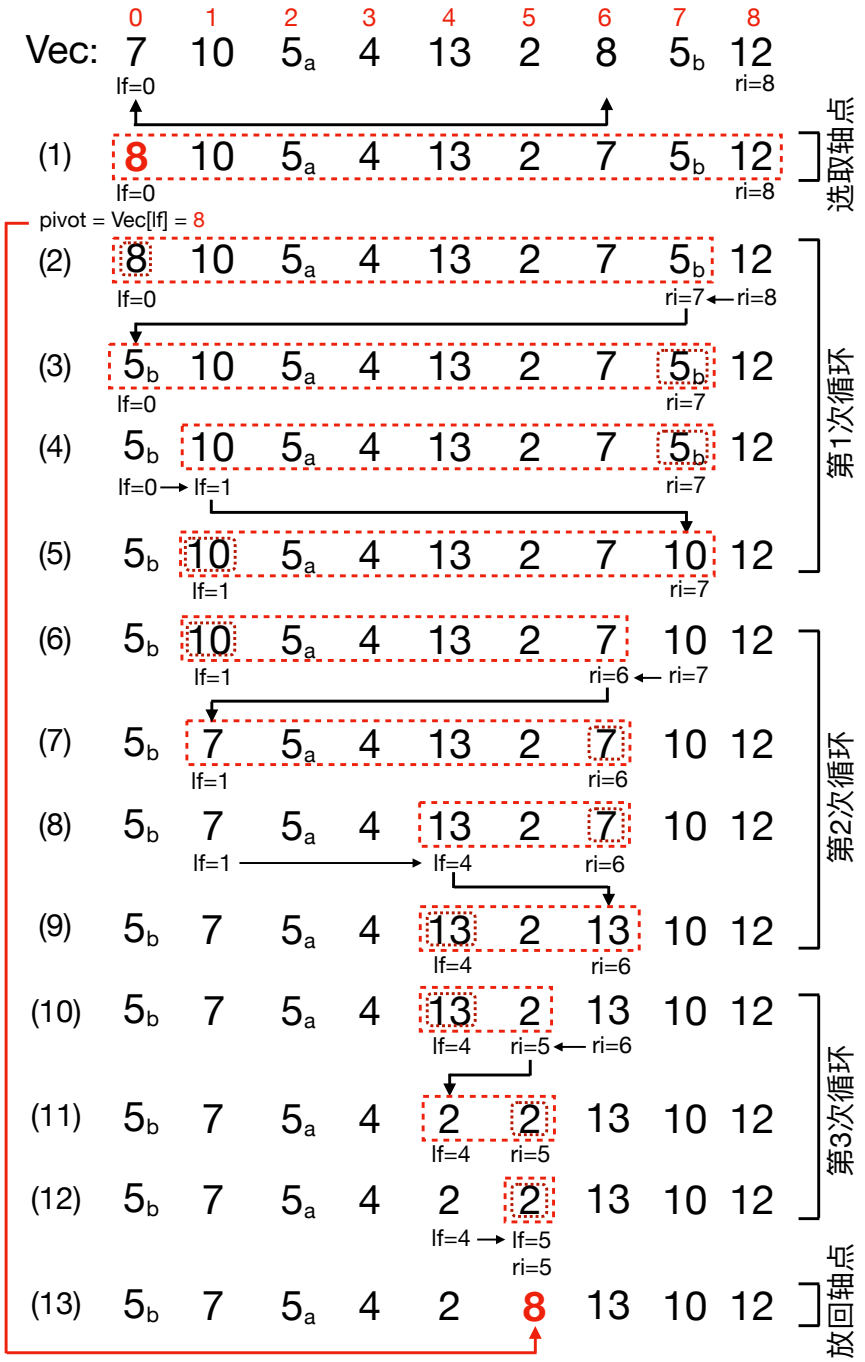


FIG. 5.43. 快速排序算法的执行过程(1/3)。 初始向量Vec $= \{7, 10, 5_a, 4, 13, 2, 8, 5_b, 12\}$，长度为9，排序范围为[lf, ri] $= [0, 8]$。 这里展示了第一次切分函数的操作过程。 虚线方框表示向量元素范围[lf, ri]，点线方框标记的单个元素为已经赋值给其它元素的元素。

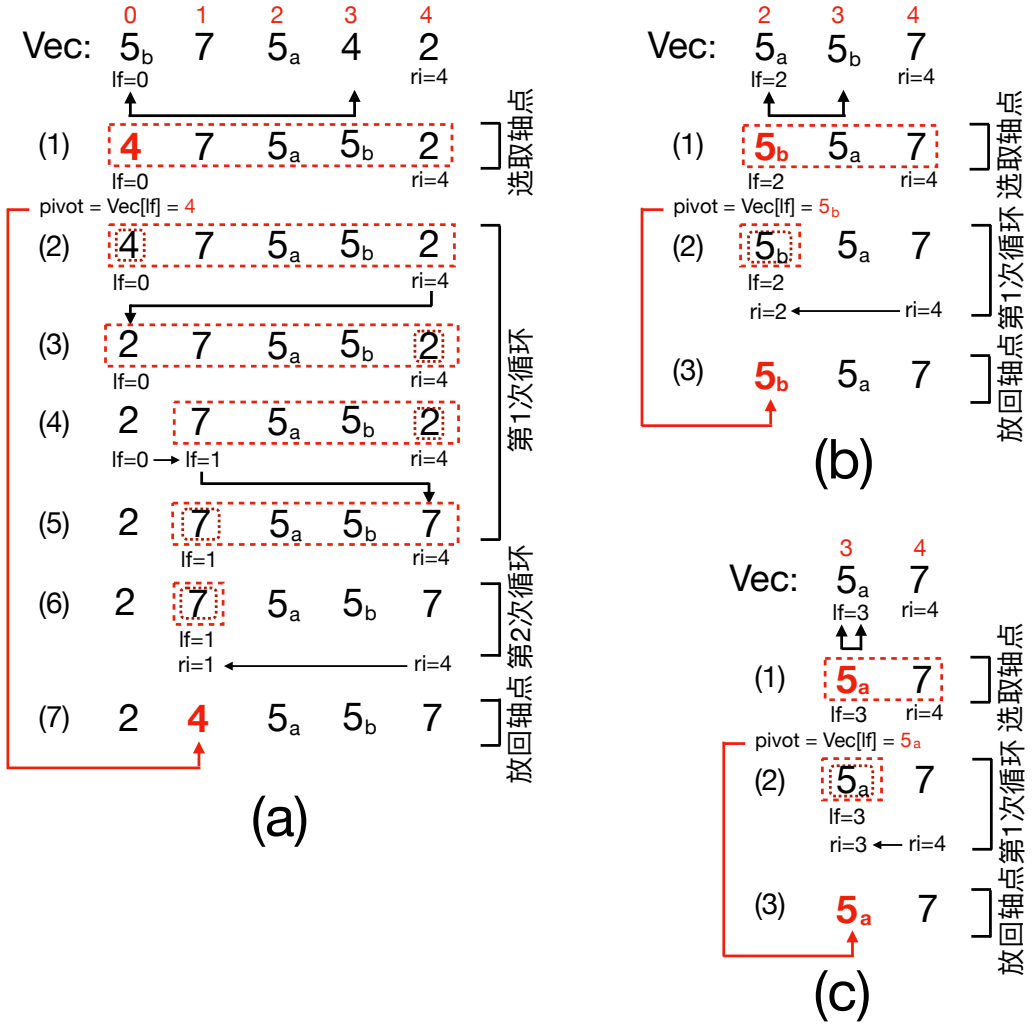


FIG. 5.44. 快速排序算法的执行过程(2/3)。 这里描述了Figure 5.43里步骤(13)中Vec[0, 4]上的三次切分函数的执行过程。 (a)执行了Vec[0, 4]上的一次切分函数。 (b)进而执行了Vec[2, 4]上的一次切分函数。 (c)进而执行了Vec[3, 4]上的一次切分函数。 最终得到排好序的Vec[0, 4] = $\{2, 4, 5_b, 5_a, 7\}$。

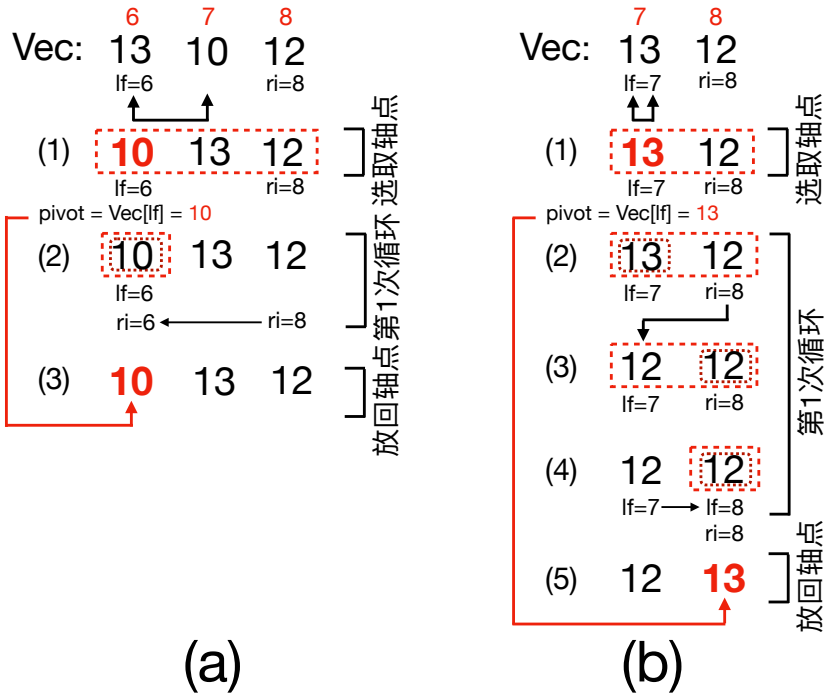


FIG. 5.45. 快速排序算法的执行过程(3/3)。 这里描述了Figure 5.43里步骤(13)中Vec[6, 8]上的两次切分函数的执行过程。 (a)执行了Vec[6, 8]上的一次切分函数。 (b)进而执行了Vec[7, 8]上的一次切分函数。 最终得到排好序的Vec[6, 8] $= \{10, 12, 13\}$。

习题

5.1 课程分数的统计性质

考虑对某门课里多个同学的成绩进行一些统计。下标为0开始的10位同学的分数如下：

$$\hat{s} = \{91,\ 85,\ 95,\ 84,\ 95,\ 92,\ 91,\ 84,\ 96,\ 88\}. \tag{5.35}$$

编程要求：

请选择合适的数据结构来存储以上数据，并求解以下问题：

（1）计算并输出课程分数中的最高分、以及对应的下标。注意可能存在多个最高分。

（2）计算并输出课程分数中的最低分、以及对应的下标。注意可能存在多个最低分。

（3）计算并输出所有分数的均值。

（4）计算并输出所有分数所对应的标准分（standard score），即z-分数（z-score）。z-分数可以视为一个数相对于其所属序列的均值的偏移程度。给定序列$\hat{x}$的数据成员$x_i, i \in [0, N-1]$，其对应z-分数z_i如下：

$$z_i = \frac{x_i - \mathrm{E}(\hat{x})}{\sigma(\hat{x})}. \tag{5.36}$$

其中，$\mathrm{E}(\hat{x})$和$\sigma(\hat{x})$分别为$\hat{x}$的期望值（expectation value，即均值）和标准差（standard deviation）。$\mathrm{E}(\hat{x})$和$\sigma(\hat{x})$的形式如下：

$$\mathrm{E}(\hat{x}) = \frac{1}{N}\sum_{i=0}^{N-1} x_i, \tag{5.37}$$

$$\sigma(\hat{x}) = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}[x_i - \mathrm{E}(\hat{x})]^2}. \tag{5.38}$$

5.2 生日问题

这里我们考虑在给定人群中人们同一天过生日的问题。我们假设每个人的生日日期都是随机的。在给定人数后，我们统计出多个人同一天过生日的分布情况。

在使用代码实现时，你可以生成一个长度为$D = 365$的数组来表示非闰年一年内的每天。然后使用随机数来分别生成人数为N的生日。最终在数组上统计有多少个人同时在某一天过生日、及其人数占总人数的比例。

编程要求：

（1）选择随机数生成器时，你可以使用课堂里提到的rand()函数。

（2）请完成代码。测试以下不同人数的情形：

- $N = 74$：这是2024级数据科学与大技术技术专业两个班的选课人数；
- $N = 240$：这是本学院2024级本科生的总人数；
- $N = 365$：这是一年的总天数；
- $N = 8007$：这是华师2025级本科生新生的总人数。

对于以上情况，请分别从整数$0, 1, 2, \cdots$（表示某天内有$0, 1, 2, \cdots$个人生日）开始按行输出对应天数、其占全年总天数的比例、相应同一天生日的人数、其占总人数的比例。

5.3 二分查找法（版本A）的执行过程

考虑一个由整型数组成的、长度为10的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 7_{\text{a}},\ 7_{\text{b}},\ 7_{\text{c}},\ 8,\ 9_{\text{a}},\ 9_{\text{b}},\ 12\}. \tag{5.39}$$

注意此序列中有重复元素7和9。给定参数(elem, lo, hi)，我们使用二分查找法（版本A）在向量Vec里给定下标的范围[lo, hi)内查找给定元素elem。算法的参数可选择：

（1）(elem, lo, hi) = $(8, 0, 10)$；

（2）(elem, lo, hi) = $(6, 1, 8)$；

（3）(elem, lo, hi) = $(8, 1, 10)$；

（4）(elem, lo, hi) = $(6, 1, 9)$。

参考讲义第5.5节“查找算法”里Figure 5.7的格式，画出示意图以展示查找算法执行时切分点下标、子区间的范围、查找长度的变化。

注意：此题画出示意图即可，无需写代码。

5.4 斐波那契查找法的执行过程

考虑一个由整型数组成的、长度为10的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 7_{\text{a}},\ 7_{\text{b}},\ 7_{\text{c}},\ 8,\ 9_{\text{a}},\ 9_{\text{b}},\ 12\}.$$

给定参数(elem, lo, hi)，我们使用斐波那契查找法在向量Vec里给定下标的范围[lo, hi)内查找给定元素elem。 算法的参数可选择：

（1）(elem, lo, hi) = $(8, 0, 10)$；

（2）(elem, lo, hi) = $(6, 1, 9)$。

参考讲义第5.5节“查找算法”里Figure 5.12的格式，画出示意图以展示查找算法执行时切分点下标、子区间的范围、查找长度的变化。

注意：此题画出示意图即可，无需写代码。

5.5 插值查找法的执行过程

考虑一个由整型数组成的、长度为10的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 7_{\text{a}},\ 7_{\text{b}},\ 7_{\text{c}},\ 8,\ 9_{\text{a}},\ 9_{\text{b}},\ 12\}.$$

给定参数(elem, lo, hi)，我们使用插值查找算法在向量Vec里给定下标的范围[lo, hi)内查找给定元素elem。 算法的参数可选择：

（1）(elem, lo, hi) = (8, 0, 10)；

（2）(elem, lo, hi) = (6, 1, 9)。

参考讲义第5.5节“查找算法”里Figure 5.16的格式，画出示意图以展示查找算法执行时切分点下标、子区间的范围、查找长度的变化。

注意：此题画出示意图即可，无需写代码。

5.6 二分查找法（版本B）的执行过程

考虑一个由整型数组成的、长度为10的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 7_{\text{a}},\ 7_{\text{b}},\ 7_{\text{c}},\ 8,\ 9_{\text{a}},\ 9_{\text{b}},\ 12\}.$$

给定参数(elem, lo, hi)，我们使用二分查找法（版本B）在向量Vec里给定下标的范围[lo, hi)内查找给定元素elem。 算法的参数可选择：

（1）(elem, lo, hi) = (8, 0, 10)；

（2）(elem, lo, hi) = (7, 0, 10)；

（3）(elem, lo, hi) = (6, 1, 9)。

参考讲义第5.5节“查找算法”里Figure 5.19的格式，画出示意图以展示查找算法执行时切分点下标、子区间的范围、查找长度的变化。

注意：此题画出示意图即可，无需写代码。

5.7 二分查找法（版本C）的执行过程

考虑一个由整型数组成的、长度为10的有序序列：

$$\text{Vec} = \{2,\ 4,\ 5,\ 7_{\text{a}},\ 7_{\text{b}},\ 7_{\text{c}},\ 8,\ 9_{\text{a}},\ 9_{\text{b}},\ 12\}.$$

给定参数(elem, lo, hi)，我们使用二分查找法（版本C）在向量Vec里给定下标的范围[lo, hi)内查找给定元素elem。 算法的参数可选择：

（1）(elem, lo, hi) = (8, 0, 10)；

（2）(elem, lo, hi) = (7, 0, 10)；

（3）(elem, lo, hi) = (6, 1, 9)。

参考讲义第5.5节“查找算法”里Figure 5.22的格式，画出示意图以展示查找算法执行时切分点下标、子区间的范围、查找长度的变化。

注意：此题画出示意图即可，无需写代码。

5.8 查找算法的平均查找长度

这里我们从定量角度来验证排序算法的平均查找长度对序列长度的依赖关系。

我们首先生成一个长度为n、由整型元素构成的有序序列，然后在序列上分别使用查找算法，统计并计算其平均成功查找长度$\mathrm{SL}^S(n)$和平均失败查找长度$\mathrm{SL}^F(n)$。对于不同的n，我们可以得到对应的$\mathrm{SL}^S(n)$和$\mathrm{SL}^F(n)$。最终在结果图上，将查找算法对应的两个平均查找长度$\mathrm{SL}(n)$的数据点画出来。这里选择一些比较大的n，比如$n = 10^5 \sim 10^6$。

以下是计算和做图上的一些要求：

（1）测试二分查找法（版本A）和斐波那契查找法。

（2）在序列上使用上述查找算法来计算某个平均查找长度时，可以选择计算15 ∼ 20个数据点。这个数目的数据点既能够表现出数据的一些趋势，但不至于太过拥挤。

（3）为了将数据点以近似线性的方式展示出来，请把x轴从n换成$\log_2 n$。

（4）在数据点附近加上理论预测的结果，即Eqs.(5.13)和(5.14)所示。其形式如Figure 2.6(b)所示。这样可以和序列上的计模拟结果进行对比。

5.9 插入排序算法的执行过程

考虑一个长度为9的、由整型数构成的序列：

$$\mathrm{Vec} = \{7,\ 9,\ 5,\ 4_{\mathrm{a}},\ 2,\ 12,\ 8,\ 4_{\mathrm{b}},\ 6\}. \tag{5.40}$$

使用插入排序算法在上述序列上。

请画出算法的执行过程。格式请参考讲义第5.6节“排序算法”里Figure 5.26。

注意：此题画出示意图即可，无需写代码。

5.10 选择排序算法的执行过程

考虑一个长度为9的、由整型数构成的序列：

$$\mathrm{Vec} = \{7,\ 9,\ 5,\ 4_{\mathrm{a}},\ 2,\ 12,\ 8,\ 4_{\mathrm{b}},\ 6\}.$$

使用选择排序算法在上述序列上。

请画出算法的执行过程。格式请参考讲义第5.6节“排序算法”里Figure 5.29。

注意：此题画出示意图即可，无需写代码。

5.11 冒泡排序算法的执行过程

考虑一个长度为9的、由整型数构成的序列：

$$\text{Vec} = \{7,\ 9,\ 5,\ 4_{\text{a}},\ 2,\ 12,\ 8,\ 4_{\text{b}},\ 6\}.$$

使用冒泡排序算法在上述序列上。

请画出算法的执行过程。格式请参考讲义第5.6节“排序算法”里Figure 5.32。

注意：此题画出示意图即可，无需写代码。

5.12 反向冒泡排序算法

在冒泡排序算法里，我们通过不断遍历无序向量以把最大的元素通过交换操作挪到序列最后的位置。但从另外一个角度来讲，如果我们通过不断遍历序列以把最小的元素挪到序列靠前的位置的话，同样也可以得到一个有序序列。

考虑一个长度为9的、由整型数构成的序列：

$$\text{Vec} = \{7,\ 9,\ 5,\ 4_{\text{a}},\ 2,\ 12,\ 8,\ 4_{\text{b}},\ 6\}.$$

请描述此算法的流程，并使用代码来实现。并请测试以上序列、输出每次遍历过程之后的序列。

5.13 归并排序算法的执行过程

考虑一个长度为9的、由整型数构成的序列：

$$\text{Vec} = \{7,\ 9,\ 5,\ 4_{\text{a}},\ 2,\ 12,\ 8,\ 4_{\text{b}},\ 6\}.$$

使用归并排序算法在上述序列上。

请画出算法的执行过程。格式请参考讲义第5.6节“排序算法”里Figure 5.36。

注意：此题画出示意图即可，无需写代码。

5.14 快速排序算法中切分函数的执行过程

考虑一个长度为9的、由整型数构成的序列：

$$\text{Vec} = \{7,\ 9,\ 5,\ 4_{\text{a}},\ 2,\ 12,\ 8,\ 4_{\text{b}},\ 6\}.$$

我们在这个序列上使用快速排序算法里利用轴点来切分序列的函数int QuickPartition(int lf, int ri)。此时输入的函数参数为[lf, ri] = [0, 8]。请选择以下元素作为轴点，即首先把该元素和下标为0的元素7进行互换，之后使用切分函数把这个元素放置到合适的下标位置。轴点可以选择为：

（1）下标为2的元素5;

（2）下标为6的元素8。

请画出这个函数的执行过程。格式请参考讲义第5.6节“排序算法”里Figure 5.43。

注意：此题画出示意图即可，无需写代码。

5.15 排序算法的时间复杂度

这里我们定量地验证排序算法的计算时间$T(n)$对序列长度n的依赖关系。

我们首先生成一个长度为n、由整型元素构成的随机序列，然后使用排序算法进行排序、并记录其计算时间$T(n)$。请选择比较大的n，比如$n = 10^5 \sim 10^6$。这里可以测试多个n以及对应的$T(n)$。最终我们把$T(n)$对n进行做图。

以下是计算和做图上的一些要求:

（1）请依次测试冒泡排序算法、归并排序算法、快速排序算法。

（2）我们关心的算法步骤的运行时间指的是算法在排序过程中的计算时间。请避开统计生成、读取、输出序列等其它过程的计算时间。请在代码里选择合适的位置使用clock()函数。

（3）做图时为了让计算时间的数据点尽可能呈现出来直线形式，请选择合适的x轴，比如n^2或$n \log_2 n$。

（4）在数据点附近加上理论预测的大致斜率，以和序列上的模拟计算结果做比较。其形式如Figure 2.6(b-d)中所示。

6. (LIST)

参考内容：邓俊辉教材第3.1-3节；李春葆教材第2.3节。

本章介绍了线性数据结构的第二种实现方式：列表。 这里首先介绍了列表和向量因为物理实现方式的不同所导致的在静态操作和动态操作上的差别。 之后继续介绍了列表里双链表的ADT和代码实现。 本章后续介绍了链表上常见的操作及其代码实现，比如列表的初始化、给定位置后的查询、列表单元的插入、列表单元的删除。

6.1. 列表的基本性质

本质上，列表和向量都为线性数据结构。向量内的所有元素（节点）在内存依次排列、没有间隙。这本身对向量本身做了很强的结构约束。但在列表里，元素在内存里可以随意排列，元素之间可以通过指针进行连接。 向量和列表在结构上差别可见如Figure 6.1(a)。

本质上，向量上能够实现的各类算法在列表里同样都能实现。 但因为物理实现方式的不同，导致差别两者在静态和动态基本操作上的不同。

• 静态操作和动态操作

定义在数据结构上的操作，大致可以分成静态操作和动态操作两种。

（1）**静态操作过程**：这类操作在执行中只读取向量的信息、而不会修改向量本身，比如返回数据结构里元素个数、返回指定位置的某个元素、查找某个元素等等。

（2）**动态操作过程**：这类操作在执行时能够修改数据结构的元素，比如从结构里删除元素、在结构里插入元素、删除整个结构等等。

• 静态存储策略

序列的元素在内存内连续排列，比如向量。这样在访问给定下标的元素时，可以直接通过简单的计算获取元素所在的内存地址。这个过程的操作时间是一个很小的常数。但执行动态操作时，往往需要附带地对大量元素进行操作。比如:

（1）从序列中间删除某个元素时，为了保证删除元素之后序列依然保持连续排列的结构特征，就需要把序列里被删除元素之后的所有元素依次往前挪动、并释放最后一个元素。

（2）从序列中间插入某个元素时，同样为了保证插入元素之后序列依然保持连续排列的结构特征，需要把插入位置之后的所有元素依次往后挪动、并在插入位置放入元素。

平均意义上来说，上述两个操作涉及到的取值和赋值的操作次数是等比于序列删除和

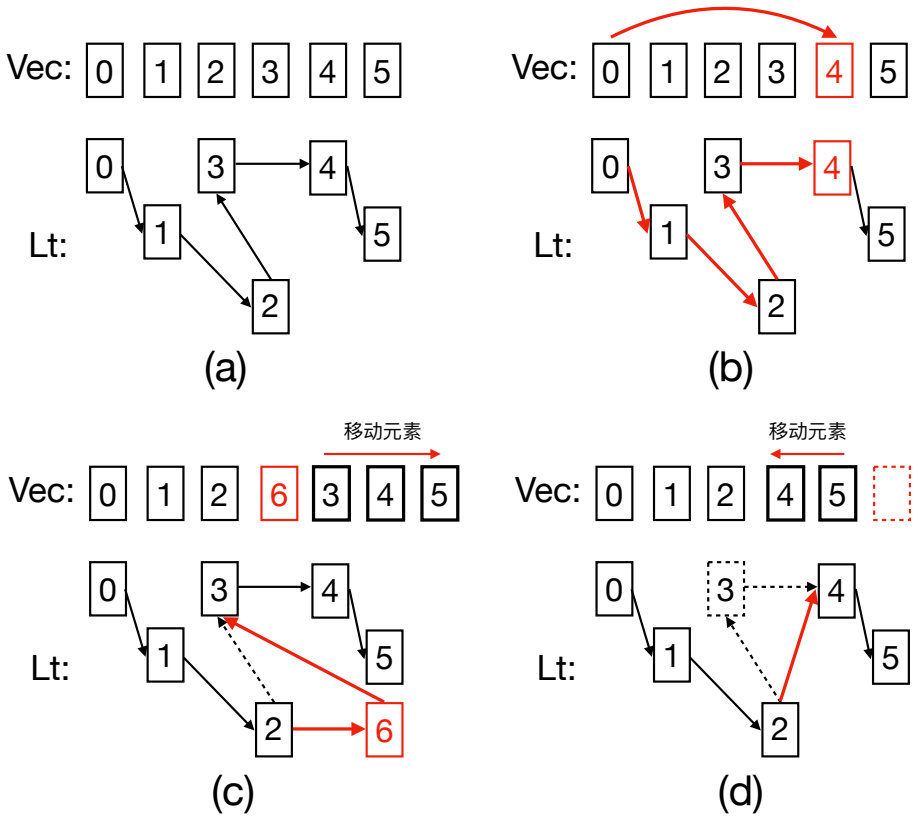


FIG. 6.1. 向量和列表的结构及其上的静态操作和动态操作。 (a)为一个向量Vec和一个列表Lt。它们分别包含6个元素，编号为0 ～ 5。其元素在二维平面上的相对位置大致为它们在内存里的相对位置。列表Lt里的箭头为元素之间的引用关系。 (b)分别为向量Vec上访问秩为4的元素的操作、在列表Lt上访问编号为4的元素的操作。在向量里，直接通过取对应元素的内存地址访问。但在列表里，必须沿着元素之间的引用（标记为红色箭头）来访问目标元素。 (c)分别为在向量Vec上秩为3的元素之前插入一个标记为6的元素的操作、在列表Lt上编号为3之前插入一个元素的操作。 (d)分别为在向量Vec上删除秩为4的元素的操作、在列表Lt上删除编号为4的元素的操作。 在(c)-(d)里，向量在插入和删除时都需要挪动元素，而在列表里只需要重新设置元素之间的引用（标记为红色箭头）即可。

插入元素位置之后的所有元素的个数。相应地这种动态操作正比于序列长度，在长序列上的操作会非常耗时。

- 动态存储策略

序列的元素在内存里不用连续排列，相邻之间通过指针或者引用联系起来，比如列表。这种结构在执行动态操作时非常迅速，因为只需要在局域结构里改变单元之间的连接

方式就可以了，并不需要在全局上挪动元素。但代价是，因为这种结构里只有纯粹的局域信息，导致在访问某个给定位置的元素时，是没法直接得到此元素的内存位置的信息，只能在序列里从头开始遍历整个序列、直到到达给定位置为止。平均意义上这个操作过程的此时是正比于序列的长度。

向量和列表在静态操作和动态操作上的差别可见如Figure 6.1(b)-(d)。

• 线性结构和半线性结构

在线性结构里，静态操作和动态操作的快速性不能兼得。如果选择了向量，访问元素这类静态操作会非常迅速、但修改序列这类动态操作会很慢；如果选择了列表，动态操作会非常快、但静态操作会很慢。

在为具体问题选择数据结构时，向量和列表并不是二选一。在实际情况下，两者可以结合在一起使用。比如对于某个数据集合而言，有些部分的大小是不变、但有些部分是频繁变化的。在设计数据结构ADT时，那些不变的部分可以使用向量、那些频繁需要进行动态操作的部分可以使用列表来表示。比如在图的数据结构中，格点部分比连边部分的变化频率要低很多。此时可以使用向量来表示格点、用列表来表示每个格点的最近邻，即需要使用一个向量和格点数目的列表来表示一个图。

如果要兼得比较平衡而且快速的静态操作时间和动态操作时间，就得换到一个不为线性结构的场景。这就是在后面章节我们会讲到的“半线性结构”，比如二叉搜索树。对于节点数为n的二叉搜索树，使用其来作为排序任务时，其静态和动态操作的时间均为$\mathcal{O}(\log_2 n)$。

• 列表的典型结构

通过列表里元素之间的引用或指针关系，可以定义双链表（Doubly Linked List）、单链表（Singly Linked List）等等。在双链表里，节点里有指向前后元素的指针。在单链表里，只有一个指向相邻元素的指针。Figure 6.2展示了双链表和单链表的结构。

Figure 6.2中的链表结构里，除了虚线方框里用来存储具体数据的节点，还有并不会存储数据的头节点和尾节点。头节点和尾节点也属于列表节点的类型。它们只是用来借助标记序列开始和结束的位置。列表本身已经有了首节点和末节点来表示列表的开始和结束，为何还需要定义头节点和尾节点？ 这类节点的存在使得算法在遍历序列时，可以统一地使用比较操作符! =来判断是否已经到了序列末尾。 整体上这类节点并不存储数据、但其存在可以使得代码在实现上更加直观简练。这类节点就叫哨兵节点(sentinel node)。 其它场合也存在哨兵节点，比如C++ STL的vector容器中迭代器的end()和逆序迭代器的rbegin()函数，如Figure 6.3所示。

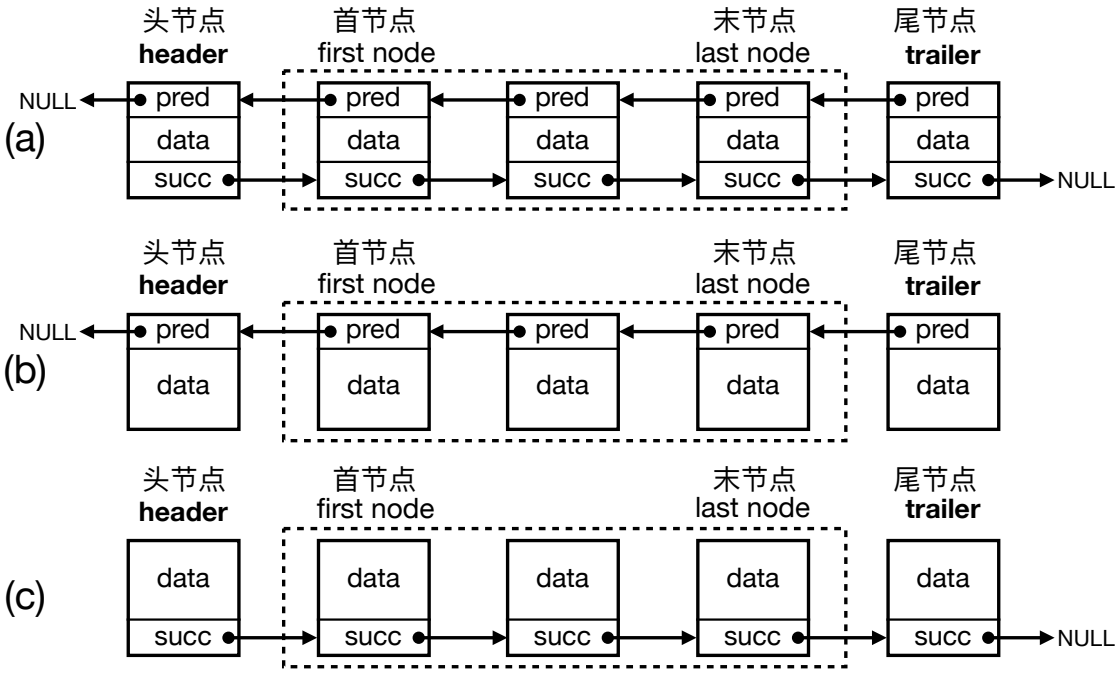


FIG. 6.2. 列表的常见结构。 虚线方框表示了列表内存储了数据的部分，其内部第一个节点叫首节点(first node)、最后一个节点叫末节点(last node)。两类链表都设置了同为列表节点类型的哨兵节点(sentinel node)：头节点(`header`)和尾节点(`trailer`)。 (a)为双链表。列表单元里除了数据项data，好包含了指向前一个节点的指针`pred`、指向后一个节点的指针`succ`。 (b)为一类单链表。数据单元里除了数据项data，只有指向前一个节点的指针`pred`。 (c)为另一类单链表。数据单元里除了数据项data，只有指向后一个节点的指针`succ`。

6.2. 列表的ADT

在定义列表前，我们首先需要定义列表节点的结构体，因为它是列表的基本存储单元。定义列表时，我们还需要定义了列表的长度、头节点（`header`）、尾节点（`trailer`）这类列表的宏观结构信息，以及定义在列表上的操作函数。 我们把具体的代码部分组织成了头文件`list.h`，在使用列表时可以引用它。Figure 6.5展示了类中定义部分的代码。

列表上的指针操作直接涉及到元素在内存里读取和移动。在实现具体的功能时，需要很周密地设计好流程。相比于一般的算法，这种流程更接近底层操作的层面。

以下介绍列表上常见的操作过程及其代码实现。

（1）列表里元素的查找

这里实现的给定一个位置参数nth，函数返回从首节点开始的第nth个节点的数据内

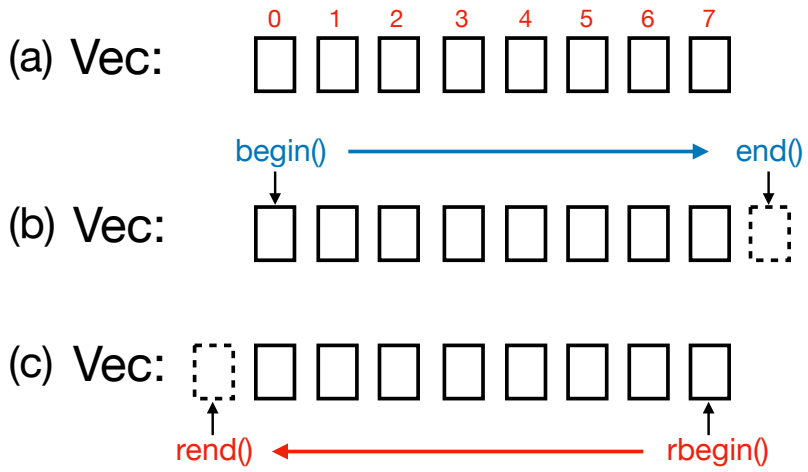


FIG. 6.3. 向量里的迭代器和哨兵节点。 (a)为一个长度为8的向量Vec。 (b)中向量的begin()和end()返回顺序迭代器的头尾迭代器，分别指向Vec的第一个节点所在位置和最后一个节点之后的位置。 (c)中向量rbegin()和rend()返回逆向迭代器的头尾迭代器，分别指向Vec的最后一个节点所在位置和第一个节点之前的位置。 end()和rend()所指向的元素并不真实存在。它们均为哨兵节点。

```
//列表节点的结构体；
struct ListNode {
  //数据成员；
  struct ListNode* pred;//节点的前驱指针；
  struct ListNode* succ;//节点的后继指针；

  int    val;//节点的数据部分；可使用基本数据类型、结构体等；

  //初始化函数；
  ListNode();
  ListNode(ListNode*, ListNode*, int);
};
ListNode::ListNode() {
  pred = NULL;
  succ = NULL;

  val  = 0;
};
ListNode::ListNode(ListNode* ipred, ListNode* isucc, int ival) {
  pred = ipred;
  succ = isucc;

  val  = ival;
};
```

FIG. 6.4. 列表节点的结构体ListNode。 此结构体包括了3个数据成员和2个初始化函数。

```
//列表的类；
class List {
 public:
  // *************** 数据成员 ***************
  //列表的结构参数；
  struct ListNode* header; //列表的头节点；
  struct ListNode* trailer;//列表的尾节点；

  int    lt_size;//列表中节点的数目；

  // *************** 操作函数 ***************
  //从头节点开始查询第nth的节点，并返回其val；
  int    FindViaSucc(int);//nth; 利用节点的后继指针进行查询；
  int    FindViaPred(int);//nth; 利用节点的前驱指针进行查询；

  //构造空列表；
  void   ConstructANullList();

  //在给定节点的附近插入单个新节点；
  struct ListNode* InsertAsSucc(ListNode*, int);//val; 在给定节点的后方插入新节点；
  struct ListNode* InsertAsPred(ListNode*, int);//val; 在给定节点的前方插入新节点；

  //删除单个节点；
  int    RemoveANodeFromList(ListNode*);//plt;

  //为列表元素赋值；
  void   SetOrderedListVal();//按其下标顺序赋值；
  void   SetRandomListVal(); //随机赋值；

  //在序列里连续插入节点；
  void   AddNodesToList(int);//size; 在列表里不断插入大小为size的节点；

  //检查列表是否有序；
  void   CheckSortedLt();

  //输出列表；
  void   OutputList();
};
```

FIG. 6.5. 列表List的ADT。 这里使用class定义了列表List，包括其宏观结构信息：列表的头节点(header)、列表的尾节点(trailer)、列表中元素的个数(lt_size)，以及定义在列表上的静态和动态操作函数。

容。 其操作过程和代码可参考Figure 6.6。

（2） 空列表的初始化

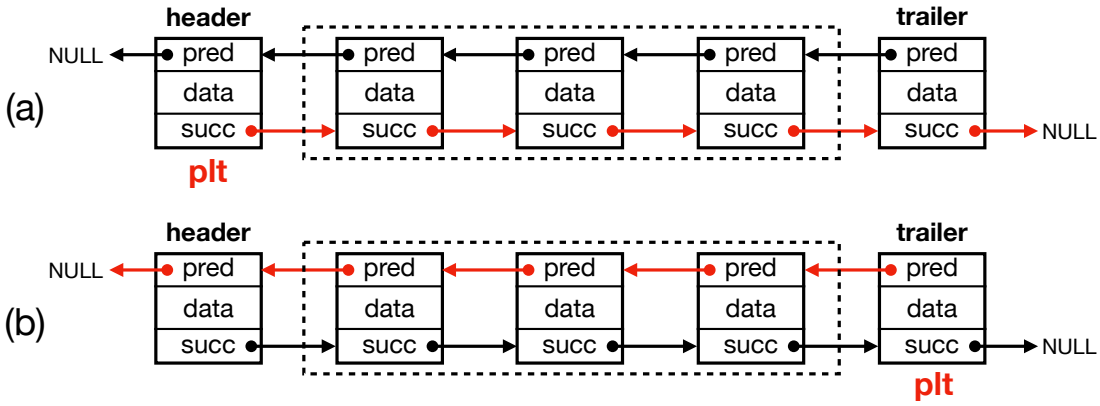
列表的初始化是最基本的动态操作过程。 这个函数用来构造一个空的列表，即只含有头节点和尾节点、没有具体的数据项。后续我们可以使用插入节点的方式来生成用来存储具体数据的列表。 在对应的函数里，头尾节点生成后，设置好相应的指针即可。 其操作过程和代码可参考Figure 6.7。

（3）列表里节点的后插入过程

此操作实现了在给定节点的之后位置里插入某个节点的操作。 在对应的函数里，参数包括节点指针和新插入节点的数据项。 其操作过程和代码可参考Figure 6.8。

（4）列表里节点的前插入过程

此操作实现了在给定节点的之前位置里插入某个节点的操作。 在对应的函数里，参数




```
//给定一个下标nth，从首节点开始，沿着succ指针返回第nth个节点的数据项；
int List::FindViaSucc(int nth)
{
  //检查nth的大小；
  if (nth < 0 || nth >= lt_size) {
    cerr<<"Error: n is out of range. "<<endl;
    exit(-1);
  }

  //找到下标n对应的元素，返回其数据项；
  ListNode* plt = header;

  while (nth >= 0) {
    plt = plt->succ;

    nth--;
  }

  return plt->val;
};
```

FIG. 6.6. 列表上按照给定位置查询元素。（下图a）为基于后继指针的查询过程。指针plt从首节点开始沿着后继节点指针访问第nth个节点，返回此节点的数据项。（上图b）为基于前驱指针的查询过程。指针plt从末节点开始沿着前驱节点指针访问第nth个节点，返回此节点的数据项。红色箭头标记了查询过程所依赖的后继指针和前驱指针。（下图）此图为基于后继节点指针的方式来查询元素过程的参考代码。

包括节点指针和新插入节点的数据项。其操作过程和代码可参考Figure 6.9。

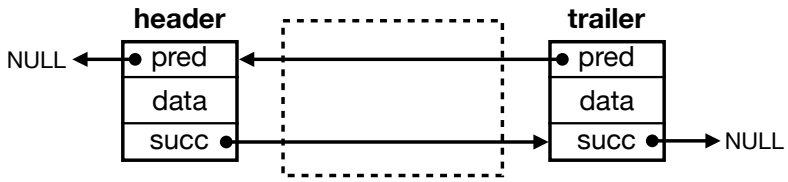
（5）列表里节点的删除

此操作实现了删除给定指针对应节点的操作。在对应的函数里，函数参数为被删除节点的指针。其操作过程和代码可参考Figure 6.10。

例子1: 列表上元素的插入和删除

代码生成了空的列表，并进而实现插入节点、删除节点的操作。

参考代码：`ch6-list.cc`中main()函数里的BasicOperations()。




```
//生成一个空列表；此时无数据项；
void List::ConstructANullList()
{
  //生成头节点和尾节点；
  ListNode* pheader  = new ListNode();
  ListNode* ptrailer = new ListNode();

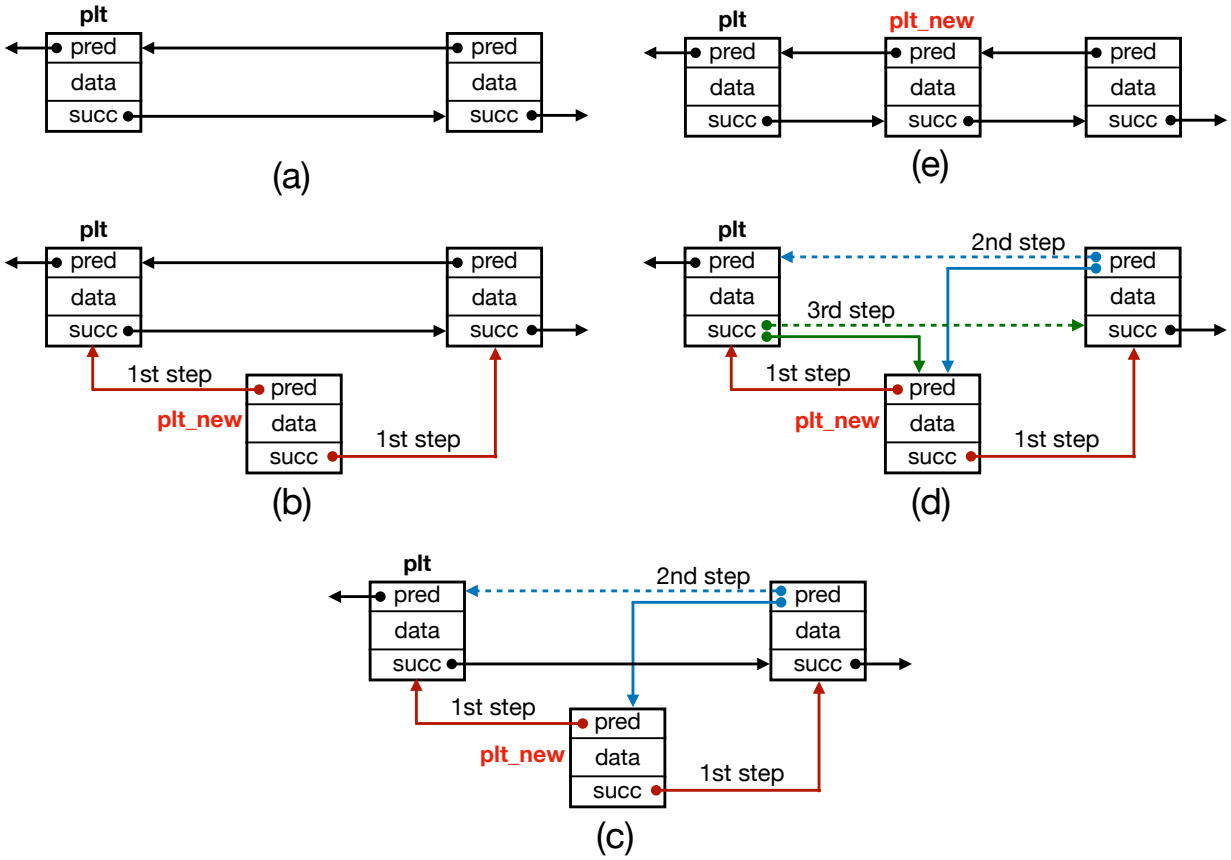
  header  = pheader;
  trailer = ptrailer;

  //设置头节点和尾节点的指针；
  header->pred  = NULL;
  header->succ  = trailer;

  trailer->pred = header;
  trailer->succ = NULL;

  //设置列表的长度；
  lt_size = 0;
};
```

FIG. 6.7. 空列表的初始化。 （上图）为空列表的图示。虚线方框表示列表中没有存储数据项的节点。 （下图）为初始化空列表的参考代码。



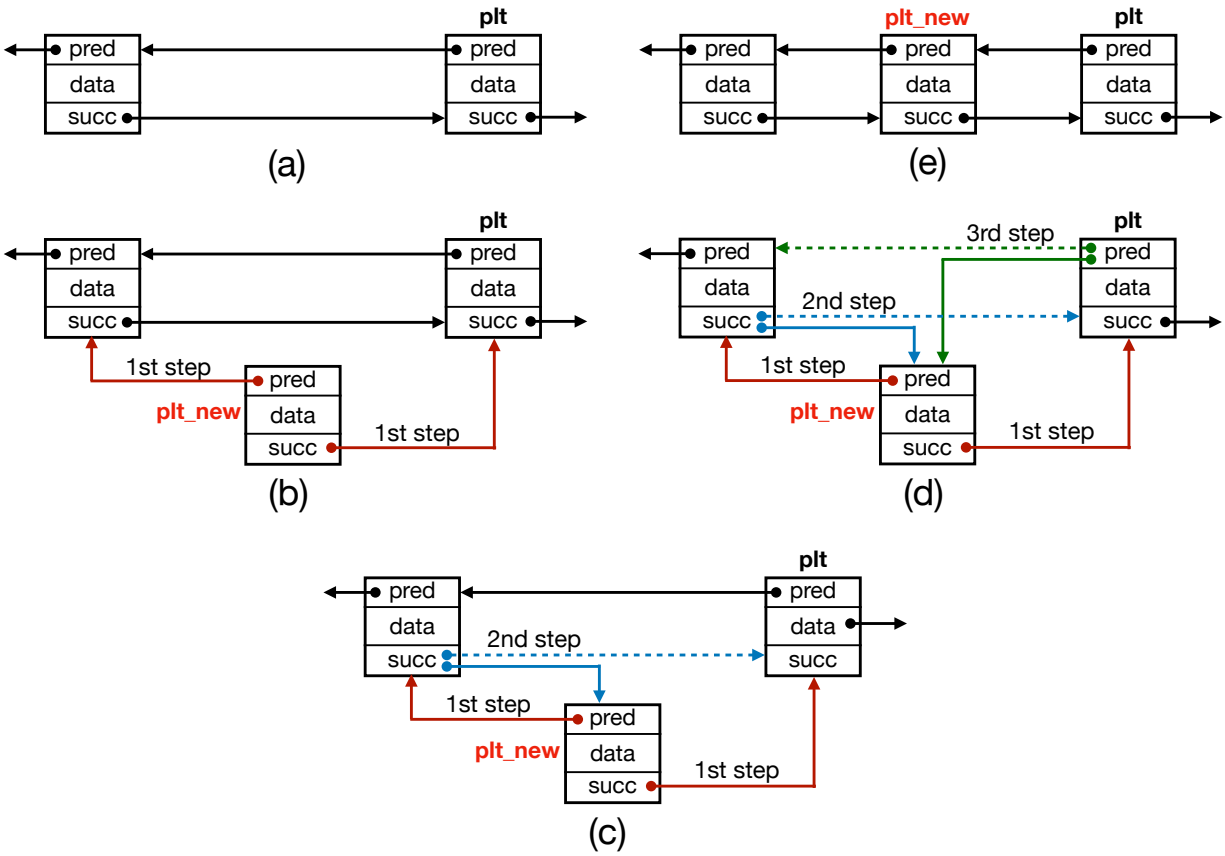

```
//在指定元素的后继节点的位置插入新节点，并返回其指针；
ListNode* List::InsertAsSucc(ListNode* plt, int val)
{
  ListNode* plt_new = new ListNode(plt, plt->succ, val);//1st step;

  plt->succ->pred = plt_new;//2nd step;
  plt->succ       = plt_new;//3rd step;

  lt_size++;

  return plt_new;
};
```

FIG. 6.8. 列表上节点的后插入过程。 此时在列表给定的指针plt之后插入一个新的节点，并返回插入节点的地址。 （上图）（a-e）为节点插入过程中指针设置的流程。图中不同颜色带箭头的线段分别表示操作过程的1st step、2nd step、3rd step。图中划线表示被改变的指针。 （下图）为后插入节点过程的参考代码。



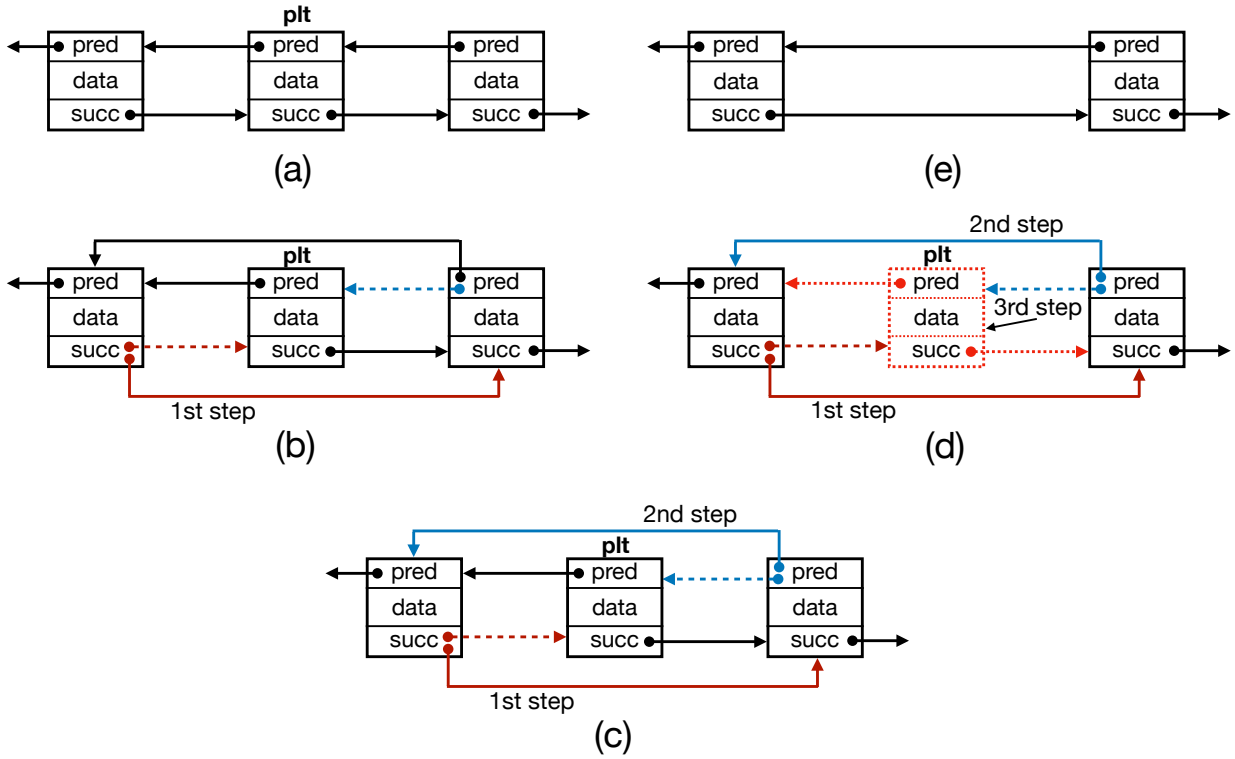

```
//在指定元素的前驱节点的位置插入新节点，并返回其指针；
ListNode* List::InsertAsPred(ListNode* plt, int val)
{
  ListNode* plt_new = new ListNode(plt->pred, plt, val);//1st step;

  plt->pred->succ = plt_new;//2nd step;
  plt->pred       = plt_new;//3rd step;

  lt_size++;

  return plt_new;
};
```

FIG. 6.9. 列表上节点的前插入过程。 此时在列表给定的指针plt之前插入一个新的节点，并返回插入节点的地址。 （上图）（a–e）为节点插入过程中指针设置的流程。图中不同颜色带箭头的线段分别表示操作过程的1st step、2nd step、3rd step。图中划线表示被改变的指针。 （下图）为列表上节点的前插入过程的参考代码。




```
//从列表里移除给定位置的元素，并返回此元素的数据项；
int List::RemoveANodeFromList(ListNode* plt)
{
  int val = plt->val;

  plt->pred->succ = plt->succ;//1st step;
  plt->succ->pred = plt->pred;//2nd step;

  delete plt;//3rd step;

  lt_size--;

  return val;
};
```

FIG. 6.10. 列表里节点的删除。此时在列表中删除给定的指针plt对应的节点，并返回此节点的数据项。（上图）（a–e）为删除节点的流程。图中不同颜色带箭头的线段分别表示操作过程的1st step、2nd step、3rd step。图中划线表示被改变的指针、点线表示被删除的结构体。（下图）为节点删除过程的参考代码。

6.3. 比较向量和列表上的操作时间

向量和列表虽然都是线性数据结构，但两者物理结构的不同，导致它们在操作层面的流程的不同，最终在具体的计算任务里计算时间的不同。这里我们从定量的角度来比较向量和列表上的操作时间。

为了让操作时间具有可比性，我们定义在向量和列表上的操作流程具有类似的流程。我们考虑以下三个基本操作过程：

（1）查询元素：查询序列的最后一个元素。

（2）插入元素：从空序列开始，在序列的首元素位置不断插入新的元素，直到序列到达给定的长度。

（3）删除元素：删除序列的首元素，直到把给定长度的序列里的元素全部删除。

具体的操作流程可见Figure 6.11。具体的操作细节如下：

◦ Figure 6.11(a)表示向量上的查询元素的操作。此时需要查询的元素为向量的最大秩对应的元素。 在向量上，这是一个执行乘法和加法的代数运算后得到内存地址、直接读取内存内容的过程。我们统计的是m次查询末尾元素的全部计算时间之和。

◦ Figure 6.11(b)表示向量上的插入元素的操作。每次操作都是在向量的首元素位置插入新的元素，直到指定的序列长度n。 在向量里，我们使用迭代器来执行在给定位置插入元素的操作。

◦ Figure 6.11(c)表示向量上的删除元素的操作。每次操作都是删除向量的首元素，直到元素全部被删除。 对向量来说，我们使用迭代器来删除向量的首元素。

◦ Figure 6.11(d)表示列表上的查询元素的操作。此时需要查询的元素为列表的末节点。在列表里，查询过程从头节点`header`开始，通过节点的后继指针依次访问列表节点、一直到序列的最后一个节点。

◦ Figure 6.11(e)表示列表上的插入节点的操作。 这里我们使用的是后插入节点的方式，每次在头节点`header`的后方插入新的节点，直到列表里的节点数目达到指定的序列长度。选择这个简单的插入节点的位置，可以避免在列表遍历多个节点后再进行插入节点的操作。

◦ Figure 6.11(f)表示列表上删除节点的操作。此时每次删除的都是列表的首节点，直到所有可用节点都被删除。 在列表上，我们通过不断删除列表的首节点，即头节点`header`的后继节点，来删除节点。同样地，选择这个简单的删除节点的位置，可以避免在列表遍历多个节点后再进行删除节点的操作。

例子2: 向量和列表上插入和删除过程的操作时间

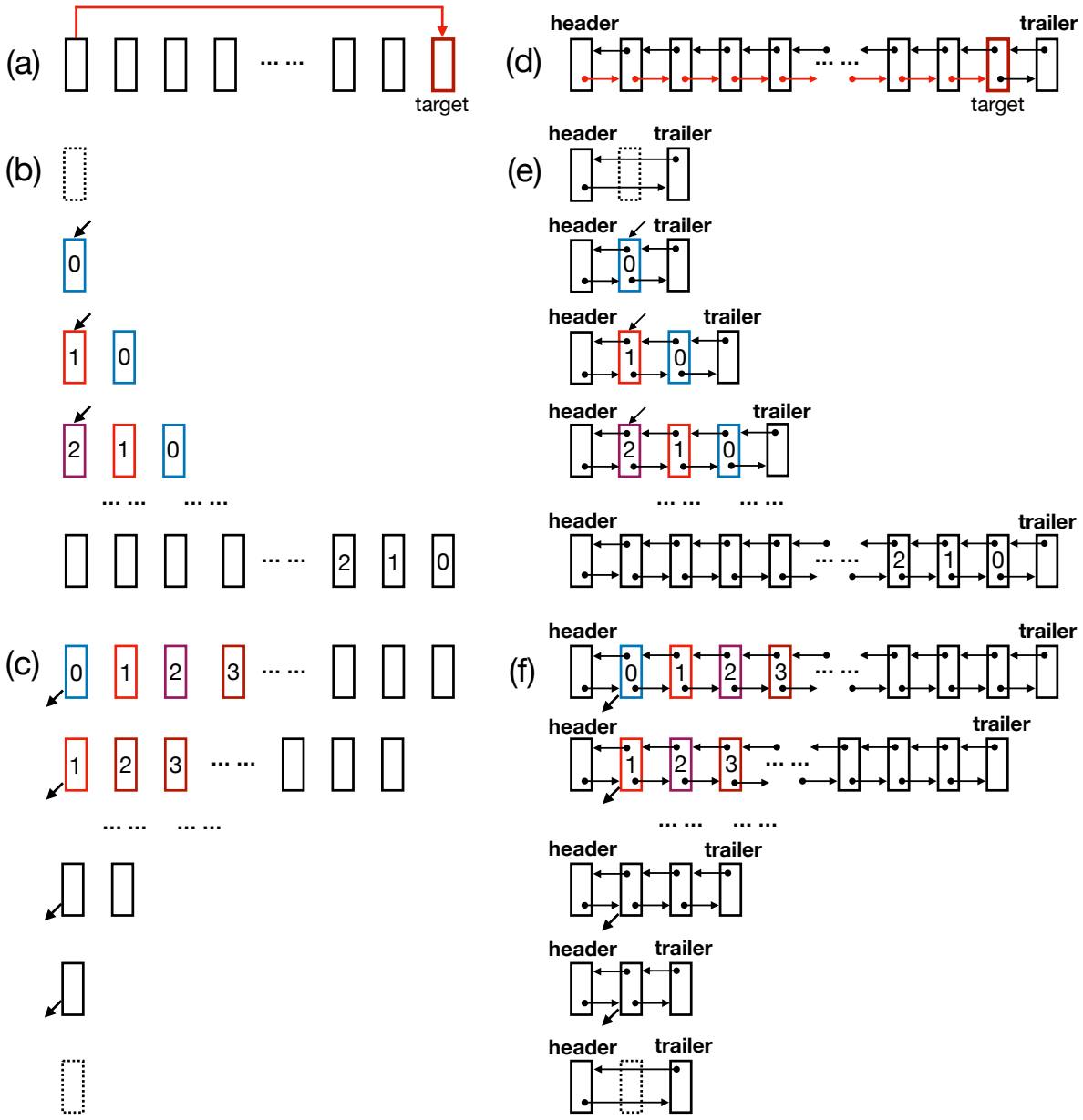


FIG. 6.11. 向量和列表上的静态和动态操作过程。 (a)-(c)分别为向量上的查询末尾元素、连续插入首元素、连续删除首元素的操作。 (d)-(f)分别为列表上的查询末节点元素、连续插入首元素、连续删除首元素的操作。 图中节点方框的颜色和内部数字用来区分插入和删除元素的顺序。 节点方框附近的斜向箭头表示相应节点被插入或者删除。

我们比较了在同样长度的向量和列表上执行查询元素、插入元素、删除元素操作的计算时间。

参考代码：`ch6-list.cc`中main()函数里的OperationTime()。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch6-list.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@向量和列表上操作过程的耗时(微秒/mu secs):
 算法参数: 序列长度 =  100000; 查询次数 =  10000
 (1) 向量:
     总时间: 查询元素 =            7, 插入元素 =       305229, 删除元素 =       274319
 总时间/n^2: 查询元素 =            -, 插入元素 =  3.05229e-05, 删除元素 =  2.74319e-05
 (2) 列表
     总时间: 查询元素 =       734552, 插入元素 =          934, 移除元素 =          450
   总时间/n: 查询元素 =      7.34552, 插入元素 =      0.00934, 删除元素 =       0.0045
 (3) 向量/列表
     总时间: 查询元素 =  9.52962e-06, 插入元素 =      326.798, 移除元素 =      609.598
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch6-list.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@向量和列表上操作过程的耗时(微秒/mu secs):
 算法参数: 序列长度 =  200000; 查询次数 =  10000
 (1) 向量:
     总时间: 查询元素 =           11, 插入元素 =  1.16112e+06, 删除元素 =  1.11611e+06
 总时间/n^2: 查询元素 =            -, 插入元素 =  2.90279e-05, 删除元素 =  2.79027e-05
 (2) 列表
     总时间: 查询元素 =    1.576e+06, 插入元素 =         1916, 移除元素 =          925
   总时间/n: 查询元素 =      7.87998, 插入元素 =      0.00958, 删除元素 =     0.004625
 (3) 向量/列表
     总时间: 查询元素 =  6.97971e-06, 插入元素 =      606.011, 移除元素 =       1206.6
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch6-list.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@向量和列表上操作过程的耗时(微秒/mu secs):
 算法参数: 序列长度 =  400000; 查询次数 =  10000
 (1) 向量:
     总时间: 查询元素 =            8, 插入元素 =  4.56689e+06, 删除元素 =   4.5558e+06
 总时间/n^2: 查询元素 =            -, 插入元素 =  2.85431e-05, 删除元素 =  2.84738e-05
 (2) 列表
     总时间: 查询元素 =  2.78446e+06, 插入元素 =         3730, 移除元素 =         1839
   总时间/n: 查询元素 =      6.96114, 插入元素 =     0.009325, 删除元素 =    0.0045975
 (3) 向量/列表
     总时间: 查询元素 =  2.87309e-06, 插入元素 =      1224.37, 移除元素 =      2477.33
```

FIG. 6.12. 向量和列表上操作过程的计算时间。 图中展示了向量和列表分别在查询、插入、删除元素三种操作过程中的计算时间，其相对时间相对于序列长度的比例，以及两者计算时间的比例。 时间单位为微秒($1\mu s = 10^{-6}s$)。 图中依次考虑的序列长度分别为$n = \{1, 2, 4\} \times 10^5$的计算结果。在查询元素的操作里，查询末元素的次数均为$m = 10^4$。

在代码里，为了在一个完整的流程里能够执行完三个操作，对于向量和列表，我们首先分别使用插入元素的方式来生成序列，之后在其上多次查询元素，最后不断删除序列的首元素直到序列为空。

代码的计算结果可见Figure 6.12。我们可知:

（1）向量上查询元素是一个非常快速的、不依赖于序列长度和查询位置的操作。

（2）列表上查询末节点的计算时间正比于序列长度n，约为$7 \times 10^{-4} n$ μs。

（3）向量上的插入和删除首元素的操作时间正比于序列长度n的平方，约为$3 \times 10^{-5} n^2$ μs。

（4）列表上插入和删除元素的操作时间正比于序列长度n，分别约为$0.009n$ μs和$0.0046n$ μs。

以上的计算结果从定量角度验证了之前我们所做的“列表上的静态操作会慢于向量、但动态操作会快于向量”的结论。

下面是一些直观的解释:

（1）对于上述向量上连续插入首元素的操作，每次插入新元素前，需要把序列后续元素依次往后挪一位。插入操作的时间主要来源于元素的移动。对于长度为n的序列，生成过程中需要移动元素的总次数为:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \propto n^2. \tag{6.1}$$

（2）对于上述向量上连续删除首元素的操作，每次删除首元素后，序列的后续所有元素需要往前依次挪动。如此删除长度为n的序列需要挪动元素的总次数为:

$$\sum_{i=n-1}^{1} i = \frac{n(n-1)}{2} \propto n^2. \tag{6.2}$$

（3）对于上述列表上插入和删除元素的过程，其基本操作是释放一个节点的内存、重新设置前后节点之间的指针。这是常数级的操作时间。因此对于生成或者删除长度为n的序列，都只需要$\propto n$的操作时间。

6.4. 列表的应用

以下介绍几个列表有关的应用例子。

■ 列表

■ 列表的倒置

参考：邓俊晖《数据结构习题解析》，第3章3.18题。

给定元素个数为n的一个列表Lt，请倒置这个列表。

解答:

我们首现用一个示意图来解释倒置列表时，列表节点中的`pred`和`succ`的变化。 主要步骤的图示见Figure 6.13。

对于一个给定的列表，算法主要经过两次遍历来改变每个元素内的指针。

• 在第一次遍历里，从`header`到`trailer`，把每个节点的`pred`依次设为最初顺序里它的`succ`，并重设`trailer`的`pred`。 此时的遍历过程基于的是列表里没有被修改的`succ`信息。

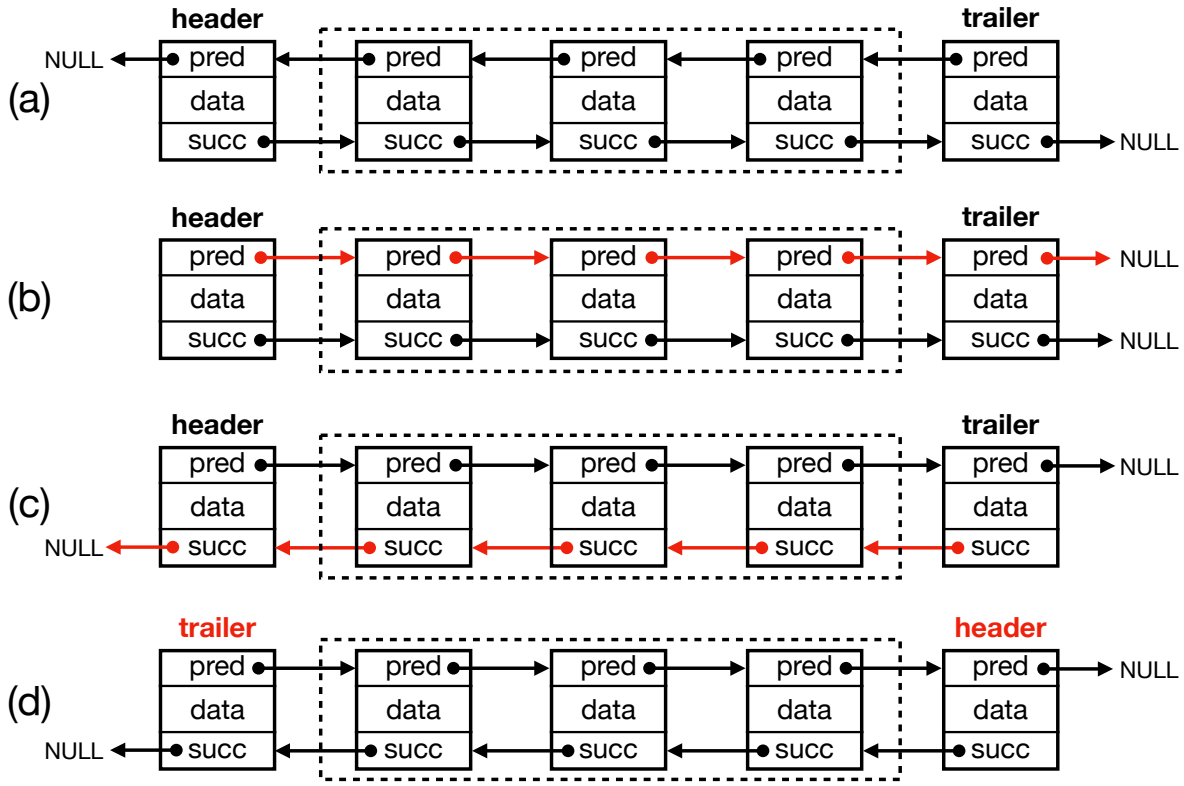


FIG. 6.13. 列表倒置的操作过程。 列表节点里的pred、succ、data分别为列表单元的前驱指针、后继指针、数据项。 (a)为元素个数$n = 3$的原始列表，其同时包括了头节点和尾节点(header和trailer)。 (b)对应对列表的第一次遍历，以依次改变每个元素内的pred。 (c)对应对列表的第二次两次遍历，以依次改变每个元素内的succ。 (d)对应头尾元素对应指针的交换，最终就得到了倒置后的列表。 图中红色部分标记了每个子图中需要改变的量。

- 在第二次遍历里，从header到trailer，把每个节点的succ依次设为最初顺序里它的pred，并重设header的succ。 此时的遍历过程基于的是列表里刚被修改了的pred信息。
- 两次遍历之后，交换列表的header和trailer。最终得到倒置后的列表。

例子3: 列表的倒置

生成给定元素个数n的、元素大小随机的一个列表，之后对列表进行倒置。

参考代码：ch6-list.cc中main()函数里的Reversion()。

计算结果见Figure 6.14。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch6-list.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@初始列表：
{  7, 19, 23,  8, 10,  2, 24,  8, 23, 19}
@倒置后的列表：
{ 19, 23,  8, 24,  2, 10,  8, 23, 19,  7}
```

FIG. 6.14. 列表倒置的输出结果。 这里测试了一个$n = 10$、由随机生成的整数（范围为$[0, 20)$）构成的列表。 代码输出了初始列表的元素、以及倒置之后的列表的元素。

习题

6.1 列表中插入和删除节点的操作

在列表上执行插入和删除节点等动态操作时，需要设置节点的多个指针。为了避免列表的结构出错，需要设计好操作流程中的各个步骤。

考虑以下三个动态操作及其中的两个步骤：

（1）Figure 6.8中后插入节点操作的2nd step和3rd step；

（2）Figure 6.9中前插入节点操作的2nd step和3rd step；

（3）Figure 6.10中移除节点操作的1st step和2nd step。

请回答以下问题：

（a）如果互换了上面小题内的两个步骤，列表的结构会成为怎样?

（b）如果此时列表的结构出错、但在代码中依然首先运行原先的第二个步骤，那我们将如何相应地修改原先的第一个步骤来使得操作依然能够正确地进行?

注意：此题需要完成必要的示意图及文字说明，无需写代码。

6.2 有序列表的合并

向量和列表同属线性数据结构。本质上，能用向量表示的数据和实现的算法，也能用列表来完成。 这里我们用列表来实现归并排序算法里的二路归并过程。请参考Figure 5.38中的二路归并过程的算法流程。 假设当下有两个由整型数构成的有序列表Lt1和Lt2，请把Lt1直接合并到Lt2里，得到合并后的Lt2。 请使用双链表的形式来实现上述过程，并输出合并后的Lt2。 作为测试，请使用代码实现以下列表的合并：

$$\mathrm{Lt1} = \{4, 5, 7, 10\}, \tag{6.3}$$

$$\mathrm{Lt2} = \{2, 5, 8, 12, 13\}. \tag{6.4}$$

使用代码来实现算法时，可以参考课堂演示代码里列表的头文件`list-class.h`。算法中请直接将列表Lt1合并到Lt2，而不能通过“首先复制Lt1的节点、然后把它们插入Lt2”的

方式来实现。算法流程可参考Figure 6.15。注意此示意图中的列表使用了单链表的形式。为了保证排序算法的稳定性，对于序列中的重复元素，我们把Lt1中的这类节点放在Lt2相应节点之前。请查看示意图对5这个元素的处理。

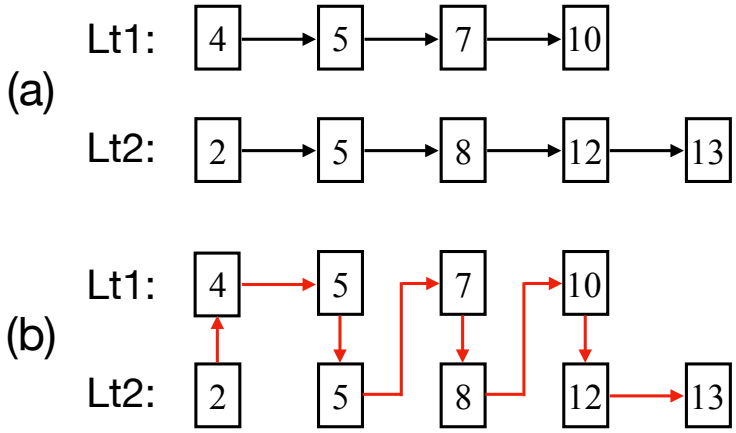


FIG. 6.15. 单链表上的二路归并过程。列表节点中的数字为其数据成员。 (a)为两个需要合并的有序序列Lt1和Lt2。 (b)为重新设置指针后的Lt和Lt2，其中Lt2表示列表合并之后的序列。图中的箭头表示相应节点的后继指针。

上述直接合并列表的过程显然不同于基于向量的二路归并过程，因为前者算法直接操作指针来合并序列、无需借用某个辅助数据结构，而后者得额外构造一个和原始向量一样长度的辅助向量来存储合并过程中的所有有序向量。容易看出来，上述基于列表的有序序列的合并算法是一个就地算法。

7. (STRING)

参考内容：邓俊辉教材第11.1-3节。李春葆教材第4.1-3节。

本章主要介绍字符串的含义、随机字符串的生成、串匹配问题的蛮力算法、串匹配的KMP算法。

字符串本质上是一个线性结构，可以使用向量或列表来实现。但因为其在应用中有很多具体场合，所以会有相应的问题和相应算法。

之后我们讨论了随机字符串的生成，也就是基于字符集的概率分布来生成带有随机的字符串。这种字符串可以像随机有序序列和随机无序序列那样用来作为算法的对象来对不同算法进行比较。

之后我们定义了字符串的匹配问题，以及相应的蛮力算法和KMP算法。所有求解问题的算法在于如何在所有可能的解构型所组成的空间里寻找到真正的解。蛮力算法的逻辑是对所有可能解进行对比，而KMP算法则利用了已经比较过的子串元素所积累的信息来排除掉某些可能的解，使得算法可以在一个更小的可能解所构成的空间上运行。KMP算法借用的中间数据是Next表，包括初版Next表和更新版Next表。两个版本的Next表都能实现模式串相对于文本串的多步移动，但后者进一步考虑到滑动模式串后元素对的相等与否、能够继续跳过不可能匹配的子串，因而可以在一个更小的可能解构成的空间内进行搜索，最终导致算法可以执行得更快。

7.1. 字符串的基本性质

■ 背景

计算机处理的信息：数字、字符、音频、视频等等。有更大量的数据是以字符形式存在的。

和字符串相关的后续专业课程：

- 自然语言处理(Natural Language Processing)
- 信息检索(Information Retrieval)

■ 基本概念

由n个字符构成的字符串(string)记为S =“$a_1a_2\cdots a_{n-1}$”，其中$a_i \in \sum$，$i \in [0, n)$

$\sum$为字符表(alphabet)，其大小可定义为D。比较常见的字符表为二进制数、ASCII码、Unicode字符集等等。

◦ 字符串S的子串(substring)：Sub =“$a_ma_2\cdots a_k$”，其中$1 \leqslant m \leqslant k \leqslant n-1$。

◦ 字符串S的前缀(prefix)：Pre =“$a_1a_2\cdots a_k$”，其中$1 \leqslant k \leqslant n-1$。

◦ 字符串S的后缀(suffix)：Suf =“$a_ka_2\cdots a_{n-1}$”，其中$1 \leqslant k \leqslant n-1$。

◦ 字符串里的判等：如何判断长度的n的字符串S和长度为m的字符串T?

7.2. 字符串的ADT

■ C++ STL: string

string作为C++ STL里的一种容器，能够字符串上的求长度、返回子串等等功能。其用法可参考：

`https://zhuanlan.zhihu.com/p/353910845`

7.3. 随机字符串的生成

1. 为何需要随机字符串?

首先，我们为什么需要随机字符串?

这个问题的出发点和我们在讨论排序算法时需要生成随机序列的出发点是一样的。当我们在讨论单个算法的时间复杂度时，我们需要分辨算法的表现到底是因为问题对象（比如排序问题中的待排序序列）本身内部的结构，还是因为算法本身的设计。同样的情况也出现在我们在比较不同算法的表现的时候。此时，如果问题对象带有随机性，那么可以认为问题对象本身是没有内部结构的。此时算法的表现依赖于算法本身，对应着它的平均复杂度。

2. 问题的描述

字符组成字符串。字符串最终会形成更加复杂的词语、句子，最终形成语言。容易得之，字符串天生就是内部结构的。如果使用概率来描述字符串，我们可以定义它的单个字符概率、两体字符的概率、三体字符的概率等等。为了让问题简单化，我们考虑的是字符串中的单个字符的概率。给定一个文本，我们可以将其视为一个很长的字符串T。考虑其字符表大小为D的字符表$S = \{s_0, s_1, \cdots, s_{D-1}\}$。相应地我们可以很容易统计出来其中单个字符的概率，也即是其字符表的离散概率分布(discrete probability distribution)$P(S) = \{P(s_0), P(s_1), \cdots, P(s_{D-1})\}$。比如对于英文的论文或者书籍，我们可以统计其英文字母（不区分大小写）出现的概率。此时$D = 26$。对于中文的文本，我

们可以单个文字出现的概率。此时中文的字符集是个很大的书，比如$D \geqslant 5000$。这是从字符串到概率分布的过程，即$\mathrm{T} \rightarrow \{S, P(S)\}$。但在随机字符串生成过程中，我们考虑的是相反的过程，也即是从概率分布到字符串的过程，即$\mathrm{T} \leftarrow \{S, P(S)\}$。

随机字符串生成问题的描述如下： 给定了一个字符集S及其概率分布$P(S)$，我们如何生成一个给定长度的字符串T，使得其最终的字符表中字符的出现概率尽可能地和$P(S)$接近。

Figure 7.1给出了基于实际文本来生成随机字符串的一个例子。

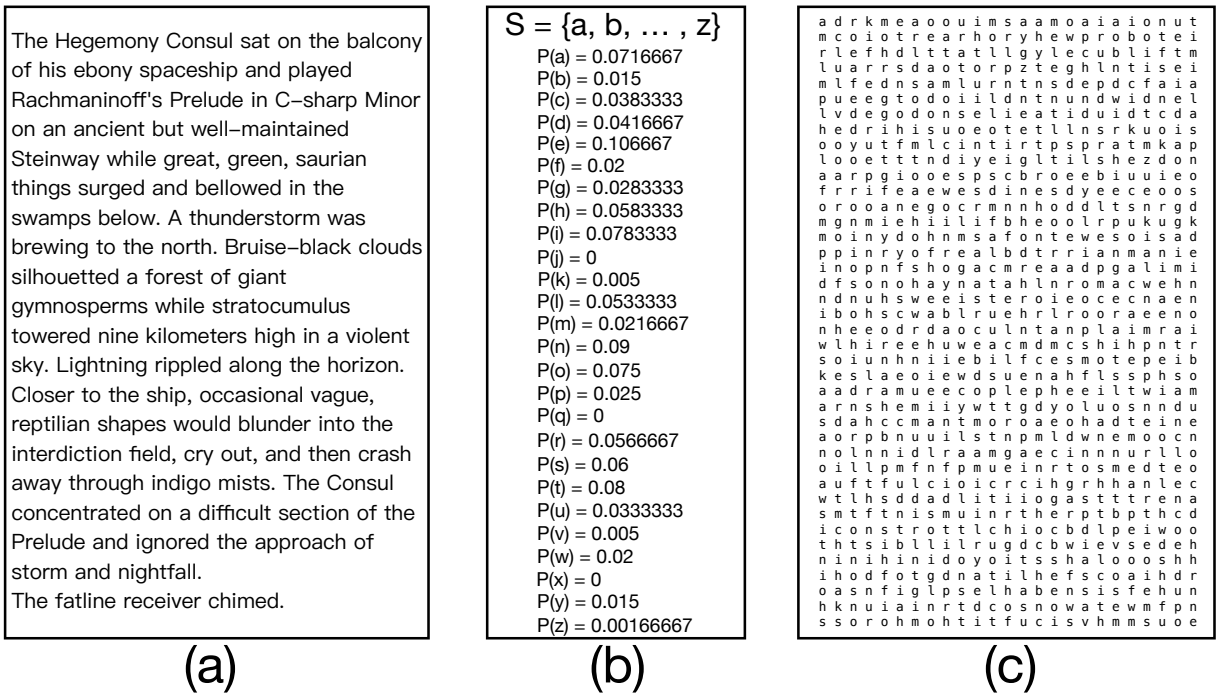


FIG. 7.1. 从实际文本、字符概率、到随机字符串的过程。 (a)为原始文本的两段文字。摘自Hyperion（《海伯利安》）的Prologue（序言）的第一二段文字。此科幻小说为作家Dan Simmons（丹·西蒙斯）于1989年的作品。这段文本中共600个英文字符、17个标点符号。 (b)为基于(a)统计得到的字母概率。我们只考虑英文字符，且不区分大小写英文字符。。S为字符集，包括26个英文字母。P(a)到P(z)表示相应英文字符的字母概率。在统计过程中，我们直接统计每个英文字符的个数，最后计算其在英文字符总数里的所占比例。 (c)为利用(b)的字母概率所生成的随机字符串。此字符串共1000个字符，其中每行25个字符、共40行。

例子1: 统计给定文本中英文字符的概率

给定Figure 7.1(a)所示的文本，统计26个英文字符所占的比例。

文本所在的路径为：./ch7-data/text-hyperion-prologue.txt

参考代码：`ch7-string.cc`中的ExtractPSFromTexts()函数。

3. 生成序列的算法

在具体的算法里，我们利用一个随机数生成器来生成一个字符串T。整个算法的大致流程如下：我们定义字符表概率对应的累积概率，从而得到这些累积概率组成有序序列；之后我们通过随机数生成器来不断生成$[0,1)$之间的随机数，通过查找相应随机数所对应的累积概率的下标、以及对应的字符表中的字符。

以下为我们从字符集概率分布到字符串的过程：

（1）累积概率(cumulative probability distribution)的计算：假如$P(S)$已经归一化，我们把$P(s_i)$以大小为长度、按照$i=0,1,\cdots,D-1$的顺序拼接成一条头尾相连的链。可知它们刚好构成$[0,1]$区间。此时我们可以定义$P(s_i)$对应的累积概率分布$P_c(s_i)$，$i\in\{0,1,\cdots,D-1\}$，意为某个字符$s\in\{s_0,s_1,\cdots,s_i\}$的概率之和，即

$$P_c(s_i)\equiv\sum_{j=0}^{i}P(s_j). \tag{7.1}$$

容易得知

$$P_c(s_{D-1})=\sum_{j=0}^{D-1}P(s_j)=1. \tag{7.2}$$

可知$P_c(S)$对应了一个大小为D的有序序列。

（2）使用随机数生成器：此时我们只考虑单个字符的概率，则算法中字符串里每个字符可以独立生成。假如我们手头有一个能够生成$(0,1)$内均匀分布实数的随机数生成器Rand()。每次在生成某个字符s时，我们首先生成一个数$t=\text{Rand}()\in(0,1)$。容易可知，t落在某个$P(s_i)$对应区间内的概率刚好等于$P(s_i)$的大小。

这里简单证明采取上述查找过程里返回相应字符下标的方式是能够生成给定字符概率的字符串。因为随机数$t\in(0,1)$是均匀分布的，t落入下标i对应区间的概率就是区间$(P_c(s_{i-1}),P_c(s_i)]$的长度。可知它的长度为$P_c(s_i)-P_c(S_{i-1})$。按照累积概率的定义，此长度即$P(s_i)$。

（3）从随机数到字符的查找过程：对于前一过程里的任意数t，我们把t在$P_c(S)$这个数组里进行比对。对于某个下标i，如果满足$P_c(s_{i-1})<t\leqslant P_c(s_i)$，则返回此下标$i$，即生成相应的字符$s=s_i$。

在上述生成过程中，需要确定一个随机数$t \in (0,1)$所对应的字符下标。这里需要从一个由累积概率构成的、长度为D的有序序列里找到对应的下标。此时的场景对应一个由双精度类型数元素背景下的查找问题。注意到我们需要查找的随机数t对应的下标（即字符s_i里的i）为$\geqslant t$的元素中最小的下标。

此时我们可以使用以下方法：

（3.a）顺序查找算法：对于t，可以遍历$P_c(s_i)$，直到找到某个下标满足$P_c(s_{i-1}) < t \leqslant P_c(s_i)$。则对应$i$为返回下标。

（3.b）二分查找法（版本C）：容易可知，寻找某个随机数$t \in [0,1)$所对应的字符s_i意味着在字符集的累积概率构成的有序序列中查找$\geqslant t$的元素中最小的下标i。回想下里参数(elem, lo, hi) = (t, 0, D)情况下返回的参数lo。此时lo为$\leqslant t$的序列元素的最大下标。这个算法可以同时处理查找元素为重复元素、查找元素不存在这些特殊情形。在双精度类型变量情形下，我们可以忽略等号的情况。此时下标lo +1对应的就是$> t$的序列元素的最小下标，也就是此时查找问题需要返回的字符集下标。在实际计算中，对于任意随机数t，我们使用二分查找法（版本C）得到参数(elem, lo, hi) = (t, 0, D)对应的返回值lo，相应地lo+1即为t对应的字符表下标。

（4）上述第2–3步骤重复N次后，我们可以得到一个长度为N的文本串T。

Figure 7.2使用图示的方式展示了从字符集概率分布到字符串的生成过程。

按照上述算法生成一个字符串T后，我们还需要量化它的经验字符概率在多大程度上符合给定的字符概率$P(S)$。我们首先统计出字符串T种所有字符s_i出现的经验概率$P_{\rm em}(S) \equiv \{P_{\rm em}(s_i), i = 0, 1, \cdots, D-1\}$，然后将字符集里对应的经验概率和给定概率的差的绝对值进行求和，即

$$P_{\rm diff} \equiv \sum_{i=0}^{D-1} |P_{\rm em}(s_i) - P(s_i)|. \tag{7.3}$$

可知当$P_{\rm diff}$越小时，生成字符串的字符集概率越接近给定字符集概率。可知当$P_{\rm diff} = 0$时，生成字符串的字符集概率完全符合给定字符集概率。

4. 代码实现

源代码可参考：`ch7-string.cc`中的GenerateStrings()函数。

在这个函数里，字符集里字幕的相对概率存放在源数据文件里。生成程序需要首先读取这个参数文件，归一化字符概率，之后再执行字符生成过程。代码分别处理了以下两个字符串的生成问题，分别对应$D = 2$和$D = 26$。

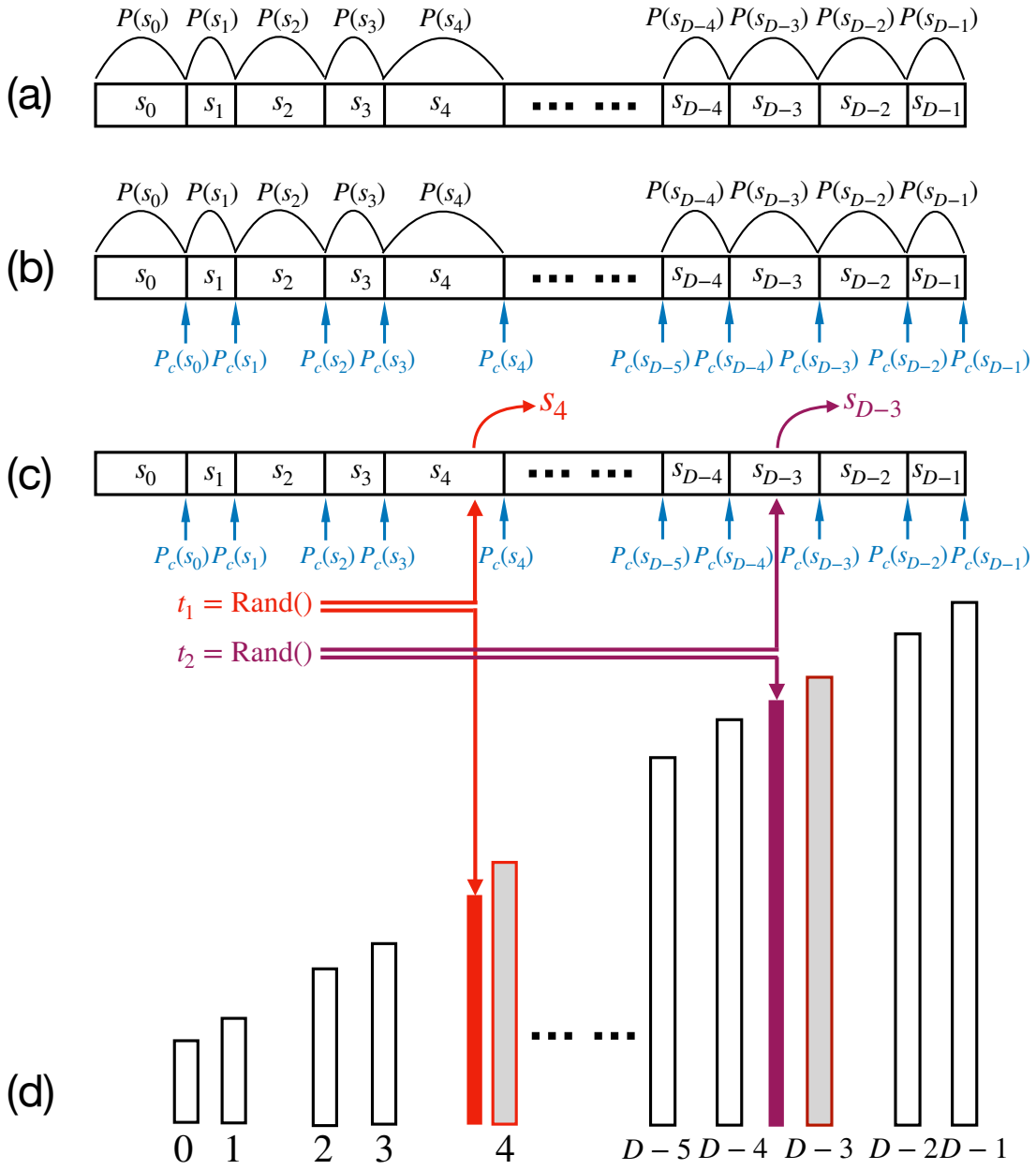


FIG. 7.2. 从字符集概率到字符串的生成过程。 这里字符集为$S = \{s_0, s_1, \cdots, s_{D-1}\}$，$D$为字符集的大小，$P(s_i)$为字符$s_i$对应的概率，其中$i = \{0, 1, \cdots, D-1\}$。 (a)展示了字符集概率。此时每一个子段对应一个字符、其宽度为其概率大小。 (b)为从字符的概率到累积概率的过程。$P_c(s_i)$为字符s_i对应的累积概率。 (c)和(d)为基于$(0, 1)$之内的一个实数寻找其对应字符的过程，其中后者为前者的柱状图形式。图中对于Rand()输出的实数t_1和t_2，通过在累积字符概率组成的有序序列上使用查找法可知其对应字符分别为s_4和s_{D-3}。

例子2: 生成满足伯努利分布(Bernoulli Distribution)的二元字符串

伯努利分布为只有两个变量取值的离散分布。其概率分布为

$$P(x) = p\delta(x,0) + (1-p)\delta(x,1). \tag{7.4}$$

上式中$\delta(x,t)$或者δ_{xt}为克罗内克符号(Kronecker symbol)，满足:

$$\delta(x,t) = \begin{cases} 1, \text{ if } x = t; \\ 0, \text{ else.} \end{cases} \tag{7.5}$$

可知式子Eq.(7.4)的含义即为变量x取值为$x = 0$时的概率为p、$x = 1$的概率为$1 - p$。

上式中的1和0可以用来表示任何二元状态的取值。比如在一些实验中，可以认为1为“实验成功”、0为“实验失败”。在这里，我们可以用来表示大小$D = 2$时的字符集。我们可以按照给定两个字符的相对概率值来生成字符串。

例子3: 生成满足给定字母概率(letter frequency)的英文字符串

在大量文本信息中，如果我们只考虑英文字母、且不区分它们的大小写，那么我们可以得到这些文本信息种的字母概率。这里我们利用这些经验统计概率来生成随机字符串。已知字母概率分布大致如下:

TABLE III. 英文字母的出现频率。$L(\in \{\text{a, b, } \cdots, \text{z}\})$为字母，$p_L$为其出现概率。

L	p_L	L	p_L	L	p_L	L	p_L	L	p_L	L	p_L	L	p_L
a	8.167%	b	1.492%	c	2.782%	d	4.253%	e	12.702%	f	2.228%	g	2.015%
h	6.094%	i	6.966%	j	0.153%	k	0.772%	l	4.025%	m	2.406%	n	6.749%
o	7.507%	p	1.929%	q	0.095%	r	5.987%	s	6.327%	t	9.056%		
u	2.758%	v	0.978%	w	2.360%	x	0.150%	y	1.974%	z	0.074%		

上述Table III中字母概率分布已存放在数据文件里。 Figure 7.3展示了26个英文字母对应的字符概率的柱状图，包括原始字母概率、以及按字母概率大小重新排列后的结果。

基于上述给定的26个字符的概率分布，我们将生成相应的随机字符串。

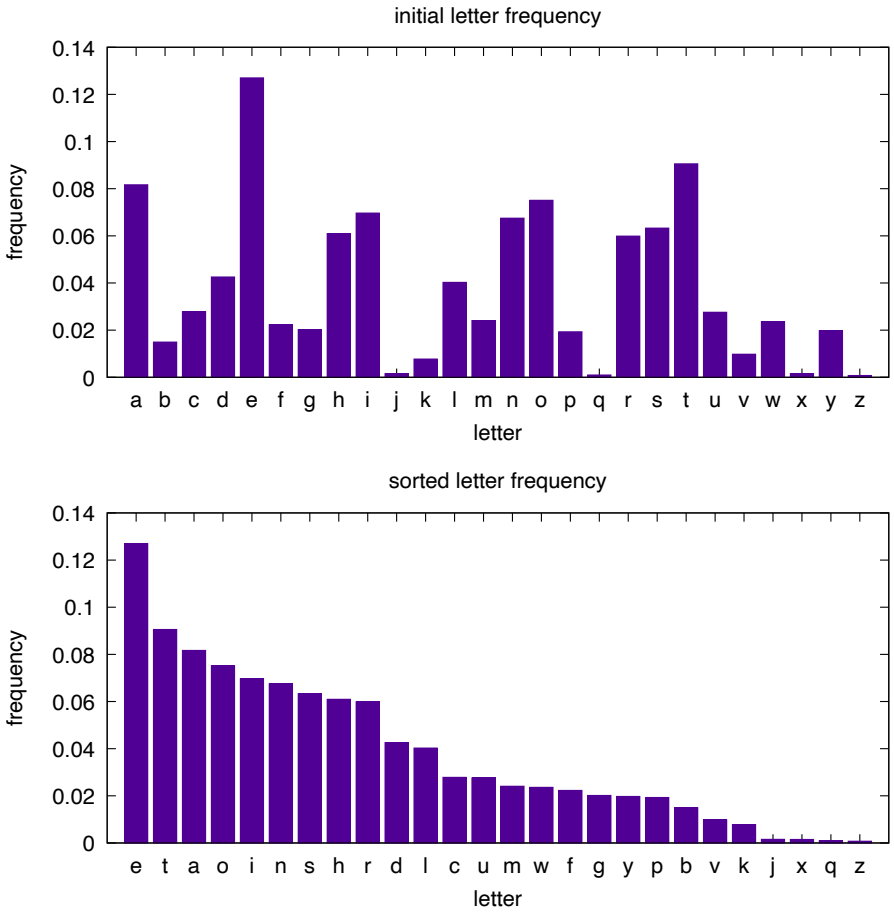


FIG. 7.3. 英文字符的字母频率。 上图为26个字母的统计概率对应的柱状图，下图为按照字母频率从高到低重新排列后得到的柱状图。

7.4. 串匹配问题

串匹配（string matching）问题描述：在给定的文本里找出指定的字符串。后者称为模式串。

比如在在给定文档里搜索到相应的文字。

> “真实的宇宙就是这么黑。”罗辑伸手挥挥，像抚摸天鹅绒般感受着黑暗的质感，“宇宙就是一座黑暗森林，每个文明都是带枪的猎人，像幽灵般潜行于林间，轻轻拨开挡路的树枝，竭力不让脚步发出一点儿声音，连呼吸都小心翼翼……他必须小心，因为林中到处都有与他一样潜行的猎人。如果他发现了别的生命，不管是不是猎人，不管是天使还是魔鬼，不管是娇嫩的婴儿还是步履蹒跚的老人，也不管是天仙

般的少女还是天神般的男孩，能做的只有一件事：开枪消灭之？在这片森林中，他人就是地狱，就是永恒的威胁，任何暴露自己存在的生命都将很快被消灭。这就是宇宙文明的图景，这就是对费米悖论的解释。”

大史又点上了一支烟，仅仅是为了有点光明。

“但黑暗森林中有一个叫人类的傻孩子，生了一堆火并在旁边高喊：我在这儿！我在这儿！”罗辑说。

摘自：《三体2：黑暗森林》，刘慈欣著

如何选择**模式串P**：在**文本串T**内选取给定长度的字符串做为模式串P。

串匹配问题、乃至其它类型问题，可定义成以下不同的版本：

（1）解的检测(detection)：问题是否有解?

例如：在上述给定文本里，是否存在“黑暗森林”这个模式串？答案为：“是”。

（2）解的定位(location)：寻找到一个解的构型(configuration)。注意此问题可能有多个解。

例如：输出文本串中第一个“黑暗森林”模式串的位置。这里“解的构型”对应着文本串中模式串对应的起始位置。答案为：“43”。此时下标从0开始计数，而且考虑标点符号、换行符，不考虑空格。

（3）解的计数(counting)：计算所有的解的总数。

例如：输出文本串中模式串“黑暗森林”的数目。答案为：“2”。

（4）解的枚举(enumeration)：输出所有的解的构型。

例如：输出文本串中所有模式串“黑暗森林”的位置。答案为：“43, 298”。

上面四种版本的问题的求解难度是不断增加的。

- 串匹配问题的复杂度

串匹配过程的执行过程可参考Figure 7.4。对于长度为N的文本串T和长度为$M(\ll N)$的模式串P，解空间的大小（即全部可能解的数目）对应着文本串中长度为M的子串的数目，即$N-M+1$。容易得知，在最简单的情况下，文本串的第一个长度为M的子串即为模式串。此时只需要进行M次元素的比较。而在最坏的情况下，文本串的最靠后的长度为M的子串才为模式串。此时需要进行$(N-M+1)\times M$次元素的比较。由此可知，此问题的解空间和问题规模N的大小成线性关系。串匹配算法的时间复杂度也大致写

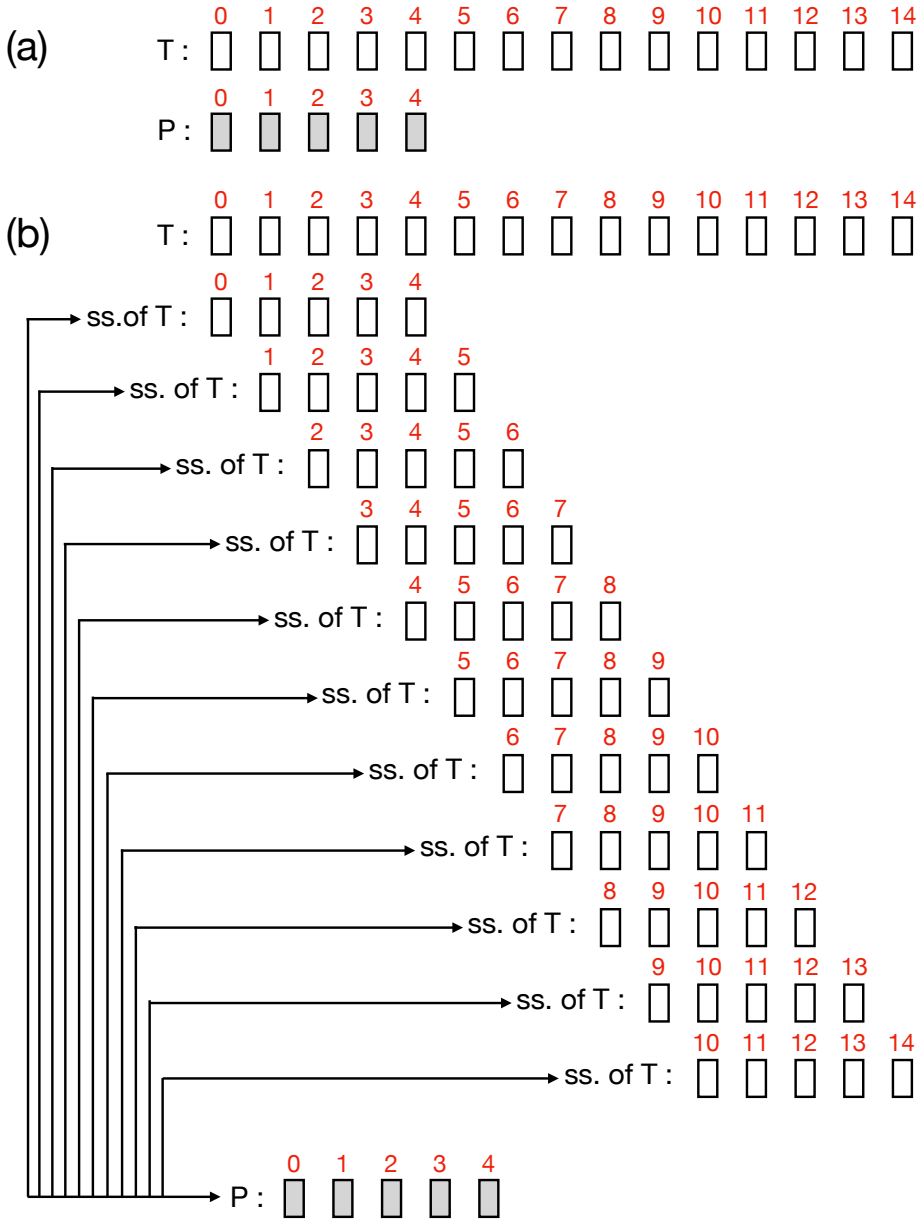


FIG. 7.4. 串匹配问题的求解过程。 (a)文本串T的大小为$N = 15$，模式串的大小为$M = 5$。 (b)文本串T里的$S = N - M + 1 = 11$个长度为M的子串都为匹配问题的可能解。 在求解串匹配问题时，可以逐个将这S个文本串子串和模式串P做对比，直到从这些子串里找到真正的解。 图中的符号ss表示子串(substring)，两端带箭头的线段表示作为可能解的文本串子串和模式串的比较。

为$\mathcal{O}(NM)$。 所以，本质上串匹配算法是个简单问题。

7.5. 串匹配问题的蛮力算法

这里我们介绍两个版本的蛮力算法(Brute-Force Algorithm)。 蛮力算法（版本A）对应着邓俊晖参考书里P310的代码11.2“蛮力串匹配算法（版本二）”。 蛮力算法（版本B）对应着邓俊晖参考书里P309的代码11.1“蛮力串匹配算法（版本一）”。 两个蛮力算法遵循同样的逻辑：

（1）算法把文本串T里长度为M的子串都认为是可能的解，然后依次和模式串P进行对比。容易得知，这些子串的数目为共有$N - M + 1$。

（2）在比较文本串子串和模式串时，我们逐个字符地对比它们的字符元素。如果某个位置的对应字符相同，则继续比较下一个位置的字符；但发现有对应字符不相同，说明此子串并不是模式串，则停止比对这个子串、并开始下一个可能解的比对。

两个算法在表述上不一样的在于如何使用下标来比较两字符串的元素。 考虑文本串T的下标为i，模式串P的下标为j。 在蛮力算法（版本A）里，需要比较的元素为T$[i+j]$和P$[j]$。此时意味着P$[0, j-1]$和T中的元素对应相等。考虑以下两种情况：

（A.1）如果T$[i+j]$ = P$[j]$：$i+j$和j往后挪一步，以比较下一对元素；

（A.2）如果T$[i+j] \neq$ P$[j]$：P的下标j挪回到首元素位置$j = 0$、T的下标i往回挪到P[0]对应的位置。

在蛮力算法（版本B）里，需要比较的元素为T$[i]$和P$[j]$。此时意味着P$[0 \sim j]$和T中的元素对应相等。 考虑以下两种情况：

（B.1）如果T$[i]$ = P$[j]$：下标i和下标j往后挪一步，以比较下一对元素；

（B.2）如果T$[i] \neq$ P$[j]$：P的下标j设置到首元素位置$j = 0$、T的下标i往回挪到P[0]对应的位置。

Figure 7.5展示了上述描述的蛮力算法两个版本中下标的变化过程。

Figure 7.6展示了上述描述的蛮力算法两个版本中下标的变化过程。

Figure 7.7展示了在给定同样的文本串和模式串后，两个版本算法的执行过程。

例子4: 串匹配问题的蛮力算法

给定文本串和模式串，使用蛮力算法的两个版本来寻找模式串的位置。

代码可参考：`ch7-string.cc`中的BruteForceAlgorithm()函数。

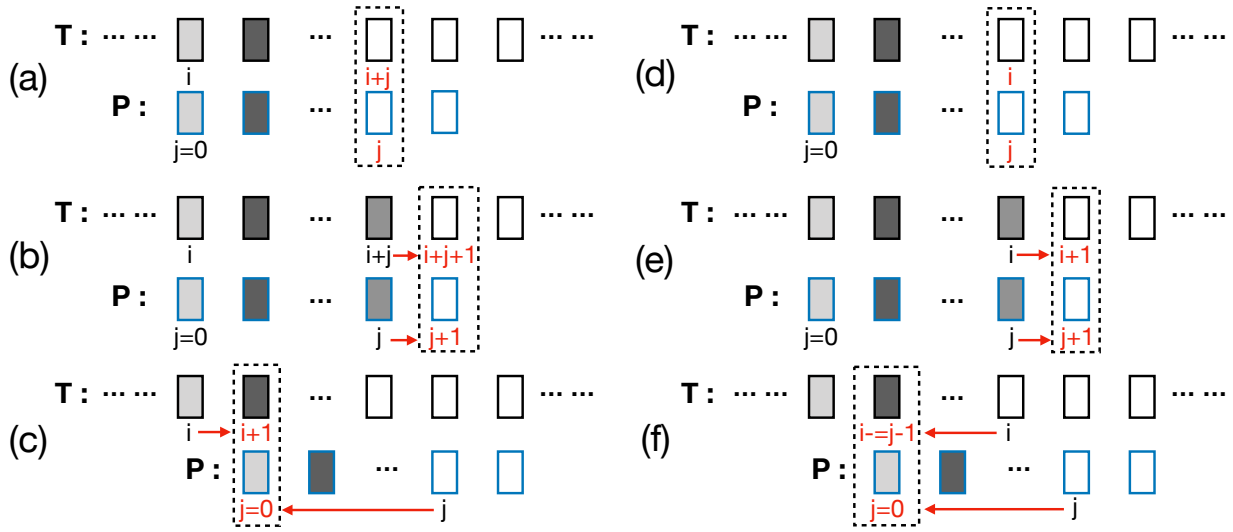


FIG. 7.5. 蛮力算法中元素匹配的基本步骤。 (a-c)对应着蛮力算法（版本A）中的过程。 (d-f)对应着蛮力算法（版本B）中的过程。 图中虚线方框包含的两个元素表示当下需要比较的一对元素。

```cpp
//BFProcedure1(): 比较T[i+j]和P[j];
int String::BFProcedure1()
{
  int i = 0;
  int j = 0;

  for (i = 0; i <= N - M; i++) {
    //依次匹配相应的T和P中的字符;
    for (j = 0; j < M; j++) {
      if ( T[i + j] != P[j] ) {
        break;
      }
    }

    //匹配完成后退出循环;
    if (j == M) {
      break;
    }
  }

  return i;
};
```

```cpp
//BFProcedure2(): 比较T[i]和P[j];
int String::BFProcedure2()
{
  int i = 0;
  int j = 0;

  while (i < N && j < M) {
    if (T[i] == P[j]) {
      i++;
      j++;
    } else {
      i = i - j + 1;
      j = 0;
    }
  }

  return i - j;
};
```

FIG. 7.6. 蛮力算法中元素匹配的参考代码。 左图和右图别对应蛮力算法（版本A）和蛮力算法（版本B）的操作过程。

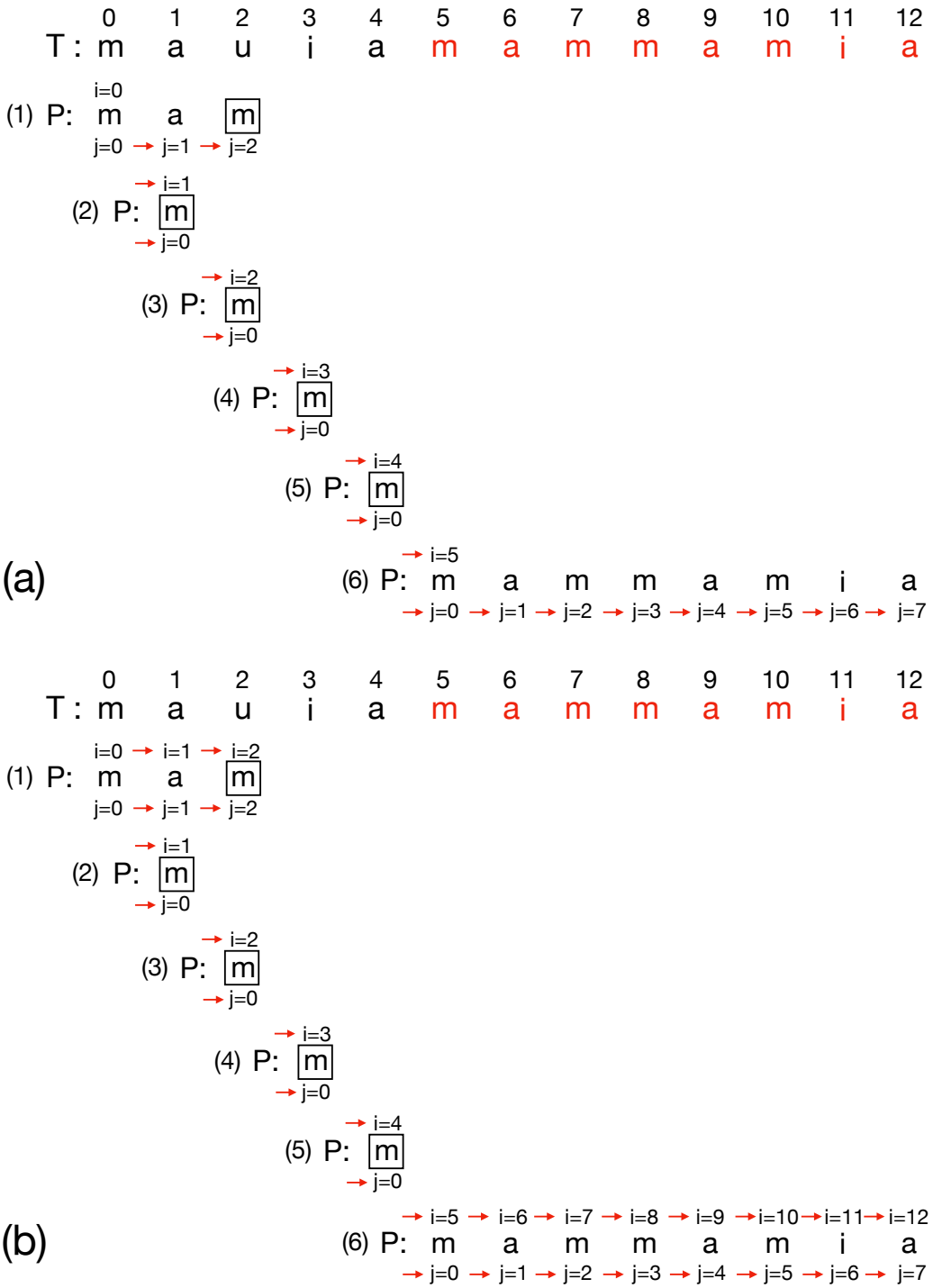


FIG. 7.7. 蛮力算法的两个版本的执行过程。文本串T= “mauiamammamia”，模式串P = “mammamia”。文本串中的红色字符部分表示的是模式串所对应的子串。文本串下标为i，模式串下标为j。(a)和(b)中均进行了6次模式串和文本串子串的比较。图中红色箭头表示两个下标的更新，实线方框表示模式串中某个元素和文本串中对应位置的元素不相等。

7.6. 串匹配问题的KMP算法

KMP算法的最初文献课可参考[23]。

蛮力算法需要对文本串中所有可能的解（即长度为M的子串）进行比较来寻找模式串。 KMP算法(Knuth-Morris-Pratt Algorithm)是对蛮力算法的改进。KMP算法的出发点是：基于已经比较的元素的结果，可以判断出来有些元素的比对步骤、乃至子串是不必要的，是可以直接跳过的。本质上，这种方式可以更快地筛选可能的解，因而比蛮力算法更加快速。

*1. Next*表的含义

这里首先讨论Next表的含义和需要满足的条件。

在蛮力算法（版本B）中，每次出现T[i] $\neq$ P[j]时，需要重新滑动i和j，即：

（1）i往回滑动到之前模式串开始位置的后一位，即$i-=j-1$;

（2）j重新设置为$j=0$。

之后，新的T[i]和P[j]继续开始比较。在这个过程里，P会往前滑动，目的是用来和文本串的下一个子串来比较。

在新的算法里，考虑匹配过程已经比较了一部分子串字符，我们可以利用已经比较过的子串的信息来避免没必要执行的元素比较的步骤，这样可以加快算法。 此时我们假设元素比较的一些基本步骤。在每次比较元素对T[i]和P[j]时，如果T[i] $\neq$ P[j]，我们采取以下操作:

（1）T的下标i并不往后退，也就是i保持原值;

（2）P往前滑动到某个新的下标$t(0 \leqslant t < j)$，之后继续比较T[i]和P[t]。

上述假设看起来很随意，但基于这些假设我们推导出相应的t值的性质及其匹配算法。如果能够得到自洽的结果，则说明这些假设是合理的。

根据上述假设，匹配算法中的每一个j，对应着一个下标t。最终这些t值构成的序列就是Next表。可知:

- 当$t=j-1$时，算法把字符串P往前移动一步。此时算法跳过了当下的文本串子串，大致对应着蛮力算法里移动模式串来进行比较的操作。
- 当$t<j-1$时，算法把字符串P往前移动了多步。此时算法本质上跳过了多个文本串子串。这是基于Next表的匹配算法真正区别于蛮力算法的地方。

上述假设及其过程可见Figure 7.8。

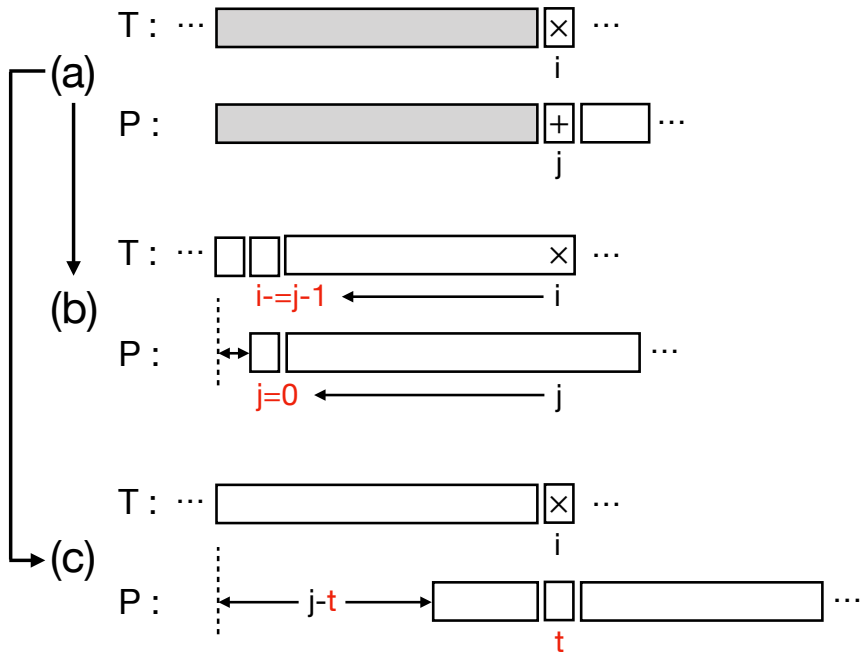


FIG. 7.8. Next表的出发点。 (a)对应文本串T和模式串P在匹配问题中的一次元素比较。此时T[i] ≠ P[j]。 (b)对应蛮力算法（版本B）里下标i和j的变化过程。整体上P往前滑动一步。 (c)对应新的匹配算法里的假设。此时下标i不变，但下标j改变为$j = \text{Next}[j](0 \leqslant t < j)$。这对应着P往前滑动$j - t$步。

上述滑动步骤基于已经比对的子串的信息，因此新下标t的具体数值需要满足一定的条件。 下面我们将陆续处理相关的两个问题：

问题1：下标t如何依赖于文本串T、模式串P、以及下标j?

问题2：移动模式串的最终目的在于寻找文本串中的解。那么为了保证算法没有跳过可能的解，t需要额外满足什么条件?

Figure 7.9展示我们解决这两个问题所使用的推导过程。

我们首先讨论问题1。Figure 7.9(a)中给出了一个模式串和一个文本串的比对。在某个步骤，假设满足：

$$\text{T}[i-j, i) = \text{P}[0, j). \tag{7.6}$$

Figure 7.9(b)里，模式串P进行了一次相对移动，其相对于文本串的移动位数为$j - t$。此时需要模式串P$[0, t)$和文本串T中对应的元素相等，即

$$\text{T}[i-t, i) = \text{P}[0, t). \tag{7.7}$$

从Figure 7.9(a)和(b)，我们将推导t所满足的条件。 如Figure 7.9(c)所示，基于Figure

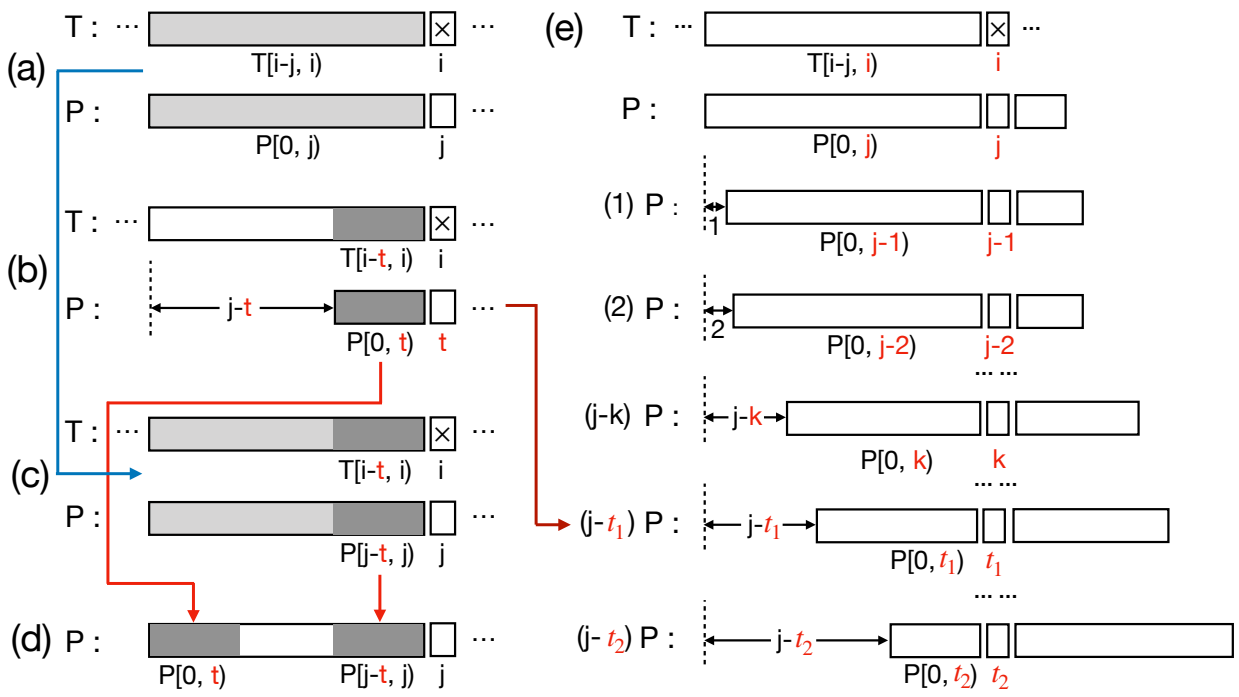


FIG. 7.9. Next表所满足的条件。 (a)为文本串和模式串的相对位置。此时满足T$[i-j,i)$ = P$[0,j)$。但我们并不假设一定满足T$[i]$ ≠ P$[j]$。 (b)为按照Next表滑动模式串P、将模式串下标t对齐文本串下标i。相应地模式串移动的位数为$j-t$。此时必定满足T$[i-t,i)$ = P$[0,t)$。 (c)使用了(a)的结果。可知T$[i-j,i)$和P$[0,j)$的对应子段满足相等的关系。因为$0 \leqslant t < j$，可知T$[i-t,i)$ = P$[j-t,j)$。 (d)中对比(b)和(c)的结果后，可以得到P$[0,t)$ = P$[j-t,j)$。 (e)为列出来模式串的单步移动。假如t_1和t_2（满足$t_1 > t_2$）均为满足要求的t值。可知其第$j-t_1$个步骤对应着(b)中的步骤。 图中相同灰度的色块表示相等的子串。

7.9(a)中的结果，我们有两个子串之间的关系：

$$\mathrm{T}[i-t,i) = \mathrm{P}[j-t,j). \tag{7.8}$$

从Figure 7.9(d)可知，结合(b)和(c)的结果后可得：

$$\mathrm{P}[0,t) = \mathrm{P}[j-t,j). \tag{7.9}$$

这就是为了能够实现算法里多步移动操作，t需要满足的条件。直观上来看，t值依赖于模式串本身前后子串的相等关系。可知，t值只依赖于模式串P以及下标j本身，并不依赖于文本串T。

对于问题2，我们需要根据Next表以t步的步数来移动模式串P时不会漏掉T子串中的可能解。这里需要处理两个子问题：

问题2.1：对于给定的P和相应的j，如何证明所有的解一定对应着某个t。

问题2.2：对于给定的P和相应的j，如果存在多个t值，那么应该如何选择t?

对于问题2.1，我们使用反证法(Proof by Contradiction)来解决。在Figure 7.9(e)里，我们展示了模式串的多次的单步移动。对于给定的P和j，假设只存在两个t值：t_1和t_2 $(t_1 > t_2)$。此时假设存在某个值$k(\neq t_1 \neq t_2)$，满足：

$$\mathrm{T}[i-k, i-k+M) = \mathrm{P}[0, M). \tag{7.10}$$

这对应着Figure 7.9(e)中的步骤(k)。为简单起见，我们假设$k > t_1$。 Eq.(7.10)说明T中本来存在一个子串作为解，即本应该把P$[k]$对齐T$[i]$。但我们却用P$[t_1]$对齐了T$[i]$，导致在依据Next表移动时跳过了这个解。 这里我们将证明上述的k其实并不存在。因为Eq.(7.10)成立，则T和P对应子串也相等，即满足：

$$\mathrm{T}[i-k, i) = \mathrm{P}[0, k). \tag{7.11}$$

上述形式和Eq.(7.7)相同。这意味着本质上k也是属于j对应的Next值。这与我们最初的假设相反。这也就证明了所有可能的解都能够通过t值体现出来。

对于问题2.2， 给定P和j后，t值的数目有以下情况：

• 如果对应的t值是唯一的：那么t值对应的移动就不会漏掉可能的解，此时设置Next$[j]$ $= t$即可。

• 但如果t值有多个，那么应该如何选择t值？从问题2.1的解答可知，对于Figure 7.9(e)中的t值存在t_1和t_2的情形，选择t_2后会导致漏掉t_1 $(> t_2)$对应的可能解。此时选择t_1才是正确的选择。在普遍意义上来说，如果t值有多个时，选择最大的t（也就是使得移动步骤次数$j - t$最小）来进行移动，将不会遗漏所有子串可能解。

这里总结一下问题1和问题2的最终结论。给定长度为N的模式串P之后，使用如下规则来构造对应的Next表：

（1）对于一个下标$j(0 \leqslant j < M)$，我们寻找所有的$t(1 \leqslant t < j)$值以满足：

$$\mathrm{P}[0, t) = \mathrm{P}[j-t, j).$$

（2）对于一个下标$j(0 \leqslant j < M)$，所有的t组成了一个集合S，即满足：

$$S(\mathrm{P}, j) = \{t | \mathrm{P}[0, t) = \mathrm{P}[j-t, j), 1 \leqslant t < j\}. \tag{7.12}$$

其中t对应某个重复子串P$[0, t)$，以及模式串的滑动步数$j - t$。t的最大值对应着最长的重复子串、模式串移动的最小步数$j - t$。此时我们选择t的最大值作为Next$[j]$的值，即：

$$\mathrm{Next}[j] = \max S(\mathrm{P}, j). \tag{7.13}$$

这里讨论下Next[0]的含义。当$j = 0$时，P[0, j)为空集，导致无法使用上述规则来计算Next[0]。这时候可以返回匹配问题最开始的步骤来讨论。示意图可见Figure 7.10。由Next表的含义可知，当文本串T[i]和模式串P[0]（对应$j = 0$）不同时，i更新为$i + 1$、j更新为0。这等价于：i不变、设置$j = -1$。此时：

（3）我们设置：

$$\text{Next}[0] = -1. \tag{7.14}$$

上述（1）–（3）构成了Next表的基本性质。

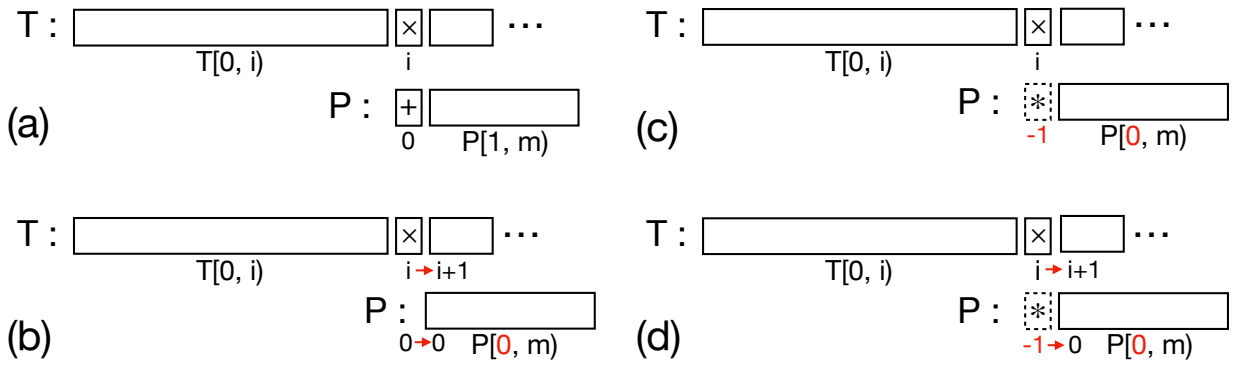


FIG. 7.10. Next[0]的含义。(a)为文本串T[i]不等于P[0]的情形。此时对应$j = 0$。(b)对应着蛮力算法视角中的下一步：i更新为$i + 1$，$j = 0$更新为$j = 0$。(c)为基于Next表的处理方式。此时认为i没有变化，且$j = -1$。这对应着在P的下标为-1的位置，定义了一个通配符“$*$”。通配符和任意字符相比较时都为相等。(d)为基于Next表操作的下一步。下标i更新为$i + 1$，j更新为0。此时的形式和(b)一样。

2. Next表的直接构造法

我们可以使用直接构造法来利用Next表的元素对应的的原始含义来计算Next表。对于任意P和给定下标j，直接找出来子串P[0, j)内的头尾重复子串，Next[j]即为重复子串中的最大长度。Figure 7.11展示根据Next表的定义直接计算某个Next[j]的过程。

根据Next表的定义计算所有的Next表的元素后，就得到了完整的Next表。Figure 7.12给出三个模式串例子的完整的Next表的计算过程：

（1）模式串的字符全同；

（2）模式串的字符各异；

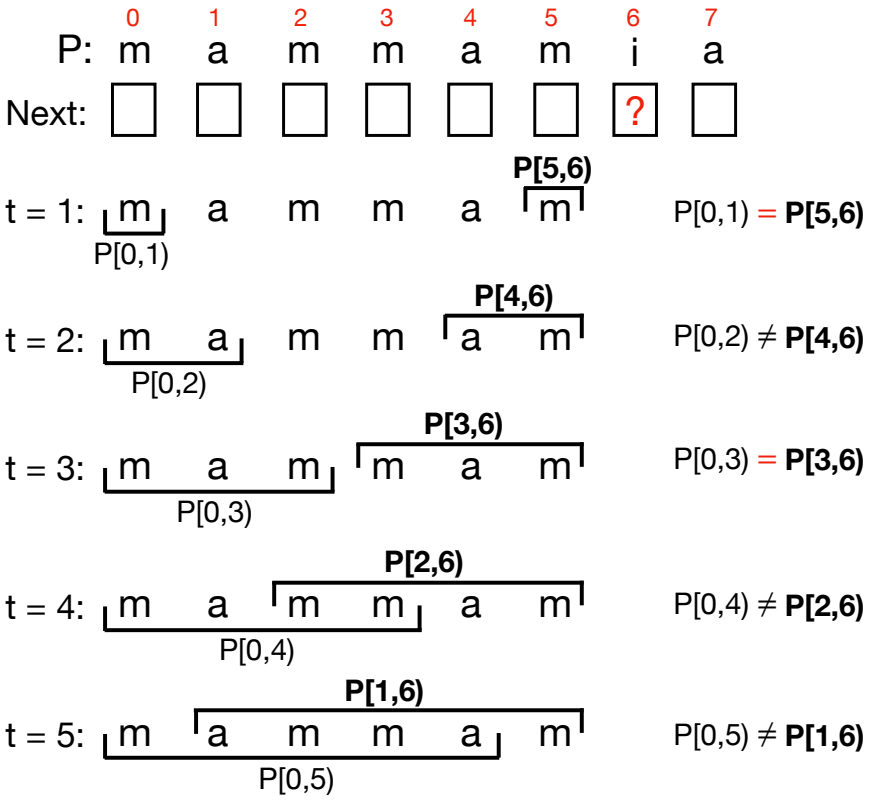


FIG. 7.11. Next表中元素的计算过程。 这里展示了给定模式串P后，计算Next[j] ($j = 6$)的过程。图中依次测试了$1 \leqslant t \leqslant 5$对应的P[$0, t$)和P[$j-t, t$)。可以得出m和mam均为模式串里的重复子串。对应$t = \{1, 3\}$。最终Next[6] $= \max\{1, 3\} = 3$。

（3）模式串具有一些重复的字符组合。

由之前的讨论结果可知，如果Next[j] $= t = j - 1$时，此时对应的字符匹配过程和蛮力算法相同；当Next[j] $= t < j - 1$时，Next表的优势才能体现出来。从三个例子里可知，当模式串里出现完全重复这类结构时，Next表就会出现第一种情况。

3. *Next*表的迭代构造法

除了直接构造法，我们还可以使用迭代算法来构造Next表。如上所述，我们已经知道Next[0] $= -1$。如果进一步有了Next[j]和Next[$j+1$]的关系，可以依次构造出$j = 1, 2, \cdots$对应的Next[j的值]，也就是Next表的所有元素。 如果已知Next[j]的值，并定义Next[j] $= t$。考虑以下两种情况：

（1）如果P[j] $=$ P[t]： 可知对于P[$0, j+1$)来说，子串P[$0, t$)和P[j]组成的新子串P[$0, t+$

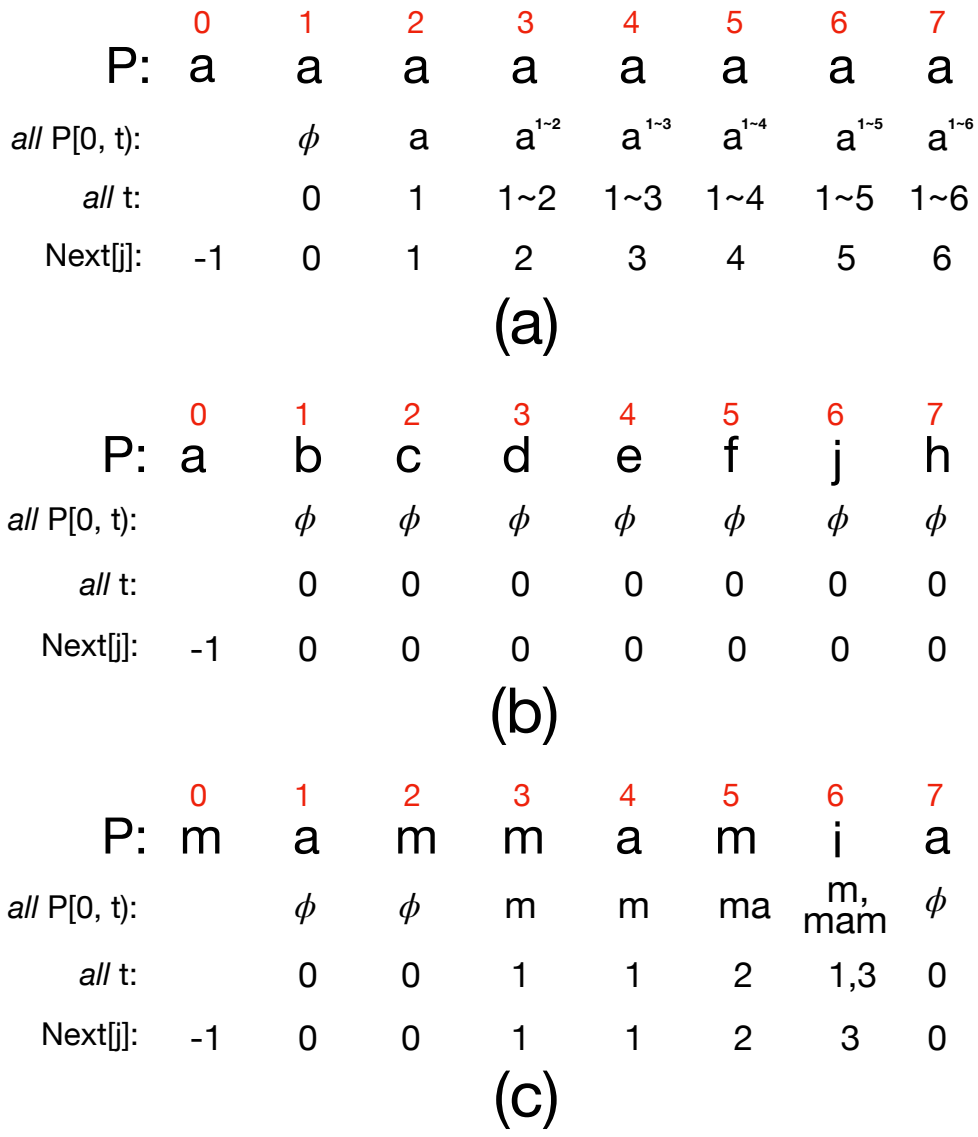


FIG. 7.12. Next表的直接构造方法的例子。 对于每一个字符下标$j \in [0,7]$，图中依次展示了子串P[0, j)中所有重复子串P$[0,t)$、相应所有的t、以及作为Next$[j]$的最大的t。

1)为它的一个头尾重复子串，而且为最长的头尾重复子串。此时：

$$\begin{aligned} \text{Next}[j+1] &= t+1 \\ &= \text{Next}[j]+1. \end{aligned} \tag{7.15}$$

Figure 7.13给出了上述对应的计算过程。

（2）如果P$[j] \neq$ P$[t]$：可以继续往后滑动模式串，将新的P[Next$[t]$]对齐T$[i]$，也就

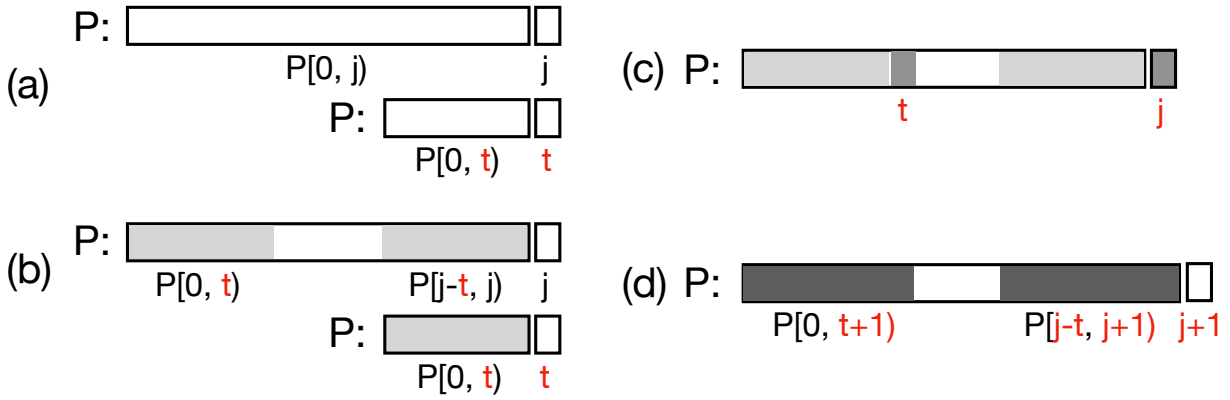


FIG. 7.13. Next表的迭代计算（P[j] = P[t]的情形）。 (a)为Next表对应的一次移动操作。此时t = Next[j]。 (b)为移动操作中Next表所满足的条件，也就是P$[0,t)$ = P$[j-t,j)$。 (c)标识出来了元素P[t]。此时假设P[t] = P[j]。 (d)中把两个相等的子串P$[0,t)$=P$[j-t,j)$、P[t] = P[j]分别合并为新的子串。 此时可知P$[0,t+1)$ = P$[j-t,j+1)$。即$t+1$为模式串P的下标$j+1$对应的最长头尾重复子串，即Next$[j+1] = t+1$。

是P[j]。如果此时出现类似（1）的情况的P[j] = P[Next[t]]，我们可以得到：

$$\begin{aligned} \text{Next}[j+1] &= \text{Next}[t]+1 \\ &= \text{Next}[\text{Next}[j]]+1. \end{aligned} \tag{7.16}$$

（3）如果P[j] $\neq$ P[Next[t]]，可以按照Next表已有的值继续移动模式串到t‘ = Next[Next[t]]，并继续使用第（1）和（2）步骤。在基于Next表移动模式串的任意第t_k步骤，如果得到P[j] = P[t_k]，我们可以得到Next$[j+1]$ = Next$[t_k]+1$。上述步骤有一个终点，也就是Next[0] $= -1$。也就是说，上述寻找给定Next[j]必然最多执行j次之后会得到具体的值。

Figure 7.14给出了上述步骤对应的计算过程。

Figure 7.15给出了上述步骤对应的计算过程。

Figure 7.16给出了Next表中两个元素的具体计算步骤的例子。其算法来自邓俊晖参考书P313，代码11.3“KMP主算法”。

例子5: 计算模式串的Next表

给定模式串P，使用迭代构造法得到对应的Next表。

参考代码：`ch7-string.cc`的TestNextInKMPAlgorithm()函数。

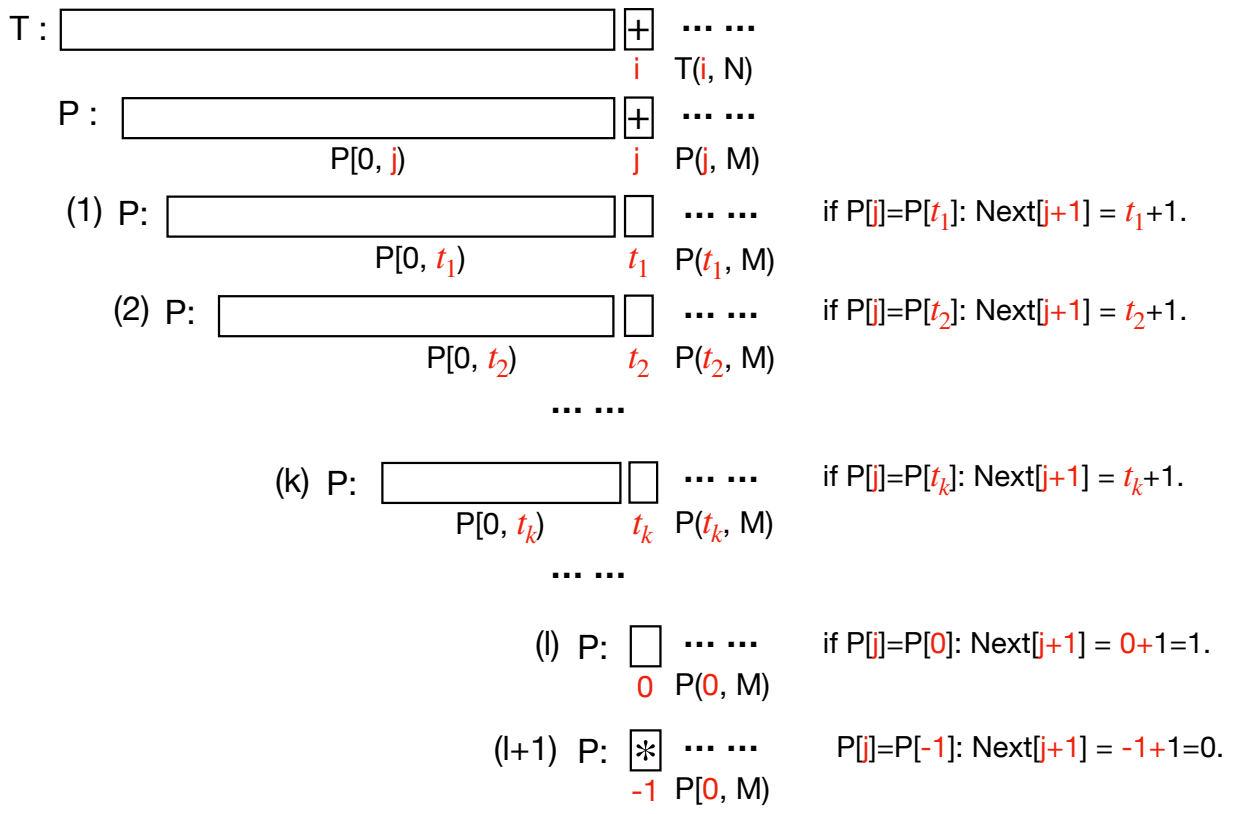


FIG. 7.14. Next表的迭代计算（P[j]和P[t]的一般情形）。第(1)到($l+1$)步为连续的Next表的移动步骤。在第($l+1$)步里，文本串元素P[j]和模式串元素P[-1]（即通配符$*$）相对应。此时算法结束。

```
//计算Next表；
void String::CalculateNext1()
{
  //初始化Next表；
  for (int nind = 0; nind < M; nind++) {
    Next[nind] = 0;
  }

  //逐个计算Next表的元素；
  Next[0] = -1;

  int   j = 0;//模式串的下标；
  int   t = -1;//Next表的临时值；

  while (j < M - 1) {
    if (t < 0 || P[j] == P[t]) {
      j++;
      t++;

      Next[j] = t;
    } else {
      t = Next[t];
    }
  }
};
```

FIG. 7.15. Next表的迭代计算的参考代码。

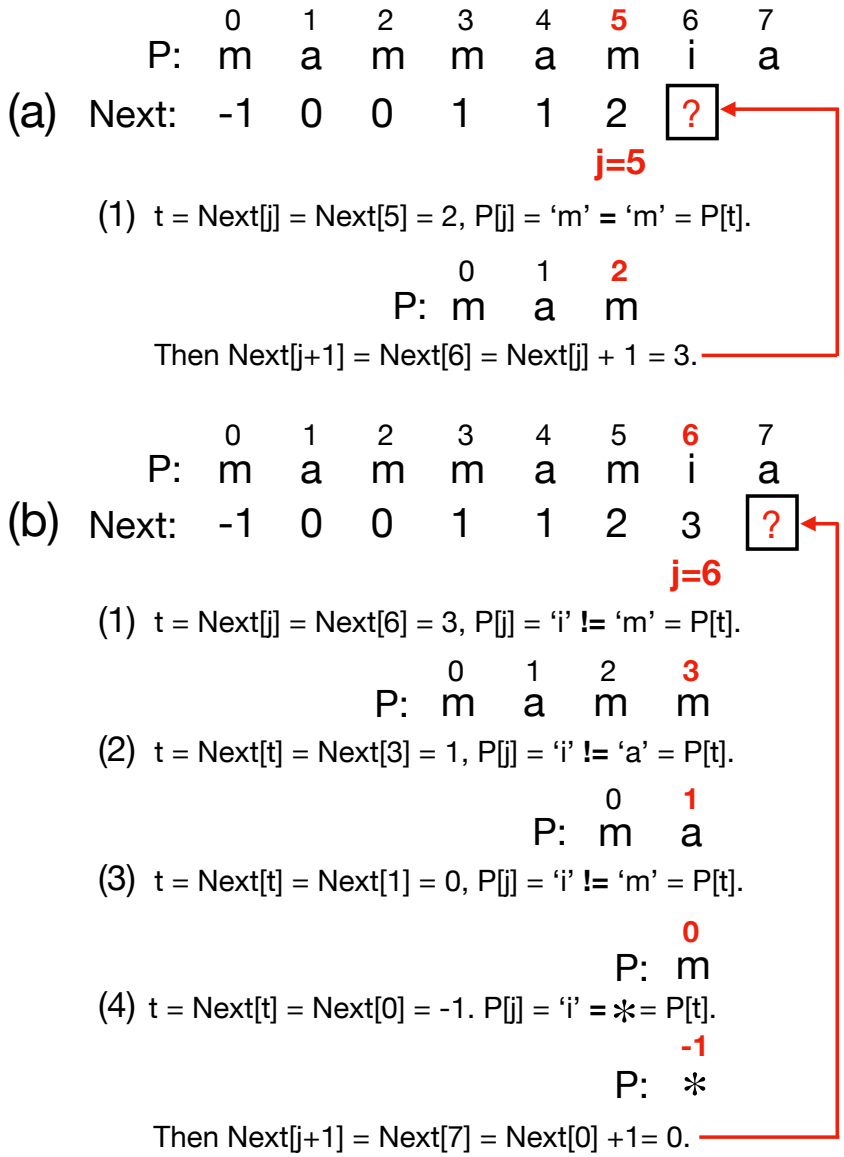


FIG. 7.16. Next表的迭代构造算法的执行过程。 (a)对应利用Next[0, 6)来计算Next[6]的计算过程。 (b)对应利用Next[0, 7)来计算Next[7]的计算过程。

4. 基于*Next*表的*KMP*算法

有了Next表后，我们可以用它来修改蛮力算法（版本B）。在对比模式串P和文本串T时，如果出现T[i] $\neq$ P[j]，模式串的移动按照如下方式：文本串T的下标i不变，但使得模式串P的下标j更新为Next[j]；T[i]和P[Next[j]]对齐后，继续进行元素比较。这种依据模式串的Next表来移动模式串的串匹配算法就是KMP算法。

Figure 7.17展示了给定模式串T和模式串P后的KMP算法的参考函数。函数返回模式串匹配成功后，其首元素在文本串的下标。

```
//KMP算法；
int String::KMPProcedure()
{
  int i = 0;//文本串的下标；
  int j = 0;//模式串的下标；

  while (i < N && j < M) {
    if (j < 0 || T[i] == P[j]) {
      i++;
      j++;
    } else {
      j = Next[j];
    }
  }

  return i - j;
};
```

FIG. 7.17. KMP算法的参考代码。

Figure 7.18给出了在给定文本串T和模式串P之后完整的执行过程。从图中可知，算法跳过了蛮力算法中的一个步骤，对应着跳过了比较文本串下标2开始的字符子串'auiamamm'的操作。

例子6: 串匹配问题的KMP算法

给定文本串T和模式串P，利用KMP算法来寻找匹配子串的位置。

参考代码：`ch7-string.cc`的KMPAlgorithm()函数。

5. 更新版*Next*表

Next表的构造本质上是为了在寻找作为匹配解的文本串子串时，能够基于已经比较过的元素的信息来跳过一些没必要进行比较的文本串子串。这本质上是对解空间进行的删

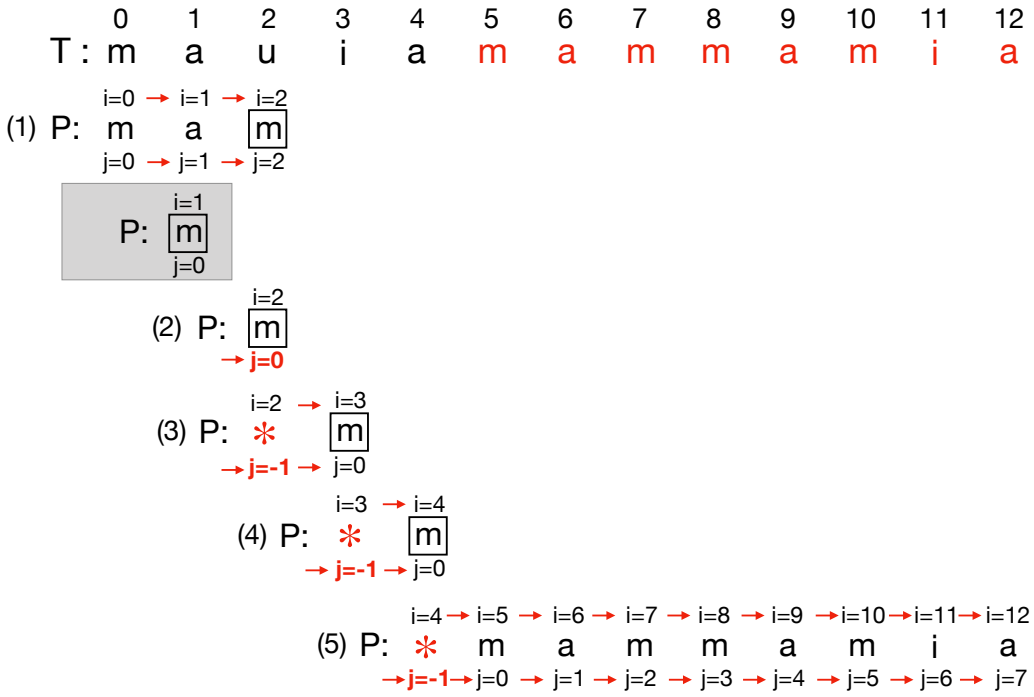


FIG. 7.18. KMP算法的执行过程。 灰色方框覆盖的字符比较步骤为KMP算法相对于蛮力算法所跳过的子串比较的操作。 下标之前的红色箭头表示算法中下标的变化。

减。

但上述Next表其实并不是最优的。其导致的结果可以通过Figure 7.18来展现。图中第(1)步骤里，因为T[2] $\neq$ P[2]，此时滑动P把P[0]对其T[2]。此时容易可见T[2] $\neq$ P[0]，需要继续把P挪到P[-1]对齐T[2]的位置。容易看到滑动P使得P[0]对齐T[2]这一步操作其实是多余的，因为P[0] = P[2]。在一般的情况下，如果T[i] $\neq$ P[j]，而且P[j] = P[t]（t=Next[j]），那么自然能够得到T[i] $\neq$ P[t]。此时可知滑动到P[t]这步操作本身是多余的。此时最合适的做法是再次滑动P，一直到P[t_k] $\neq$ P[j]的情况，其中t_k为执行k次的取值Next表的操作。此时可以赋值Next[j]为t_k，这样可以保证T[i] $\neq$ P[j]时，可以把P直接滑动到一个新的元素P[t_k]来和T[i]继续比较。

上述方法对应的Next表意味着需要对最初的Next表进行修改。新的Next表可以命名为更新版Next表，以和之前的初版Next表做区分。 给定长度为M的模式串P，对比于初版Next表的3个条件，更新版Next表所满足的3个条件可以写为：

（1）对于一个下标$j(0 \leqslant j < M)$，我们寻找所有的$t(1 \leqslant t < j)$值以满足：

$$\mathrm{P}[0,t) = \mathrm{P}[j-t,j), \mathrm{P}[j] \neq \mathrm{P}[t]. \tag{7.17}$$

（2）对于一个下标$j(0 \leqslant j < M)$，所有的t组成了一个集合S，即满足：

$$S(\mathrm{P}, j) = \{t | \mathrm{P}[0, t) = \mathrm{P}[j-t, j), \mathrm{P}[j] \neq \mathrm{P}[t], 1 \leqslant t < j\}. \tag{7.18}$$

其中t对应某个重复子串$\mathrm{P}[0, t)$，以及模式串的滑动步数$j - t$。t的最大值对应着最长的重复子串、模式串移动的最小步数$j - t$。此时我们选择t的最大值作为$\mathrm{Next}[j]$的值，即：

$$\mathrm{Next}[j] = \max S(\mathrm{P}, j).$$

（3）我们设置：

$$\mathrm{Next}[0] = -1.$$

上述3个条件和初版Next表的差别在于红色文字标记的新的限制：$\mathrm{P}[j] \neq \mathrm{P}[t]$。

6. 更新版*Next*表的直接构造法

利用上述原始定义，对于任意模式串P，我们可以先直接构造出来初版Next表、然后构造出来更新版Next表。 这里更新表Next表记为nNext。

具体来说，给定下标j，我们比较$\mathrm{P}[j]$和$\mathrm{P}[\mathrm{Next}[j]]$的值：

（1）如果$\mathrm{P}[j] \neq \mathrm{P}[\mathrm{Next}[j]]$，则$\mathrm{nNext}[j] = \mathrm{Next}[j]$；

（2）如果$\mathrm{P}[j] = \mathrm{P}[\mathrm{Next}[j]]$，则$\mathrm{nNext}[\mathrm{j}] = \mathrm{nNext}[\mathrm{Next}[j]]$。

上述过程利用了已经构造了的Next表和nNext表的结果。 上述步骤可以不断进行下去，直到完整的nNext被构造出来。

Figure 7.19给出来了三个模式串的更新版Next表的直接构造方法的执行过程。 容易看出来，对于全同元素组成的模式串，其nNext表比Next表的对应元素要小。而对于全异元素的模式串，其nNext表和Next表相同。对于实际情况中常见的具有部分结构的模式串，上述两种情况都会出现。

7. 更新版*Next*表的迭代构造法

除了直接构造法，我们还可以使用迭代构造法来构造更新版Next表。 这类方法适合使用代码来构造任意模式串的nNext表。

Figure 7.20展示了构造nNext表的参考代码。

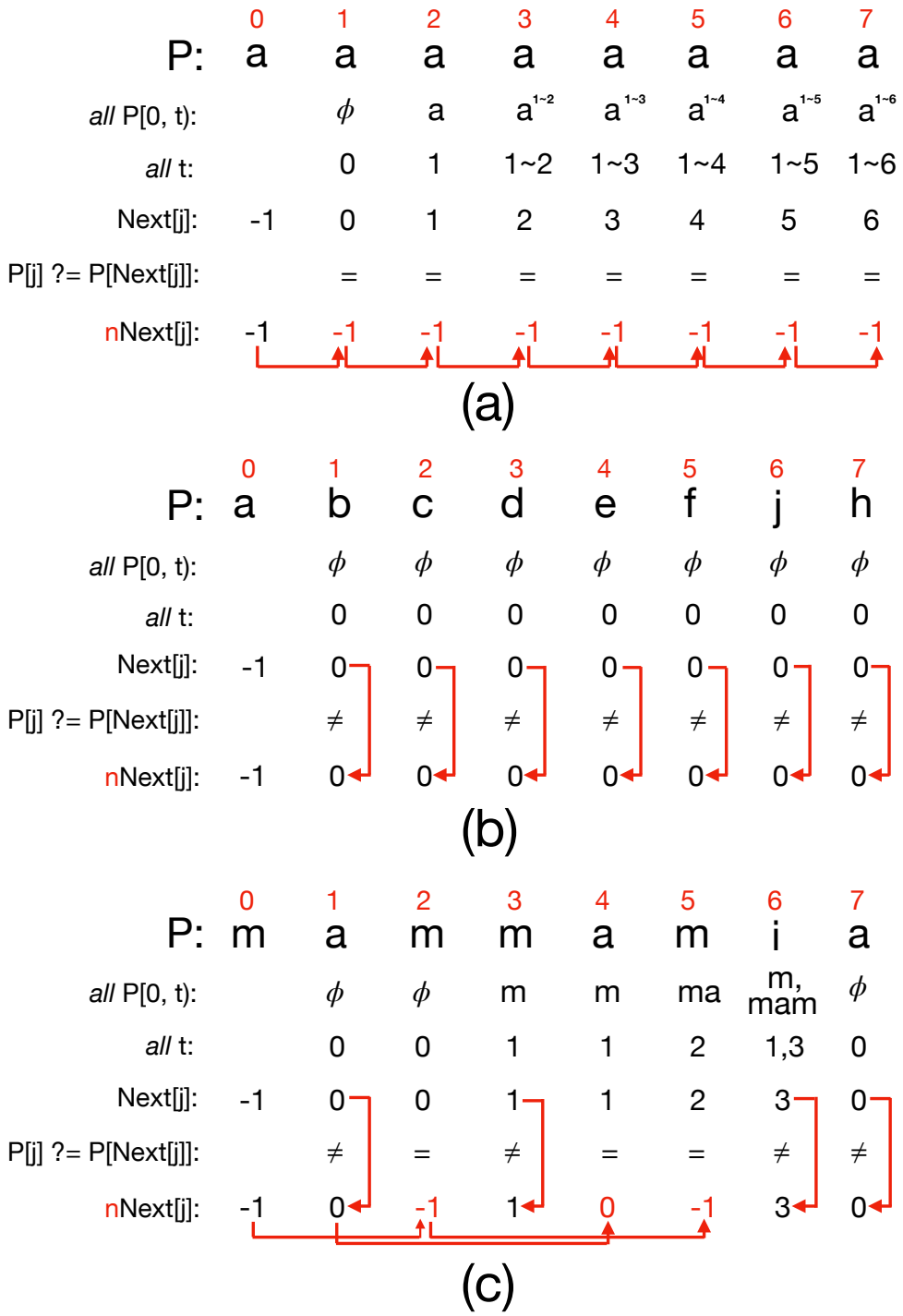


FIG. 7.19. 更新版Next表的直接构造方法的三个例子。 对于下标$j \in [0,7]$，图中依次展示了子串P$[0,j)$中所有重复子串P$[0,t)$、相应的所有t、作为Next$[j]$的最大的t。此部分的格式和Figure 7.12相同。 最后两行是利用初版Next表来构造更新版Next表（此时标记为nNext表）。图中的红色箭头表示赋值过程。

```
//计算改进版Next表；
void String::CalculateNext2()
{
  //默认初始化Next表；
  for (int nind = 0; nind < M; nind++) {
    Next[nind] = 0;
  }

  //逐个计算Next表的元素；
  Next[0] = -1;

  int   j = 0;//模式串的下标；
  int   t = -1;//Next表的临时值；

  while (j < M - 1) {
    if (t < 0 || P[j] == P[t]) {
      j++;
      t++;

      Next[j] = (P[j] != P[t]? t: Next[t]);
    } else {
      t = Next[t];
    }
  }
};
```

FIG. 7.20. 更新版Next表的迭代构造法的参考代码。

例子7: 模式串的更新版Next表

给定模式串P，计算其对应的更新版Next表。

参考代码：`ch7-string.cc`的TestNextInKMPAlgorithm()函数。

8. 基于更新版*Next*表的*KMP*算法

把更新版Next表整合到KMP算法后，我们可以得到新的KMP算法。

在代码实现上，基于更新版Next表的KMP算法和初始版Next表的KMP算法完全相同，差别只在于前者需要使用新的Next表。

Figure 7.21给出来在给定文本串T和模式串P之后完整的执行过程。 容易看出来，使用更新版Next表之后，KMP算法可以继续跳过一些之前需要匹配的子串。这样使得匹配比较的次数减少，最终算法速度会更快。

例子8: 串匹配问题的基于更新版Next表的KMP算法

在给定文本串T和模式串P，使用基于更新版Next表的KMP算法来寻找匹配子串的位置。

参考代码：`ch7-string.cc`的KMPAlgorithm()函数。

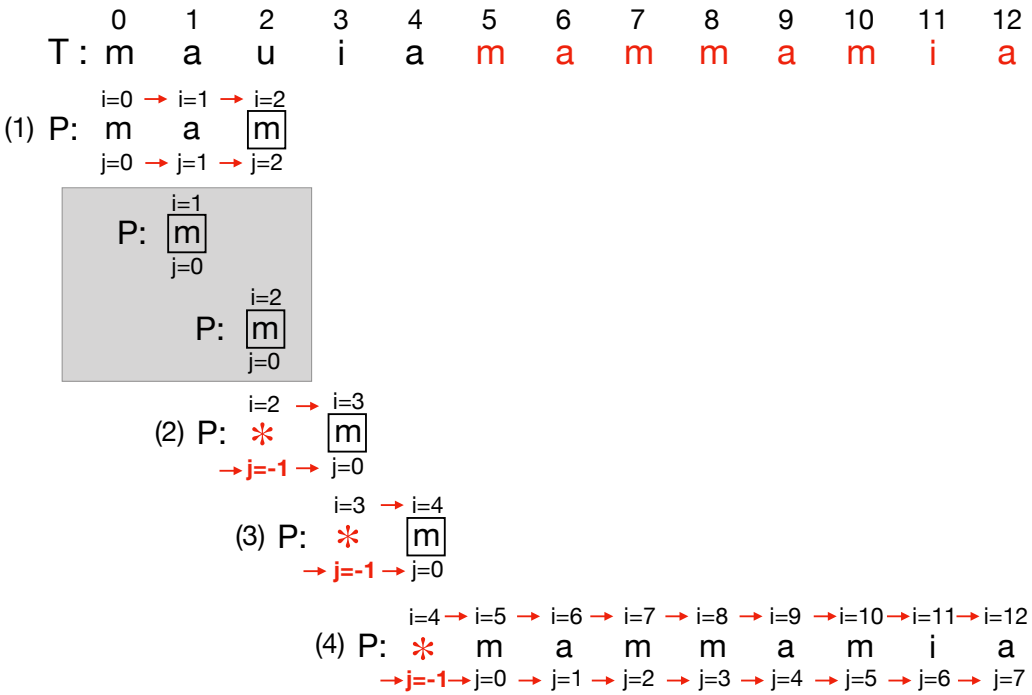


FIG. 7.21. 基于更新版Next表的KMP算法的执行过程。 图中灰色方框覆盖的步骤为KMP算法相对于蛮力算法所跳过的子串比较的步骤。

*9. KMP*算法和蛮力算法的比较

为了更直观地比较蛮力算法和KMP算法，我们可以在给定文本串和模式串后，比较三种算法在某一步的滑动步骤。 Figure 7.22和7.23给出了一个具体的例子。两图展示的是T[5] $\neq$ P[5]后的操作过程。 可以看出来，使用Next表和更新版Next表的KMP算法都到达文本串下标保持为$i = 5$、模式串下标j从$j = 5$设置为$j = 0$的过程。 从图中标记的i和j的值可以直接统计算法步骤的数目。 可以看出来，蛮力算法、基于Next表的KMP算法、基于更新版Next表的KMP算法分别使用了9步、3步、1步操作。 总结这里的结论，KMP算法本质上在比较字符过程中跳过了一些无需比较的子串，总体上可以加速串匹配算法。

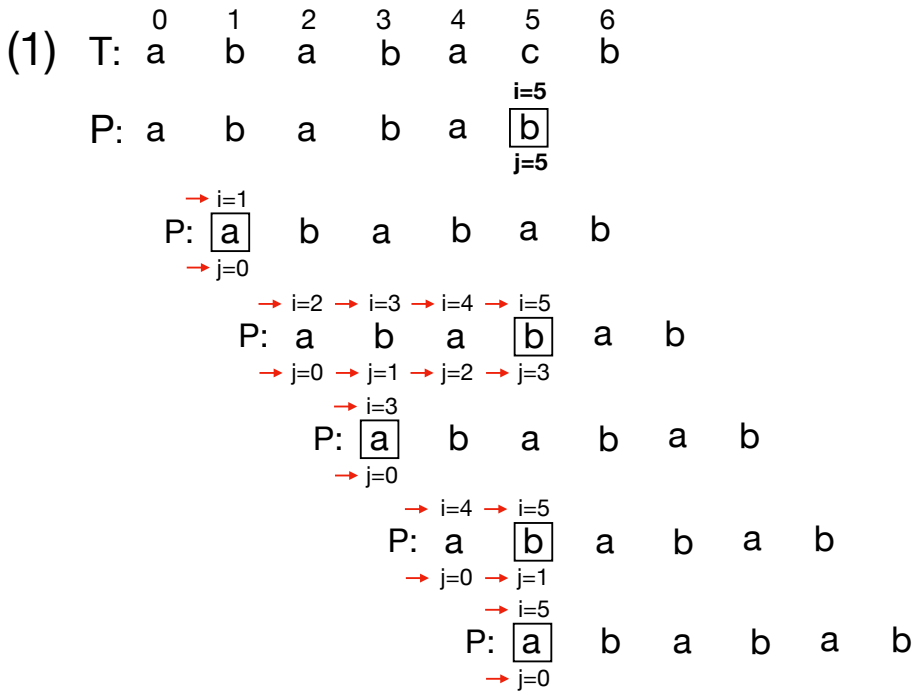


FIG. 7.22. KMP算法和蛮力算法的比较（1/2）。 图中考虑的是文本串T和模式串P错配时(T[5] $\neq$ P[5])的后续操作过程。 (1)为蛮力算法的全部流程。 图中的方框表示模式串和文本串的对应元素出现了错配，红色箭头标记了两个字符串下标的更新。

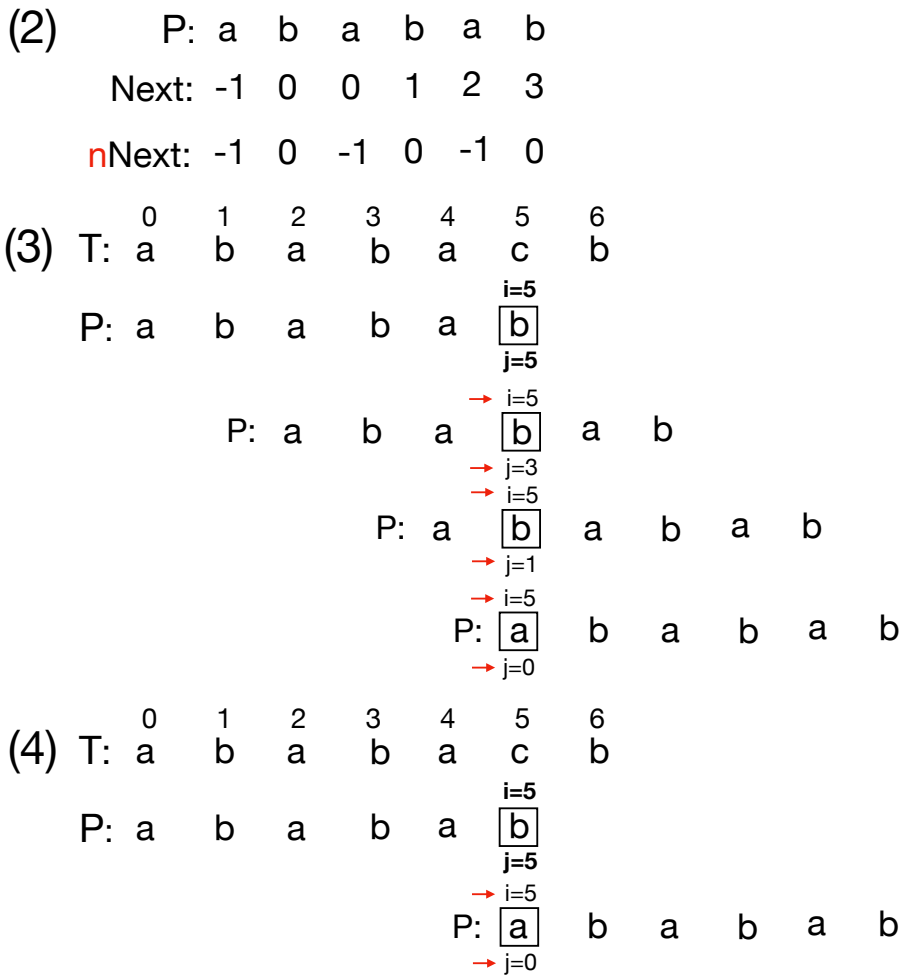


FIG. 7.23. KMP算法和蛮力算法的比较（2/2）。图中考虑的是文本串T和模式串P错配时(T[5] ≠ P[5])的后续操作过程。(2)为模式串P对应的Next表和更新版Next表。(3)为基于Next表的KMP算法的流程。(4)为基于更新版Next表的KMP算法的流程。图中的方框表示模式串和文本串的对应元素出现了错配，红色箭头标记了两个字符串下标的更新。

习题

7.1 Next表的构造

请使用直接构造法完成以下模式串P对应的Next表:

（1）P = “abababababab”;

（2）P = “abbaabbaabba”;

（3）P = “abcabcabcabc”;

（4）P = “cbacbacbacba”;

（5）P = “abcdabcdabcd”;

（6）P = “abcddcbaabcd”;

（7）P = “dingdingdang”.

请画出相应的计算流程示意图。其格式可参照讲义Figure 7.12。此题无需写代码。

7.2 更新版Next表的构造

请使用直接构造法写出以下模式串P对应的更新版Next表。

（1）P = “abababababab”;

（2）P = “abbaabbaabba”;

（3）P = “abcabcabcabc”;

（4）P = “cbacbacbacba”;

（5）P = “abcdabcdabcd”;

（6）P = “abcddcbaabcd”;

（7）P = “dingdingdang”.

请画出相应的计算流程示意图。其格式可参照讲义Figure 7.19。此题无需写代码。

7.3 串匹配算法的时间复杂度

这里我们在随机文本串上运行串匹配问题的蛮力算法和KMP算法，并比较它们的计算时间。 这里我们选择蛮力算法（版本B）以及基于更新版Next表的KMP算法。

我们首先基于字符概率来生成随机字符串。此时考虑字符集大小$D = 2$，字符比例可选择为60% : 40%。我们首先生成长度为N（比如$N = 10^5 \sim 10^6$）的字符串T。在T中我们多次（比如次数$S = 200$）选取长度为M（比如$M = 30, 25, 20, 15$）的模式串P，然后分别使用蛮力算法和KMP算法进行串匹配，并记录其计算时间。

为了充分地利用文本串T，在选取模式串时，我们在T的靠后位置，比如文本串最后比例为f_P的长度（比如$f_P = 0.1$），来连续选择长度为M的字符子串作为模式串P。选取子

串的起始下标为$N(1-f_P)-M$，之后按同一步长Nf_P/N_P来选取N_P个模式串。上述从文本串T中选择模式串P的过程如Figure 7.24所示。

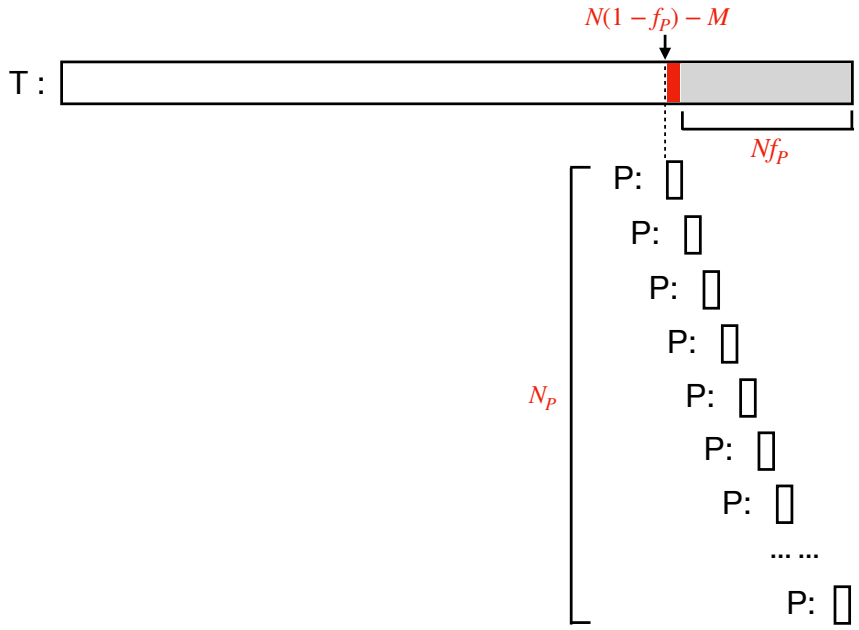


FIG. 7.24. 从文本串中选择子串作为模式串的过程。

请测试不同的N和M，然后在同一幅图里画出两个匹配算法的计算时间$T(N)$。

8. (STACK AND QUEUE)

参考内容：邓俊辉教材第4.1–6节；李春葆教材第3.1–2节。

这一章讲解了两种基于向量的受限结构：栈(stack)和队列(queue)。

对于一个可变长数组或者向量（vector）来说，对它的查询、插入、删除等操作可以是它的任意一个元素，也就是说其所有元素对静态和动态操作是开放的。 栈作为一种线性结构。栈对外的操作接口只有自身的一个端口，即栈顶。此时外界对此结构内元素的操作都只能对栈顶来进行。 队列也为一种线性结构。队列对外的操作接口为自身的两个端口。此时外界对此结构内元素的操作只能通过这两个端来进行。 本质上，栈和队列都能通过向量来实现。 但在具体的问题中，依据问题的自身特点，选择一种受限结构在处理一些具有局域结构和宏观流程的问题时会更直观一些。

8.1. 栈

1. 栈的基本性质

栈是一种受限的线性结构。在栈里，开放的结构部分只有在某一个端的元素，即栈顶元素。栈顶是栈唯一对外界可见的部分。外界只能通过在栈顶来查询或操作元素：查询栈的元素时，只能查询栈顶元素；删除栈的元素时，只能删除栈顶元素。

栈的上述特点导致先进入栈的元素在栈是离栈顶更远的元素、后进入栈的元素是最靠近栈顶的元素。最终栈能够实现先进后出(first-in last-out, FILO)、后进先出(last-in first-out, LIFO)的顺序。

上述结构特点导致对栈的操作函数有一些特别的地方。在一般意义上，我们在栈上定义以下静态和动态的操作函数：

- size(): 返回栈内元素的个数。
- top(): 返回栈顶元素的值。
- push(elem): 把给定元素elem压入栈顶。
- pop(): 把栈顶元素删除掉。

Figure 8.1展示了栈的结构和基于栈的操作。

2. 栈的ADT

本质上栈本身可以通过对向量的数据结构修改来得到，主要是通过限定访问和修改的

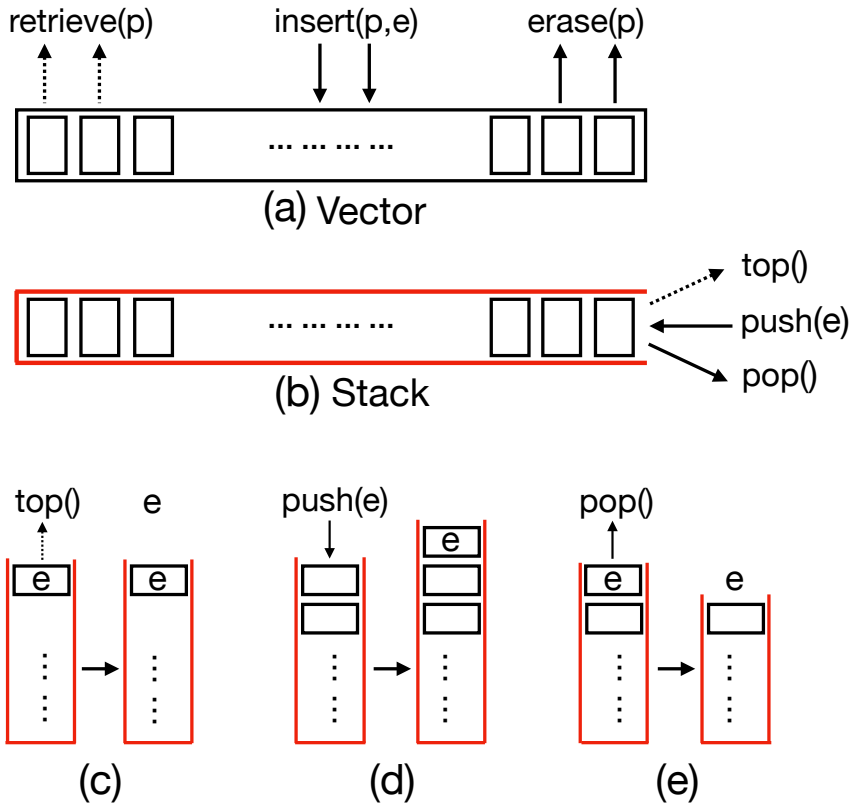


FIG. 8.1. 栈和向量在结构和操作函数上的差别。 (a)展示了一个向量Vector的基本结构。此时有关查询、插入、删除元素的操作可以在向量内的任意位置来实现。p表示操作对应的位置，比如元素对应的秩、指针、或者迭代器。 (b)展示了一个栈Stack的基本结构。红色边界表示栈结构对外封闭的部分，此时只有一端是对外开放，即栈顶。相应地有关对栈的查询、插入、删除元素的操作只能对栈顶来操作。 (c)为查询元素的操作函数top()，对应需返回栈顶元素e。此时无需修改栈内的元素。 (d)为插入元素的操作函数push(e)，对应元素e被压入栈顶。 (ce)为删除元素的操作函数pop()，对应栈顶元素e被弹出（即删除）。 图中带虚线的箭头表示静态操作、带实线的箭头表示动态操作。

秩的范围。

在这里我们可以直接使用C++ STL里的容器stack来实现栈的基本功能。

容器stack可查阅链接：`https://cppreference.cn/w/cpp/container/stack`

stack里的基本操作函数如下：

- size()：返回栈中元素的个数。
- empty()：判断栈是否为空。栈内没有元素时返回true，否则返回false。

- top()： 返回栈顶元素的值。
- push(elem)：把给定元素elem压入栈顶。
- pop()：把栈顶的元素弹出，即删除元素。

例子1: 使用C++ STL容器stack的基本操作函数

使用代码展示C++ STL容器stack里基本的查询和操作元素的函数的用法，包括size(), empty(), top(), push(e), pop()。

参考代码：`ch8-staq.cc`里的StackOperations()函数。

Figure 8.2展示了在栈中连续压入元素和弹出元素的过程。注意操作过程中元素变化的顺序。

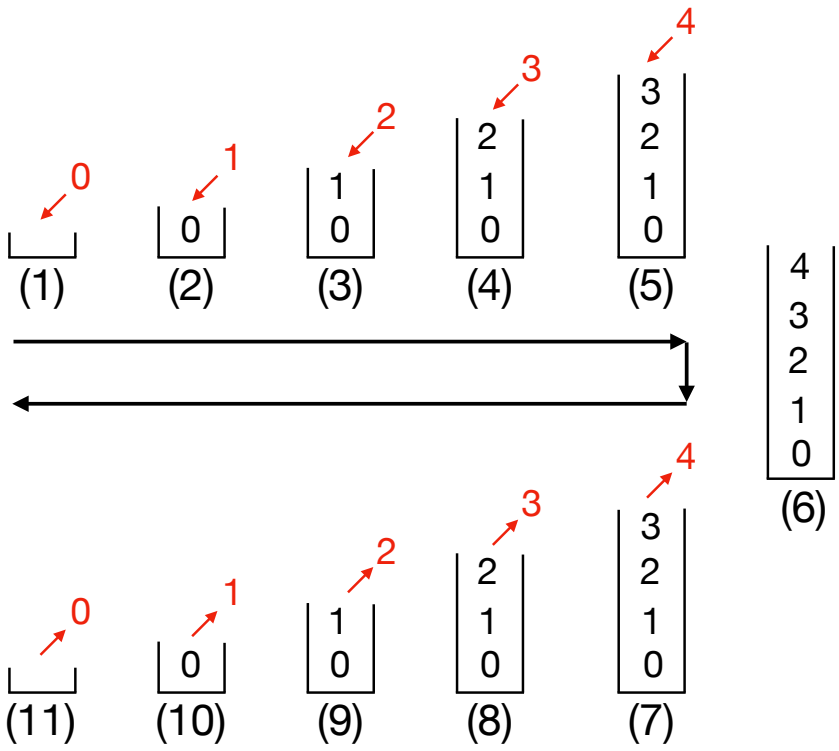


FIG. 8.2. 栈上插入和删除元素的操作。 (1) - (5)展示了从空的栈开始，依次压入元素，直到得到(6)中的包含了5个元素的栈。 (7) - (11)展示了从栈里逐个弹出栈顶元素，直到得到一个空的栈。

3. 栈的应用

整体上，栈能够用来协助和时间顺序有关的一些操作，比如顺序、倒序、对称等等。这些顺序既可以发生在局域，也可以是长距离的范围。

以下我们讨论两个涉及到栈的两个例子。

■ 括号匹配问题

对于编程，从代码文件到可执行文件，必须通过把代码文件中的字符转变成结构化数据、并构建起语法结构。这两部分可以通过词法分析器(lexer)和语法分析器(parser)来实现。

下面我们考虑算术计算里的一个简单例子，也就是对于算术计算的字符。我们首先得判断它们是否符合语法规则。比如以下的数学计算表达式:

$$[(4+5)\times 9]\%10, \tag{8.1}$$

$$[(4+5]\times 9)\%10. \tag{8.2}$$

显然第一个是符合语法规则的，但第二个的括号在搭配上出了问题。此时，我们如何能够简单地来判断括号的搭配是否正确? 假如去掉数学表达式里的数字、算术符号等等，只留下有方括号和圆括号表达式组成的字符串。此时判断语法规则的问题就变成了如何判断括号字符串具有匹配的结构。 所谓“匹配”，就是每一个左括号都有一个后续的右括号相对应，而且两者之间的括号字符串也是左右匹配的。对于方括号、圆括号等，可以依次类推。 比如以下情形是合法的:

$$[\ ]\ [\ ]\ (\ )\ (\ ), \tag{8.3}$$

$$[\ [\ (\ )\ (\ )\ ]\ ]. \tag{8.4}$$

但下述情形是非法的:

$$[\ ]\ [\ (\ ]\ )\ (\ ), \tag{8.5}$$

$$[\ (\ [\ )\ (\ )\ ]\ ]. \tag{8.6}$$

这里考虑一个简化的情况：如果字符串里只含有”(”和“)”，此时如何判断这个符号串是否是匹配的。 如果一个括号符号串是匹配的，则输出提示信息“true”；如果符号串不是匹配的，返回匹配之后剩下的符号以及其个数。 比如对于以下字符串:

$$(\ (\ )\ (\ (\ )\ )\ )\ (\ )\ ). \tag{8.7}$$

算法返回残留子串的长度为3、对应子串为())。

那么，我们如何通过算法来进行判断一个圆括号的字符串具有合法的匹配结构?

直接相邻的左括号和右括号是最容易判断的匹配结构。 这个问题的难点在于：某个左括号所匹配的右括号可能相离很远，比如下述字符串中红色标记的一对括号：

$$(\ (\ )\ (\ (\ )\ )\ )\ (\ )\ ). \tag{8.8}$$

此时我们可以不断删除相邻的左右括号的匹配结构。最终不相邻符号之间的匹配结构最终会变得相邻、并被删去。

上述算法可以通过向量、列表等数据结构来实现。但使用不同的数据结构在代码实现的直观程度上会很不一样。这里我们使用栈来实现。Figure 8.3为算法的流程示意图。 在具体算法里，我们首先把字符串的首元素压入栈内。之后依次读取字符串内的元素，并和栈Stk的栈顶元素做对比。这里考虑两种情况：

（1）如果栈顶元素为左括号“(”、字符串元素为右括号“)”，则两元素匹配，此时弹出栈顶元素“(”，并继续读取字符串的下一个字符。

（2）如果两个元素不匹配、栈为空、字符串元素为“(”等情况，则把字符压入栈，并继续读取字符串的下一个字符。

这里弹出栈顶元素的步骤实现了算法里删除匹配括号的操作。相应地，栈Stk记录的是当下没有被匹配的符号。如果算法最后得到的栈是空的，则括号字符串是左右匹配的；如果得到的栈不是空的，则可以输出残留的字符串。

> 思考：
>
> 这里大家可以继续思考一下，如果使用数组或向量的话，将如何延续上述类似的算法流程来处理括号匹配问题?

使用栈虽然能够让符号匹配的操作更加直观，但也带来了一个新的问题，也就是使用栈来输出字符时的顺序。在输出栈内元素时，因为其结构只允许访问栈顶，导致没法如向量那这样通过下标来直接遍历并输出元素。输出的栈的元素只能通过查询栈顶元素、删除栈顶元素来不断进行。此外，从Figure 8.3(10)可知，如果直接输出栈的元素，其顺序和输入字符串内的原始子串的顺序是相反的。原因在于，我们在遍历字符串时，是从左到右的（下标从小到大）。如果要保证输出的顺序和原始子串一样，可以有两种处理方法：

（1）如果在遍历序列时依然使用下标从小到大的过程，最终输出栈内元素时可以结合另外一个栈来使用。这样可以把倒序的输出重新变成正序。

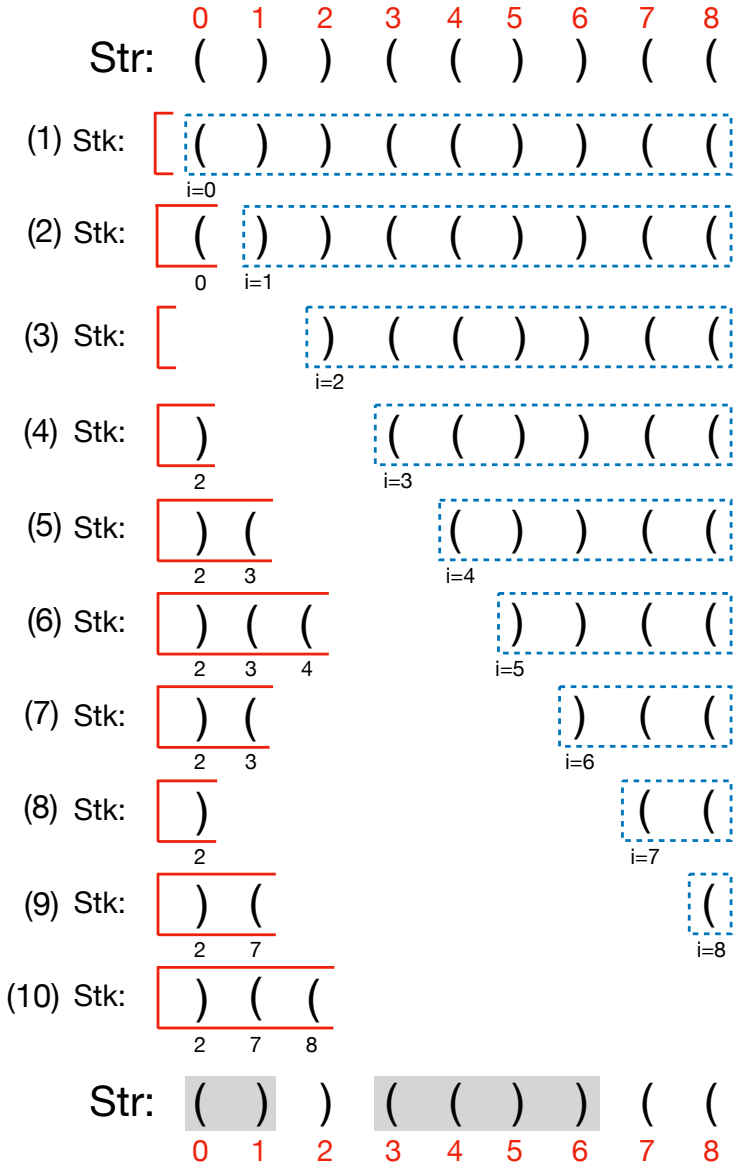


FIG. 8.3. 使用栈来处理括号匹配问题的算法流程。 输入的字符串为Str。(1)-(10)标记了借助Stk来匹配左右括号符号的过程，其中每一步需要判断当前栈顶元素和依次读取的字符串元素是否匹配。栈内元素下方还标记了它们在原先字符串里的下标。虚线方框标记了暂未处理的字符串子串。(10)中Stk里放置着最终不能匹配的字符子串。在最后的Str里，我们用色块标记了匹配的部分。

（2）如果在遍历序列时使用下标从大到小的倒序过程，则使用一个栈就能实现最终残留子串的正序输出。但在匹配过程里，我们需要修改匹配的条件：即栈顶的元素为“)”、字符串元素为“(”时，两个字符元素相匹配。

例子2: 使用栈来处理括号匹配问题

给定一个只含有“(”和“)”的符号串，请判断这个符号串是否左右匹配。如果字符串为匹配的，则输出提示信息；如果字符串未能匹配，则输出匹配过程之后的残留子串。

参考代码：`ch8-staq.cc`里的ValidParentheses()函数。

代码里遍历输入字符串时使用的正序遍历。在处理最终残留子串的输出时，借用了另外一个栈来实现残留子串的正序输出。

■ 栈混洗(stack permutation)

对于放置在栈中的元素序列，我们可以使用一定的格式来表示它们的顺序。比如对于栈中的一个序列$\{1,2,3\}$，如果元素3为栈顶，则序列可以写成$[1,2,3\rangle$，其中[表示栈底、$\rangle$表示栈顶。

考虑存储在栈里的、元素个数为n的一个序列$\{a_0,a_1,a_2,\cdots a_{n-1}\}$。我们借助另外一个栈可以得到它的栈混洗。其过程可以参考Figure 8.4。Figure 8.4(0)为原始的栈A、存储栈混洗序列的栈B、以及用来存储中间序列的栈S。从栈A内的元素出发，我们可以执行多个栈操作步骤。在每一步骤里，我们只能执行以下两步中的一步：

（1）从栈A中弹出元素、同时压入栈S；

（2）从栈S中弹出元素、同时压入栈B。

当栈A中的所有元素都被转移到栈B后，栈B中的序列就是栈A序列的一个栈混洗。Figure 8.4中，从序列$[3,2,1\rangle$出发，我们最终得到一个栈混洗序列$[2,3,1\rangle$。

对于一个长度为n、元素各异的序列，其所有元素构成的序列的数目为全排列数$n!$。栈混洗本质上代表了借助栈能够实现的所有序列的可能性。容易理解，栈混洗过程中有相应的栈的操作的约束，使得元素之间的一些想对顺序会一直保持，最终栈混洗能够实现的序列的数目会少于全排列数。相应地和栈混洗问题相关的有三个重要问题。

问题1：如何判断一个序列为栈混洗序列。比如：初始序列构型为$[3,2,1\rangle$，那么如何判断$[2,1,3\rangle$为一个栈混洗构型?

问题2：如何计算栈混洗序列的所有数目。

下面我们将会证明栈混洗的总数目对序列长度n的依赖关系为一个Catalan数。Catalan数的形式为

$$\text{Catalan}(n)=\frac{C_{2n}^{n}}{n+1},n\geqslant 1. \tag{8.9}$$

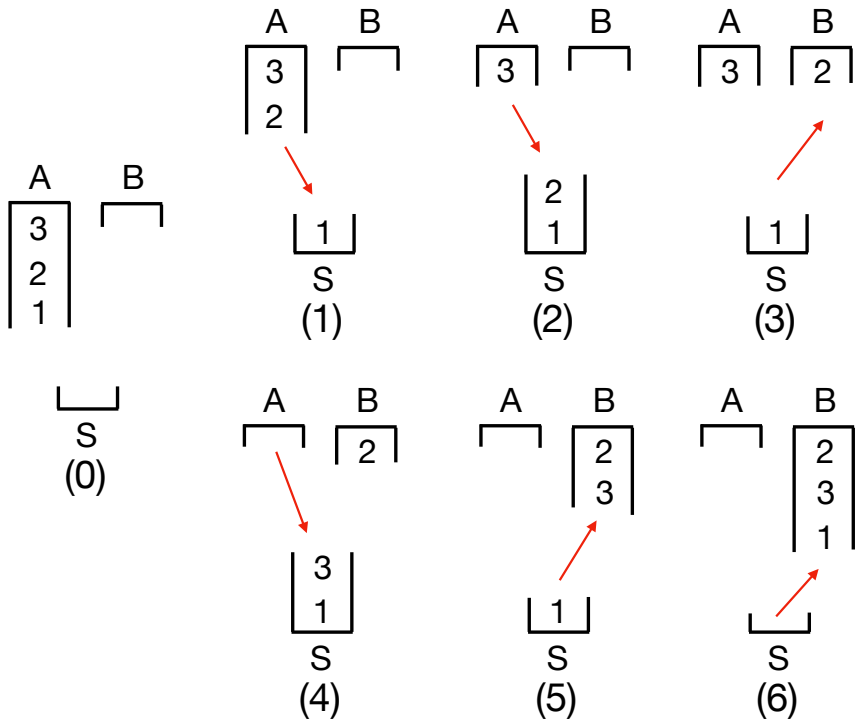


FIG. 8.4. 栈混洗的操作过程。 (0)为原始的栈A、B、S。 (1-6)为从栈A中弹出元素并压入栈S、从栈S中弹出元素并压入栈B的操作过程。 最终得到(6)中栈B中的栈混洗序列。

如上所述，全排列数$P(n)$为$P(n)=n!$。 此时，Catalan(n)和$P(n)$的比值$f(n)$为

$$f(n)=\frac{\text{Catalan}(n)}{P(n)}=\frac{C_{2n}^{n}}{n+1}\times\frac{1}{n!}=\frac{(2n)!}{(n+1)!n!}\times\frac{1}{n!}=\frac{(2n)!}{(n+1)!(n!)^2}. \tag{8.10}$$

$f(n)$的递推公式可以写为

$$\begin{aligned} f(n)&=\frac{2n(2n-1)(2n-2)!}{(n+1)n!\cdot[n(n-1)!]^2}\\ &=\frac{2n(2n-1)}{(n+1)n^2}\times\frac{[2(n-1)]!}{n![(n-1)!]^2}\\ &=\frac{2(2n-1)}{n(n+1)}\times f(n-1). \end{aligned} \tag{8.11}$$

容易得知$f(0)=1$。 对于任意n，$f(n)$的值可以通过上述递推公式来计算得到。借助递推公式可以避免一些重复的阶乘计算。

Catalan数的计算结果见Figure 8.5。可知，当n增加时，$f(n)$的值从1开始迅速变小。尤其是当$n\geqslant 10$时，$f(n)<0.01$，即全排列中绝大部分的构型都不是栈混洗构型。

问题3：输出某个序列对应的所有栈混洗序列。

这个问题本质上是一个枚举问题，也是三个问题里最难的问题。

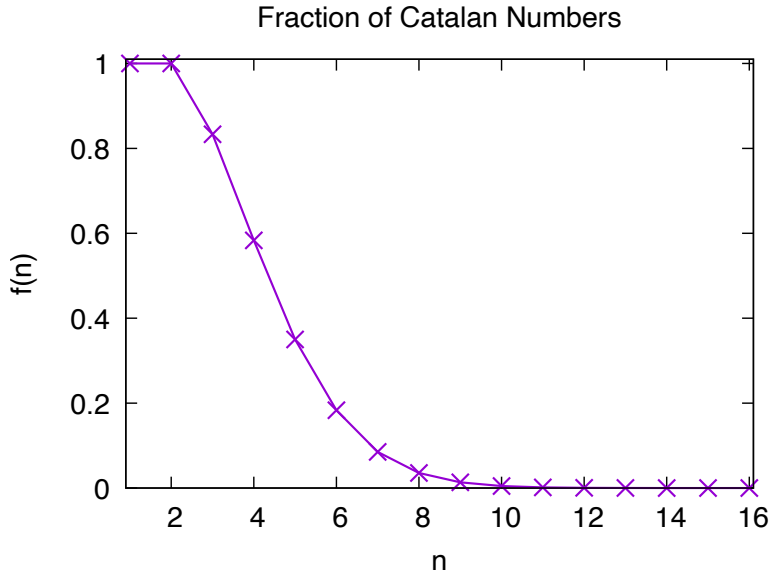


FIG. 8.5. Catalan数Catalan(n)与全排列数$P(n)$之间的比值。 图中符号为横轴自然数n对应的两个数的比值。

8.2. 队列

1. 队列的基本性质

队列也是一种受限的线性序列结构。队列面向操作的唯一结构是它的两端。两个端分别为队尾(rear)和队首(front)。对队列的查询、增加元素、删除元素都只能在队首和队尾上执行。在查询元素时，可以分别返回队首或者对队尾的元素。在插入元素时，只能从队尾插入元素。在删除元素时，只能删除队首的元素。

队列的元素从队尾进入序列、最终从队首离开序列。相应地，最开始进入队列的元素会最早离开序列，最后进入队列的元素会在最晚离开序列。本质上，队列实现了先进先出(first-in first-out, FIFO)、后进后出(last-in last-out, LILO)的顺序。

在一般意义上，可以是在队列上定义以下静态和动态操作:

- size(): 返回队列内元素的个数。
- front(): 返回队首元素。
- rear(): 返回队尾元素。
- enqueue(elem): 在队尾插入给定元素elem。
- dequeue(): 在队首删除元素。

Figure 8.6展示了队列的结构特征及其上的操作函数。

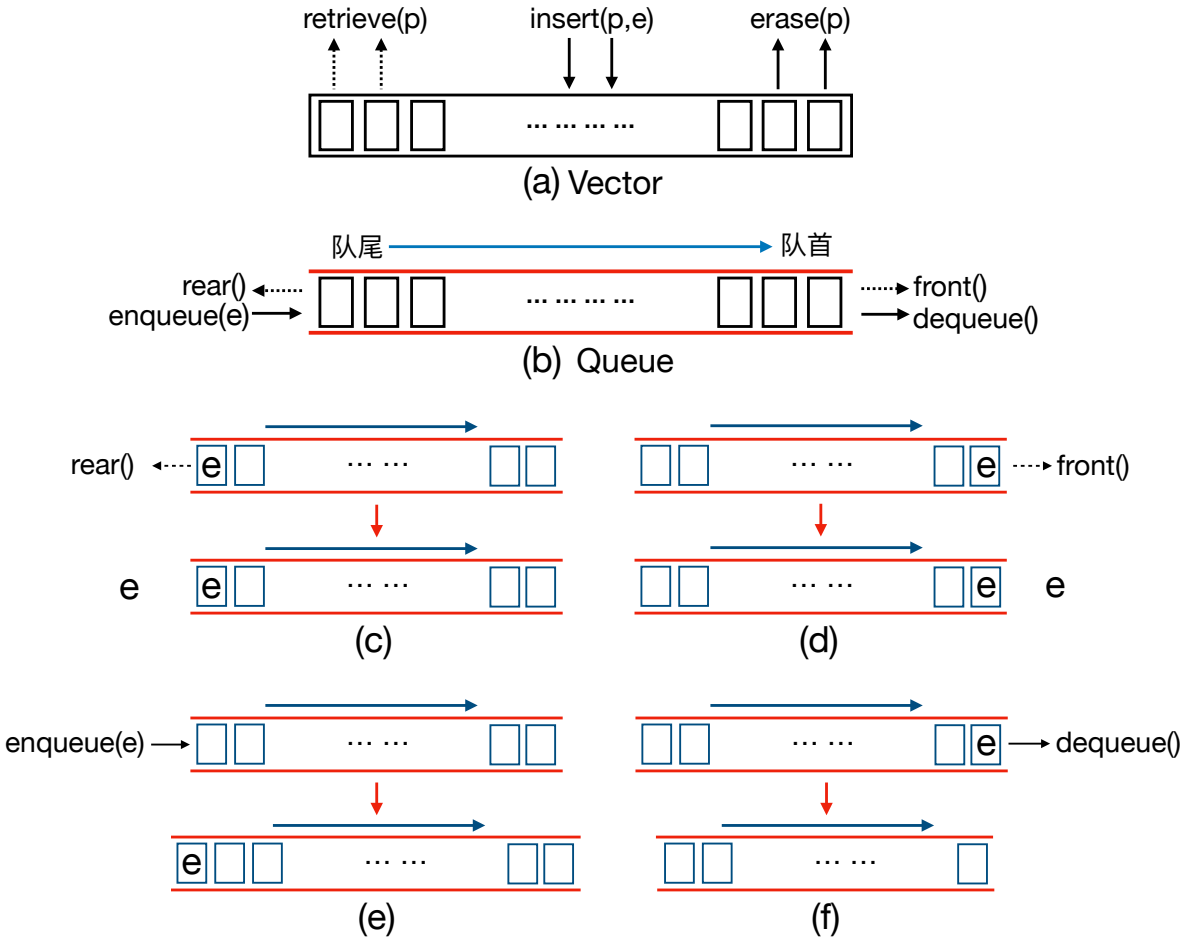


FIG. 8.6. 队列和向量在结构上的差别、及队列上的基本操作函数。 (a)展示了一个向量Vector的基本结构。此时有关查询、插入、删除元素的操作可以在向量内的任意位置来实现。p表示操作对应的位置，比如元素对应的秩、指针、或者迭代器。 (b)展示了一个队列Queue的基本结构。红色边界表示栈结构对外封闭的部分。此时只有两端是对外开放的。 蓝色箭头表示从队尾到队首的方向。 (c)为查询元素的操作函数rear()，对应返回队尾元素的值。 (d)为查询元素的操作函数front()，对应返回队首元素的值。 (e)为插入元素的操作函数enqueue(e)，对应把元素e插入队尾的位置。 (f)为删除元素的操作函数dequeue()，对应把队首元素删除。 图中带虚线的箭头表示静态操作、带实线的箭头表示动态操作。

2. 队列的*ADT*

我们可以使用C++ STL里的容器queue来实现队列的功能。

queue的使用可参考链接：`https://cppreference.cn/w/cpp/container/queue`

容器queue的基本操作函数如下：

∘ size()：返回队列中元素个数。

∘ empty()：判断队列是否为空。如果队列是空，则返回true；如果否，则返回false。

∘ front()： 返回队首元素。

∘ back()： 返回队尾元素。对应Figure 8.6中的rear()函数。

∘ push(elem)：把给定元素elem压入队尾。对应对应Figure 8.6中的enqueue(elem)函数。

∘ pop()：把队首元素删除掉。对应对应Figure 8.6中的dequeue(e)函数。

例子3: 使用C++ STL容器queue的基本操作函数

这里展示了C++ STL容器queue里基本的查询和操作元素的函数的用法。函数包括size(), empty(), front(), back(), push(e), pop()。

参考代码：`ch8-staq.cc`里的QueueOperations()函数。

Figure 8.7展示了队列里连续插入和删除元素的操作过程。注意操作中元素的顺序。

3. 队列的应用

队列的结构特点比较适合用来处理宏观上的有序流程。以下为两个具体的例子：

∘ 二叉树上的层次遍历

∘ 图上的广度优先搜索过程

上面两个例子我们将分别在第11章“二叉树”、第13章“图”里讲解。

8.3. 栈与队列的比较

栈与队列都为基于向量的受限数据结构。两者受限端口的不同导致操作函数的不同，最终导致两者适合处理不同的流程和问题的结构。两者的比较可见Table IV。

所有能用栈与队列处理的结构，本质上同样可以使用向量来处理。差别只是对于具体的问题，在算法流程和代码实现上，向量没有栈与队列那么直观。

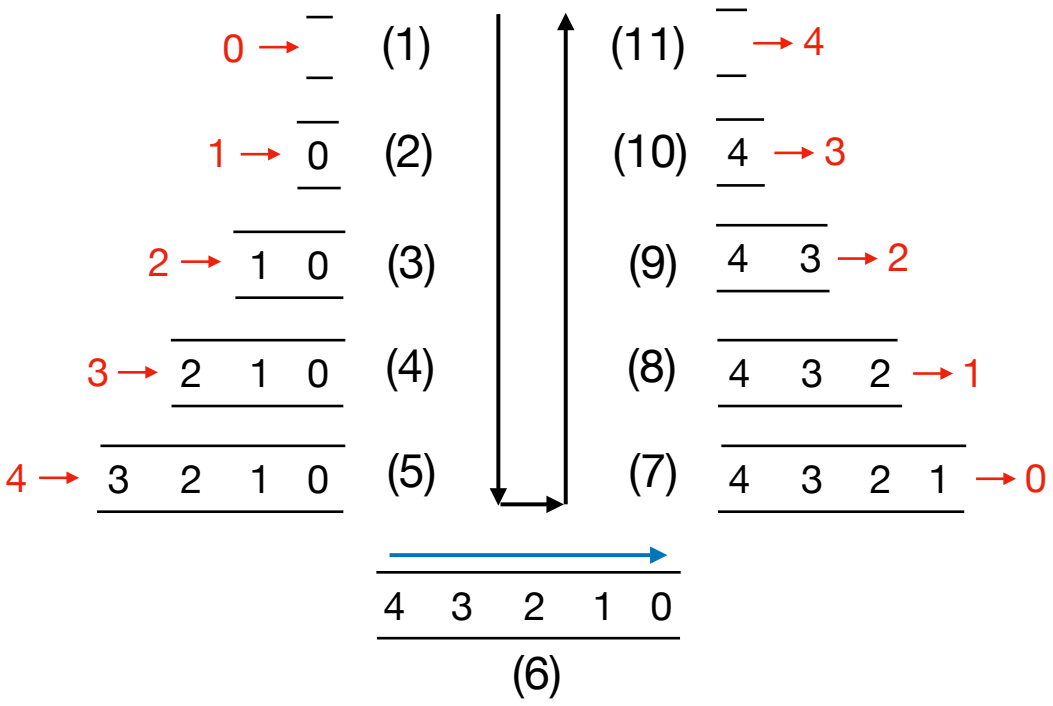


FIG. 8.7. 队列上插入和删除元素的操作。 (1) - (5)展示了从空的队列开始依次插入元素，直到得到(6)中的包含了5个元素的队列。 (6)中的蓝色箭头表示从队尾到队首的方向。 (7) - (11)展示了从队列里逐个删除元素，直到得到一个空的队列。

TABLE IV. 栈与队列的比较。

数据结构	C++ STL容器	实现的顺序	适合场合
栈(stack)	stack	FILO, LIFO	局域上的对称结构等
队列(queue)	queue	FIFO, LILO	宏观上需要顺序处理的流程等

习题

8.1 有效圆括号表达式(Valid Parentheses, VP)中的遍历

在本章小节“栈的应用”里，我们讨论了使用栈来处理圆括号字符串的匹配问题。在遍历字符串以消除匹配的圆括号子串时，我们使用的是从头到尾的、正序的遍历过程。最终在输出最后的残留字符串时，我们需要借用第二个栈来按原先字符串里的顺序来输出它们。但如果我们在遍历字符串时使用从尾到头的、倒序的遍历过程，只用一个栈就能按原先顺序来输出残留字符串。

请参考Figure 8.3，使用图中已有的Str来画出示意图来描述算法的运行，即需要描述读取括号字符串时栈的相应变化。请辅以必要的文字。

注：此处不用写代码。

8.2 有效圆括号表达式(Valid Parentheses, VP)中的匹配圆括号

在讲义里，我们使用栈来实现圆括号字符串里的匹配过程，最终得到不能匹配的残留字符串。 在这里，我们关注的重点在于匹配子串本身，也就是有效圆括号表达式。在一般情况下，一个圆括号字符串可能包含多个匹配子串。比如对于字符串

$$\text{Str} = \text{“())(())((”}. \tag{8.12}$$

其匹配子串分别为“()”和“(())”，其起始下标分别为0和3、长度分别为2和4。 注意，第二个有效圆括号表达式“(())”的内部其实包含了一个有效圆括号表达式“()”。但我们把“(())”作为单独的一个匹配子串。

给定长度为n的一个字符串Str，请使用栈来实现括号匹配算法，并依次输出它所有的有效圆括号表达式，以及它们在初始字符串的起始下标和长度。请使用题目中的Str来测试代码、并输出结果。

注：在实现括号匹配算法时，可以参考习题8.1来使用倒序来遍历Str。这样可以直接实现匹配子串的顺序输出。

8.3 寻找满足给定条件的所有栈混洗序列

给定一个存储在栈内的序列A $= [5, 4, 3, 2, 1\rangle$，请使用栈操作来生成满足给定要求的所有栈混洗序列B。 栈混洗序列B的条件有如下几种:

（1）B = [2 ? ? ? 4 $\rangle$;

（2）B = [4 ? ? 2 ? $\rangle$;

（3）B = [3 ? 2 ? ? $\rangle$;

（4）B = [? 3 ? 2 ? $\rangle$。

上述条件中的“?”表示未知元素。上述条件（1）的大致操作流程如Figure 8.8所示。

请参考Figure 8.4中的操作流程图，画出相关的操作过程、以得到所有满足条件的栈混洗序列。

注：此题画出操作的示意图即可，不用写出代码。

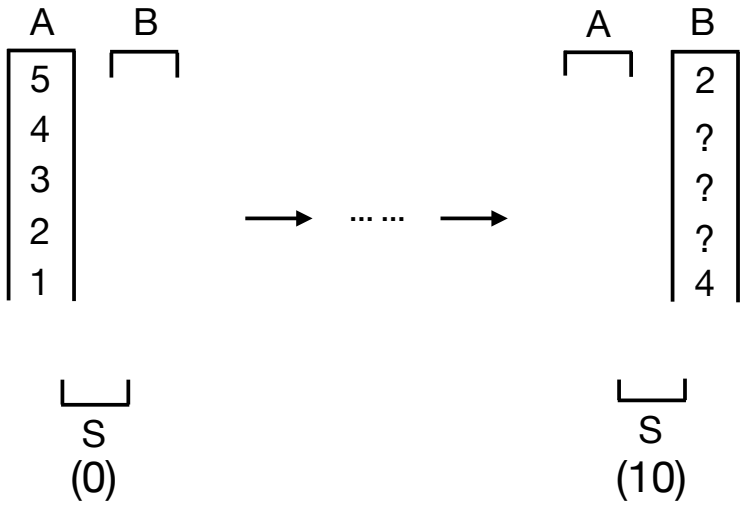


FIG. 8.8. 栈混洗的一个操作流程。 (0)为初始的栈A、栈B、栈S，其中栈A存储了初始序列存储。(10)为经过多次栈操作过程后得到的栈混洗序列。栈混洗序列存储在栈B，其中栈底元素2和栈顶元素4已经标记了出来，其它未确定的元素用问号表示。

9. (HASHTABLE)

参考内容：邓俊辉教材第9.1–4节；李春葆教材第9.4节。

这里我们介绍散列表(hashtable)的概念及其基本应用。

9.1. 应用场景：词典

散列表的应用场景为词典(dictionary)。词典的基本形式为<key, value>（<关键码，值>），本质上实现了关键码和值的组合。例子如下：电话号码和人名的对应。比如

<137xxxx3210，张三>

<152xxxx4510，李四>

<155xxxx9010，王五>

一般情况下，我们可以使用C++ STL容器中的map来实现。

9.2. 基本性质

■ 散列表的应用场景

■ 散列函数(hash function)

■ 冲突(collision)

冲突的定义

装填因子(load factor)

完美散列(perfect hashing)

9.3. 散列函数(hashing function)

■ 设计规则

■ 散列函数的一些例子

散列函数可以使用算术计算的方式，比如以下的(1)和(2)。也可以是非算术计算的方式，比如结合了逻辑操作和集操作，比如以下函数中的(3)-(6)。

常见的散列函数如下：

（1）除余法

（2）MAD法

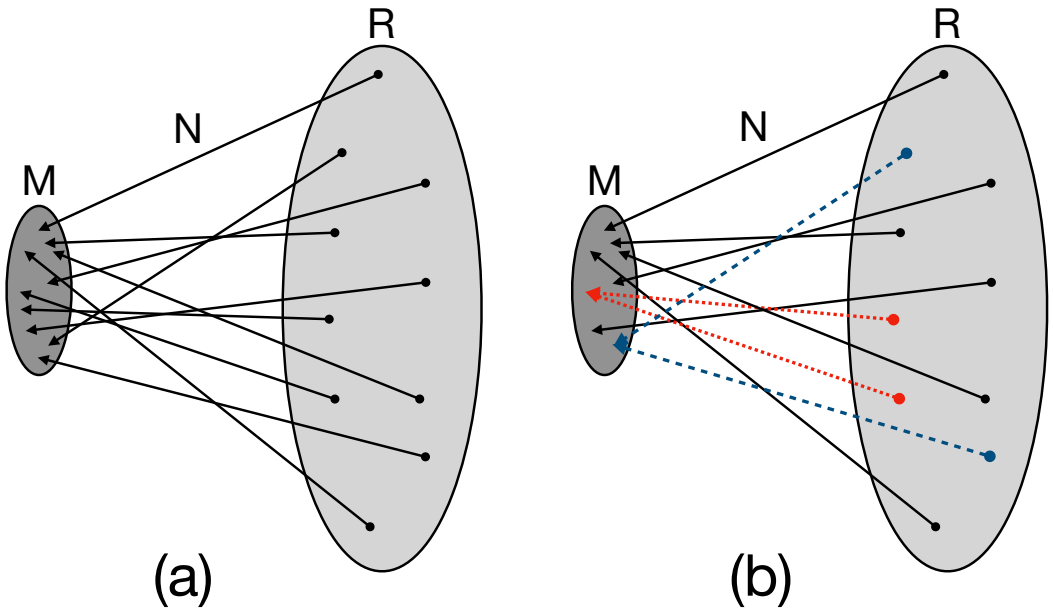


FIG. 9.1. 散列表的基本概念。 (a)中的右侧色块表示了原始数据里关键码的取值范围，其大小表示为R。左侧色块表示了经过散列函数计算后得到的新的更小的取值范围，其大小为M。从右侧色块指向左侧色块的箭头表示散列函数对右侧色块里的具体关键码之后所得到的左侧色块里的关键码。使用散列函数计算的关键码的个数为N。 (a)中所有N个关键码对应的散列函数值都不同。(b)和(a)有着相同的格式，但点线和划线分别对应了两个关键码经过散列函数计算后得到相同的值。此时对应了两次冲突。 此时有$N < M \leqslant R$。

（3）数字分析法
（4）平方取中法
（5）折叠法
（6）位异或法

9.4. 冲突排解

■ 开散列(open hashing)或者封闭定址(closed addressing)

开散列的思路在于，当冲突发生之前或者之时，需要已有的散列表空间之外，额外构造空间来存储产生冲突的关键码。

（1）多槽位法(multiple slots)
（2）独立链法(separate chaining)
（3）公共溢出区(overflow area)

■ 闭散列(closed hashing)或者开放定址(open addressing)

闭散列的思路在于，当冲突发生时，算法继续在已有的散列表空间里寻找地址来存储产生冲突的关键码。这使得无需额外构造新的存储空间。

试探法

（1）线性试探法(linear probing)

线性试探法会加堆积现象。

（2）平方试探法(quadratic probing)

（3）双散列法(double hashing)

9.5. 散列表的应用

■ 桶排序算法（Bucketsort）

这里考虑一个非负整数排序的问题：给定N个非负整数，其不存在重复元素，请对其进行排序。 从散列表的角度出发，可以使用桶排序算法来完成这个排序问题。

算法流程如Figure 9.2所示。其过程大致如下：

（1）对于一个非负整数序列，寻找其最大值$N_{\max}$。相应地构造一个长度为$N_{\max}+1$的状态数组，其所有元素初始值为0。

（2）遍历这个整数序列，把元素直接作为数组下标，标记状态数组对应的元素为1。

（3）遍历状态数组，如果元素为1，则输出其下标。遍历后得到的下标组成的数组，即为排好序的序列。

■ 最大间隙问题(Maximum Gap)

问题描述：考虑N个数构成的一个序列。它可以视为一维直线上的多个点的位置。直线上相邻两点之间有相应的距离。请找到这些距离之间的最大距离、以及对应的两个点。其问题定义如Figure 9.3所示。

这里我们考虑任意实数构成的序列。这个序列可以是直接给定的，也可以按照具体要求来生成。

对于最大间隙问题，最直观的方法是可以先使用实数域上的排序算法首先将点序列进行排序，然后遍历一次序列以得到相邻点间隙中的最大值。 这里我们可以选择快速排序算法。但因为是在实数域上考虑，我们需要对之前的整数域上的快速排序算法稍微做些修改，比如把$\leqslant$ ($\geqslant$)修改为$<$ ($>$)。

我们也可以借助散列表来求解最大间隙问题。以下是算法的主要步骤：

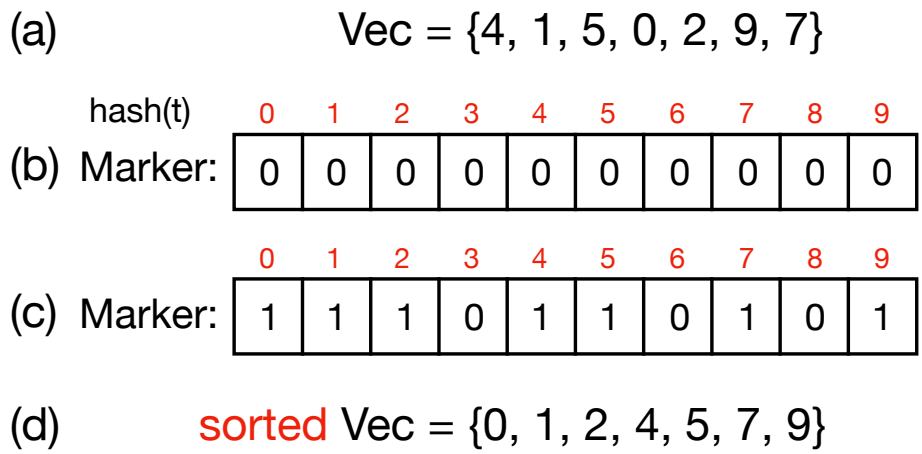


FIG. 9.2. 排序问题的桶排序算法。 (a)为待排序的非负整数序列。 (b)为基于序列最大元素构造的状态数组。其元素的初始值均为0。 (c)为遍历整数序列后得到的状态数组：对于序列中的整数元素，其作为下标的状态数组的元素为1。 (d)为遍历状态数组后得到的非零元素对应的下标，对应着最终排好序的整数序列。

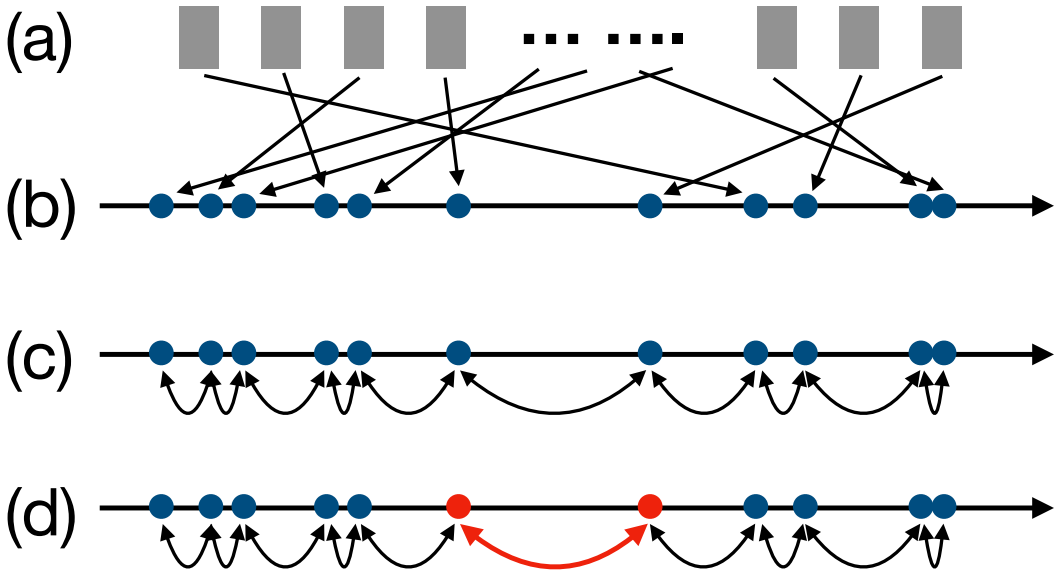


FIG. 9.3. 最大间隙问题的描述。 (a)中阴影色块表示为一个实数序列。 (b)把实数序列映射到一条一维直线上。此时序列元素对应着直线上的圆点。 (c)直线上两个相邻的圆点之间有相应的间距。 (d)中红色双向箭头标记了所有间距中的最大间隙，红色节点表示最大间隙所对应的两个圆点（即序列元素）。

（1）对于给定序列，首先寻找其最小值lo和最大值hi，对应于点位置的最左侧和最右侧。hi - lo即为点序列的分布区间。

（2）把N个点按散列函数放入相应的桶。散列表中桶的数目同为N。每个桶覆盖的点位置的范围相同。对于任意点位置x，其散列函数的值对应于桶的编号，即：

$$\mathrm{ht}(x) = \lfloor (N-1) \times (x - \mathrm{lo})/(\mathrm{hi} - \mathrm{lo}) \rfloor. \tag{9.1}$$

（3）对于散列表的每个桶内的所有点，确定其最左侧点和最右侧点的编号和位置。

（4）相邻的、都落有点的桶可以用来计算点之间的间隙，即**左侧桶最右侧点**和**右侧桶最左侧点**之间的距离即为点序列里的一个间隙。此时遍历一次桶里存的最左侧点和最右侧点的位置，即可找到最大间隙、以及它对应的两个端点的编号和位置。

步骤（4）为何如此描述？它其实来基于对点位置的性质的判断。可知点之间的间隙的统计性质有：

- 最小间隙 $\leqslant (\mathrm{hi} - \mathrm{lo})/(N-1)$；
- 最大间隙 $\geqslant (\mathrm{hi} - \mathrm{lo})/(N-1)$。

上述等号只发生在当所有相邻点之间的间距都相等的时候。此时对应于所有点都处在桶的边缘。 对于任意的点序列，可以忽略上述均匀分布的情形。可知最小间隙发生在桶内部的两个点之间，而最大间隙发生在跨越至少两个桶之间的点之间。

在使用代码实现上述过程时，我们使用一个大小为N的数据结构来表示点序列以及算法的中间计算结果。 其数据结构如Figure 9.4。 其单元包含了点位置本身、快速排序算法所需的点位置和下标、散列表方法所需的信息。

例子1: 最大间隙问题的求解算法

> 在区间$(0, 1)$内随机生成$N = 20$个位置点。 请输出这些点的原始序列、排序后的序列、最大间隙、以及最大间隙对应的两个点。 这里可以检查两种算法的正确性。
>
> 参考代码为`ch9-hashtable.cc`。
>
> 两种方法的测试输出结果见Figure 9.5。 为了能够实现后续的时间复杂度的计算，我们可以用$(0, 1)$之间的随机数生成器来生成均匀分布的、长度为N的点位置的实数序列。 为了保证在给定N后，点的分布比较均匀，我们选择了随序列大小变化的取值范围。 为简单起见，我们选定点位置的取值范围为$(0, fN)$，其中f为调节因子。点的位置即可以如此生成：$x_i = f \times N \times \mathrm{Rand}(), i \in [0, N-1]$。 在图中可以看出两种方法的计算结果符合。

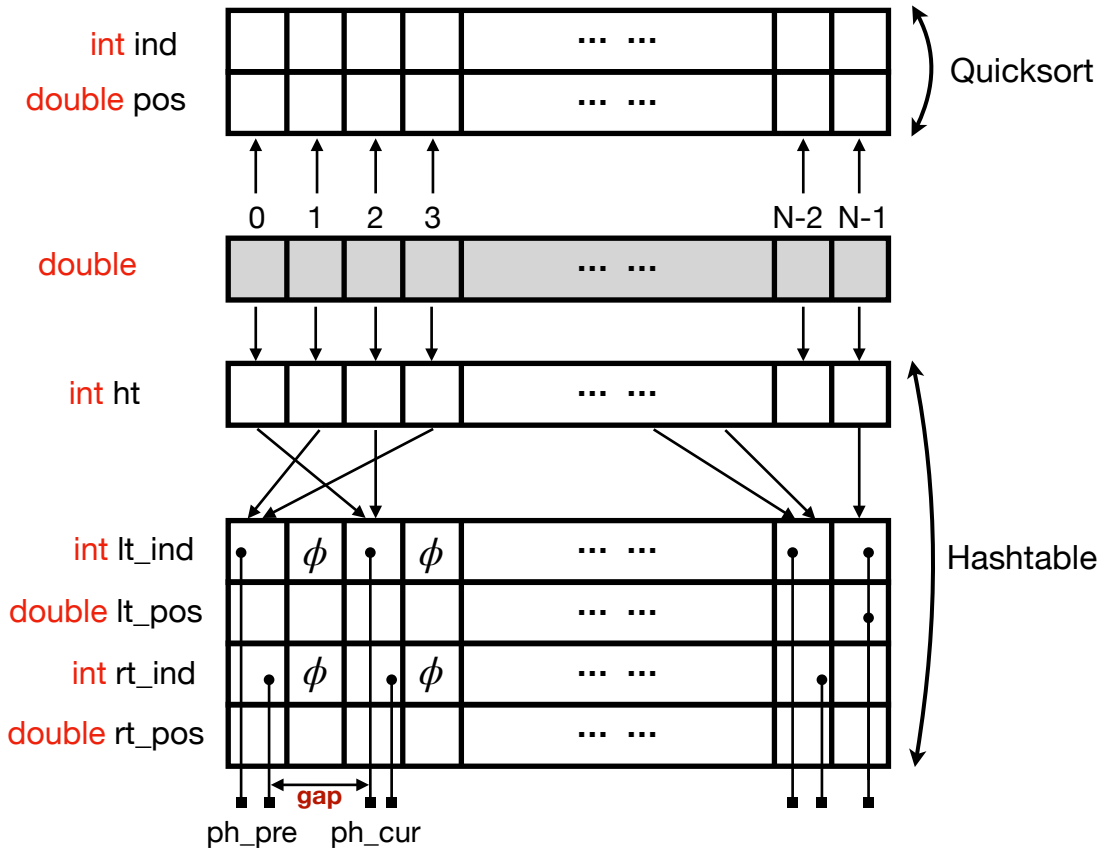


FIG. 9.4. 最大间隙问题中快速排序方法和散列表方法所使用的结构体。 此结构体构成的的序列长度为N，其构成单元如图中纵轴所示的格点，共包括8个数据部分， 包含了快速排序(Quicksort)和散列表方法(Hashtable)所需的数据单元。 在散列表方法中，每个桶单元里端点为方块的线条标识了桶内的左侧点编号lt_ind和右侧点编号rt_ind。 点编号里的ϕ表示空集，即对应桶里没有点落入。 遍历桶序列时通过读取相邻的两个均落有点的桶的左右侧点位置，可以计算出点序列里的一个间隙。 最后，算法利用ph_pre和ph_cur两个指针来遍历桶序列以确定最大间隙对应的两个端点。

习题

9.1 整数序列的桶排序算法

讲义中的桶排序问题处理的是非负整数序列。这里我们考虑使用桶排序算法来排序任意整数序列。

比如以下没有重复元素的序列：

$$\text{Vec} = \{7, -1, 6, 9, -3, 5, 6, 4, 2, -12, 8, 0, 4, -3, 6\}. \tag{9.2}$$

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch9-hashtable.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@Output result:
 Dots[ 0]: pos =   2.08852| Quicksort: [ 6]  0.283617| Hashtable: ht =   9; lt:[  6]  0.283617, rt:[17]  0.403982
 Dots[ 1]: pos =   1.77503| Quicksort: [17]  0.403982| Hashtable: ht =   8; lt:[ -1]        -1, rt:[-1]        -1
 Dots[ 2]: pos =  0.935237| Quicksort: [19]  0.854978| Hashtable: ht =   3; lt:[ -1]        -1, rt:[-1]        -1
 Dots[ 3]: pos =   2.52519| Quicksort: [ 2]  0.935237| Hashtable: ht =  12; lt:[ 19]  0.854978, rt:[ 4]  0.942848
 Dots[ 4]: pos =  0.942848| Quicksort: [ 4]  0.942848| Hashtable: ht =   3; lt:[ -1]        -1, rt:[-1]        -1
 Dots[ 5]: pos =   2.45042| Quicksort: [15]   1.46405| Hashtable: ht =  11; lt:[ -1]        -1, rt:[-1]        -1
 Dots[ 6]: pos =  0.283617| Quicksort: [ 8]   1.65376| Hashtable: ht =   0; lt:[ 15]   1.46405, rt:[15]   1.46405
 Dots[ 7]: pos =    2.7511| Quicksort: [18]   1.72386| Hashtable: ht =  13; lt:[  8]   1.65376, rt:[18]   1.72386
 Dots[ 8]: pos =   1.65376| Quicksort: [ 1]   1.77503| Hashtable: ht =   7; lt:[  1]   1.77503, rt:[ 1]   1.77503
 Dots[ 9]: pos =    2.7467| Quicksort: [13]   2.00463| Hashtable: ht =  13; lt:[ 13]   2.00463, rt:[ 0]   2.08852
 Dots[10]: pos =   3.76548| Quicksort: [ 0]   2.08852| Hashtable: ht =  19; lt:[ -1]        -1, rt:[-1]        -1
 Dots[11]: pos =   2.38111| Quicksort: [16]   2.33358| Hashtable: ht =  11; lt:[ 16]   2.33358, rt:[ 5]   2.45042
 Dots[12]: pos =   3.25495| Quicksort: [11]   2.38111| Hashtable: ht =  16; lt:[  3]   2.52519, rt:[ 3]   2.52519
 Dots[13]: pos =   2.00463| Quicksort: [ 5]   2.45042| Hashtable: ht =   9; lt:[  9]    2.7467, rt:[ 7]    2.7511
 Dots[14]: pos =   3.73258| Quicksort: [ 3]   2.52519| Hashtable: ht =  18; lt:[ -1]        -1, rt:[-1]        -1
 Dots[15]: pos =   1.46405| Quicksort: [ 9]    2.7467| Hashtable: ht =   6; lt:[ -1]        -1, rt:[-1]        -1
 Dots[16]: pos =   2.33358| Quicksort: [ 7]    2.7511| Hashtable: ht =  11; lt:[ 12]   3.25495, rt:[12]   3.25495
 Dots[17]: pos =  0.403982| Quicksort: [12]   3.25495| Hashtable: ht =   0; lt:[ -1]        -1, rt:[-1]        -1
 Dots[18]: pos =   1.72386| Quicksort: [14]   3.73258| Hashtable: ht =   7; lt:[ 14]   3.73258, rt:[14]   3.73258
 Dots[19]: pos =  0.854978| Quicksort: [10]   3.76548| Hashtable: ht =   3; lt:[ 10]   3.76548, rt:[10]   3.76548
@最终结果：
 -QuickSort: val = 0.521204, lt = [ 4] 0.942848, rt = [15]  1.46405
 -Hashtable: val = 0.521204, lt = [ 4] 0.942848, rt = [15]  1.46405
```

FIG. 9.5. 最大间隙问题中快速排序方法和散列表方法的测试结果。 这里测试了参数$N = 20$、$f = 0.1$时的结果。 第一个@后输出了数据结构MG本身，包括最初的点序列、使用快速排序算法后得到的点位置序列、对应的散列表信息。 第二个@后为使用快速排序法和散列表方法后的计算结果。输出数据依次为最大间隙、其对应的左侧点和右侧点在原始序列中的下标和数值。

注意此序列同时含有正数、负数、和零。此外序列包含了重复元素4和6。 排序后的序列输出为：

$$\text{Vec} = \{-12, -3, -3, -1, 0, 2, 4, 4, 5, 6, 6, 6, 7, 8, 9\}. \tag{9.3}$$

注意：无需保证桶排序算法是稳定的。

请设计并实现相应的桶排序算法，对上述序列进行排序，并输出排序后的序列。

9.2 最大间隙问题的散列表方法的时间复杂度

考虑一维实数轴上的最大间隙问题。请完成代码生成长度为$N = 10^5 \sim 10^6$的随机实数序列，并使用快速排序算法和散列表方法寻找最大间隙，并在一幅里画出两种方法的计算时间$T(N)$随N的变化。

10. (PRIORITY QUEUE)

优先级队列(priority queue)。

10.1. 基本性质

10.2. ADT

10.3. 应用

11. (BINARY TREE)

参考内容：邓俊辉教材第5.1–4节；李春葆教材第7.1–5节。

本章介绍半线性数据结构：二叉树(binary tree)。 本章将会介绍二叉树的基本结构性质、其代码实现、遍历等。

11.1. 树的基本性质

■ 为什么要定义“树”这种结构

回想一下，在第5章“向量”这一章里，我们一般考虑的数据形式和计算任务一般如下：

（1）给定100个[0, 1)之间生成的随机数，查找某个数对应的下标；

（2）给定一个班所有同学的某门课的分数，对其进行排序；

这些数据的特点是所有元素本质上是一类，彼此之间没有隶属、关联等关系。

现在考虑另外一些数据及其计算任务：

（1）给定一些文件夹和子文件夹，从某个文件夹出发，找到某个文档所在的位置。

（2）给定几百万年人类演化的谱系，确定某个人种的演化路线；

（3）给定某个姓氏的族谱，找到从某个祖先如何繁衍到当下的某一家人。

上述数据成员之间已经不是同一类数据了，而是一般具有隶属关系，比如文件夹和子文件夹、某个人种和后续的人种、族谱里的一代人和下一代人。真实数据中普遍存在着层次化特征或层级(hierarchical)结构。此时为了更好的处理数据，更适合的数据存储方式是使用具有层级结构的数据形式。 这就是这一章将要讲解的树(tree)这个数据结构。

■ 数据和树的对应

真实的树包含了树根、树干、树叶，从树根到树叶为从下到上的顺序。在抽象的树结构里，为了符合文字的顺序，我们会把树倒着进行作图。 Figure 11.2给出了数据和树对应的一个例子。树有唯一的根节点(root)。树的结构包括节点(node)、节点之间的连边。节点之间的连边具有方向。一个节点可有其它一个节点指向它，同时它可以指向多个节点。

数据成员可以存储在树的节点里。通过跟节点到给定节点有着具体的顺序，此时对应的序列可以用来存储那些具有层次结构的数据。Figure 11.1中，节点k对应着一个具有层次结构的序列(a, c, g, k)。

■ 树的局域结构性质

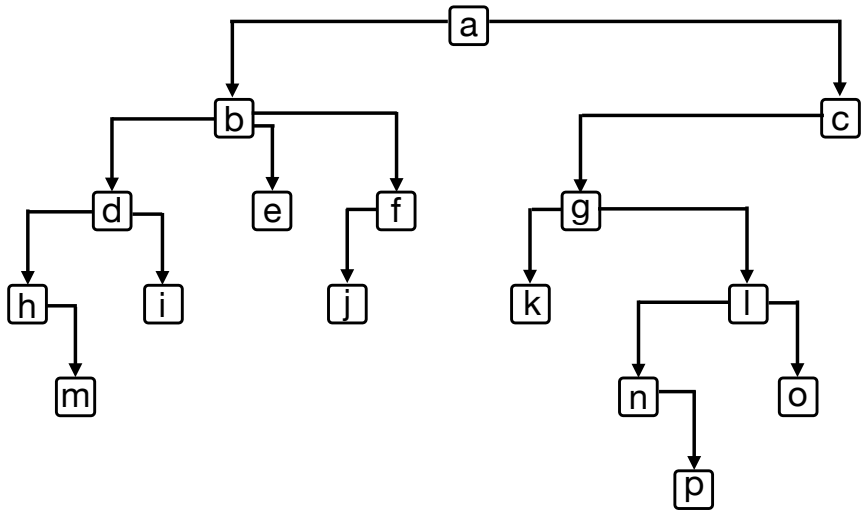


FIG. 11.1. 数据和树的对应。 这棵树包括16个节点、15条连边，其中节点a为树的根节点。

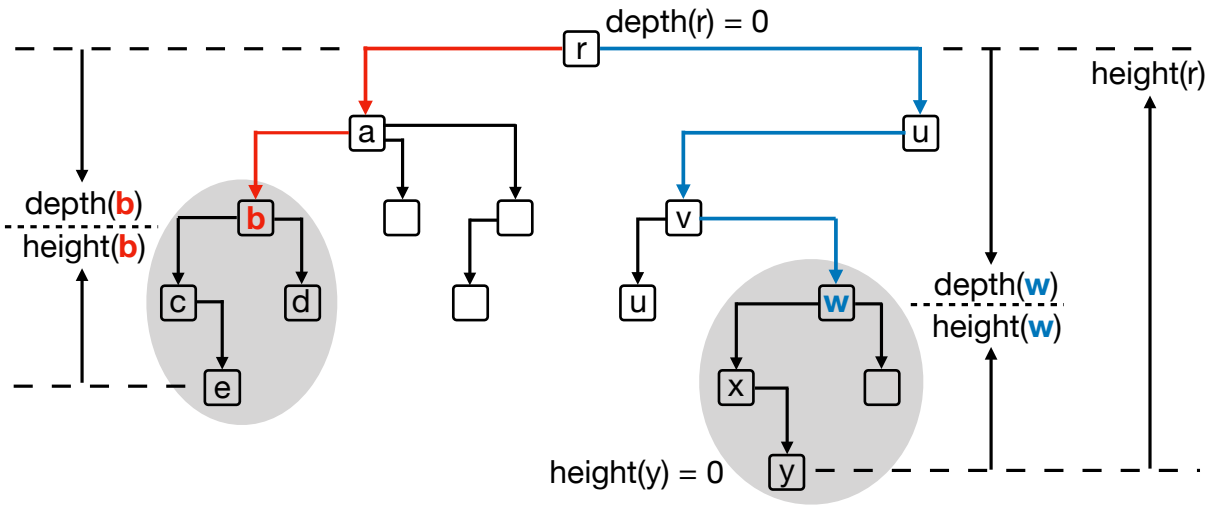


FIG. 11.2. 树的结构性质。 图中展示了树上的一些结构性质，包括从根节点到达指定节点的路径、子树的高度、节点的深度等定义。其中根节点深度定义为0、叶子节点的高度定义为0。

• 节点的父亲(parent)和孩子(child)：给定一个节点，指向它的节点就是它的父亲或者父节点。它指向的节点就是它的孩子或者子节点。 Figure 11.2中，节点b的父节点为节点a，它的子节点为节点c和d。

在列表里，我们也定义了列表节点的前驱(predecessor)节点和后继(successor)节点。但注意此时的两个节点是一类节点，它们之间没有隶属关系。

• 节点的祖先(ancestor)和后代(descendent)：给定节点，沿着连边能够到达它的节

点都是它的祖先，而从它出发沿着连边能够到达的其它节点都叫它的后代。节点的祖先这个节点集包含了节点自身。如果除去节点本身，这个集就是节点的真祖先(proper ancestor)。同样地，节点的后代除开节点本身，就是它的真后代(proper descendent)。Figure 11.2中，节点b的祖先包括节点b、a、r，其中其真祖先为a、r；其后代为b、c、d、e，其中真后代为c、d、e。

• 节点的度(degree)：某个节点的所有子节点的数目。Figure 11.2中，节点a的度为3，b的度为2，而e的度为0。

• 叶节点(leaf)和内部节点(internal nodes)：叶节点表示没有子节点的节点，如果节点有子节点即为内部节点。Figure 11.2中，节点d、e为叶子节点，c、d、a等均为内部节点。

• 树的子树(subtrees)：给定一个节点v，其所有后代构成一棵子树，且其根节点为v。Figure 11.2中，两块色块所覆盖的结构分别为节点b和w作为根节点的子树。

• 节点之间的通路或路径(path)：两个节点之间沿着连边如果能够相连，则连边的序列就为路径。注意，只有节点和它的祖先或者后代节点之间才有连边相连，而且其路径是唯一的。Figure 11.2中，节点b和根结点r之间的路径为(r, a, b)，节点w和根结点r之间的路径为(r, u, v, w)，而节点a和w之间是不相连的。根节点r到节点b和w的路径分别用红色和蓝色标记的连边表示。

• 节点的深度(depth)：树中的任意节点v，如果沿着其父节点可以一直到达根节点。也就是说节点v到根节点r只有唯一的路径。这条通路上连边的数目即为节点v的深度。根节点的深度定义为0。Figure 11.2中，节点a、b、c、d、e的深度分别为1、2、3、3、4。

注意：在一些教材里，根节点的深度定义为1。

• 树的高度(height)：树中每个节点都有其深度，其中最大值即为树的高度。Figure 11.2中树的高度为5，而以节点b和w作为根节点的子树的高度均为2。

• 深度和高度概念之间的关系：节点的深度与它作为根节点的子树的高度之间的关系为：

$$\text{height}(v) + \text{depth}(v) \leqslant \text{height}(\text{root}). \tag{11.1}$$

比如Figure 11.2中，对于节点b来说，其高度为2、深度为2，满足这个不等式。不等式里的等号只有当节点刚好处在树中路径最长的那条序列上才满足，比如Figure 11.2中的节点u、v、w、x、y。

■ 树的一些特殊类型

以下为树的一些特殊构型。例子如Figure 11.3所示。

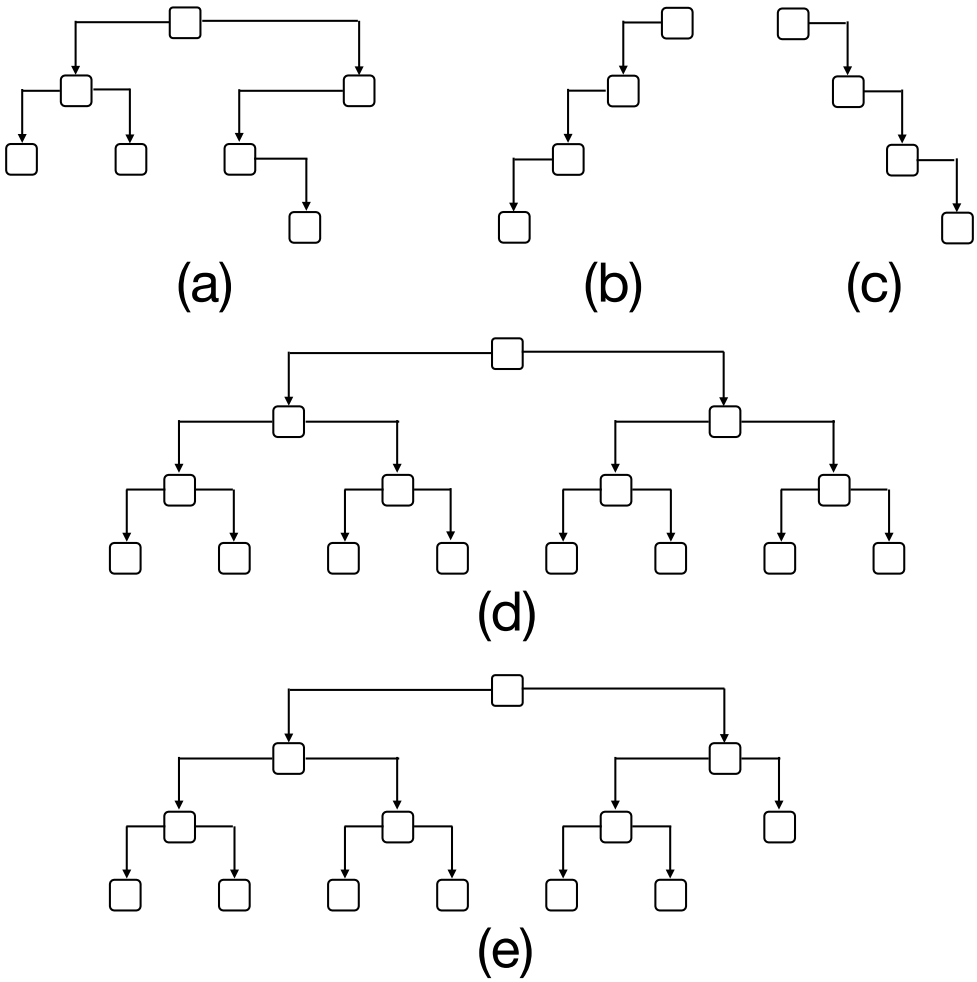


FIG. 11.3. 树的一些特殊类型。 (a)为一个典型的二叉树结构。 (b-c)的树结构里，每个节点只和上下两个节点相连。此时的树退化成了线性序列结构。 (d)为一棵满二叉树。 (e)为一棵完全二叉树。这里的5棵二叉树的深度均为3。

- k叉树(k-ary tree)：每个节点均不超过k个有根树。
- 有序树(ordered tree)：同一个父节点的孩子都可以左右相互区分，具有某一个线性次序。
- 二叉树(binary tree)：树的每个节点的度不超过2，即为0、1、或者2。
- 真二叉树(proper binary tree)：树的内部节点的度都为2。
- 满二叉树(full binary tree)：所有叶节点同处在最底层。

给定二叉树高度h后，可以得知，其对应仅仅一棵满二叉树。我们可以计算出来其节点数目。 容易得知，对于距离跟节点的距离为d的一层节点，其节点数最多为2^d。可以得到

满二叉树的节点数$S(h)$：

$$S(h) = \sum_{l=0}^{d=h} 2^d = 2^{h+1} - 1. \tag{11.2}$$

满二叉树的叶节点在最远的那层，其数目为：

$$S_{\text{leaves}} = 2^h. \tag{11.3}$$

容易得到其内部节点数为：

$$S_{\text{interior}} = S(h) - S_1 = 2^{h+1} - 1 - 2^h = 2^h - 1. \tag{11.4}$$

即满二叉树的叶子节点数比内部节点数大1。

• 完全二叉树(complete binary tree)：叶节点只能出现在最底层的两层，而且最底层中的叶子节点均处在次底层叶子节点的左侧。可以看出来，满二叉树为完全二叉树的一个特例。

给定二叉树高度h后，可知我们可以有多个完全二叉树的构型。但我们计算出来其节点数的范围。 容易看出来，其节点数的最小值对应着高度为$h-1$的满二叉树，再加上一个节点作为距离为l的那一层。 此时的节点数最小值$S_{\min}$为：

$$S_{\min} = \sum_{l=0}^{d=h-1} 2^d + 1 = 2^h. \tag{11.5}$$

节点数的最大值对应着高度为h的满二叉树。此时最大值$S_{\max}$为：

$$S_{\max} = S(h) = 2^{h+1} - 1. \tag{11.6}$$

综合以上结果可知，完全节点数S的范围为$[2^h, 2^{h+1} - 1]$。

完全二叉树里，叶节点数和内部节点数相等、或者前者大1。

■ 从k叉树到二叉树的转换

11.2. 二叉树的ADT

■ 如何表示和存储二叉树的结构?

二叉树的核心结构信息在于连边，也就是某个节点和它的父节点、或者某个节点和它所有子节点所表示的连边。直观上有以下存储方式：

（1）存储每个节点的所有子节点：此时连边存储了一遍；在查询给定节点的子节点时，其操作时间是常数时间；但查询其父节点时，需要遍历整个二叉树。

（2）存储每个节点的父节点：此时连边也存储了一遍；在查询给定节点的父节点时，其操作时间是常数时间；但查询其子节点时，需要遍历整个二叉树。

为了能够同时方便查询所有节点的父节点和子节点，我们采取的是以下的存储方式：

（3）同时存储每个节点的父节点和所有子节点：此时连边被存储了两遍；但在查询给定节点的父节点和子节点时，其操作时间都是常数级。

Figure 11.4展示了给定二叉树的三种存储方式。 上述的存储方式体现了算法设计里一个典型的取舍问题：存储空间和运算时间往往很难同时兼顾；我们往往只能用更大的存储空间来缩短计算时间，或者用更长的运算时间来减少存储空间。上述第3种存储方式是典型的“用空间换时间”的取舍方式。

■ 二叉树结构的ADT

基于上述的存储方式，对于二叉树的每个节点，我们同时存储了其父节点和子节点的指针。 二叉树节点的结构体可以定义为BTNode，其组成单元如Figure 11.5所示。 其基本结构如Figure 11.6(a)所示。

对于二叉树本身，我们可以使用列表或者向量来存储其所有节点。如Figure 11.6(b)中的二叉树，我们使用了Figure 11.6(c–d)中两种描述方式。在列表或向量的描述方式中，对于每个节点都必须设置好其父节点和子节点的指针。容易看出来，列表的描述方式非常贴近二叉树结构本身。但向量描述在内存上更加紧凑。在本章，我们延续了二叉树的向量描述方式。

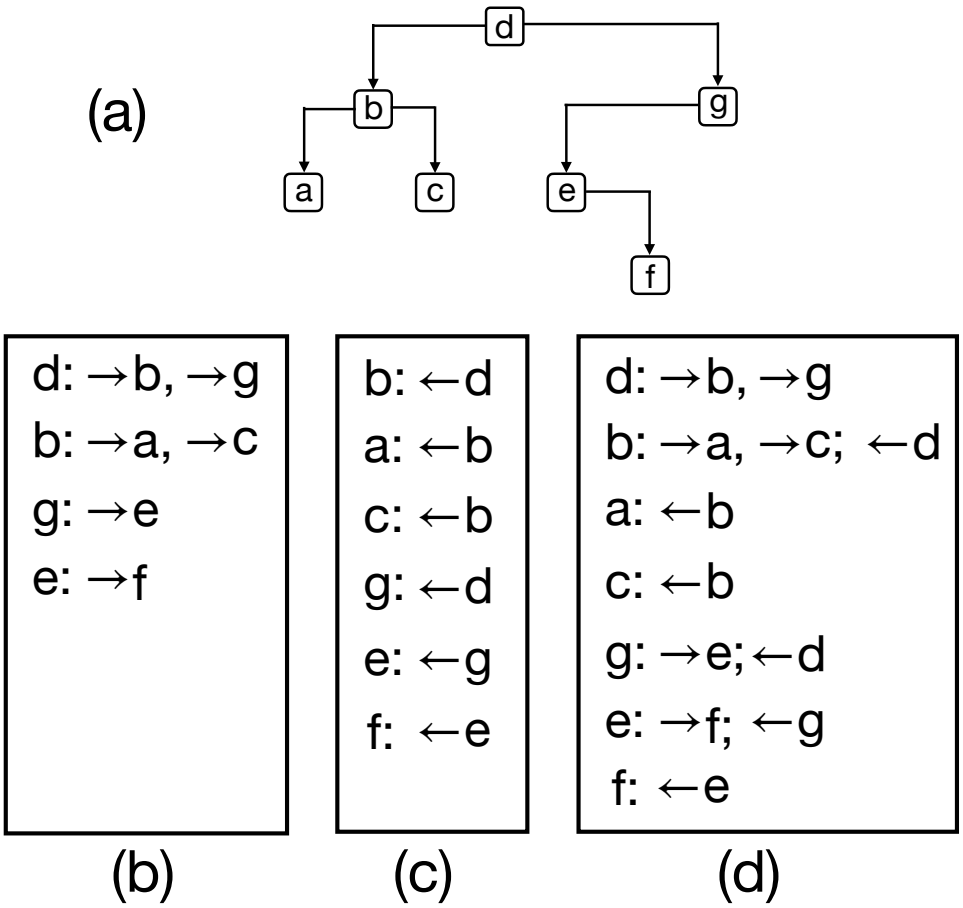


FIG. 11.4. 二叉树的三种存储方式。 (a)为一个包含7个节点、6条连边的二叉树。 (b–d)为此二叉树的三种存储方式。 箭头标记了父节点指向子节点的连边关系。 (b)中存储每个节点的子节点，共包含了6条连边。 (c)中存储每个节点的父节点，共包含了6条连边。 (d)中同时存储每个节点的子节点和父节点，共包含了12条连边。

```
//二叉树的节点的结构体；
struct BTNode {
  //以下为描述二叉树的结构所需的信息；
  char   key;//节点的关键码(key);

  //节点的父节点、子节点信息；
  struct BTNode* ppa;//指向父节点的指针；
  struct BTNode* plc;//指向左子节点的指针；
  struct BTNode* prc;//执行右子节点的指针；

  //节点在二叉树中的位置信息；
  int    dep;//节点的深度(depth)；
  int    het;//节点对应子树的高度(height)；
  int    ord;//节点在遍历过程中的次序(order)；

  // ----------------------------------------
  //以下为二叉树的实际使用中需要的数据成员；

  //节点在连边数据里的信息；
  int    as_pa;//连边数据集中节点作为父节点(parent)的次数；
  int    as_lc;//连边数据集中节点作为左子节点(left-child)的次数；
  int    as_rc;//连边数据集中节点作为右子节点(right-child)的次数；

  //作图矩阵BTMatrix中节点的位置；
  int    xind; //节点的横向位置；
  int    yind; //节点的纵向位置；

  BTNode();
};
```

FIG. 11.5. 二叉树节点的结构体BTNode。

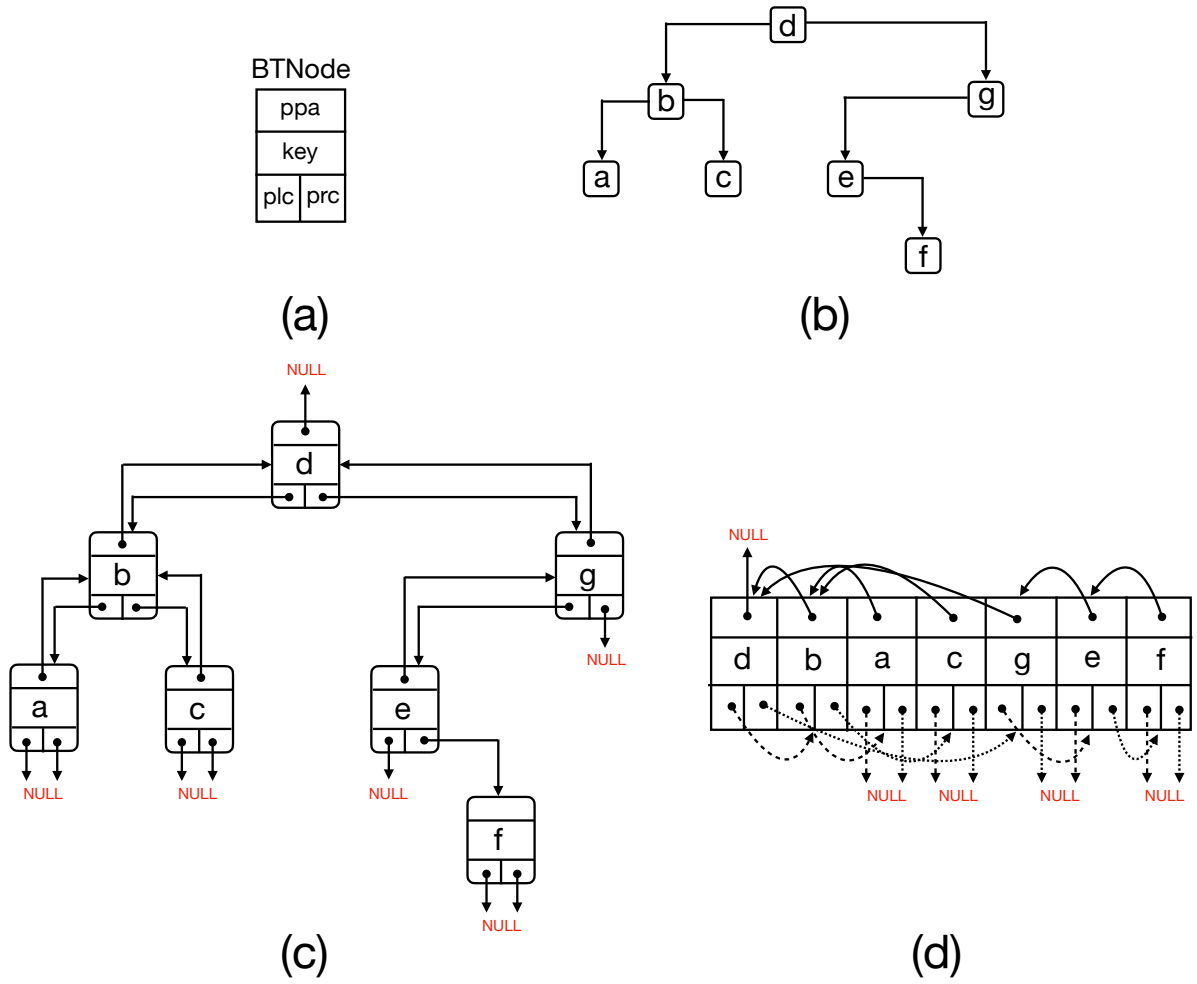


FIG. 11.6. 二叉树的描述方式。 (a)为二叉树节点的结构体的主要内容，包括节点的关键码key、其父节点指针pp、其左子节点指针plc、右子节点prc。 (b)为一棵包含了7个节点、6条连边的二叉树。 (c)为(b)中二叉树的节点序列的列表描述。 (d)为(b)中二叉树的节点序列的向量描述。 (c)和(d)中每个节点的三个指针都已经设置好。 其中实线表示节点指向其父节点的指针，划线表示父节点指向左子节点的指针，点线表示父节点指向右子节点的指针。

■ 二叉树的构建：从连边的数据集来构建二叉树

在构造二叉树时，我们试着实现以下结果：输入二叉树的连边的信息，程序会自动判断数据是否对应真是的二叉树结构；程序构造出来相应的二叉树节点及其连边结构；程序也会得到二叉树的一些全局结构信息，比如树的节点数、树的边数、根节点、树的高度等等。

对于二叉树的连边数据，可以有多种格式来考虑。这里我们考虑的连边信息为三节点组成的三元数，即（父节点，其左子节点、其右子节点）。三节点的数据为向量BTTriplets。其结构体单元为BTTriplet。如Figure 11.7所示。为简单期间，二叉树节点的关键码范围为'a' ~ 'z'，同时空节点可以用'0'表示。

```
//二叉树中三节点对数据的结构体；
struct BTTriplet {
  char pa;
  char lc;
  char rc;
};
```

FIG. 11.7. 二叉树里三节点连边数据的结构体BTTriplet。

对于二叉树的结构，我们使用向量描述方式，即向量BTNodes。构造二叉树时，我们基于连边数据BTTriplets来构造二叉树结构BTNodes。在构造过程的代码里，我们不断读入连边数据；同时构造新的节点，并建立节点之间的连接。同时程序还需要判断二叉树是否出现结构错误，比如有些点同时作为左右子节点、有些节点的出度大于2。此外，程序还要考虑到输入的边集有可能构成一棵森林、而非一棵二叉树。因此代码也需要核实此边集仅对应一个根节点。

假设我们利用三节点对数据来生成一课二叉树，其结构如Figure 11.8所示。其所对应的BTTriplets和BTNodes的构造过程可见Figure 11.9。其构造过程包括以下三个步骤：

（1）**构造BTTriplets**：按边的数据格式把边集数据一条一条地放入BTTriplets。

（2）**利用BTTriplets构造BTNodes**：遍历一次BTTriplets，以构造BTNodes并赋值其中的关键码key。具体来说，逐次读取BTTriplets里的边。如果边里的节点里有BTNodes内并不存在的key，就在BTNodes里加入一个新节点，并赋值这个新的key。BTNodes在内存里以向量形式存储，但节点之间通过父子节点的指针联系起来。

BTNodes的组成单元中，as_pa, as_lc, as_rc直接来自三节点对数据集。每次读取节点时，如果此点为父节点/左子节点/右子节点，则节点相应的as_pa/as_lc/as_rc加1。

边集数据读取完后，我们依然没法直接确定它对应的结构就是一棵二叉树。此时我们

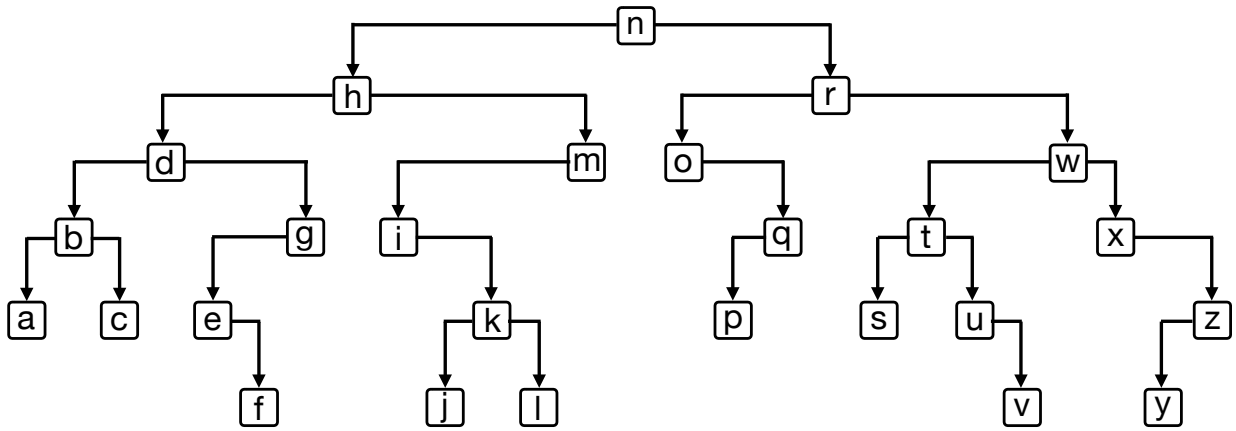


FIG. 11.8. 通过连边数据文件来读取一个二叉树。 此二叉树树包括26个节点和17个父节点左右子节点的三元组合。

可以基于节点的这三个变量来检查它是否满足二叉树结构的约束。容易得知，在一个结构正确的二叉树里，各类节点需要满足以下要求:

（2.a）根节点：as_pa $\in \{1, 2\}$, as_lc $= 0$, as_rc $= 0$;

（2.b）内部节点：as_pa $\in \{1, 2\}$, as_lc + as_rc $= 1$;

（2.c）叶子节点：as_pa$= 0$, as_lc + as_rc $= 1$。

我们可以分别统计上述三类节点的数目。容易得知:

（2.d）如果根节点的数目大于1，则数据集对应多个二叉树组成的一片森林（forest），或者其对应的二叉树结构本身出现了错误。

（2.e）如果上述三类节点的数目之和小于二叉树的节点数，则某些节点可能包含多个父节点、或者超过2个子节点。此时数据集对应的二叉树结构也出现了错误。

（3）**设置BTNodes内节点之间的指针**：再次遍历BTTriplets，以设置BTNodes里的指针。 具体来说，对于BTTriplets里的父子点对，首先在BTNodes找到对应关键码的两个单元，然后给父节点赋予子节点所在单元的指针（plc或prc）、给子节点赋予父节点所在单元的指针（ppa）。根节点指向父节点的指针设为空指针（NULL）。

例子1: 二叉树的构建

给定一个二叉树的连边数据集，构造相应的二叉树结构。

代码见`ch11-bintree.cc`中main()主函数里的ConstructABinaryTree()。

■ 二叉树中查询节点

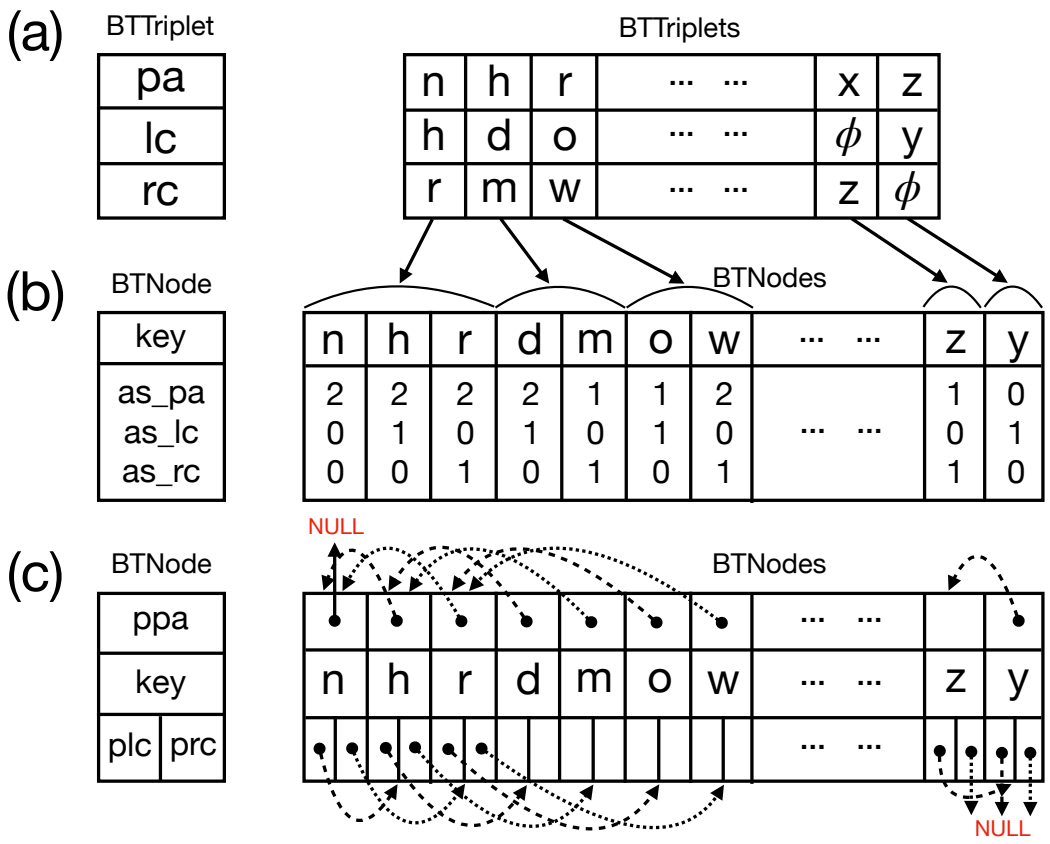


FIG. 11.9. 二叉树的构造过程。 构造过程主要利用两个数据结构BTTriplets和BTNodes。 (a-c)三个子图的左侧为基本的组成单元，右侧为写成向量r形式的数据结构。 (a)对应输入的三节点数据BTTriplets，每条边依次包含父节点、左子节点、右子节点。 空节点用空集符号ϕ表示，在代码中可用'0'表示。 (b)和(c)均为向量形式的BTNodes，但展示了结构题内的不同的数据成员。 (b)对应于遍历一次BTTriplets，以利用连边中的节点信息来构造BTNodes、并赋值key。 基于as_pa/as_lc/as_rc，我们可以检查所生成二叉树的结构是否合理、并确定根节点。 (c)对应于再次遍历ETTriplets，以搭建父子节点之间的指针关系。 图中展示了结构体BTNode里的部分指针，其中实线表示节点指向其父节点的指针，划线表示父节点指向左子节点的指针，点线表示父节点指向右子节点的指针。 根节点n的父节点指针设为空指针（NULL）。

■ 二叉树中插入节点

■ 二叉树中删除节点

11.3. 二叉树的先序、中序、后序遍历

1. 遍历过程的定义

使用数据结构来对数据进行具体操作时，首先得能够实现对数据内各个成员的访问，也就是遍历(traversal)。遍历过程对于线性结构是简单的，但对于二叉树这种结构来说没那么直观。 二叉树的结构为父节点和子节点之间的连接共三个成员。对于二叉树的任意节点r，都可以把二叉树的结构粗粒化为节点r、节点r的左子节点构成的子树（简称为节点r的左子树）、节点r的右子节点构成的子树（简称为节点r的右子树）三部分。 我们可以在局域上递归式地定义访问它们的相对访问顺序。相应地我们考虑以下三种遍历方式：

- 先序遍历(preorder traversal)：依次访问节点r、节点r的左子树、节点r的右子树。
- 中序遍历(inorder traversal)：依次节点r的左子树、节点r、节点r的右子树。
- 后序遍历(postorder traversal)：依次节点r的左子树、节点r的右子树、节点r。

上述“先序”、“中序”、“后序”来源于访问局域结构里的根节点相对于其左子树和右子树的顺序。 上述三种遍历过程的差别如Figure 11.10所示。

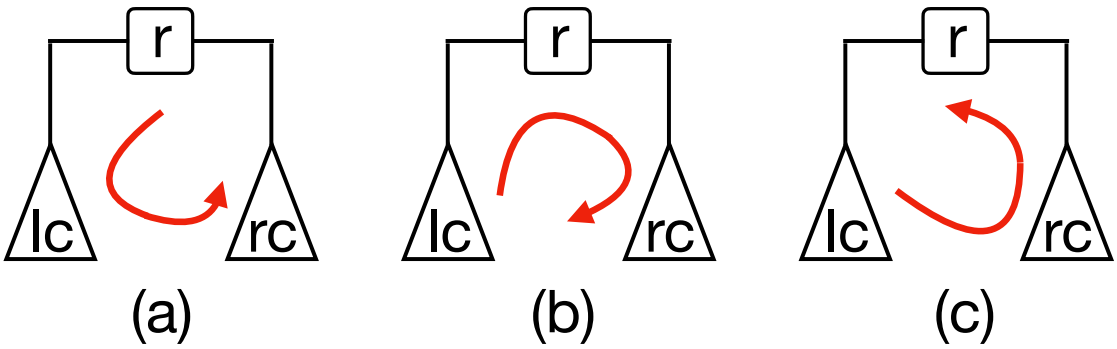


FIG. 11.10. 二叉树上的先序、中序、后序遍历。 图中红色带箭头的曲线表示三种局域结构的访问顺序。

2. 遍历过程的递归版本

在代码实现上，我们可以直接基于遍历过程的递归形式的定义写出相应的递归形式的算法版本。 Figure 11.11展示了三种遍历过程对应的函数。

先序遍历递归版本的算法流程可见Figure 11.12。

中序遍历递归版本的算法流程可见Figure 11.13。

```
void BinTree::PreOrderTraversal_Recursive_AStep(BTNode* pn)
{
  if (pn == NULL) return;

  //访问当下节点；
  VisitANode(pn);

  //访问当下节点的左子节点；
  PreOrderTraversal_Recursive_AStep(pn->plc);

  //访问当下节点的右子节点；
  PreOrderTraversal_Recursive_AStep(pn->prc);
};
void BinTree::InOrderTraversal_Recursive_AStep(BTNode* pn)
{
  if (pn == NULL) return;

  //移动到左子节点；
  InOrderTraversal_Recursive_AStep(pn->plc);

  //访问当下节点；
  VisitANode(pn);

  //移动到右子节点；
  InOrderTraversal_Recursive_AStep(pn->prc);
};
void BinTree::PostOrderTraversal_Recursive_AStep(BTNode* pn)
{
  if (pn == NULL) return;

  //移动到左子节点；
  PostOrderTraversal_Recursive_AStep(pn->plc);

  //移动到左右子节点；
  PostOrderTraversal_Recursive_AStep(pn->prc);

  //访问当下节点；
  VisitANode(pn);
};
```

FIG. 11.11. 二叉树上遍历过程函数的代码。 三个子图分别展示了先序、中序、后序遍历中，在给定节点指针pn后，访问pn对应节点、pn对应的左子树、pn对应的右子树的过程。

中序遍历的序列有一个简单直观的解释。对一个二维平面上的二叉树，可以将它投射到横轴上，也就是不考虑节点在深度上的差别。此时得到的从左到右的序列就是二叉树对应的中序遍历序列。

后序遍历递归版本的算法流程可见Figure 11.14。

三个遍历过程的快速实现。

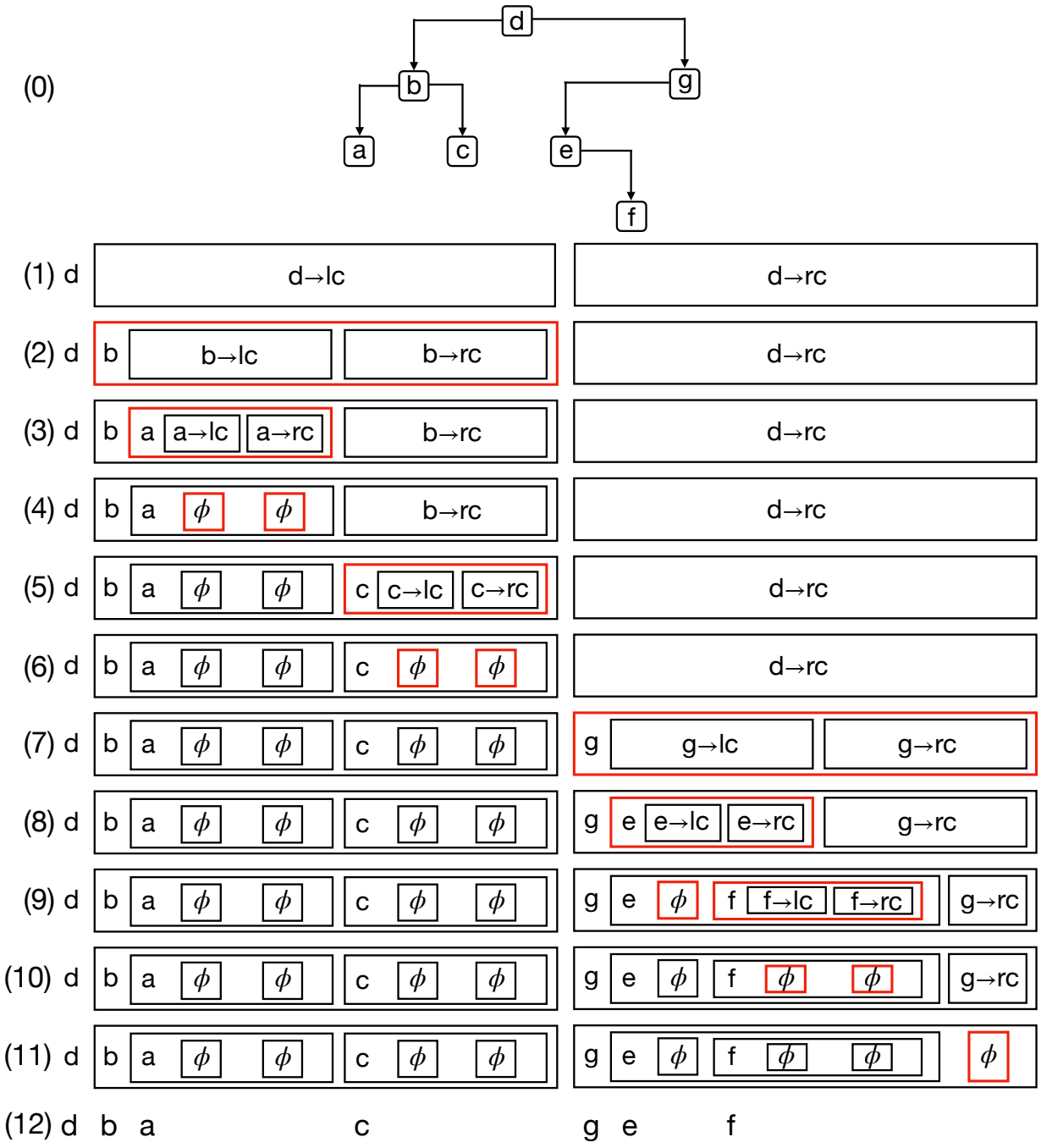


FIG. 11.12. 二叉树上先序遍历的计算流程。 (0)为一个二叉树，包括7个节点。 步骤(1)-(11)为此二叉树上先序遍历算法的递归版本对应的执行过程。 步骤(12)为从(11)中去除空节点后的序列，也就是遍历算法最终得到的节点序列。 图中的方框表示某个节点的左子树或右子树，ϕ表示空节点。图中红色方框表示算法执行中的当前步骤。 经过多次递归过程后，算法的分支到达空节点后进行返回。

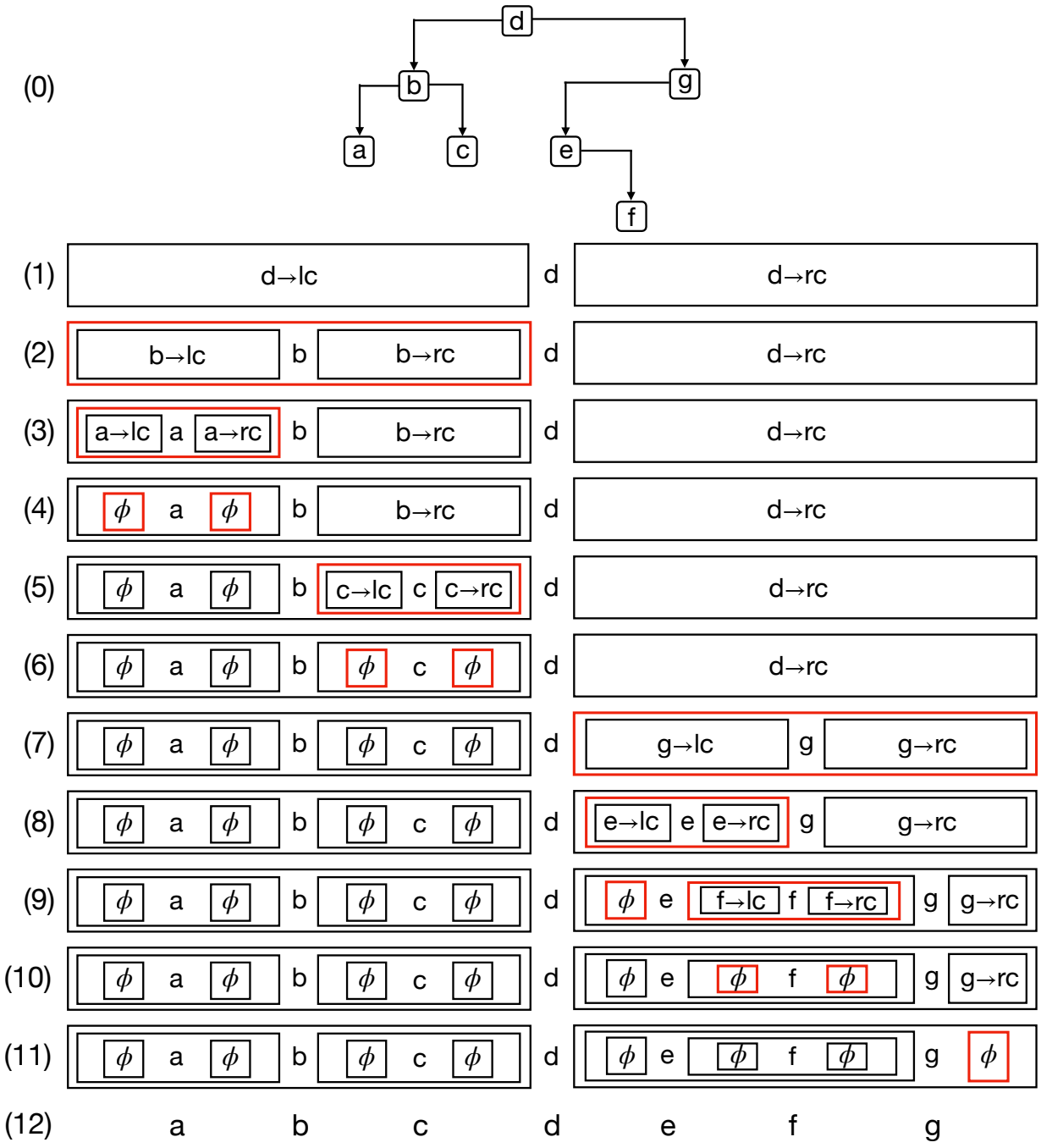


FIG. 11.13. 二叉树上中序遍历的计算流程。 图中的展示方式如Figure 11.12。 步骤(1)-(11)为(0)中二叉树上中序遍历算法的递归版本对应的执行过程。 步骤(12)为从(11)中去除空节点后的序列，也就是遍历算法最终得到的节点序列。

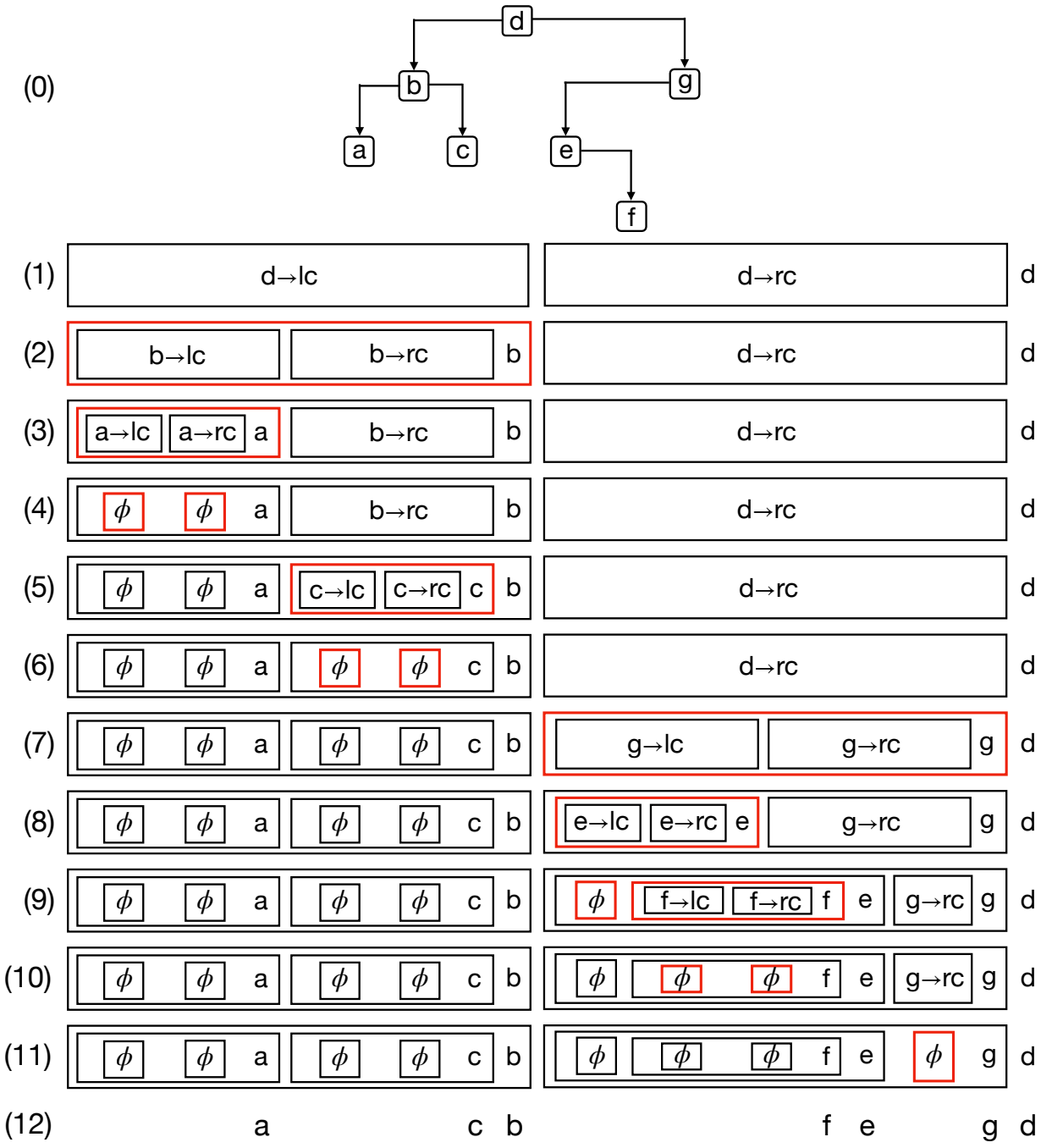


FIG. 11.14. 二叉树上后序遍历的计算流程。 图中的展示方式如Figure 11.12。 步骤(1)-(11)为(0)中二叉树上后序遍历算法的递归版本对应的执行过程。 步骤(12)为从(11)中去除空节点后的序列，也就是遍历算法最终得到的节点序列。

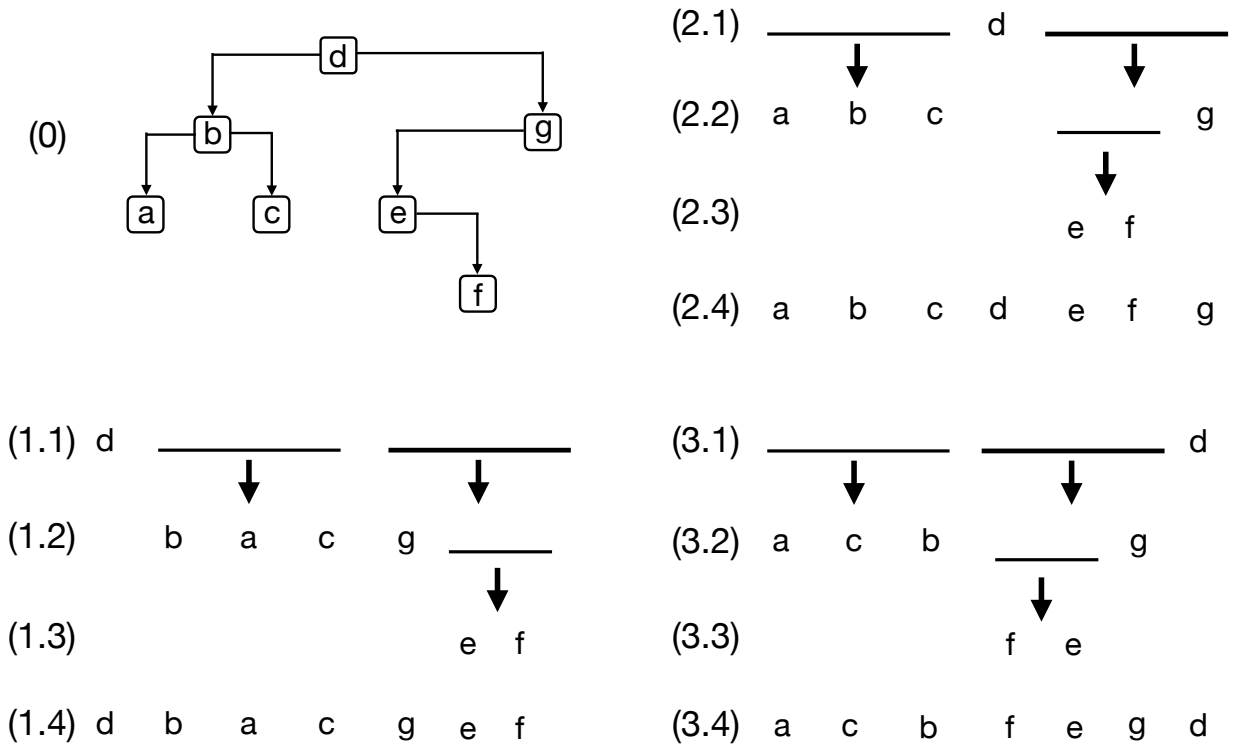


FIG. 11.15. 二叉树上先序、中序、后序遍历的快速计算流程。 步骤(1.1)-(1.3)为(0)中二叉树上先序遍历对应的快速执行过程。 稍细和稍粗的横线分别对应当前步骤里未确定的左子树和右子树的节点序列。 从(1.1)到(1.2)、从(1.2)到(1.3)，我们只写出来了当前确定下来的子树的节点序列。步骤(1.4)为最终的先序遍历序列。 步骤(2.1)–(2.4)、步骤(3.1)–(3.4)分别对应中序和后序遍历过程，其格式和(1.1)–(1.4)相同。

3. 递归算法和迭代算法的比较

这里我们以整数求和的问题为例来展示递归算法和迭代算法之间的差别。 求和问题定义为：

$$f(n) = 0 + 1 + 2 + \cdots + n, n \geqslant 0. \tag{11.7}$$

这个求和可以写成：

$$f(n) = f(n-1) + n, \tag{11.8}$$

$$f(0) = 0. \tag{11.9}$$

在递归版本算法里，对于任意$n \geqslant 0$，算法需要依次调用求和函数来计算$f(n-1), f(n-2), \cdots, f(2), f(1), f(0)$，其中$f(0)$为递归基。之后算法会重新返回计算$f(1), f(2), \cdots, f(n-2), f(n-1), f(n)$。其过程如下：

$$f(n) = f(n-1) + n, \tag{11.10}$$

$$f(n-1) = f(n-2) + n - 1, \tag{11.11}$$

$$f(n-2) = f(n-3) + n - 2, \tag{11.12}$$

$$\cdots \tag{11.13}$$

$$f(2) = f(1) + 2, \tag{11.14}$$

$$f(1) = f(0) + 1, \tag{11.15}$$

$$f(0) = 0, \tag{11.16}$$

$$f(1) = f(0) + 1, \tag{11.17}$$

$$f(2) = f(1) + 2, \tag{11.18}$$

$$\cdots \tag{11.19}$$

$$f(n-2) = f(n-3) + n - 2, \tag{11.20}$$

$$f(n-1) = f(n-2) + n - 1, \tag{11.21}$$

$$f(n) = f(n-1) + n. \tag{11.22}$$

这个过程中，我们需要使用多个中间数据结构来存储所有的$f(i), i \in [1, n-1]$。 而在算法的迭代版本里，对于每一个$f(i), i \in [1, n-1]$，我们都可以相应地计算$f(i+1) = f(i) + i$。这样给定初始值$f(0) = 0$，可以依次直接计算得到$f(1), f(2), f(3), \cdots, f(n-1), f(n)$。其

过程如下：

$$f(0) = 0, \tag{11.23}$$

$$f(1) = f(0) + 1, \tag{11.24}$$

$$f(2) = f(1) + 2, \tag{11.25}$$

$$\cdots \tag{11.26}$$

$$f(n-2) = f(n-3) + n - 2, \tag{11.27}$$

$$f(n-1) = f(n-2) + n - 1, \tag{11.28}$$

$$f(n) = f(n-1) + n. \tag{11.29}$$

最终只需要一次从0到n的循环，就能计算相应的$f(n)$。此时我们只需要一个整数作为中间变量就可以实现算法过程。 Figure 11.16展示了求和问题里递归版本和迭代版本的计算过程。

一般来说，算法的递归版本和迭代版本相比，在代码实现上，前者更符合直观的计算流程，而后者需要透彻地分析算法的执行才能设计出相应的迭代过程，其算法设计的要求比后者要高很多、算法实现的流程也会比前者复杂。在内存空间和计算时间方面，前者需要的中间数据结构和计算时间依赖于迭代过程的深度，而后者一般只需要常数级的中间数据结构和计算时间。这使得递归版本在空间和时间利用效率上都不如迭代版本。

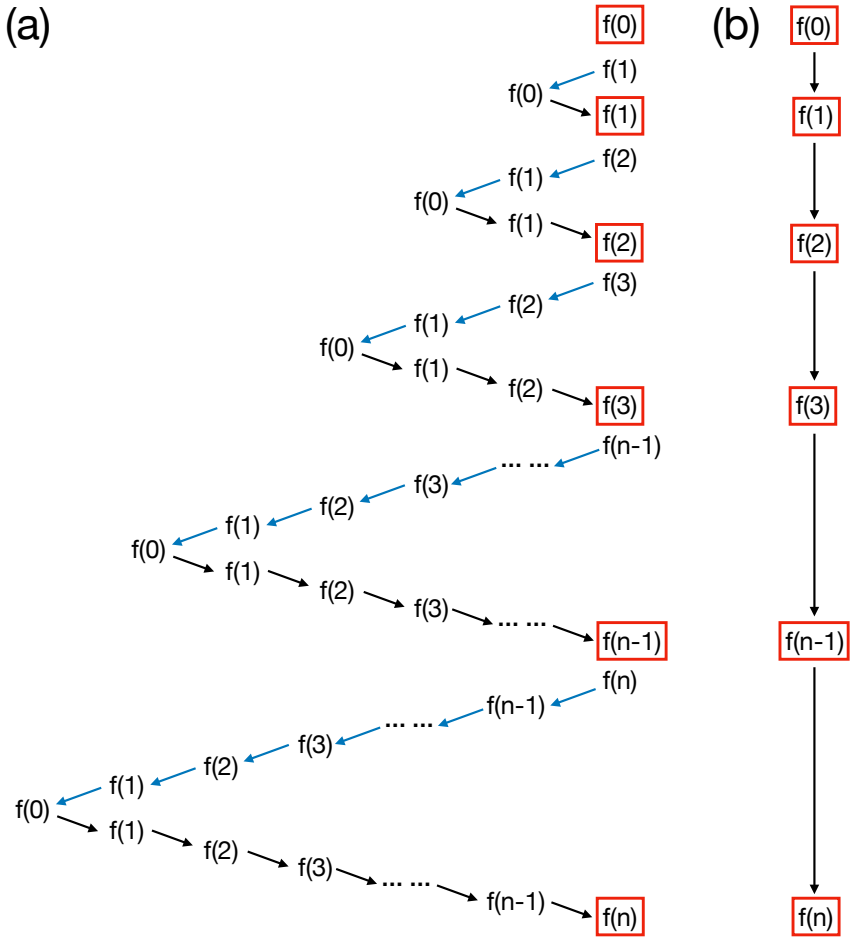


FIG. 11.16. 整数求和的递归算法和迭代算法的差别。 (a)对应着算法的迭代版本，展示了所有计算$f(1)$到$f(n)$的迭代步骤。 (b)对应这算法的迭代版本，展示了从$f(1)$依次到$f(n)$的计算过程。 图中的箭头表示计算的顺序。

4. 遍历过程的非递归版本

上述先序、中序、后序遍历在算法上都是通过递归过程来实现的。除此之外，还可以使用非递归版本的算法。相对于后者，前者虽然在代码实现上非常直接，但需要额外的内存空间来存储中间变量、而且其计算时间依赖于最长的递归深度。而在非递归版本的算法里，一般无需额外的中间变量，而且在计算时会快很多。

- 先序遍历的迭代实现
- 中序遍历的迭代实现
- 后序遍历的迭代实现

11.4. 二叉树的层次遍历

除了基于先序、中序、后序遍历过程之外，还有一类遍历方式：层次遍历(level-order traversal)。 此时我们需要从根节点开始按照离根节点的距离（即深度）依次输出节点。在图这个数据结构上，层次遍历对应着广度优先搜索算法（Breath-first search）。

层次遍历的算法流程可见Figure 11.17。 此遍历过程使用了队列来实现对一层一层节点的连续访问。

11.5. 二叉树的遍历序列与节点的顺序性

给定任意一棵二叉树，我们都可以得到其对应的先序、中序、后序、层次遍历的序列。 对于一个遍历序列，每个节点在其中有具体的位置。此时的遍历序列本质上可以视为遍历过程对应的有序序列。 如Figure 11.18所示，四种遍历序列对应着二叉树节点的四种顺序。

二叉树本质上是一个二维的空间结构：节点在左右方向上可以通过左右子节点进行区分、在上下方向上离根节点有相应的距离（即节点的深度）。但从遍历的角度，这个二维结构可以通过一维序列来表示。 上述四种遍历序列本质上是节点的一种排列，其遵循的特定顺序依赖于遍历过程的定义。 如果用一个线性序列表示它们，此时四种序列的每一个节点都有它确定的下标。 对于任意节点，可以定义其在相应序列的前一个点为其前驱节点、后一个结点为其后继节点。 如Figure 11.18所示，对于节点k来说：

- 在先序遍历情形下，其前驱节点为i、后继节点为j;
- 在中序遍历情形下，其前驱节点为j、后继节点为l;
- 在后序遍历情形下，其前驱节点为l、后继节点为i;

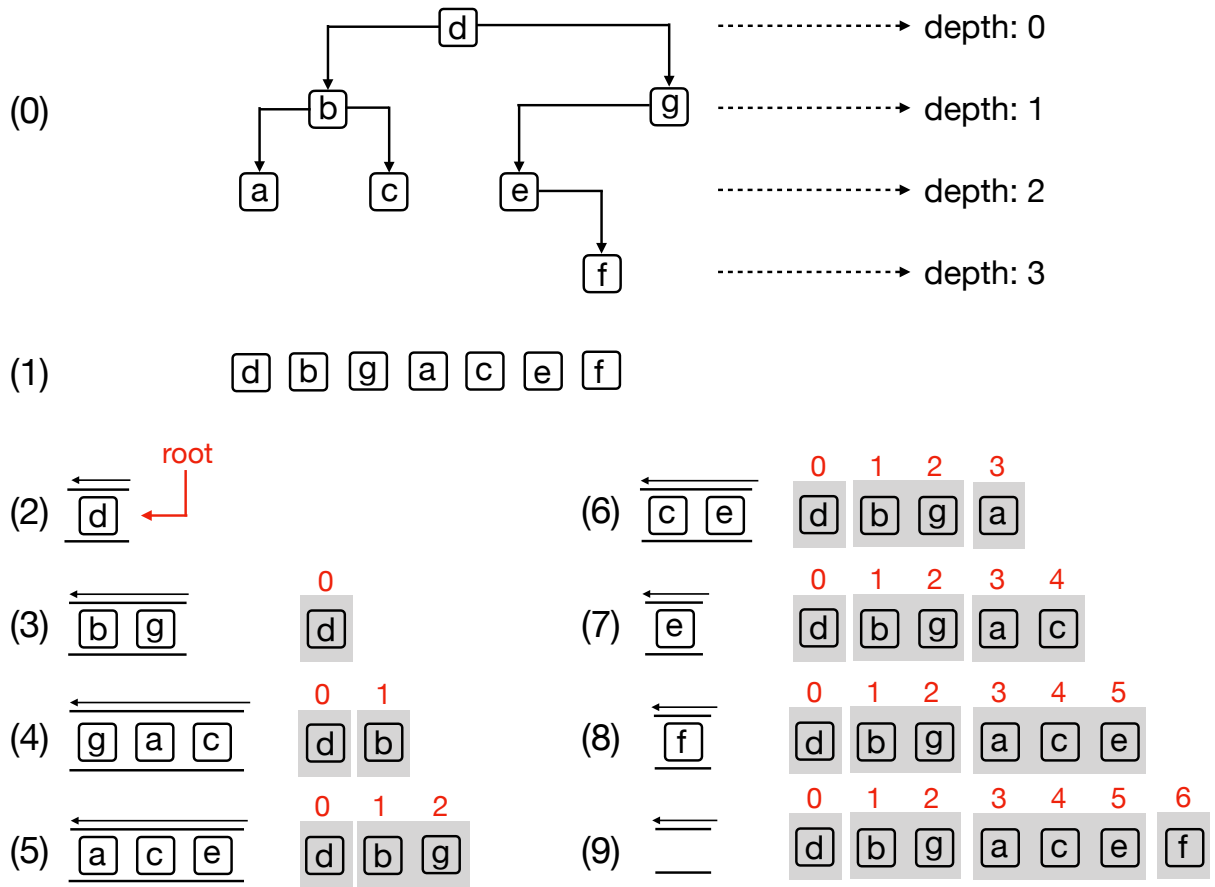


FIG. 11.17. 二叉树上层次遍历的计算流程。 (0)为一棵二叉树。 (1)为相应的层次遍历的序列。步骤(2)-(9)为此二叉树上层次遍历对应的遍历过程。 每行包括队列内部的元素（箭头表示从队尾到队首的方向），以及访问过的节点。步骤(9)中队列为空，相应地遍历过程结束。访问过的节点上方的数字表示访问顺序的编号。图中不同色块表示不同的节点深度。

◦ 在层次遍历情形下，其前驱节点为e、后继节点为p。

这里有个直接的问题：给定一棵二叉树，可否使用算法来得到任意节点在某个遍历序列意义上的前驱节点和后继节点?

以下我们以中序遍历意义下的顺序性为例。

■ 中序遍历序列背景下的顺序性

对于二叉树里的任意节点，我们可以把寻找其前驱节点和后继节点的过程写成函数形式。

Figure 11.19展示了在中序遍历意义下寻找一些节点的前驱节点和后继节点的过程。

容易得知，对于任意节点t的前驱节点：

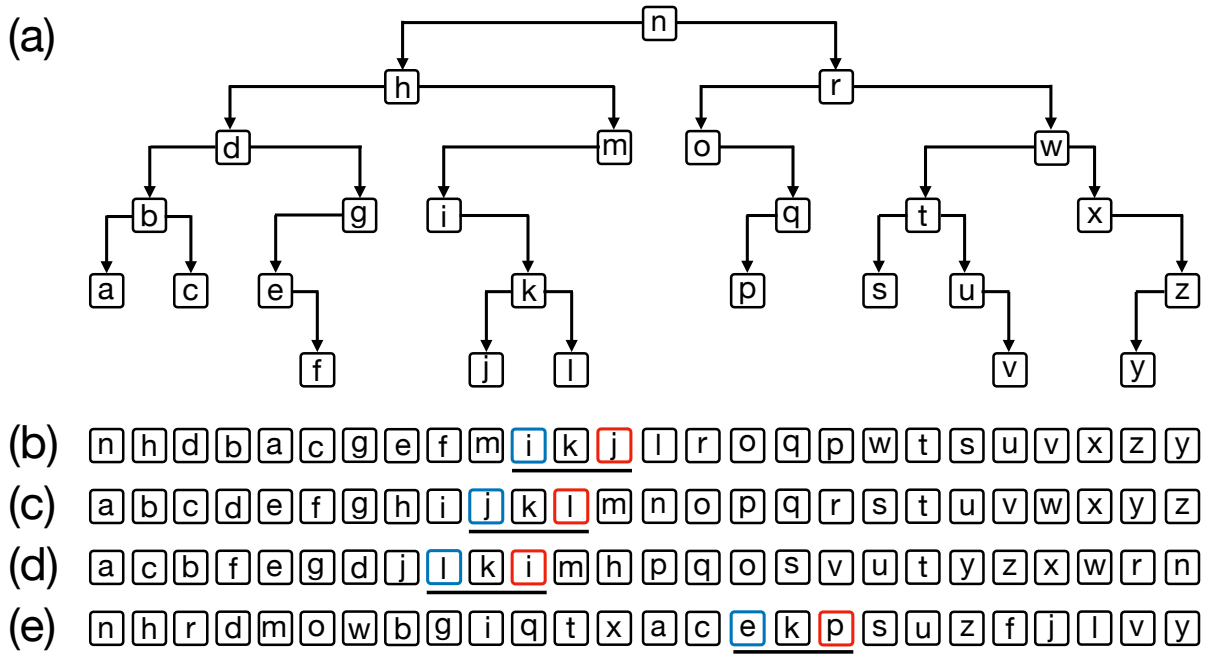


FIG. 11.18. 二叉树结构对应的遍历序列。 (a)为Figure 11.8中的二叉树，包含了26个节点。 (b) - (d)分别为(a)中二叉树的先序、中序、后序、层次遍历的序列。遍历序列里的横线标记了各自序列里的节点k及其前驱和后继节点。

（1）如果t有左子树： 假设其左子节点为lc，t的前驱节点即为左子树lc对应的中序遍历序列的最大值。在二叉树里，前驱节点对应着沿着左子树lc的右子树一直遍历下去，直到没有右子节点的节点p为止。节点p即为t的前驱节点。

（2）如果t没有左子节点：意味着t本身是某个子树的中序遍历序列里的最小值。此时t的前驱节点即为此二叉树根节点作为右子节点的父节点。那么如何从t出发寻找到这棵二叉树？可以从t往上遍历：如果t为某个节点的左子节点，则移到此节点。如此不断进行下去，直到一个节点s不再为某个节点的左子节点。节点t即为节点s对应子树的中序遍历序列里的最小者。相应地节点s的父节点即为节点t的前驱节点。

对于任意节点t的后继节点：

(1) 如果t有右子节点：假设其右子节点为rc，t的后继节点即为右子树rc对应的中序遍历序列的最小值。在二叉树里，后继节点对应着沿着右子树rc的左子树一直遍历下去，直到没有左子节点的节点s为止。节点s即为t的后继节点。

(2) 如果t无右子节点：意味着t本身是某个子树的中序遍历序列里的最大值。此时t的后继节点即为此二叉树根节点作为左子节点的父节点。那么如何从t出发寻找到这棵二叉树？可以从t往上遍历：如果t为某个节点的右子节点，则移到此节点。如此不断进行下

去，直到一个节点s不再为某个节点的右子节点。节点t即为节点s对应子树的中序遍历序列里的最大者。相应地节点s的父节点即为节点t的后继节点。

■ 层次遍历序列里的顺序性

完全二叉树和满二叉树里的顺序性。

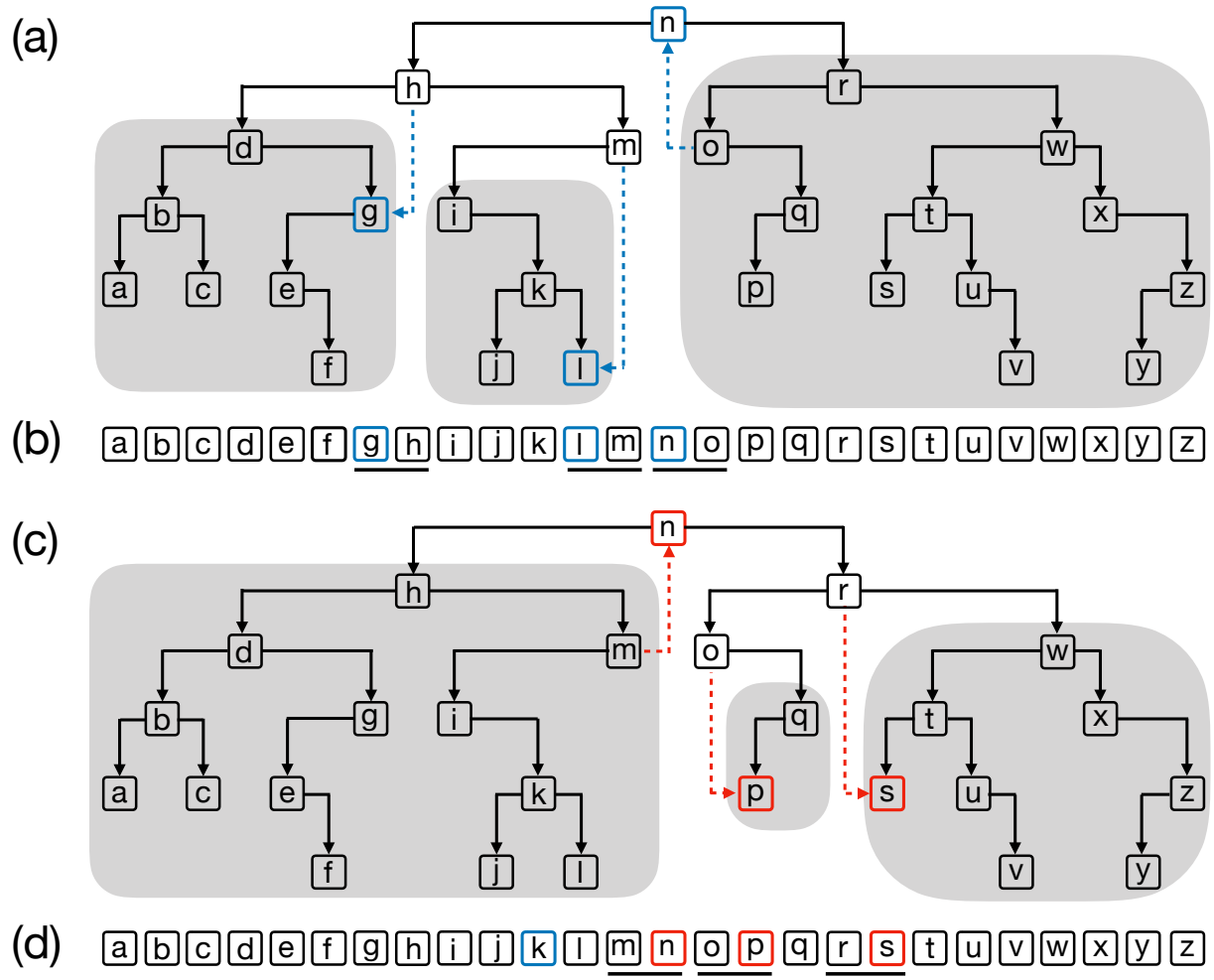


FIG. 11.19. 在二叉树里寻找基于中序遍历序列的前驱节点和后继节点。 (a)展示了分别寻找节点h、m、o的前驱节点的过程。其中h和m均有左子节点、o点无左子节点。 对于节点h来说，对应的色块分别为在中序遍历序列里刚好在h之前的子树，其中g为这个子树里最靠后的节点。 节点m和l的情况类似。 对于节点o来说，对应的色块为o节点刚好在中序遍历序列里最靠左的最大子树，n为相应子树的父节点。 (b)为节点h、m、o在中序遍历序列里的情形。 (c)展示了分别寻找节点m、o、r的后继节点的过程。其中o和r均有右子节点、m点无右子节点。 对于节点o来说，对应的色块分别为在中序遍历序列里刚好在o之后的子树，其中p为这个子树里最靠前的节点。 节点r和s的情况类似。 对于节点m来说，对应的色块为m节点刚好在中序遍历序列里最靠右的最大子树，n为相应子树的父节点。 (d)为节点m、o、r在中序遍历序列里的情形。

11.6. 从二叉树的遍历序列反推其构型

对于二叉树的遍历，最常见的问题场景是给定了二叉树的构型，然后写出对应的遍历序列，比如先序、中序、后序、层次遍历的序列。如Figure **??**所示，给定(a)里的结构后，可以得到(b)里对应的遍历序列。 但我们还可以考虑一个反方向的问题，也就是给定了某个二叉树的遍历序列，可否反推出来此二叉树的构型。如Figure 11.20所示，我们从(b)中的遍历序列出发，来反推出来(a)中的二叉树构型，如果能够实现的的话。 上述两个问题典型的正问题和反问题(inverse problem)，对应着从模型预测结果、从结果反推模型。

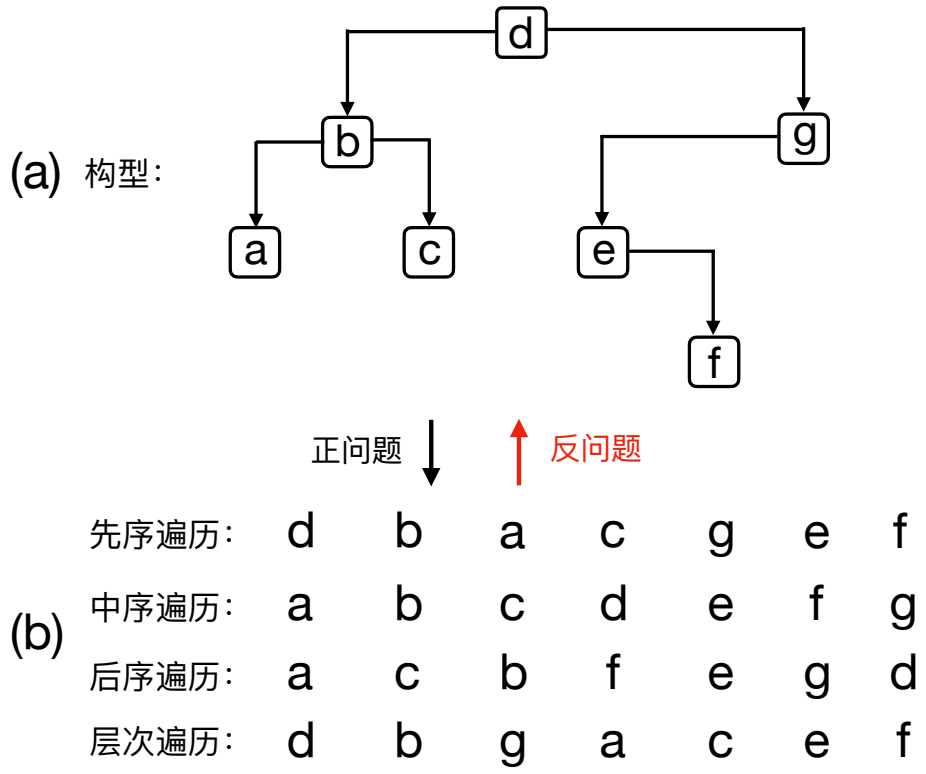


FIG. 11.20. 二叉树的构型和遍历序列之间的正反问题。 (a)为一棵包含7个节点的二叉树。 (b)为二叉树对应的先序、中序、后续、层次遍历的序列。

如何从二叉树的遍历序列反推其构型？主要思路还是利用了遍历序列在二叉树局域结构里的基本性质。 对于当下二叉树的根节点root及其左子树lt、右子树rt，其先序遍历、中序序列、后序遍历的顺序依次为：

$$P = \{\text{root}, \text{lt}, \text{rt}\}, \tag{11.30}$$

$$I = \{\mathrm{lt}, \mathrm{root}, \mathrm{rt}\}, \tag{11.31}$$

$$O = \{\mathrm{lt}, \mathrm{rt}, \mathrm{root}\}. \tag{11.32}$$

通过比对不同的遍历序列，我们可以在局域层面不断确定当下二叉树的根节点和左右子树。如此过程可以迭代进行下去，最终以确定二叉树的所有节点。

这里为了简单考虑，我们考虑以下三种情形。

1，从先序和中序遍历序列反推二叉树的构型

2，从中序和后序遍历序列反推二叉树的构型

3，从先序和中序遍历序列反推二叉树的构型

■ 1，从先序和中序遍历序列反推二叉树的构型

假设二叉树的节点数为n，其先序和中序遍历序列分别为：

$$P = \{P_0, P_1, \cdots, P_{n-1}\}, \tag{11.33}$$

$$I = \{I_0, I_1, \cdots, I_{n-1}\}. \tag{11.34}$$

由先序遍历的定义可知，先序遍历序列里的P_0为二叉树的根节点。此时中序序列I中有一节点为根节点，记为I_m，$0 \leqslant m \leqslant n-1$。此时我们可以通过中序遍历序列来区分和标记左右子树对应的节点集，同时相应地在先序遍历序列里标记出来左右子树的节点集。此时有

$$P = \{P_0, \underline{P_1, P_2, \cdots, P_m}, \underline{P_{m+1}, P_{m+2}, \cdots, P_{n-1}}\}; \tag{11.35}$$

$$I = \{\underline{I_0, I_1, \cdots, I_{m-1}}, I_m, \underline{I_{m+1}, I_{m+2}, \cdots, I_{n-1}}\}. \tag{11.36}$$

此时，根节点的左子树和右子树的遍历序列分别为：

$$P = \{P_1, P_2, \cdots, P_m\}, \{P_{m+1}, P_{m+2}, \cdots, P_{n-1}\}; \tag{11.37}$$

$$I = \{I_0, I_1, \cdots, I_{m-1}\}, \{I_{m+1}, I_{m+2}, \cdots, I_{n-1}\}. \tag{11.38}$$

容易得知，上述过程能够唯一地确定对应子树的根节点及其左右子树的节点集。这是反推构型算法的基本过程。这个过程可以迭代地进行下去。随着算法的执行，未确定状态的子树节点会逐渐减少，直到最终所有节点的状态都被确定下来。

下面我们考虑两种简单情形：

（1）子树有3个节点，其先序和中序遍历序列为：

$$P = \{P_0, P_1, P_2\}; \tag{11.39}$$

$$I = \{I_0, I_1, I_2\}. \tag{11.40}$$

可知：

（1.a）当$P_0 = I_0$为根节点时，左子树为空，右子树节点集为$\{P_1, P_2\}$、即$\{I_1, I_2\}$；

（1.b）当$P_0 = I_1$为根节点时，左子树为$P_1 = I_0$，右子树为$P_2 = I_2$；

（1.c）当$P_0 = I_2$为根节点时，左子树节点集为$\{P_1, P_2\}$、即$\{I_0, I_1\}$，右子树为空。

（2）子树有2个节点，其先序和中序遍历序列为：

$$P = \{P_0, P_1\}, \tag{11.41}$$

$$I = \{I_0, I_1\}. \tag{11.42}$$

可知：

（2.a）当$P_0 = I_0$为根节点时，左子树为空，右子树为$P_1 = I_1$；

（2.b）当$P_0 = I_1$为根节点时，左子树为$P_1 = I_0$，右子树为空。

综上所述，如果两个遍历序列正确，使用上述迭代过程后，可以得到唯一的二叉树构型。

例子2: 某个二叉树的先序和中序遍历序列分别为：

$$P = \{d, b, a, c, g, e, f\}, \tag{11.43}$$

$$I = \{a, b, c, d, e, f, g\}. \tag{11.44}$$

请反推出其二叉树的构型。

解：

第1步：由先序遍历序列可知d为当下的根节点，其左右子树的遍历序列分别为：

$$P = \{b, a, c\}, \{g, e, f\}, \tag{11.45}$$

$$I = \{a, b, c\}, \{e, f, g\}. \tag{11.46}$$

第2步：对于第1步中的左子树，由其先序遍历序列可知其根节点为b。继续由其中序遍历序列可知，其左右子树分别为a和c。

对于第1步中的右子树，由其先序遍历序列可知其根节点为g，其左子树的遍历序列为：

$$P = \{e, f\}, \tag{11.47}$$

$$I = \{e, f\}, \tag{11.48}$$

右子树为空。

第3步：对于第2步里的右子树，由其先序遍历序列可知e为其根节点，其左子树为空，右子树为f。

综上所述，上述算法步骤以及最终的二叉树构型可见Figure 11.21。

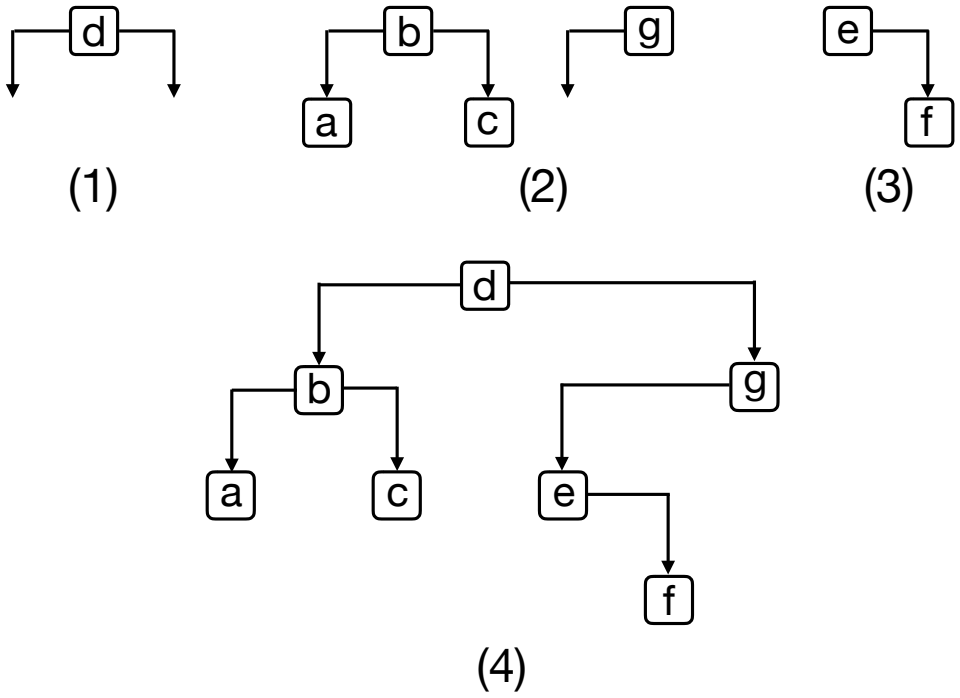


FIG. 11.21. 利用二叉树的先序和中序遍历序列来反推二叉树的构型。 子图(1-3)分别表示算法描述中的第1-3步。子图(4)对应于算法最终得到的二叉树构型。

■ 2，从中序和后序遍历序列反推二叉树的构型

假设二叉树的节点数为n，其中序和后序遍历序列分别为：

$$I = \{I_0, I_1, \cdots, I_{n-1}\}, \tag{11.49}$$

$$O = \{O_0, O_1, \cdots, O_{n-1}\}. \tag{11.50}$$

由后序遍历的定义可知，后序遍历序列里的O_{n-1}为此二叉树的根节点。则序列I中有一节点为根节点，记为I_m，$0 \leqslant m \leqslant n-1$。 此时我们可以通过中序遍历序列来区分和标记左右子树对应的节点集，同时相应地在先序遍历序列里标记出来左右子树的节点集。此时

有

$$I = \{\underline{I_0, I_1, \cdots, I_{m-1}}, I_m, \underline{I_{m+1}, I_{m+2}, \cdots, I_{n-1}}\}; \tag{11.51}$$

$$O = \{\underline{O_0, O_1, \cdots, O_{m-1}}, \underline{O_m, O_{m+1}, \cdots, O_{n-2}}, O_{n-1}\}. \tag{11.52}$$

此时根节点的左子树和右子树的遍历序列分别为

$$I = \{I_0, I_1, \cdots, I_{m-1}\}, \{I_{m+1}, I_{m+2}, \cdots, I_{n-1}\}; \tag{11.53}$$

$$O = \{O_0, O_1, \cdots, O_{m-1}\}, \{O_m, O_{m+1}, \cdots, O_{n-2}\}. \tag{11.54}$$

上述确定子树的根节点及其左右子树节点集的划分是确定的。其划分步骤可以不断进行下去，直到所有节点的状态都被确定。

下面我们考虑两种简单情形：

（1）子树有3个节点，其中序和后序遍历序列为：

$$I = \{I_0, I_1, I_2\}, \tag{11.55}$$

$$O = \{O_0, O_1, O_2\}. \tag{11.56}$$

可知：

（1.a）当$I_0 = O_2$为根节点时，左子树为空，右子树节点集为$\{I_1, I_2\}$、即$\{O_0, O_1\}$；

（1.b）当$I_1 = O_2$为根节点时，左子树为$I_0 = O_0$，右子树为$I_2 = O_1$；

（1.c）当$I_2 = O_2$为根节点时，左子树节点集为$\{I_0, I_1\}$、即$\{O_0, O_1\}$，右子树为空。

（2）子树有2个节点，其先序和中序遍历序列为：

$$I = \{I_0, I_1\}, \tag{11.57}$$

$$O = \{O_0, O_1\}. \tag{11.58}$$

可知：

（2.a）当$I_0 = O_1$为根节点时，左子树为空，右子树为$I_1 = O_0$；

（2.b）当$I_1 = O_1$为根节点时，左子树为$I_0 = O_0$，右子树为空。

综上所述，如果两个遍历序列正确，使用上述迭代过程，最终可以得到唯一的二叉树构型。

例子3: 某个二叉树的中序和后序遍历序列分别为：

$$I = \{a, b, c, d, e, f, g\}, \tag{11.59}$$

$$O = \{a, c, b, f, e, g, d\}. \tag{11.60}$$

请反推出其二叉树的构型。

解：

第1步：由后序遍历序列可知d为当下的根节点，其左右子树对应的遍历序列分别为：

$$I = \{a, b, c\}, \{e, f, g\}, \tag{11.61}$$

$$O = \{a, c, b\}, \{f, e, g\}. \tag{11.62}$$

第2步：对于第1步中的左子树，由后序遍历序列可知其根节点为b。进而由中序遍历序列可知，其左右子树分别为a和c。

对于第1步中的右子树，由后序遍历序列可知其根节点为g。进而由中序遍历序列可知，其左子树的遍历序列为：

$$I = \{e, f\}, \tag{11.63}$$

$$O = \{f, e\}, \tag{11.64}$$

右子树为空。

第3步：对于第2步里的右子树，由后序遍历序列可知e为其根节点。由中序遍历序列可知其左子树为空，右子树为f。

综上所述，上述步骤以及最终的二叉树构型和Figure 11.21相同。

■ 3，从先序和后序遍历序列反推二叉树的构型

假设二叉树的节点数为n，其先序和后序遍历序列分别为：

$$P = \{P_0, P_1, \cdots, P_{n-1}\}, \tag{11.65}$$

$$O = \{O_0, O_1, \cdots, O_{n-1}\}. \tag{11.66}$$

由先序和后序遍历的定义可知，$P_0 = O_{n-1}$为根节点，其左右子树合并后的遍历序列分别为：

$$P = \{P_1, P_2, \cdots, P_{n-1}\}, \tag{11.67}$$

$$O = \{O_0, O_1, \cdots, O_{n-2}\}. \tag{11.68}$$

注意上述序列是左右子树合并后的节点集。这里我们已经没法如之前两种情况那样可以分别区分出来左子树和右子树的节点集。下面我们考虑两种情况：

（1）如果$P_1 \neq O_{n-2}$：容易得知，此时同时存在非空的左子树和右子树。而且P_1为左子树的根节点，O_{n-2}为右子树的根节点。 在后序遍历序列O中，我们设左子树的根节点为$O_m = P_1$，$0 \leqslant m \leqslant n-3$。相应可知$P_{m+2} = O_{n-2}$为右子树的根节点。此时左右子树的遍历序列分别如下：

$$P = \{P_1, P_2, \cdots, P_{m+1}\}, \{P_{m+2}, \cdots, P_{n-1}\}, \tag{11.69}$$

$$O = \{O_0, O_1, \cdots, O_m\}, \{O_{m+1}, \cdots, O_{n-2}\}. \tag{11.70}$$

此时我们可以唯一地确定左右子树的节点集。

（2）如果$P_1 = O_{n-2}$：此时有两种子情况：

（2.a）左子树为空。左右子树的节点集如下：

$$P = \varnothing, \{P_1, P_2, \cdots, P_{n-1}\}, \tag{11.71}$$

$$O = \{O_0, O_1, \cdots, O_{n-2}\}, \varnothing. \tag{11.72}$$

其中，$\varnothing$表示空集。

（2.b）右子树为空。左右子树的节点集如下：

$$P = \{P_1, P_2, \cdots, P_{n-1}\}, \varnothing, \tag{11.73}$$

$$O = \varnothing, \{O_0, O_1, \cdots, O_{n-2}\}. \tag{11.74}$$

因为上述情形（2）的存在，导致我们在处理此类情况时可能会有多种可能的左右子树的结构。这意味着如果用先序和后序遍历序列来反推二叉树构型的话，我们可能不会得到唯一的构型。

下面我们考虑两种简单情形：

（1）子树有3个节点，其先序和后序遍历序列为：

$$P = \{P_0, P_1, P_2\}, \tag{11.75}$$

$$O = \{O_0, O_1, O_2\}. \tag{11.76}$$

由先序和后序遍历序列可知，$P_0 = O_2$为根节点。此时其左右子树合并后的遍历序列为：

$$P = \{P_1, P_2\}, \tag{11.77}$$

$$O = \{O_0, O_1\}. \tag{11.78}$$

（1.a）如果$P_1 \neq O_1$，则$P_2 = O_1$。此时其左子树为$P_1 = O_0$，右子树为$P_2 = O_1$。

（1.b）如果$P_1 = O_1$，此时有两种可能性：

（1.b.1）左子树为$\{P_1, P_2\}$、即$\{O_0, O_1\}$、且其根节点为$P_1 = O_1$。右子树为空。

（1.b.2）左子树为空。右子树为$\{P_1, P_2\}$、即$\{O_0, O_1\}$、且其根节点为$P_1 = O_1$。

（2）子树有2个节点，其先序和后序遍历序列为：

$$P = \{P_0, P_1\}, \tag{11.79}$$

$$O = \{O_0, O_1\}. \tag{11.80}$$

由先序和后序遍历序列可知，$P_0 = O_1$为根节点。此时存在两种情形：

（2.a）左子树为$P_1 = O_0$，右子树为空；

（2.b）左子树为空，右子树为$P_1 = O_0$。

结合上述2个节点的结果，我们可以把3个节点情况下所有的可能构型，即（1.a）、（1.b.1）、（1.b.2）三种情形所对应的5种二叉树构型都确定下来。其构型可见Figure 11.22。

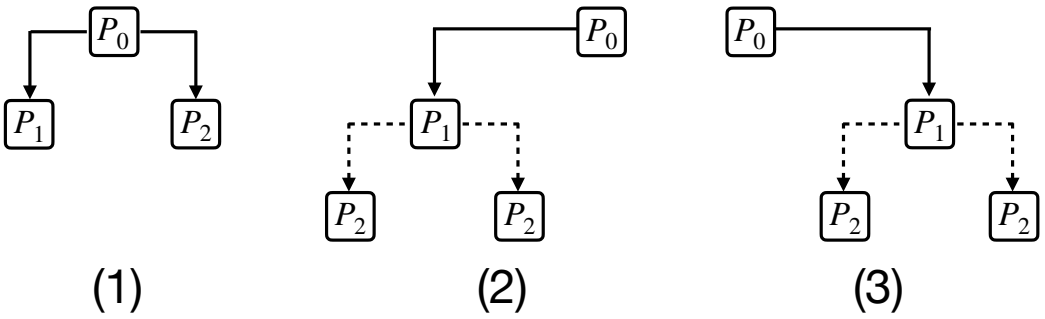


FIG. 11.22. 利用二叉树的先序和后序遍历序列来反推二叉树的构型。 此时考虑3个节点的情形。子图（1-3）分别对应着文中的情形（1.a）、（1.b.1）、（1.b.2）。 图中的虚线表示可能的子节点结构。

例子4: 某个二叉树的先序和后序遍历序列分别为：

$$P = \{d, b, a, c, g, e, f\}, \tag{11.81}$$

$$O = \{a, c, b, f, e, g, d\}. \tag{11.82}$$

请反推出其二叉树的构型。

解：

第1步：由先序和后序遍历序列的定义可知，d为当下的根节点。左右子树合并后的遍历序列分别为：

$$P = \{b, a, c, g, e, f\}, \tag{11.83}$$

$$O = \{a, c, b, f, e, g\}. \tag{11.84}$$

因为$b \neq g$，此时可以区分左子树和右子树各自对应的遍历序列，如下所示：

$$P = \{b, a, c\}, \{g, e, f\}, \tag{11.85}$$

$$O = \{a, c, b\}, \{f, e, g\}. \tag{11.86}$$

第2步：对于第1步得到的左子树，由先序和后序遍历序列可知b为根节点。其左右子树合并后的遍历序列为：

$$P = \{a, c\} \tag{11.87}$$

$$O = \{a, c\}. \tag{11.88}$$

因为$a \neq c$，此时存在非空的左右子树，且其左右子树分别为a和c。

对于第1步得到的右子树，由先序和后序遍历序列可知g为根节点。其左右子树合并后的遍历序列为：

$$P = \{e, f\}; \tag{11.89}$$

$$O = \{f, e\}. \tag{11.90}$$

第3步：对于第2步得到的右子树，可知其先序遍历序列的头元素和后序遍历序列的尾元素相等，即$e = e$。此时有两种情况需要考虑：

（1）左子树为空，且右子树的遍历序列为：

$$P = \{e, f\}; \tag{11.91}$$

$$O = \{f, e\}. \tag{11.92}$$

此时e为根节点，但其左右子树仍然有两种子情况：

（1.a）左子树为空，右子树为f；

（1.b）左子树为f，右子树为空。

（2）右子树为空，且左子树的遍历序列为：

$$P = \{e, f\}; \tag{11.93}$$

$$O = \{f, e\}. \tag{11.94}$$

此时e为根节点，但其左右子树仍然有两种子情况：

（2.a）左子树为空，右子树为f；

（2.b）左子树为f，右子树为空。

综合上述结果可知，对于给定的两个遍历序列，我们能够得到四种可能的二叉树构型。 算法对应的过程和最终得到的二叉树构型可见Figure 11.23。

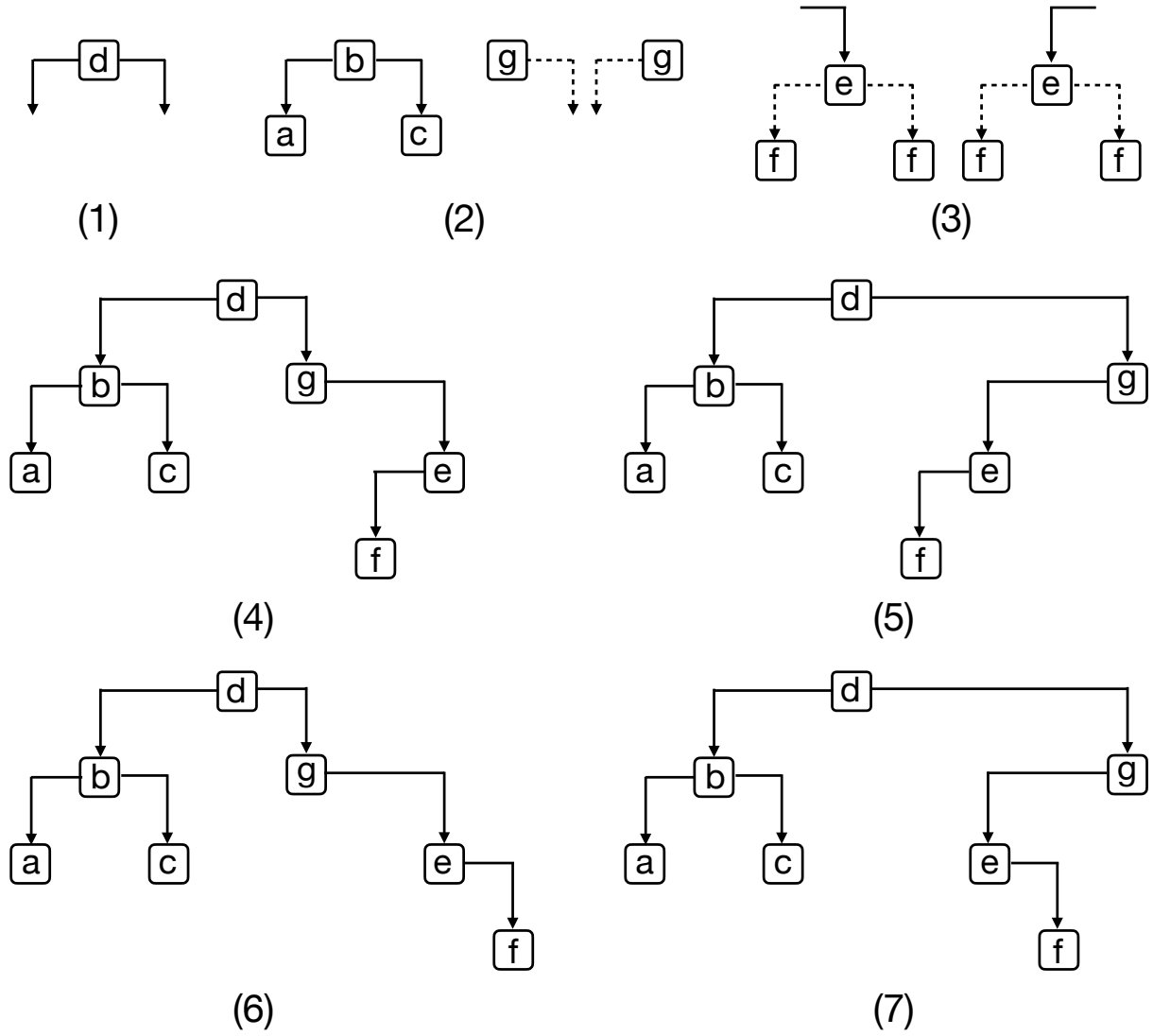


FIG. 11.23. 利用二叉树的先序和后序遍历序列来反推二叉树的构型。 子图(1-3)分别表示算法描述中的第1-3步。图中的虚线表示可能的子节点结构。 子图(4-7)对应于序列对应的4种二叉树构型。

11.7. 二叉树的作图

二叉树的结构存储信息在其数据结构里。为了直观地展示二叉树的结构本身，我们可以使用算法把任意二叉树的数据结构作为输入、以二维图片的形式把它的结构展示出来。展示格式类似于Figure 11.24。

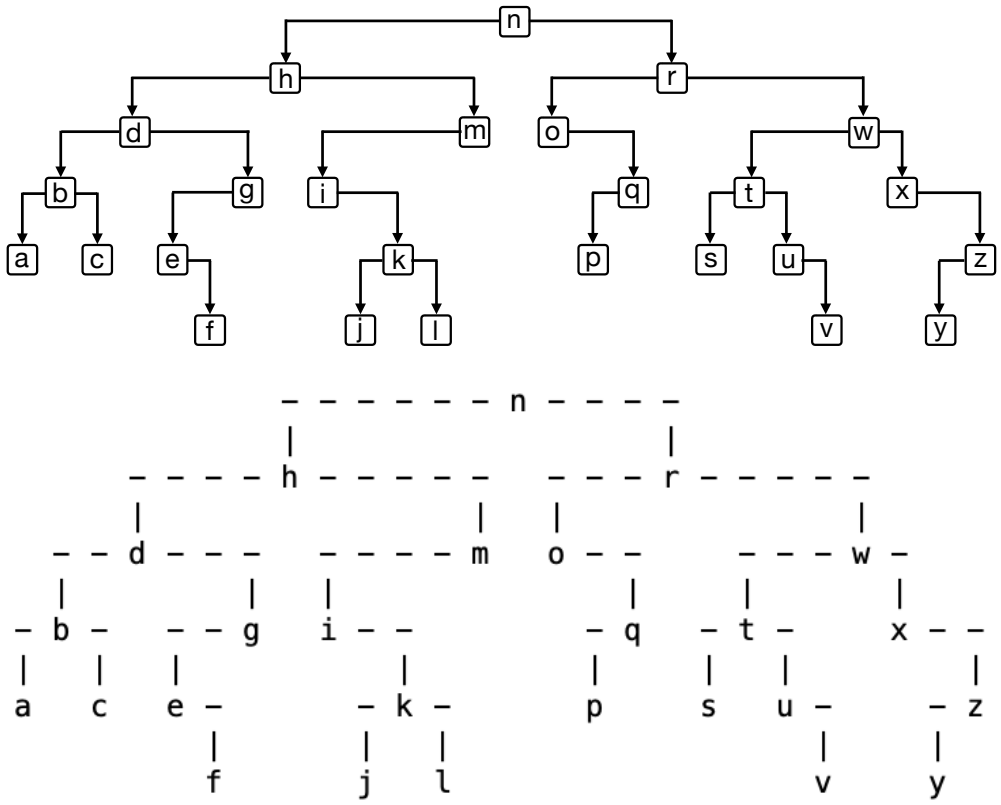


FIG. 11.24. 二叉树的作图。（上图）一棵二叉树，包括26个节点。其编号依次为字符{a ∼ z}。（下图）使用二叉树作图算法所得到的输出图像。

为了更直观地展示二叉树的二维构型，可以将任意二叉树按照其拓扑结构以二维矩阵格式直接输出。二叉树的节点用其关键码标识，父子节点之间的连线可用线段符号表示。

这里作图的基本逻辑是把二叉树的连接结构转换成二维字符矩阵的形式： 节点的关键码本身为字符型变量，可以直接输出；节点之间的横向和纵向线段分别用字符'−'和'|'表示。二叉树的结构信息体现在父节点和子节点之间的连接关系里，但这个信息是完全局域的。作图算法的核心问题是如何把这种二叉树的局域结构信息在二维平面上体现出来。这里我们可以利用二叉树遍历来寻找节点之间的相对位置。具体来说，就是使用二叉树的中序遍历来寻找节点在横轴方向上的相对顺序，使用层次遍历来寻找节点在纵轴方向上的相

对顺序。这样利用两个遍历算法可以得到每个节点在二维平面上的相对位置，通过设置坐标的起始点、节点在横向和纵向上的相对间隔，就能得到每个节点在二维平面上的坐标。之后在图中的具体坐标画出节点、连接父子节点作为连边等过程也就简单很多了。

整体的算法流程如下：

（1）首先读取二叉树的连边数据，在内存中构造出二叉树。

（2）使用中序遍历得到每个节点在横轴上的编号，从0开始。同时使用层次遍历来得到每个节点在纵轴上的深度。此时可以得到二叉树的高度。对于每个节点，高度减去其深度，也就是节点相对于最深层节点的距离。这个值可以作为节点在纵轴上的编号，也是从0开始。

（3）利用节点在横轴和纵轴上的编号来计算其二维坐标。最终在用来做图的字符矩阵上的相应位置里画上节点的序号。

（4）基于二叉树里连边的信息，在字符矩阵上标记出父节点到子节点之间的连边。

（5）对于已经构造好了的二维字符矩阵，逐行输出即可得到对应的二叉树的图示。

具体的数据结构和计算过程如下：

（1） 二维字符矩阵的表示。 我们使用数据结构为vector⟨char⟩ BTMatrix来作为作图的二维字符矩阵。 二维矩阵的宽度为int N_col，高度为int N_row，分别为列数和行数。 二维字符矩阵的初始值为空格，关键码和线段在后续过程中赋值相应的字符。BTMatrix在内存里本质上是一个一维矩阵。对于二维矩阵里的坐标(int xind, int yind)，其对应的一维矩阵里的下标可以写为int mind (= xind + yind * N_col)。因为这种二维坐标到一维下标的转换很简单，后续我们将直接在二维绘图矩阵的语言下描述矩阵元素的操作。 考虑到二叉树的深度从根节点处为0开始计算，我们约定二维矩阵的横纵坐标的顺序为从左到右、从上到下。

（2）节点在二维矩阵上以模块单元来展示。 使用二维字符矩阵来表示二叉树的难点在于如何确定节点的相对间隔和节点的具体位置。 对于前者，我们使用基本构成模块来作为矩阵的构成单元以确定节点之间的间隔； 对于后者，我们使用节点在中序遍历和层次遍历中的两个序号来作为它在二维矩阵里的坐标。 为了定量地界定图像中节点之间的间隔大小，我们首先假定二维矩阵由基本长方形模块构成。 基本模块的宽度和高度分别为int col_size ($\geqslant 1$)和int row_size ($\geqslant 2$)。 基本模块中或者有关键码、或者只有线段、或者没有任何字符。 为简单起见，我们假设关键码处在基本模块的左上角元素，即$(0,0)$元素。 此时节点间的线段处在两个关键码在横轴和纵轴方向上的矩阵元素。 示意图可见Figure 11.25，展示了如何给二叉树对应的二维矩阵单元赋值。

（3）节点在二维平面上的坐标。 二维矩阵中基本模块的数目可以由关键码在横向和

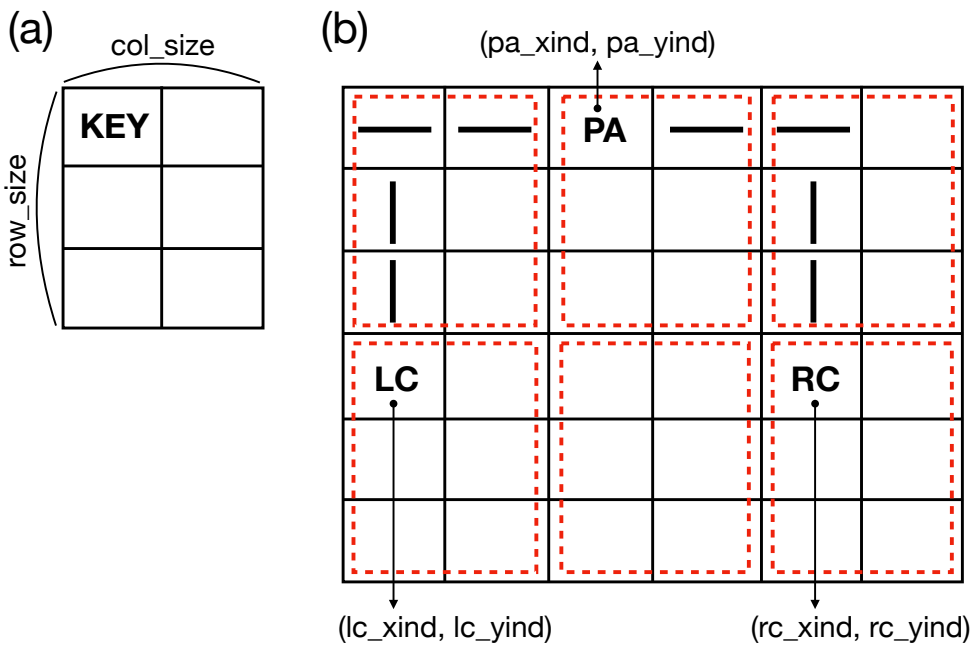


FIG. 11.25. 二叉树作图的二维字符矩阵表示。 (a)为一个节点在二维字符矩阵里的显示单元。它包含了多个基本字符单元，其横向包括col_size个字符单元、纵向包括row_size个字符单元。节点key放在显示单元的左上角。 (b)展示了如何上对节点之间的连边进行作图。PA、LC、RC分别表示一个父节点、左子节点、右子节点对应的key值。三个节点在二维字符矩阵的坐标用箭头进行了标识。父节点和子节点之间的连边通过横向和纵向的线段字符来表示。

纵向上的投影数目来确定。 横向上，可以对二叉树进行中序遍历，得到每个点在中序遍历意义下的下标ord。 纵向上，可以对二叉树进行层次遍历，得到每个点的深度dep。中序遍历下标和深度的组合(ord, dep)可以视为节点在二叉树里的“坐标”。 相应地我们可以得到节点关键码在二维字符矩阵BTMatrix中的坐标(xind, yind)， 其中xind = ord * col_size, yind = dep * row_size。 在二维矩阵表示中，这样选择使得节点在横轴和纵轴上的分布都比较均匀。

（4）节点之间的连边的作图。 在二维字符矩阵BTMatrix里表示节点时，可以直接在节点的坐标(xind, yind)处赋值相应的关键码key。 在表示节点之间的连线时，我们考虑由连边连接三个节点的情形： 节点分别为父节点、左子节点、右子节点(pa, lc, rc)， 其坐标分别为(pa_xind, pa_yind)，(lc_xind, lc_yind)，(rc_xind, rc_yind)。 从父节点引出到左子节点的连边包括两段：

（4.a）横向线段部分的横坐标为[lc_xind, pa_xind），纵坐标为pa_yind；

（4.b）纵向线段部分的横坐标为lc_xind，纵坐标为[pa_yind + 1, lc_yind)。

从父节点引出到右子节点的连边包括两段：

（4.c）横向线段部分的横坐标为[pa_xind + 1, rc_xind]，纵坐标为pa_yind；

（4.d）纵向线段部分的横坐标为rc_xind，纵坐标为[pa_yind + 1, rc_yind)。

对于只有左子节点或右子节点的连边，按上面方式选择相应的一条连边即可。

最终，基于BTMatrix，逐行输出其字符即可得到二叉树的图示。

例子5: 二叉树的作图

这里按照给定的二叉树结构，即Figure 11.8(a)，使用作图算法画出对应的图示。

代码：`ch11-bintree.cc`中main()主函数里的DrawABinaryTree()。

绘图结果在Figure 11.24的下图。其中二维作图模块单元的大小为cols_per_elem = 1, rows_per_elem = 2。

习题

11.1 二叉树的构型

考虑一棵包含了11个节点的二叉树。 以下对其构型的一些要求：

（1）其中序遍历的序列为{a, b, c, d, e, f, g, h, i, j, k}。

（2）其高度为4。（此时假定根节点的深度为0。）

（3）其根节点的两棵子树的高度不相等，且高度差的绝对值为1。

（4）此二叉树有一棵子树为满二叉树。

注意可能存在多棵二叉树能够同时满足上述所有约束。 请画出其中一棵二叉树，并写出此二叉树对应的先序遍历和后序遍历对应的序列。

11.2 二叉树的遍历

考虑Figure 11.8中的二叉树。 请完成代码以实现此二叉树的构建、及对它的遍历，并按照要求输出树节点编号的序列。

此处所考虑的遍历过程和输出格式的要求如下：

（1）依次实现先序遍历、中序遍历、后序遍历，并输出遍历序列。

（2）实现层次遍历的代码，并分行输出节点所在的深度、同一层所有节点的关键码。按图中二叉树的结构，前3行输出为：

0 : n

1 : h r

2 : d m o w

11.3 二叉树的遍历与操作

考虑Figure 11.8中的二叉树。这里我们将基于二叉树的遍历对节点来执行一些简单的任务。 请完成代码以实现此二叉树的构建、及相应的遍历算法。 然后按照中序遍历的顺序来输出满足一些结构要求的节点编号、及其总数。其结构要求如下：

（1）节点编号的范围为$\geqslant d$、且$\leqslant v$。

（2）节点的深度$\geqslant 2$。（此时假设根节点的深度为0。）

（3）节点为内部节点。

11.4 二叉树的遍历序列和节点有序性

考虑Figure 11.8(a)中的二叉树。 请完成代码以实现此二叉树的构建、及对它的中序遍历，并按照要求输出树节点编号的序列。

（1）实现寻找给定节点的前驱节点的代码，并输出节点g、l、o、r的结果。

（2）实现寻找给定节点的后继节点的代码，并输出节点g、l、o、r的结果。

11.5 从二叉树的遍历序列反推其构型

考虑一棵包含了11个节点的二叉树，其节点标记为{a, b, c, d, e, f, g, h, i, j, k}。 假如现在你手头有三个遍历序列：

- 先序遍历序列：{b, k, j, g, a, i, e, h, d, f, c}；
- 中序遍历序列：{g, j, a, k, b, e, h, i, d, c, f}；
- 后序遍历序列：{g, a, j, k, h, e, c, f, d, i, b}。

请验证上述三个遍历序列是否能够同时对应某个二叉树结构。此时你可以首先选择两个序列以构造出对应的二叉树，然后把这棵树和第三个序列做对比、以核对它们是否相容。 请写出自己的推导过程。 如果三个序列对应同一棵二叉树，请画出这棵树的结构；如果它们不对应于某个棵树，请指出并改正遍历序列中的不相容的地方，并画出对应的树结构。

注：此题无需写代码。

11.6 从二叉树的遍历序列反推其构型

考虑一棵包含了11个节点的二叉树，其节点标记为{a, b, c, d, e, f, g, h, i, j, k}。 假如现在你手头有三个遍历序列：

- 先序遍历序列：{g, c, f, a, h, d, j, b, i, k, e}；

- 中序遍历序列：{c, f, a, g, j, d, h, k, i, b, e}；
- 后序遍历序列：{a, f, c, j, d, k, i, e, b, h, g}。

请验证上述三个遍历序列是否能够同时对应某个二叉树结构。此时你可以首先选择两个序列以构造出对应的二叉树，然后把这棵树和第三个序列做对比、以核对它们是否相容。 请写出自己的推导过程。 如果三个序列对应同一棵二叉树，请画出这棵树的结构；如果它们不对应于某个棵树，请指出并改正遍历序列中的不相容的地方，并画出对应的树结构。

注：此题无需写代码。

12. (BINARY SEARCH TREE)

参考内容：邓俊辉教材第7.1–4节。李春葆教材第9.3.1–2节。

在“向量”一章里，我们能看到它在静态操作上为常数级计算时间$\mathcal{O}(1)$、动态操作的操作时间依赖于序列长度$\mathcal{O}(n)$。在“列表”一章里，我们看到它在静态操作上的时间依赖于序列长度$\mathcal{O}(n)$、动态操作的时间为常数级$\mathcal{O}(1)$。也就是说，向量和列表在静态操作和动态操作上是无法兼顾的。但这个“无法兼顾”是在线性序列的场景下来说的。如果换了一个场景，两种操作的计算时间可能同时兼顾。

本章介绍了二叉搜索树的一些基本概念及其基本操作。 同时介绍了渐近平衡二叉树的概念。AVL树为一种常见的渐近平衡二叉树，我们介绍了其结构定义、以及重平衡操作过程。

12.1. 二叉搜索树的基本性质

• 二叉树里的节点

此时我们考虑节点使用关键码（key）来表示。关键码支持比较（大小）和比对（相等）的操作。常见的关键码比如数字、具有字典序的字符等等。

• 二叉树搜索（Binary Search Tree, BST）：BST可以认为是一种特别的、带有顺序性的二叉树。 从二叉树到二叉搜索树，我们需要对每个节点的关键码的大小使用约束：任意节点均不小于其左子树、不大于其右子树。 这不是节点与节点之间的比较，而是节点和节点集合之间的比较，是比“任意节点不小于其左子节点、不大于其右子节点”更严格的要求。其要求如Figure 12.1(1)所示。容易得知，Figure 12.1(2-3)中的二叉树为二叉搜索树，但(4)中的二叉树不为二叉搜索树。

在局域上，这个有序性的要求只发生在当下节点及其左右子树之间。但当二叉树里所有节点满足这个要求时，二叉树的所有节点会呈现出来一个有序性。这个有序性对应着一个有序的中序遍历序列。

12.2. 二叉搜索树的ADT

BST的实现可使用二叉树的ADT。 这里讨论以下BST的ADT中的基本静态操作和动态操作。

一些操作函数:

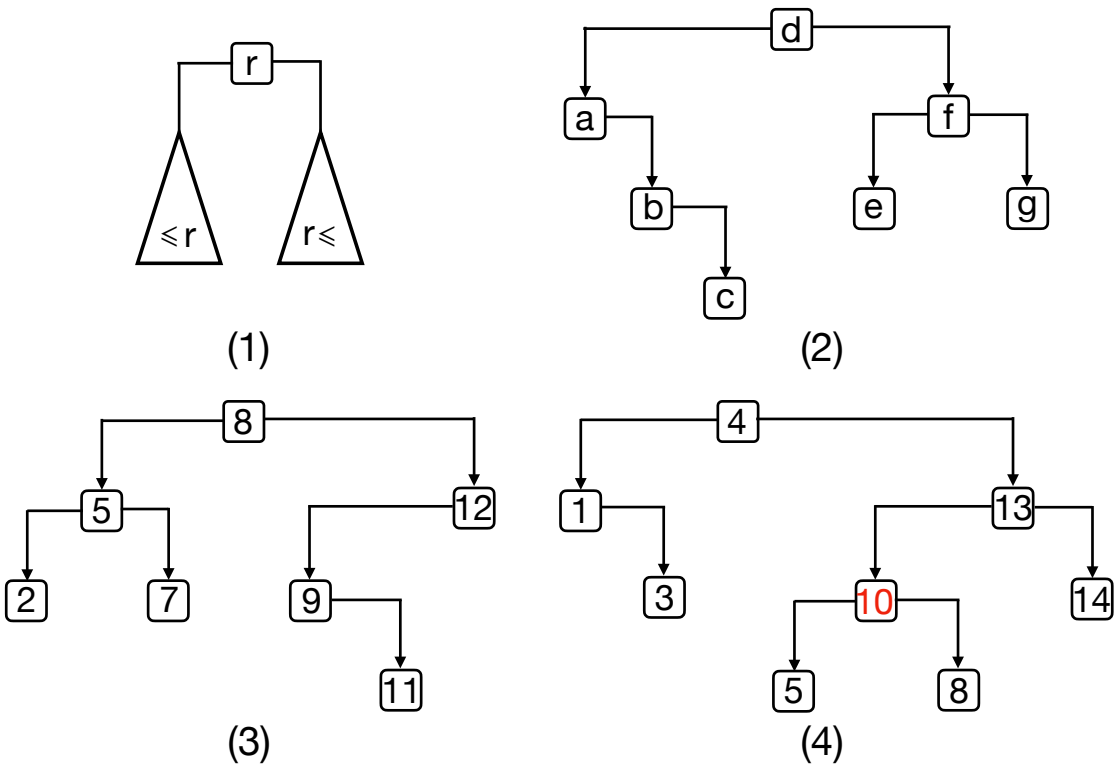


FIG. 12.1. 二叉搜索树中的有序性。 （1）为二叉搜索树中对每个节点、其左子树、其右子树中节点的要求。 （2）为包含了7个节点的二叉搜索树，其节点的关键码为$\{a, b, \cdots, f\}$。 （3）为包含了7个节点的二叉搜索树，其节点的关键码为$\{2, 5, 7, 8, 9, 11, 12\}$。 （4）不为二叉搜索树，因为关键码为10的节点不满足二叉搜索树的有序性。

■ 1，查找节点

二叉搜索树本身对应的一个有序的线性序列。 从树的根节点开始，通过比较节点和查找元素的大小，来确定此根节点是否为查找元素、查找元素处在根节点的左子树、查找元素处在根节点的右子树。这个过程持续下去，直到找到该元素、或者确定元素在树中不存在。这个过程非常类似与有序线形序列上的二分查找过程。其过程可见Figure 12.2。

在代码实现上，我们除了记录查找元素所在节点的指针之外，还可以记录其父节点的指针。

- 当树中存在查找元素时，返回此节点的指针、及其父节点指针。
- 当树中不存在节点时，返回空指针、及其父节点指针。此父节点指针表示的是查找元素的位置信息。在构造二叉搜索树的场景下，这个位置可以提供新节点在书中的位置信息。

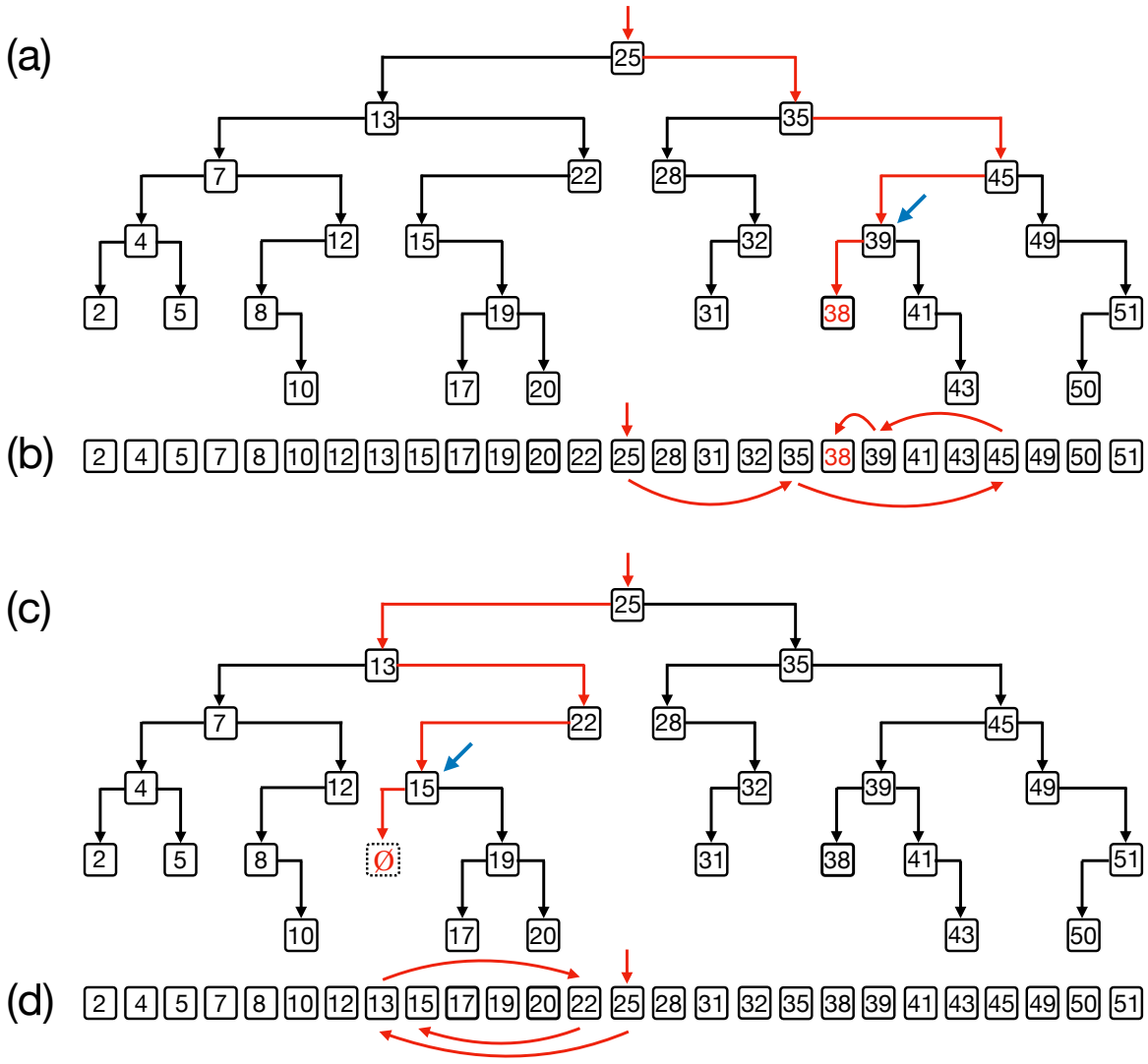


FIG. 12.2. 二叉搜索树中的查找过程。 （a）为含有26个节点的二叉搜索树。此时在树中查找元素38。查找过程使用红色线段和箭头来表示。 （b）为查找过程在二叉搜索树对应的中序遍历序列上的对应过程。 （c）为（a）中的二叉搜索树上查找元素14的过程。可知此元素在树中不存在，但对应为节点15的左子节点的位置。 （d）为查找过程在二叉搜索树对应的中序遍历序列上的对应过程。 图中蓝色箭头标记了查找元素对应节点的父节点。

■ 2，插入节点

先使用search(e)判断元素是否存在，并确定插入位置。

若关键码e不存在，则将新节点作为叶子节点插入，并以hot为父节点。

其操作过程如图Figure 12.3。

■ 3，删除节点

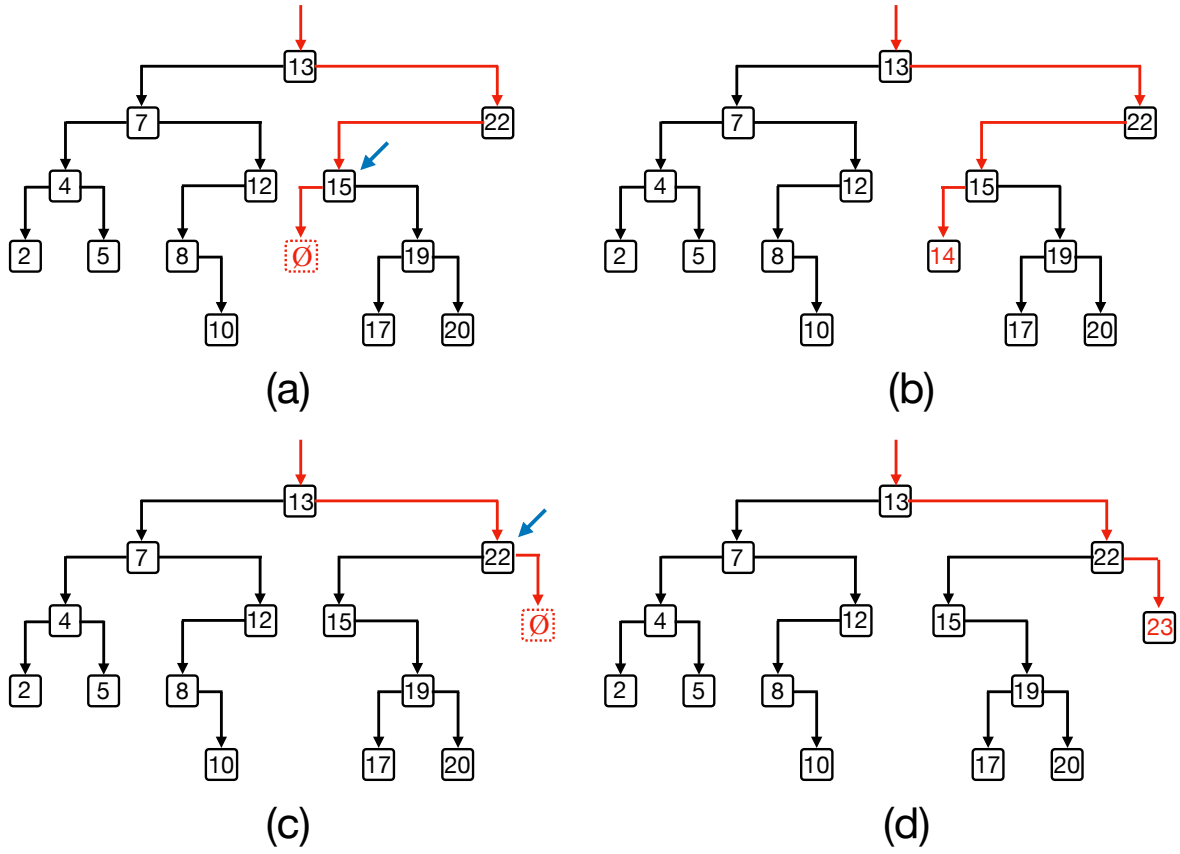


FIG. 12.3. 二叉搜索树中的插入节点的过程。 （a - b）为在给定二叉搜索树中查找、并插入元素14的操作过程。 （c - d）为在给定二叉搜索树中查找、并插入元素23的操作过程。 图中蓝色箭头标记了查找元素对应节点的父节点。

从二叉搜索树中删除节点时，可能会相应改变树中的顺序性，相应地需要对树做一些结构调整。 这里分别描述一下三种情况。

（3.1）被删除节点无子节点

此时可以直接删除相应节点，二叉搜索树的顺序性不受影响。

其操作过程如Figure 12.4所示。

（3.2）被删除节点仅有一个子节点（单侧情形）

如果被删除节点仅有左子节点，此时只需将其左子树连接到被删除节点的父节点、删除相应节点即可。 如果被删除节点仅有右子节点，此时也只需将其右子树连接到被删除节点的父节点、删除相应节点。

其操作过程如Figure 12.5所示。

（3.3）被删除节点有两个子节点（双侧情形）

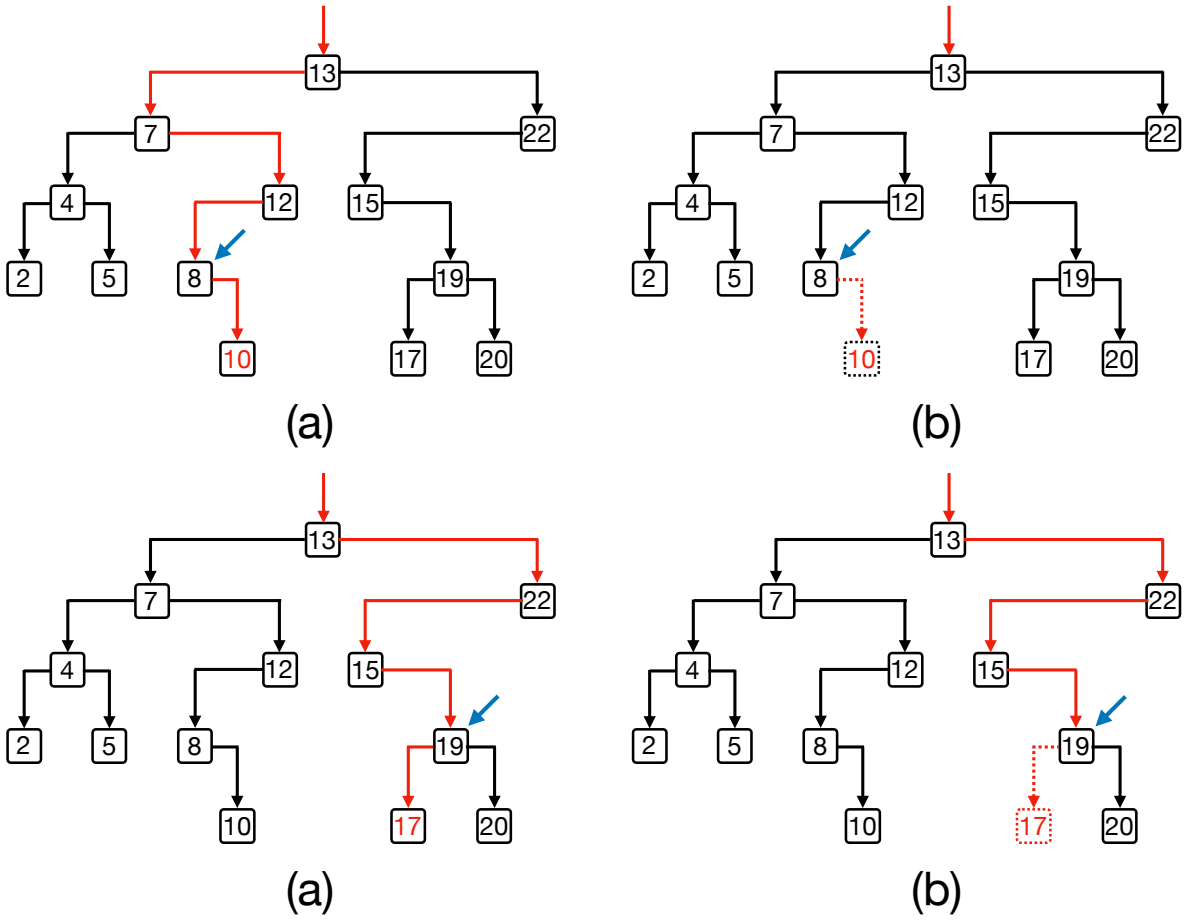


FIG. 12.4. 二叉搜索树中的删除节点的过程。此时被删除节点无子节点。 （a - b）为在给定二叉搜索树中查找、并删除元素10的操作过程。 （c - d）为在给定二叉搜索树中查找、饼删除元素17的操作过程。 蓝色箭头指向了查找元素对应节点的父节点。虚线表示被删除的连边和节点。

如果被删除节点同时具有左子节点和右子节点，删除操作对树的结构影响会大一些。此时我们首先寻找被删除节点在中序遍历意义下的后继节点，交换被删除节点和此后继节点，然后删除被删除节点。 容易得知，此后继节点是没有左子节点的。此时的删除过程可以利用上述仅有右子节点或者同时无左右子节点的情况来进行处理。

其操作过程如Figure 12.6所示。

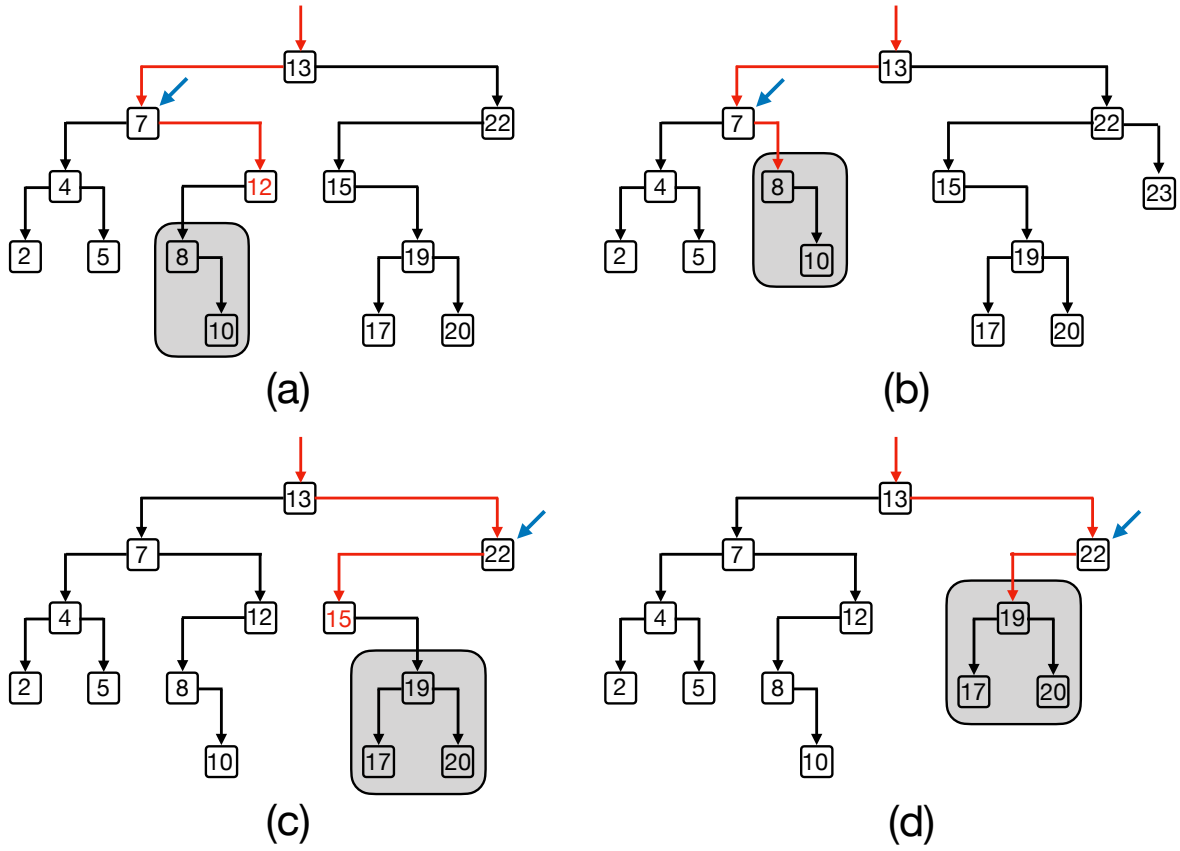


FIG. 12.5. 二叉搜索树中的删除节点的过程。此时被删除节点仅有一个子节点。 （a - b）为在给定二叉搜索树中查找、并删除元素12的操作过程。 元素12对应的节点在原二叉搜索树中只有左子节点。 （c - d）为在给定二叉搜索树中查找、并删除元素15的操作过程。 元素15对应的节点在原二叉搜索树中只有右子节点。 阴影部分覆盖的子树为被删除节点在原二叉搜索树中的唯一子树。蓝色箭头指向了查找元素对应节点的父节点。

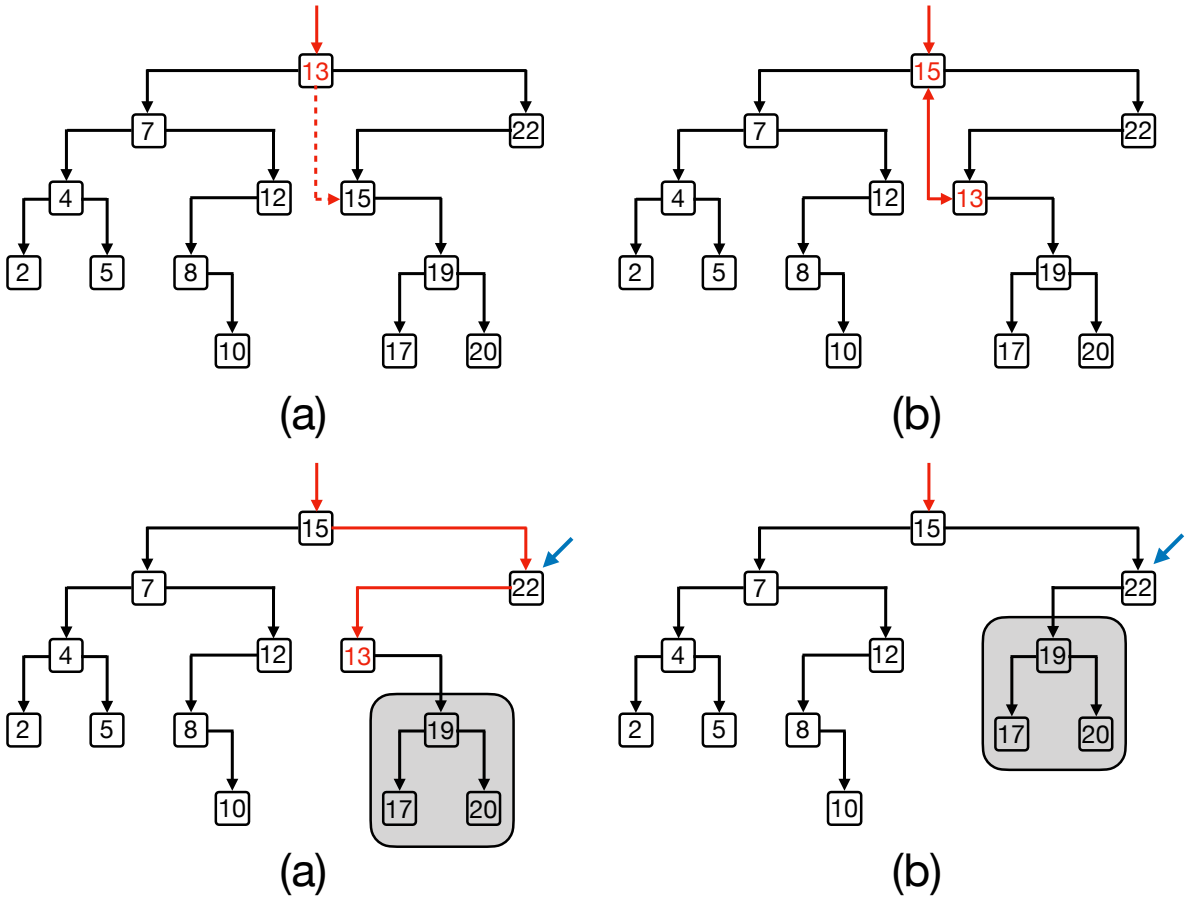


FIG. 12.6. 二叉搜索树中的删除节点的过程。此时被删除节点有两个子节点。 (a) 被删除节点为13。节点15为被删除节点13在中序遍历意义下的后继节点。 (b) 交换节点13和15。 (c) 确定节点13的位置。 (d) 把节点13从二叉搜索树中删除。 阴影部分覆盖的子树为被删除节点在原二叉搜索树中的右子树。

12.3. 渐近平衡与旋转操作

■ 树的高度、理想平衡、渐近平衡

容易得知，二叉搜索树中查找、插入、删除节点的操作依赖于节点在树中的深度。对整棵树来说，这些算法的计算时间依赖于树的高度，也就是说树的高度越低，平均意义上算法的时间越短；树的高度越高，平均意义上算法的时间越长。

对于一个高度$h(\geqslant 0)$来说，二叉树的节点数$N(h)$的范围为多少?

节点数最少的情况对应着整棵二叉树退化称为一个线形序列，此时有：

$$N(h) = h + 1. \tag{12.1}$$

节点数最多的情况对应着树为一棵满二叉树。此时节点数为：

$$N(h) = 2^0 + 2^1 + 2^2 + \cdots + 2^h = 2^{h+1} - 1. \tag{12.2}$$

• 理想平衡：为了构造高度尽可能低的二叉树，我们可以尽量从根节点开始填入节点、并尽量填满每一层节点。此时的树类似一个完全二叉树、或者为一棵满二叉树。树的高度$\sim \log_2 N$。

同时，我们应该注意到这种树的结构意味很强的结构约束。但我们对其进行动态操作，比如插入和删除节点时，可能需要对多个节点进行操作、以保证尽可能地填满树的每一层。 相应地，这种结构的维护成本很高。

• 渐近平衡：为了同时保证树的高度尽可能低、但又不至于需要很高的维护成本，我们可以选择树的高度为$\log_2 N$的量级，即$< \mathcal{O}(\log_2 N)$。此时这种树为平衡二叉搜索树（Balanced Binary Search Tree, BBST）。

■ 等价变换

对于二叉搜索树来说，其核心功能使用二叉树这种结构来存储有序序列。本质上，如果其存储的有序序列不变，其节点之间的连接细节并不重要。对于能够实现同一有序序列的二叉搜索树，我们认为它们本质上是等同的。那么如何改变二叉搜索树的结构、但又能实现同样的有序序列? 可以使用旋转操作。

以下介绍两种旋转操作：右旋操作（zig）和左旋操作（zag）。经过旋转操作，不同的BST之间可互相转换。

（1）右旋操作

如Figure 12.7展示了二叉搜索树中的一次右旋操作。可知旋转操作之后，节点u的深度增加1，节点v的深度减小1，子树T1中所有节点的深度减少了1，子树T2中所有节点的深

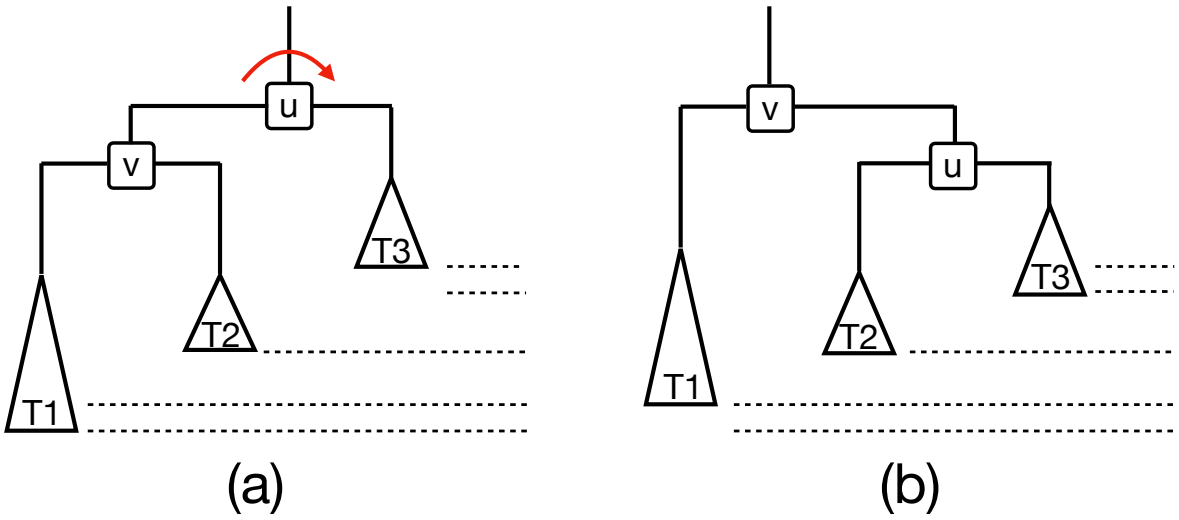


FIG. 12.7. 二叉搜索树中的右旋操作。 （a）此二叉搜索树上，将以节点u为轴点使用右旋操作。此时子树T1的高度高于子树T2。 （b）右旋操作后的结果。 图中序线表示旋转操作之前和之后子树中最深节点对应的深度。

度不变，子树T3中所有节点的深度增加1。相应地，子树T1和子树T2之间的高度差比旋转之前减少了1、子树T2和子树T3之间的高度差比之前减少了1。

（2）左旋操作

如Figure 12.8展示了二叉搜索树中的一次左旋操作。可知旋转操作之后，节点u的深度增加1，节点v的深度减小1，子树T1中所有节点的深度减少了1，子树T2中所有节点的深度不变，子树T3中所有节点的深度增加1。相应地，子树T1和子树T2之间的高度差比旋转之前减少了1、子树T2和子树T3之间的高度差比之前减少了1。

12.4. AVL树

■ 平衡因子和AVL树的结构要求

节点的平衡因子（balance factor）定义为其子左右子树的高度差，即BaFac(v) = height(v→lc) - height(v→rc)。

对于空节点，我们可以设置它们的高度为−1。相应的所有叶子节点的高度均为0，其平衡因子为0。

AVL树作为一种特殊的二叉搜索树，对于树中的任意节点v，满足BaFac(v) $\in \{-1, 0, 1\}$。判断一棵二叉搜索树是否为AVL树时，可以先逐个标记所有节点的平衡

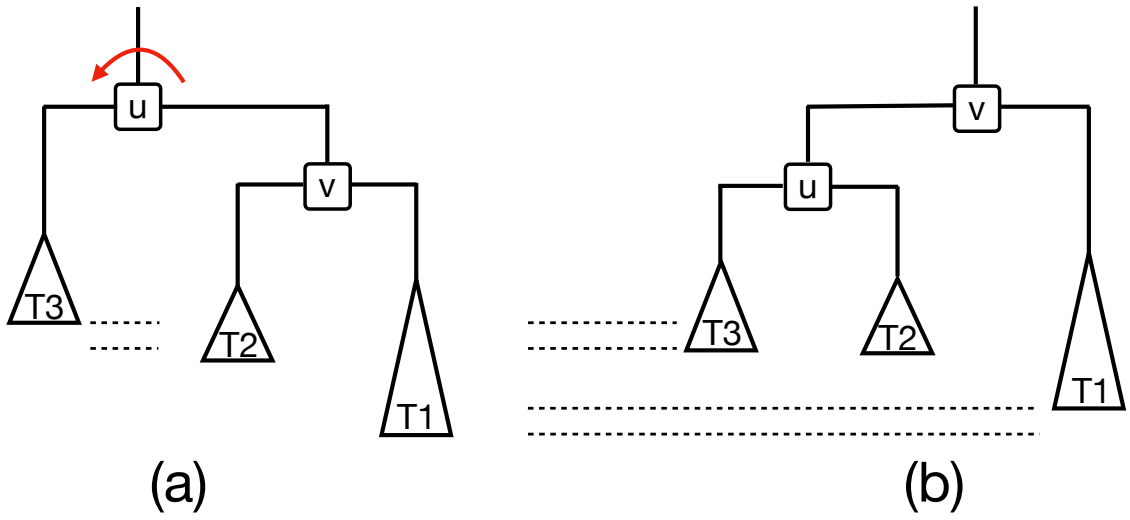


FIG. 12.8. 二叉搜索树中的左旋操作。 （a）此二叉搜索树上，将以节点u为轴点使用左旋操作。此时子树T1的高度高于子树T2。 （b）左旋操作后的结果。 图中序线表示旋转操作之前和之后子树中最深节点对应的深度。

因子。如果所有的平衡因子都满足上述要求，则此二叉搜索树就为AVL树。

Figure 12.9给出了一个AVL树的例子。容易得知子图(a)中的二叉搜索树不为AVL树，因为节点12、22的平衡因子违反了要求。而子图(b)中的二叉搜索树为一棵AVL树。

■ 高度为h的AVL树的节点数$S(h)$的范围

之前我们计算过高度为h的二叉树的节点数范围为$[N_{\min}(h), N_{\max}(h)]$，其中最小值$N_{\min}(h) = h + 1$、最大值$N_{\max} = 2^{h+1} - 1$。分别对应着线性二叉树和满二叉树。

容易得知，线性的二叉树显然不是AVL树，但满二叉树为一棵AVL树。所有本质上AVL树有个最小的节点数。我们可以简略地称这类AVL树为“最瘦AVL树”。对于最瘦AVL树，我们可以保证其内部节点的平衡因子刚好在临界值，也就是-1或1。为简单起见，我们假设其内部节点的平衡因子都为1。这类AVL树的一些例子如Figure 12.10所示。

对于图中高度为$\{1, 2, 3, 4, 5\}$的AVL树来说，节点数分别为$\{2, 4, 7, 12, 20\}$。这里我们可以直接构造出来给定高度h后的最瘦AVL的节点数$S_{\min}(h)$。从Figure 12.10(b-e)可知，构造高度为h的最瘦AVL树时，可以直接使用高度为$h - 1$的最瘦AVL树作为其左子树、高度为$h - 2$的最瘦AVL树作为其左子树。此时可以得到三者节点数之间的关系：

$$S_{\min}(h) = S_{\min}(h - 1) + S_{\min}(h - 2) + 1. \tag{12.3}$$

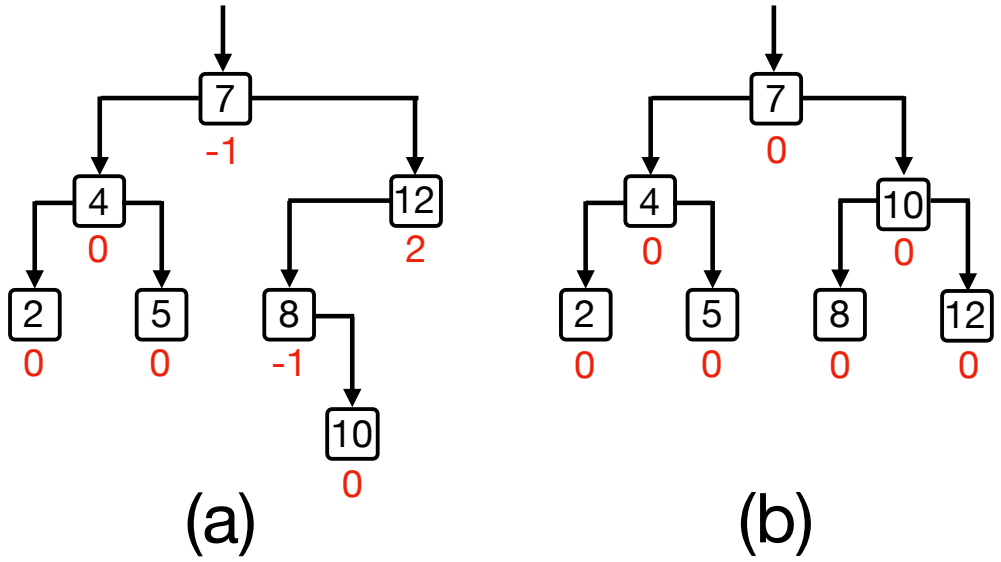


FIG. 12.9. 二叉搜索树与AVL树。 （a）为一棵二叉搜索树。 （b）为将（a）中的树进行一些结构变换，得到的新的二叉搜索树。 注意到两棵二叉搜索树的中序遍历序列相同。 每个节点下面的红色数字为节点的平衡因子。

上式右侧最后的一项+1来源于根节点。 上式可以写为：

$$S_{\min}(h)+1=[S_{\min}(h-1)+1]+[S_{\min}(h-2)+1]. \tag{12.4}$$

可以等效写为：

$$T_{\min}(h)\equiv S_{\min}(h)+1, \tag{12.5}$$

$$T_{\min}(h)=T_{\min}(h-1)+T_{\min}(h-2). \tag{12.6}$$

$T_{\min}(h)$的迭代关系其实对应着一个Fibonacci序列$fib(s)$，$s=1,2,\cdots$。 这里我们假定

$$fib(s)=\{1,1,2,3,5,8,13,21,\cdots\}. \tag{12.7}$$

但我们还需要确定两者之间系数h和s之间的关系。 由Figure 12.10，可知

$$T_{\min}(1)=S_{\min}(1)+1=3, \tag{12.8}$$

$$T_{\min}(2)=S_{\min}(2)+1=5, \tag{12.9}$$

$$T_{\min}(3)=S_{\min}(3)+1=8, \tag{12.10}$$

$$T_{\min}(4)=S_{\min}(4)+1=13, \tag{12.11}$$

$$T_{\min}(5)=S_{\min}(5)+1=21. \tag{12.12}$$

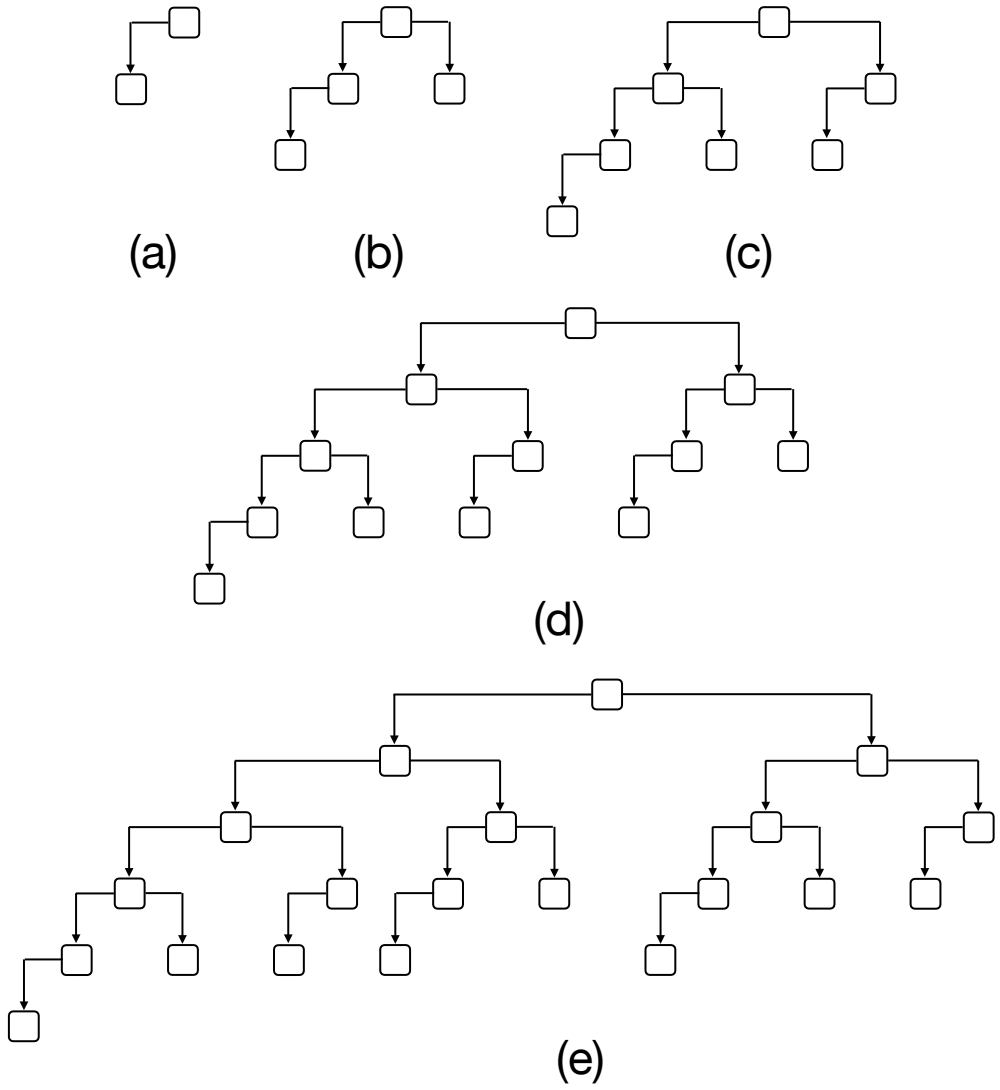


FIG. 12.10. AVL树的一些例子。 (a-e)分别展示了高度为1到5的节点数最少的AVL树的构型。对于任意内部节点v，$\mathrm{BF}(v) = 1$。 容易得知，这些AVL树的节点数分别为$\{2, 4, 7, 12, 20\}$。

容易得知：

$$T_{\min}(h) = fib(h + 3). \tag{12.13}$$

对应于：

$$S_{\min}(h) \equiv T_{\min}(h) - 1 = fib(h + 3) - 1. \tag{12.14}$$

综合以上结果可知，AVL树的节点数的取值范围为$[S_{\min}(h), S_{\max}(h)]$，其中最小值$S_{\min}(h) = fib(h + 3) - 1$、最大值$S_{\max}(h) = 2^{h+1} - 1$。

以下我们可以继续得到，当h很大时，AVL树的节点数$S(h)$和h的标度关系。 由第4章的例子5可知，当h很大时，$fib(h) = \Theta(2^{ch})$，且$c = \log_2 \frac{1+\sqrt{5}}{2} \approx 0.694$。容易得知：

$$S_{\min}(h) = fib(h+3) - 1 = \Theta(2^{c(h+3)}) = \Theta(2^{ch}). \tag{12.15}$$

而对于AVL树的节点数的最大值，可知：

$$S_{\max}(h) = 2^{h+1} - 1 = \Theta(2^{ch}). \tag{12.16}$$

对于AVL树的任意节点数$n \in [S_{\min}(h), S_{\max}(h)]$，由Eqs.(12.15)和(12.16)，可知AVL树的节点数和高度h满足，$n = \Theta(2^{ch})$，等价于$h = \Theta(\log_2 n)$。即这里再次证明了AVL树的结构满足渐近平衡的要求。

12.5. 构造AVL树的重平衡策略

考虑以下两个问题： （1）对于任意一棵二叉搜索树，可否通过等价操作来把它变成一棵AVL树？ （2）对于一棵AVL树，在对它的节点进行动态操作时，可能会有节点出现失衡情况，那么如何通过局域操作把它重新变成满足AVL树的结构要求？ 这两个问题可以统一使用重平衡策略来处理，也就是对于任意一棵二叉搜索树，通过局域操作，使得所有节点满足AVL树的结构约束。

这里我们讨论一个具体的场景，也就是在AVL树里加入或删除节点时导致的节点失衡的情况。 加入或删除节点后，能够导致的原有节点的高度变化为1。 容易得知如果出现失衡的节点，那么其平衡因子为2或者-2。 从删除或加入节点的位置开始沿着其父节点往上，第一个出现失衡情形的节点，我们标记为g。 g的两棵子树中，高度更高的那个节点标记为p。p的两棵子树中，高度更高的那个节点标记为v。 节点g、p、v可以作为子树状态的标记节点。 容易得知，给定g后，g和p、v的相对位置情形有4种。这四种情况有着相应的旋转操作。

（1）p为g的左子节点、v为p的左子节点

如Figure 12.11所示。 对应操作为单旋操作：右旋zig()。 此时g的平衡因子为2、p的平衡因子为1、v的平衡因子为1或者-1。 通过旋转操作后，g的平衡因子为0、p的平衡因子为0、v的平衡因子不变。

（2）p为g的左子节点、v为p的右子节点

如Figure 12.12所示。 对应操作为双旋操作：左旋zag()+右旋zig()。 此时g的平衡因子为2、p的平衡因子为-1、v的平衡因子为1或者-1。 通过旋转操作后，g的平衡因子为-1或者0、p的平衡因子为0或者1、v的平衡因子为0。

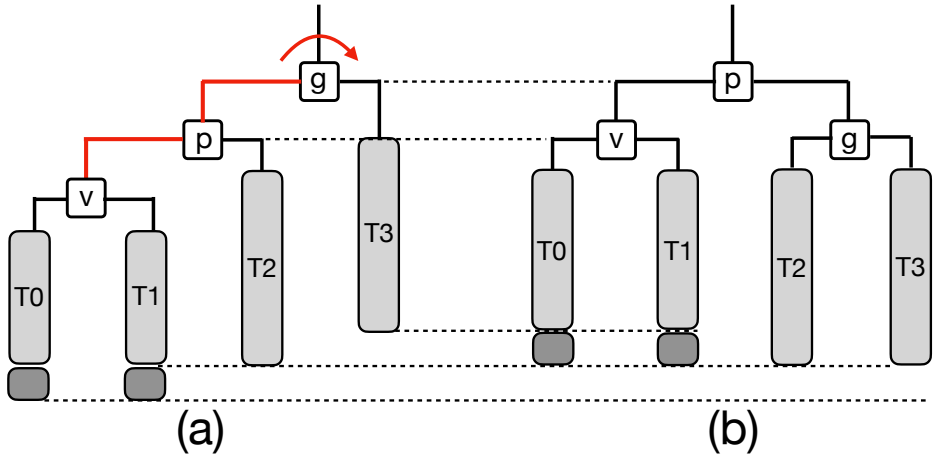


FIG. 12.11. AVL树里的右旋操作。

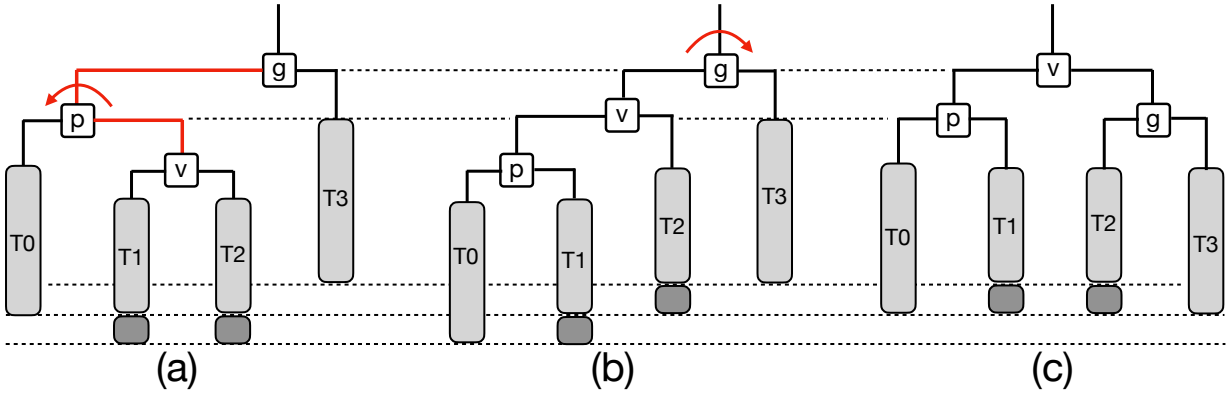


FIG. 12.12. AVL树里的左旋右旋操作。

（3）p为g的右子节点、v为p的右子节点

如Figure 12.13所示。 对应操作为单旋操作：左旋zag()。 此时g的平衡因子为−2、p的平衡因子为−1、v的平衡因子为1或者−1。 通过旋转操作后，g的平衡因子为0、p的平衡因子为0、v的平衡因子不变。

（4）p为g的右子节点、v为p的左子节点

如Figure 12.14所示。 对应操作为双旋操作：右旋zig()+左旋zag()。 此时g的平衡因子为−2、p的平衡因子为1、v的平衡因子为1或者−1。 通过旋转操作后，g的平衡因子为0或者1、p的平衡因子为−1或者0、v的平衡因子为0。

上述旋转操作的的核心思想，简单来说：当一个节点的平衡因子超过1时，可以把左子树里的节点适当移动到右子树，这对应着右旋操作（zig）；当一个节点的平衡因子小

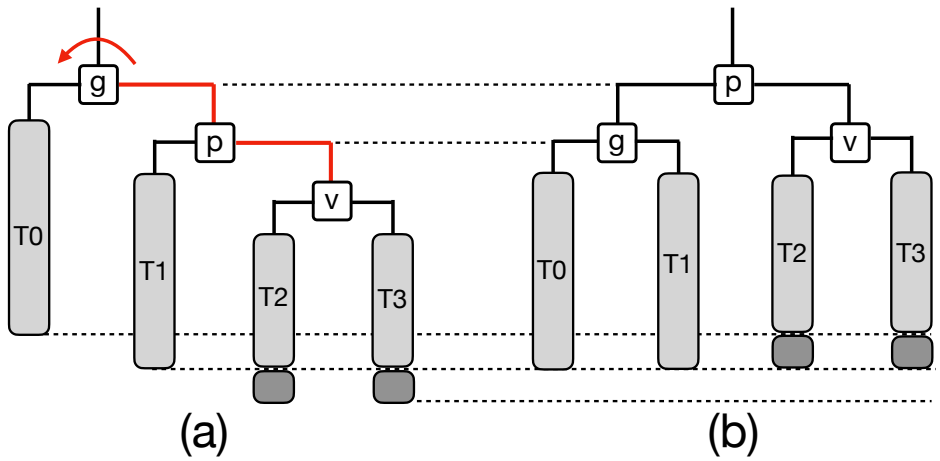


FIG. 12.13. AVL树里的左旋操作。

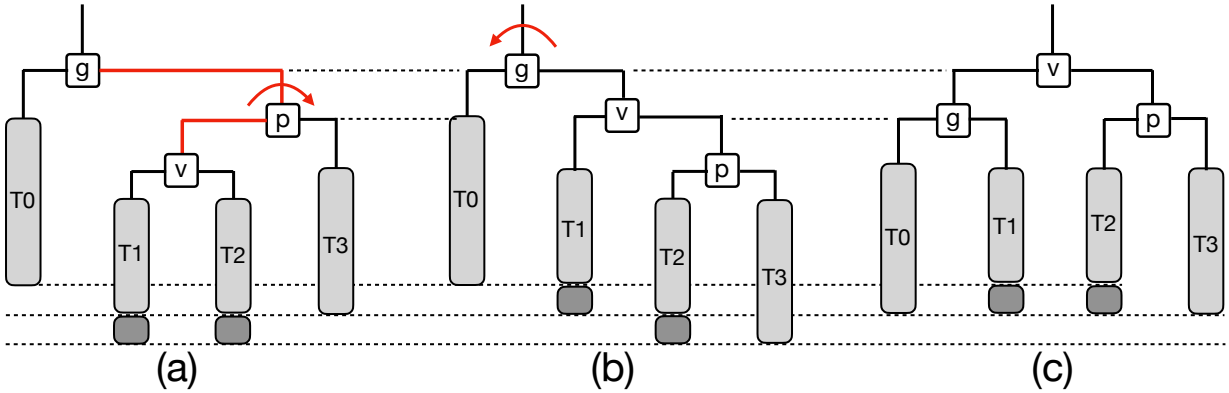


FIG. 12.14. AVL树里的右旋左旋操作。

于−1时，可以把右子树里的节点适当移动到左子树，这对应着左旋操作（zag）。两种操作都能降低左右子树的高度差，直到最终所有节点都能满足AVL树的结构要求。

容易得知，上述四个步骤都能重新将失衡的节点变得重新满足AVL树的要求。上述四个步骤针对的是平衡因子为2或者−2的情形。对于其它情形，此操作能够降低子树之间的高度差。如果多次使用这个操作，最终使得所有节点满足AVL树的结构要求。

Figure 12.15给出了一个使用旋转操作来实现AVL树的例子。图中的二叉搜索树的初始构型为Figure 12.2为节点数为13的二叉搜索树中根节点的左子树。

■ 统一的算法流程：(3+4)重构

上述四种旋转操作本质上可以视为同一个过程的四种具体情形。其过程如Figure 12.16所示。Figure 12.16(a-d)的四种情形里，进行重构操作之后，四棵子树的相对位

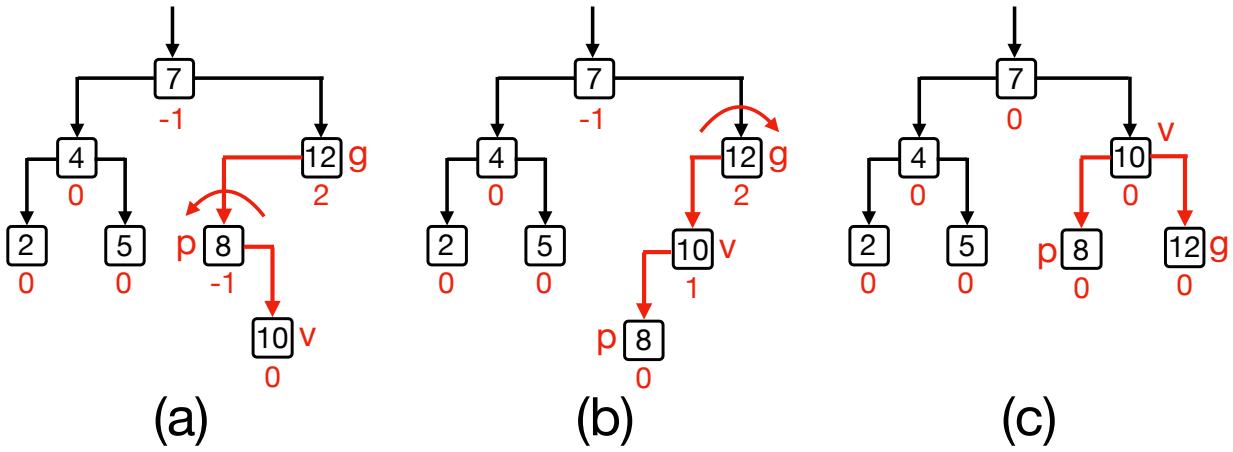


FIG. 12.15. AVL树里的旋转操作的例子。 （a）给出了一棵二叉搜索树。节点标记了平衡因子以及相应的g、p、v节点。 由g、p、v节点的位置关系可知，此时可以使用左旋右旋操作来构造AVL树。 （b）为围绕节点p进行左旋操作后的结果。 （c）为围绕节点g进行右旋操作后的结果。

置一直没变，改变的是g、p、v三个节点的相对位置。 Figure 12.16(e-f)中，我们用节点a、b、c统一标识g、p、v。重构过程中，我们把g、p、v按具体情形赋值给a、b、c节点即可。 这个过程就是AVL树的重平衡策略：(3+4)重构。

例子1: AVL树的构造

对于上题中的二叉搜索树：

（1）证明它不是一棵AVL树。

（2）对它使用重平衡策略，直到它变成一棵AVL树。

参考代码：`ch11-bintree.cc`中的函数BST2AVL()。

使用重平衡策略之后，得到的BST继续使用绘图函数，得到结果如Figure 12.17所示。

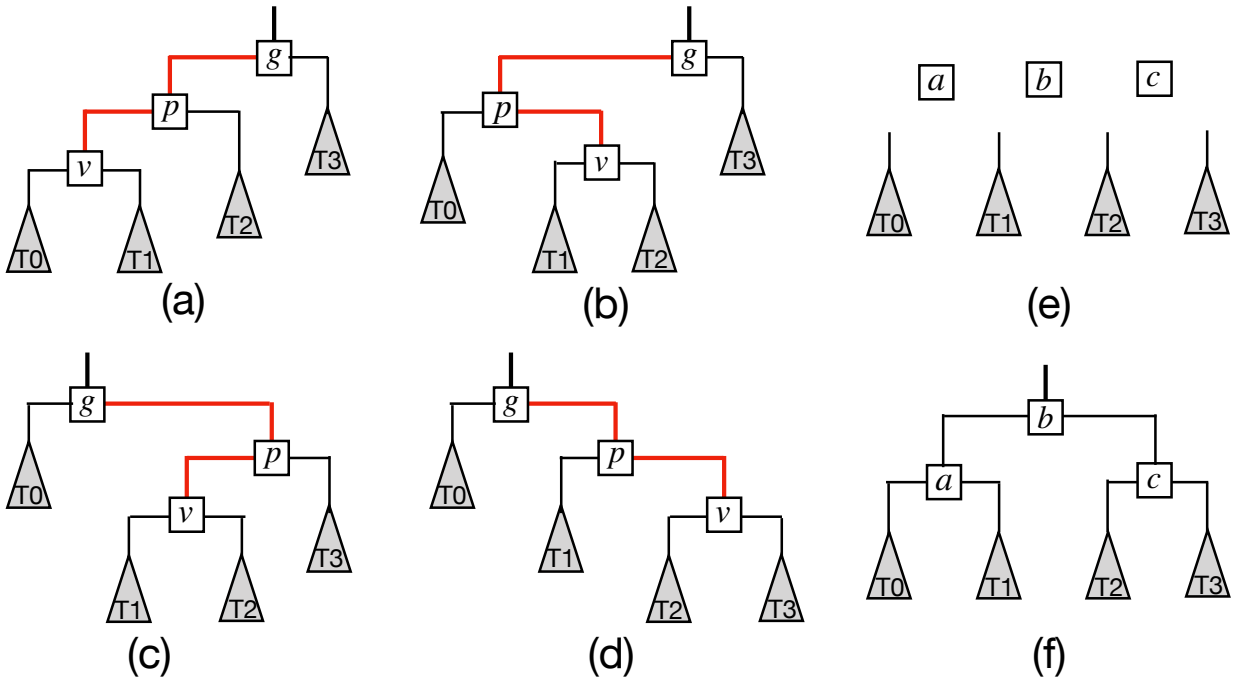


FIG. 12.16. (3+4)重构操作过程。 （a-d）对应着节点g、p、v相对位置的四种可能性。 （e-f）上述四种情形对应着把3个节点、4棵子树按照相应的方式连接起来。 此时按照（a-d）中g、p、v三节点的相对位置，对a、b、c进行赋值即可。

./figures-ch11/ch11-20-bintree-n26-plot.png

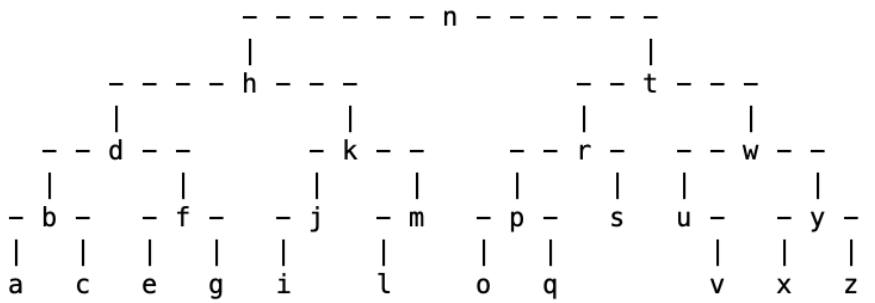


FIG. 12.17. 二叉搜索树到AVL树的结构变化。（上图）为Figure 11.8中的二叉树，包括26个节点。（下图）对其进行结构变化后得到的AVL树。其中二维作图模块单元的大小为cols_per_elem = 1, rows_per_elem = 2。

习题

12.1 AVL树的手动构造

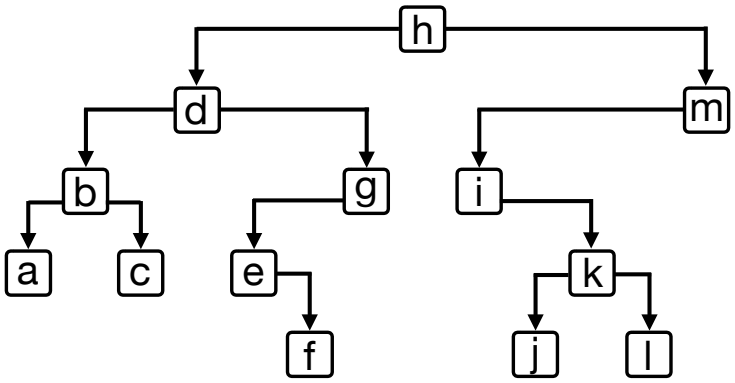


FIG. 12.18. 一棵二叉搜索树，其节点数为13。此树对应于Figure 12.2中根节点的左子树。

课堂里我们使用旋转操作这种标准算法流程来把任意一棵二叉搜索树调整成为一棵AVL树。 Figure 12.18为节点数为13的二叉搜索树。其根节点的左子树变为AVL树的过程，在Figure 12.15里已经完成。 对于**其右子树**，请完成以下问题：

（1）请在每个节点边写出其平衡因子，以证明此二叉搜索树不是AVL树。

（2）请使用课程里提到的标准的左旋和右旋搭配的操作，手动做图把它调整成一棵AVL树。请画出旋转操作中每一步的树结构、以及每个节点的平衡因子。

注：此题无需写代码，手动画出必要的示意图即可。

12.2 AVL树的代码构造

对于上题Figure 12.18中的二叉搜索树，请完成代码来回答以下问题：

（1）证明它不是一棵AVL树。

（2）使用“（3+4）重构策略”来把它变成一棵AVL树。完成重构操作之后，你可以输出每个节点的平衡因子，以证明重构后的二叉搜索树的确为一棵AVL树。

13. (GRAPH)

参考内容：邓俊辉教材第6.1–12节。李春葆教材第8.1–5节。

本章介绍了图的基本概念、ADT、及算法。 在图的算法层面，我们介绍了其上的遍历算法、顶点间最短路径问题及算法、最小生成树问题及算法。

13.1. 图的基本性质

图(graphs)和网络(networks)都使用了“顶点”和“边”来描述其结构。可以认为两者是同一个概念。 两者的差别主要在于学科和研究重点的差别：前者来自图论(graph theory)[25]，侧重解析和理论结果； 后者来自网络科学(network science)[26, 27]，侧重真实世界网络启发的问题。

图和网络是复杂系统(complex systems)中相互作用的一种抽象描述。复杂系统指的是具有多种类构成单元、多种类相互关系的系统。 在拓扑结构上，复杂系统可以使用顶点(nodes/vertices)和边(links/edges)构成，其中顶点来描述其组成单元、用边表示表示单元之间相互作用的结构。我们可以按照具体情况来定义顶点和边。网络结构既可以描述真实的物理系统，也可以用来描述抽象概念之间的关系。这里有一个约定俗成的名词搭配：(vertices, edges) = (nodes, links)。

• 图论的起源：图论起源于哥尼斯堡七桥问题(The Bridges of Königsberg）。【注：哥尼斯堡意为国王（koenig）之山（berg），曾为普鲁士和德国的领土。二战后陆续属于苏联和俄罗斯，并改名为加里宁格勒市（Kaliningrad）。】此问题描述为可否走过河上的7座桥、而不用重复走过某座桥。欧拉(Euler)在1735年证明了此问题是不可能的。他的证明过程需要把桥所连接的岛和桥本身抽象成通过顶点和边。此问题经过抽象，可以描述为在给定图上，能否一次性走过全部7条边、而不用重复通过某条边。如Figure 13.1所示。此证明过程依赖于图中顶点度的奇偶性。欧拉对七桥问题的研究可以视为图论的开端。这里已经有了从具体问题里抽象出来顶点和边的概念，并讨论其拓扑结构本身的问题。

• 网络科学：网络科学本质上来源于图论，开端于1998年的小世界网络[36]和1999年的无标度网络[37]的研究。 图论和网络科学虽然用了一套语言，但关心的问题有一些差别。前者关系是一些特定图下的解析或者近似结果，后者更加关心真实世界网络(real-world networks)的结构、动力学、功能、演化等等主题，涉及到解析理论、数值求解、模拟过程、数据分析等多种处理方法。

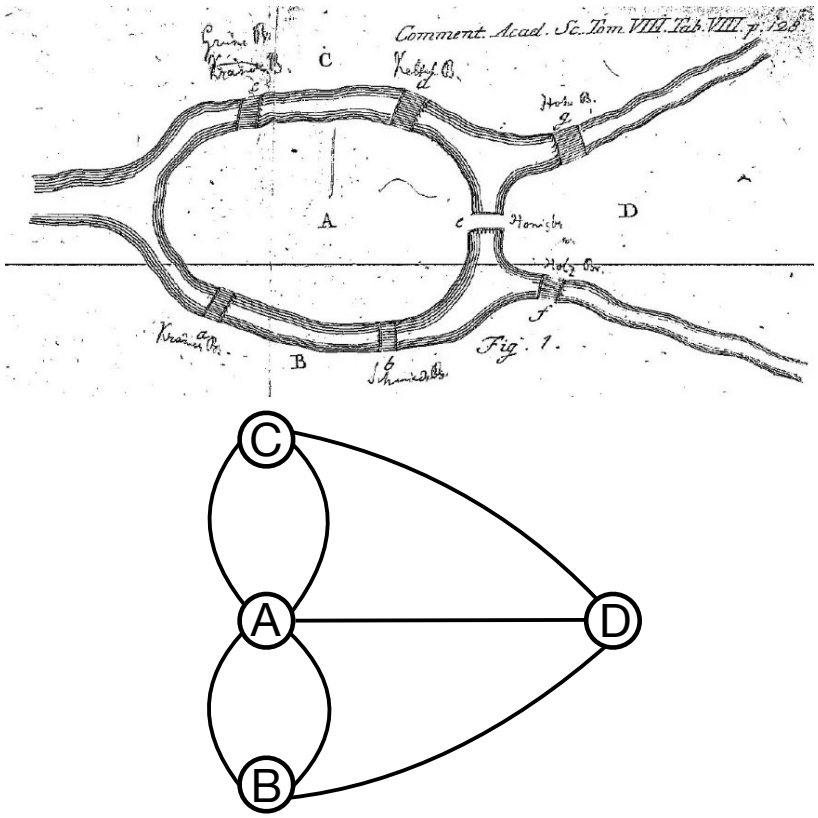


FIG. 13.1. 哥尼斯堡七桥问题。（上图）为当时欧拉有关七桥问题的手稿。（下图）为七桥问题抽象出的图结构。顶点表示连通的陆地，边表示连接两块陆地的桥。图中共4个顶点、7条边。

以下是一些真实世界网络的例子：

◦ 细胞网络(celluar networks)：由基因、基因表达的蛋白、蛋白执行具体生物功能所产生代谢产物等等构成的网络。这个网络提供了生物活动的生化基础。

◦ 神经网络(neuronal networks)：由10^{10}量级的神经元所构成的神经网络。它执行了计算、记忆、认知等等功能。

◦ 社交网络(social networks)：人通过职业、友谊、家庭等关系联系在一起。

◦ 通信网络(communication networks)：基站、终端设备通过通信基础设施连接起来。

◦ 电网(power grids)：电源从发电厂、高压电线、变电器一直输送到家庭用户。电网是现代社会基础设施里的重要一环。

◦ 交易网络(transaction networks)：包括传统银行服务、现代的在线支付等等，把处在金融服务中的人、机构等联系在一起。

网络科学诞生于1998年。催生网络科学最直接的原因在于上世纪90年代末，互联网和生物科技等等先进技术积累了大量大规模网络系统的经验数据。客观上，从各学科得到的

网络结构需要有一个统一的、普适的定量理解和数学描述的框架。这是一个典型的由技术和数据积累所激发出来的新研究方式或学科的例子。

总体上，网络科学体现了典型跨学科性。其研究群体覆盖了多专业背景，比如计算机科学、自动化、物理学、生物学、脑科学等等。同时在研究方法上，覆盖了以下各类方法:

◦ 定量与数学方法：图论、随机性、普适性等数学和物理的概念、控制论、信息论等学科;

◦ 经验数据驱动方法：规模更大的真实世界数据集往往能催生网络科学的新结果和新理解;

◦ 数据计算方法：大规模网络数据集下的数值计算方法、数据库、机器学习等研究方法。

■ 网络的可视化和分析工具

Gephi → `https://gephi.org`

Cytoscape → `https://cytoscape.org`

igraph → `https://igraph.org`

NetworkX → `https://networkx.org/en/`

■ 一些常见的真实世界网络数据集:

`https://snap.stanford.edu`

`https://networks.skewed.de`

■ 网络的一些类型

• 无向边(undirected edges)和有向边(directed edges)：边表示顶点之间的关系或者相互作用。边上可以定义方向：如果顶点之间的方向为双向的或者无所谓方向，则边为无向边；如果顶点之间的关系定义上了单向的方向，则边为有向边。当图的边是有向时，图为有向图(directed graphs)。当图上的边没有方向时，图为无向图(undirected graphs)。如Figure 13.2(a)和(b)分别所示。

• 带权网络(weighted networks)：网络的顶点或边有大小、重要性等等的差别。这些信息是无法通过相同的顶点或者边来表示的。比如：用图描述城市之间的交通网络时，城市的面积和人口时有很大差别的：用图来描述社交关系时，注意到社交关系里亲人关系、朋友关系、熟人关系之间的关系强度一般是递减的；用图来描述过国家或地区之间贸易额时，双边的贸易额可能有很大差别的。此时可以通过顶点的大小、边的粗细来表示它们之

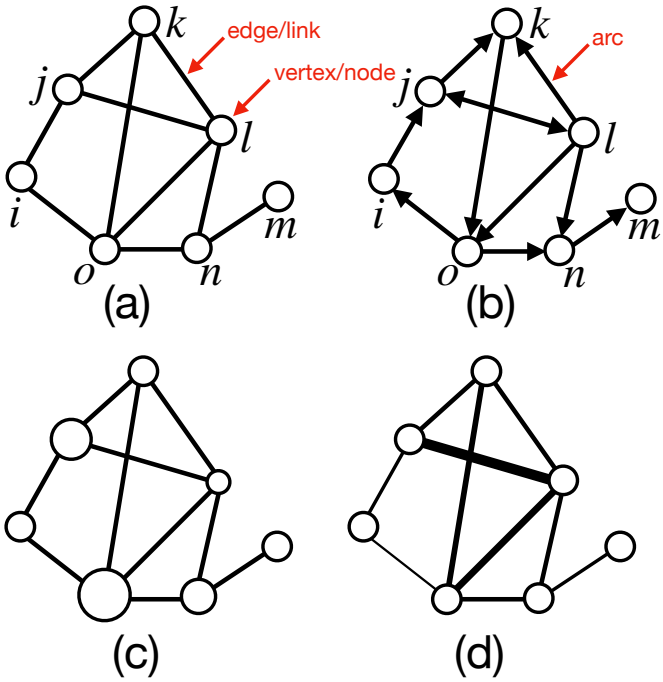


FIG. 13.2. 图的一些常见类型。 (a)为一个无向图，包含了7个顶点、10条边。 (b)为一个有向图。(c)为顶点带权重的无向图。顶点的大小大致表示定义在顶点上的权重。 (d)为边带权重的无向图。顶点的跨度大致表示定义在顶点上的权重。

间的相对权重。如Figure 13.2(c)和(d)分别所示。

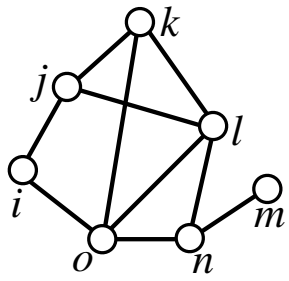
• 空间网络(spatial networks)：随机图本身没有空间维度的概念，顶点之间唯一重要的信息是它们是否相连、以及如何相连。但在一些真实世界场景里，顶点之间可以存在距离的概念。比如城市中之间的交通网络，城市之间是有距离的，而且一般来说距离越近、交通量会越大。一般来说，当网络定义在某个有限维的空间里，都可以视为空间网络。

• 含时网络(temporal networks)：图对应的复杂系统一般不是一直处在静态的，其在演化时，其顶点和边也会出现变化，比如顶点的加入和退出、边的增加或者删除等等。此时可以用含时网络来表示：此时我们使用多个时间戳来表示网络，其中每个时间戳对应着一个网络结构。

■ 图的邻接矩阵

图的最核心的结构性质就是顶点之间的相连关系。此时可以使用矩阵来表示。相应矩阵的性质，比如本征值的大小和分布，和网络结构的宏观性质直接相关。 以下为常考虑的两个矩阵：

• 邻接矩阵（adjacency matrix）A



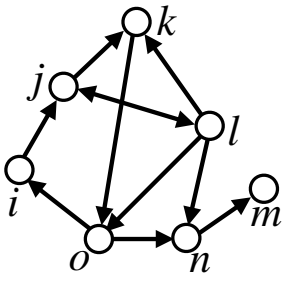
(a)

$A_{(i,j)}$	i	j	k	l	m	n	o
i	0	1	0	0	0	0	1
j	1	0	1	1	0	0	0
k	1	0	0	1	0	0	1
l	0	1	1	0	0	1	1
m	0	0	0	0	0	1	0
n	0	0	0	1	1	0	1
o	1	0	1	1	0	1	0

(b)

$A_{(i,j)}$	i	j	k	l	m	n	o
i	2	-1	0	0	0	0	-1
j	-1	3	-1	-1	0	0	0
k	-1	0	3	-1	0	0	1
l	0	-1	-1	4	0	-1	-1
m	0	0	0	0	1	-1	0
n	0	0	0	-1	-1	4	-1
o	-1	0	-1	-1	0	-1	4

(c)



(d)

$A_{\langle i,j \rangle}$	i	j	k	l	m	n	o
i	0	1	0	0	0	0	0
j	0	0	1	1	0	0	0
k	0	0	0	0	0	0	1
l	0	1	1	0	0	1	1
m	0	0	0	0	0	0	0
n	0	0	0	0	1	0	0
o	1	0	0	0	0	1	0

(e)

FIG. 13.3. 图的矩阵表示。 (a)为含有顶点数$N = 7$、边数$M = 10$的无向图。 (b)为(a)对应的邻接矩阵A。(c)为(a)对应的拉普拉斯矩阵D。 (d)为含有顶点数$N = 7$、有向边数$M = 10$的有向图。 (e)为(d)对应的邻接矩阵。

邻接矩阵用来表示图的连接信息，包括是否相连，以及边的权重。 对于顶点数为N的网络，其邻接矩阵为$A_{N\times N}$。 在无向图中，如果顶点i和j相连，我们用(i, j)表示这条边，相应地$A_{ij} = A_{ji} = 1$；如果顶点i和j相连，则$A_{ij} = A_{ji} = 0$。 在有向图中，如果从顶点i到j有一条有向边，我们用$i, j\rangle$表示这条边，相应地$A_{ij} = 1$；如果顶点i和j没有有向边，则$A_{ij} = 0$。一般来说$A_{ij} \neq A_{ji}$。 Figure 13.3展示了两个邻接矩阵的例子。

• 拉普拉斯矩阵（Laplacian matrix）L

矩阵定义为$L \equiv D - A$，其中D为度矩阵，即$D_{ij} = k_i\delta_{ij}$。

■ 基于度的一些结构性质

• 顶点的度(degree)：表示和顶点相连的边的数目。在无向图中，顶点的度统计了所有和它相连的无向边。对于顶点i，度表示为k_i。 可知Figure 13.2(a)无向图里顶点i和j的度分别为2和1。 对于无向图$G = \{V, E\}$（其中$|V| = N$, $|E| = M$），容易得

知，$\sum_{i\in V} k_i = 2M$。原因在于，在统计全部顶点的度时，每条边都被统计了两次。

对于有向图来说，需要对边的方向做区分。边$\langle i,j\rangle$为一个有顺序的格点对，表示i是j的入邻居(in-neighbors)，j是i的出邻居(out-neighbors)。对于顶点i，其入邻居的大小为入度(in-degree)k_i^{in}、其出邻居的大小为出度(out-degree)k_i^{ou}。可知Figure 13.2(b)中顶点i的入度和出度分别为1，顶点m的入度为1、出度为0。对于有向图$D = \{V, E\}$（其中$|V| = N$, $|E| = M$），容易得知，$\sum_{i\in V} k_i^{\text{in}} = \sum_{i\in V} k_i^{\text{ou}} = M$。原因在于，在统计全部顶点的出度和入度时，每条边只被统计了一次。

• 图的度分布(degree distribution)：度分布$P(k)(k \geqslant 0)$为图中顶点的度的一个概率分布，即随机取一个点、其度为k的概率。对于一个真实的图来说，可以计算其经验度分布(empirical degree distribution)$P_{\text{em}}(k), k \geqslant 0$：我们可以直接数出度为$k$的顶点数$N(k)$，则得到$P_{\text{em}}(k) \equiv N(k)/N$。一幅图的经验度分布的计算可见Figure 13.4。

• 图的余度分布(excess degree distribution)：在随机图有关的理论框架中，比如在第15章的图渗流模型，我们会利用迭代概率框架来计算一些子图的性质。余度分布$Q(k)$的定义和度分布类似，但指的是随机选择一条边、并从一个端点到另外一个端点，此端点的度为k的概率。$Q(k)$和$P(k)$直接相关，容易得到

$$Q(k) = \frac{kP(k)}{\sum_{k=0}^{+\infty} kP(k)} = \frac{kP(k)}{c}, k \geqslant 1. \tag{13.1}$$

容易可知：$Q(k)$的定义里$k \geqslant 1$，而$P(k)$的定义里$k \geqslant 0$。

• 图的平均度(mean degree)：考虑一个无向图$G = \{V, E\}$，其中V为图的顶点集($|V| = N$)、E为图的边集($|E| = M$)。对于一个顶点i，其连接的边数目为它的度k_i。对于一个图来说，平均意义上，每个点的度可以定义为平均连接度c，易知$c = 2M/N$。

• 稀疏图(sparse graphs)和稠密图(dense graphs)：对于顶点数为N的图，其最大的边数为$M_{\max} = N(N-1)/2$。如果图的边数$M \sim M_{\max}$，此时$c \sim O(N)$，则此图的度随着顶点数的增加而增加，则此图就是稠密的(dense)。一个简单的例子为完全图(complete graph)，即其所有顶点都和图里其它所有的点相连接。即对于顶点数为N的全连接图，所有顶点的度均为$N-1 \sim N$。如果$M \sim N$，即对应的平均度$c \sim O(1)$，则图的平均度不随顶点数的增加而增加，则此图是稀疏的(sparse)。一个简单的稀疏图为一棵树：其顶点数为N时，边数一直为$N-1$，则平均度$c \sim 2$。Figure 13.5给出了稀疏图和稠密图的例子。

完全图一般在一些解析计算有应用，但注意到在真实世界中：

Real-world networks are sparse.（真实世界网络是稀疏的。）

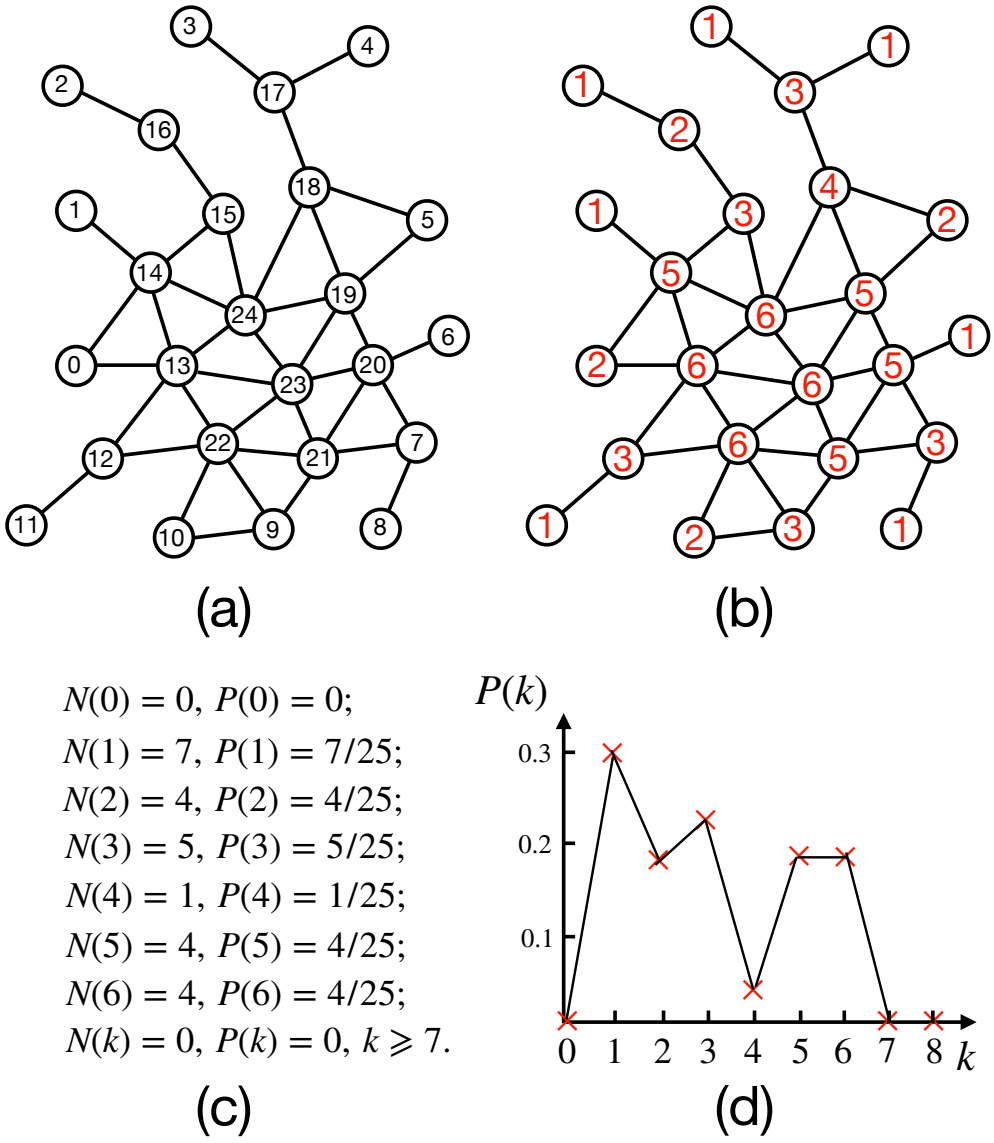


FIG. 13.4. 无向图的经验度分布的计算。 (a)为一个包含了$N = 25$个顶点、$M = 39$条边的无向图。顶点内标记的是顶点的序号。 (b)在每个顶点内标记了它的度。 (c)里统计了图中度为$k(\geqslant 0)$的顶点数$N(k)$，以及相应的度的概率$P(k)(= N(k)/N)$。 (d)对统计的度分布$P(k)$数据进行作图。图中横轴为度k、纵轴为度分布的$P(k)$，符号×表示每个k对应的$P(k)$值。

■ 顶点之间路径的相关性质

• 顶点之间的路径或通路(path)

表示两顶点之间的头尾相连的边的序列。对于无向图来说，比如Figure 13.6(b)中顶

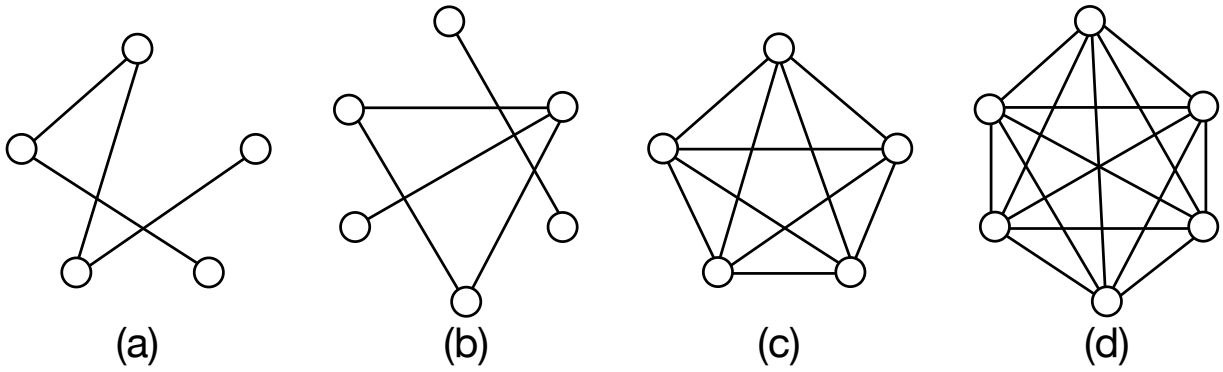


FIG. 13.5. 稀疏图和稠密图。 (a-b)分别为顶点数$N = 5$和$N = 6$的稀疏图。此时的平均度设置为$c = 2$。相应地两图的边数分比为$M = 5$和$M = 6$。 (c-d)分别为顶点数$N = 5$和$N = 6$的全连接图。此时的边数分比为$M = 5 \times 4/2 = 10$、$M = 6 \times 5/2 = 15$。

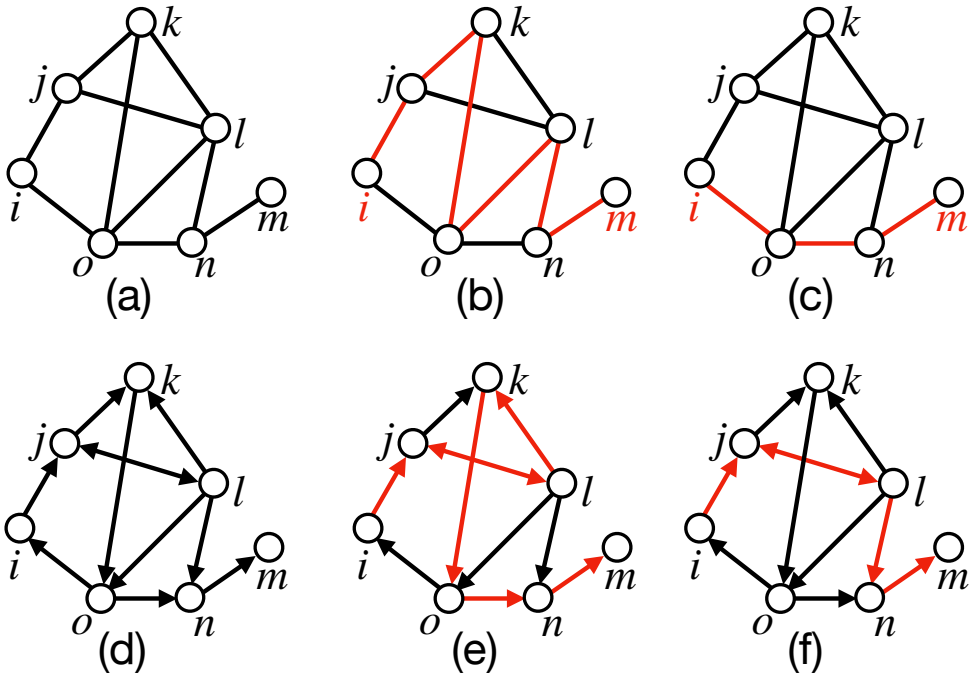


FIG. 13.6. 图中的路径。 (a)为7个顶点、10条边的一副无向图。(b-c)中红色边标记了从顶点i到顶点m的一条路径。 (d)为一副有向图。(e-f)中红色边标记了从顶点i到顶点m的一条路径。

点i到m有多条路径，比如

$$P_1 = (i, j, k, o, l, n, m); \tag{13.2}$$

$$P_2 = (i, j, l, n, m); \tag{13.3}$$

$$P_3 = (i, o, n, m). \tag{13.4}$$

对于有向图来说，序列里的边需要遵循边上的方向。 比如Figure 13.6(e)中顶点i到m的路径包括

$$P_1 = \langle i, j, l, k, o, n, m \rangle; \tag{13.5}$$

$$P_2 = \langle i, j, l, n, m \rangle. \tag{13.6}$$

• 顶点之间的距离(distance)：两顶点之间可能有多条路径，且每条路径都会有相应的距离。其中最短路径长度即为顶点之间的距离。图中的任意两个顶点之间，我们都能得到两者之间的距离。同时从给定的顶点出发，我们也可以计算得到其它所有顶点到它的距离。

• 图的连通单元(connected component/component)和连通性(connectedness)： 如上所述，不互为最近邻的两个顶点，可以通过通过路径间接连接起来。彼此之间都有路径连接的顶点所构成的子图叫连通单元。如果图中只有一个连通单元，则此图是连通的、为一个连通图(connected graph)。 如果图中有多个连通单元，则它就是不连通的(disconnected)。

当图中有多个连通单元时，我们一般尤其关注其中的最大连通单元(giant component)，即包含顶点数最大的那个连通单元。图的最大连通单元的比例，即其内部顶点数占图中所有顶点的比例，是衡量图的宏观连通性(macroscopic connectedness)的参数。 在研究图对应的复杂系统在面对外部扰动时的结构稳定性时，我们一般关注的就是最大连通单元的比例随着外部扰动强度的变化。

• 图的直径(diameter)

“直径”这个概念一般出现在圆这个场景。在圆中，直径对应着圆周两点相连、并且穿过圆心的线段的距离。从一个更广义的角度来看，这个距离本质上对应着圆中任意两点之间的欧几里得距离(Euclidean distance)的最大值。

这里我们继续把这个概念推广到任意连续几何形状上。此时几何形状里的“直径”可以定义为其内部任意两点之间欧几里得距离的最大值。本质上，任意连续几何形状的“直径”对应着用来完全覆盖这个形状的圆的最小直径。整体上，连续几何结构的“直径”这个概念可以用来度量它的宏观尺寸。 在Figure 13.7(c)，其几何形状的最大距离为d，则其直径为d。

我们可以把连续几何形状的直径概念继续推广到任意离散结构里。此时只要能够定义出来两点之间的“距离”，相应地也就能定义结构的“直径”。如上所述，在图这种典

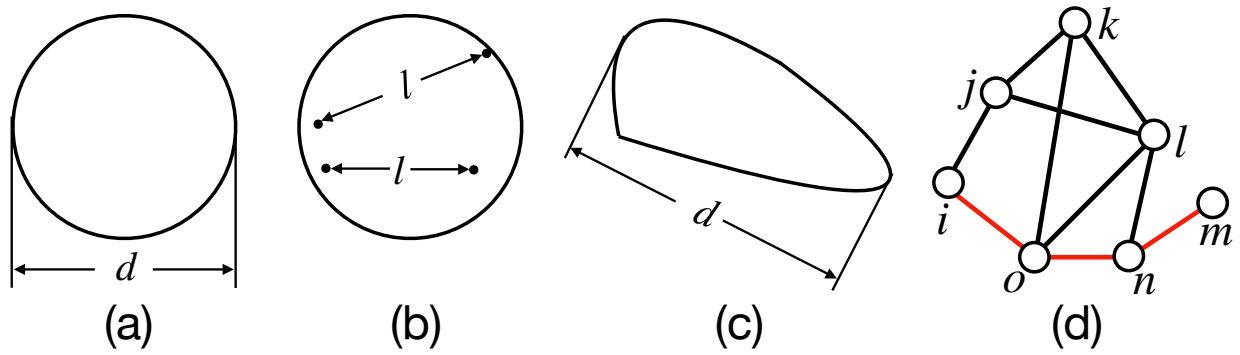


FIG. 13.7. 连续和离散几何结构的直径。 (a)为一个直径为d的圆。 (b)为连续几何结构里“直径”的定义。对应着图中任意两点之间距离l的最大值。 (c)为一个不规则的几何图形。图中所示距离d对应着这个几何结构里两点之间距离的最大值。此时d为此几何形状的“直径”。 (d)为7个顶点、10条边的一幅无向图。 顶点i和m之间的距离为3，对应标记为红色的路径，即(i,o,n,m)，的长度。3也是图中两个顶点间的最大距离。

型离散结构里，“距离”表示顶点之间所有路径的最小值。在一个连通的图里，我们首先计算出来任意两个顶点之间的距离。最终这些距离的最大值，即为图的直径。 对于图$G=\{V,E\}$，其直径d的定义可以写成:

$$d=\max_{\{i,j\}\in V} D_{ij}. \tag{13.7}$$

其中D_{ij}表示顶点i和j之间的距离。

对于Figure 13.7(d)，连通图中共有7个顶点$\{i,j,k,l,m,n,o\}$，则顶点对数目为$7\times 6/2=21$。我们首先可以列举出来所有顶点对之间的所有最短路径、及对应距离。 可知:

$$(1)\ i\longleftrightarrow j:(i,j);L(i,j)=1; \tag{13.8}$$

$$(2)\ i\longleftrightarrow k:(i,j,k);L(i,k)=2; \tag{13.9}$$

$$(3)\ i\longleftrightarrow l:(i,j,l);L(i,l)=2; \tag{13.10}$$

$$(4)\ i\longleftrightarrow m:(i,o,n,m);L(i,m)=3; \tag{13.11}$$

$$(5)\ i\longleftrightarrow n:(i,o,n);L(i,n)=2; \tag{13.12}$$

$$(6)\ i\longleftrightarrow o:(i,o);L(i,o)=1; \tag{13.13}$$

$$(7)\ j\longleftrightarrow k:(i,k);L(j,k)=1; \tag{13.14}$$

$$(8)\ j\longleftrightarrow l:(j,l);L(j,l)=1; \tag{13.15}$$

$$(9)\ j\longleftrightarrow m:(j,l,n,m);L(j,m)=3; \tag{13.16}$$

$$(10)\ j \longleftrightarrow n : (j,l,n); L(j,n) = 2; \tag{13.17}$$

$$(11)\ j \longleftrightarrow o : (j,l,o),(j,i,o); L(j,o) = 2; \tag{13.18}$$

$$(12)\ k \longleftrightarrow l : (k,l); L(k,l) = 1; \tag{13.19}$$

$$(13)\ k \longleftrightarrow m : (k,l,n,m),(k,o,n,m); L(i,j) = 3; \tag{13.20}$$

$$(14)\ k \longleftrightarrow n : (k,l,n),(k,o,n); L(k,n) = 2; \tag{13.21}$$

$$(15)\ k \longleftrightarrow o : (k,o); L(k,o) = 1; \tag{13.22}$$

$$(16)\ l \longleftrightarrow m : (l,n,m); L(l,m) = 2; \tag{13.23}$$

$$(17)\ l \longleftrightarrow n : (l,n); L(l,n) = 1; \tag{13.24}$$

$$(18)\ l \longleftrightarrow o : (l,o); L(l,o) = 1; \tag{13.25}$$

$$(19)\ m \longleftrightarrow n : (m,n); L(m,n) = 1; \tag{13.26}$$

$$(20)\ m \longleftrightarrow o : (m,n,o); L(m,o) = 2; \tag{13.27}$$

$$(21)\ n \longleftrightarrow o : (n,o); L(n,o) = 1. \tag{13.28}$$

从上面结果可知，顶点间距离最大者为3，对应着顶点对(i,m), (j,m), (k,m)之间的距离。对于上面的式子，如果忽略掉具体路径的信息，我们可以把任意顶点对的距离用一个矩阵表示出来，即用矩阵元素$D(i,j)$表示顶点i和j之间的距离。Table V展示了Figure 13.7(a)对应的距离矩阵。

TABLE V. 无向图中所有顶点对$(i,j) \in E$之间的距离$D(i,j)$。一个顶点到本身的距离定义为0，对应着矩阵中的对角线元素。红色数字标记了每行距离数据中的最大值。

$D(i,j)$	i	j	k	l	m	n	o
i	0	1	2	2	3	2	1
j	1	0	1	1	3	2	2
k	2	1	0	1	3	2	1
l	2	1	1	0	2	1	1
m	3	3	3	2	0	1	2
n	2	2	2	1	1	0	1
o	1	2	1	1	2	1	0

- 图的半径(radius)

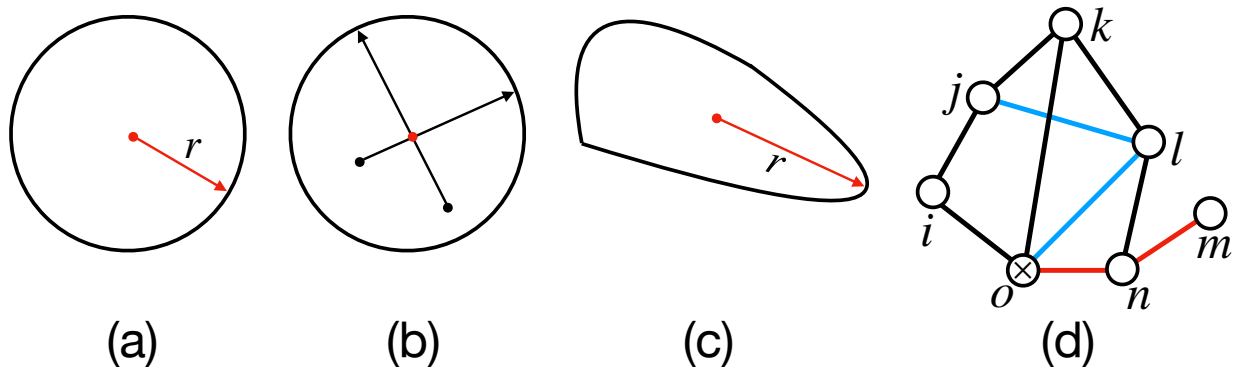


FIG. 13.8. 连续和离散几何结构的半径。 (a)为一个半径为r的圆。 (b)圆中从原点伸出的箭头表示此原点到圆中其它点的最大距离。 (c)为一个不规则的几何结构。距离r为其内部点到其它点的距离的最大值的最小值。相应地，r为此不规则几何结构的“半径”。 (c)为7个顶点、10条边的一幅无向图。 从文中可知，2对应着此图的半径。蓝色和红色标记的两条路径的长度均为2。相应地，顶点o为图的一个中心点。

对于二维平面上的一个圆，我们可以很容易理解“半径(radius)”这个概念，即圆心到达圆周上某一点的距离。 我们可以从另外一个角度来理解“半径”的含义。 对于圆中任意一点，我们可以得到它到达圆中其它点的距离。容易得知，这个距离当中的最大值对应着它穿过圆心、继续到达圆周的直线的距离。对于给定的圆中每个点，我们都可以得到对应的最大距离。其中的最小值就对应着圆的“半径”。

延续上述的表述方式，我们把圆这种结构的“半径”拓展到任意连续几何结构上。对于任意连续几何结构，可以得到其内部某一点到结构中其它所有点的距离，相应地我们可以得到其中的最大值。如此可以得到此结构中任意点到其它点的最大距离。而这些最大距离中的最小值即可以定义为这个几何结构的“半径”。 和“直径”一样，“半径”同样用来大致衡量连续几何结构的宏观尺寸。

在离散结构里，如果能够定义距离的概念，我们同样也能定义半径这个结构特征。 对于一个无权重的无向图，我们首先得到所有顶点离图中其它顶点的距离，从而到此顶点离其它顶点的最大距离。这些最大距离中的最小值就对应着图的半径。相应的距离原点为图的”中心点”。 对于图$G = \{V, E\}$，其半径r的定义可以写成:

$$r = \min_{i \in V} \max_{j \in V} D_{ij}. \tag{13.29}$$

对于Figure 13.8(d)的图，为了得到其半径，我们可以利用图的顶点间距离矩阵。从Table V可知，对于顶点$\{i, j, k, l, m, n, o\}$来说，它们离其它顶点的最大距离分别

为$\{3,3,3,2,3,2,2\}$。可知这些最大距离的最小值为2，即此图的半径为2。相应地，顶点$\{l,n,o\}$均为图的中心点。

■ 图中的圈或者环路

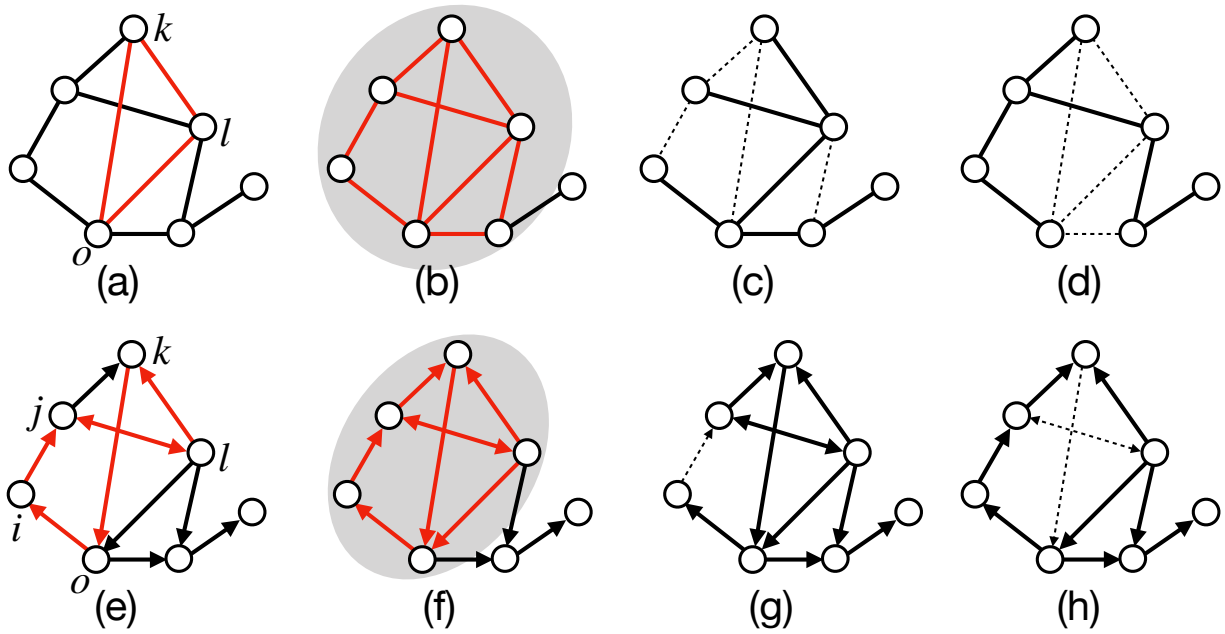


FIG. 13.9. 图中的圈或环路。(a-d)对应无向图的情形。(a)的红色的边标记了图中的一个圈。(b)为图中所有的圈所构成的结构。(c-d)为从图中去除一些边之后得到的无圈图。(e-h)对应有向图情形。(e)中的红色有向边标记了图中的一个圈。(f)为图中所有的圈所构成的结构。(g-h)为从图中去除一些边之后得到的无圈图。

• 图的圈或环路(cycle/loop)：表示图中头尾相连的路径，此时的路径中的边是不重复的。在无向图中，圈为无向边的集合。Figure 13.9(a)中(k,l,o)对应了无向图中的一个圈。在有向图中，圈中环路为头尾相连的有向边。Figure 13.9(e)中的有向路径$\langle i,j,l,k,o\rangle$对应了有向图中的一个圈。

在无向图中，所有的圈的顶点和边构成了一个子图：从此子图中任意顶点出发，沿着子图内的边能够再次回到此顶点。Figure 13.9(b)展示了无向图中的这个结构。那么如何得到无向图里的这个结构？容易得知，此结构中所有顶点的度$\geqslant 2$。为了得到这个子图，可以删除度为0以及1的所有顶点。删除度为1的顶点的过程对应于移叶过程(leaf removal)。注意到这个过程是迭代进行的过程，也就是图中可能有一些顶点在最开始度大于2。但随着移叶过程的执行，它的度会变成1，从而被删除。此过程留下的子图结构本质

上对应着2核结构。 感兴趣的同学可参考第15章中的“K-核渗流”一小节。

而在有向图中，所有的圈的顶点核边也构成了一个特别的子图：从此子图中的任意一个顶点出发，沿着子图内的有向边移动，最终可以重新回到此顶点。有向图里的这个结构叫强连通单元(Strongly Connected Component, SCC)。 Figure 13.9(f)展示了(e)对应的SCC。 那么如何得到有向图里的强连通单元? 容易得知，SCC结构中所有顶点的入度1且出度$\geqslant 1$，原因在于顶点作为圈中路径的一部分，至少有一个出边和一条入边。容易可知，那些入度为0、或者出度为0的顶点肯定不属于SCC结构。此时，我们可以迭代地删除所有入度为0、或者出度为0的顶点。那么最终得到的结构能保证所有的顶点的入度1且出度$\geqslant 1$，此时也就是一个SCC结构。 感兴趣的同学可参考第15章中的“有向图的强连通单元”一小节。

• 无环图或无圈图(acyclic graphs)：如果图中没有圈，则此图为无环图或无圈图。如果无环图是连通的，则此图为一棵树(tree)；如果此图不是连通的，此结构对应着一个者森林(forest)。 Figure 13.9(c-d)分别为无向图情形下的树结构，(g-h)表示了从(e)中有向图情形下的两棵树。

容易得知，无向图的树本身为一个连通单元，即如果删去一条边，则它会分裂成两个连通单元；但如果在它里面加入一条边，结构里会出现圈。这意味着树既为极大无环图、也为极小连通图。

■ 网络中的一些关键顶点

• 图的关节点(articulation points)和桥边(bridges)

图中的关节点和桥边与连通单元的数目有关：当移除某个点或边后，如果图的连通单元数目增加，则此点或边即为关节点或桥边。图示可见Figure 13.10。 不存在关节点的子图叫重连通单元(biconnected component)。

因为关节点和桥边连接着不同的连通单元，它们的移除可以认为是破坏图宏观连接性的一种便捷的方式。 如何解析地计算给定随机图上关节点和桥边的数目、 如何解析地预估需要多少个关节点才能得到双连通单元(biconnected component)， 可参考[29, 30]。

■ 数据结构意义下的树和图

作为数据结构的树和作为图结构的树。基于图结构的术语可知，作为一种数据结构的k叉树可以视为一种特殊的图结构。如果把一棵k叉树中父节点到其子节点的关系视为从父节点指向子节点的一条有向边，则此k叉树可以转换为有向图意义下的一棵树。此时有向图中根节点的入度为0、其它节点的入度为1、同时所有节点的出度$\leqslant k$。如果把k叉

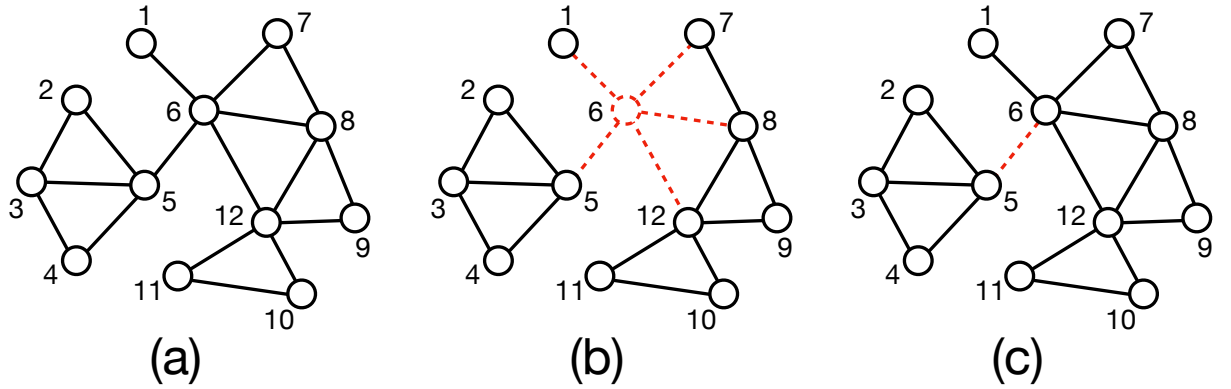


FIG. 13.10. 图的关节点和桥边。 (a)为包含12个顶点、17条边的一幅无向图。此时图中只有一个连通单元。 (b)中顶点6被移除，和它相邻的所有边一同也被移除。此时图分裂成三个连通单元，顶点数目从小到大依次为1、4、6。易知顶点6在原图里为关节点。同理可知顶点5和12也为关节点。 (c)中边$(5,6)$被移除后，图分裂成两个连通单元。易知边$(5,6)$在原图里为桥边。同理可知$(1,6)$也为桥边。

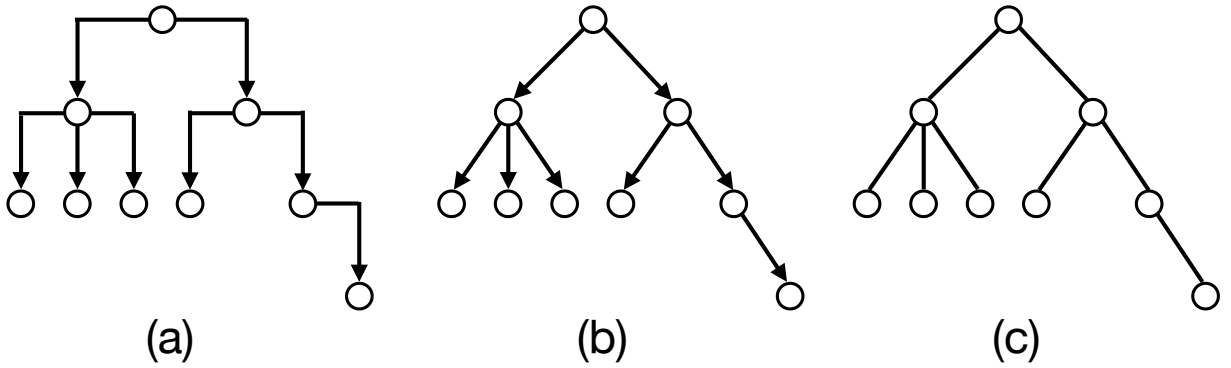


FIG. 13.11. 树作为一种特殊的图结构。 (a)中为由9个节点、8条边构成的三叉树。(b)和(c)分别为(a)对应的有向图和无向图。

树中父节点到子节点的关系视为一条无向边，则此k叉树可以视为无向图意义下的一棵树。 Figure 13.11展示了一棵3叉树转换为有向图和无向图的过程。

■ 网络的高阶结构

• 社区结构(communities)

社区结构为网络中边比它处密集的局域结构。作为网络的介观结构，它是真实网络系统的动力学行为和功能的重要结构基础。网络科学研究领域里是一个经典的社交网络就是

空手道俱乐部网络(Zachary's Karate club network)[32]。其结构如图Figure 13.12所示，可知它包含了两个社区结构。 社区发现问题[31]也就是如何从图的结构信息来寻找图中的社区结构及其结构特征。 空手道俱乐部网络常常作为图例，用来测试和比较复杂网络上的各种社区发现算法。

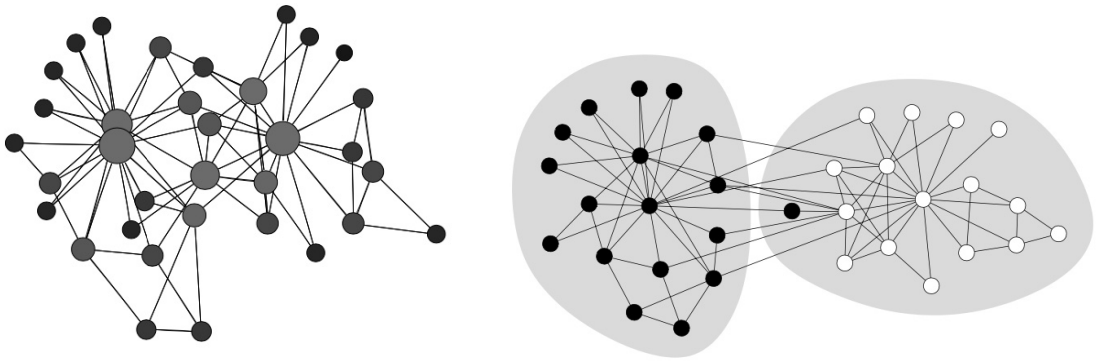


FIG. 13.12. 图的社区发现问题。 空手道俱乐部网络(Zachary's Karate club network)为无向连通图，包含34个顶点、78条边。 顶点表示俱乐部的学员和教练，边表示学员和教练之间的相识关系。 左图为网络的大致结构。 右图中的两个色块所覆盖的子图为使用某个社区发现算法划分出来的两个网络社区结构， 黑色和白色的点对应着人直观上区分出来的两个网络社区结构。 两幅子图分别来自[26]的Figure 1.2和14.3。

模块系数（modularity）Q的定义。

- 层级结构（Hierarchical structure）
- 度关联（Degree correlation）
- 团聚系数（Clustering coefficient）

■ 超越简单网络的网络结构

- 相互依赖网络(interdependent networks)
- 网络的网络(networks of networks)
- 高阶相互作用网络(networks of higher-order interactions)

截止到现在为止，我们考虑的图的边只涉及到两个顶点。但在一些场合，比如化学反应和桥牌等游戏里，相互作用发生在多个($\geqslant 3$)作用单元之间。此时我们使用超边(hyperedge)来描述这种涉及到多个顶点的边。如此抽象出来的图结构叫高阶网

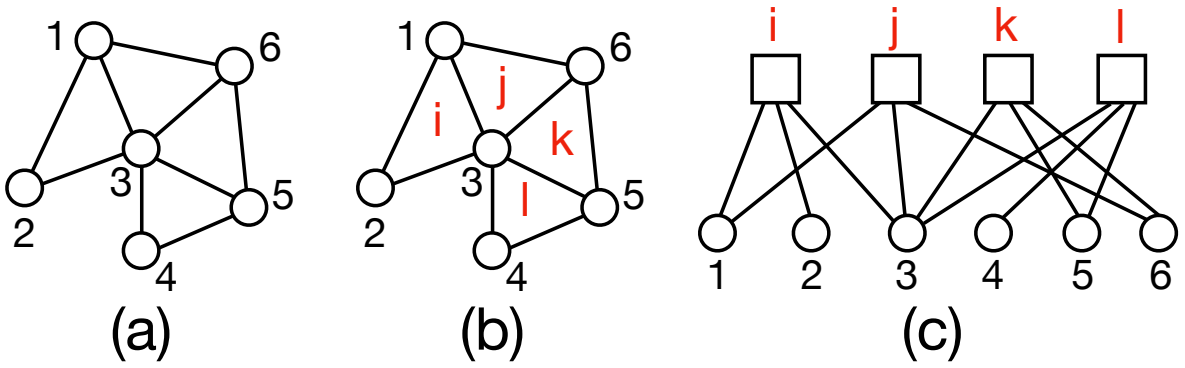


FIG. 13.13. 高阶网络的表示方式。 (a)为表示二阶相互作用的一个图，包括6个顶点、9条边。(b)为表示三阶相互作用的一个图，包括6个顶点、4条边。i, j, k, l为三体相互作用（边）的编号，分别包含了顶点$(1, 2, 3)$、$(1, 3, 6)$、$(3, 5, 6)$、$(3, 4, 5)$。 (c)为(b)的二部图(bipartite graphs)表达方式，其中圆点表示顶点，方块顶点表示边，方块顶点和圆点的连线表示每条边对顶点的包含关系。

络。Figure 13.13展示了高阶网络的一些例子。

13.2. 无向图的ADT

这里我们讨论如何使用代码来存储图的结构，以进行后续的算法处理。

在考虑使用ADT来存储某个数据结构时，我们可以问以下三个问题：

- 问题1：什么才是图里最重要的结构信息?
- 问题2：如果直接使用上述结构信息来描述图时，是否是个好的选择?
- 问题3：使用图来处理图上的各种算法时，如何把图结构本身和具体的问题进行区分?

考虑一副无向图$G = \{V, E\}$，其中顶点集为$V(|V| = N)$、边集为$E(|E| = M)$。相应地其平均度为$c = 2M/N$。 对于图G来说，上述问题1的答案是图的边，也即是“哪些顶点之间有边相连”。此时对应的数据集为边数据集，其大小为M。

对于问题2来说，仅仅存储图的边在算法层面上显然不是最优的。比如对于图这种非线形结构，输出一个顶点的所有邻居这么简单的任务，就需要把整个边集遍历一遍。此时更适合的结构描述需要从“边”的视角转移到“顶点”的视角，也就是存储每个顶点以及他们的最近邻。容易看出来，这种方法本质上把所有的边存储了两遍。之后我们将会看到，这种视角会加速算法，但代价是需要更大的存储空间。这是一个典型的“用空间换时间”的算法

选择。

而对于问题3，在后面的图算法小节里，比如图上的遍历算法、优化问题、动力学问题等等，一般可以需要在图中基于顶点或者边的等等信息来选择顶点或者边。此时可以通过不改变已有的图结构信息、而是在顶点和边的数据结构里添加新的数据成员来实现。此外有些算法涉及到删除顶点和边，但本质上我们无需真的去删除顶点或者边。我们同样可以通过给顶点和边设置“存在”和“被删除”的状态即可进行区分。

这里我们继续比较一下边视角对应的数据描述和顶点视角对应的数据描述。 假设顶点序号用整型int记录，其内存单元为S_{int}。一个邻居指针的内存为S_{ptr}。 最基本的结构也就是边的ADT，即：

• 边Edges：大小为M，包括每条边的两个顶点。其内存大小为$M \times 2S_{\text{int}}$。

但如果我们需要基于顶点和其邻居的数据结构，如Figure 13.14所示，整体上我们还需要实现以下两个部分：

• 顶点Vertices：大小为N，顶点相关的信息，比如度、指向其最近邻的指针等。其内存大小为$N \times (S_{\text{int}} + S_{\text{ptr}})$。注意此时因为顶点按顺序排列，无需记录顶点对应的序号。

• 顶点的邻居Neighbors：大小为$2M$，存储了所有顶点的邻居，包括其邻居顶点编号、边序号等等。其内存大小为$2M \times (S_{\text{int}} + S_{\text{int}})$。

容易得知，两个数据结构所需的内存空间的比例为：

$$\begin{aligned}\frac{S_{\text{E}} + S_{\text{V}} + S_{\text{N}}}{S_{\text{E}}} &= \frac{M \times 2S_{\text{int}} + N \times (S_{\text{int}} + S_{\text{ptr}}) + 2M \times (S_{\text{int}} + S_{\text{int}})}{M \times 2S_{\text{int}}} \\ &= \frac{c + 1 + \frac{S_{\text{ptr}}}{S_{\text{int}}} + 2c}{c} \\ &= 3 + \frac{1}{c}\left(1 + \frac{S_{\text{ptr}}}{S_{\text{int}}}\right). \end{aligned} \tag{13.30}$$

可知当$c \gg 1$时，顶点视角的数据结构描述的内存大约为3倍的基于边的数据结构描述。

在本讲义里，我们一般使用的就是顶点视角的数据结构描述。 在图背景的问题里，我们一般需要从数据文件里读取边数据（包括的边的权重）来构造图结构。这样使得我们能够去处理具有各种真实网络。基于数据集来生成图的读图过程可以写成通用函数，作为其它算法的预处理步骤。

有了边数据后，我们可以按要求来构造图结构。考虑到不同的场景里，邻居Neighbors的实现会有一些差别。 这里我们考虑两种场景：

（1）图的生成：此时我们需要往顶点集里不断添加边，相应地图结构处在动态之中。此时顶点的邻居是变动的。对于每个顶点，使用单独的一个向量来存储它的邻居，是一个直观、简单的方法。但缺点是，在内存里这种多个变化的向量需要很大的（原大于邻居总

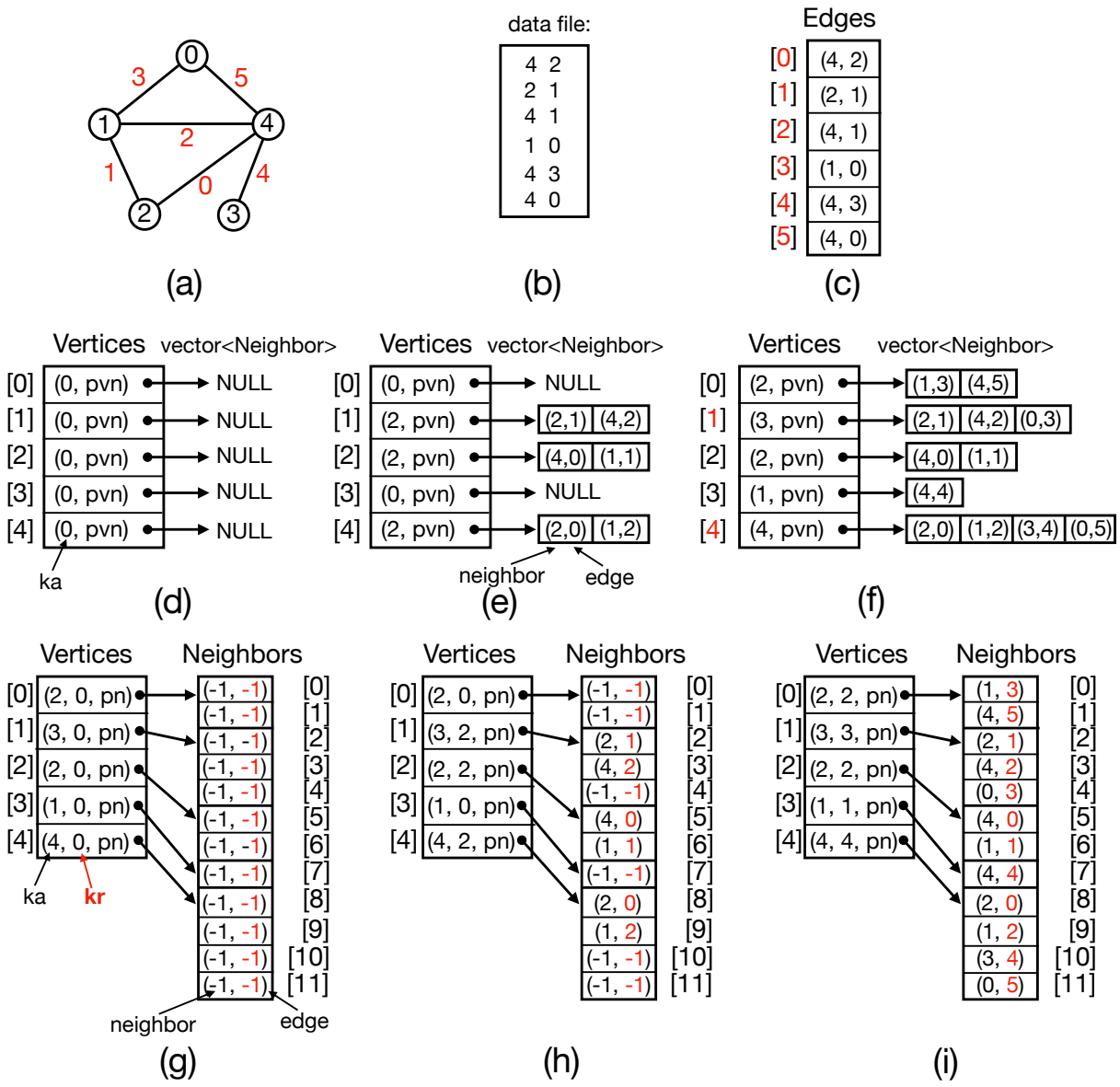


FIG. 13.14. 无向图的ADT。 (a)为一个包含了顶点数$N = 5$、边数$M = 6$的图结构。顶点内部为其编号，边附近的为其编号。 (b)为读入图结构的边数据集。 (c)为边的存储结构Edges。 (d-f)对应图的动态结构。(g-i)对应图的静态结构。 (e-f)和(h-i)分别展示了读入3和6条无向边时数据成员的变化。

数$2M$）存储空间。

这个场景的一个简单例子就是生成随机图，也就是在给定生成边的规则下不断为顶点集构造边。

（2） 给定图结构上的动力学过程：此时我们在一个固定的图结构上执行动力学过

程，或者包含了顶点和边被删除的情形。这些情形都意味着网络的顶点和边数都是有上限的。此时我们可以使用一个连续的结构（大小为$2M$）来放置所有顶点的最近邻。 如果此动力学过程涉及到删除顶点和边，只需要在把顶点和边的状态标记为“被删除”即可，无需真的从ADT中删除顶点或者边。图上的动力学过程一般需要首先在顶点或边上定义状态（比如二元状态），通过具体的算法来不断更新状态，以最终选择属于某种状态的顶点或边。这个过程涵盖了绝大部分定义在图上的遍历和搜索过程、优化问题的求解等等。

对于动态的图结构ADT，我们首先构造出来顶点的ADT，即Vertices。此时顶点的邻居都是空的。之后在读取边数据时，不断为顶点添加邻居。比如对于序号为eind的边$(v1, v2)$，我们在Vertices结构里为$v1$添加一个邻居结构体Neighbor，其包含了邻居编号$v2$、边序号eind，相应地更新$v1$的度ka；同时为$v2$添加一个邻居结构体Neighbor，其包含了邻居编号$v1$、边序号eind，相应地更新$v2$的度ka。当我们把给定的边数读取之后，就能得到一个图结构ADT。

对于静态的图结构ADT来说，其构造图的流程会稍微复杂一些。我们首先构造出来大小为N的Vertices和大小为$2M$的Neighbors。从读取边集到构造完整的图结构ADT的过程中，我们需要读取两次。

- 第一次读取边是为所有的顶点赋予度ka。之后我们利用顶点的度来为每个顶点指向其邻居序列赋值。可知对于顶点i，其对应的Neighbors中的邻居的起始位置为：$n_i = \sum_{j<i} k_j$，即序号小于i的所有顶点的度之和。
- 第二次读取边为了给顶点的邻居赋值。此时我们依次读取边，不断填充Neighbors，直到所有边都读取完。

需要说明的是，Vertices结构里的成员kr记录的是不断读入边时顶点当下的度。但它在图结构的描述里并不是的必须的。它的设置是考虑到随机图或者随机化的图的场景。在随机图模型里，边的构造是相互独立的。当我们构造了一个顶点数为N、平均度为c的某个随机图时，此时我们有一个长度$cN/2$的边数据集。如果取这个边数据集的前面部分、且大小为$c'N/2$，我们可以构造出平均度为c'的另外一个随机图。此时利用已有的边数据，我们可以构造出来边数更少的多个随机图。在ADT的层面，度更小的图可以在度更大的图上来实现。Vertices结构里的kr就是为了控制顶点的邻居范围以实现度更小的图结构ADT。如Figure 13.14，我们可以在边数为6的一个图结构(i)里实现边数为3的一个图(h)。

有了图的ADT之后，可以在图上实现多种具体的算法。比如：

（1）输出所有顶点的邻居。

（2）统计给定图的经验度分布。

例子1: 对于一副无向图，我们有它的边的数据文件。请读入数据，输出其所有顶点的邻居，并统计其度分布。

参考代码：ch13-graph.cc里的函数ReadAnUndirectedGraph_Dynamic()和ReadAnUndirectedGraph_Static()。

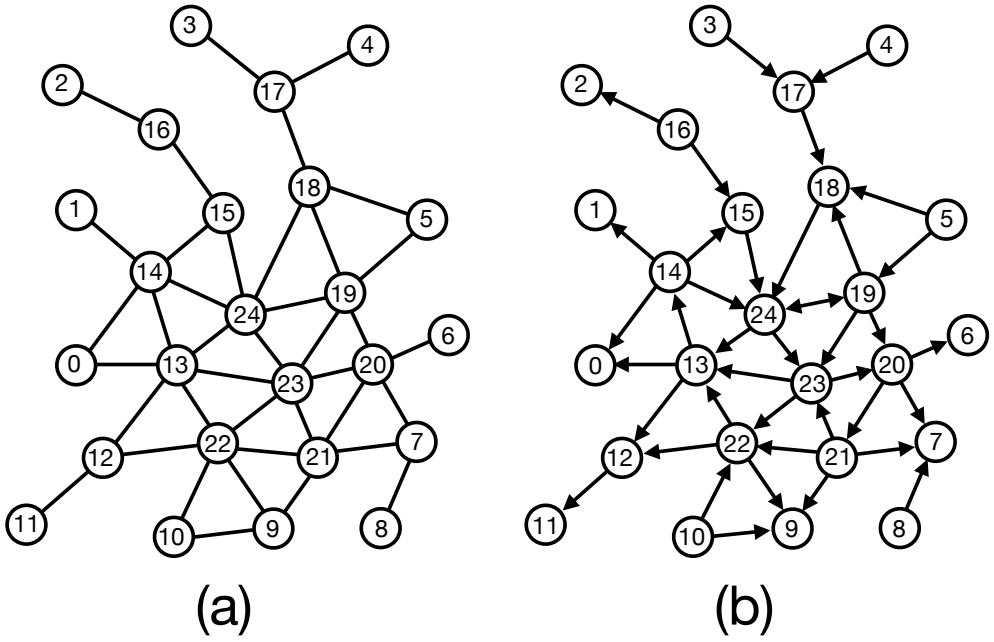


FIG. 13.15. 无向图和有向图的两个例子。 (a)为顶点数$N = 25$、边数$M = 39$的一幅无向图。(b)为顶点数$N = 25$、边数$M = 40$的一幅有向图。 上述的有向图来自于为无向图的边赋予方向。注意顶点19和24之间的边为双向边（此时边数按2计算），其余边为单向边。

Figure 13.15(a)对应一幅无向图。此无向图即为Figure 13.4(a)中的无向图。其度分布的代码计算结果可以和图中的结果做对比。

计算度分布的函数为CountPQ()。过程如下： 首先遍历图的所有节点，以寻找图的最大度$k_{\max}$； 之后构造大小为$k_{\max}+1$的PQ，将用来记录度从0到$k_{\max}$的节点数和相应概率； 再次遍历图的所有节点。对于每一个节点i，读取它的度k_i，对应PQ[k_i]的数据成员Nk增加1。

13.3. 有向图的ADT

在有向图的场景下，我们遇到的问题和无向图类似。 此时我们可以构造类似的顶点视角的动态和静态数据结构。 有向图和无向图的差别在于边的有向性。在ADT层面，我们

需要给顶点区分入邻居和出邻居，同时区分入度和出度。此时在读入有向图的数据时，需要设置顶点的顺序。 比如图数据文件中的每行数据包含两个数字编号$v1$和$v2$。我们可以设置格式$(v1, v2)$对应边方向为$v1 \leftarrow v2$。

Figure 13.16给出了有向图的ADT，描述了从有向边数据集读入输入以构造图结构的过程。 在动态数据结构里，我们为顶点分别设置了入度和出度，以及入邻居和出邻居的向量的指针。 在静态数据结构里，我们把所有顶点的入邻居和出邻居依次放入Neighbors。每次读如边$(v1, v2)$时，我们需要把$v1$放到$v2$的出邻居里、把$v2$放到$v1$的入邻居里。

例子2: 对于一副有向图，我们有它的边的数据文件。请读入数据，输出其所有顶点的入邻居和出邻居。

参考代码：`ch13-graph.cc`里的函数`ReadADirectedGraph_Dynamic()`和`ReadADirectedGraph_Static()`。

Figure 13.15(b)对应一幅有向图。 代码读入此图对应的边数据，以构造图结构。最终输出每个顶点的入邻居和出邻居。

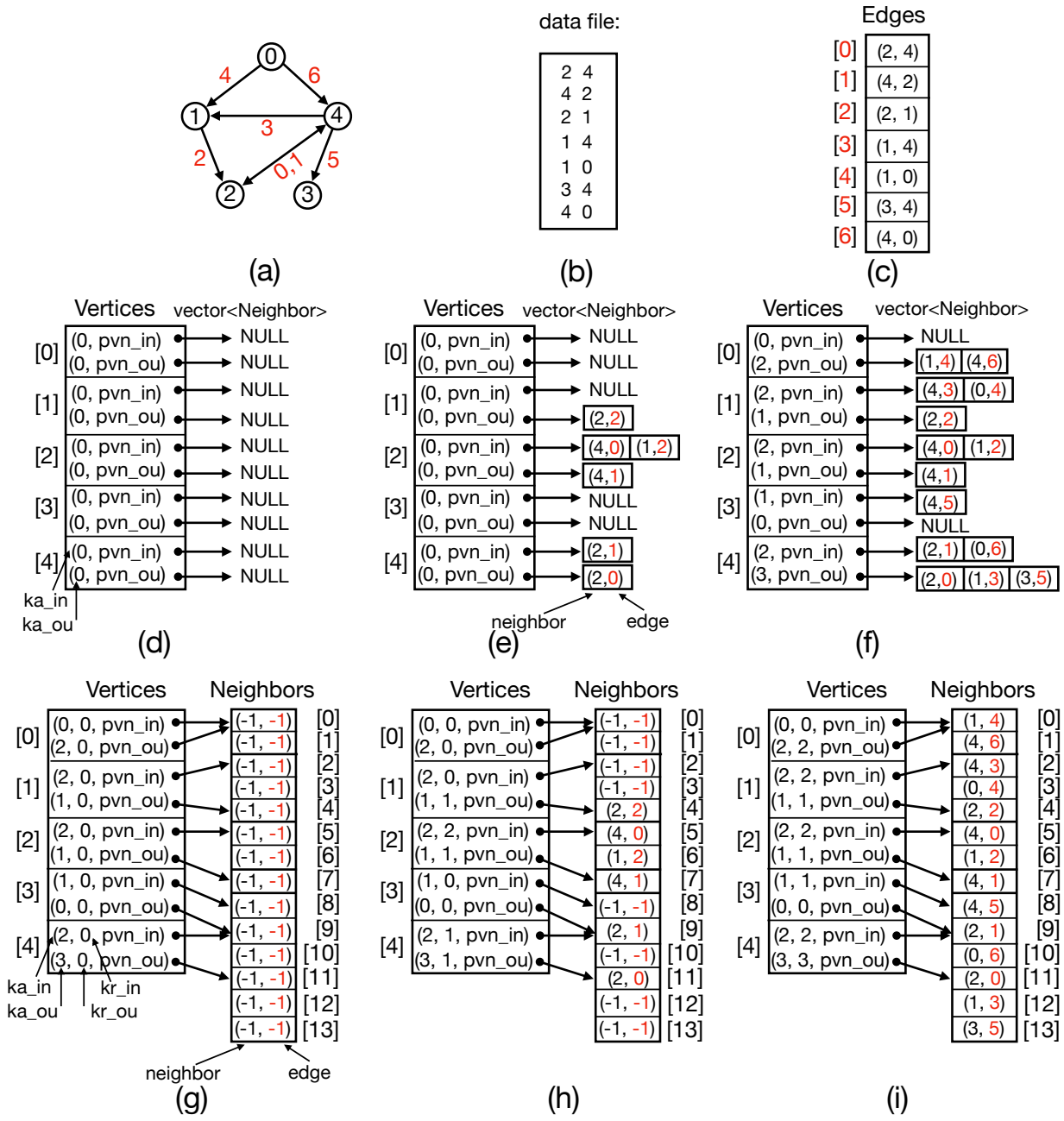


FIG. 13.16. 有向图的ADT。 (a)为一个包含了顶点数$N = 5$、边数$M = 7$的图。顶点内部为其编号，边附近的为其编号。 顶点2和4之间的边是双向的，此时按照两条边来计算。 (b)为读入图结构的边数据集。 (c)为边的存储结构Edges。 (d-f)对应图的动态结构。(g-i)对应图的静态结构。(e-f)和(h-i)分别展示了读入3和7条有向边时数据成员的变化。

13.4. 图的遍历

如同我们在二叉树章节一样，我们也可以在图结构里通过遍历来访问图的所有顶点和边。 本质上，在无向图的意义下，从任意一个顶点出发，我们可以通过边访问这个顶点所在的连通单元。 从局域上来看，最基本的操作是从一个顶点出发、通过边来访问它的邻居。这是图上遍历算法的基本步骤。但以什么样的顺序来访问邻居导致存在不一样的遍历算法。以下我们讨论两种遍历算法：广度优先搜索(Breath-First Search, BFS)算法和深度优先搜索(Depth-First Search, DFS)算法。

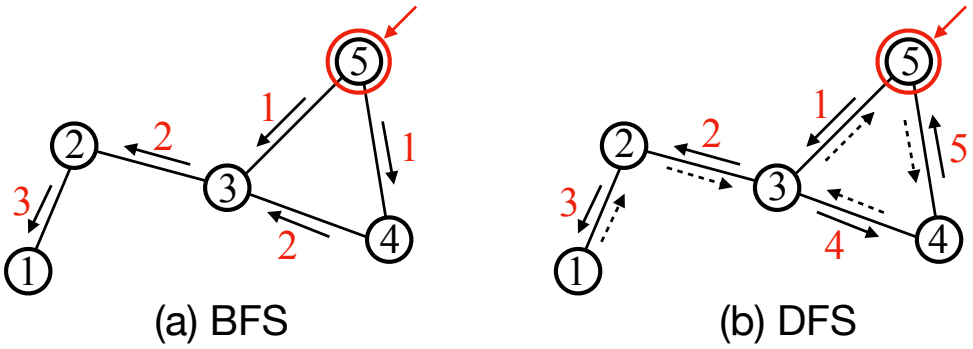


FIG. 13.17. 图中遍历算法的图示。 考虑顶点数$N = 5$、边数$M = 5$的一个无向图。 顶点内的的数字为顶点的编号，边附近箭头边的数字为算法访问步骤的标记。 (b) 图中的广度优先搜索算法。从顶点5开始进行顶点的遍历。 (a) 图中的深度优先搜索算法。虚线表示递归算法步骤的返回步骤。

对于BFS，我们依次访问给定顶点相距距离为1、2、3等等的所有邻居。在算法实现上，这样的顺序访问方式，我们可以使用队列来实现。我们首先压入算法的起始顶点，之后依次取出并访问队首的元素、然后把此元素的还未访问的邻居压入队列的队尾。这个流程不断访问距离起点越来越远的顶点，直到最终所有和起点相连的顶点被访问。容易看出来，BFS算法的中间数据结构也就是这个队列。

对于DFS，给定一个起始顶点后，我们访问当下顶点的一个邻居，然后继续访问这个邻居自身的一个未被访问的邻居。这个过程一直尽心下去，直到所有的顶点被访问。在算法实现上，这个过程是个递归算法，需要使用栈来存储它的中间执行过程和数据。

对于两个遍历算法，我们都需要在顶点上定义新的状态来表示顶点是否被访问。

例子3: 无向图上的遍历算法

问题：对于Figure 13.15(a)中的无向图，使用遍历算法来遍历图，以输出得到和给定起点相连的所有顶点。

使用BFS算法时，代码可参考`ch13-graph.cc`中的函数`BFS()`。 计算结果如Figure 13.18所示。

使用DFS算法时，代码可参考`ch13-graph.cc`中的函数`DFS()`。 计算结果如Figure 13.19所示。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 0
TraSeq:
[ 0: 0->13][ 1: 0->14][ 2:13->12][ 3:13->22][ 4:13->23][ 5:13->24][ 6:14-> 1][ 7:14->15][ 8:12->11][ 9:22-> 9]
[10:22->10][11:22->21][12:23->19][13:23->20][14:24->18][15:15->16][16:21-> 7][17:19-> 5][18:20-> 6][19:18->17]
[20:16-> 2][21: 7-> 8][22:17-> 3][23:17-> 4]
Component: { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,}.
Its size = 25
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 20
TraSeq:
[ 0:20-> 6][ 1:20-> 7][ 2:20->19][ 3:20->21][ 4:20->23][ 5: 7-> 8][ 6:19-> 5][ 7:19->18][ 8:19->24][ 9:21-> 9]
[10:21->22][11:23->13][12:18->17][13:24->14][14:24->15][15: 9->10][16:22->12][17:13-> 0][18:17-> 3][19:17-> 4]
[20:14-> 1][21:15->16][22:12->11][23:16-> 2]
Component: { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,}.
Its size = 25
```

FIG. 13.18. 无向图上从单个顶点出发的广度优先搜索算法(BFS)的计算结果。 图中展示了在Figure 13.21(a)中分别以顶点0、20为起始点得到的遍历结果。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 0
TraSeq:
[ 0: 0->13][ 1:13->12][ 2:12->11][ 3:12->22][ 4:22-> 9][ 5: 9->10][ 6: 9->21][ 7:21-> 7][ 8: 7-> 8][ 9: 7->20]
[10:20-> 6][11:20->19][12:19-> 5][13: 5->18][14:18->17][15:17-> 3][16:17-> 4][17:18->24][18:24->14][19:14-> 1]
[20:14->15][21:15->16][22:16-> 2][23:24->23]
Component: { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,}.
Its size = 25
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 20
TraSeq:
[ 0:20-> 6][ 1:20-> 7][ 2: 7-> 8][ 3: 7->21][ 4:21-> 9][ 5: 9->10][ 6:10->22][ 7:22->12][ 8:12->11][ 9:12->13]
[10:13-> 0][11: 0->14][12:14-> 1][13:14->15][14:15->16][15:16-> 2][16:15->24][17:24->18][18:18-> 5][19: 5->19]
[20:19->23][21:18->17][22:17-> 3][23:17-> 4]
Component: { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,}.
Its size = 25
```

FIG. 13.19. 无向图上从单个顶点出发的深度优先搜索算法(DFS)的计算结果。 图中展示了在Figure 13.21(a)中分别以顶点0、20为起始点得到的遍历结果。

对于Figure 13.18和 13.19，容易得知，当图中只有一个连通单元时，算法从图中任意顶点出发，都能访问顶点所在连通单元的所有顶点。

对于BFS和DFS，如果把遍历算法访问的边依次连接起来，可以看出来算法遍历图的顺序。基于Figure 13.18和13.19，Figure 13.20展示了把顶点20作为源点后的BFS和DFS遍历算法的中边的访问顺序。 容易看出来，这些边集都呈现为树的结构。

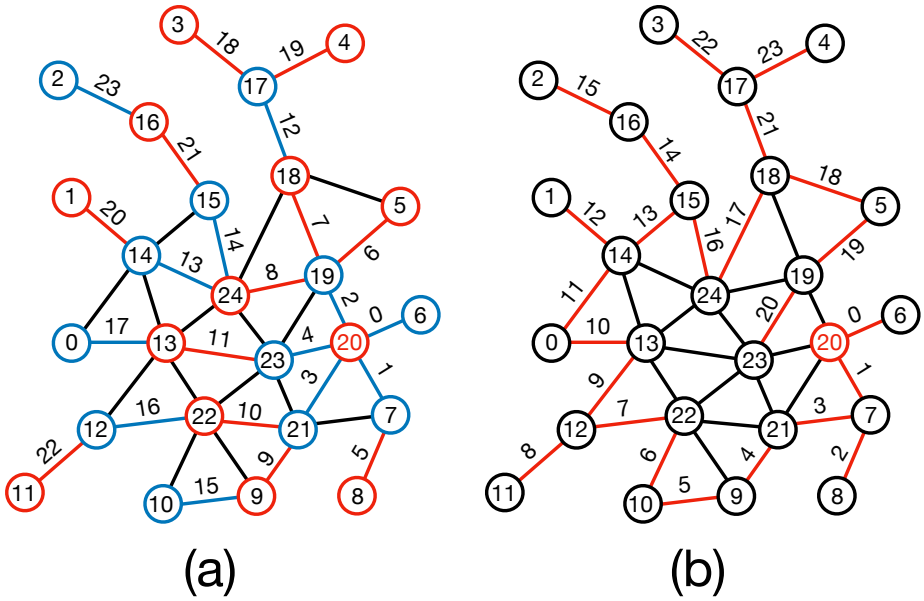


FIG. 13.20. 无向图中遍历算法对应的序列。 此时把顶点20作为遍历算法的源点。边上的序号标记了边的访问顺序。 (a)BFS对应的边的访问序列。顶点边缘的蓝色和红色表示这些顶点到源点的距离为奇数和偶数。 (b)DFS对应的边的访问序列。标记为红色边的标识了边的访问顺序。

13.5. 图的遍历：多个连通单元的情形

当我们能够利用遍历算法来访问图的所有顶点和边时，我们可以得到有关图的更加细致的宏观统计性质。比如顶点之间的路径、判断图是否为连通图等等。

无向图中所有顶点可能都属于一个连通单元，也可能包含多个连通单元。 比如Figure 13.21(a)中的无向图包含了4个连通单元。此时我们如何来遍历这样的图?

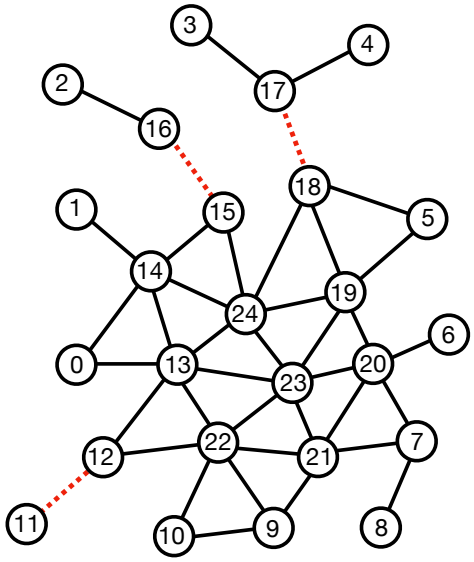


FIG. 13.21. 有着多个连通单元的无向图。 此图为从Figure 13.15(a)中删除3条边之后的结果。被删除的边为$(11,12)$、$(15,16)$、$(17,18)$，已用红色划线标记了出来。此时图中出现了4个连通单元。

如Figure 13.22和13.23所示，当只有从多个顶点出发来进行遍历，才可能找到图中所有的连通单元。 当图中存在多个连通单元时，使用遍历算法找到的连通单元依赖于选择的顶点本身。如果需要遍历某个连通单元，需要选择此连通单元里的顶点作为源点。

那么，可否基于遍历算法来设计一个新的算法来把无向图中的所有连通单元都找出来? 此时我们可以给顶点赋予一个连通单元状态来标记顶点所属的连通单元。 我们可以多次调用遍历算法，直到所有顶点都具有各自的连通单元状态。其算法大致如下：

（1）图中所有顶点的连通单元状态初始化为默认值，比如-1。

（2）遍历图的顶点，如果其连通单元状态为默认值，则把它作为源点使用搜索算法。在搜索算法中，访问顶点时需要把其连通单元状态设置为源点的值。

（3）多次执行（2），直到图中所有顶点的连通单元状态都不为默认值。

```
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 0
TraSeq:
[ 0: 0->13][ 1: 0->14][ 2:13->12][ 3:13->22][ 4:13->23][ 5:13->24][ 6:14-> 1][ 7:14->15][ 8:22-> 9][ 9:22->10]
[10:22->21][11:23->19][12:23->20][13:24->18][14:21-> 7][15:19-> 5][16:20-> 6][17: 7-> 8]
Component: { 0, 1, 5, 6, 7, 8, 9,10,12,13,14,15,18,19,20,21,22,23,24,}.
Its size = 19
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 2
TraSeq:
[ 0: 2->16]
Component: { 2,16,}.
Its size = 2
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 4
TraSeq:
[ 0: 4->17][ 1:17-> 3]
Component: { 3, 4,17,}.
Its size = 3
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 11
TraSeq:

Component: {11,}.
Its size = 1
```

FIG. 13.22. 无向图上从单个顶点出发的广度优先搜索算法(BFS)的计算结果。 图中展示了在Figure 13.21(a)中分别以顶点0、2、4、11为起始点得到的遍历结果。

```
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 0
TraSeq:
[ 0: 0->13][ 1:13->12][ 2:12->22][ 3:22-> 9][ 4: 9->10][ 5: 9->21][ 6:21-> 7][ 7: 7-> 8][ 8: 7->20][ 9:20-> 6]
[10:20->19][11:19-> 5][12: 5->18][13:18->24][14:24->14][15:14-> 1][16:14->15][17:24->23]
Component: { 0, 1, 5, 6, 7, 8, 9,10,12,13,14,15,18,19,20,21,22,23,24,}.
Its size = 19
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 2
TraSeq:
[ 0: 2->16]
Component: { 2,16,}.
Its size = 2
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 4
TraSeq:
[ 0: 4->17][ 1:17-> 3]
Component: { 3, 4,17,}.
Its size = 3
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
Starting point: 11
TraSeq:

Component: {11,}.
Its size = 1
```

FIG. 13.23. 无向图上从单个顶点出发的深度优先搜索算法(DFS)的计算结果。 图中展示了在Figure 13.21(a)中分别以顶点0、2、4、11为起始点得到的遍历结果。

（4）再次遍历图的顶点，按照顶点的连通单元状态来统计所有连通单元内的顶点及其数目。

Figure 13.24给使用上述算法时顶点的连通单元状态的标记结果。

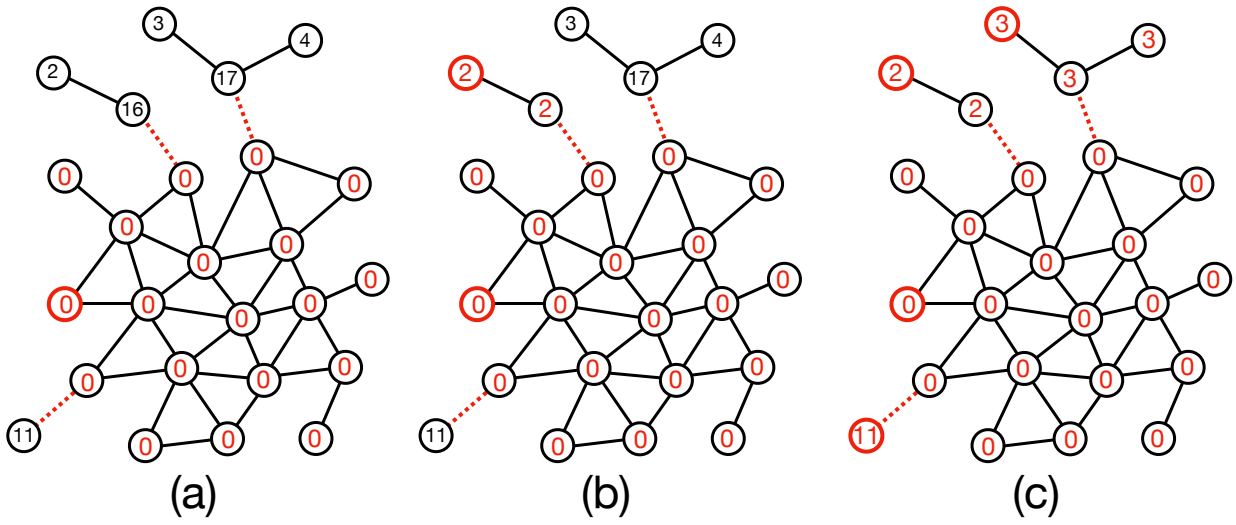


FIG. 13.24. 有着多个连通单元的无向图中的遍历算法。 (a-c)为遍历Figure 13.21中图的4个连通单元时，通过标记顶点的连通单元的编号来最终统计连通单元的大小和数目。 红色顶点表示遍历算法的起点，顶点内的红色数字为顶点所在的连通单元编号。

对于包含多个连通单元的图，我们利用遍历算法得到其包含顶点数最多的单元，也就是巨连通单元。 最大连通单元中顶点数所占图的顶点数的比例为一个重要的参数，能够用来表征图在宏观意义上的连通状态。 在第15章中，我们会考虑多种删除顶点或者边的方式，最终我们都会把图的巨连通单元作为图的宏观连接性的指标。

例子4: 含有多个连通单元的无向图上的遍历算法

问题：对于Figure 13.21中的无向图，使用BFS算法和DFS算法可以得到其所有连通单元。

代码可参考`ch13-graph.cc`中的函数`BFSOnComponents()`和`DFSOnComponents()`。 计算结果如Figure 13.25所示。

```
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
 Starting point: 0
 TraSeq:
 [ 0: 0->13][ 1: 0->14][ 2:13->12][ 3:13->22][ 4:13->23][ 5:13->24][ 6:14-> 1][ 7:14->15][ 8:22-> 9][ 9:22->10]
 [10:22->21][11:23->19][12:23->20][13:24->18][14:21-> 7][15:19-> 5][16:20-> 6][17: 7-> 8]
 Starting point: 2
 TraSeq:
 [ 0: 2->16]
 Starting point: 3
 TraSeq:
 [ 0: 3->17][ 1:17-> 4]
 Starting point: 11
 TraSeq:

 @输出所有的连通单元及其大小：
  Component: index =  0, size = 19
  Component: index =  2, size =  2
  Component: index =  3, size =  3
  Component: index = 11, size =  1
  After traversal, GC has a size = 19, or a fraction = 0.76
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
  TraSeq:
 [ 0: 0->13][ 1:13->12][ 2:12->22][ 3:22-> 9][ 4: 9->10][ 5: 9->21][ 6:21-> 7][ 7: 7-> 8][ 8: 7->20][ 9:20-> 6]
 [10:20->19][11:19-> 5][12: 5->18][13:18->24][14:24->14][15:14-> 1][16:14->15][17:24->23]
  TraSeq:
 [ 0: 2->16]
  TraSeq:
 [ 0: 3->17][ 1:17-> 4]
  TraSeq:

 @输出所有的连通单元及其大小：
  Component: index =  0, size = 19
  Component: index =  2, size =  2
  Component: index =  3, size =  3
  Component: index = 11, size =  1
  After traversal, GC has a size = 19, or a fraction = 0.76
```

FIG. 13.25. 含有多个连通单元的无向图上遍历算法的计算结果。 这里考虑Figure 13.21(a)的无向图。 这里依次给出来了BFS算法和DFS算法的计算结果。两者都能计算并输出图里的所有连通单元包含的顶点及其大小。

13.6. 顶点间的最短路径

这里我们考虑有权重无向图上顶点之间的距离和最短路径的问题。 最短路径问题可以有多个具体的场景，比如：（1）计算给定源点后图中其它顶点到它的距离和最短路径；（2）计算图中任意两个顶点之间的距离和最短路径。 Figure 13.26列举了这两类场景。第二个问题需要比第一个问题更多的距离信息，显然是更难的问题。直觉上，如果把图中所有顶点到其它顶点的距离和最短路径都计算出来，也就相应的到了任意两个顶点之间的距离和最短路。但在算法中，其实没有必要这么做。两个问题对应着不同的算法。在本小节，我们关注第一个场景。

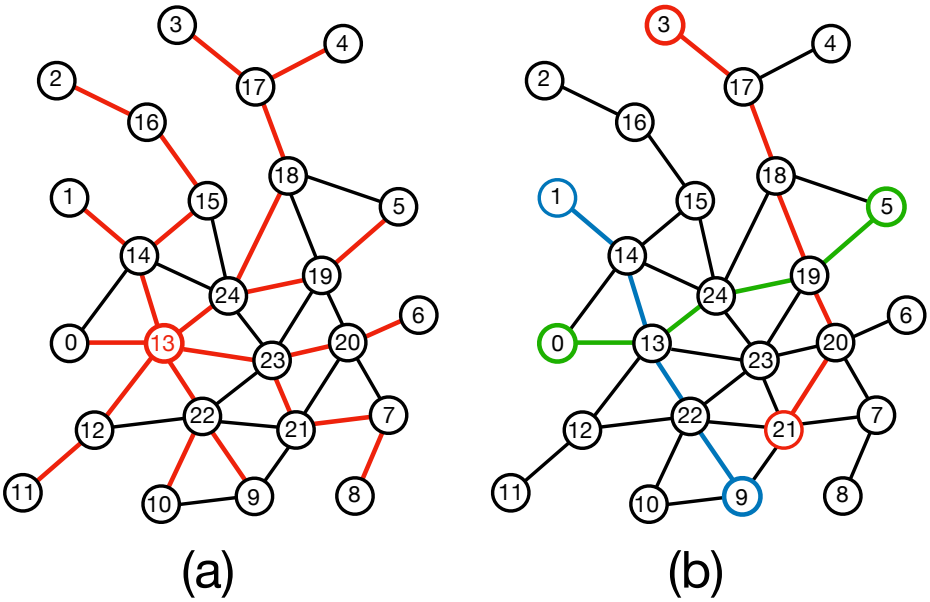


FIG. 13.26. 无向图上最短路径问题的两个场景。 (a)为包含了$N = 25$个顶点、$M = 39$个边的带权重的无向图。 图中标记了13作为源点，其它所有顶点到它的最短路径。其中最短路径用红色边表示。 (b)列举了三个顶点对之间的最短路径，分别为$(0, 5)$、$(1, 9)$、$(3, 21)$。三条路径用不同颜色做了标记。

在一般意义上，我们考虑边带权重的无向图上的最短距离的问题。此时有权重的无向图可以作为带权图的特例。 在有权图上，两个顶点之间的一条路径为头尾相连的边集，其对应的距离为边的权重之和。 两个顶点之间可能有多条路径。这些路径中的最小权重为最短距离，对应的路径也就是最短路径。 Figure 13.27展示了一个带权重无向图及其从源点出发的最短路径的问题。

那么在带权重的无向图上，我们如何使用算法来寻找这些租短路径及其距离？ 这

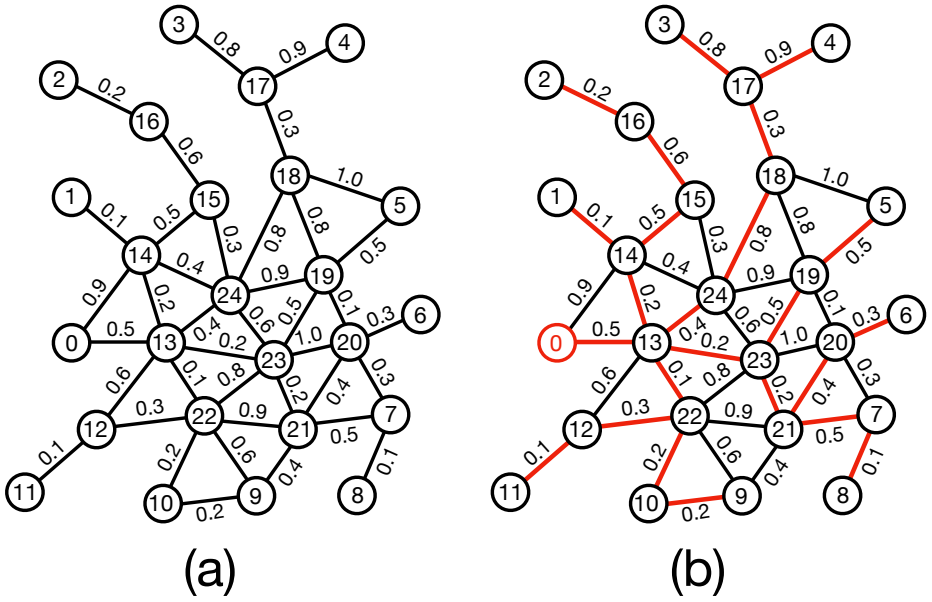


FIG. 13.27. 带权重无向图上的最短路径问题。 (a)为包含了$N = 25$个顶点、$M = 39$个边的带权重的无向图。 边附近的数字为对应边的权重。分布范围为$[0.1, 1.0]$。 (b)为从源点0出发，其它顶点到它的距离。标记了红色的边组成的路径表示了图中各个顶点到源点的最短路径。

里使用了Dijkstra算法。这里首先讨论下Dijstra算法的出发点。 如Figure 13.28所示，如果顶点0和8之间的最短路径确定了，为$(0, 13, 24, 18, 17, 4)$。那么这条路径当中的顶点13、24、18、17到0的边集也对应着这些顶点到源点的最短路径。 这个结论可以抽象为Figure 13.28(b)中的描述。如果SV为一条最短路径，而且U为路径中的某个顶点，那么SV的前缀SU也是一条最短路径。这个可以通过反证法来证明。 Dijkstra算法的基本思想在于Figure 13.28(c)：如果SU为一条最短路径，那么如何基于这个信息来构造更多顶点到源点S的距离。 这个对应于Figure 13.28(d)中的情形：如果给定源点0，13和22到它的最短路径已经确定，那么如何把非确定距离的顶点和已有的最短距离集合连接起来、以确定新的顶点到源点的距离？ Dijskstra算法的做法是：已经确定最短距离的顶点组成了一个集合，我们可以把和这个集合相邻的所有顶点的距离确定下来，然后选择其中的最短距离，并把这个顶点以及最短距离相应的边添加最短路径组成的集合。 Figure 13.28(d)中，基于顶点集$\{0, 13, 22\}$，我们确定了相邻的顶点$\{14, 24, 23, 21, 9, 10, 12\}$距离源点的距离分别为$\{0.7, 0.9, 0.7, 1.5, 1.2, 0.8, 0.9\}$。可知其中最短距离为0.7，对应边$(13, 14)$和$(13, 23)$。此时可以确定顶点14和23距离源点的距离为0.7。我们可以两个顶点加入相应的顶点集，继续执行上述步骤。

上述这个不断构造最短路径的解的步骤可以用一个通用的框架来表示。 图中每个顶

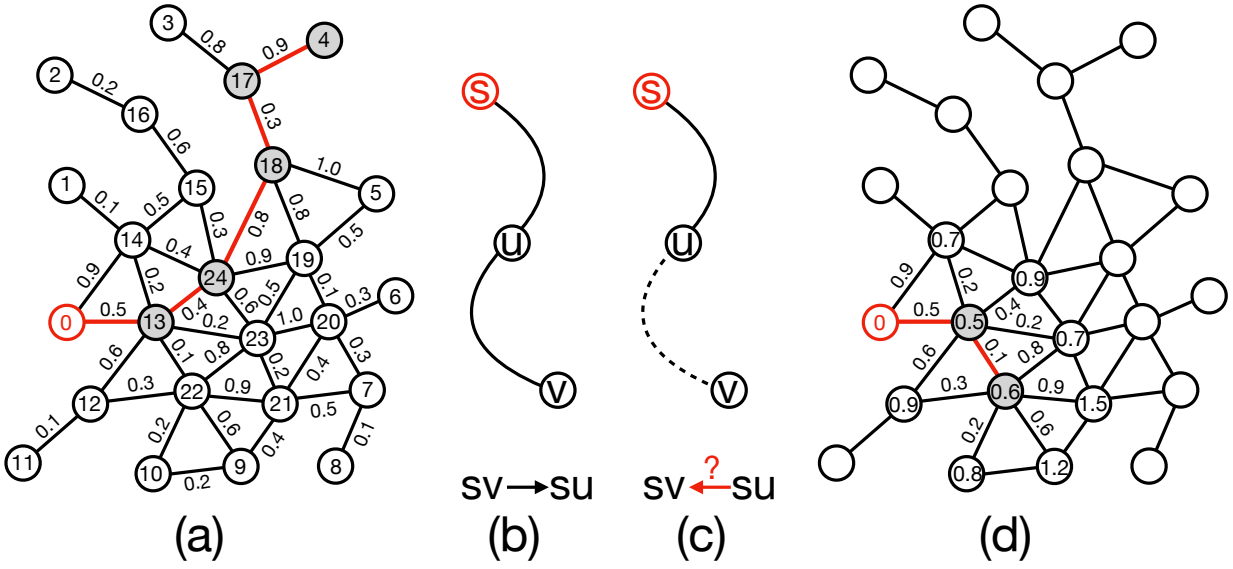


FIG. 13.28. Dijkstra算法的基本假设。 (a)为Figure 13.27中顶点0和7之间的一条路径。 (b)中S为图中某个源点，V为某个顶点，SV对应S到U的一条最短路径。 (c)中S为图中某个源点，U为某个顶点。SU对应S到U的一条最短路径。 (d)中为(a)的图，但省掉了部分顶点的编号和边的权重。图中顶点0到13、22之间的距离已经确定。顶点中的数字为此顶点到顶点0的距离。此时可以基于已有的最短路径和路径来计算它们的近邻到顶点0的最短路径和距离。

点i上定义了距离$p(i)$，表示相应顶点到源点的距离。 在上述算法里，解的构造是一个从初始解（即源点）开始的增长过程。 在增长过程的基本操作中，算法从已有解的所有未访问的最近邻顶点里选择一个顶点加入已有解。 上述算法过程的核心在于$p(i)$如何更新、并基于$p(i)$的值来选择顶点。 在初始状态下，$p(i) = \infty$，$i \in V$。从T_k构造T_{k+1}的步骤中，需要从∂T_k中选择顶点加入T_k中。 对于未访问的顶点j来说，其距离$p(j)$一般依赖与和它相连的已访问的最近邻顶点$t(t \in \partial j)$的优先级数$p(t)$、以及对应跨区域边的权重$w(t, j)$。 在Dijkstra算法里，顶点距离$p(j)$的更新规则为：

$$p(j) = \min_{t \in \partial j} \{p(j), p(t) + w(t, j)\}. \tag{13.31}$$

在算法中，我们每次选择所有距离中最小的顶点j，并把它放入已确定最短路径的顶点集中。 上述流程如Figure 13.29所示。

容易得知，上述流程可以用在很多逐个构造解的问题里。此时顶点参数$p(i)$的含义视具体问题而定。 上述框架也就是优先级优先搜索算法(Priority-first Search, PFS)。Dijkstra算法可以视为一个具体的例子。

在Figure 13.30里，我们给出了一个无向图上使用Dijkstra算法的完整流程。 Figure

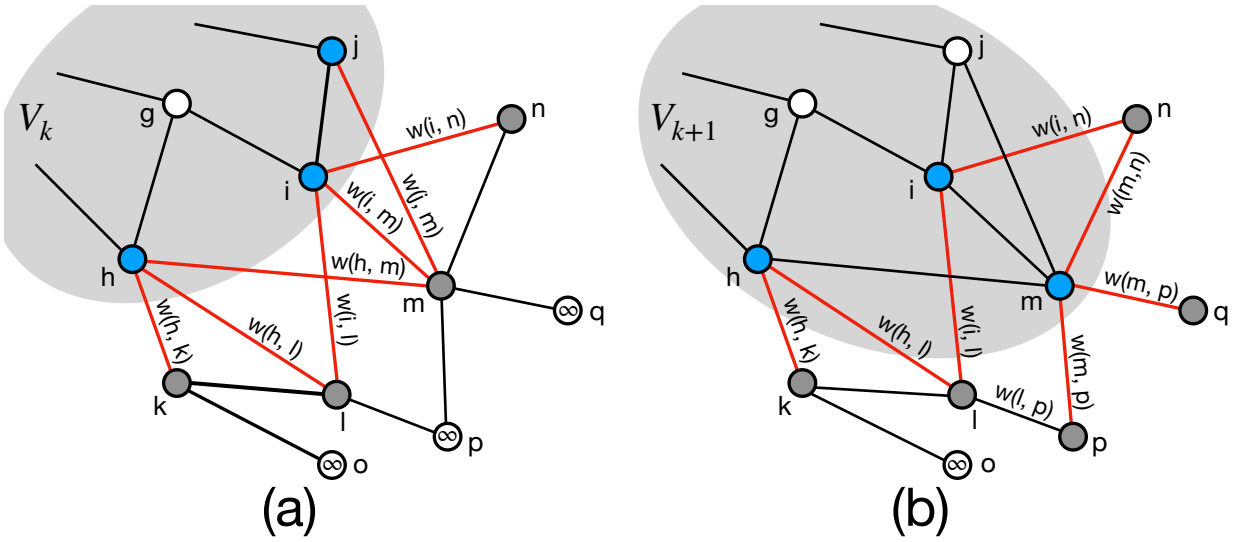


FIG. 13.29. Dijkstra算法里顶点距离参数的更新过程。 灰色色块覆盖的区域为Dijkstra算法已经访问过的子图V_k，其外为算法还没访问的子图$\hat{V_k}$。两部分字符分别为用蓝色和深灰色圆圈表示，其边用红色实线表示。 (a)中跨越两个区域的有7条边。此时需要更新顶点k、l、m、n四个顶点的距离参数。 (b)中，假设顶点m因为其距离最小，被加入了访问子图，得到新的访问子图V_{k+1}。此时需要继续更新顶点k、l、n、p、q的距离。

13.30(a)中，所有顶点的距离的初始值设置为无穷大(∞)。 Figure 13.30(b)中，源点为顶点3。此时可以更新其最近邻($\{2, 4, 8, 9\}$)的距离值。 Figure 13.30(c)中，未确定距离顶点中的最小距离值对应边$(3, 9)$。此时顶点9的距离为2，相应可以更新9的最近邻5和7的距离。 Figure 13.30(d)中，未确定距离顶点中的最小距离值对应边$(3, 8)$。此时顶点8的距离为4，相应可以更新4的最近邻的距离。 这些过程可以一直进行下去，直到Figure 13.30(k)。此时所有顶点到源点3的最短路径和距离都能确定下来。

注意到Figure 13.30(h)中，顶点1到源点3的两条路径$(1, 2, 3)$和$(1, 7, 9, 3)$有着同样的距离8。此时我们可以选择路径$(1, 3)$或者$(1, 7)$添加到最短路径的边集里。 我们可以最短路径问题的构型定义为其包含的边集。那么这个问题除了Figure 13.30(k)中的构型，还有第二个构型，即Figure 13.30(l)所示的构型。

由上述结果可知，最短路径问题中的每个顶点到源点的距离的值是唯一的，但最短路径的构型可能不是唯一的。如果图中某个顶点到源点有着多条最短路径具有同样的权重，则给定源点后，其最短路径问题最终也就具有多个构型。

例子5: 图上源点到其它顶点的最短路径

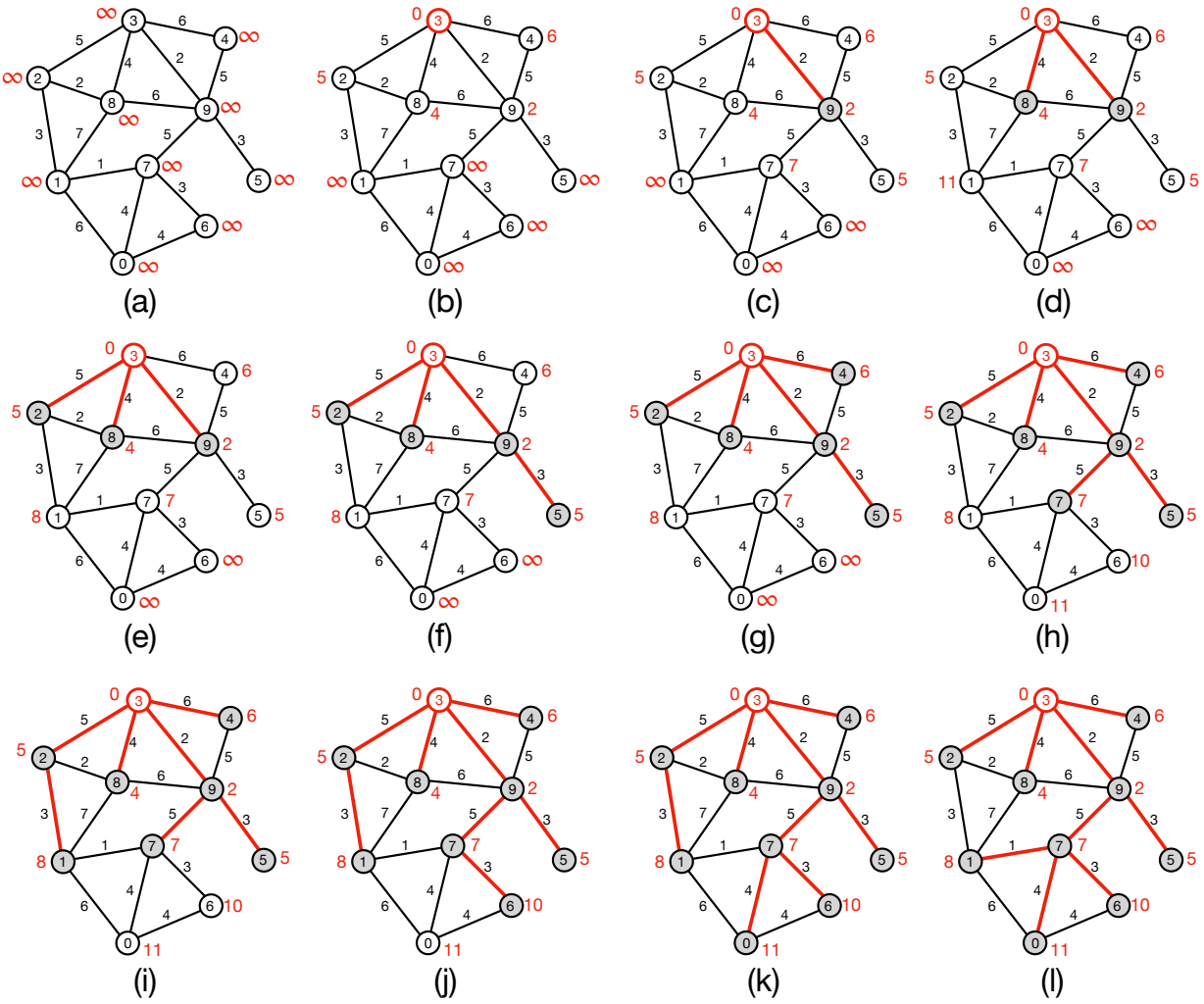


FIG. 13.30. 无向图上最短路径问题的Dijkstra算法的流程。 (a)为一个带权无向图，其顶点数$N = 10$、边数$M = 16$。边附近的数字为其权重。 (b-k)给出了以顶点3为源点，不断更新其它顶点到源点的距离。 (k)为此图上最短路径问题的一个构型。 (l)为此图上最短路径问题的第二个构型。 顶点附近的数字为其到达源点的距离，其初始值为无穷大(∞)。 标记了红色的边属于最短路径的边集。 带深色色块的顶点表示算法中距离参数被确定的顶点。

问题：给定有权图，使用Dijkstra算法计算这些顶点到给定顶点到其它顶点的最短路径、并输出。

代码可参考`ch13-graph.cc`中函数`Path_Dijkstra()`。

计算结果如Figure 13.31所示。 容易得知，选择不同的顶点作为源点时，其它顶点到源点的距离显然各不相同。但宏观上我们可以用一个平均距离，即$N-1$个顶点到源点的距离之和的平均值，来刻画其它顶点到源点的距离。相应地，这个距离可以

视为源点在图中的中心性(centrality)的一个指标：如果平均距离越小，意味着源点在图中的位置更加靠“中间”；如果平均距离越大，意味着源点在图中靠近“边缘”。Figure 13.31中的三个顶点里，容易得知，顶点4的度为1，为图中最小的度，此时它对应的平均距离是最大的；顶点24的度为6，为图中最大的度，此时对应的平均距离最小。这里的结果表明度的大小和顶点的平均距离有一些相关性。但真实情况可能比这个复杂的多：除了度，有很多结构特征会影响源点到其它顶点的平均距离。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@With Dijkstra's algorithm, for source vertex [4] with a degree 1:
 [ 0] dist = 2.9; [ 1] dist = 2.5; [ 2] dist = 3.1; [ 3] dist = 1.7;
 [ 4] dist =   0; [ 5] dist = 2.2; [ 6] dist = 2.4; [ 7] dist = 2.4;
 [ 8] dist = 2.5; [ 9] dist = 2.9; [10] dist = 2.7; [11] dist = 2.9;
 [12] dist = 2.8; [13] dist = 2.4; [14] dist = 2.4; [15] dist = 2.3;
 [16] dist = 2.9; [17] dist = 0.9; [18] dist = 1.2; [19] dist =   2;
 [20] dist = 2.1; [21] dist = 2.5; [22] dist = 2.5; [23] dist = 2.5;
 [24] dist =   2;
 Average dist: 2.3625
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@With Dijkstra's algorithm, for source vertex [0] with a degree 2:
 [ 0] dist =   0; [ 1] dist = 0.8; [ 2] dist =   2; [ 3] dist = 2.8;
 [ 4] dist = 2.9; [ 5] dist = 1.7; [ 6] dist = 1.6; [ 7] dist = 1.4;
 [ 8] dist = 1.5; [ 9] dist =   1; [10] dist = 0.8; [11] dist =   1;
 [12] dist = 0.9; [13] dist = 0.5; [14] dist = 0.7; [15] dist = 1.2;
 [16] dist = 1.8; [17] dist =   2; [18] dist = 1.7; [19] dist = 1.2;
 [20] dist = 1.3; [21] dist = 0.9; [22] dist = 0.6; [23] dist = 0.7;
 [24] dist = 0.9;
 Average dist: 1.32917
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
@With Dijkstra's algorithm, for source vertex [24] with a degree 6:
 [ 0] dist = 0.9; [ 1] dist = 0.5; [ 2] dist = 1.1; [ 3] dist = 1.9;
 [ 4] dist =   2; [ 5] dist = 1.4; [ 6] dist = 1.3; [ 7] dist = 1.3;
 [ 8] dist = 1.4; [ 9] dist = 0.9; [10] dist = 0.7; [11] dist = 0.9;
 [12] dist = 0.8; [13] dist = 0.4; [14] dist = 0.4; [15] dist = 0.3;
 [16] dist = 0.9; [17] dist = 1.1; [18] dist = 0.8; [19] dist = 0.9;
 [20] dist =   1; [21] dist = 0.8; [22] dist = 0.5; [23] dist = 0.6;
 [24] dist =   0;
 Average dist: 0.95
```

FIG. 13.31. 有权无向图上Dijkstra算法的计算结果。 对应的带权重无向图为Figure 13.27(a)。 这里输出了分别选择三个顶点（4, 0, 24）作为源点，计算其它所有顶点到源点的距离。 最终结果可以和Figure 13.27(b)中的最短路径做对比。

13.7. 图的最小生成树

在一个带权无向图上，我们考虑定义在其上的最小生成树(Minimum Spanning Tree, MST)问题。 生成树(Spanning Tree)对应一个边的集合。这些边构成了一棵树，并且覆盖了图中所有的顶点。 对于顶点数为N、边数为M的图来说，其生成树对应的边数显然

为$N-1$。本质上其构型具有多种可能性，相应地其边之上的权重之和会很不同。 而最小生成树考虑的是这些树中权重之和最小时对应的构型。

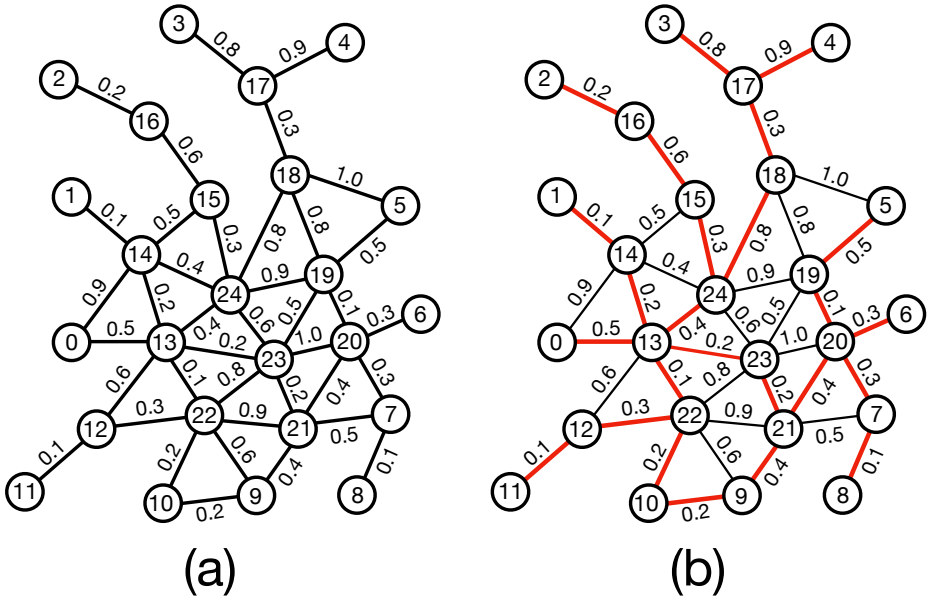


FIG. 13.32. 有权无向图上的最小生成树。 (a)中为包含了顶点数$N=25$、边数$N=39$的带权无向图。 边附近的数字为对应边的权重。 (b)中标记了红色的边标记了一个最小生成树。

这个问题本质上是一个多项式问题（P问题）。可以使用了Prim算法来进行求解。 此算法基于的基本结论是：对于图的一个割(cut)，其跨边中权重最小的边，属于此图的最小生成树。 这个结论可以使用反证法来证明。 假设图为$G=\{V,E\}$，顶点集中的一个割为$\{S,V\backslash S\}$。我们把跨边中权重最小的一条边表示为(u,v)。此时我们假设图的最小生成树M里并不存在这条边。如果我们把(u,v)加入这个最小生成树，显然会出现环。这个环包括了边(u,v)，相应地也包含了另外一条跨边。假设这条跨边为(s,t)，而且这条边的权重$w_{(s,t)}$显然大于(u,v)的权重$w_{(u,v)}$。如果我们从最小生成树和(u,v)中去掉(s,t)，那么此时对应的边集也是一个最小生成树，而且具有比M的权重小$w_{(s,t)}-w_{(u,v)}$。此时和假设相矛盾。由此证明上述的基本结论是正确的。

Prim算法的实现框架也采取了优先级优先搜索的框架。 我们首先随机选择一个顶点作为算法的起点。此顶点作为已访问顶点的集合。 之后我们选择已访问顶点和未访问顶点之间连边的权重最小的连边，并把相应的未访问顶点放入这个集合。 此过程一直进行下去，一直到所有顶点都被访问。 在顶点i上我们可以设置参数$p(i)$。容易得知，算法中未

访问顶点j的优先级系数的更新规则为：

$$p(j) = \min_{t \in \partial j}\{p(j), w(t,j)\}. \tag{13.32}$$

Figure 13.33展示了无向图上Prim算法的完整流程。 Figure 13.33(a)中，顶点的权重参数的初始值均为无穷大(∞)。 Figure 13.33(b)中，选择顶点9作为初始顶点。此时它的最近邻顶点$\{3,4,5,7,8\}$的顶点权重相应更新。 Figure 13.33(c)中，顶点中权重的最小值为2，对应边$(3,9)$加入最小生成树。此时继续更新顶点3的最近邻的权重。 Figure 13.33(d)中，顶点中权重的最小值为3，对应边$\{9,5\}$加入最小生成树。因为顶点5只有9这一个邻居，此时图中无需因此更新顶点的权重。 Figure 13.33(e)中，顶点中权重的最小值为4，对应边$\{3,8\}$加入最小生成树。此时继续更新顶点8的最近邻的权重。 这个过程一直进行下去，直到所有顶点都加入了最小生成树。最终得到Figure 13.33(k)中最小生成树的边集。

在Figure 13.33(i)中，我们能够发现边$(7,0)$和$(6,0)$有着同样的权重4。 在最终的最小生成树中，起始有两个构型Figure 13.33(k)和(l)。两者的差别在于前者包含$(7,0)$、后者包含$(6,0)$。 在最小生成树问题里，生成树对应的边的权重之和作为最小值时唯一的，但对应的边的构型不一定是唯一的。 如果多条边的权重相等，则可能存在多个最小生成树的构型。

例子6: 寻找给定图上的最小生成树

问题：给定有权图，使用Prim算法寻找此图上的最小生成树，并输出相应的边集和总权重。

代码可参考`ch13-graph.cc`中函数`MST_Prim()`。

13.8. 无向图上的移叶过程

对于无向图来说，度为1的顶点成为叶子节点(leaf node)。大家可以参考无向图中叶子节点的定义（仅有父节点、无左右子节点）。如果移除图中的所有叶子节点、包括孤立节点（度为0），这样可以得到一个紧凑一些的结构。此时剩余子图里所有顶点的度$\geqslant 2$。这个删除叶子节点和孤立节点（以及和它相连的边）的过程就是移叶过程(Leaf Removal Procedure)。 注意移叶过程是一个迭代过程(iterative process)。移叶过程本身是一个同时减少顶点和边的过程。图中某些顶点本身在最开始可能不是叶子节点。但随着移叶过程的执行，这些顶点最终也变成了叶子节点。如Figure 13.35(b)中的顶点16和17。它们在初始

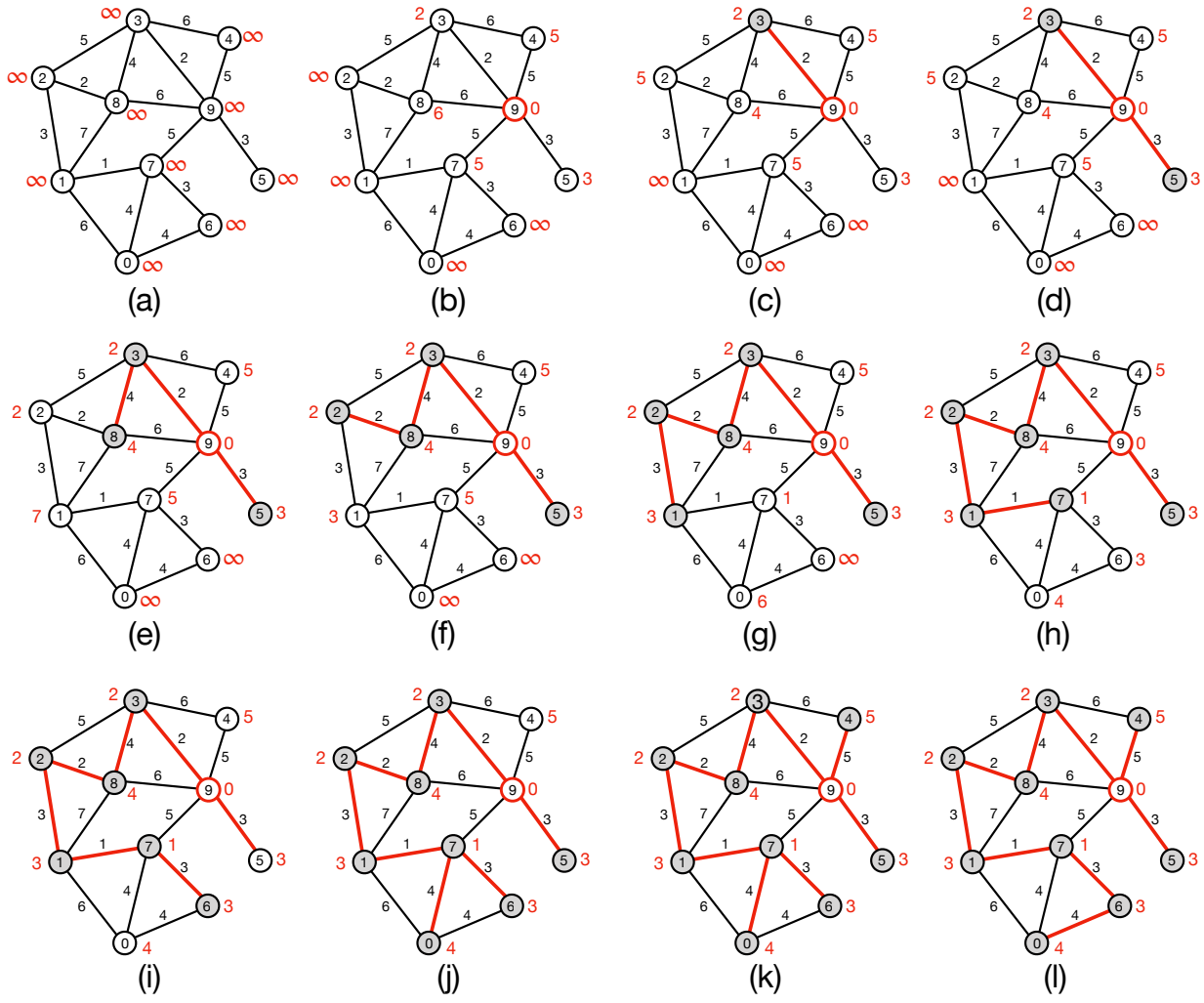


FIG. 13.33. 无向图上Prim算法的流程。 (a)为最初的无向图，其中顶点数$N = 10$、边数$M = 16$。边附近的数字为对应的权重。 (b-k)给出了Prim算法中，首先把顶点9作为初始顶点，不断更新其它顶点的权重信息的过程。 (k)为最小生成树的一个构型。 (l)为最小生成树的第二个构型。 顶点附近的数字为其对应的权重，其初始值为无穷大(∞)。标记了红色的边属于最小生成树。 带深色色块的顶点表示算法中距离参数被确定的顶点。

图中的度分别为2和3。但当它的最近邻的叶子节点被删除后，它们最终也变成了一个叶子节点。

算法过程中我们使用移除顶点的方式来实现移叶算法的过程。但在代码层面，我们不用真正地从数据结构里删除“需要移除的顶点”，而只是在原来的图解里标记它的状态为“被移除状态”即可。在代码里，我们首先找到并存储叶子节点，然后迭代地删除它们。我们用队列(queue)来存储叶子节点的序号：每次移除一个顶点，我们就把它从队列里删

```
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
 @With Prim's algorithm, we have a MST:
  [ 0] ( 8, 7); [ 1] (10, 9); [ 2] (12,11); [ 3] (13, 0); [ 4] (14, 1); [ 5] (14,13);
  [ 6] (16, 2); [ 7] (16,15); [ 8] (17, 3); [ 9] (17, 4); [10] (18,17); [11] (19, 5);
  [12] (20, 6); [13] (20, 7); [14] (20,19); [15] (21,20); [16] (22,10); [17] (22,12);
  [18] (22,13); [19] (23,13); [20] (23,21); [21] (24,13); [22] (24,15); [23] (24,18);
```

FIG. 13.34. 有权无向图上Prim算法的计算结果。 这里按顺序输出了MST里所有的边。 其计算结果可以和Figure 13.32(b)中的结果做对比。

除；每次在移除顶点时新产生了叶子节点，我们就把它压入队列里。 注意，图中的孤立顶点也被存储在队列里，并最终被删除。

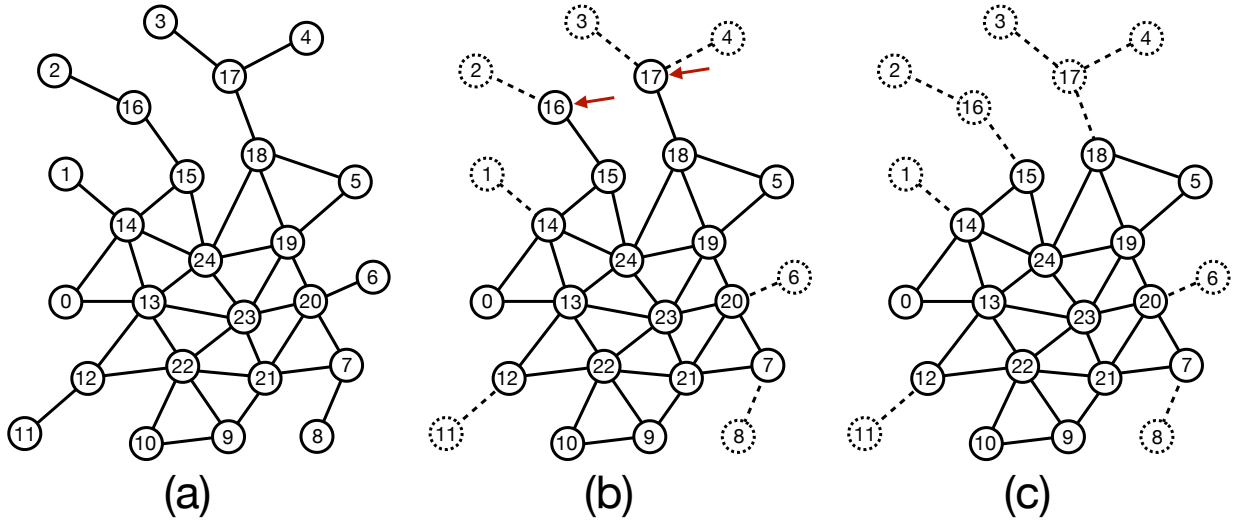


FIG. 13.35. 无向图上的移叶过程。 (a)为包含了顶点数$N = 25$、边数$M = 39$的无向图。 (b)为删除初始图中度为1的顶点的操作。用点线标识的边和顶点表示在移叶过程中被删除的边和顶点。(c)为在(b)中子图中继续删除度为1的顶点，即$\{16, 17\}$。 最终实线标记的顶点和边为使用移叶过程所得到的子图。

例子7: 无向图里的移叶算法

问题：对于Figure 13.35(a)，使用移叶算法以得到最终的残留子图。

代码可参考`ch13-graph.cc`中的函数`LRP()`。

代码的结果可见Figure 13.36。

```
ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
ZhaoJH@MacBook-Pro 3 code class % ./ob
 Residual nodes: { 0, 5, 7, 9,10,12,13,14,15,18,19,20,21,22,23,24,}.
 Node size = 16, or fraction = 0.64
```

FIG. 13.36. 无向图上移叶过程的计算结果。 其结果可以和Figure 13.35(b)的图示结果做对比。

13.9. 有向图上的强连通单元

在无向图中，我们把连通单元定义为彼此之间都可以通过边到达的所有顶点构成的子图。 在无向图，顶点之间可以通过没有方向（或者双向）的边到达。但在有向图里，从一个顶点i能直接到其最近邻顶点j必须因为它们之间存在有向边$\langle i,j\rangle$。边的方向性导致顶点能够访问其最近邻是也必须依据这个有向性，最终导致在“连通性”的定义上，有向图和无向图会有些差别。 在有向图中，如果存在一个子图，其内部任意顶点之间都可以通过有向边构成的路径到达彼此，那么此结构就是有向图的强连通单元(Strongly Connected Component, SCC)。这个连通性的定义类似无向图里的连通单元，但约束更强。 另一方面，如果只关注有向图里顶点之间是否有边（无论其方向），此时对应的图也就是一个忽略边方向的无向图，此时对应的图结构里的连通单元叫做有向图里的弱连通单元(Weakly Connected Component, WCC)。

这里我们讨论下，对于任意一个有向图，如何得到其SCC。 假设考虑的图为$G=\{V,E\}$，其中V和E分别为图的顶点集和边集。 G对应的SCC为$G_s=\{V_s,E_s\}$，其中V_s和E_s分别为图的SCC对应的顶点集和边集。 可知$V_s\subseteq V$、$E_s\subseteq E$。 对于V_s内的任意顶点$i\in V_s$，可知i在V_s里必有入邻居顶点、也必有出邻居顶点：前者保证V_s内的其它顶点能够到达i，后者保证i能够到达V_s的其它顶点。 这意味着i需要同时满足$k_i^{\rm in}\geqslant 1$、$k_i^{\rm ou}\geqslant 1$。 需要强调的是，“一个顶点i同时满足$k_i^{\rm in}\geqslant 1$、$k_i^{\rm ou}\geqslant 1$”只是“它属于SCC”的必要条件，而非充分条件。比如Figure 13.37(a)中顶点17满足$k^{\rm in}=2$、$k^{\rm ou}=1$，但它显然不属于图的SCC。 那么如何得到图中满足上述对度的要求的所有顶点？其实只需要不断删除$k_i^{\rm in}=0$或者$k_i^{\rm ou}=0$的顶点即可。 我们的算法如下：对于任意有向图，如果某个顶点i满足$k_i^{\rm in}=0$或者$k_i^{\rm ou}=0$，则我们删除i及其相邻的所有边；这个过程迭代地进行下去，直到不存在$k_i^{\rm in}=0$或者$k_i^{\rm ou}=0$的顶点；最终剩下的子图就是SCC。

需要提到的是，如同我们在移叶算法里做的一样，在算法上执行删除顶点和边的操作时，我们无需在数据结构里真的去移除相应的数据成员，而只需标记相应顶点和边的状态为“被移除”即可。 Figure 13.37给出了一个在有向图中寻找SCC的具体例子。

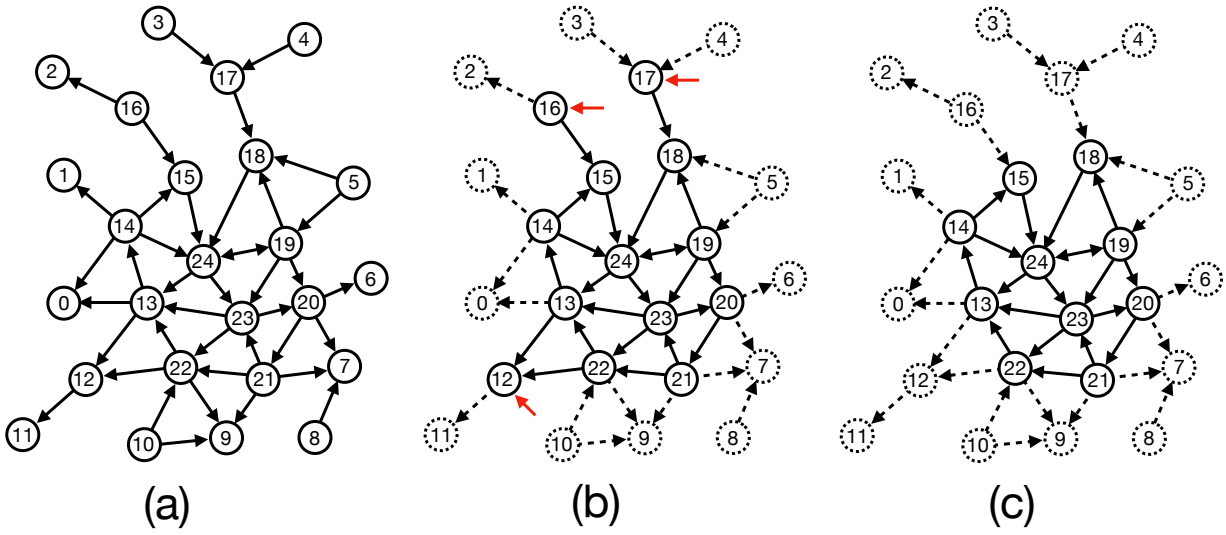


FIG. 13.37. 有向图的强连通单元。 (a)为包含了顶点数$N = 25$、边数$M = 40$的有向图。注意顶点19和24之间的边为双向边，在边数统计中算作2条单向边。 (b)为从初始图中删去入度为0、或者出度为0的顶点。被删除的边和顶点用点线和划线标识。 (c)为在(b)中的图中继续删除入度为0、或者出度为0的顶点，即$\{12, 16, 17\}$。 最终实线标记的顶点和边为原图的强连通单元。

例子8: 有向图里的强连通单元

问题：对于Figure 13.37(a)，使用算法以得到其强连通单元。

代码可参考`ch13-graph.cc`中的函数`SCC()`。

代码的结果可见Figure 13.38。这里可以和Figure 13.37(b)的图示结果做对比。

```
[ZhaoJH@MacBook-Pro 3 code class % g++ -o ob ch13-graph.cc
[ZhaoJH@MacBook-Pro 3 code class % ./ob
 Residual nodes: {13,14,15,18,19,20,21,22,23,24,}.
 Node size = 10, or fraction = 0.4_
```

FIG. 13.38. 有向图的强连通单元的计算结果。 这里的结果可以和Figure 13.37里的结果做对比。

习题

13.1 图的结构性质

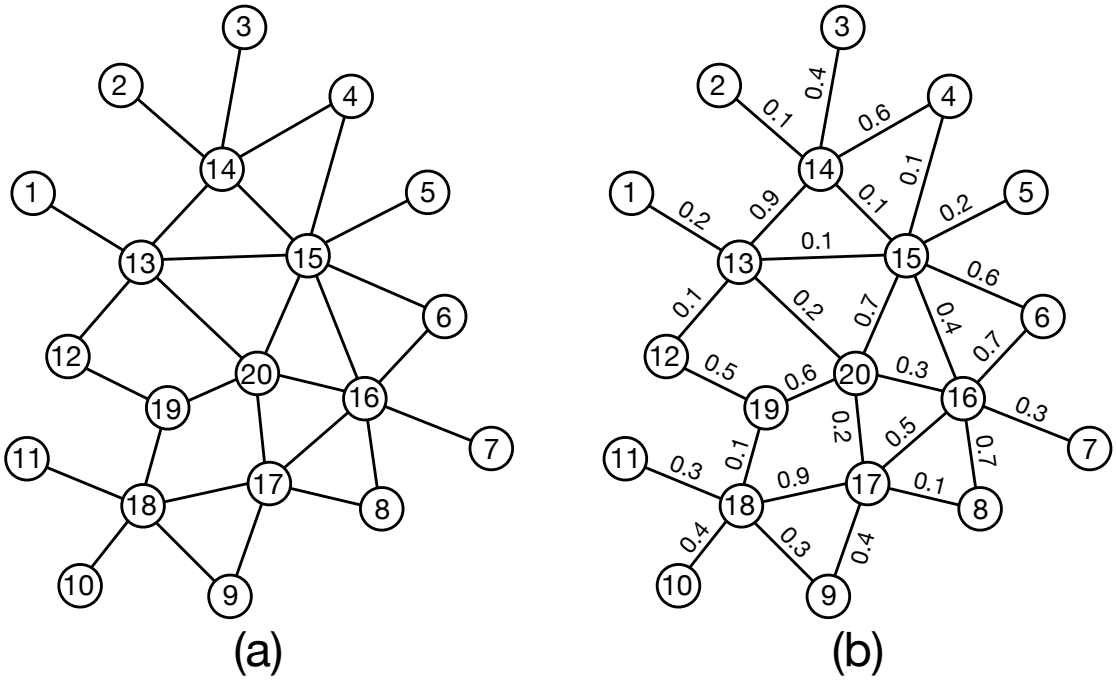


FIG. 13.39. 无向图的两个例子。 (a)无权重的无向图，其中顶点数$N = 20$、边数$M = 29$。顶点内的数字为其编号。 (b)为基于(a)的带权重的无向图。边附近的数为对应边的权重。

对于Figure 13.39(a)中的无向图，请回答以下问题：

（1）图中顶点的最小和最大的度为多少？请写出对应的顶点编号。

（2）请写出各个度对应的顶点数，并手画出大致的度分布。

（3）请写出图中顶点8和13之间的所有最短路径。

（4）请写出此图的直径，并写出它对应的顶点间路径。

（5）请写出此图的半径，并写出它对应的顶点间路径。

注：此题无需写代码。

13.2 图的遍历算法

对于Figure 13.39(a)，把顶点15作为源点，可以对图使用遍历算法。

请直接画出来广度优先搜索算法和深度优先搜索算法对应的顶点序列。注意遍历算法对应的顶点序列有多种，画出来一种即可。

13.3 图上顶点间的最短路径的算法示意图

对于Figure 13.39(b)，可以把顶点15作为求解顶点间最短路径的源点。

请直接手画出来Dijkstra算法执行过程中顶点上最短路径值的更新过程，以及算法最后得到的源点到其它所有顶点的最短路径。

13.4 图的最小生成树的算法示意图

对于Figure 13.39(b)，请手画出来Prim算法执行过程中边的选择过程，以及最终得到的生成树。

13.5 图上顶点间的最短路径和最小生成树

这里考虑的图结构为Karate club network（空手道俱乐网络）。其构型如Figure 13.12所示。此图为无权重的无向图，顶点数$N = 34$、边数$M = 78$。相应地边数据集为课堂演示代码里文件夹`ch13-data-source/karate-club-n34m78.txt`。

我们将基于此网络来实现一些图算法。请完成以下问题:

（1）此网络本身为一个无权重的无向图。这里首先我们在其边上定义权重来把它变成一个带权重的网络。这里考虑一个简单的情况：边的权重反比于两个端点的度。对于顶点i和j之间的边(i, j)，其权重为$w_{ij} = 1/k_i k_j$，其中k_i和k_j分别为i和j的度。

（2）请寻找图中度最大和最小的顶点。注意此类顶点可能有多个。然后把这些顶点分别作为源点，使用Dijkstra算法计算它们和其它所有顶点之间的距离。最后请逐行输出源点和目标顶点的顶点对、以及它们之间的距离。

（3）使用Prim算法计算此图的最小生成树。请输出此最小生成树包含的所有边，并计算这些边的权重之和、以及它占边的总权重的比例。

14.

随机图(random graphs)指的是按照某个添加顶点和边的规则来生成的图。随机图没有复杂的介观结构(mesoscopic structure)，使得我们可以使用很少的拓扑参数来描述它们，也因此它们能够用来作为测试不同算法的基准(benchmark)。同时，因为随机图的结构足够简单，它们的一些宏观性质具有解析的形式，这使得我们更容易理解这类图。生成随机图的模型就叫随机图模型(random graph models)。

首先，我们为什么需要随机图?

真实世界网络在不同尺度上往往有丰富的、复杂的结构。如果在这样的图结构上比较某个计算问题的不同方法或算法，往往很难分辨出来得到的不同结果的差别是来源于方法本身、还是方法所使用的图结构。为了尽可能消除图结构本身对计算方法的影响，我们一般考虑内部结构尽可能少的图。此外，我们也希望能够避免使用具体的图来比较，而是可以从图的一个系综（使用少量参数生成的所有图）里随机选择图例来比较。这使得我们在选择图结构进行计算时有很高的自由度。对于随机图来说，上述两个要求都能满足。对于不同的方法或算法，在利用图参数随机生成的、顶点数比较大的一个图上进行测试，可以直接来比较它们的效果。

14.1. 随机正规图(Random regular graph)

对于图来说，正规(regular)表示图中顶点的度是相等的。在正规随机图(regular random graphs)里，图中每个格点的度均为$k_0(\in \{2,3,\cdots,+\infty\})$，但点与点之间的连接是随机构造的。

这种图可以视为格点系统(lattices)的推广。格点系统一般定义在有限维空间里，比如二维、三维空间，系统中的格点和空间上最近的格点相邻。对于格点邻居数都为k的格点系统和正规随机图来说，两者的度分布是一样的，但本质上前者结构内嵌于一个有限维系统，而后者可视为处在无限维的系统里，两者在宏观结构特征、动力学结果上会很不同。

和RR随机图关系密切的一个模型是格点系统(lattice system)。考虑由一个多条平行、且互相垂直的直线所构成的平面。直线相交的点可以视为一个顶点，顶点之间的直线段可以视为连边。这样构造的图是一个无穷大的格点系统。这类系统的每个点的度为4，存在很多长度很短的圈。其结构显然完全不同于$k_0 = 4$的RR随机图。一个例子如Figure 14.1所示。

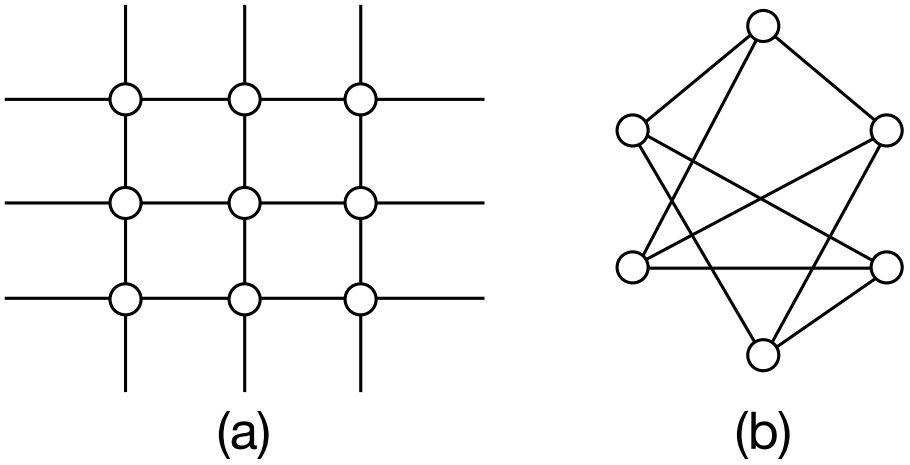


FIG. 14.1. 正规随机图和格点系统。 (a)顶点数$N = 6$，度$k_0 = 3$的正规随机图。 (b)顶点数$N = 12$、度$k_0 = 4$的二维平面上的格点系统。此时格点系统满足周期性边界条件，即最左和最右的顶点依次相连接、最上和最下的顶点依次相连接。

1. 生成过程

生成格点。

构造连边。

2. 度分布的推导

对于度为$k_0(\in \{2, 3, \cdots, +\infty\})$的正规随机图，其度分布和余度分布分别为

$$P(k) = \delta(k, k_0), \tag{14.1}$$

$$Q(k) = \delta(k, k_0 - 1). \tag{14.2}$$

在一些具体问题里，我们可以对图进行稀疏化(dilution)以得到不均匀的度分布。一个简单的稀疏化形式是从图中随机移除比例为$1 - \rho \in [0, 1]$的连边。此时残留图的平均度为$c = k_0\rho$，其度分布和余度分布分别为

$$P(k) = \binom{k_0}{k}\rho^k(1-\rho)^{k_0-k}, 0 \leqslant k \leqslant k_0, \tag{14.3}$$

$$Q(k) = \binom{k_0-1}{k-1}\rho^{k-1}(1-\rho)^{k_0-k}, 1 \leqslant k \leqslant k_0. \tag{14.4}$$

对于求和公式Eqs.(14.11)-(14.12)，我们有：

$$\sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s} = \sum_{k=s}^{k_0}\binom{k_0}{k}\rho^k(1-\rho)^{k_0-k}\binom{k}{s}x^{k-s}, \tag{14.5}$$

$$\sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}x^{k-1-s} = \sum_{k=s+1}^{k_0}\binom{k_0-1}{k-1}\rho^{k-1}(1-\rho)^{k_0-k}\binom{k-1}{s}x^{k-1-s}. \tag{14.6}$$

上式中，s的取值范围为$0 \leqslant s \leqslant k_0$和$0 \leqslant s \leqslant k_0 - 1$。

14.2. ER随机图(Erdös-Rényi random graph)

Erdös-Rényi随机图（ Erdös-Rényi random graph, ER random graph） [33, 34]在早前的图论研究中被视为标准的随机图模型。

1. 生成过程

构造ER随机图的过程可以通过两种模型来实现：(N,p)过程和(N,M)过程，其中N为图的顶点数、p为某个概率、M连边数。两个模型都是从只有N个顶点、没有连边的空图$G=\{V,\varnothing\}$开始，不断在图中选取两个顶点、并构造连边。如果选取的两个节点之间没有连边，则构造一条连边；如果它们之间有连边，则重新随机选择两个节点并检查它们之间有没连边。最终我们得到图$G=\{V,E\}$。但两个过程不同之处在于构造连边时选择顶点的方式。

- (N,p)过程：对于顶点集中的任意顶点i，其它任意一个顶点和它相连的概率为概率p。对于每个节点，我们可以如此生成连边。通过这种概率方式生成的图$G=\{V,E\}$的连边数为$|E|=pN(N-1)$，对应平均度$c=2|E|/N=2p(N-1)$。
- (N,M)过程：此过程中，我们逐步在图中加入M条连边。每次加入一条连边时，首先随机选择两个节点。如此生成的图$G=\{V,E\}$的平均度为$c=2M/N$。

可知，当N很大、$pN=\Theta(1)$时，上述两个过程是等价的。

在使用代码来从模拟过程上构造ER随机图时，我们选择更直观的(N,M)过程。 生成ER随机图时，我们使用动态数据结构来存储不断增加连边的图结构。 假如我们需要生成一幅ER随机图$G=\{V,E\}$，其顶点数$|V|=N$，连边数$|E|=M$。 初始的Vertices大小为N，此时对应一个只有N个顶点、没有连边的图。之后我们逐步加入M条连边。 每次加入一条连边时，首先利用随机数生成器随机选择两个顶点$(v1,v2)$。然后检查两个顶点是否相邻。此时有两个结果：

▷ 如果两个顶点相邻，则随机选择一个顶点作为新的$v2$，然后再次测试新的$(v1, v2)$是否相邻。

▷ 如果两个顶点不相邻，则把$v2$放入$v1$的最近邻序列、把$v1$放入$v2$的最近邻序列，同时把$v1$和$v2$的度增加1。

按上述基本过程，逐条把M条连边加入图里。

按照上述过程我们就可以生成一个具体的ER图的图例。 有了图例之后，我们可以使用它来测试各种算法。

2. 度分布的推导

下面我们来做一件简单的图性质的统计，也就是图的经验度分布。此时我们遍历一遍图结构，统计其每个度$k(\geqslant 0)$对应的顶点数$N(k)$，即可得到这幅图的经验度分布$P_{\mathrm{em}}(k) = N(k)/N$。

在上述(N, p)过程里，当N很大时，我们可以解析地计算得到ER随机图的度分布。 按照模型的描述，可知图中某个顶点i的度为k的概率为

$$P(k) = C_{N-1}^{k} p^{k} (1-p)^{N-1-k}, k \geqslant 0. \tag{14.7}$$

当$N \gg 1$时，$P(k)$的形式可以继续如下整理:

$$\begin{aligned}
P(k) &= C_{N-1}^{k} p^{k} (1-p)^{N-1-k} \\
&= \frac{(N-1)!}{(N-1-k)!k!} p^{k} (1-p)^{N-1-k} \\
&= \frac{(N-1)(N-2)\cdots(N-k)}{k!} p^{k} (1-p)^{N-1-k} \\
&\overset{(1)}{\approx} \frac{N^{k}}{k!} p^{k} (1-p)^{N} \\
&= \frac{(Np)^{k}}{k!} \left(1 - \frac{pN}{N}\right)^{N} \\
&\overset{(2)}{=} \frac{c^{k}}{k!} \left(1 - \frac{c}{N}\right)^{N} \\
&\overset{(3)}{\approx} \frac{c^{k}}{k!} \mathrm{e}^{-c}, k \geqslant 0.
\end{aligned} \tag{14.8}$$

上式中，符号(1)利用了当$N \gg 1$时，$N - i \approx N$, 其中$i \in \{1, 2, \cdots, k+1\}$； 符号(2)中定义了当$N \gg 1$时$Np$为一个和$N$无关的常数$c$； 符号(3)中利用了自然底数e的定义。 最终的Eq.(14.8)的形式即为泊松分布（Poisson distribution）。 在(N, M)过程里，我们同样能得到泊松分布，且$c = 2M/N$。

由Eq.(14.8)可知，ER随机图的余度分布为

$$Q(k) \equiv \frac{kP(k)}{c} = \mathrm{e}^{-c}\frac{c^{k-1}}{(k-1)!}. \tag{14.9}$$

它同样具有泊松分布的形式。

- 从阶乘$k!$到函数$\Gamma(k+1)$

度k为自然数，度的阶乘$k!(\geqslant 0)$可以直接计算。 我们可以继续把阶乘$(k!)$推广到Γ函数（Gamma function）的形式。 Γ函数定义为

$$\Gamma(s) \equiv \int_0^{+\infty} \mathrm{d}t t^{s-1}\mathrm{e}^{-t}. \tag{14.10}$$

当$s(\geqslant 0)$为自然数时，有递推公式$\Gamma(s+1) = s\Gamma(s)$，$\Gamma(1) = 1$。 则Γ函数退化成阶乘形式$\Gamma(s) = (s-1)!$,。

当我们用$\Gamma(k+1)$代替$k!$后，就可以把之前的离散值k推广到任意实数$k \geqslant 0$。Γ函数的计算可以使用GSL (GNU Scientific Library)[35]里的特殊函数计算模块。

ER随机图的度分布如Figure 14.2所示。这里我们比较了图例上经验度分布和泊松分布的结果。 首先，我们生成了$N = 10^5$、不同c的ER随机图，然后统计了其经验度分布。作为对比，我们还解析计算了给定c的泊松分布。最终度分布的模拟结果和解析结果可见Figure 14.2。

在有关图相关的一些理论计算中，我们常常需要计算基于度分布$P(k)$和余度分布$Q(k)$的求和计算，其形式如下：

$$\mathrm{SumP}(s,x) \equiv \sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s}, \tag{14.11}$$

$$\mathrm{SumQ}(s,x) \equiv \sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}x^{k-1-s}, \tag{14.12}$$

其中因子$s \geqslant 0$, 参数$x \in [0,1]$。

在ER随机图下，我们可以推导得知

$$\sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s} = \mathrm{e}^{-c(1-x)}\frac{c^s}{s!}, \tag{14.13}$$

$$\sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}x^{k-1-s} = \mathrm{e}^{-c(1-x)}\frac{c^s}{s!}. \tag{14.14}$$

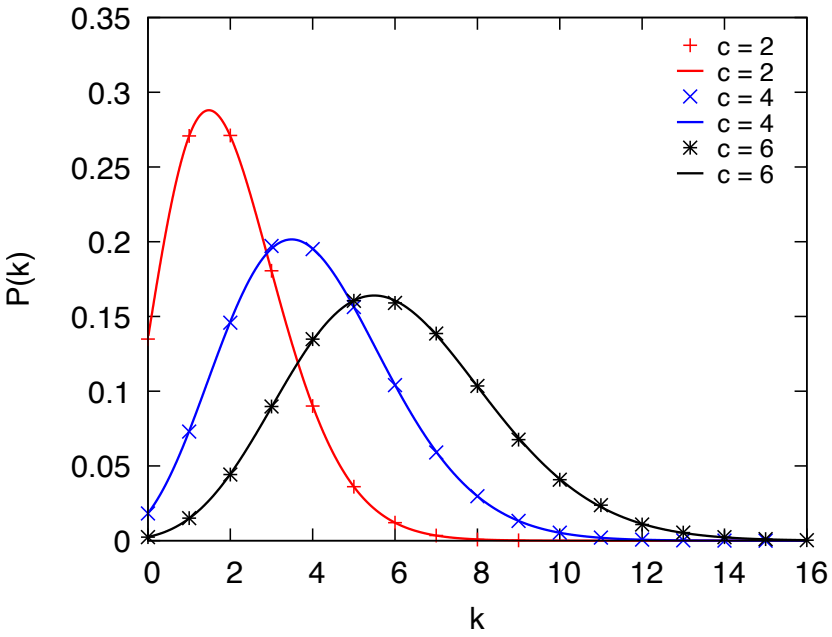


FIG. 14.2. ER随机图的度分布。这里展示了平均度$c = 2, 4, 6$时对应的度分布。曲线为使用$\Gamma(k+1)$函数代替阶乘$k!$后得到的度分布的解析结果。符号为顶点数$N = 10^5$的一幅ER随机图上的经验度分布$P_{\rm em}(k)$。

以下是求和计算的具体步骤。对于Eq.(14.13)，我们有

$$\begin{aligned}\sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s} &= \sum_{k=s}^{+\infty} \mathrm{e}^{-c}\frac{c^k}{\cancel{k!}}\cdot\frac{\cancel{k!}}{s!(k-s)!}x^{k-s}\\ &= \mathrm{e}^{-c}\sum_{k=s}^{+\infty} c^s c^{k-s}\frac{x^{k-s}}{s!(k-s)!}\\ &= \mathrm{e}^{-c}\frac{c^s}{s!}\cdot\sum_{k=s}^{+\infty}\frac{(cx)^{k-s}}{(k-s)!}\\ &= \mathrm{e}^{-c}\frac{c^s}{s!}\cdot\mathrm{e}^{cx}\\ &= \mathrm{e}^{-c(1-x)}\frac{c^s}{s!}.\end{aligned} \tag{14.15}$$

对于Eq.(14.14)，我们有

$$\begin{aligned}\sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}x^{k-1-s} &= \sum_{k=s+1}^{+\infty}\mathrm{e}^{-c}\frac{c^{k-1}}{\cancel{(k-1)!}}\cdot\frac{\cancel{(k-1)!}}{s!(k-1-s)!}x^{k-1-s}\\ &= \mathrm{e}^{-c}\sum_{k=s+1}^{+\infty} c^s c^{k-1-s}\frac{x^{k-1-s}}{s!(k-1-s)!}\end{aligned}$$

$$
\begin{aligned}
&= \mathrm{e}^{-c}\frac{c^s}{s!}\cdot\sum_{k=s+1}^{+\infty}\frac{(cx)^{k-1-s}}{(k-1-s)!}\\
&= \mathrm{e}^{-c}\frac{c^s}{s!}\cdot\mathrm{e}^{cx}\\
&= \mathrm{e}^{-c(1-x)}\frac{c^s}{s!}.
\end{aligned}
\tag{14.16}
$$

14.3. 度分布的无标度性质

> Grau, teurer Freund, ist alle Theorie,
> Und grün des Lebens goldner Baum.
>
> – *Faust*, by Johann Wolfgang von Göthe
>
> 亲爱的朋友，一切理论都是灰色的，
> 唯有生命之树常青。
>
> 摘自：《浮士德》，约翰·沃尔夫冈·冯·歌德著

在随机图模型里，我们只是对顶点之间的连接方式做了假设。但真实世界网络(real-world networks)往往具有远远超过随机图的丰富的结构特点，在微观、介观、宏观层次上都偏离了随机图模型的预测。 在最直观的度分布意义上，真实网络的分布远离上述ER随机图的泊松分布。真实世界网络的度分布的经验统计结果[37]表明，诸如演员合作网络、网页的超链接、电网结构等等，具有类似幂律的分布特征，即$P(k)\sim k^{-\gamma},\gamma>2.0$，其中$\gamma$为度分布的指数。 这种分布性质为网络的无标度律(scale-free property)，对应的网络被称为无标度网络（scale-free networks）。 系统的标度(scale)描述的是系统在时间、空间、或者某个标度上的一个大致尺度：当时间或空间大于这个尺度之后，系统的性质、概率等等会指数级地变小。对于ER随机图来说，这个尺度就是平均度c。但某些系统可能具有无标度性质，这意味这些系统在某些指标上分布非常宽。 在无标度网络里，大部分的点的度是很小的，但存在着少量度很大的点。这种结构特征导致真实网络在动力学和功能上都偏离了随机图模型。

在构造无标度网络上，我们可以使用多种模型来实现。 本质上，无标度网络是在顶点数N和度k都很大时才会出现的渐进性质。这导致使用不同模型来生成无标度网络时，就算度参数γ选择为一样，图的结构都可能相差很大。

以下我们考虑无标度网络的模型：构型模型、BA模型、静态模型。

14.4. 纯无标度网络(Pure scale-free networks)

如果直接用幂律分布来描述无标度网络的度分布，可得到$P(k)=k^{-\gamma}/\zeta(\gamma)$，相应地其余度分布为$Q(k)=k^{1-\gamma}/\zeta(\gamma-1)$。其中$\zeta(\gamma)=\sum_{k=1}^{+\infty}k^{-\gamma},(\gamma>2)$为黎曼Zeta函数(Riemann Zeta function)。相应地，我们可以计算Eqs.(14.11)-(14.12)的一些结果，如下：

$$\sum_{k=1}^{+\infty}P(k)x^k=\frac{\mathrm{Li}_\gamma(x)}{\zeta(\gamma)}, \tag{14.17}$$

$$\sum_{k=1}^{+\infty}P(k)kx^{k-1}=\frac{\mathrm{Li}_{\gamma-1}(x)}{x\zeta(\gamma)}, \tag{14.18}$$

$$\sum_{k=1}^{+\infty}Q(k)x^{k-1}=\frac{\mathrm{Li}_{\gamma-1}(x)}{x\zeta(\gamma-1)}. \tag{14.19}$$

上式中，$\mathrm{Li}_n(x)=\sum_{k=1}^{+\infty}x^k/k^n$ 为参数x的n阶多对数(n-th polylogarithm)。

14.5. BA模型(Barabási-Albert Model)

Barabási-Albert模型（Barabási-Albert model, BA模型）[37] 为第一个能够构造出无标度网络的模型。提出该模型的论文被视为网络科学诞生标志的两篇论文之一。

1. 生成过程

此模型的要素主要有两个：网络生长过程(network growth)、优先连接法则(preferential attachment)。 它们模拟了真实网络生成过程中的两个经验性观察。

∘ 网络生长过程：在生成随机图过程中，我们一般在给定节点数的图中不断加入连边，但在真实的网络生成过程中，网络里会同时加入节点和连边。比如微信账号构成的社交网络里，存在着已有账号之间互加好友，但也同时存在着新账号不断加入社交网络。

∘ 优先连接法则：新的节点加入已有网络时，并不会均匀地与已有节点构成连边，而是具有一些倾向性。BA模型选择的法则是：已有网络的任一节点i和新加入节点相连接的概率正比与它的度k_i。

BA模型的描述如下：

（1）初始状态下，给定n_0个节点，且它们之间没有连边。

（2）之后不断地加入$N-n_0$个节点，且每个节点和已有网络里的节点构造$k_0\leqslant n_0$条连边。已有网络的节点i和新加入节点构造连边时，连接概率$\propto k_i$。

依据以上过程可以构造节点数为N、连边数为$k_0(N-n_0)$的图。

2. 度分布的推导

在N很大时，它的平均度为k_0，度分布可以解析得到为：

$$P(k) = \frac{2k_0^2}{k^3} \sim k^{-3}. \tag{14.20}$$

也就是说，如此构造的图的度分布具有无标度性质，且$\gamma = 3$。这种度分布在做图展示上，使用双对数坐标轴会更直观一些。

3. 计算结果

BA模型生成图的度分布模拟结果和解析预测可见Figure 14.3。

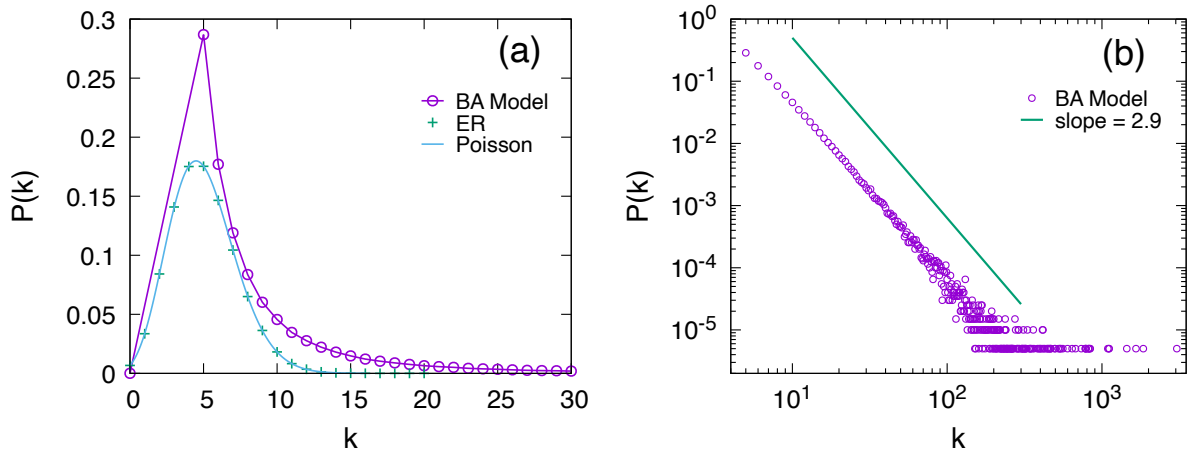


FIG. 14.3. BA模型生成图的度分布。 (a)展示了BA模型生成的一个图的经验度分布，对应节点数$N = 2 \times 10^5$、初始节点数$n_0 = 10$、新加入节点的度$k_0 = 5$。 作为对比，图中也展示了ER随机图的度分布的模拟结果和解析结果。模拟结果中，选择节点数$N = 10^6$、平均度$c = 5$。图中还给出了$c = 5$对应的泊松分布，即无穷大ER图的度分布。 (b)中，把(a)的BA模型的度分布数据点使用双对数坐标轴重新做图。

14.6. 静态模型(Static Model)

这里我们讨论构造渐进意义上满足幂律度分布的随机图。 这里我们考虑使用静态模型(static model) [38, 39] 来生成渐进无标度网络(asymptotic scale-free networks)。

1. 生成过程

静态模型为利用隐参数模型 (hidden parameter model)来生成图的一个具体例子。 对于隐参数模型，节点i具有某个权重$w(i,\vec{a})$，其中$\vec{a}$为图生成过程的参数。构造连边时，节点选择的概率正比于$w(i,\vec{a})$。 假如我们使用静态模型来生成一个节点数为N、平均度为c、度指数为γ的无标度网络。可以相应地定义一个参数$\xi = 1/(\gamma-1)$。 对于点集V，我们给所有顶点以编号$i = 0, 1, \cdots, N-1$。每个节点上定义权重为$w_i \sim (i+1)^{-\xi}$。 构造图的过程是从只有N个节点组成的点集V、没有连边的空图$G = \{V, \varnothing\}$开始，不断加入$M = cN/2$条连边。每次加入连边时，我们首先随机选择两个节点，且顶点被选择的概率p_i正比于其权重w_i，即$p_i \propto w_i$。如果两个顶点之间没有连边，则把它们连接起来；如果两点之间已经有了连边，我们可以再次随机选择第二个顶点，直到两点之间没有连边为止。按照上述过程加入M条边之后，我们得到$G = \{V, E\}$。

• 具体来说，使用静态模型生成一幅图的过程如下:

（1）给定节点数N和连边数M，构造并初始化Vertices、Edges、Weights。 对于Weights，每个节点的未归一化权重为$(i+1)^{-\xi}$（$i = 0, 1, \cdots, N-1$），之后计算其归一化权重w_i、累积权重cw_i。

（2）对于Edges，依次生成M条边。生成一条边时，使用$(0,1)$间均匀分布的随机数生成器生成$r \in (0,1)$，然后使用二分法在Weights的数据成员cw里找到对应的节点编号v。如此可以得到两个节点$(v1, v2)$。使用函数IsAnEdge($v1$, $v2$)来判断两个节点之间是否已有连边。如果没有，则把连边加入图中。

（3）在图中加入一条连边的函数为AddAnEdge$(v1, v2)$。加入连边$(v1, v2)$时，在$v1$的最近邻矢量里加入$v2$，$v1$的度增加1。使用同样过程把$v2$放入$v1$的最近邻矢量。

• 在代码层面，这里还需要实现如何从顶点的权重概率分布来生成顶点的序号。这个实现过程类似于“字符串”一章里，我们基于字符概率来生成随机字符串。

Weights的数据成员有w和cw。对于Weights$[i]$，$i \in [0, N-1]$，其

$$w_i = \frac{(i+1)^{-\xi}}{\sum_{i=0}^{N-1}(i+1)^{-\xi}}. \tag{14.21}$$

相应的$cw_i = \sum_{j=0}^{i} w_j$。给定任一随机数$r \in (0,1)$，如果某个$l \in [0, N-1]$满足条件$cw_{l-1} < r \leqslant cw_l$，则$r$对应的节点编号为$l$。原因在于：按照这种方式，节点$l$被选择的概率为区间$(cw_{l-1}, cw_l]$的宽度，即$w_l$。

通过节点权重的形式$w_i = i^{-\xi}$可知，当γ很大时，顶点权重不依赖于具体的编号、且趋于相等。这意味着通过选择顶点来生成连边时，每个顶点被选择的概率相等，即此时静态

模型退化为ER随机图模型。

2. 度分布的推导

在推导随机图的度分布时，我们考虑的是热力学局限(thermodynamic limit)的情形，即$N, M \to \infty$、且$M/N \to \Theta(1)$。按照上述过程，给定顶点权重w_i的具体形式，我们都能得到生成图在热力学极限下的度分布$P(k)$，但其对应形式的推导依赖于w_i本身。 在w_i为如上的指数形式时，使用解析理论方法 [38, 39]能够得到此时生成图的度分布$P(k)$的形式如下：

$$\begin{aligned} P(k) &= \frac{1}{\xi}\frac{[c(1-\xi)]^k}{k!}\int_1^{+\infty} \mathrm{d}t t^{k-1-\frac{1}{\xi}}\mathrm{e}^{-c(1-\xi)t} \\ &= \frac{1}{\xi}\frac{[c(1-\xi)]^k}{k!}\mathrm{E}_{-k+1+\frac{1}{\xi}}[c(1-\xi)]. \end{aligned} \tag{14.22}$$

上式中第二个等号利用了特殊函数$\mathrm{E}_a(x)$的定义。$\mathrm{E}_a(x)$为广义指数积分(generalized exponential intergral, GEI)，其形式为

$$\mathrm{E}_a(x) \equiv \int_1^{+\infty} \mathrm{d}t e^{-xt}t^{-a}, a, x > 0. \tag{14.23}$$

当$k \gg q$时，$P(k)$满足$P(k) \propto k^{-\gamma}$，此时$\gamma = 1 + 1/\xi$，其中γ为图幂律分布的指数。相对于BA模型，静态模型可以生成$\gamma > 2.0$的无标度网络。 相应的图的余度分布为：

$$\begin{aligned} Q(k) &= \left(\frac{1}{\xi} - 1\right)\frac{[c(1-\xi)]^{k-1}}{(k-1)!}\int_1^{+\infty} \mathrm{d}t t^{k-1-\frac{1}{\xi}}\mathrm{e}^{-c(1-\xi)t} \\ &= \left(\frac{1}{\xi} - 1\right)\frac{[c(1-\xi)]^{k-1}}{(k-1)!}E_{-k+1+\frac{1}{\xi}}[c(1-\xi)]. \end{aligned} \tag{14.24}$$

以下我们讨论如何计算特殊函数$\mathrm{E}_a(x)$。 首先，我们可知上不完全Γ函数(upper incomplete gamma function)定义为

$$\Gamma(s, x) = \int_x^{+\infty} \mathrm{d}t t^{s-1}\mathrm{e}^{-t}, x > 0. \tag{14.25}$$

我们可以推导求得$\mathrm{E}_a(x)$和$\Gamma(s, x)$的关系。 对于$\mathrm{E}_a(x)$的右侧(right-handed side, RHS)的积分，我们定义一个新的变量$xt = u$。此时有

$$\begin{aligned} \mathrm{E}_a(x) &= \int_x^{+\infty} \mathrm{d}\left(\frac{u}{x}\right)\left(\frac{u}{x}\right)^{-a}\mathrm{e}^{-u} \\ &= \int_x^{+\infty} \mathrm{d}u u^{-a}\mathrm{e}^{-u}\cdot x^{a-1} \\ &= x^{a-1}\Gamma(1-a, x). \end{aligned} \tag{14.26}$$

上述的Γ函数(gamma function)可以使用GNU Scientific Library (GSL)[35]来计算，指令为：*double gsl_sf_gamma_inc(double a, double x)*。相应地，我们也就能解析计算$\mathrm{E}_a(x)$、以及度分布Eqs.(14.22)和(14.24)。

和上述推导过程相关的一些特殊函数：

- 指数积分(exponential integral)：

$$\mathrm{E}_1(x) \equiv \mathrm{Re}\int_1^{+\infty} \mathrm{d}t t^{-1}\mathrm{e}^{-xt}, x > 0. \tag{14.27}$$

- 广义指数积分(generalized exponential integral, GEI)

$$\mathrm{E}_\alpha(x) \equiv \int_1^{+\infty} \mathrm{d}t t^{-\alpha}\mathrm{e}^{-xt}, a, x > 0. \tag{14.28}$$

- Γ函数(gamma function)

$$\Gamma(s) \equiv \int_0^{+\infty} \mathrm{d}t t^{s-1}\mathrm{e}^{-t}. \tag{14.29}$$

如果$s(\geqslant 1)$为整数，Γ函数退化成阶乘形式$\Gamma(s) = (s-1)!, \Gamma(0) = 1$.

- 上不完全Γ函数(upper incomplete gamma function)：

$$\Gamma(s,x) \equiv \int_x^{+\infty} \mathrm{d}t t^{s-1}\mathrm{e}^{-t}, x > 0. \tag{14.30}$$

- 下不完全Γ函数(lower incomplete gamma function)

$$\gamma(s,x) \equiv \int_0^{x} \mathrm{d}t t^{s-1}\mathrm{e}^{-t}, x > 0. \tag{14.31}$$

对于求和公式Eqs.(14.11)-(14.12)，我们有($s \geqslant 0$)：

$$\sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s} = \frac{1}{\xi}\frac{[c(1-\xi)]^s}{s!}E_{-s+1+\frac{1}{\xi}}[c(1-\xi)(1-x)], \tag{14.32}$$

$$\sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}x^{k-1-s} = \left(\frac{1}{\xi}-1\right)\frac{[c(1-\xi)]^s}{s!}E_{-s+\frac{1}{\xi}}[c(1-\xi)(1-x)]. \tag{14.33}$$

以下为具体的推导过程。对于Eq.(14.32)，我们有：

$$\sum_{k=s}^{+\infty} P(k)\binom{k}{s}x^{k-s}$$

$$
\begin{aligned}
&= \sum_{k=s}^{+\infty} \frac{1}{\xi} \frac{[c(1-\xi)]^k}{\cancel{k!}} \int_1^{+\infty} \mathrm{d}t t^{k-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \frac{\cancel{k!}}{s!(k-s)!} x^{k-s} \\
&= \sum_{k=s}^{+\infty} \frac{1}{\xi} \frac{[c(1-\xi)]^{k-s+s}}{s!} \int_1^{+\infty} \mathrm{d}t t^{k-s+s-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \frac{x^{k-s}}{(k-s)!} \\
&= \sum_{k=s}^{+\infty} \frac{1}{\xi} \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \frac{[c(1-\xi)xt]^{k-s}}{(k-s)!} \\
&= \frac{1}{\xi} \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \sum_{k=s}^{+\infty} \frac{[c(1-\xi)xt]^{k-s}}{(k-s)!} \\
&= \frac{1}{\xi} \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \mathrm{e}^{c(1-\xi)xt} \\
&= \frac{1}{\xi} \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)(1-x)t} \\
&= \frac{1}{\xi} \frac{[c(1-\xi)]^s}{s!} E_{-s+1+\frac{1}{\xi}}[c(1-\xi)(1-x)].
\end{aligned} \tag{14.34}
$$

对于Eq.(14.33)，我们有：

$$
\begin{aligned}
&\sum_{k=s+1}^{+\infty} Q(k) \binom{k-1}{s} x^{k-1-s} \\
&= \sum_{k=s+1}^{+\infty} \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^{k-1}}{\cancel{(k-1)!}} \int_1^{+\infty} \mathrm{d}t t^{k-1-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \frac{\cancel{(k-1)!}}{s!(k-1-s)!} x^{k-1-s} \\
&= \sum_{k=s+1}^{+\infty} \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^{k-1-s+s}}{s!} \int_1^{+\infty} \mathrm{d}t t^{k-1-s+s-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \frac{x^{k-1-s}}{(k-1-s)!} \\
&= \sum_{k=s+1}^{+\infty} \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \frac{[c(1-\xi)xt]^{k-1-s}}{(k-1-s)!} \\
&= \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \sum_{k=s+1}^{+\infty} \frac{[c(1-\xi)xt]^{k-1-s}}{(k-1-s)!} \\
&= \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)t} \cdot \mathrm{e}^{c(1-\xi)xt} \\
&= \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^s}{s!} \int_1^{+\infty} \mathrm{d}t t^{s-\frac{1}{\xi}} \mathrm{e}^{-c(1-\xi)(1-x)t} \\
&= \left(\frac{1}{\xi} - 1\right) \frac{[c(1-\xi)]^s}{s!} E_{-s+\frac{1}{\xi}}[c(1-\xi)(1-x)].
\end{aligned} \tag{14.35}
$$

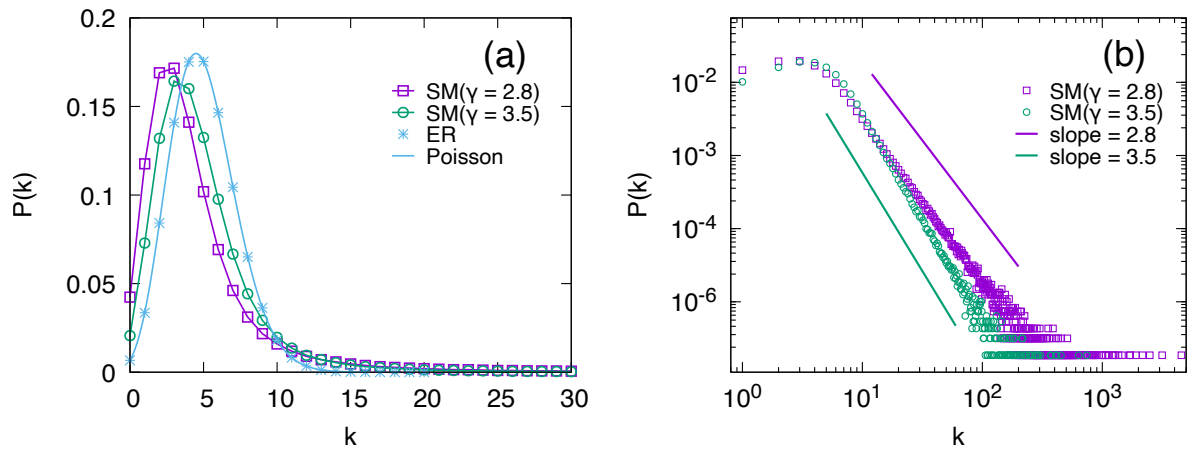


FIG. 14.4. 使用静态模型生成的无标度网络的度分布。 (a)展示了两个无标度网络的经验度分布。我们分别生成了节点数$N = 10^6$、平均度$c = 5$、度指数分别为$\gamma = 2.8$和3.5的两个图例，然后对图例统计其度分布。这里使用的是线性坐标轴。 作为对比，图中还展示了N和c相同的一幅ER随机图的度分布，以及对应的柏松分布。 (b)把(a)中的无标度网络的度分布数据点使用双对数坐标轴(log10-log10)重新做图。 作为对比，图中还画出来了两个度指数对应斜率的直线。

3. 计算结果

Figure 14.4给出使用静态模型生成随机图的例子，分别用线性坐标和双对数坐标对度分布做了图。 由Figure 14.4(a)可知，无标度网络里度概率最大时对应的度是小于c的，而且随着γ的减小会越来越小。此外，在k比较大的区域，$P(k)$显然不为0。 由Figure 14.4(b)可知，在k比较大时，度分布表现出幂律的趋势。

这里讨论下无标度网络和ER随机图在度分布上的差别。 对于节点数N和平均度c相同情况下的ER随机图，大部分节点的度在$k \approx c$附近。当k很大时，度对应概率会变的非常小。这意味着在节点数有限的图里，k很大的点将很难存在。 相比之下，静态模型生成的图在度$k < c$区域的比例要高于ER随机图，而且当k很大时，对应的$P(k)$依然不明显地趋于0。这意味着在节点数相同的无标度网络里，存在少数节点有着很大的度。这种节点一般叫做中心节点(hubs)。这里有一些经验性的结果：Figure 14.4里ER随机图里最大度为18、对应节点数为3；但在$\gamma = 3.5$对应的无标度网络里，最大度为743、对应节点数为1；在$\gamma = 2.8$对应的网络里，最大度为4562，对应节点数为1。由此可见，随着γ的减小，有着同样节点数的无标度网络中节点的最大度将越来越大。 整体上来说，对于具有同样平均度的ER随机图和使用静态模型生成的无标度网络，后者出现度偏小和度很大的

节点的概率都会比ER随机图要高。

14.7. 构型模型(Configuration Model)

1. 生成过程

构型模型(Configuration Model)是一种直接从度分布到到图例的生成图的方式：它首先基于给定的度分布来生成相应的度序列，然后基于度序列构造连边以生成相应的图结构。这种方式可以用来说生成任意给定合适度分布$P(k)$的图例，不限于无标度网络。

这里我们考虑一般情况下构造图的过程。对于具体的图例，除了度分布$P(k)$，我们还需要设置一些参数：图的顶点数N、最小度$k_{\min}$、最大度$k_{\max}$。

这里我们估计一下按照构型模型构造出来的图例的平均度c。 在构型模型中，根据顶点数N和度分布$P(k)$，我们可以得到任意的度$k \in [k_{\min}, k_{\max}]$对应的顶点数$N_k = NP(k)$。相应的图的连边数$M$满足$2M = \sum_{k=k_{\min}}^{k_{\max}} kN_k$。 相应的图的度分布为$c = 2M/N$。容易可知，在这个模型里，给定参数$N$和结构参数之后，平均度$c$也就给定了。

构型模型的具体过程如下：

（1）从度分布到度的序列：基于顶点数N和度分布$P(k)$，可以得到$k \in [k_{\min}, k_{\max}]$对应的顶点数$N_k$。此时对于大小为$N$的顶点序列，我们按照顺序给顶点依次赋予度，也就是我们依次给大小为N_k的顶点赋予其度为k。给所有顶点赋予度之后得到一个度序列，之后可以对度序列进行随机化，也就是随机选择两个顶点（顶点i和j）、然后交换两者的度（$k_i \leftrightarrow k_j$）。

（2）从半连边到连边：此时我们有顶点数为N、没有连边的图，相应的顶点i的度为k_i。 可以视为任意顶点i有k_i个半连边。之后通过连接两个半连边来构造完整的连边。

（3）随机化图的连边：当初始的图构型生成以后，可以进行连边的随机化。

代入具体的无标度度分布之后，则可以利用上述构型模型来构造无标度网络的图例。此时我们需要指定幂律度分布的指数γ。 则我们想要生成的图的度分布为：

$$P(k) = \frac{k^{-\gamma}}{\sum_{m=k_{\min}}^{k_{\max}} m^{-\gamma}}. \tag{14.36}$$

其中，$k_{\min} \leqslant k \leqslant k_{\max}$。最小度$k_{\min}$我们一般取$\geqslant 2$的值。最大度$k_{\max}$的设置一般用来影响着网络里的度度关联(degree-degree correlation)。一般我们选择$k_{\max} = \sqrt{N}$，此时可以消除网络里的度度关联[41]。

2. 度分布的推导

对于使用构型模型生成的图例的度分布Eq. (14.36)，其余度分布为：

$$Q(k) = \frac{k^{1-\gamma}}{\sum_{m=k_{\min}}^{k_{\max}} m^{1-\gamma}}. \tag{14.37}$$

上式中，$k_{\min} \leqslant k \leqslant k_{\max}$。

使用上述方式生成的图中最小度为$k_{\min}$。这个图例可以继续进行稀疏化，类似于正规随机图的稀疏化过程，也就是我么可以随机选择比例为$1-\rho$的连边并删除。这种稀疏化过程会在整体上降低连边的密度。此时稀疏化图的平均度为$c = c_0\rho$。对于一个度分布为$P_0(k)$、对应余度分布$Q_0(k)$、平均度为c_0的无标度网络的图例，其稀疏化后的度分布和余度分布分别为：

$$\begin{aligned} P(k) &= \sum_{\hat{m}=k}^{k_{\max}} P_0(\hat{m}) \binom{\hat{m}}{k} \rho^k (1-\rho)^{\hat{m}-k} \\ &= \frac{1}{\sum_{m=k_{\min}}^{k_{\max}} m^{-\gamma}} \sum_{\hat{m}=k}^{k_{\max}} \hat{m}^{-\gamma} \binom{\hat{m}}{k} \rho^k (1-\rho)^{\hat{m}-k}, \end{aligned} \tag{14.38}$$

$$\begin{aligned} Q(k) &= \sum_{\hat{m}=k}^{k_{\max}} Q_0(\hat{m}) \binom{\hat{m}-1}{k-1} \rho^{k-1} (1-\rho)^{\hat{m}-k} \\ &= \frac{1}{\sum_{m=k_{min}}^{k_{\max}} m^{1-\gamma}} \sum_{\hat{m}=k}^{k_{\max}} \hat{m}^{1-\gamma} \binom{\hat{m}-1}{k-1} \rho^{k-1} (1-\rho)^{\hat{m}-k} \end{aligned} \tag{14.39}$$

上述两式中，$k_{\min} \leqslant k \leqslant k_{\max}$。

3. 计算结果

计算结果。

14.8. 图的随机化

真实世界网络往往具有复杂的内部结构。在真实世界网络上使用算法时，其结果显然来自真实世界网络内部结构的影响。那么如何度量这种内部结构对结果的影响?

我们可以把真实世界网络的结果视为对某个无内部结构的网络的偏移。算法在无内部结构的网络上的结果视为算法的基准(benchmark)。无内部结构的网络可以通过对网络结构进行随机化得到。比较常见的随机化过程是对连边进行操作：网络的中每一个顶点，

保证顶点的度不变，但连边的选择是随机的。这就是图的保度随机化(degree-preserving randomization)。使用保度随机化后得到的图依然具有相同的度分布，但随机化消除了图的内部结构。

在实现保度随机化时，我们可以不断随机选择两条连边、交换彼此的端点来实现。比如对于图中的任意两条边(u_1, v_1)和(u_2, v_2)，如果(u_1, v_2)和(u_2, v_1)在图中并不存在，则可以交换两条边的端点，生成这两条新的连边；如果(u_1, v_2)和(u_2, v_1)在图中已经存在，我们再次随机选择(u_2, v_2)、直到连边(u_1, v_2)和(u_2, v_1)在图中并不存在为止。连边的端点交换过程如Figure 14.5所示。

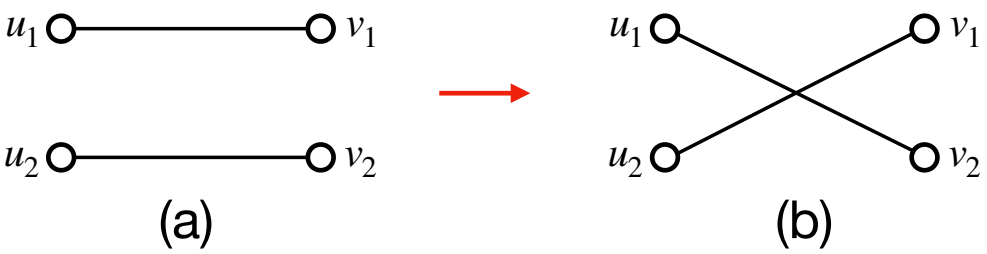


FIG. 14.5. 保度随机化中连边交换端点的操作。 (a)为初始图中的两条连边(u_1, v_1)和(u_2, v_2)。(b)中经过端点的交换，得到新的两条连边(u_1, v_2)和(u_2, v_1)。

习题

14.1 正规随机图的生成

首先使用代码生成正规随机图。

之后使用连边的稀疏化过程来得到新的图。然后统计其经验度分布。 请比较此时图的度分布的模拟和解析结果。

14.2 ER随机图的生成

生成一副ER随机图，其顶点数目为$N = 10^6$、平均度为$c = 6$。请完成代码统计其度分布，并在一幅结果图上同时画出来这副图的度分布和$c = 6$对应的泊松分布。

14.3 使用静态模型生成无标度网络

使用代码实现静态模型(static model)以生成无标度网络，并重复出来Figure 14.4中的结果。

14.4 图的保度随机化

给定一个真实世界网络(karate club network)，在经过保度随机化之后，图的一些结构性质的变化，比如同一个顶点作为源点事其它顶点到它的平均距离、图的直径等等。

15.

15.1. 图上的渗流现象(Percolation phenomena)

广义的渗流现象(percolation phenomena) [43, 44] 研究的是宏观连接性(macroscopic connectedness)的产生。 宏观连接性可以通过最大连通单元(giant component)的相对大小来标识。图的一个集团(cluster)或者连通单元(component)指的是一个子图、其中任意两个点之间都有路径相连。最大连通单元指的是图的最大的连通单元。 对于一个给定的图，我们可以使用图遍历算法列出来图所有的子图及其大小，然后选择其中最大的那个。

一般情况下，当渗流现象定义在图上时，我们关注的是在给定的删除个点和连边的流程下，随着图结构参数或者控制参数的变化，图的最大连通单元的产生和变化。图上的渗流现象是图上的一种结构相变现象(structural phase transitions)。值得指出的是图的第二大连通单元(second largest component)也是比较重要的拓扑性质，能够用来确定结构相变现象的阶数。

为何要研究图上的渗流现象?

网络的脆弱性（fragility）、韧性（resilience）、和鲁棒性（robustness）都和渗流现象有关系。以上例子的主题是网络系统里失效的级联传播(cascading failures)。网络系统里，组成单元能够正常地运行以实现某个功能，依赖于其它单元的正常运行。某个单元的失效会继续影响和它相连的单元，导致失效的传递。在失效的级联传播例子里，常常会出现一个局域的、少量构成单元的失效造成大范围的、数目巨大的构成单元的失效。

渗流理论(percolation theory)关注的是渗流模型的构造、分析、和解析结果。 对于图上渗流模型的结果，我们关注两个方面：理论上预测最大连通单元的比例的变化、确定转变点的位置以及分析临界点附近的临界行为(critical behaviour)。 图上渗流现象的两种常见情形基于渗流曲线在临界点附近的变化。当控制参数增加到某个值时，最大连通单元从0连续地变成正值、并且逐渐变大，这对应着连续或二阶相变(continuous or second-order phase transitions)。但当控制参数增加到某个值时，最大连通单元从0跳到某个值、然后逐渐变大，这对应着不连续或一阶相变(discontinuous or first-order phase transitions)。Figure 15.1展示了两种情形的例子。

对于随机图上的渗流模型，我们一般能够使用少量的变量来得到一套自洽方程(self-consistent equations)来描述最大连通单元的变化。但在真实世界网络上，从解析角度来描述渗流现象是困难的，往往需要更加复杂的平均场理论(mean-field theory)框架。

在随机图上，我们可以使用解析方式来求解渗流模型。其解析方式一般为概率形式，

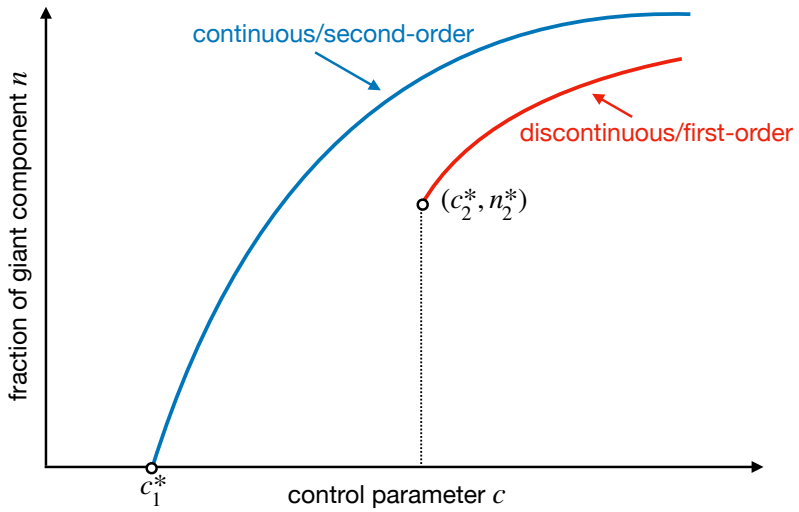


FIG. 15.1. 图上渗流现象的两种常见情形。 图中横轴为模型控制参数c，纵轴为图的最大连通单元的比例n。 对于图中的连续/二阶相变曲线，当控制参数$c < c_1^*$时，$n = 0$；但当$c \geqslant c_1^*$时，n从0连续地逐渐增加。 对于图中的不连续/一阶相变曲线，当控制参数$c < c_2^*$时，$n = 0$；但当$c = c_2^*$时，n从0直接跳到$n_2^*(> 0)$。

并写成生成函数的形式(generating function method) [45]。

15.2. 最大连通单元的生成

1. 模型

在无向图里，我们更关心的是最大连通单元(giant component, GC)，即连通单元中点数最大的那个 其相对大小可以通过它内部顶点数目和图的顶点数目的比例n来表示。图上的最大连通单元的产生（emergence of giant component）对应着图上宏观结构连通性(macroscopic connectedness)的形成，为图论里最简单的渗流现象（percolation phenomenon）[43, 44]。渗流现象是图中典型的结构相变(structural phase transitions)。

如Figs.15.2和15.3所示。

我们考虑一个很大的稀疏随机图$G = \{V, E\}$，其中$V(|V| = N)$和$E(|E| = M)$分别为点集和边集， $c(\equiv 2M/N)$为图的平均连接度。此时我们考虑如何来定量地计算图中的最大连通单元(GC)的产生和变化。为了在更普适的框架下考虑最大连通单元的变化，我们定义初始稀疏化概率$p(\in [0, 1])$：在图G上，比例为$1 - p$的顶点首先被随机选择并移除，之

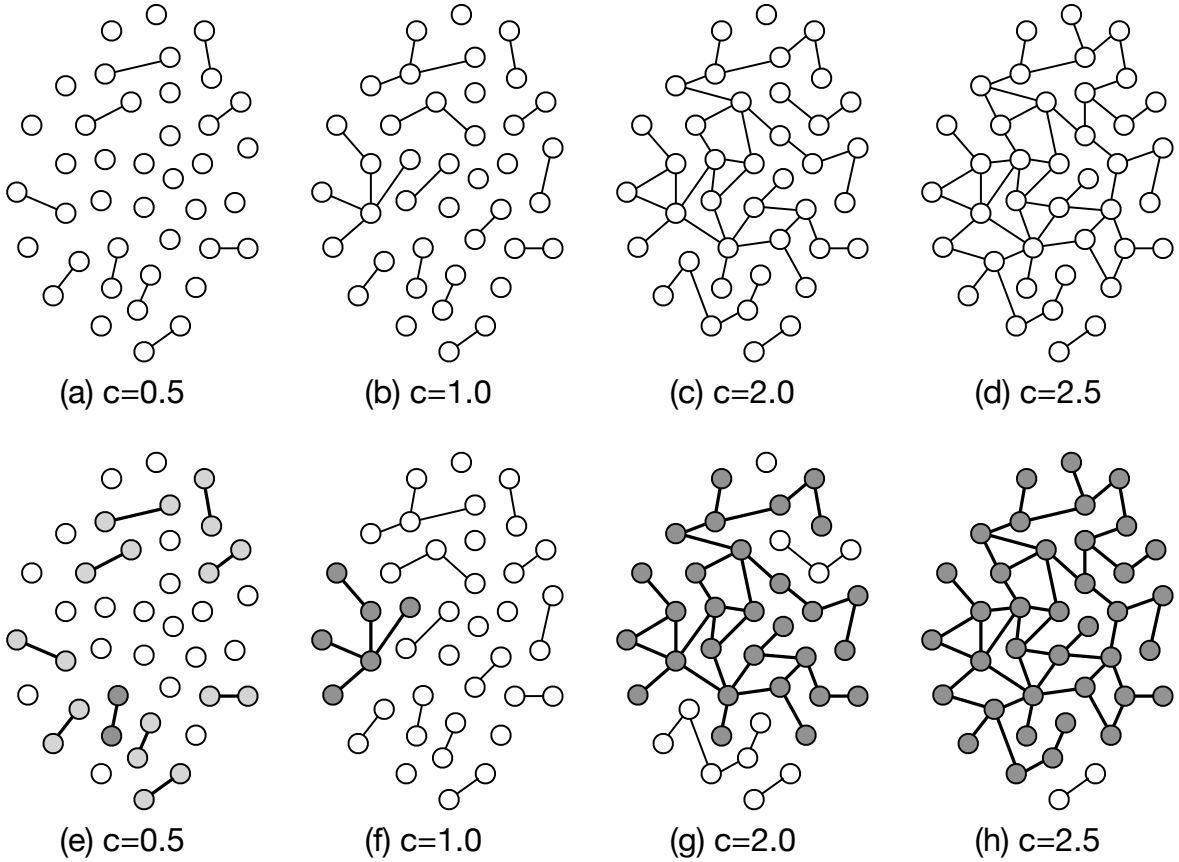


FIG. 15.2. 图上最大连通单元的生成（基于添加连边的过程）。 这里考虑从顶点数$N = 40$、没有连边的图开始，不断往图中添加连边的过程。 (a)中图的连边数$M = 10$，对应平均度$c = 0.5$。 (b)中图的连边数$M = 20$，对应$c = 1.0$。 (c)中图的连边数$M = 40$，对应$c = 2.0$。 (d)中图的连边数$M = 50$，对应$c = 2.5$。 (e) - (h)用深色的顶点和粗线的连边分别标记了(a) - (d)中图的最大连通单元，其包含顶点数分别为2、6、29、38。 注意：(a)中最大连通单元有10个，(e)中用深色色块标记了其中一个、用浅色色块标记了其它单元。

后我们在残留图上使用广度或深度优先搜索算法来寻找最大连通单元，并计算其比例n。

在模拟结果方面，我们依据顶点数N和平均度c来生成ER随机图的一个图例。然后我们在单个图例上完成广度优先算法或深度优先算法以找到其最大连通单元，继而得到其顶点数目$N_{\rm gc}$和比例$n_{\rm em}(= N_{\rm gc}/N)$。这样选择不同的$c$，可以得到对应图的$n_{\rm em}$组成的曲线。

使用单次的广度优先搜索算法（BFS）或者深度优先搜索算法（DFS）可得到图中的一个连通单元。 但找到其中的最大连通单元需要多次使用搜索算法直到所有的顶点都被遍历、并被指定为属于某个连通单元。 完成了这个遍历过程后，我们统计所有连通单元里顶点的数目，即可找出最大的那个。

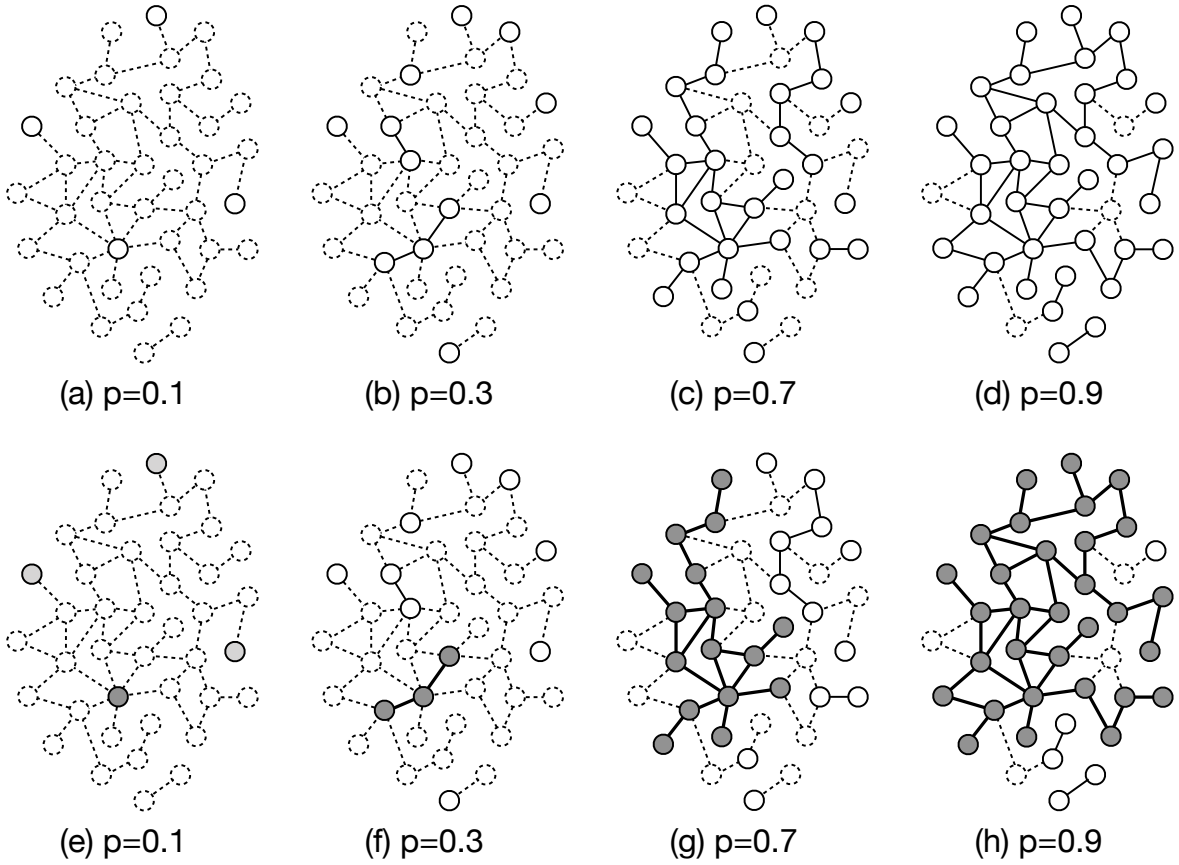


FIG. 15.3. 图上最大连通单元的生成（基于删除顶点和连边的过程）。 这里考虑从顶点数$N = 40$、连边数$M = 50$的一个图开始，不断删除顶点及其相连的连边的过程。 (a)中图的残留顶点数$N = 4$，对应移除后剩余概率$p = 0.1$。 (b)中图的残留顶点数$N = 12$，对应$p = 0.3$。 (c)中图的残留顶点数$N = 28$，对应$p = 0.7$。 (d)中图的残留顶点数$N = 36$，对应$p = 0.9$。 图中虚线表示的圆圈和直线分别表示被移除的顶点和连边。 (e) - (h)用深色的顶点和粗线的连边分别标记了(a) - (d)中图的最大连通单元，其包含顶点数分别为1、3、16、31。 注意：(a)中最大连通单元有4个，(e)中用深色色块标记了其中一个、用浅色色块标记了其它单元。

在具体的代码实现中，我们在顶点的结构体里定义了char qs和int cind。 前者用来记录顶点在稀疏化过程后是否依然在图中。$p = 1$意味着所有点都在图中。 后者用来记录顶点属于的连通单元的编号。此编号即为使用单次BFS或DFS时形参里的顶点编号。 图上的遍历过程完成后，我们统计每个顶点所属的连通单元的编号，即可得到每个连通单元的大小。

2. 解析理论

在稀疏图上，我们可以进一步得到其最大连通单元大小的解析理论框架。这个理论框架本质上是无穷大随机图上的概率计算框架。 在求解这类问题时，一般使用基于度分布的生成函数方法（generating function） [45]。

在稀疏图G中，我们首先假设概率$1-p$的顶点被随机选择并移除。我们随机选择一条边$(i,j)\in E$，从一个顶点i到达另外一个顶点j。我们假设j处在最大连通单元的概率为$x(\in[0,1])$。此时x为空穴概率(cavity probability)。

对于图G里的任一顶点i，i处在最大连通单元的概率为n。可知它处在最大连通单元需要求其最近邻中少有一个邻居处在最大连通单元。x本质上为边际概率(marginal probability)。我们有

$$n=p\sum_{k=0}^{+\infty}P(k)[1-(1-x)^k]. \tag{15.1}$$

图中顶点的状态彼此之间是相关的。容易得知，一个顶点属于GC，是因为它的邻居中必然有一个顶点也属于GC。通过这关系可以构造出来概率x的自洽方程。 在x的定义中，j如果能处在最大连通单元，此时要求它的最近邻中至少一个和最大连通单元相连，除了i之外。 我们有：

$$x=p\sum_{k=1}^{+\infty}Q(k)[1-(1-x)^{k-1}]. \tag{15.2}$$

Figure 15.4给出了空穴概率和边际概率的图示。 Eq.(15.2)-(15.1)组成的理论框架可以求得相应的最大连通单元的理论预计结果。 对于任意真实图例或者随机图，有了度分布$P(k)$和p，通过求自洽方程Eq.(15.2)可得到x的稳定解，之后代入Eq.(15.1)右侧即可得到n。

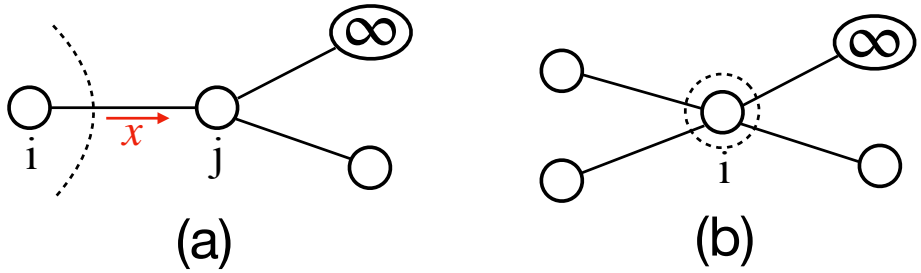


FIG. 15.4. 稀疏图G上最大连通单元的解析理论。 (a)中(i,j)为图G中随机选择的一条连边。 图中包含∞符号的椭圆表示最大连通单元。 (b)中顶点i为图G中任意选择的一个顶点。

3. 自洽方程的不动点分析

从普遍的意义上来说，我们能够直接求得任意随机图中出现最大连通单元时的临界值p。如果给定图的度分布为$P(k)$。定义Eq.(15.2)的右侧为$f(x;p)$，我们有$x = f(x;p)$。容易可知，$f(x;p)$是变量x的单调增加函数。可以有

$$f(x;p)|_{x=0} = 0, \tag{15.3}$$

$$f(x;p)|_{x=1} = pQ(1) \in [0,1]. \tag{15.4}$$

继而有：

$$\frac{\partial f(x;p)}{\partial x} = p\sum_{k=1}^{+\infty} Q(k)(k-1)(1-x)^{k-2}. \tag{15.5}$$

上述方程是x的单调递减函数。此时有：

$$\begin{aligned}\frac{\partial f(x;p)}{\partial x}|_{x=0} &= p\sum_{k=1}^{+\infty} Q(k)(k-1)\\ &= p\left[\sum_{k=1}^{+\infty} Q(k)k - \sum_{k=1}^{+\infty} Q(k)\right]\\ &= p\left[\sum_{k=1}^{+\infty} \frac{k^2P(k)}{\langle k\rangle} - 1\right]\\ &= p\left(\frac{\langle k^2\rangle}{\langle k\rangle} - 1\right) > 0.\end{aligned} \tag{15.6}$$

图中第二个等号利用了$Q(k)$的定义。

此时我们有个对于函数$f(x;p)$在区间$x \in [0,1]$里的大致图像： $f(x;p)$从$x = 0$时的0单调增加到$x = 1$时的$pQ(1)$，此时其倒数从$x = 0$时单调递减。 对于Eq.(15.6)， 如果$\frac{\partial f(x;p)}{\partial x}|_{x=0} \leqslant 1$， 则此时只有$x = 0$这么一个稳定不动点。对应于此时没有宏观大小的连通单元。 如果$\frac{\partial f(x;p)}{\partial x}|_{x=0} > 1$， 则除了$x = 0$，会出现一个新的稳定不动点对应于$x > 0$。此时对应着出现了一个宏观大小的连通单元。 $\frac{\partial f(x;p)}{\partial x}|_{x=0,p=p^*} = 1$对应着一个临界情形。通过其对应的$p$，我们求出其临界值。此时有：

$$p^*\left(\frac{\langle k^2\rangle}{\langle k\rangle} - 1\right) = 1. \tag{15.7}$$

即：

$$p^* = \frac{\langle k\rangle}{\langle k^2\rangle - \langle k\rangle}. \tag{15.8}$$

*4. ER*随机图上的结果

这里我们考虑Erdös-Rényi随机图上最大连通单元的生成过程。 在ER随机图下，由Eqs.(14.13)- (14.14)可以得知

$$\sum_{k=1}^{+\infty} Q(k)x^{k-1} = \mathrm{e}^{-c(1-x)}, \tag{15.9}$$

$$\sum_{k=0}^{+\infty} P(k)x^{k} = \mathrm{e}^{-c(1-x)}. \tag{15.10}$$

相应地，我们可以得到

$$x = p(1-\mathrm{e}^{-cx}), \tag{15.11}$$

$$n = p(1-\mathrm{e}^{-cx}) = x. \tag{15.12}$$

为简单起见，我们考虑初始稀疏化概率$p = 1$，对应着在空图里不断添加连边的过程。 我们定义Eq.(15.11)的右侧为$f(x;c) \equiv 1-\mathrm{e}^{-cx}$。 此时可以参考第2章里“自洽方程的不动点分析”的计算结果。可以利用自洽方程的稳定不动点来计算对应最大连通单元的比例n。

利用上述自洽方程的求解结果，我们可以得到任意c对应的稳定不不动点x，进而计算最终的n。 Figure 15.5展示了ER随机图上最大连通单元比例n随平均度c的变化。 我们将有限大小的图上模拟计算结果n_{em}和无穷大图（$N = \infty$）的解析计算结果n_{th}画在同一幅结果图里。

容易得知:

（1）当$c < 1$时，此时方程只有一个不动点$x_1 = 0$，且为稳定不动点。 此时$n(= x_1) = 0$。

（2）当$c > 1$，此时方程有了$x_1 = 0$之外的第二个不动点$x_2 > 0$。 $x_1 = 0$变成不稳定的，而x_2是稳定的。 稳定的不动点x_2离0越来越远。 此时对应着$n(= x_2)$逐渐增大。

（3）$c = 1$对应着一种临界情况，

图的平均度c可以用来衡量连边的稀疏程度。上述n随着c的变化趋势有一个简单直观的物理图像。 考虑在只有顶点的空图里不断加入连边的过程。在初始图里，所有的顶点各自组成一个连通单元。 当图中的连边数很少时，绝大部分的连通单元是彼此分离的，而且只是单个顶点或者组成一条连边的两个顶点。随着连边的不断加入，分离的连通单元会彼此连接起来形成更大的连通单元。 但因为已有的连通单元数目多而且分散，新的融合之后的连通单元对于图来说依然是很小的。 随着连边的不断加入，连通单元之间的合并

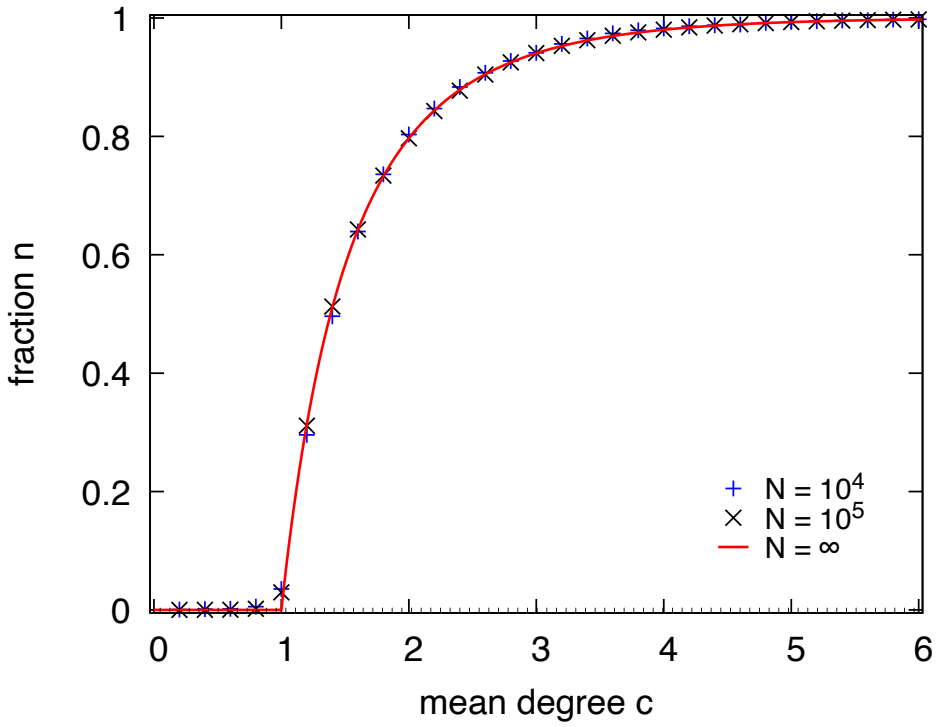


FIG. 15.5. ER随机图上的最大连通单元n。 符号分别对应顶点数$N = 10^4$和10^5的单幅ER随机图上的模拟结果，实线为无穷大图上解析计算的结果。

会持续进行。当加入的连边足够多时，才会出现一个宏观大的连通单元。此时的临界点为$c = 1$。当$c > 1$时， 图中只有一个占据主要比例的连通单元，其余的连通单元相比之下很小。随着连边的加入，别的连通单元会不断加入最大连通单元，n会不断增加。当c很大时，最大连通单元会几乎包含图的所有顶点，即$n \to 1$。

15.3. K-核渗流(K-core percolation)

1. 模型

图的K-核(K-core) [47–49] 表示的是图的一种介观结构，一般定义在无向图上，表示的是图中的一个子图、且每个顶点的度都$\geqslant K$，$K \geqslant 1$。对于一个图，通过不断地删除度$< K$的顶点，则可以得到对应的K-核结构。 Figure 15.6给出了一个图上的K-核及其删除过程。

在代码层面上，Figure 15.7给出了相应的代码计算结果。

和K-核渗流模型密切相关的一个模型是bootstrap percolation [47]。 这里可以从网络上

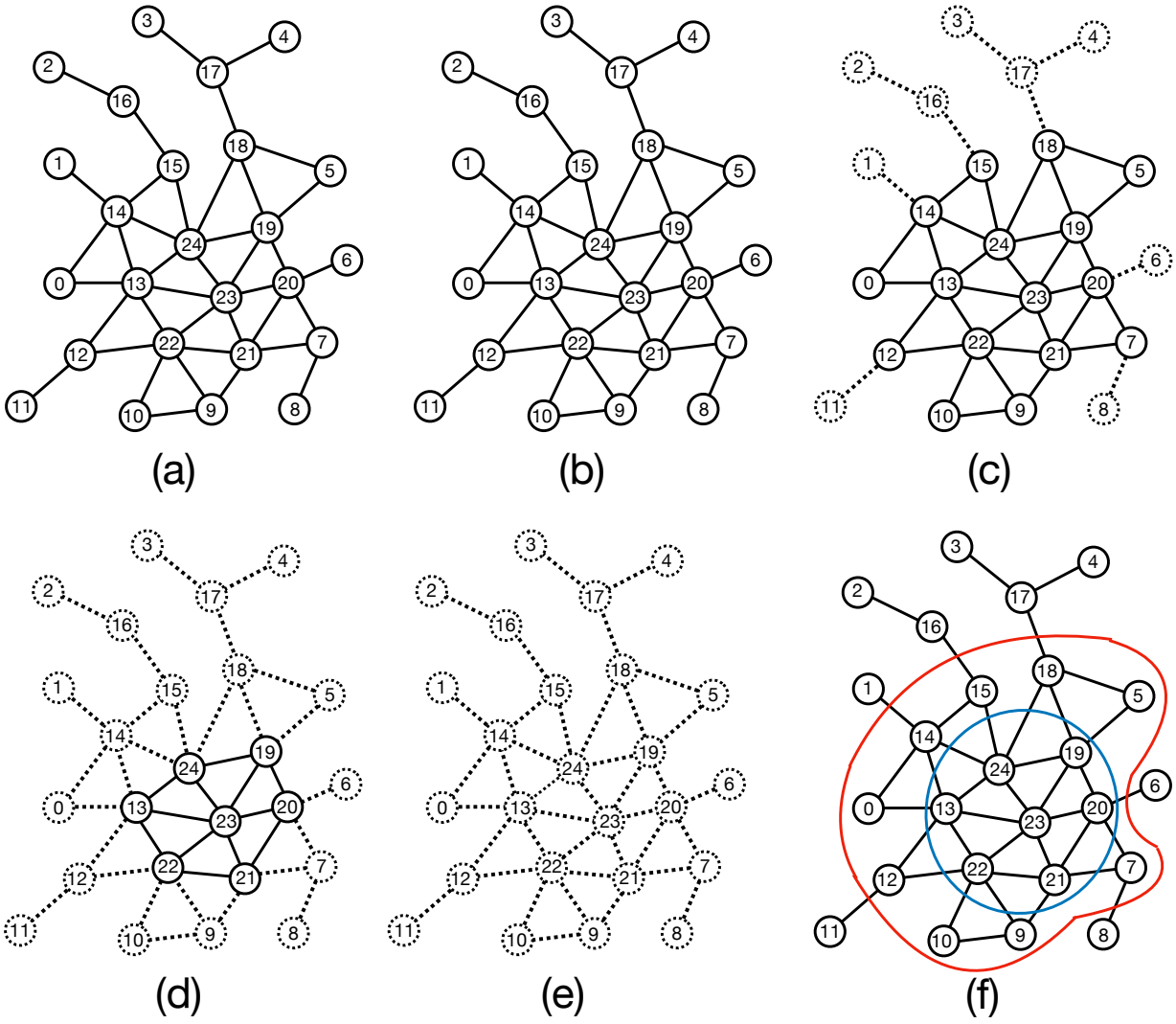


FIG. 15.6. 图上K-核结构的示意图。 (a)为初始的无向图，包含的顶点数$N = 25$、连边数$M = 39$。 (b)-(e)分别为使用迭代删除过程后得到的图的1-核、2-核、3-核、4-核。 用虚线表示的圆圈和实线表示被删除的顶点和连边。 (a)中的图没有度为0的顶点，所以图的1-核就是图本身。此外可知图的4核为空。 (e)中把图的1-核、2-核、3-核同时画了出来。其中2-核用红色实线标记、3-核用蓝色实现标记。

的激发过程(activation process)来理解这个模型。 假设在一个网络上，每个顶点只有两个状态：活跃的(active)和非活跃的(non-active)。而且活跃顶点可以一直处在活跃状态，但从非活跃的状态在一定条件下可以进入活跃状态。 假设在一个图的初始状态里，每个顶点处在二元状态中的一个。之后在状态演化过程里，顶点可以从非活跃状态进入活跃状态，其规则为：如果某个非活跃状态的顶点中有$\geqslant K$个活跃顶点，则这个顶点也进入活跃

```
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % g++ -o ch ch15c-kcore.cc
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % ./ch
@执行K核删除过程。
@检查K核构型：无约束违反。
@K核结构(K = 1):
 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
20 21 22 23 24
 K核内格点总数目： 25
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % g++ -o ch ch15c-kcore.cc
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % ./ch
@执行K核删除过程。
@检查K核构型：无约束违反。
@K核结构(K = 2):
 0  5  7  9 10 12 13 14 15 18 19 20 21 22 23 24
 K核内格点总数目： 16
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % g++ -o ch ch15c-kcore.cc
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % ./ch
@执行K核删除过程。
@检查K核构型：无约束违反。
@K核结构(K = 3):
13 19 20 21 22 23 24
 K核内格点总数目： 7
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % g++ -o ch ch15c-kcore.cc
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % ./ch
@执行K核删除过程。
@检查K核构型：无约束违反。
@K核结构(K = 4):
 K核内格点总数目： 0
```

FIG. 15.7. 图中K-核结构的代码计算结果。 这里依次展示了Figure 15.6中无向图上1-核、2-核、3-核、4-核的顶点数目。

状态。

对于图上的K-核渗流模型，顶点的状态处在被移除或者未移出状态。K-核删除过程对应着对于处在未移出状态的顶点，如果其最近邻中处在未移出状态的顶点数$< K$，则顶点从未移出状态变化为被移除状态。可知，K-核渗流模型和bootstrap模型，都把顶点的度K作为顶点状态改变的阈值。

K-核结构作为图的一种介观结构，首先可以用来描述图的层级结构。 直观上，我们可知，图的K-核结构包含着$(K-1)$-核结构。通过增加K这个参数，图中所有顶点，通过调整K这个参数，图中所有顶点都会被赋予某个K值参数，表示它属于$(K-1)$核、但不属于K核。可以参考Figure 15.6。这种方式成为K-核分解过程(K-core decomposition)[51]。这种方式可以展示图中顶点基于度的层级连接特性。

另外一方面，网络科学的基本思想是网络系统上的动力学和功能都依赖于其结构。上述网络的层级结构也影响网络上的动力学过程，比如网络的稳定性、顶点状态的传播过程等等。比如研究表明，图中的顶点的K-值参数越大，其传播能力越强[52]。网络机构里的影响力传播者(influential spreaders)表示的是其在动力学影响力上有远高于其它顶点的特

殊顶点，而顶点的K-核参数可以视为一个可行的参数指标。

K-核结构中的K可以理解为度的阈值。在最初的K-核定义里，图中每个顶点的阈值K可以选择为同一个值。但我们也可以对每个顶点设置不同的的K值，比如随机地为每个顶点选择某个范围内的阈值，这对应着不均匀的K-核渗流模型(heterogeneous K-core percolation)[53–55]。

2. 解析理论

这里我们推导出来无关联随机图上K-核的解析框架。考虑无向图$G = \{V, E\}$，其顶点集为$V(|V| = N)$、连边集为$E(|E| = M)$。在使用K-核删除过程之前，我们在图中首先使用顶点删除过程：给定初始移除概率p，每个顶点有p的概率被删除。容易得知，图中使用K-核删除过程后，图中顶点要么被移除、要么处在K-核结构中。我们随机选择一条连边$(i, j) \in E$，其中i和j均为图中顶点。从顶点i到顶点j，我们定义空穴概率x为i已经处在K-核中、同时j在K-核结构中的概率。图中的任意顶点，其属于K-核的概率为n。可知顶点属于K-核的概率依赖于其邻居顶点的状态：如果给定顶点在K-核中，从次顶点出发，其邻居中至少有K个最近邻也处在K-核里。我们相应有：

$$n = p\sum_{k=K}^{+\infty} P(k)\sum_{s=K}^{k}\binom{k}{s}x^s(1-x)^{k-s}. \tag{15.13}$$

容易可知，图中的K-核移除过程是一个迭代过程，顶点的状态依赖于其邻居顶点的状态。从这个角度出发，我们可以得到空穴概率x的自洽方程。从x的定义可知，如果j点处在K-核中，则j的邻居中，有$\geqslant K$个顶点处在K-核中；相应地，在邻居$\partial j\backslash i$中，有$\geqslant K-1$个顶点也处在K-核中，因为按照定义顶点i也处在K-核中。我们有：

$$x = p\sum_{k=K}^{+\infty} Q(k)\sum_{s=K-1}^{k-1}\binom{k-1}{s}x^s(1-x)^{k-1-s}. \tag{15.14}$$

Eqs. (15.13)和(15.13)构成了随机图上K-核结构的理论框架的主要内容。通过给定具体的随机图度分布，我们利用Eq. (15.14)计算出来空穴概率x的稳定不动点，然后使用Eq. (15.13)得到相应的n。

利用上述x和m的结果，我们可以得到更多有关K-核的结构信息。如果把K-核结构本

身是为一个图，其度分布$P_{\mathrm{n}}(k)$和平均度c_{n}分别为：

$$P_{\mathrm{n}}(k)=\frac{1}{n}p\sum_{s=k}^{+\infty}P(s)\binom{s}{k}x^{k}(1-x)^{s-k},k\geqslant K, \tag{15.15}$$

$$c_{\mathrm{n}}=\sum_{k=K}^{+\infty}kP_{\mathrm{n}}(k)=\frac{cx^{2}}{n}. \tag{15.16}$$

3. 解析理论公式的简化

使用求和公式的一些等效转换后，我们可以把Eqs. (15.13)和(15.14)简化为如下公式：

$$n=p\left[1-\sum_{s=0}^{K-1}x^{s}\cdot\sum_{k=s}^{+\infty}P(k)\binom{k}{s}(1-x)^{k-s}\right], \tag{15.17}$$

$$x=p\left[1-\sum_{s=0}^{K-2}x^{s}\cdot\sum_{k=s+1}^{+\infty}Q(k)\binom{k-1}{s}(1-x)^{k-1-s}\right]. \tag{15.18}$$

以下为推导过程。

对于Eq.(15.13)，我们有：

$$\begin{aligned}n&=p\sum_{k=K}^{+\infty}P(k)\sum_{s=K}^{k}\binom{k}{s}x^{s}(1-x)^{k-s}\\&=p\sum_{k=K}^{+\infty}P(k)\left[1-\sum_{s=0}^{K-1}\binom{k}{s}x^{s}(1-x)^{k-s}\right].\end{aligned} \tag{15.19}$$

之后，我们把求和形式$\sum_{k=K}^{+\infty}P(k)[\cdot]$写成$\sum_{k=0}^{+\infty}P(k)[\cdot]$。 当$k^{'}\leqslant K-1$，我们有：

$$\begin{aligned}1-\sum_{s=0}^{K-1}\binom{k^{'}}{s}x^{s}(1-x)^{k^{'}-s}&=1-\sum_{s=0}^{k^{'}}\binom{k^{'}}{s}x^{s}(1-x)^{k^{'}-s}\\&=1-(x+1-x)^{k^{'}}\\&=0.\end{aligned} \tag{15.20}$$

最终，我们有：

$$\begin{aligned}n&=p\left\{\sum_{k=0}^{+\infty}P(k)\left[1-\sum_{s=0}^{K-1}\binom{k}{s}x^{s}(1-x)^{k-s}\right]-\sum_{k=0}^{K-1}P(k)\left[1-\sum_{s=0}^{K-1}\binom{k}{s}x^{s}(1-x)^{k-s}\right]\right\}\\&=p\sum_{k=0}^{+\infty}P(k)\left[1-\sum_{s=0}^{K-1}\binom{k}{s}x^{s}(1-x)^{k-s}\right]\\&=p\left[\sum_{k=0}^{+\infty}P(k)-\sum_{k=0}^{+\infty}P(k)\sum_{s=0}^{K-1}\binom{k}{s}x^{s}(1-x)^{k-s}\right]\end{aligned}$$

$$= p\left[1 - \sum_{s=0}^{K-1} x^s \cdot \sum_{k=s}^{+\infty} P(k)\binom{k}{s}(1-x)^{k-s}\right]. \tag{15.21}$$

对于Eq.(15.14)，我们有：

$$\begin{aligned} x &= p\sum_{k=K}^{+\infty} Q(k) \sum_{s=K-1}^{k-1} \binom{k-1}{s} x^s (1-x)^{k-1-s} \\ &= p\sum_{k=K}^{+\infty} Q(k)\left[1 - \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right]. \end{aligned} \tag{15.22}$$

之后我们把求和形式$\sum_{k=K}^{+\infty} Q(k)[\cdot]$写成为$\sum_{k=1}^{+\infty} Q(k)[\cdot]$。考虑到当$k^{'} \leqslant K-1$，容易有$k^{'}-1 \leqslant K-2$。则：

$$\begin{aligned} 1 - \sum_{s=0}^{K-2} \binom{k^{'}-1}{s} x^s (1-x)^{k^{'}-1-s} &= 1 - \sum_{s=0}^{k^{'}-1} \binom{k^{'}-1}{s} x^s (1-x)^{k^{'}-1-s} \\ &= 1 - (x+1-x)^{k^{'}} \\ &= 0. \end{aligned} \tag{15.23}$$

我们继续有：

$$\begin{aligned} x &= p\{\sum_{k=1}^{+\infty} Q(k)\left[1 - \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right] \\ &- \sum_{k=1}^{K-1} Q(k)\left[1 - \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right]\} \\ &= p\left\{\sum_{k=1}^{+\infty} Q(k)\left[1 - \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right] - 0\right\} \\ &= p\sum_{k=1}^{+\infty} Q(k)\left[1 - \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right] \\ &= p\left[\sum_{k=1}^{+\infty} Q(k) - \sum_{k=1}^{+\infty} Q(k) \sum_{s=0}^{K-2} \binom{k-1}{s} x^s (1-x)^{k-1-s}\right] \\ &= p\left[1 - \sum_{s=0}^{K-2} x^s \cdot \sum_{k=s+1}^{+\infty} Q(k)\binom{k-1}{s}(1-x)^{k-1-s}\right]. \end{aligned} \tag{15.24}$$

4. 自洽方程的不动点分析

这里我们考虑K-核结构产生背后的相变行为。

我们把自洽方程Eq.(15.14)的右侧定义为$f(x,p)$。其相对于x的偏导为：

$$\frac{\partial f(x,p)}{\partial x}=p\sum_{k=K}^{+\infty}Q(k)\sum_{s=K-1}^{k-1}\binom{k-1}{s}[sx^{s-1}(1-x)^{k-1-s}-x^s(k-1-s)(1-x)^{k-2-s}]. \quad (15.25)$$

此时我们有：

$$f(x,p)|_{x=0}=0, \quad (15.26)$$

$$f(x,p)|_{x=1}=p\sum_{k=K}^{+\infty}Q(k)x^{k-1}\in[0,1], \quad (15.27)$$

$$\frac{\partial f(x,p)}{\partial x}|_{x=0}=p\sum_{k=K}^{+\infty}Q(k)\sum_{s=K-1}^{k-1}\binom{k-1}{s}[s\cdot 0^{s-1}-(k-1-s)\cdot 0^s]. \quad (15.28)$$

容易可知，不论p的具体取值，$x=0$一直为Eq. (15.14)的不动点。 方程$f(x;p)$的整体图像大致如下：$f(x;p)$从$x=0$对应的$f(0;p)=0$开始增加，一直到$x=1$。 此时方程的不动点行为依赖于 $\frac{\partial f(x;p)}{\partial x}|_{x=0}$的值。 而$\frac{\partial f(x;p)}{\partial x}|_{x=0}$依赖于$K$的值。 考虑以下情况：

▷ 当$K=2$，有$s-1\geqslant 0$。则：

$$\frac{\partial f(x;p)}{\partial x}\Big|_{x=0}=p\sum_{k=2}^{+\infty}Q(k)(k-1)=p\left(\frac{\langle k^2\rangle}{\langle k\rangle}-1\right)>0. \quad (15.29)$$

在临界点处，我们有：

$$\frac{\partial f(x;p)}{\partial x}\Big|_{x=0,p=p^*}=1, \quad (15.30)$$

即：

$$p^*=\frac{\langle k\rangle}{\langle k^2\rangle-\langle k\rangle}. \quad (15.31)$$

上式中p^*为对应的临界点值。

当$p\leqslant p^*$，Eq. (15.14)只有一个不动点$x=0$。 当$p>p^*$，一个非零的不动点出现了，而且为新的稳定不动点。 这里的结果等同于GC在ER随机图上的解析结果。

▷ 当$K\geqslant 3$，我们有：

$$\frac{\partial f(x;p)}{\partial x}\Big|_{x=0}=0. \quad (15.32)$$

假设此时的临界值为p^*。 当$p\leqslant p^*$，我们有$x=0$为唯一的不动点。 由$f(0;p)$和$\partial f(0;p)/\partial x$可知，如果函数$f(x;p)$出现新的不动点，这个不动点$x>0$肯定不是从$x=0$开始慢慢变化的，而是相对于$x=0$有一个跳边。 此时$p^*$满足以下形式：

$$f(x^*;p^*)=x^*, \quad (15.33)$$

$$\frac{\partial f(x;p)}{\partial x}\Big|_{x=x^*,p=p^*}=1. \quad (15.34)$$

通过上述两式可以得到对应的(x^*,p^*)。

5. ER随机图上的结果

在ER随机图上，代入泊松分布形式的度分布，可以得到简化的解析理论框架：

$$x = p\left[1 - \mathrm{e}^{-cx}\sum_{s=0}^{K-2}\frac{(cx)^s}{s!}\right], \tag{15.35}$$

$$n = p\left[1 - \mathrm{e}^{-cx}\sum_{s=0}^{K-1}\frac{(cx)^s}{s!}\right]. \tag{15.36}$$

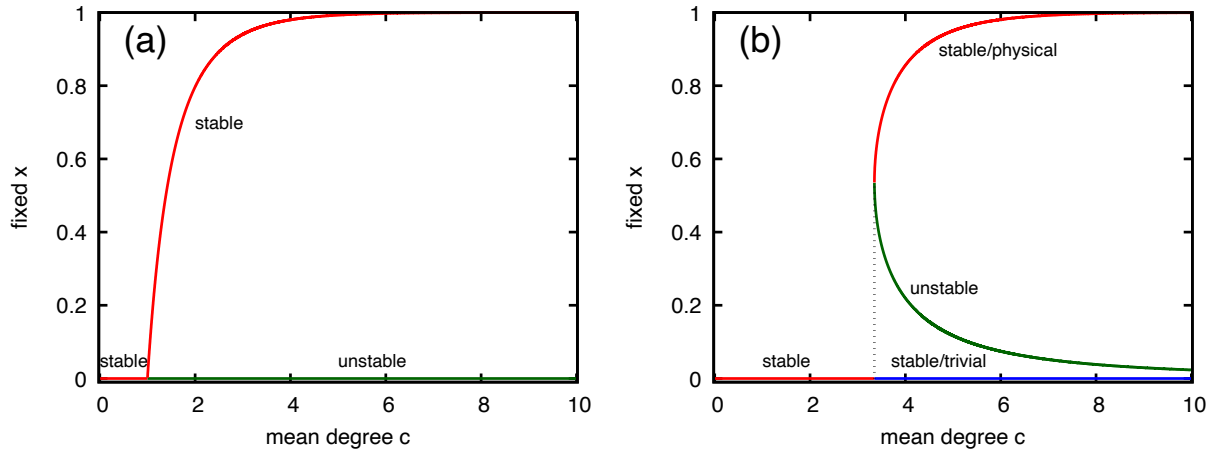


FIG. 15.8. ER随机图上K-核结构的解析理论中的不动点分析。 (a)为$K = 2$时空穴概率x的不动点。此时不动点从一个分支变为两个分支，其分叉点在$c = c^* = 1$。 红色实线对应的较大不动点构成稳定的不动点分支。 深绿色实线对应的较小不动点($x = 0$)构成不稳定的不动点分支。 (b)为$K = 3$时空穴概率x的不动点。此时不动点从一个分支变为三个分支，其分叉点在$c = c^* \approx 3.350$。 红色实线对应的较大不动点($x > 0$)构成稳定的不动点分支。 深绿色实线对应的中间大小不动点($x > 0$)构成不稳定的不动点分支。 蓝色实线对应的较小不动点($x = 0$)也构成了稳定的不动点分支。

Figure 15.8给出了自洽方程Eq.(15.35)中空穴概率x的不动点分析。 当$K = 2$时，x的不动点分叉发生在$c = c^* = 1$，此时不动点从一支变成了两支。当$c > 1$时，较大的不动点为稳定不动点，并且从$c < 1$的稳定不动点$x = 0$开始连续增加。 当$K = 3$时，x的不动点分叉发生在$c = c^* \approx 3.350$，此时不动点分支从一支变成了三支。中间大小的不动点为不稳定的。较大和较小的不动点都为稳定的，但其物理意义不一样。较大的不动点对应$x > 0$，对应着宏观大小的K-核结构。而较小的不动点对应$x = 0$，意味着没有宏观大小的K-核结构，为没有物理意义的结果。因此在计算K-核结构的知识时，我们选择

是较大的不动点分支。可知在这个不动点分支在$c = 3.350$，相对于$c < 3.350$的稳定不动点$x = 0$呈现为一个跳边，即从$x = 0$到$x \approx 0.535655$。这种情形其实发生在$K \geqslant 3$的所有情形。

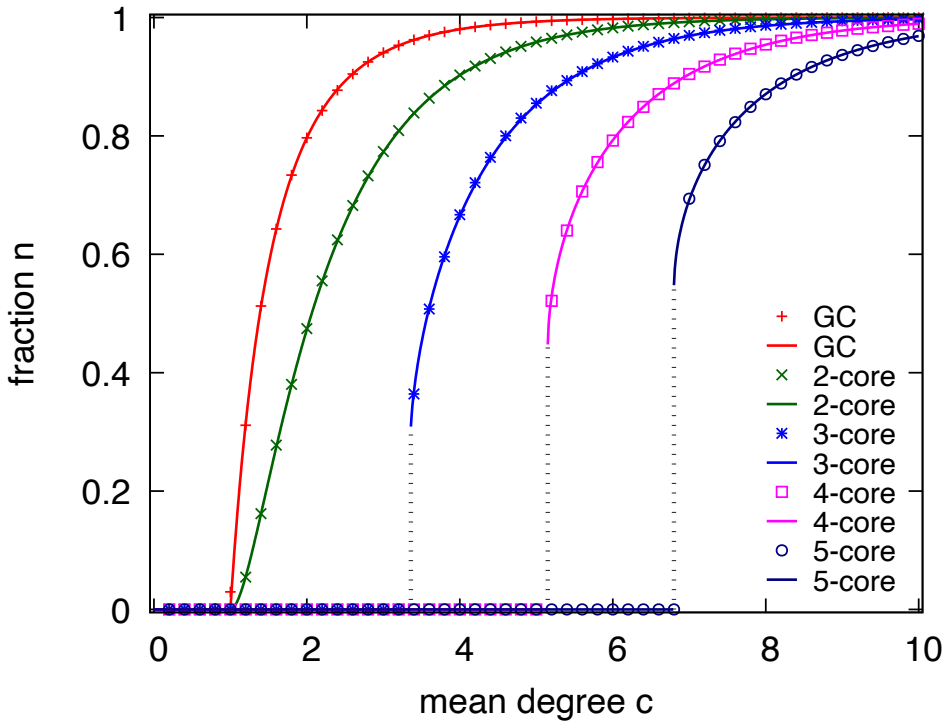


FIG. 15.9. ER随机图上的K-核结构的比例。 图中依次展示了$K = 2, 3, 4, 5$时的结果。 符号对应着顶点数$N = 10^5$的一个图例上的模拟结果。 实线为无穷大图上的解析结果，虚线为无穷大图上的解析结果中的跳变。 作为对比，图中也给出来了ER随机图上最大连通单元产生的模拟和解析结果。

Figure 15.9给出了ER随机图上K-核结构的模拟和解析计算结果。我们也能发现K-核产生的过程比较类似：当c很小时，宏观大小的K-核并不存在；只有当c足够大时，宏观大小的K-核才会产生；随着c的继续增加，K-核的比例趋近于1。 但对于不同的K-核结构，其在诞生点的具体情形不一样。 $K = 2$时，可知2-核结构的产生是连续的。这对应着x的稳定不动点在分叉点是连续地从$x = 0$开始增加的。 当$K \geqslant 3$时，可以看出K-核的产生是一个不连续的、突然产生的过程。这对应着在x的分叉点，新的稳定不动点的产生对应着从$x = 0$开始的一个跳边。

15.4. 核渗流(Core percolation)

1. 模型

无向图上核渗流(core percolation)对应着贪婪移叶算法(Greedy Leaf Removal Procedure, GLR procedure)[56–58].。 无向图中的叶子结点指的是度为1的结点，其唯一的相邻结点成为根节点(root)。 贪婪移叶算法的一个基本步骤是在图中移除叶子结点(leaf node)、以及它的邻居结点及其相邻的连边。 随着结点和连边的移除，图中之前不是叶子结点的节点可能会变成叶子结点。这样贪婪移叶算法可以迭代地进行下去，直到图中不存在叶子结点。最终剩下的子图就是图的核(core)。核结构产生的过程就是核渗流现象。上述过程可以参考Figure 15.10。

贪婪移叶算法的提出本质上是为了在最大匹配问题(Maximum Matching Problem, MM Problem)和最小顶点覆盖问题(Minimum Vertex Cover Problem, MVC problem)作为局域步骤来减少问题中的约束。

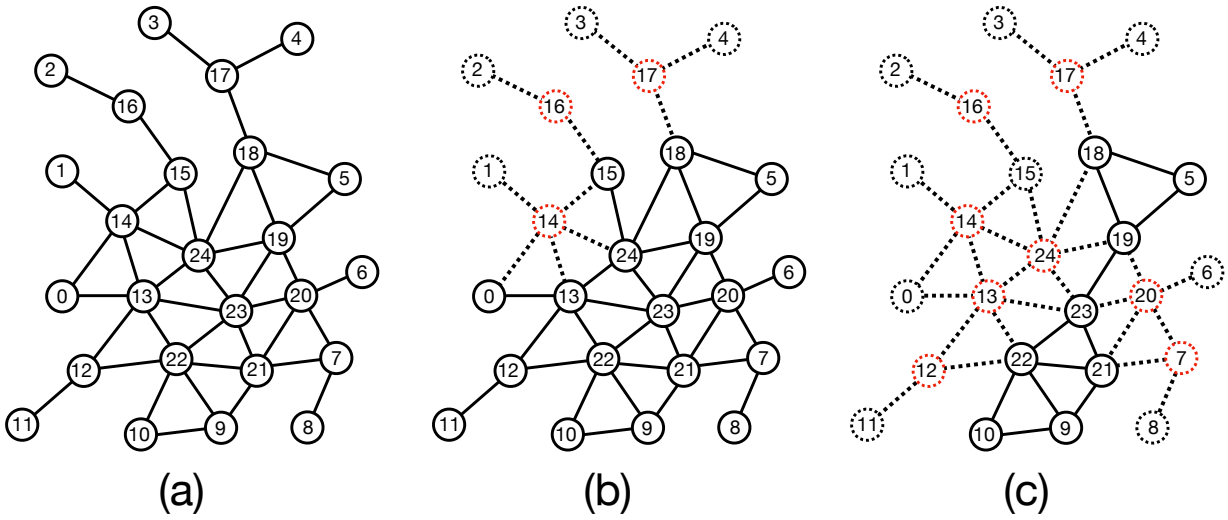


FIG. 15.10. 图的核结构。 (a)为初始的无向图，顶点数$N = 25$、连边数$M = 39$。 (b)分别使用3次贪婪移叶算法的结果。用虚线表示的圆圈和实线表示被删除的顶点和连边，红色方框标识的被移除顶点表示相应贪婪移叶算法的根节点。 (c)为贪婪移叶算法结束后得到的最终结果。最终的核结构包括了8个节点、11条连边。

```
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % g++ -o ch ch15d-core.cc
ZhaoJH@ZhaoJHdeMacBook-Pro 3 code class % ./ch
@执行GLR删除过程。
@检查核结构的构型：无约束违反。
@核结构：
 5  9 10 18 19 21 22 23
 核结构内格点总数目：8
```

FIG. 15.11. Figure 15.10中图例的核结构的结果。

2. 解析理论

对任意的无向图$G=\{V,E\}$（$N=|V|,E=|E|$），通过贪婪移叶算法，我们可以得到相应的核，并计算得到相应的节点数$N_{\rm core}$。容易得知其结点数比例为$n\equiv N_{\rm core}/N$。但如果在随机图上，我们可以从解析角度推导出来n的大小。下面我们给出了相应的理论框架。这里我们考虑没有度度关联(degree-degree correlation)的无向图$G=\{V,E\}$，其结构通过度分布为$P(k)$来描述。在图中，我们随机选择一条连边$(i,j)\in E$，此时i和j都为图中顶点。容易得知图中所有顶点在经过贪婪移叶算法之后，要么为叶子结点被移除、要么为根节点被移除、要么为孤立结点被移除、要么处在最后的核结构中。从顶点i到顶点j，我们定义j以叶子结点的角色被移除的概率为α。此时容易得知i即为j相应的根节点。从顶点i到顶点j，我们同时定义j以根节点的角色的被移除概率为β、且i并不为j相应的叶子结点。

核结构的比例n本质上为边际概率，对应于随机选择图中任意一个顶点$i\in V$、此点属于核结构的概率。顶点i能够处在核结构中，i及其邻居顶点∂i必须满足一些局域约束条件：i不是叶子结点；i不是根节点；对于i的邻居顶点∂i中，可能存在根节点、但这些根节点相应的叶子结点显然不为i；此外，邻居顶点∂i中至少有两个顶点同时处在核结构中。此时n的形式如下：

$$n=\sum_{k=2}^{+\infty}P(k)\sum_{s=2}^{k}\binom{k}{s}(1-\alpha-\beta)^s\beta^{k-s}. \tag{15.37}$$

从上述顶点属于核结构的条件可知，概率(α,β)的定义方式完全基于n的描述。

贪婪移叶算法作为一个迭代算法，叶子结点和根节点的产生是相互关联的。利用这一层关系，我们可以构造出来α和β之间的耦合关系。基于上一段的概念，当顶点j为叶子结点时，可知j的邻居∂j中除了i之外都会被移除、而且是以根节点的形式被移除。而当上述中的顶点j为根结点时，可知j的邻居顶点∂j中除了i至少有一个节点为叶子结点。此时，

随机图上的空穴概率(α, β)之间的自洽方程可以写为：

$$\alpha = \sum_{k=1}^{+\infty} Q(k)\beta^{k-1}, \tag{15.38}$$

$$\beta = \sum_{k=1}^{+\infty} Q(k)[1-(1-\alpha)^{k-1}]. \tag{15.39}$$

上述方程Eqs.(15.37)–(15.39)组成了计算核比例的计算框架。 求解得到Eqs.(15.38)–(15.39)，我们可以得到(α, β)的稳定解。之后就可以利用Eq.(15.37)来得到相应的核比例n。

对于Eq. (15.37)，我们可以进一步简化它的形式。 除去求和符号$\sum_{s=2}^{k}$，我们有：

$$n = \sum_{k=2}^{+\infty} P(k)[(1-\alpha)^k - \beta^k - k(1-\alpha-\beta)\beta^{k-1}]. \tag{15.40}$$

容易看出来，当$k=1$时,

$$(1-\alpha)^k - \beta^k - k(1-\alpha-\beta)\beta^{k-1} = (1-\alpha) - \beta - (1-\alpha-\beta) = 0. \tag{15.41}$$

当$k=0$时,

$$(1-\alpha)^k - \beta^k = 1 - 1 = 0. \tag{15.42}$$

此时有：

$$\begin{aligned} n &= \sum_{k=1}^{+\infty} P(k)[(1-\alpha)^k - \beta^k - k(1-\alpha-\beta)\beta^{k-1}] \\ &= \sum_{k=1}^{+\infty} P(k)[(1-\alpha)^k - \beta^k] - \sum_{k=1}^{+\infty} P(k)k(1-\alpha-\beta)\beta^{k-1} \\ &= \sum_{k=0}^{+\infty} P(k)[(1-\alpha)^k - \beta^k] - \sum_{k=1}^{+\infty} P(k)k(1-\alpha-\beta)\beta^{k-1} \\ &= \sum_{k=0}^{+\infty} P(k)[(1-\alpha)^k - \beta^k] - (1-\alpha-\beta)\cdot\sum_{k=1}^{+\infty} P(k)k\beta^{k-1}. \end{aligned} \tag{15.43}$$

此外，对于上式右边第二个求和，可以得到：

$$\sum_{k=1}^{+\infty} P(k)k\beta^{k-1} = \sum_{k=1}^{+\infty} cQ(k)\cdot\beta^{k-1} = c\cdot\sum_{k=1}^{+\infty} Q(k)\beta^{k-1} = c\alpha, \tag{15.44}$$

上述过程的两个等号分别利用了$Q(k)$本身定义和Eq. (15.38)。 最后，我们有：

$$n = \sum_{k=0}^{+\infty} P(k)[(1-\alpha)^k - \beta^k] - c\alpha(1-\alpha-\beta). \tag{15.45}$$

3. 自洽方程的不动点分析

不动点分析。

4. *ER*随机图上的结果

在ER随机图上，Eqs.(15.38), (15.39), (15.45)可以简化为如下形式：

$$\alpha = \mathrm{e}^{-c(1-\beta)}, \tag{15.46}$$

$$\beta = 1 - \mathrm{e}^{-c\alpha}, \tag{15.47}$$

$$\begin{aligned} n &= \mathrm{e}^{-c\alpha} - \mathrm{e}^{-c(1-\beta)} - c\alpha(1-\alpha-\beta) \\ &= 1 - \alpha - \beta - c\alpha(1-\alpha-\beta). \end{aligned} \tag{15.48}$$

Eq.(15.48)的第二个等号依次利用了Eqs.(15.47)和(15.46)的结果。Eqs.(15.47)和(15.46)可以合并为一个函数，如下：

$$\alpha = \exp(-c\exp(-c\alpha)) \equiv f(\alpha, c), \tag{15.49}$$

其中$f(\alpha, c)$为定义的自洽方程。

ER随机图中α的不动点分析结果可见Figure 15.12。 由Figure 15.12(a)可知，通过曲线$y = f(\alpha, c)$和$y = x$的相交点，可以确定自洽方程$\alpha = f(\alpha, c)$对应的不动点；通过不动点附近$y = f(\alpha, c)$的斜率，进而可以判断不动点的稳定性。 当$c \leqslant \mathrm{e} \approx 2.718$时，可知自洽方程只有一个不动点。此时对应$1 - \alpha - \beta = 0$。 当$c > \mathrm{e}$时，唯一的不动点分支分成了三支。 其中中间大小的分支（对应着$1 - \alpha - \beta = 0$）依然为不动点，但变成了不稳定的。较大和较小的不动点分支都为稳定不动点，但两者在物理意义上是不同的。 较小不动点的分支对应着$1 - \alpha - \beta > 0$，是有物理意义的分支，因为核结构的比例$n \sim 1 - \alpha - \beta$。较大不动点的分支对应着$1 - \alpha - \beta < 0$，为无物理意义的分支。因此在计算中，我们使用$\alpha$不动点中的较小者来计算核结构的性质。由图可知，$c = \mathrm{e}$处，新的不动点分支的出现是一个连续的过程。

Figure 15.13给出了ER随机图上核结构的比例n的计算结果。 容易得知，当$c \leqslant \mathrm{e}$时，ER随机图上无宏观大小的核结构，因为$n \sim 1 - \alpha - \beta = 0$。当$c > \mathrm{e}$时，图中出现了宏观大小的核结构，因为$n \sim 1 - \alpha - \beta > 0$。和不动点分析的结果作对比，容易可知宏观核结构的出现过程是连续的。

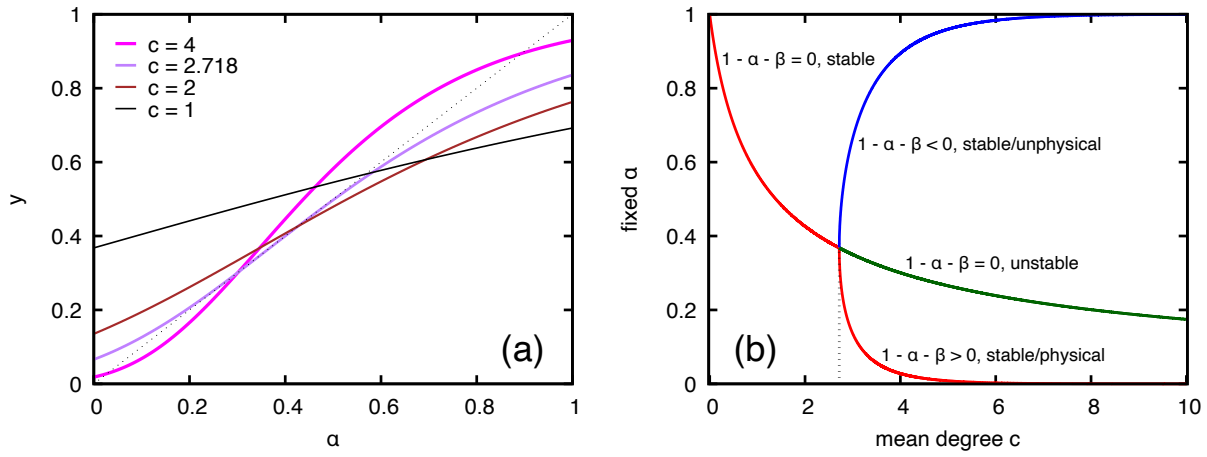


FIG. 15.12. ER随机图上核结构的解析理论中的不动点分析。 (a)展示了随着c的增加，自洽方程$f(\alpha, c)$的变化。四个不同的c对应着1个、1个、1个、3个不动点。 (b)展示了随着c的增加，空穴概率α的不动点的变化。可知不动点从一支分成了三支，分叉点在$c = c^* = \mathrm{e}$。 不动点分支附近同时标记出来了不动点α对应的$1-\alpha-\beta$的值。 红色实线对应的较小不动点构成了一支稳定的不动点分支。 深绿色实线对应的中间大小不动点构成了不稳定的不动点分支，对应着$1-\alpha-\beta=0$这个平庸解。 蓝色实线对应的较大不动点也构成了一支稳定的不动点分支，但对应着$1-\alpha-\beta<0$，即为非物理结果。

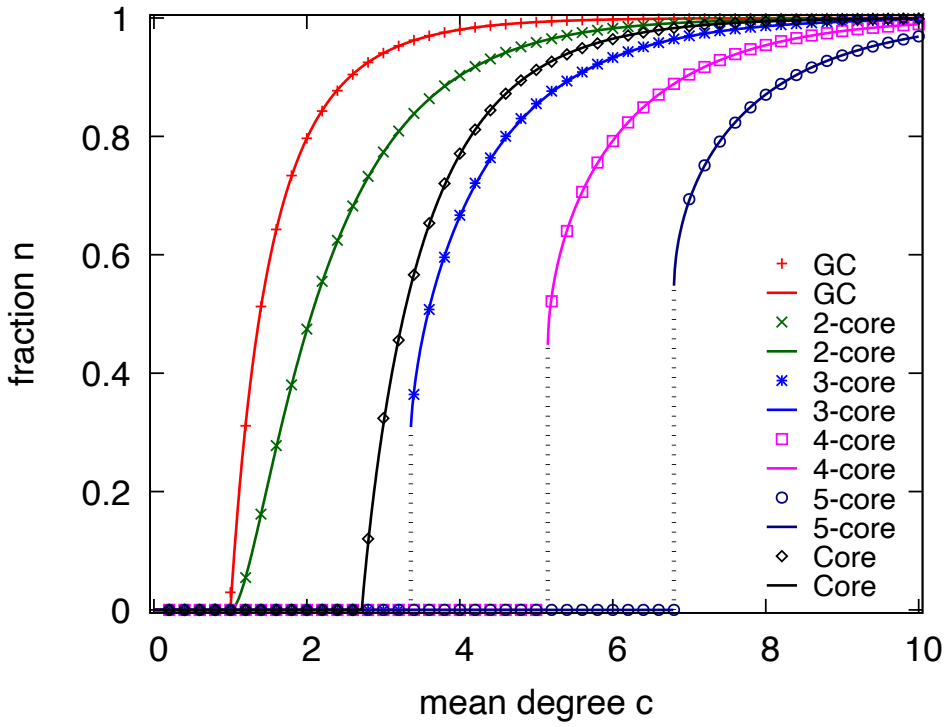


FIG. 15.13. ER随机图上的核结构的比例。 符号对应着顶点数$N = 10^5$的一个图例上的模拟结果，实线为无穷大图上的解析结果。 作为对比，图中也展示了最大连通单元和K-核的模拟和解析结果，其中虚线为K-核结构的解析计算结果中的跳边。

15.5. 有向图的强连通单元

1. 模型

在第13章里我们介绍了有向图的强连通单元(Strongly Connected Components, SCC)的概念。对于有向图$D=\{V,A\}$，其顶点集为$V(|V|=N)$、连边集为$A(|A|=M)$。有向图的SCC指的是有向图的子图，对于其中每个顶点，其至少有一个入节点、至少一个出节点。从另外一个角度来说，有向图中的所有有向环路的集合即为SCC。假设其SCC中的顶点数为$N_{\rm scc}$，则其顶点数比例为$n=N_{\rm scc}/N$。

为了得到给定有向图的SCC，可以迭代地删除图中所有入度为0或者出度为0的顶点。这个删除过程结束后，得到的子图就对应有向图的SCC结构。

2. 解析理论

在随机图有向图上，我们其实可以解析地计算出来SCC的比例n随着图结构参数的变化。其解析理论可参考[60]。对于随机有向图D，其度分布为$P(k_{\rm in},k_{\rm ou})$。在图D中我们随机选择任意一条连边$\langle i,j\rangle$，即顶点i指向顶点j的有向连边。在这条连边上，我们可以定义两个空穴概率(α,β)：从j到i（此时逆着连边的方向），i处在SCC中的概率为α；从i到j（此时顺着连边的方向），j处在SCC中的概率为β。如Figure 15.14所示。

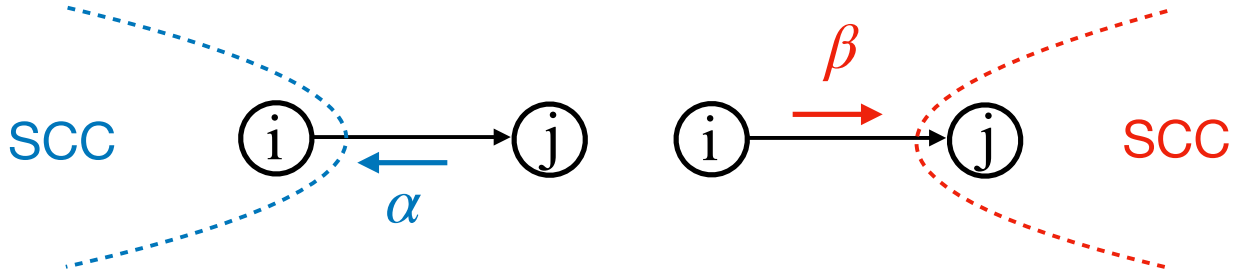


FIG. 15.14. 有向ER随机图上SCC的解析理论中的空穴概率。空穴概率α和β定义在图中任意选择的一条有向连边$\langle i,j\rangle$上。

SCC中顶点数的比例n可以视为一个边际概率，即任意选择一个顶点、此顶点处在SCC中的概率。对于任意顶点$i\in V$，如果它处在SCC中，其邻居必须满足一定的条件：它的入邻居中，至少有一个邻居顶点处在SCC中；同时其出邻居中，至少有一个顶点

处在SCC中。相应地可以求得n对于(α,β)的依赖关系。n的形式如下：

$$n=\sum_{k_{\rm in}=1,k_{\rm ou}=1}^{+\infty}P(k_{\rm in},k_{\rm ou})[1-(1-\alpha)^{k_{\rm in}}][1-(1-\beta)^{k_{\rm ou}}]. \tag{15.50}$$

我们可以继续得到α和β的耦合方程。如下：

$$\alpha=\sum_{k_{\rm in}=1,k_{\rm ou}=1}^{+\infty}Q_{\rm ou}(k_{\rm in},k_{\rm ou})[1-(1-\alpha)^{k_{\rm in}}], \tag{15.51}$$

$$\beta=\sum_{k_{\rm in}=1,k_{\rm ou}=1}^{+\infty}Q_{\rm in}(k_{\rm in},k_{\rm ou})[1-(1-\beta)^{k_{\rm ou}}]. \tag{15.52}$$

Eq. (15.51)对应的情形为i有至少一个入节点处于SCC。因为顶点i已经有了出顶点邻居j，那么此时i必然处在SCC中。 Eq. (15.52)对应的情形为j有至少一个出节点处于SCC。因为顶点j已经有了入顶点邻居i，那么此时j必然处在SCC中。

对于给定任意度分布的$P(k_{\rm in},k_{\rm ou})$的稀疏随机图，通过以上Eqs.(15.51)和(15.52)，可以得到(α,β)的稳定解，由此利用Eq.(15.50)可以得到相应的n。

可以发现，SCC的解析理论里，空穴概率α的自洽方程Eqs. (15.51)、β的自洽方程Eq.(15.52)和无向图GC理论里空穴概率x的自洽方程Eq.(15.2)的形式是相同的。 这意味着，在稀疏随机图上，SCC和GC的在临界点附近的行为是相同的。

*3. ER*随机图上的结果

在ER随机图上，Eqs. (15.50) - (15.52)对应着：

$$n=(1-{\rm e}^{-c\alpha})(1-{\rm e}^{-c\beta}), \tag{15.53}$$

$$\alpha=1-{\rm e}^{-c\alpha}, \tag{15.54}$$

$$\beta=1-{\rm e}^{-c\beta}. \tag{15.55}$$

上述方程组可以简化为：

$$n=\alpha^2, \tag{15.56}$$

$$\alpha=1-{\rm e}^{-c\alpha}. \tag{15.57}$$

Figure 15.15展示了ER随机图上的模拟和解析计算的结果。 可知当$c\leqslant c^*=1$时，图中没有宏观大小的SCC。当$c>1$时，图中才有宏观大小的SCC，而且随着c的增加不断增

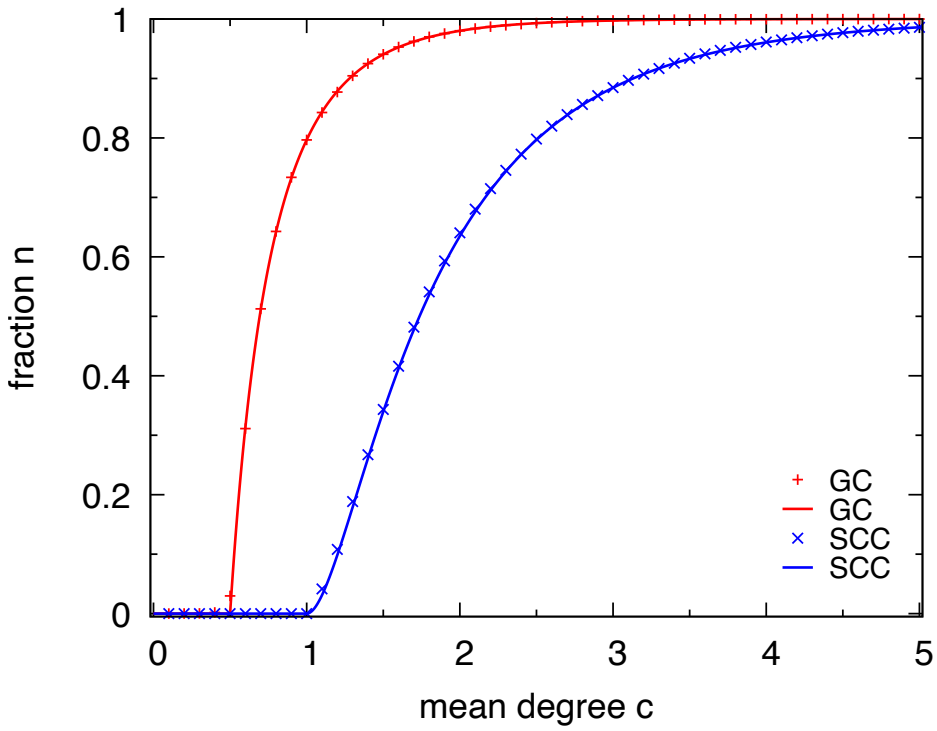


FIG. 15.15. 有向ER随机图上的SCC的比例。 符号对应着顶点数$N = 10^5$的一个图例上的模拟结果，实线为无穷大图上的解析结果。 图中还展示了最大连通单元（GC）在平均度为$2c$的无向ER随机图的模拟和解析结果。

加、并趋近于1。容易可知，SCC的产生过程是连续的。这和最大连通单元的产生的情形是相同的。

Figure 15.15还展示了无向图中最大连通单元（GC）的计算结果。 无向图上的GC和有向图上的SCC的结果比较可以理解为：在一个给定的平均度为$2c$的无向ER随机图上，可以计算其应的GC；如果对图中的连边随机地赋予方向，可以得到其对应的有向图，进而得到相应的SCC。 容易可知，确定c的值之后，SCC的大小明显要小于GC，而且其临界点$c^* = 1$大于GC的临界点$c^* = 0.5$。 GC和SCC都能用来度量图的宏观连接性。但当连边被赋予方向之后，SCC为一个更加紧致的结构。

15.6. Random walks

For a graph $G = \{V, E\}$, the adjacency matrix $\vec{A}_{N\times N}$. $\vec{A}_{ij} = 1$ if the edge (i, j) exists, and 0 otherwise.

Here is a correspondence between a Markov chain and a random walk on a graph.

(1) states $\{\vec{s}\} \leftrightarrow N$ vertices.

(2) state transition matrix $\vec{p} \leftrightarrow s_{ij}$ as the row normalized A_i as $S_{ij} = \frac{A_{ij}}{\sum_{j\in\partial i} A_{ij}}$.

(3) a stationary state by the Markov chain $\leftrightarrow$ a stationary state of a random walk.

Why we study random walks on a graph?

A reason: to evaluate the relative importance of nodes based on the stationary values of the random walks.

How about biased random walks on a graph?

Google matrix and PageRank[62].

They are the core algorithms for the Google search engine at its inception, still adopted in may versions of search engines and recommender systems.

The Google matrix $\vec{G}$ is derived as

$$G_{ij} = \alpha S_{ij} + (1-\alpha)\frac{1}{N}, \tag{15.58}$$

which are further row-normalized. The first term on the RHS of the above equation is the exploitation term, which shows the tendency of G_{ij} to approach S_{ij}. The second term on the RHS is the exploration term, a stochastic term as each node other than i can be accessed. The parameter α is the damping factor. When $\alpha = 1$, the matrix reduces to a random walk. The eigenvector corresponds to the stationary state of the random walks.

In the context of search engines, the nodes are webpages, the links as hyperlinks through clicks, the keywords in queries are also considered. The stationary values of nodes of the Google matrix is a measure of the importance of the webpages, which are the results for queries.

习题

15.1 ER随机图上的最大连通单元

这里我们考虑给定的一幅图在随机删除顶点时最大连通单元的变化。

考虑一副随机生成的ER随机图，其顶点数为N、平均度为c（比如$c = 10$）。考虑从图中随机删除节点的过程：给定一个残留概率$p \in [0, 1]$，图中每个顶点以$1 - p$的概率被删除掉。在删除过程之后的残留子图上，我们可以求解其最大连通单元的顶点数和初始顶点数N的比例n。

这里将从模拟和解析两个角度来研究上述问题。 在模拟上，请使用遍历算法得到给定一幅图（比如$N = 10^6$）上不同p对应的$N_{\rm gc}$、以及相应比例n。在解析上，请使用课件中的理论模型计算无穷大图上，不同的p所对应的n。作图时，请将上述两类结果放在一副图中、并进行比较。在模拟结果中，p的步长可选择为$\Delta p = 0.02$。在解析结果中，p的步长可选择为$\Delta p = 0.0001$。

15.2 ER随机图上的K-核结构

给定$c = 10$的ER随机图，选择不同的p，然后计算相应的K-核结构的大小。在同一个结果图里，放入模拟结果和解析结果。其中模拟结果，可以考虑顶点数$N = 10^6$的图例。

15.3 ER随机图上的核结构

重复计算结果Figure 15.12和15.13。

15.4 ER随机图上的强连通单元

重复计算结果Figure 15.15。

16.

16.1. 平时作业的安排

- 第1次平时作业（覆盖第3、4章）

  作业题目：讲义习题3.3, 4.1(15)(18)(19)(20), 4.2(1)(5)

  发布时间：第2周周四（9月17日）

  到期时间：第5周周二（10月6日）23:59:00

  截止时间：第5周周日（10月11日）23:59:00

- 第2次平时作业（覆盖第5章）
  - 作业题目：讲义习题5.8、5.15
  - 发布时间：第5周周六（10月10日）
  - 到期时间：第7周周二（10月20日）23:59:59
  - 截止时间：第7周周六（10月24日）23:59:00

- 第3次平时作业（覆盖第6 ∼ 9章）
  - 作业题目：讲义习题6.2、7.3、8.2、9.1
  - 发布时间：第9周周四（11月5日）
  - 到期时间：第11周周二（11月17日）23:59:00
  - 截止时间：第11周周六（11月21日）23:59:00

- 第4次平时作业（覆盖第11 ∼ 13章）
  - 作业题目：讲义习题11.3、12.2、13.5
  - 发布时间：第15周周五（12月18日）
  - 到期时间：第16周周三（12月23日）23:59:00
  - 截止时间：第17周周一（12月28日）23:59:00

16.2. 实验课作业的安排

- 学期第1周周三/周四（9月9日/10日）【大数据6班/5班】
  作业题目：讲义习题2.1 “安装常用的工具”

- 学期第2周周三/周四（9月16日/17日）【大数据6班/5班】
  作业题目：讲义习题2.1 “安装常用的工具”

- 学期第3周周三/周四（9月23日/24日）【大数据6班/5班】
  作业题目：讲义习题5.1 “课程分数的统计性质”
  发布题目：9月23日（周三）
  到期/截止时间：9月27日（周日）23:59:00 → 9月28日（周一）23:59:00

- 学期第4周周三（9月30日）【大数据6班】
  作业题目：讲义习题5.3 “二分查找法（版本A）的执行过程”
  到期/截止时间：10月3日（周六）23:59:00

- 学期第4周周四（10月1日）【大数据5班】

<因国庆节放假取消>

- 学期第5周周三（10月7日）【大数据6班】

<因国庆节放假调休，补课时间为学期第5周周三（10月10日）>

- 学期第5周周四（10月8日）【大数据5班】
  作业题目：讲义习题5.11 “冒泡排序算法的执行过程”
  到期/截止时间：10月10日（周六）23:59:00

- 学期第5周周六（10月10日）【大数据6班】

<因国庆节放假调休，补学期第5周周三（10月7日）的课>
  作业题目：讲义习题5.12 “反向冒泡排序算法”
  注：此题自愿完成，不计算在实验课分数内。

- 学期第6周周三（10月14日）【大数据6班】
  - 作业题目：讲义习题5.14 “快速排序算法里切分函数的执行过程”之第1小题
  - 到期/截止时间：10月16日（周五）23:59:00

- 学期第6周周四（10月15日）【大数据5班】

- 作业题目：讲义习题5.14 “快速排序算法里切分函数的执行过程”之第2小题
- 到期/截止时间：10月17日（周六）23:59:00

- 学期第7周周三（10月21日）【大数据6班】
  - 作业题目：讲义习题6.1 “列表中插入和删除节点的操作”之第1、3小题
  - 到期/截止时间：10月23日（周五）23:59:00

- 学期第7周周四（10月22日）【大数据5班】
  - 作业题目：讲义习题6.1 “列表中插入和删除节点的操作”之第2、3小题
  - 到期/截止时间：10月24日（周六）23:59:00

- 学期第8周周三（10月28日）【大数据6班】
  - 作业题目时间：讲义习题7.1 “Next表的构造”之第5小题
  - 到期/截止时间：10月30日（周五）23:59:00

- 学期第8周周四（10月29日）【大数据5班】
  - 作业题目时间：讲义习题7.1 “Next表的构造”之第2小题
  - 到期/截止时间：10月31日（周六）23:59:00

- 学期第9周周三（11月4日）【大数据6班】
  - 作业题目：讲义习题7.2 “更新版Next表的构造”之第5小题
  - 到期/截止时间：11月6日（周五）23:59:00

- 学期第9周周四（11月5日）【大数据5班】
  - 作业题目：讲义习题7.2 “更新版Next表的构造”之第2小题
  - 到期/截止时间：11月7日（周六）23:59:00

- 学期第10周周三（11月11日）【大数据6班】
  - 作业题目：讲义习题8.3 “寻找满足给定条件的所有栈混洗序列”之第2小题
  - 到期/截止时间：11月13日（周五）23:59:00

- 学期第10周周四（11月12日）【大数据5班】
  - 作业题目：讲义习题8.3 “寻找满足给定条件的所有栈混洗序列”之第4小题
  - 到期/截止时间：11月14日（周六）23:59:00

- 学期第11周周三/周四（11月18日/19日）【大数据6班/5班】
  - 作业题目：讲义习题11.1 “二叉树的构型”

- 到期/截止时间：11月20日（周五）23:59:00

- 学期第12周周三/周四（11月25日/26日）【大数据6班/5班】
  - 作业题目：讲义习题11.6 “从二叉树的遍历序列反推其构型”
  - 到期/截止时间：11月27日（周五）23:59:00

- 学期第13周周三/周四（12月2日/3日）【大数据6班/5班】
  - 作业题目：讲义习题12.1 “AVL树的手动构造”
  - 到期/截止时间：12月4日（周五）23:59:00

- 学期第14周周三/周四（12月9日/10日）【大数据6班/5班】
  - 作业题目：讲义习题13.1 “图的结构性质”
  - 到期/截止时间：12月11日（周五）23:59:00

- 学期第15周周三/周四（12月16日/17日）【大数据6班/5班】
  - 作业题目：讲义习题13.3 “图上格点间的最短距离的算法示意图”
  - 到期/截止时间：12月18日（周五）23:59:00

- 学期第16周周三/周四（12月23日/24日）【大数据6班/5班】
  - 作业题目：讲义习题13.4 “图的最小生成树的算法示意图”
  - 到期/截止时间：12月25日（周五）23:59:00

[1] Robert Sedgewick and Kevin Wayne, *Algorithms*, Fourth Edition, (Pearson Education, 2011).

[2] Jon Kleinberg and Éva Tardos *Algorithm Design*, (Pearson Education, Inc., 2006).

[3] Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani, *Algorithms*, (McGraw-Hill Higher Education, 2008).

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, Fourth Edition, (The MIT Press, 2022).

[5] 郭光灿，“量子信息技术研究现状与未来”，《中国科学：信息科学》，2020年。
https://www.sciengine.com/SSI/doi/10.1360/SSI-2020-0112

[6] 潘建伟，”量子信息科技的发展现状与展望”，《物理学报》 **73**, 010301 (2024).
https://wulixb.iphy.ac.cn/article/doi/10.7498/aps.73.20231795

[7] J. V. Uspensky, *Introduction to Mathematical Probability* (McGraw-Hill, New York, 1937).

[8] G.-L. Buffon, Essai d'arithmétique morale, Histoire naturelle, générale et particulière, Supplément 4, 46–123 (1777).

[9] P.-S. Laplace, *Théorie analytique des probabilités* (Veuve Courcier, Paris, 1812).

[10] P.-S. Laplace, *Théorie analytique des probabilités*, Third Revised Edition (Veuve Courcier, Paris, 1820).

[11] David P. Landau and Kurt Binder, *A Guide to Monte Carlo Simulations in Statistical Physics*, Fourth Edition (Cambridge University Press, Cambridge, 2015).

[12] Yan-Jie Min, De-Quan Zhu, and Jin-Hua Zhao, Buffon-Laplace Needle Problem as a geometric probabilistic approach to filtration process, Physica A 657, 130253 (2025).

[13] M. C. Cross and P. C. Hohenberg, Pattern formation outside of equilibrium, *Reviews of Modern Physics* **65**, 851–1112 (1993)

[14] Tamás Vicsek and Anna Zafeiris, Collective motion, *Physics Reports* **517**, 71–140 (2012).

[15] Tamás Vicsek, András Czirók, Eshel Ben-Jacod, Inon Cohen, and Ofer Shochet, Novel Type of Phase Transition in a System of Self-Driven Particles, *Physical Review Letters* **75**, 1226–1229 (1995).

[16] Stephen Cook, The Complexity of Theorem Proving Procedures, in Proc. 3rd Annual ACM Symposium on Theory of Computing (STOC), pp. 151–158 (1971).

[17] Richard M. Karp, Reducibility Among Combinatorial Problems, in R. E. Miller, J. W.

Thatcher, J. D. Bohlinger (eds.). Complexity of Computer Computations. New York: Plenum, pp. 85–103 (1972).

[18] Michael R. Garey and David S. Johnson *Computers and Intractability: A Guide to the Theory of NP-Completeness* (W H Freeman, New York, 1979).

[19] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Summer Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermanni Heimonen, Jakob S. Kottmann, Tim Menke, Wai-Keong Mok, Sukin Sim, Keong-Chuan Kwek, and Alán Aspuru-Guzik, Noisy intermediate-scale quantum algorithms, Reviews of Modern Physics **94**, 015004 (2022).

[20] Christos H. Papadimitriou and Kenneth Steiglitz, *Combinatorial Optimization: Algorithms and Complexity* (New York: Dover Publications, 1998).

[21] Panos M Pardalos, Ding-Zhu Du and Ronald L Graham, *Handbook of Combinatorial Optimization* Second Edition (New York: Springer, 2013).

[22] C. A. R. Hoare, Quicksort, The Computer Journal **5**, 10–16 (1962)

[23] Donald E. Knuth, James H. Morries, Jr., and Vaughan R. Pratt, Fast pattern matching in strings, SIAM Journal of Computing **6**, 323–350 (1977)

[24] Robert S. Boyer and J. Strother Moore, A fast string searching algorithm, Communications of the ACM **20**, 762–772 (1977)

[25] Béla Bollobás, *Modern Graph Theory*, (New York: Springer, 2002).

[26] Mark E. J. Newman, *Networks* 2nd Edition, (New York: Oxford University Press, 2018).

[27] S. Boccaletti, V. Latora, Y. Moreno, M. Chavez, and D.-U. Hwang, Complex networks: Structure and dynamics, Physics Reports **424**, 1175–308 (2006).

[28] Valdis E. Krebs, Mapping Networks of Terrorist Cells, Connections **24**, 43–52 (2002).

[29] Liang Tian, Amir Bashan, Da-Ning Shi, and Yang-Yu Liu, Articulation points in complex networks, Nature Communications **8**, 14223 (2017).

[30] Ang-Kun Wu, Liang Tian, and Yang-Yu Liu, Bridges in complex networks, Phys. Rev. E **97**, 012307 (2018).

[31] Santo Fortunato, Community detection in graphs, Physics Reports **486**, 75–174 (2010)

[32] Wayne Zachary, An information flow model for conflict and fission in small groups, Journal of Anthropological Research **33**, 452–473 (1976).

[33] Paul Erdös and Alfréd Rényi, On random graphs, I, Publ. Math. **6**, 290–297 (1959).

[34] Paul Erdös and Alfréd Rényi, On the evolution of random graphs, Publ. Math. Inst. Hung.

Acad. Sci. **5**, 17–61 (1960).

[35] `http://www.gnu.org/software/gsl/`

[36] Duncan Watts, Steven Strogatz, Colletive dynamis of 'small-world' networks, Nature **393**, 440–442 (1998).

[37] Albert-László Barabási and Réka Albert, Emergence of Scaling in Random Networks, *Science* **286**, 509–512 (1999).

[38] K.-I. Goh, B. Kahng, and D. Kim, Universal behavior of load distribution in scale-free networks, Phys. Rev. Lett. **87**, 278701 (2001).

[39] M. Catanzaro and R. Pastor-Satorras, Analytic solution of a static scale-free network model, Eur. Phys. J. B **44**, 241–248 (2005).

[40] J.-S. Lee, K.-I. Goh, B. Kahng, and D. Kim, Intrinsic degree-correlations in the static model of scale-free networks, Eur. Phys. J. B **49**, 231–238 (2006).

[41] M. Buguñá, R. Pastor-Satorras, and A. Vespignani, Cut-offs and finite size effects in scale-free networks, Eur. Phys. J. B **38**, 205–209 (2004).

[42] Marten Scheffer, *Critical Transitions in Nature and Society* (Princeton University Press, Princeton, 2009).

[43] Dietrich Stauffer and Amnon Aharony, *Introduction to Percolation Theory* Revised 2nd Edition (London: Taylor & Francis, 1994).

[44] Ming Li, Run-Ran Liu, Linyuan Lü, Mao-Bin Hu, Shuqi Xu, and Yi-Cheng Zhang, Percolation on complex networks: Theory and application, Physics Reports **907**, 1-68 (2021).

[45] Mark E. J. Newman, Steven H. Strogatz, and Duncan J. Watts, Random graphs with arbitrary degree distributions and their applications, Physical Review E **64**, 026118 (2001).

[46] Michael Molloy and Bruce Reed, The size of the giant component of a random graph with a given degree sequence, Combinal. Probab. Comput. **7**, 295 (1998).

[47] J. Chalupa, P. L. Leath, and G. R. Reich, Bootstrap percolation on a Bethe lattice, Journal of Physics C: Solid State Physics **12**, L31-5 (1979).

[48] B. Pittel, J. Spencer, and N. Wormald, Sudden Emergence of a Giant k-Core in a Random Graph, Journal of Combinatorial Theory B **67**, 111 (1996).

[49] S. N. Dorogovtsev, A.V. Goltsev, J. F. F. Mendes, k-core organization of complex networks, Physical Review Letters **96** 040601 (2006).

[50] G. J. Baxter, S. N. Dorogovtsev, K.-E. Lee, J. F. F. Mendes, and A. V. Goltsev, Critical

dynamics of the k-core pruning process, Phys. Rev. X **5**, 031017 (2015).

[51] Shai Carmi, Shlomo Havlin, Scott Kirkpatrick, Yuval Shavitt, and Eran Shir, A model of Internet topology using k-shell decomposition, Proc. Natl. Acad. Sci. of USA **104**, 11150–11154 (2007).

[52] Maksim Kitsak, Lazaros K. Gallos, Shlomo Havlin, Fredrik Liljeros, Lev Muchnik, H. Eugene Stanley, and Hernán A. Makse, Identification of influential spreaders in complex networks, Nature Physics **6**, 888–893 (2010).

[53] G.J. Baxter, S. N. Dorogovtsev, A. V. Goltsev, and J. F. F. Mendes, Heterogeneous k-core versus bootstrap percolation on complex networks, Phys. Rev. E **83**, 051134 (2011).

[54] Davide Cellai, Aonghus Lawlor, Kenneth A. Dawson, and James P. Glesson, Tricritical Point in Heterogeneous k-Core Percolation, Phys. Rev. Lett. **107**, 175703 (2011).

[55] Davide Cellai, Aonghus Lawlor, Kenneth A. Dawson, and James P. Glesson, Critical phenomena in heterogeneous k-core percolation, Phys. Rev. E **87**, 022134 (2013).

[56] R. M. Karp and M. Sipser, Maximum matchings in sparse random graphs in *Proceedings of the 22nd IEEE Annual Symposium on Foundations of Computer Science* (Nashville, TN, USA, October 1981) pp 364-375 (1981).

[57] M. Bauer and O. Golinelli, Core percolation in random graphs: A critical phenomena analysis, European Physical Journal B **24** 339-352 (2001).

[58] Yang-Yu Liu, Endre Csóka, Hai-Jun Zhou, and Márton Pósfai, Core Percolation on Complex Networks, Physical Review Letters **109**, 205703 (2012).

[59] Jin-Hua Zhao and Hai-Jun Zhou, Two faces of greedy leaf removal procedure on graphs, Journal of Statistical Mechanics 083401 (2019).

[60] S. N. Dorogovtsev, J. F. F. Mendes, and A. N. Samukhin, Giant strongly connected component of directed networks, Physical Review E **64**, 025101(R) (2001).

[61] G. Timár, A. V. Goltsev, S. N. Dorogovtsev, and J. F. F. Mendes, Mapping the structure of directed networks: beyond the bow-tie diagram, Physical Review Letters **118**, 078301 (2017).

[62] Sergei Brin and Larry Page, The Anatomy of a Large-Scale Hypertextual Web Search Engine, Computer Networks and ISDN Systems 30, 107–117 (1998).